

Ngspice + OpenVAF Enhancements

User Handbook and Enhancement Reference

Dr. Meisam Bahadori

July 2026

Contents

Part I — The User Handbook	7
The Enhanced ngspice + OpenVAF Handbook	7
1 · Getting started	9
2 · Verilog-A language support	12
3 · Simulating with ngspice	20
4 · Limitations and gotchas	24
5 · Enhancement index	26
Part II — The Enhancements, One by One	74
Enhancement-1 — <code>absdelay</code> support for OSDI/Verilog-A in OpenVAF + ngspice	75
Enhancement-2 — Indirect Branch Assignment support in OpenVAF	80
Enhancement-3 — Vectored/Bus-Style Net Declarations in OpenVAF	86
Enhancement-4 — Laplace-Domain Transfer-Function Operators in OpenVAF	94
Enhancement-5 — Verilog-A Module Instantiation in OpenVAF	109
Enhancement-6 — Missing Basic Verilog-A Features in OpenVAF-r	118
Enhancement-7 — <code>@(initial_step)</code> event gating and variable persistence	126
Enhancement-8 — <code>generate for/genvar</code> and <code>cross()/above()/timer()</code>	130
Enhancement-9 — Verilog-A noise sources: <code>completing noise_table / noise_table_log</code>	136
Enhancement-10 — Verilog-A statistical / random-number system functions	145
Enhancement-11 — Verilog-A file I/O and string-formatting system functions	149
Enhancement-12 — the last Verilog-A system functions: <code>probe / alias / plusargs</code>	153
Enhancement-13 — <code>limexp()</code> : investigation, kept stateless	155
Enhancement-14 — Verilog-A array literals / aggregates	157
Enhancement-15 — Verilog-A multi-dimensional arrays	160
Enhancement-16 — Verilog-A <code>\$table_model</code>	162
Enhancement-17 — multi-dimensional <code>\$table_model</code>	164
Enhancement-18 — array declaration syntax + arrays in analog functions	166
Enhancement-19 — <code>do ... while</code> loop	168
Enhancement-20 — array output/inout arguments to analog functions	169
Enhancement-21 — Verilog-AMS <code>paramset</code> blocks	170
Enhancement-22 — natural cubic-spline <code>\$table_model</code>	172
Enhancement-23 — array return values from analog functions	174
Enhancement-24 — <code>\$discontinuity(n)</code> simulator support	176
Enhancement-25 — <code>\$simparam\$str(name)</code> support	178
Enhancement-26 — <code>ac_stim(...)</code> baseline (crash fix + correct large-signal semantics)	180
Enhancement-27 — <code>idtmod(...)</code> modulo-integrator fix	181
Enhancement-28 — <code>idt(...)</code> initial-condition fix	182
Enhancement-29 — port-branch flow access <code>I(<port>)</code>	184
Enhancement-30 — <code>variadic analysis(arg1, arg2, ...)</code>	186
Enhancement-31 — complex poles/zeros in laplace/zi root forms	188
Enhancement-32 — integer persistent/event-state variables	190
Enhancement-33 — array case + array-literal function arguments	192
Enhancement-34 — <code>{...}</code> concatenation & <code>{n{...}}</code> replication	194
Enhancement-35 — lexer hang on <code>//</code> comment at end-of-file	196
Enhancement-36 — probe-only branches / ideal ammeter + flow-only signal flow	198
Enhancement-37 — operator-correctness audit + fixes	200
Enhancement-38 — operator-precedence audit + fixes	202
Enhancement-39 — derived natures & deriving natures from disciplines	204
Enhancement-40 — N-dimensional <code>\$table_model</code>	206
Enhancement-41 — implicit nets in instance connections	207
Enhancement-42 — correlated (same-named) noise sources	209

Enhancement-43 — variable initializers, completed	211
Enhancement-44 — paramset hierarchical system parameters	213
Enhancement-45 — net initialization + net attribute access	215
Enhancement-46 — escaped identifiers + integer literal bases	217
Enhancement-47 — 'default_transition + transition() fixes	219
Enhancement-48 — string literal escape sequences	221
Enhancement-49 — \$root + hierarchical names, transition() input	223
Enhancement-50 — domain binding validation	225
Enhancement-51 — full ac_stim AC injection	226
Enhancement-52 — idt() assert/reset forms	228
Enhancement-53 — @(final_step) + analysis-phase lists on step events	229
Enhancement-54 — correct + node-free noise factors	231
Enhancement-55 — simulation-control tasks + \$discontinuity rejection	233
Enhancement-56 — VA_TEST end-to-end sweep: CMC default-range idiom + noise crash . .	235
Enhancement-57 — physics-accuracy validation suite	237
Enhancement-58 — defparam hierarchical parameter override	238
Enhancement-59 — LRM-corner probe follow-up: event OR lists, \$realtime, port concatenation, recursion diagnostics	240
Enhancement-60 — multiple analog blocks: validation deliverable	242
Enhancement-61 — operator-argument audit: slow sign-convention fix	244
Enhancement-62 — ngspice analysis coverage for OSDI devices: .dc @inst[param] sweeps + .disto warning	246
Enhancement-63 — RF analyses with OSDI devices: .sp / transient noise / PSS + span.c NaN fix	248
Enhancement-64 — Touchstone export: auto-Rbase, N-port wrsnp, 1-port .sp	250
Enhancement-65 — preprocessor audit: macro-recursion guard	252
Enhancement-66 — Monte Carlo with OSDI devices: validation deliverable	254
Enhancement-67 — generate audit: genvar-substitution fix, nested loops, generate if/case	255
Enhancement-68 — enabling openvaf's own integration test suite	257
Enhancement-69 — operating-point variables end-to-end: validation deliverable	259
Enhancement-70 — behavioral-loop audit: precise loop diagnostics	260
Enhancement-71 — display-task audit: format flags/width for every conversion + the %b segfault	262
Enhancement-72 — Touchstone round 2: MA/DB formats, frequency units, Y/Z export, and the rdsnp reader	263
Enhancement-73 — the user handbook: 72 enhancements consolidated into docs/handbook/	264
Enhancement-74 — performance benchmark: OSDI vs built-in twins + flagship compile times	266
Enhancement-75 — dynamic physics validation: the reactive paths cross-checked	268
Enhancement-76 — multi-module .osdi libraries: audit + three registration fixes	269
Enhancement-77 — ngspice zero-warning build: 33 → 0	271
Enhancement-78 — casex / casez: the don't-care case statements	272
Enhancement-79 — benchmark round 2: ring oscillator, small-signal throughput, KLU vs SPARSE	274
Enhancement-80 — temperature physics validation + the dtemp alias	275
Enhancement-81 — session-lifecycle + memory audit: two resource fixes	276
Enhancement-82 — provenance and compliance documentation	277
Enhancement-83 — the transistor-level μ A741: a complete model-to-circuit demo	278
Enhancement-84 — the LRM example sweep and its six defect fixes	280
Enhancement-85 — __FILE__/LINE and connection part-selects	282
Enhancement-86 — hierarchical branch probes	284
Enhancement-87 — block-scoped parameters	286
Enhancement-88 — the legacy generate statement	288
Enhancement-89 — name-then-range ports and the Annex E primitives	290
Enhancement-90 — multi-bit input bus port bit reads	292

Enhancement-91 — multi-name name-then-range declarations and parameter-dependent widths	294
Enhancement-92 — freezing structural (width) parameters	296
Enhancement-93 — warning when a fixed (localparam) parameter is set	298
Enhancement-94 — the pyplot command (matplotlib backend)	299
Enhancement-95 — optional file name for pyplot	300
Enhancement-96 — module-level generate for without generate/endgenerate	301
Enhancement-97 — clean diagnostic for a contribution to a ground branch	302
Enhancement-98 — pyplot multi-panel subplots and style sheets	303
Enhancement-99 — pyplot vector export formats (SVG/PDF) and figure size	304
Enhancement-100 — the first hundred: a milestone audit & retrospective	305
Enhancement-101 — \$c log2 correctness (arity + value)	307
Enhancement-102 — name-then-range array-valued parameters	309
Enhancement-103 — ceil() of a runtime argument (missing LLVM intrinsic)	310
Enhancement-104 — \$rtoi / \$itor real↔integer conversion functions	311
Enhancement-105 — \$sscanf / \$fscanf honour the format conversion base	312
Enhancement-106 — string relational comparison (<, <=, >, >=)	313
Enhancement-107 — the \$fgetc file-input function	314
Enhancement-108 — the \$ungetc file-input function	315
Enhancement-109 — noise_table / noise_table_log interpolation per the LRM	316
Enhancement-110 — errpreset: coordinated accuracy presets for ngspice	317
Enhancement-111 — globalized (damped) Newton via an Armijo line search	319
Enhancement-112 — KLU support for the globalized-Newton line search	322
Enhancement-113 — KLU support for noise and pole-zero analysis	324
Enhancement-114 — KLU support for sensitivity analysis	326
Enhancement-115 — KLU support for distortion analysis (.disto)	328
Enhancement-116 — KLU wrong-DC fix for decoupled OSDI nodes (2 of 3 XFAILs)	330
Enhancement-117 — Periodic steady state (PSS) productionized	332
Enhancement-118 — PSS runs under the KLU solver	334
Enhancement-119 — retain the PSS periodic operating point	336
Enhancement-120 — periodic small-signal Jacobian harmonics	338
Enhancement-121 — the PAC conversion-matrix engine	340
Enhancement-122 — the .pac command (periodic AC)	342
Enhancement-123 — finish .pac: source stimulus + conversion sidebands	344
Enhancement-124 — periodic noise (.pnoise)	346
Enhancement-125 — periodic transfer function (.pxf)	348
Enhancement-126 — cyclostationary noise	350
Enhancement-127 — pseudo-transient continuation	352
Enhancement-128 — LTE-based dynamic integration-order control	354
Enhancement-129 — sweep progress bar	357
Enhancement-130 — built-in Nelder-Mead optimizer	359
Enhancement-131 — transient checkpoint / restart	361
Enhancement-132 — periodic S-parameters (.psp)	364
Enhancement-133 — quasi-periodic (two-tone) steady state (qpss)	367
Enhancement-134 — Harmonic Balance (hb)	369
Enhancement-135 — Harmonic-Balance source-stepping continuation	371
Enhancement-136 — Two-tone Harmonic Balance (frequency-domain QPSS)	373
Enhancement-137 — Two-tone small-signal QPAC (quasi-periodic AC)	375
Enhancement-138 — Two-tone small-signal QPnoise (quasi-periodic noise)	377
Enhancement-139 — Cyclostationary QPnoise	379
Enhancement-140 — Oscillator phase noise (autonomous HB + phasenoise)	381
Enhancement-141 — Two-tone small-signal QPXF (quasi-periodic transfer function)	383
Enhancement-142 — Input-frequency sweep for the two-tone small-signal analyses	385

Enhancement-143 — gradient least-squares curve fitting for optimize	387
Enhancement-144 — optimizing symbolic .param values (-dparam)	389
Enhancement-145 — optimizing .model-card parameters (-mparam)	391
Enhancement-146 — universal sweep command and .sweep card	393
Enhancement-147 — nested ?: no longer takes exponential time to compile	395
Enhancement-148 — compiler hardening: parser depth, include cap, array cap	397
Enhancement-149 — Latin-Hypercube Monte Carlo sampling (mcsample)	399
Enhancement-150 — high-sigma rare-event estimation (highsigma)	401
Enhancement-151 — process/mismatch correlations and a packaged yield flow	403
Enhancement-152 — KLU matrix reordering and scaling controls	405
Enhancement-153 — Levenberg-Marquardt trust-region Newton (.option trustregion)	407
Enhancement-154 — Envelope Following (envelope command)	409
Enhancement-155 — RC network reduction (reduce, TICER)	411
Enhancement-156 — Sparse RC reduction (scalable reduce)	413
Enhancement-157 — Device aging (reliability degradation flow)	415
Enhancement-158 — Power-grid EMIR (electromigration + IR-drop)	418
Enhancement-159 — Real production compact-model bring-up (BSIM4 + EKV)	420
Enhancement-160 — CMC compact-model coverage sweep	422
Enhancement-161 — Dynamic (AC / RF) compact-model validation	424
Enhancement-162 — .hb dot-card for harmonic balance	426
Enhancement-163 — .qpss, .hbosc, .phasenoise dot-cards	428
Enhancement-164 — Large-signal RF characterization of a real transistor	430
Enhancement-165 — Production compact-model noise validation	432
Enhancement-166 — Electro-thermal / self-heating validation	434
Enhancement-167 — Cross-model self-heating sweep	436
Enhancement-168 — RF noise figure of an LNA	438
Enhancement-169 — Interactive command-line syntax highlighting	440
Enhancement-170 — Semantic syntax highlighting	442
Enhancement-171 — KLU/Sparse solver audit: complex determinant + pivot tolerance	444
Enhancement-172 — KLU balanced pole-zero + full partial pivoting	446
Enhancement-173 — Eigenvalue-based pole-zero (.options pzeig)	448
Enhancement-174 — help all crash fix (help string used as printf format)	450
Enhancement-175 — RF-suite audit: the dropped parametric term in every periodic small- signal analysis	451
Enhancement-176 — Driven-mode PSS shooting: no frequency hunt, no breakpoint flood	453
Enhancement-177 — pnoise noise-folding referee + the folded-flicker frequency fix	455
Enhancement-178 — exact separable cyclostationary folding + the HB DC-source fix	457
Enhancement-179 — standard-analyses audit: referee battery + three fixes	459
Enhancement-180 — checkpoint/restart under KLU (the last Sparse-only feature falls)	461
Enhancement-181 — core-numerics audit: the integrator certified exactly, plus .options ordfix	462
Enhancement-182 — pyplot: autoscale by default, pin axes only for explicit user limits	464
Enhancement-183 — pyplot usability: distinct default names, deck-folder output, linewidth & backend	465
Enhancement-184 — sweep progress bar reaches 100%	466
Enhancement-185 — openvaf-r autodiff audit: hypot & atan2 derivative fixes	467
Enhancement-186 — openvaf-r autodiff audit: real-modulo (%) derivative fix	468
Enhancement-187 — openvaf-r math-identity simplifier: invalid function-inverse cancellations	470
Enhancement-188 — Monte Carlo warm-start (montecarlo ... -warm)	472
Enhancement-189 — Sweep waveform overlay (sweep ... -overlay)	474
Enhancement-190 — Nested multi-knob sweep (sweep ... -vs ...)	475
Enhancement-191 — .ac lin 2 / .sp lin 2 off-by-one fix	476
Enhancement-192 — Auto-checkpoint on interrupt (set autosave=<file>)	478

Enhancement-193 — <code>.pnoise</code> honors <code>sqrnoise</code> (noise-density units fix)	480
Enhancement-194 — Particle swarm optimization (<code>optimize -method pso</code>)	482
Enhancement-195 — Differential evolution (<code>optimize -method de</code>)	484
Enhancement-196 — Simulated annealing (<code>optimize -method sa</code>)	485
Enhancement-197 — A 100-parameter circuit optimization (raised <code>optimize</code> limits)	486
Enhancement-198 — <code>stb</code> stability / loop-gain analysis	488
Enhancement-199 — N-port Touchstone device (<code>snp2va.py</code>)	490
Enhancement-200 — built-in <code>pre_snp</code> command	492
Enhancement-201 — <code>pre_snp</code> scalability (fast vector fitting + shared-pole realization)	494
Enhancement-202 — <code>.sp</code> S-parameter matrix inverse is $O(N^3)$, not $O(N!)$	496
Enhancement-203 — <code>.meas</code> ac gain / phase margin (+ batch <code>vdb/vp</code> fix)	498
Enhancement-204 — auto-triggering DC convergence aids	499
Enhancement-205 — low-rank residue factorization in <code>pre_snp</code>	501
Enhancement-206 — design centering / yield optimization	503
Enhancement-207 — eye diagram / jitter analysis (<code>eye</code>)	505
Enhancement-208 — <code>pyplot -eye</code> : one-line matplotlib eye diagrams	506
Enhancement-209 — <code>hb</code> publishes its spectrum as nutmeg vectors	508
Enhancement-210 — PSS polish: <code>.pss</code> dot-card runs in batch + complex spectrum vectors	509
Enhancement-211 — code-analysis bug fixes (XSPICE DC-op else + LTSPICE table free)	511
Enhancement-212 — crash hardening: seven user-triggerable crashes	513
Enhancement-213 — <code>openvaf-r</code> crash hardening: four compiler panics	515
Enhancement-214 — whole-array type coercion: a recurring crash class, fixed at its root	518

Part I — The User Handbook

The Enhanced ngspice + OpenVAF Handbook

This handbook is the consolidated user guide to the enhanced toolchain in this repository: `openvaf-r` (the Verilog-A compiler) and `ngspice-46` (the simulator), as extended by the enhancement series (see the [index](#)). The repository's top-level [README](#) carries the one-line enhancement index; the detailed write-ups live in [enhancements_doc/](#), one per enhancement, in the order the work happened. This handbook reorganizes that material by what you want to do — so you don't need to know the project history to find out whether a language feature works, how to run an analysis, or where the sharp edges are.

Prefer a single file? The whole handbook *plus the complete text of every enhancement write-up* is compiled into one PDF with a linked table of contents: [docs/Ngspice-OpenVAF-Handbook.pdf](#). Regenerate it after edits with `python3 docs/handbook/build_pdf.py` (needs `pandoc` + `xelatex`).

The chapters

Chapter	Read it when you want to...
1 · Getting started	install or build the tools, compile your first Verilog-A model, load it in ngspice, and run the shipped example suites
2 · Verilog-A language support	know whether a language construct works — the full feature matrix, organized by LRM area, with a pointer to the example folder that proves each entry
3 · Simulating with ngspice	run analyses over your compiled models — operating point through S-parameters, parameter sweeps, Monte Carlo, noise, measurements, Touchstone import/export
4 · Limitations and gotchas	know what <i>doesn't</i> work, what is out of scope by design, and the traps that cost real debugging time
5 · Enhancement index	trace a feature back to the enhancement that built it — one line per enhancement with links to the detailed write-up and the example folder

How the material is verified

Every feature claimed in this handbook is guarded by a verify script in [examples/](#): a self-contained Python script that compiles the relevant Verilog-A model with the committed compiler, runs it through the committed ngspice, and checks the numbers against closed-form expectations. The feature-matrix entries in chapter 2 and the workflow recipes in chapter 3 each link to the folder that pins them. If you want to see a feature in action, running its verify script is the fastest way:

```
python3 examples/dowhile_examples/verify_dowhile.py
```

Two larger guards sit on top of the per-feature suites:

- **VA_TEST/** — the public VA-Models collection (BSIM4/6/BULK/CMG/IMG/SOI, PSP, HiCUM, MEXTRAM, EKV, ASM-HEMT, and more; 92 standalone models), all of which compile with `python3 VA_TEST/compile_all.py`;
- **examples/physcheck_examples/** — a physics regression suite that checks industry models against analytic device laws (60 mV/decade junction slope, $4kT/R$ thermal noise identity, AC-vs-numeric-derivative Jacobian cross-checks) through the whole toolchain.

Reference documents

The [docs/](#) folder holds the primary sources this project is written against: the Verilog-AMS Language Reference Manual (VAMS-LRM-2023.pdf — Verilog-A is its Annex C subset), the ngspice-46 manual, and the OSDI interface specification. Its [compliance/](#) subfolder holds the [Verilog-A LRM compliance document](#) (with a PDF edition) — clause-by-clause language coverage with verified code examples. Its [change_log/](#) subfolder holds the two full change reports — [ngspice](#) and [openvaf-r](#), each with a PDF edition — documenting every modification this project applied to either tool, organized by subsystem, with its reason and enhancement link. When the handbook cites "LRM 4.5.1" or similar, that's the document it means.

1 · Getting started

This chapter takes you from a fresh clone to a running Verilog-A simulation: get the two binaries, compile a model, load it in ngspice, and know where the example suites are when you want a working reference for any feature.

1.1 The two tools

- **openvaf-r** compiles a Verilog-A source file (.va) into an OSDI shared object (.osdi) — a compiled device model with analytic Jacobians generated by automatic differentiation.
- **ngspice** loads .osdi files at runtime and simulates them alongside its built-in devices, with full access to every analysis (op, DC, AC, transient, noise, S-parameters, ...).

Both binaries in this repository are patched: the compiler implements far more of the Verilog-A language than upstream, and ngspice carries the matching OSDI runtime (ABI 0.7) plus simulator-side features such as .dc @inst[param] sweeps and Touchstone I/O. Use them as a pair — a .osdi produced by this compiler requires this ngspice (stock ngspice rejects the newer ABI), and vice versa.

1.2 Option A: prebuilt binaries

CI builds both tools for five platforms and commits them under bin/:

Platform	Directory
Linux x86-64	bin/linux/intel/
Linux ARM64	bin/linux/arm/
macOS Apple Silicon	bin/macos/apple-silicon/
macOS Intel	bin/macos/intel/
Windows x86-64	bin/windows/intel/

Each platform has small runtime prerequisites (X11/readline on Linux, XQuartz + Homebrew readline on macOS, nothing on Windows — the DLLs are bundled). The [Prebuilt Binaries section of the top-level README](#) has the exact per-OS install commands and the notes for macOS Gatekeeper / Windows SmartScreen warnings.

1.3 Option B: building from source

openvaf-r needs a Rust toolchain and LLVM 18 (exactly 18 — the codegen crates are pinned to it):

```
# macOS (Homebrew)
brew install llvm@18
export LLVM_SYS_181_PREFIX=/opt/homebrew/opt/llvm@18

# Debian/Ubuntu
sudo apt-get install llvm-18-dev libpolly-18-dev libzstd-dev
export LLVM_SYS_181_PREFIX=/usr/lib/llvm-18

cd OpenVAF-master-20260610
cargo build --release --bin openvaf-r --features openvaf-driver/llvm18
# → target/release/openvaf-r
```

ngspice is a standard autotools build. The configuration used throughout this project (macOS spelling shown; adjust the readline/X11 paths for your system):

```
cd ngspice-46
mkdir -p build && cd build
```

```

../configure --with-x --with-readline=/opt/homebrew/opt/readline --disable-openmp
make -j8
# → build/src/ngspice

```

The CI workflow ([.github/workflows/build-binaries.yml](https://github.com/workflows/build-binaries.yml)) is the authoritative record of working build recipes for all five platforms, including the Windows MSYS2/MinGW path.

1.4 Your first model

Save this as `myres.va` — a behavioral resistor with an overridable parameter:

```

`include "disciplines.vams"

module myres(p, n);
    inout p, n;
    electrical p, n;
    parameter real r = 1k from (0:inf);

    analog
        I(p, n) <+ V(p, n) / r;
endmodule

```

Compile it:

```
openvaf-r myres.va -o myres.osdi
```

(Without `-o`, the output lands next to the input as `myres.osdi`. Useful extras: `-I <dir>` for include search paths, `-D MACRO [=VALUE]` for preprocessor defines, `--lints` to list tunable lints.)

Now a SPICE deck, `divider.cir`. Three conventions matter:

1. `pre_osdi <file>.osdi` in the `.control` block loads the compiled model before the circuit is parsed (that's what the `pre_` prefix means).
2. OSDI instances use the **N** device letter.
3. The `.model` card's type name is the Verilog-A module name (`myres`), and the card is where parameters are set.

* resistive divider from a Verilog-A resistor

```

vin in 0 dc 2
n1 in mid ra
n2 mid 0 rb
.model ra myres(r=1k)
.model rb myres(r=1k)

.control
pre_osdi myres.osdi
op
print v(mid)
.endc
.end

ngspice -b divider.cir
# v(mid) = 1.000000e+00

```

Title line: SPICE treats the *first line* of a deck as the title, not as a statement. Every deck must start with a comment/title line — a netlist statement on line 1 is silently swallowed.

From here everything is ordinary ngspice: dc, ac, tran, noise, .meas, plotting. Chapter 3 covers the analysis workflows in depth, including the parts specific to OSDI devices (parameter access via @n1[. . .], sweeps over device parameters, Monte Carlo).

1.5 The example suites

Every feature in this project ships with a folder under [examples/](#) containing the Verilog-A models, ready-to-run decks, and a `verify_*.py` script that checks exact numbers. They double as a cookbook: when chapter 2 says arrays work, `examples/array_examples/` shows the syntax in a complete, committed, tested model.

```
python3 examples/noise_examples/verify_noise.py
```

The scripts resolve the toolchain through `examples/_setup.py`: a locally built binary (`OpenVAF-master-20260610/target/release/opencvaf-r`, `ngspice-46/build/src/ngspice`) is preferred if present, otherwise the committed `bin/<os>/<arch>/` prebuilt for your machine is used — so the suites run on a fresh clone with no configuration.

Two folders are worth singling out:

- [examples/analyses_examples/](#) is a tutorial: nine standalone, commented decks (one per analysis, each stating its expected numbers), a walk-through README, and committed PNG plots of the sweep results.
- [examples/physcheck_examples/](#) validates industry compact models against analytic device physics — a good template for validating your own models quantitatively.

1.6 The bigger test assets

- **VA_TEST/** — 124 `.va` files of public industry compact models; `python3 VA_TEST/compile_all.py` compiles all 92 standalone models and regenerates `VA_TEST/compile_report.md`.
- The compiler's own integration suite — descriptor snapshots and live numeric tests over real compact models:

```
cd OpenVAF-master-20260610
RUN_DEV_TESTS=1 cargo test --release -p opencvaf --features opencvaf/llvm18
↪ --test integration
```

2 · Verilog-A language support

This is the feature matrix: what the compiler accepts, what the compiled model actually *does* at simulation time, and where each claim is proven. The scope of the project is Verilog-A — the analog subset defined in Annex C of the Verilog-AMS LRM — not full mixed-signal AMS (see [chapter 4](#) for exactly where that line runs).

Every table row links the enhancement that built or audited the feature (E-n → the detailed write-up in `enhancements_doc/`) and, in the notes, the `examples/` folder whose verify script pins it. A recurring lesson of this project is that *compiling proves nothing* — several features compiled for years while silently doing the wrong thing at runtime — so a row only says “works” if a committed script checks the simulated numbers.

2.1 Modules, hierarchy, and elaboration

Hierarchy is resolved by a compile-time elaboration pass: instantiated modules are recursively inlined (alpha-renamed per instance, ports bound to the caller’s nets, parameters bound to overrides) into a single flat module before the rest of the compiler runs. Everything in this section is therefore resolved at compile time.

Feature	Notes	Since
Module instantiation	Named <code>.p(net)</code> and positional port connections, open ports, named <code>#(.r(1e3))</code> and positional <code>#(1e3)</code> parameter overrides, instance arrays <code>res rarr[0:3]</code> (https://github.com/javaNoviceProgrammer/Ngspice_OpenVAF_Enhancements/blob/main/docs/handbook/...), arbitrary nesting, across <code>`include</code> boundaries; cyclic instantiation is a clean error. <code>examples/instantiation_examples/</code>	E-5
generate for / genvar	Structural loops generating nets, instances, variables, parameters; nested loops (inner bounds may use outer genvars); genvars usable in any expression, e.g. <code>#(.r(1e3*(i+1)))</code> . <code>examples/generate_examples/</code>	E-8 , E-67
generate if / generate case	else/else-if chains, multi-value case arms, default; conditions fold over genvars and literals. A condition on a module <i>parameter</i> is an intentional error — parameters bind at simulation time and cannot shape structure (see §4.3).	E-67
Hierarchical names	Instance-path references <code>u1.u2.node</code> , <code>\$root.top.net</code> , from expressions and probes. <code>examples/hiername_examples/</code>	E-49
defparam	Compile-time hierarchical parameter override at any depth (<code>defparam u1.u2.r = 2e3;</code>), multi-assignment form, expression values; takes LRM-mandated precedence over instance <code>#()</code> overrides. <code>examples/defparam_examples/</code>	E-58
paramset	Verilog-AMS paramset blocks, including <code>.\$mfactor/position</code> hidden-system-parameter assignments inside the paramset. <code>examples/paramset_examples/</code> , <code>examples/paramsethsp_examples/</code>	E-21 , E-44
Implicit nets	An undeclared identifier in an instance connection becomes an implicit scalar net, discipline derived from the connected port. <code>examples/implicitnet_examples/</code>	E-41
Net concatenation in ports	<code>u1({a, c})</code> onto a vectored port, expanded bit-by-bit (leftmost = msb), positionally or named, nested through instance levels; width mismatch is a hard error.	E-59
Multiple analog blocks	Several <code>analog / analog initial</code> blocks per module execute as-if concatenated in source order (LRM 6.2), including across instance flattening. <code>examples/multianalog_examples/</code>	E-60

2.2 Nets, disciplines, natures

Feature	Notes	Since
Vectored (bus) nets and ports	<code>electrical [3:0] bus;</code> with bit-select access in branches and probes; buses slice element-wise onto instance arrays and bus ports. examples/bus_examples/	E-3
ground declarations	Both orderings (<code>electrical ground gnd;</code> and <code>ground electrical gnd;</code>). examples/ground_examples/	E-9
Derived natures	<code>nature X : Parent</code> , deriving from a discipline's nature (<code>: electrical.flow</code>), with full attribute inheritance. examples/derivednature_examples/	E-39
Net initializers (nodesets)	<code>electrical a = 5.0;</code> becomes an OSDI nodeset — an initial-guess hint for the solver, essential for bistable circuits. examples/netinit_examples/	E-45
Nature-attribute access	<code>net.potential.abstol</code> , <code>branch.potential.abstol</code> readable in expressions.	E-45, E-59
Domain binding	<code>domain continuous</code> accepted; <code>domain discrete</code> on a discipline with natures is a proper two-label error (LRM 3.6.2.2). examples/domainbind_examples/	E-50
Signal-flow / flow-only disciplines	Disciplines with only a flow (or only a potential) nature work, including probe-only usage (§2.7). examples/signalflow_examples/	E-36

2.3 Data types, variables, and arrays

Feature	Notes	Since
<code>real</code> , <code>integer</code> , <code>string</code> variables	Full support, including uninitialized strings defaulting to <code>""</code> . examples/vartype_examples/	E-9
Variable persistence	Module-level variables genuinely keep their value across evaluations (<code>accum = accum + 1.0;</code> accumulates) — for <code>real</code> <i>and</i> <code>integer</code> state. examples/variable_persistence_examples/ , examples/intstate_examples/	E-7, E-32
Declaration initializers	<code>real x = 2.0*p;</code> , including on (multi-dimensional) arrays; parameter-only initializers evaluate once at setup. examples/varinit_examples/	E-43
Arrays, 1-D to N-D	Both declaration orders (<code>real [0:3] x;</code> and LRM-style <code>real x[0:3];</code>), any number of dimensions, constant <i>and dynamic</i> indexing (<code>m[i][j]</code> with runtime <code>i, j</code>). examples/array_examples/ , examples/mdarray_examples/	E-14, E-15, E-18
Array aggregates	Whole-array assignment <code>c = {v0, v1, v2};</code> , nested for N-D, array-to-array copy <code>c = d;</code> .	E-14, E-15
Concatenation / replication	<code>{a, b, c}</code> and <code>{n{x}}</code> as real operators over arrays and strings (<code>{...}</code> stays the typed aggregate). examples/concat_examples/	E-34
Arrays in case	Element-wise array case statements and array-literal case labels. examples/arraycase_examples/	E-33

2.4 Parameters

Feature	Notes	Since
Ranges and excludes	from (0:inf), exclude 0, etc., enforced on user-given values. Per the CMC convention, a parameter's default is exempt from its own range — industry models use an out-of-range default as the "feature disabled" state.	E-56
localparam	Non-overridable per the LRM; derived localparams (localparam G = 1/R) track their inputs. examples/localparam_examples/	E-9
aliasparam	Works, including \$param_given queried through the alias.	E-59
Array-valued parameters	parameter real [0:3] w = '{...}'; (any dimensionality) expands to one scalar OSDI parameter per element, each individually overridable from SPICE: <code>.model m dev(w[1][1]=0.9)</code> .	E-14, E-15
String parameters	Including case dispatch on a string parameter.	E-59
Hidden system parameters	\$mfactor (device multiplicity — automatically scales flows, noise, and Jacobians) and the position parameters, readable in the model and settable per instance (<code>n1 a b mod m=4</code>).	E-44
Instance-line parameters	(* type="instance" *) on a parameter exposes it on the instance line and to <code>alter @n1[p] / .dc @n1[p]</code> (see §3.3).	E-62
\$param_given, \$port_connected	Both work; industry models use them for configuration guards.	—

2.5 Operators and literals

The operator surface was audited systematically — every operator against the LRM, every precedence level against LRM Table 4-2 — with self-checking modules whose failure modes are numerically visible.

Feature	Notes	Since
Arithmetic, comparison, logical, bitwise operators	Audited; fixes included ~ (was arithmetic negate) and const-folded unsigned >>. examples/operator_examples/	E-37
Precedence and associativity	Audited against LRM Table 4-2; % now binds like *//, <code>2**3**2 = 64</code> (left-assoc), unary binds above **. examples/precedence_examples/	E-38
Arithmetic shifts <<< / >>>	examples/shift_examples/	E-6
Ternary ?:	Including over string operands.	E-37
Based integer literals	'h1F, 'b101, 'o17, 'd42, with underscores and explicit widths. examples/escid_examples/	E-46
Escaped identifiers	\my-net! per the LRM, everywhere identifiers appear.	E-46
String literal escapes	The full set (\n, \t, \\, \", octal \ddd) via a single-pass unescaper. examples/stresc_examples/	E-48
Integer min/max/abs, away-from-zero rounding	Verified integer-typed, per the LRM.	E-59
Math functions & scale factors	ln/log/exp/sqrt/pow/trig/hyperbolic/floor/ceil, SI suffixes (1k, 1u, ...).	—

2.6 Analog operators (filters, integrators, delays)

All analog operators produce correct Jacobian contributions via automatic differentiation, so convergence behaves like a hand-written stamp.

Operator	Notes	Since
<code>ddt(x)</code>	Time derivative; fully working (the reactive residual/Jacobian path).	—
<code>idt(x[, ic])</code>	Time integral; the initial condition survives into transient (it used to be zeroed at the IC phase). examples/idthic_examples/	E-28
<code>idt(x, ic, assert[, tol])</code>	Reset form: while <code>assert</code> is nonzero the state holds at <code>ic</code> ; smooth reset dynamics keep self-referential resets bounded. Relaxation oscillators work. examples/idtassert_examples/	E-52
<code>idtmod(x, ic, modulus[, offset])</code>	Modulo integrator (VCO phase idiom): integrates unbounded internally, wraps the returned value. examples/idtmod_examples/	E-27
<code>ddx(x, probe)</code>	Symbolic derivative w.r.t. a potential <i>or flow</i> probe. examples/ddx_examples/	E-13, E-59
<code>absdelay(x, td[, maxdelay])</code>	Transport delay via the synthetic-node DAE approach. examples/absdelay_examples/	E-1
<code>transition(x[, td, rise, fall])</code>	3/4/5-argument forms; <code>`default_transition</code> supplies defaults; DC is well-posed (no singular matrix at the operating point). examples/transition_examples/ , examples/defaulttransition_examples/	E-6, E-47
<code>slew(x[, max_pos, max_neg])</code>	Rate limiter; honors the LRM's <i>negative</i> <code>max_neg_slew_rate</code> convention (and tolerates the legacy positive-magnitude spelling). examples/slew_examples/ , examples/opargs_examples/	E-6, E-61
<code>laplace_nd/np/zd/zp</code>	Continuous-time filters via exact state-space realization; complex pole/zero pairs per the LRM's (re, im) vector convention; parameter-dependent coefficients. examples/laplace_examples/ , examples/complexpole_examples/	E-4, E-31
<code>zi_nd/np/zd/zp</code>	Z-domain filters via bilinear transform onto the laplace machinery. examples/zi_examples/	E-6, E-31
<code>last_crossing(x, dir)</code>	Interpolated crossing times, backed by simulator-side waveform history (an additive OSDI extension). examples/last_crossing_examples/	E-6
<code>limexp(x)</code>	Stateless limited exponential (tangent-continued above the overflow cutoff) — a documented decision; see §4.4.	E-13
<code>\$limit(x, "pnjlim", ...)</code> <code>/ \$limit(x, fn, ...)</code>	Genuinely engages SPICE-style iteration limiting (stiff diodes converge without <code>gmin</code> stepping), for the built-in and user-function forms. examples/opargs_examples/	E-61
<code>\$bound_step(t)</code>	Genuinely bounds the transient step.	E-61
<code>\$discontinuity(n)</code>	(Clamps the <i>next</i> step and makes the integrator bisect onto the event instead of extrapolating across it. examples/discontinuity_examples/	E-24, E-55
Trailing tolerance arguments	The optional <code>abstol</code> args of <code>ddt/idt/idtmod/laplace_*/zi_*</code> and event tolerances are accepted and honored.	E-61

Analog operators are statically forbidden inside conditionals whose condition depends on the solution, and inside loops (LRM 4.5.1) — the diagnostics name the actual construct and suggest hoisting or

generate unrolling (E-70).

2.7 Contributions, branches, and probes

Feature	Notes	Since
V(a,b) <+ / I(a,b) <+ contribu- tions	Including switch branches (V-then-I) and accumulation across statements, loops, and multiple analog blocks.	—
Indirect branch assignment	V(out): V(x) == 0; — the LRM's ideal-opamp construct, one implicit equation per statement. examples/indirect_assignment_examples/	E-2
Port-flow probes	Reading the total flow through a port (previously returned 0). examples/portflow_examples/	E-29
I(<p>) Probe-only branches	Probing a branch that is never contributed to reads its actual flow (an ideal ammeter) instead of 0 + open circuit; makes flow-only signal-flow disciplines work. examples/portflow_examples/ , examples/signalflow_examples/	E-36
Grounded branch declarations	branch (a) br;	E-59

2.8 Events

Feature	Notes	Since
@(initial_ step) / @(final_ step)	Genuinely gated (once at the start; once at the end of a <i>successful</i> analysis, seeing the converged solution — the classic "report a tracked peak" pattern works). An op fires both. examples/initial_step_examples/ , examples/finalstep_examples/	E-7, E-53
Analysis- phase lists	@(initial_step("ac","tran")) filters by analysis type, per LRM 5.10.2.	E-53
cross(expr[, dir, time_ tol, expr_ tol])	Edge detection with persistent state across evaluations; tolerances honored. examples/cross_examples/	E-8, E-61
above(expr[, ...])	Like cross but also fires in DC.	E-8, E-59
timer(start[, period, tol])	Periodic firing verified at exact counts. examples/timer_examples/	E-8, E-61
Event OR lists	@(cross(...) or timer(...) or initial_step) — fires exactly when any member fires (LRM 5.10.3). examples/lrmcorner_examples/	E-59

2.9 Analog functions

Feature	Notes	Since
analog function basics	Inlined at the call site; derivatives flow through automatically.	—
Array arguments	Whole-array input args (E-18), output/inout writeback (E-20), array return values (real [0:2] f; — E-23), array-literal arguments (E-33), array locals. examples/funcarray_examples/, examples/arrayout_examples/, examples/arrayret_examples/	E-18/ 20/23/ 33
Loops inside functions	Iterative algorithms (Newton, factorial) verified exact.	E-70
Integer/ untyped arguments	Untyped args default to real; integer output args work.	E-43, E-59
Recursion	Not legal in Verilog-A; both direct and mutual recursion are clean, cycle-naming errors (mutual recursion used to crash the compiler).	E-59

2.10 Procedural statements and loops

Feature	Notes	Since
if/else, case, casex/casez	case works on strings and arrays; casex/casez treat x/z/? digits of item literals as comparison masks (the priority-encoder idiom), with don't-care literals rejected everywhere else. examples/casexz_examples/	E-33, E-59, E-78
for, while, repeat(n), do...while	All four, audited: nesting, loops over arrays, solution-dependent conditions, contributions accumulating inside loops. Parameter-dependent trip counts honor model-card overrides at simulation time. examples/analogloop_examples/, examples/dowhile_examples/, examples/repeat_examples/	E-9, E-19, E-70
disable <block>;	Verilog-A's early exit — the idiom from which break/continue are built (the language has neither keyword). examples/disable_examples/	E-9

2.11 System tasks and functions

Display and I/O

Feature	Notes	Since
\$strobe, \$display, \$write, \$monitor, \$debug	All five kinds; full format-specifier surface — [flags] [width] [.precision] on every conversion (%5d, %-8s, %08.3f, dynamic %*d), %e/f/g/r (engineering notation), %d/h/o/b/c, %s, %m (module path), %%. examples/display_examples/	E-71
File I/O	\$fopen/\$fclose/\$fdisplay/\$fwrite/\$fstrobe/\$fmonitor/\$fdebug/\$fflush/\$fseek/\$ftell/\$fseek/ examples/fileio_examples/	E-11
String formatting/ parsing	\$swrite, \$sformat, \$sscanf, \$fgets, \$fscanf. examples/stringio_examples/	E-11

Modeling functions

Feature	Notes	Since
<code>\$table_model</code>	1-D piecewise-linear with constant/linear extrapolation controls; multilinear interpolation in any number of dimensions (self-describing grid file); natural cubic-spline control ("3") for C ¹ -smooth 1-D tables. Lowered to differentiable MIR, so <i>all</i> partial derivatives feed the Jacobian (a table-based MOSFET gets exact gm and gds). examples/table_model_examples/ , examples/mdtable_examples/ , examples/ndtable_examples/ , examples/cubic_table_examples/	E-16 , E-17 , E-22 , E-40
Noise sources	<code>white_noise</code> , <code>flicker_noise</code> , <code>noise_table</code> , <code>noise_table_log</code> (inline data or file). Same-named sources sum coherently (LRM 4.6.4, correlation networks and anti-phase cancellation exact); operating-point-dependent factors and <code>ddt()</code> -shaped noise (induced-gate idiom) are exact and add no matrix unknowns. examples/noise_examples/ , examples/noisecorr_examples/ , examples/noisejw_examples/	E-9 , E-42 , E-54
<code>analysis("name", ...)</code>	Variadic; matches ngspice's analysis phases, including the AC/noise operating point counting as "ac"/"noise" per LRM 4.6.1. examples/analysis_examples/	E-30 , E-53
<code>ac_stim([name, mag, phase])</code>	Full AC right-hand-side injection — a Verilog-A module can <i>be</i> the AC stimulus, with exact magnitude and phase. examples/acstim_examples/	E-26 , E-51
Random / distributions	<code>\$random</code> , <code>\$arandom</code> , <code>\$dist_*</code> and <code>\$rdist_*</code> (uniform, normal, exponential, poisson, chi-square, t, erlang) — deterministic per (<code>seed</code> , <code>call site</code>), stable across Newton iterations (reproducible Monte Carlo, no convergence breakage). examples/rng_examples/	E-10

Environment and control

Feature	Notes	Since
<code>\$temperature</code> , <code>\$vt[(T)]</code>	Simulator temperature (tracks <code>.dc temp</code> sweeps per point) and thermal voltage.	E-62
<code>\$abstime</code> , <code>\$realtime</code>	Identical in the continuous-time analog context.	E-59
<code>\$simparam("name"[, default])</code> , <code>\$simparam\$str("name")</code>	Numeric and string simulator parameters; ngspice exposes <code>analysis_name</code> and <code>simulator</code> among others. examples/simparamstr_examples/	E-25
<code>\$finish</code> , <code>\$stop</code> , <code>\$fatal</code>	Honored at the accepted-point boundary: <code>\$finish</code> ends the analysis cleanly (firing <code>@(final_step)</code>), <code>\$stop</code> pauses resumably, <code>\$fatal</code> aborts with its message — including under solution-dependent conditions, and during setup ("device rejected its configuration"). examples/simctrl_examples/	E-55 , E-56
Fallback group	<code>\$simprobe</code> , <code>\$analog_node_alias</code> , <code>\$analog_port_alias</code> , <code>\$test\$plusargs</code> , <code>\$value\$plusargs</code> compile and return their LRM "mechanism unavailable" fallbacks (see §4.2). examples/alias_examples/	E-12

2.12 Preprocessor and compiler directives

Feature	Notes	Since
<code>`define</code> with arguments	Macros-using-macros, macro calls as macro arguments, backslash continuations, multi-line calls; recursive expansion is a clean, located error (both direct and mutual), while legitimate same-macro nesting inside arguments still works. <code>examples/preproc_examples/</code>	E-65
<code>`ifdef`/`ifndef`/`ifndef`/`endif`</code>	Chains of <code>`ifdef`/`ifndef`/`endif`</code> nesting inside module bodies; unbalanced conditionals are located errors.	E-65
<code>`include`</code>	Nested chains; <code>-I</code> search paths.	—
<code>`default_` transition</code>	Supplies default rise/fall for bare <code>transition()</code> calls. <code>examples/defaulttransition_examples/</code>	E-47
Housekeeping directives	<code>`celldefine`/`endcelldefine`</code> , <code>`unconnected_drive`/`nounconnected_drive`</code> , <code>`timescale`</code> , <code>`line`</code> , <code>`pragma`</code> , <code>`undefineall`</code> , <code>`resetall`</code> , <code>`default_nettype`</code> — accepted (previously fatal errors). <code>examples/directive_examples/</code>	E-6

2.13 Attributes

Attributes (`(* ... *)`) are how a model talks to the simulator's UI:

Attribute	Effect
<code>(* desc="..." *)</code> on a variable	Exposes it as an operating-point variable: <code>print @n1[var], .save @n1[var], .meas</code> over it — for real <i>and</i> integer variables (E-69, <code>examples/opvar_examples/</code>)
<code>(* type="instance" *)</code> on a parameter	Exposes it on the instance line and to <code>alter/.dc @n1[param]</code> (E-62)
<code>(* desc="...", units="..." *)</code> on a parameter	Description/units metadata in the OSDI descriptor

A symmetry worth remembering: instance access to *parameters* needs `type="instance"`; instance access to *variables* needs `desc`.

3 · Simulating with ngspice

This chapter covers the simulator side: how compiled Verilog-A devices participate in ngspice's analyses, how to reach their parameters and internal variables, and the statistical / RF workflows built on top.

The single best companion to this chapter is [examples/analyses_examples/](#) — a tutorial folder with one standalone, commented deck per analysis, each stating the numbers it expects, plus committed PNG plots. When a recipe below feels terse, the corresponding deck there is the full worked example.

3.1 Devices, model cards, instance lines

An OSDI device instance is an N-letter element referencing a .model card whose type name is the Verilog-A module name:

```
n1 in out mymod          ; instance (ports in declaration order)
.model mymod myres(r=1k) ; model card: module `myres`, parameter r
```

- Model-card parameters are the default place to set parameters.
- Instance-line parameters (n1 in out mymod r=2k) work for parameters the model marks (* type="instance" *) — plus the built-in m=<mult> device multiplicity, which correctly scales currents, charges, and noise (\$mfactor in the model's own source).
- Multiple instances of one model card get independent state and independent per-instance values of position/multiplicity parameters.

3.2 Reading data out of a model

Verilog-A variables carrying a (* desc="..." *) attribute are operating-point variables, visible wherever ngspice vectors are:

```
print @n1[ids]                ; after op
.save @n1[ids] @n1[region]    ; per-point recording in tran/dc/ac
.meas tran ipk MAX @n1[ids]    ; measurements over opvar vectors
.meas tran tcross WHEN @n1[ids]=0.5m RISE=1
show n1                        ; all opvars incl. string variables
```

Real *and integer* variables record per point; a variable without desc is deliberately not exposed (clean "no such parameter" error). String opvars display via show but cannot become vectors (vectors are numeric). Pinned in [examples/opvar_examples/](#).

3.3 Parameter access, **alter**, and sweeps

Parameters are readable and writable from the control language:

```
print @n1[r]                  ; read (instance param, (* type="instance" *))
alter @n1[r] = 2k              ; change + re-run setup on the next analysis
altermod @mm[r] = 2k           ; same for a model-card parameter
```

.dc sweeps over device parameters work for OSDI (and built-in) devices — the sweep refreshes the device per point exactly as alter would, restores the original value afterwards, and nests with other sweep variables:

```
.dc @n1[r] 500 1500 100        ; sweep an OSDI instance parameter
.dc @n1[r] 500 1500 500 vin 0 2 1 ; nested with a source sweep
.dc temp -40 125 5             ; temperature sweep; $temperature
                                ; tracks each point (°C in the deck,
                                ; K inside the model)
```

3.4 Analysis coverage

All core analyses treat OSDI devices as full citizens. The audited status of everything beyond op/dc/ac/tran:

Analysis	Status with OSDI devices
op, .dc, .ac, .tran	Fully supported; analytic Jacobians from autodiff (cross-checked against numeric derivatives on PSP103 to $\sim 10^{-5}$ in <code>examples/physcheck_examples/</code>)
.noise	Fully supported — all four noise-source types, correlated sources, op-dependent and frequency-shaped factors (§2.11)
.tf	Exact (transfer function, input/output impedance)
.pz	Exact for linear devices, bit-identical to built-in twins; nonlinear .pz failures are a stock ngspice quirk affecting built-ins identically
.sens (DC and AC)	Exact against analytic derivatives
.disto	Not supported for OSDI devices — the distortion kernel needs Taylor coefficients beyond first derivatives, which the OSDI ABI cannot provide. ngspice prints a prominent warning naming each affected device (it used to report silent zeros)
.sp (S-parameters)	Fully supported, any port count (1-port reflection through N-port); donoise (NF, noise parameters) is inherently 2-port
Transient noise (TRNOISE sources)	Propagates through OSDI devices correctly; device- <i>internal</i> noise does not enter .tran (same as built-ins)
.pss (experimental, needs <code>--enable-pss</code> at configure time)	OSDI devices converge like built-ins; strongly nonlinear circuits defeat the shooting method for both alike

Details and the exact pinned numbers: [examples/analyses_examples/](#) and [examples/rfanalyses_examples/](#).

3.5 S-parameters and Touchstone files

Ports for .sp are voltage sources tagged with a port number and reference impedance (V1 in 0 DC 0 AC 1 portnum 1 z0 50). After an sp run the plot holds complex S_{i_j} (and Y_{i_j}/Z_{i_j}) vectors plus Rbase (published automatically from port 1's z0; a manual `let Rbase = ...` still overrides).

Export — Touchstone v1, any port count, full option surface:

```
wrsnp out.s2p           ; classic: # Hz S RI R 50
wrsnp out.s2p ma ghz    ; magnitude/angle, GHz frequency column
wrsnp out.s2p db        ; dB-magnitude/angle
wrsnp out.y2p y         ; Y-parameters (normalized to Rbase per spec)
wrsnp out.z2p z mhz     ; Z-parameters, MHz
```

Options (ri|ma|db, s|y|z, hz|khz|mhz|ghz) combine in any order; the 2-port default output is byte-identical to the classic wrs2p, and $N \geq 3$ files use the spec's row-major layout.

Import — rdsnp reads any Touchstone v1 file (yours or a VNA's) into a new plot with a Hz frequency scale and complex vectors matching the .sp conventions, so measured data diffs against simulation in one expression:

```
sp lin 100 1MEG 1G
let s21sim = S_2_1
```

```
rdsnp measured.s2p          ; port count from the extension (or: rdsnp f 3)
let err = maximum(mag(S_2_1 - {sp1}.s21sim))
```

MA/DB files convert back to real/imaginary on read, Y/Z de-normalize to absolute values, and the imported plot's Rbase lets it round-trip back out through wrsnp. Pinned round-trip accuracy: 4×10^{-8} (the file's own 6-digit precision is the limit). See [examples/touchstone_examples/](#).

3.6 Monte Carlo

Both standard ngspice MC idioms reach OSDI parameters ([examples/montecarlo_examples/](#)):

The **reset** idiom — a random-valued .param feeds a model card; each reset re-throws the dice and re-runs OSDI setup:

```
.param rr = agauss(1k, 100, 3)      ; nominal 1k,  $\sigma = 100/3$ 
.model mm myres(r={rr})
...
.control
let n = 0
while n < 200
  reset
  op
  ...collect...
  let n = n + 1
end
.endc
```

The **alter** idiom — draw in the control language and alter the parameter (no re-parse, and matched devices can share one draw):

```
setseed 42                                ; whole ensembles bit-reproducible
let r = 1k + 33.3*sgauss(0)
alter @n1[r] = r
```

Three gotchas, pinned by the verify suite: every textual occurrence of a random {param} draws independently (use the alter idiom for matched devices); ngspice's sunif(0) is uniform on $[-1, 1]$, not $[0, 1]$; and wrdata cannot export control-created vectors (parse print output instead — see §4.5).

3.7 Statistical modeling inside the device

Monte Carlo can also live in the Verilog-A source itself: \$rdist_normal and friends give each *instance* an independent, reproducible draw (stable across Newton iterations — see §2.11), which is the right tool for per-device mismatch, with the simulator-side idioms above layered on top for lot-level variation.

3.8 XSPICE code models

Alongside the OpenVAF/OSDI device path, this ngspice is built with XSPICE enabled, so it can also load ngspice's code models — the A-device library of behavioural analog and event/digital blocks (gain, summer, limit, oscillators, ADC/DAC bridges, controlled sources, transmission lines, ...). Code models are compiled .cm shared libraries loaded with the codemodel command.

The prebuilt bin/<os>/<arch>/ bundle ships them ready to use:

```
bin/<os>/<arch>/
  ngspice, openvaf-r      the executables
  codemodels/*.cm        analog, digital, spice2poly, xtradev,
                          xtraevt, table, tlines
```


scripts/spinit loads the above relative to \$SPICE_LIB_DIR

Point **SPICE_LIB_DIR** at that bundle directory; ngspice reads scripts/spinit at startup, which loads every code model:

```
export SPICE_LIB_DIR="$PWD/bin/macos/apple-silicon"    # your platform's dir
./bin/macos/apple-silicon/ngspice -b my_deck.cir
```

The example scripts do this automatically — `_setup.py` sets `SPICE_LIB_DIR` to the resolved bundle, so codemodel A-devices work with no extra setup. A minimal use is a gain block ($v(out) = 2 \cdot v(in)$):

```
* xspice gain
Vin in 0 3
a1 in out gainblk
.model gainblk gain(gain=2.0)
Rl out 0 1k
.control
op
print v(out)
.endc
.end
```

which prints $v(out) = 6$. The loads are silent and gated on if `$?xspice_enabled`, so a deck that uses no A-device (or an ngspice built without XSPICE) is unaffected.

3.9 When something misbehaves

- Compile-time: `openvaf-r` diagnostics are located and specific (wrong construct in a condition, recursion cycles, width mismatches). `--lints` lists tunable lints; `-A/-W/-E` adjust their level.
- A model that rejects its configuration via `$fatal/$finish` *during setup* surfaces as “a Verilog-A device rejected its configuration during setup”, naming the device.
- `$strobe/$display` output appears on ngspice’s stdout — `printf-exact` formatting (§2.11) makes temporary debug output dependable.
- For convergence work: `$limit` genuinely engages iteration limiting, `nodesets` (electrical `n = 5.0;`) seed the solver, and `$discontinuity / $bound_step` steer the transient integrator (§2.6).

4 · Limitations and gotchas

An honest map of the edges: what is out of scope by design, what carries fallback semantics, the deliberate design decisions, and the traps that cost real debugging time in this project. Everything here is *documented behavior* — most entries are pinned by a verify script so they can't regress silently into something different.

4.1 Scope: Verilog-A, not full AMS

The project targets Verilog-A — the analog-only subset defined in Annex C of the Verilog-AMS LRM. Features that exist only in full Verilog-AMS are out of scope, most notably:

- digital/mixed-signal constructs — `reg`/`wire` logic, `always/initial` (digital) blocks, delays/event control on digital signals, connect modules and auto-insertion;
- **``default_discipline`** — the directive is AMS-only. (Implicit nets *are* Verilog-A, and work: an undeclared net takes its discipline from the connected port — [E-41](#).)
- discrete-domain disciplines — `domain discrete` combined with `natures` is rejected with a proper error per LRM 3.6.2.2.

When in doubt whether a construct is Verilog-A or AMS-only, Annex C of the LRM (in [docs/](#)) is the authority the compiler follows.

4.2 Features with fallback semantics

These compile and behave predictably, but the OSDI/ngspice target has no underlying mechanism for them, so they return their LRM-specified "mechanism unavailable" values ([E-12](#)):

Function	Behavior
<code>\$test\$plusargs / \$value\$plusargs</code>	<code>false</code> / default (ngspice has no <code>plusargs</code>)
<code>\$simprobe</code>	the supplied default
<code>\$analog_node_alias / \$analog_port_alias</code>	<code>0</code> (no runtime hierarchical aliasing)

4.3 Compile time vs. simulation time

The single most useful mental model for this toolchain:

- Structure binds at compile time. Hierarchy, generate blocks, `genvars`, `defparam`, `paramsets`, bus widths — all resolved by the elaboration pass. Consequently a generate condition or bound on a module parameter is an intentional, explained error: parameters arrive from model cards at simulation time, after structure is frozen ([E-67](#)).
- Behavior binds at simulation time. Loop trip counts, `if` arms, and expressions may depend on parameters (and the solution) freely — a model-card override genuinely changes how many times a for loop runs ([E-70](#)).

4.4 Documented design decisions

- **`limexp()`** is stateless ([E-13](#)): `exp(x)` below the overflow cutoff, tangent-continued above — exact in every analysis. A stateful, previous-iterate limiting version was built and reverted: correct converged values under limiting require SPICE's limiting-RHS correction, which applies to circuit unknowns, not to `limexp`'s derived argument. Use `$limit` for genuine iteration limiting.
- Random draws don't advance a seed ([E-10](#)): each `$random/$dist_*` call site is a deterministic function of (seed, call site). An advancing seed would return different values on every Newton iteration and destroy convergence. Sequences within one evaluation are therefore not available; per-instance/per-call-site independence is.

- **.disto** warns instead of pretending (E-62): the small-signal distortion kernel needs higher-order Taylor coefficients the OSDI ABI cannot carry; OSDI devices are skipped with a loud warning (they used to contribute silent zeros).
- **@(final_step)** fires only on success: a failed or interrupted analysis never fires it — “final” means the converged end of the run (E-53).

4.5 ngspice control-language traps

Pinned during this project’s own verification work — they will bite anyone scripting ngspice:

- The first line of a deck is the title, not a statement. A netlist line placed first is silently swallowed.
- **wrdata** mispairs control-created vectors (it attaches the current plot’s scale, which for a control-made vector is length 1). Export such vectors by parsing `print` output, or `wrdata` only plot-native vectors (E-66).
- **print** of an imported/foreign plot omits the scale column — `print frequency[0]`-style expressions separately (E-72).
- **sunif(0)** is uniform on $[-1, 1]$, not $[0, 1]$; and every textual occurrence of a random-valued `{param}` draws independently — matched devices need the `alter` idiom (E-66).
- Control scripts continue past aborted analyses — detect completion from data, not from echo markers after the analysis command (E-56).
- **sp lin 2 f1 f2** yields one point (a stock ngspice quirk; `lin 3` and up behave; E-63).
- A recompiled **.osdi** does not reload in-session — `pre_osdi` on an already-loaded path notes it and keeps the existing registration; restart ngspice to pick up a recompiled file (E-81).
- Module names that collide with ngspice built-in device types (`cccs`, `vccs`, `vcvs`, ...) are skipped with a warning — the built-in keeps the name, so pick another module name to actually use the model (this used to be a hard crash; E-76).

4.6 Assorted edges

- **break/continue** don’t exist in Verilog-A — the compiler rejects them with the `disable <block>;` idiom as the alternative (E-9, E-70).
- Analog operators can’t sit in loops or solution-dependent conditionals (LRM 4.5.1) — hoist them or unroll with `generate`; the diagnostics say which (E-70).
- Recursion in analog functions is illegal — direct and mutual forms are clean errors naming the cycle (E-59).
- Analog function arguments use the separated (non-ANSI) declaration form — after `analog function <type> <name>;`, give the direction and the data type as *separate* statements: `input x; real x;` (this is the classic Verilog-A style every LRM example and compact model uses). The combined declaration `input real x;` and ANSI-style function headers (`analog function real f(input real x); ...`) are not accepted — both are clean parse errors (unexpected token `'real'` / unexpected token `'('`). This applies to array arguments too: `output w; real w[0:3];`, not `output real w[0:3];`.
- String opvars display via `show` but can’t become vectors (ngspice vectors are numeric) — `print @n1[strvar]` fails with a clear message (E-69).
- **sin** is a reserved word in Verilog-A — don’t name a net `sin` (E-36).
- OSDI ABI pairing: this ngspice requires $\text{OSDI} \geq 0.7$; `.osdi` files from older compilers must be recompiled, and this compiler’s output won’t load into stock ngspice (E-54).
- Parameter default ranges: a parameter’s default is exempt from its own `from range` (the CMC “disabled feature” idiom) — but every user-given value is fully validated. Don’t rely on an out-of-range *given* value sneaking through (E-56).
- Corpus models that “fail” by design: some industry models reject default configurations on purpose (HiSIM’s `$port_connected` guards, VBIC floating internal nodes at $\text{RCX}=0$ — use `.option rshunt`, FBH-HBT needing `fb>0`). The triage lives in E-56.

5 · Enhancement index

One line per enhancement, in order. Doc links the detailed write-up in `enhancements_doc/`; Examples links the folder whose verify script pins the behavior (the result plots live in the example folders). This same table is the enhancement index of the top-level [README](#).

#	What it delivered	Doc	Examples
1	<code>absdelay()</code> transport delay via synthetic-node DAE	doc	absdelay
2	Indirect branch assignment (<code>V(out): V(x)==0</code> — ideal op-amps)	doc	indirect_assignment
3	Vectored (bus) nets and ports with bit-select access	doc	bus
4	<code>laplace_*</code> filter operators + array-variable declarations	doc	laplace , bessel_filter
5	Module instantiation (hierarchy via compile-time flattening)	doc	instantiation
6	Directives, <code><<</>>></code> , <code>slew/transition</code> , <code>zi_*</code> , <code>last_crossing</code>	doc	directive , shift , slew , zi , last_crossing
7	<code>@(initial_step)</code> gating + genuine variable persistence	doc	initial_step , variable_persistence
8	<code>generate for/genvar</code> + <code>cross()/above()/timer()</code> events	doc	generate , cross , timer
9	<code>noise_table(_log)</code> , <code>localparam</code> , <code>ground</code> , strings, <code>repeat/disable</code>	doc	noise , repeat , disable
10	<code>\$random</code> + all <code>\$dist_*/\$rdist_*</code> distributions	doc	rng
11	File I/O (<code>\$fopen...</code>) + string formatting/parsing (<code>\$sscanf...</code>)	doc	fileio , stringio
12	<code>\$simprobe</code> /aliases/plusargs as LRM fallbacks (last unsupported builtins)	doc	alias
13	<code>limexp()</code> kept stateless (documented decision); <code>ddx()</code> demo	doc	ddx
14	Array literals/aggregates, array parameters, dynamic indexing	doc	array
15	Multi-dimensional arrays (N-D, per-element param override)	doc	mdarray
16	<code>\$table_model</code> 1-D lookup tables (differentiable)	doc	table_model
17	2-D/3-D <code>\$table_model</code> (multilinear, exact Jacobian partials)	doc	mdtable
18	<code>real x[0:n]</code> declaration order + arrays in analog functions	doc	funcarray
19	<code>do ... while</code> loops	doc	dowhile
20	Array output/inout function arguments	doc	arrayout
21	<code>paramset</code> blocks	doc	paramset
22	Natural cubic-spline <code>\$table_model</code> (control "3")	doc	cubic_table
23	Array return values from analog functions	doc	arrayret
24	<code>\$discontinuity(n)</code> next-step clamp	doc	discontinuity
25	<code>\$simparam\$str</code> (+ <code>ngspice analysis_name/simulator params</code>)	doc	simparamstr

#	What it delivered	Doc	Examples
26	<code>ac_stim</code> crash fix + correct large-signal baseline	doc	acstim
27	<code>idtmod()</code> modulo-integrator fix	doc	idtmod
28	<code>idt()</code> initial condition survives into transient	doc	idtic
29	Port-flow probes <code>I(<port>)</code>	doc	portflow
30	Variadic analysis(<code>"ac", "tran", ...</code>)	doc	analysis
31	Complex poles/zeros in <code>laplace_*/zi_*</code> root forms	doc	complexpole
32	Integer persistent/event-state variables (compiler crash fix)	doc	intstate
33	Array case + array-literal function arguments	doc	arraycase
34	<code>{...}</code> concatenation and <code>{n{...}}</code> replication operators	doc	concat
35	Lexer hang on <code>//</code> comment at EOF	doc	comment
36	Probe-only branches (ideal ammeters, flow-only disciplines)	doc	signalflow
37	Operator-correctness audit (<code>~</code> , <code>const-folded >></code> , <code>string ?:</code>)	doc	operator
38	Precedence audit vs LRM Table 4-2 (% level fix)	doc	precedence
39	Derived natures (nature <code>X : Parent, : electrical.flow</code>)	doc	derivednature
40	<code>\$table_model</code> in any number of dimensions	doc	ndtable
41	Implicit nets in instance connections	doc	implicitnet
42	Correlated (same-named) noise sources sum coherently	doc	noisecorr
43	Variable declaration initializers, completed (N-D arrays)	doc	varinit
44	Paramset hidden system parameters (<code>.</code> , <code>\$mfactor</code>)	doc	paramsethsp
45	Net initializers (nodesets) + nature-attribute access	doc	netinit
46	Escaped identifiers + based integer literals	doc	escid
47	<code>`default_transition + transition()</code> fixes	doc	defaulttransition
48	String-literal escape sequences (single-pass unescaper)	doc	stresc
49	Hierarchical names + <code>\$root</code>	doc	hiername
50	Domain-binding validation (LRM 3.6.2.2)	doc	domainbind
51	Full <code>ac_stim</code> AC-RHS injection (OSDI ABI 0.6)	doc	acstim
52	<code>idt()</code> assert/reset forms (relaxation oscillators)	doc	idtassert
53	<code>@(final_step)</code> + analysis-phase lists on step events	doc	finalstep
54	Correct, node-free noise factors (OSDI ABI 0.7)	doc	noisejw
55	<code>\$finish/\$stop/\$fatal</code> honored + <code>\$discontinuity</code> step rejection	doc	simctrl

#	What it delivered	Doc	Examples
56	Corpus sweep: CMC default-range idiom + noise crash fix	doc	paramrange
57	Physics-accuracy validation suite	doc	physcheck
58	defparam hierarchical override	doc	defparam
59	LRM corners: event OR lists, \$realtime, port concat, recursion diags	doc	lrncorner
60	Multiple analog blocks — validation	doc	multianalog
61	Operator-argument audit — slew sign fix, \$limit, \$bound_step	doc	opargs
62	.dc @inst[param] sweeps + .disto warning + analyses tutorial	doc	analyses
63	RF analyses: N-port .sp, trnoise, PSS + span.c NaN fix	doc	rfanalyses
64	Touchstone export: auto-Rbase, N-port wrsnp, 1-port .sp	doc	touchstone
65	Preprocessor audit — macro-recursion guard	doc	preproc
66	Monte Carlo with OSDI — validation (+ zero-warning build chore)	doc	montecarlo
67	Generate audit — genvar fix, nesting, generate if/case	doc	generate
68	The compiler's own integration test suite, enabled (28 models)	doc	—
69	Operating-point variables end-to-end — validation	doc	opvar
70	Behavioral-loop audit — precise loop diagnostics	doc	analogloop
71	Display-task audit — full format surface + %b segfault fix	doc	display
72	Touchstone round 2 — MA/DB, units, Y/Z, rdsnp reader	doc	touchstone
73	This handbook, its PDF edition, and the README index	doc	docs/handbook
74	Performance benchmark — OSDI-vs-built-in twins at parity (RC ladder 0.99×), flagship compile times	doc	benchmark
75	Dynamic physics validation — reactive paths cross-checked (Cgg AC ≡ transient, charge conservation, tran-sine ≡ .ac)	doc	dynphys
76	Multi-module .osdi libraries — audit + registration fixes (duplicate warning, double-load skip, stock .model segfault)	doc	multimod
77	ngspice zero-warning build (33 → 0) — SDK macro clashes, %Id→%zu (readable plot-memory errors), codemodel dynamic_lookup	doc	—
78	casex/casez — don't-care digits in item literals as comparison masks (priority-encoder idiom)	doc	casexz

#	What it delivered	Doc	Examples
79	Benchmark round 2 — BSIM4 ring-oscillator twin (1.1% freq match), .ac/.noise throughput, KLU-vs-SPARSE + solver-independence pin	doc	benchmark
80	Temperature physics — $v_t \equiv kT/q$, dtemp alias fix, noise $\propto T$, MEXTRAM $E_g = 1.25$ eV, PSP103 ZTC flip	doc	tempphys
81	Session-lifecycle audit — reset loops leak-free, destroy all verified; once-per-excursion memory warning + no_mem_check, pre_osdi restart hint	doc	lifecycle
82	Provenance + compliance docs — full change reports for both tools (docs/change_log/) and the Verilog-A LRM compliance document (docs/compliance/)	doc	—
83	Transistor-level μ A741 demo — a Verilog-A BJT powering the textbook 20-transistor 741; datasheet figures emerge (104 dB, 0.75 MHz, 0.54 V/ μ s)	doc	opamp741
84	LRM example sweep — all 231 Verilog-AMS LRM-2023 code examples compile, plus 6 compiler defect fixes (port-branch panics, silent undefined modules, dead-op codegen, exit-0-on-error)	doc	lrn
85	<code>__FILE__</code> / <code>__LINE__</code> predefined macros + part-selects in instance connections (<code>inst (out[3:2], in)</code>) — the last two LRM-sweep findings; all 8 sweep defects now fixed	doc	filemacro , partselect
86	Hierarchical branch probes — <code>V(top.a1.b)</code> , <code>V(inst.branch(a,b))</code> , <code>I(inst.branch(<p>))</code> via synthesized 0V ammeters; + 2 DAE fixes (V-source-to-internal-node open circuit, collapse-of-probed-branch)	doc	hierbranch
87	Block-scoped parameters (<code>parameter/localparam inside begin: label, read label.name</code>) — feature validated end-to-end + clean diagnostic for the LRM's illegal <code>#(.blk.p(4))</code> override	doc	blockparam
88	Legacy generate <code><id> (start, end)</code> statement (obsolete Verilog-A 1.0 analog-block loop-unroll, LRM Annex C.4) with constant bounds	doc	legacygen
89	Name-then-range net/port declarations (<code>input in[0:2], electrical out[0:2]</code>) + an Annex E SPICE-primitives library (resistor/capacitor/inductor/sources/square-law MOS)	doc	arrayport , annexe

#	What it delivered	Doc	Examples
90	Multi-bit input bus port bit reads: fix scrambled terminal order when a vectored port (<code>input [0:2] in</code>) is not the last port in a non-ANSI header, so <code>V(in[k])</code> maps to the correct terminal	doc	busport
91	Multi-name name-then-range declarations (<code>input a[0:1], b[0:3], c;</code>) + parameter-dependent declaration widths (<code>electrical [0:N-1] out;</code> , <code>real w[0:N-1];</code> , folded from the parameter default)	doc	paramwidth
92	Freeze structural (width) parameters to <code>localparam</code> so a netlist override cannot desync the frozen width from behavioural code (fixes a silent out-of-bounds in E-91)	doc	paramfreeze
93	Warn when a netlist sets a fixed (<code>localparam</code>) parameter: <code>openvaf</code> flags it non-settable (<code>PARAM_FLAG_FIXED</code>), <code>ngspice</code> warns instead of silently ignoring the value	doc	paramnonset
94	New <code>ngspice</code> <code>pyplot</code> command — plot simulated vectors with <code>matplotlib</code> (a Python counterpart to <code>gnuplot</code>); <code>pyplot_</code> <code>terminal=png</code> renders headless to a PNG	doc	pyplot
95	Make the <code>pyplot</code> output file name optional — <code>pyplot v(out)</code> (or bare node names) defaults the base name to <code>pyplot</code> ; an explicit name still works	doc	pyplot
96	Parse a module-level <code>generate</code> <code>for/if/case</code> written without the optional <code>generate/endgenerate</code> keywords (was unexpected token <code>'for'</code> , or silently dropped the loop)	doc	baregenerate
97	Clean diagnostic instead of a compiler panic when a contribution's branch is entirely ground (<code>V(gnd) <+ ...</code>)	doc	groundcontrib
98	<code>pyplot</code> multi-panel subplots (<code>set pyplot_</code> <code>subplots=N</code> , <code>N</code> traces per stacked panel) + <code>matplotlib</code> style sheets (<code>set pyplot_</code> <code>style=dark</code>)	doc	pyplotpanel
99	<code>pyplot</code> vector export formats (<code>set pyplot_</code> <code>terminal=svg/pdf</code>) + figure size (<code>set</code> <code>pyplot_figsize="W,H"</code>)	doc	pyplotexport
100	Milestone audit & retrospective — full-tree re-verification (90/90 suites + 28/28 integration), provenance/link audit, and a look back at the first hundred	doc	—
101	<code>\$clog2</code> correctness — accept one argument (was a bad 2-arg signature) and return <code>ceil(log2 n)</code> (was off by one on exact powers of two)	doc	clog2

#	What it delivered	Doc	Examples
102	Name-then-range array parameters — parameter <code>real c[0:2]</code> (dims after the name), completing the name-then-range line (vars/nets/ports already had it)	doc	paramarray
103	<code>ceil()</code> of a runtime argument no longer crashes the compiler (the <code>llvm.ceil.f64</code> intrinsic was unregistered; <code>floor</code> worked)	doc	ceil
104	<code>\$rtoi / \$itor</code> real↔integer conversion functions (<code>\$rtoi</code> truncates toward zero, distinct from the rounding implicit cast)	doc	convert
105	<code>\$sscanf / \$fscanf</code> honour the format base (<code>%h/%x</code> hex, <code>%o</code> octal, <code>%b</code> binary) instead of ignoring it	doc	sscanf
106	String relational comparison (<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>) via lexicographic <code>strcmp</code> (completes the string comparison surface; <code>==</code> / <code>!=</code> already worked)	doc	stringcmp
107	<code>\$fgetc(fd)</code> single-character file read (completes the file I/O family: <code>\$fgets/\$fscanf/\$ftell/...</code> already existed)	doc	fgetc
108	<code>\$ungetc(c, fd)</code> one-character pushback (the peek/look-ahead companion to <code>\$fgetc</code>)	doc	ungetc
109	<code>noise_table/noise_table_log</code> interpolation corrected to the LRM (linear-in-f / log-log; both take Hz input)	doc	noisetable
110	<code>ngspice .option</code> <code>errpreset=conservative moderate liberal</code> — one knob for a coordinated tolerance/robustness set (Spectre-style); explicit options override regardless of order, <code>moderate</code> = historical defaults	doc	errpreset
111	<code>ngspice .option linesearch</code> — globalized (damped) Newton via Armijo backtracking on a new KCL-residual merit $\ F\ =\ G \cdot x-b\ $ (the merit ngspice lacked); result-neutral, off by default	doc	linesearch
112	ngspice KLU support for .option linesearch — fixed a SIGSEGV where <code>SMPmultiply</code> 's KLU path passed NULL ordering maps to <code>klu_matrix_vector_multiply</code> ; NULL now means identity ordering, so line search runs under both KLU and Sparse	doc	linesearch
113	ngspice KLU for noise + single-ended pole-zero — fixed the KLU adjoint solve (was non-transposed, silently wrong on asymmetric circuits) so KLU noise matches Sparse exactly	doc	analyses

#	What it delivered	Doc	Examples
114	ngspice KLU for sensitivity (<code>.sens</code> , DC & AC) — fixed a segfault where KLU setup gated on the main matrix's flag dereferenced the auxiliary <code>delta_Y</code> 's NULL matrix; now matches Sparse	doc	analyses
115	ngspice KLU for distortion (<code>.disto</code>) — the complex solve ran on a real-mode matrix so every harmonic came back zero; now converts <code>real↔complex</code> around the solve like <code>acan.c</code>	doc	analyses
116	ngspice KLU wrong-DC fix for decoupled OSDI nodes — a ground-referenced internal node got an all-zero, structurally-singular solver row; now tied to ground at setup (fixes 2 of 3 <code>KLU_XFAILs</code>)	doc	groundcontrib , hierbranch
117	ngspice PSS productionized — <code>.pss</code> was experimental (~230 trace lines/run); now built by default with the shooting trace gated behind <code>set ngdebug</code> . Foundation for the RF small-signal suite	doc	rfpss
118	ngspice PSS under KLU — <code>.pss</code> hung because reused KLU pivots exploded the timestep; a full re-factor each step makes KLU match Sparse. Now runs under both solvers	doc	rfpss
119	ngspice retain the PSS operating point — capture node voltages and device states per sample (with frequency + dims) as the substrate the periodic small-signal suite linearizes around	doc	rfpss
120	ngspice periodic small-signal Jacobian harmonics — walk the retained op-point, re-linearize and stamp $G+jC$ per sample, DFT to harmonics — the blocks the PAC conversion matrix is built from	doc	rfpss
121	ngspice PAC conversion-matrix engine — assemble the $(2M+1)N$ harmonic conversion matrix $H_{\{nm\}}=G_{\{n-m\}}+j\omega_m C_{\{n-m\}}$ and solve it (dense complex LU) — the numerical heart of PAC/pnoise/PXF	doc	rfpss
122	ngspice .pac command — user-facing periodic AC: <code>.pac <pss> <dec oct lin> N f0 f1</code> sweeps a small-signal input frequency, solving the conversion matrix per point → complex plot	doc	rfpss
123	ngspice finish .pac — adds a source-referenced stimulus (true transfer / conversion gain) and multi-sideband output (<code>maxsideband K → <node>_usb<k>/_lsb<k></code>). PAC complete	doc	rfpss

#	What it delivered	Doc	Examples
124	ngspice .pnoise command — fold each device's noise through the conversion-matrix adjoint $H^T \Psi = e_{\{out,0\}}$ per sideband; reduces exactly to .noise in the linear limit. RF small-signal #2	doc	rfpss
125	ngspice .pxf command — periodic transfer function, the adjoint of PAC; sideband-0 is bit-identical to the PAC response. Completes the RF periodic small-signal suite (PSS→PAC→Pnoise→PXF)	doc	rfpss
126	ngspice cyclostationary noise (.pnoise ... cyclo) — evaluate each device's PSD $S(t)$ at every PSS sample's bias and fold in the time domain; captures pumped devices' bias-dependent shot/flicker noise	doc	rfpss
127	ngspice pseudo-transient continuation (.option ptcont) — a $\dot{X}$ -embedded DC homotopy that follows a stable trajectory to the correct root on stiff circuits where plain Newton overshoots. Off by default	doc	ptcont
128	ngspice LTE-based dynamic integration order (.option dynorder) — pick the Gear order per step from the truncation-error limit (unlocking the unused orders 3–6); up to 8.9× fewer steps at matched accuracy. Off by default	doc	dynorder
129	ngspice sweep progress bar — the throttled Reference value status line gains a live progress bar + percentage during tran/AC/DC/noise sweeps (stdout only, never in the rawfile). 22/22	doc	progressbar
130	ngspice built-in optimizer — new optimize command: a derivative-free Nelder-Mead search that varies alter parameters, re-runs an analysis, and minimizes an objective in normalized [0,1] space. 9/9 vs analytic optima	doc	optimize
131	ngspice transient checkpoint / restart — savestate/loadstate serialize the full transient integration state to disk and resume it, even in a fresh process (stock could only resume in memory). Sparse-only; 19/19 bit-identical	doc	checkpoint
132	ngspice periodic S-parameters (.psp) — small-signal S-parameters around a PSS op-point including input↔sideband conversion (mixers/switched circuits); sideband-0 reduces exactly to .sp . Both solvers; 8/8	doc	psp

#	What it delivered	Doc	Examples
133	ngspice quasi-periodic two-tone steady state (qpss) — two-tone spectrum incl. IM3 via transient + direct DFT at each exact intermod frequency (commensurate tones). Solver-independent; 11/11 incl. the 3:1 IP3 law	doc	qpss
134	ngspice Harmonic Balance (hb) — frequency-domain periodic steady state by Newton, using the E-121 conversion matrix as the exact Jacobian; nonlinear reactive via $C(v)v'$ (no charge extraction). Both solvers, built-in + OSDI; 8/8	doc	hb
135	ngspice HB source-stepping continuation — an adaptive $\lambda:0 \rightarrow 1$ source homotopy that makes E-134 HB converge on strongly-driven circuits where a cold Newton diverges; bit-identical to the plain solve when it isn't needed. 9/9	doc	hb
136	ngspice two-tone Harmonic Balance (qpss ... hb) — a frequency-domain 2-D HB engine = true incommensurate QPSS; devices sampled on a 2-D phase grid, sources via oversampled least-squares APFT. Both solvers; 7/7	doc	qpss
137	ngspice two-tone QPAC (qpac <f_in>) — small-signal quasi-periodic AC around the QPSS-HB op-point, reporting the response at every sideband $f_{in} + k_1 \cdot f_1 + k_2 \cdot f_2$. One dense solve, solver-independent by construction. 7/7	doc	qpac
138	ngspice two-tone QPnoise (qpnoise <out> <f_in>) — quasi-periodic noise folding every device's noise over all sidebands via one adjoint solve $H^T \Psi = e_{\{out, (0,0)\}}$; reduces exactly to .noise with no pump. 6/6	doc	qpnoise
139	ngspice cyclostationary QPnoise (qpnoise ... cyclo) — time-varying device PSD $S(t)$ folded in the time domain (per-sample junction settling); a hard-pumped diode shows $\sim 8\times$ switching-mixer noise enhancement. 10/10	doc	qpnoise
140	ngspice oscillator phase noise (hbosc + phasenoise) — autonomous HB (bordered Newton for $V + \omega 0$) plus a carrier-sideband adjoint giving $L(df)$ in dBc/Hz; -20 dB/dec skirt and thermal $L \propto T$ validated. 8/8	doc	phasenoise

#	What it delivered	Doc	Examples
141	ngspice two-tone QPXF (qpxf <out> <f_in>) — quasi-periodic transfer function, the adjoint of QPAC; (0,0) transfer bit-identical to QPAC by reciprocity. Completes the two-tone small-signal suite. 6/6	doc	qpxf
142	ngspice QP small-signal frequency sweep — qpac/qnoise/qpxf gain a <dec oct lin> N f0 f1 sweep of f_in, emitting a plottable ngspice plot (conversion gain / NF / image-rejection curves). 5/5	doc	sweep
143	ngspice least-squares curve fitting for optimize — a gradient-based -target least-squares mode (Levenberg-Marquardt) over one or more -analysis stages; ~27 vs 67 evals vs the simplex; OSDI diode extraction. 23/23	doc	fit
144	ngspice optimize tunes symbolic .param values — new knob kind -dparam varies netlist .param symbols (via alterparam + a quiet reset re-source), mixing freely with -param device knobs. Closes the last E-143 follow-up. 31/31	doc	fit
145	ngspice optimize tunes .model-card parameters — a third knob kind -mparam varies model params named @<model>[<param>] (e.g. @dmod[is]), which are neither alter-reachable nor .dc-sweepable; applied in place with altermod (no re-source, like -param). All three kinds — -param (instance), -mparam (model), -dparam (.param) — mix in one run, so the optimizer now covers every circuit knob. In com_optimize.c only. Verified 39/39 (adds OSDI + built-in model-param fits, model+instance joint fit)	doc	fit
146	ngspice universal sweep command + .sweep card — sweeps any knob, generalizing .dc (sources/resistors/instance params only). Auto-detects the kind: device/instance/source via alter, @model[param] via altermod, .param via alterparam+reset — so model params and .params (impossible with .dc) are sweepable. Runs a chosen inner analysis (default op) per point and records outputs into a plot; .sweep card does it from the netlist (re-entrancy-guarded). Verified 11/11: sweep R1 == built-in .dc R1, model/.param sweeps vs analytic, AC/tran inner analyses, card == command	doc	sweep

#	What it delivered	Doc	Examples
147	openvaf-r nested ?: no longer compiles in exponential time — a robustness campaign (117 production models + adversarial inputs + 4000 fuzz iterations) found the body validator re-validated both ternary branches (fell through the Select arm to walk_child_exprs), so N nested ? : were validated 2^n times — depth ~30 hung the compiler. Fix: return from the Select arm. $O(2^n) \rightarrow O(N)$ (depth 160: hopeless $\rightarrow$ 0.11 s); behaviour-preserving (0 verdict flips across all 117 models). Campaign found 0 panics/segfaults on 4000 fuzz iterations. Verified 7/7	doc	nested
148	openvaf-r compiler hardening: parser depth / include cap / array cap — closes E-147's three lower-severity findings, turning pathological input from crashes/hangs into clean diagnostics: a parse-time expression-depth limit (1000) stops deep nesting/chains overflowing the recursive-descent parser; an <code>`include depth cap (64, new IncludeRecursionLimit)</code> stops a self-including file overflowing the stack; an array element cap (~1M, new <code>ArrayTooLarge</code>) refuses to materialize <code>real x[0:1000000000]</code> (variable/param/net-bus/instance arrays). Behaviour-preserving (0 verdict flips across 117 models; unit tests pass). Verified 17/17	doc	robustness

#	What it delivered	Doc	Examples
149	ngspice Latin-Hypercube Monte Carlo sampling (mcsample) — low-discrepancy statistical sampling, closing a $\times$ gap vs commercial tools. <code>mcsample lhs <N></code> [seed <s>] puts the netlist stochastic functions (agauss/gauss/aunif/unif/limit) into a stratified sampler: each random dimension's range is split into N equal strata hit exactly once (independent permutation per dimension; Gaussians stratified in probability space via an inverse-normal-CDF probit), so N runs cover each distribution evenly instead of the clumping of plain MC. The existing reset-driven idiom is unchanged — one reset = one stratified sample (advanced on the NUPADECKCOPY pass edge); <code>mcsample random/off</code> reverts. Front-end only, so solver-independent. Verified 5/5 under both solvers: stratification (each of 48 strata once vs random's gaps), multi-dimension, $\sim 130\times$ lower Var(mean) at same N, reproducibility, analytic correctness	doc	lhs
150	ngspice high-sigma rare-event estimation (highsigma) — reaches the rare tail plain MC can't: $4-6\sigma$ failure probabilities (SRAM/standard-cell yield) needing 10^7-10^9 runs. <code>highsigma <N> [-scale <λ>] -metric <expr> [-max <hi>] [-min <lo>]</code> uses scaled-sigma importance sampling — inflates every Gaussian .param σ by λ so the failure region is sampled often, then reweights each sample by $p_{\text{nom}}/p_{\text{inflated}}$ for an unbiased estimate; direction-free (no gradient/MPFP). Reports P(fail), relative error, equivalent sigma-to-fail (in <code>highsigma_*</code> vectors). Front-end/solver-independent. Verified vs analytic $\Phi(-\beta)$: $\beta=4 \rightarrow \sigma 4.000$, P $3.16e-5$; $\beta=5$ (P=$2.87e-7$) from 6000 runs where plain MC sees ~ 0; two-sided, reproducibility, multi-parameter. Sparse-only (heavy deck)	doc	highsigma

#	What it delivered	Doc	Examples
151	ngspice process/mismatch correlations + packaged yield (mccorr/mvnorm, montecarlo) — closes the last two $\triangle$ statistical rows. mccorr <k> <matrix> registers a k×k correlation matrix (Cholesky); a new .param function mvnorm(i) returns the i-th correlated standard normal per sample ($y=L \cdot z$), so matched-device process/mismatch correlation is native and composes with LHS (E-149) + importance sampling (E-150). montecarlo <N> [-lhs] (-spec <metric> [-max <hi>] [-min <lo>])... packages the yield flow: pass only if all specs hold, reports yield + Wilson 95% CI + per-spec violations (corners via .lib). Front-end/solver-independent. Verified: corr 0.711/−0.614, non-PD rejected, single/multi-spec yield vs analytic, -lhs ~800× lower yield variance, correlation raises joint yield; demo ~100% correlated vs ~74% independent	doc	yield
152	ngspice KLU matrix reordering + scaling controls — the KLU direct solver ran on hard-coded defaults; now .option klu_ordering=amd colamd, klu_scale=none sum max, klu_bt f=on off (friendly names → KLU integer codes), plus a fixed klu_memgrow_factor (was a no-op boolean). Follows the existing option chain (optdefs → cktsopt → TSK/CKT/SMP structs → cktdojob/niinit → klsmp.c sets Common→ordering/scale/bt f after klu_defaults). KLU-only, changes only how the matrix factors, never the solution. Verified: all settings physically identical (rel spread 2.8e-14); AMD≠COLAMD and scale=max≠none in the last digits (proof the knobs reach KLU); invalid values warn; default bit-for-bit unchanged	doc	klu

#	What it delivered	Doc	Examples
153	ngspice Levenberg-Marquardt trust-region Newton (.option trustregion) — completes the damped/trust-region-Newton row alongside the E-111 line search: damps the Jacobian $(x_{k+1} = x_k - (J + \mu I)^{-1} F, \mu = \lambda \cdot \ \text{diag}(J)\ , \text{Marquardt-scaled})$ rather than just the step length, re-aiming the step to regularize an ill-conditioned Jacobian; rejected residual-increasing steps retry with grown $\lambda, \lambda \rightarrow 0$ on success. Result-neutral (bit-identical to plain Newton, both solvers; new SMPdiagNorm). Honest finding: measured zero rejections — ngspice globalizes at the device level (junction limiting) before the residual, so a solver-level trust-region is inert on typical circuits	doc	trustregion
154	ngspice Envelope Following (envelope) — the last $\times$ in the RF/periodic-steady-state suite: for a carrier-driven circuit with a slow amplitude/phase envelope over many carrier periods, samples the state once per period and integrates the slow drift, jumping M periods at a time with an implicit monodromy step $([(1+M)I - M \cdot \Phi] \Delta Y = -G, \Phi = d\phi/dY)$ that is A-stable — tracking a resonator's envelope where the naive explicit jump blows up (which had shelved a forward-Euler attempt). Adaptive-M LTE control; self-started trapezoidal one-period map. New EFanalysis + envelope command, emits an envelope plot. Verified vs full .tran (<3% on Q~3160, ~1.6% on Q~316, bounded over 3000 periods, both solvers); 26 samples reproduce ~3000 periods. Completes the RF suite	doc	envelope
155	ngspice RC network reduction (reduce) — collapses a post-layout parasitic R/C network into a small equivalent .subckt of R's and C's preserving port behaviour over DC..fmax, via TICER Schur-complement elimination of interior nodes (realizable R/C, DC exact, factor trades reduction vs in-band accuracy). Ports auto-detected as nodes touched by non-R/C devices (sources, transistors, OSDI) + ground + keep. New CKTreduceRC engine + reduce command; both solvers. Verified: identity reduction bit-exact, 4× fewer nodes at <0.25 dB in-band, OSDI auto-port. Moves the post-layout RC-reduction row to ✓	doc	reduce

#	What it delivered	Doc	Examples
156	ngspice Sparse RC reduction — scales E-155's reduce from the dense ~2500-node cap to real post-layout size (millions of nodes): per-node adjacency + minimum-degree elimination (like sparse LU, zero fill on chains) + a maxdeg fill guard (leaves a dense mesh core intact instead of blowing up — unguarded a mesh made 3308 fill edges/step, guarded 9). Same TICER math/DC-exact/auto-port; 65k nodes in ~4 s. Also prints the correct .subckt instantiation (terminal order). Verified both solvers: identity bit-exact, 4× at <0.25 dB, 8001-node network reduces	doc	reduce
157	ngspice Device aging (aging command) — the reliability stress→degrade→re-simulate flow (HCI/NBTI/TDDDB). aging <t_target> [dynamic <tstop>] finds every aging-capable device, computes its degradation after t_target seconds, writes it back, and re-stamps the circuit so later analyses see the aged devices. Model-agnostic: a Verilog-A/OSDI model opts in with a degradation-rate opvar (agerate) + a per-instance age parameter; the engine integrates the rate into a dose and feeds it back, the model owns dose→shift. static = rate(op)·t; dynamic = time-weighted mean rate·t (duty cycle). New frontend/com_aging.c; solver-independent. Verified both solvers: dose = rate·t exactly, NBTI $\Delta V_{th} \propto t^{0.25}$ to 5 sig figs, current drops, monotone, near-threshold degrades more in relative current, 30%-duty gate ages 0.30×, sub-threshold accrues zero. Moves the reliability aging rows to ✓	doc	aging
158	ngspice Power-grid EMIR (emir command) — electromigration + IR-drop sign-off, completing the reliability trilogy. After a DC solve reports IR-drop (each node's sag below the rail — worst node + violations) and electromigration (per wire-segment current density 'J=	I	/(w·thick), ranked, with Black's-equation $MTTF/ref=(J_{max}/J)^n$. Key physics: EM is set by current *density* not current — a fat trunk is safe while a thin low-current wire voids first (why grids taper width). Reads node voltages from the plot + @Rk[i]/@Rk[w]via nutmeg. Newfrontend/com_emir.c; solver-independent. Verified both solvers: exact IR-drop + linear scaling, $J=I/(w \cdot thick)$ exact, narrow-wire-worst physics, $BlackJ^n$, violation count, rail auto-detect, OSDI load. Moves the reliability EMIR row to ✓

#	What it delivered	Doc	Examples
159	Real production compact-model bring-up (BSIM4 + EKV) — runs the CMC models people tape out with through <code>openvaf-r</code> → OSDI → ngspice. BSIM4 (~12.6k-line industry MOSFET) compiles in ~6 s and matches ngspice's built-in BSIM4 to ~2% (output) / ~4.5% (transfer) — a rigorous self-check. EKV (no ngspice built-in) is brought up purely via OSDI. Finding: OSDI keeps internal nodes static (no collapsing), so BSIM4's internal S/D nodes float with default <code>rdsmode=0</code> → <code>Id=0</code> ; <code>rdsmode=1</code> (near-shorts them) is the fix. Validation/example (no source change); both solvers, 6 checks	doc	compactmodels
160	CMC compact-model coverage sweep — drives every real CMC model bundled with OpenVAF through <code>openvaf-r</code> → OSDI → ngspice (like the E-84 LRM sweep, for production models). 19/19 compile, 19/19 load across every device class (MOSFET, FinFET, SOI, GaN HEMT, surface-potential, bipolar/HBT, diode). Per-class conduction validated: BSIM4/BSIM3 vs ngspice built-in (~2%/~7%), EKV, HICUM/L2 $\beta \approx 100$ (Gummel), DIODE_CMC exp turn-on. Findings: the E-159 <code>rdsmode=1</code> issue is BSIM4-specific (BSIM3 works out of box; its "zero" at $V_{gs}=1$ is a high ~1.7 V default V_{th}); HiSIMHV needs <code>cosubnode=1</code> . Validation/example (no source change); SPARSE-only + regression-excluded (~35 s), 7 checks	doc	cmcsweep
161	Dynamic (AC/RF) compact-model validation — extends E-159/160 into the dynamic domain, exercising OSDI reactive (charge) Jacobian stamping + <code>.ac</code> . BSIM4 C-V: $C_{gg}(V_{gs})$ from <code>.ac</code> (~40→129 fF, subthreshold→inversion) matches ngspice built-in BSIM4 to <1%. f_T (	h21	=1): OSDI BSIM4 ~3.5 GHz matches built-in ~1%; HICUML2 SiGe HBT (realistic $t_0=10$ ps + 1 fF caps) $f_T \approx 15.5$ GHz at the transit-time limit $1/(2\pi \cdot t_0)$, rises with I_c . Validation/example (no source change); both solvers, 4 checks

#	What it delivered	Doc	Examples
164	Large-signal RF of a real transistor (P1dB / IP3) — drives a HICUM/L2 SiGe-HBT common-emitter amp hard via transient+Fourier/FFT: AM-AM compression (gain 17.8→20.7 dB expansion then P1dB), HD3 with the 3:1 slope, two-tone IM3 (~-30 dBc). $IIP3 = A/\sqrt{(3 \cdot HD3)} \approx 0.134$ V constant across drive. Finding: HB/QPSS (E-134/136) don't converge on this stiff production model in an amp config (hb error 103; two-tone qpss too slow) → transient+FFT is the robust route. Closes the production-model validation loop (DC/coverage/small-signal/large-signal). Validation/example (no source change); both solvers, 4 checks	doc	rfpa
165	Production compact-model noise validation — exercises OSDI's .noise stamping (white_noise/flicker_noise) on production models. BSIM4 output-noise $S_v(f)$ matches ngspice built-in BSIM4 to ~1.5% across the band — $1/f$ flicker ($\sqrt{S_v} \propto 1/\sqrt{f}$) + flat thermal floor. HICUM (no built-in) checked vs physics: white noise (shot+thermal, no default flicker), output floor tracks collector shot $\sqrt{(2q \cdot I_c \cdot RC^2)}$, scales $\sqrt{I_c}$. Completes the production-model validation loop. Validation/example (no source change); both solvers (.noise under KLU via E-113), 5 checks	doc	modelnoise

#	What it delivered	Doc	Examples
166	Electro-thermal / self-heating validation — exercises a model's own internal self-heating node, which OSDI stamps as an extra thermal terminal ("voltage" = junction temperature rise, "current" = dissipated power). On HICUM/L2 (flsh=1, params rth/cth) the electro-thermal analogy holds: current-biased (stable OP), $V(tnode) = P_{diss} \cdot rth$ to machine precision over a decade of power and two rth; at fixed I_c self-heating lowers V_{be} by a consistent -1.53 mV/K (classic bipolar TC), more for larger rth; the thermal transient is single-pole $\tau = rth \cdot cth$ (63.2% at one τ , scaling with both); under fixed V_{be} it is positive feedback $\rightarrow$ thermal runaway (5.9 mA isothermal vs tens of A self-heated at $V_{be} = 0.82$ V). KLU note: a floating thermal node ($rth=0$) is structurally singular under KLU, so the isothermal reference ties it to ground. Completes the production-model validation loop end to end. Validation/example (no source change); both solvers, 6 checks	doc	electrothermal
167	Cross-model self-heating sweep — breadth follow-up to E-166 (like E-160 was to E-159): the same electro-thermal analogy $V(tnode) = P_{diss} \cdot rth_{eff}$ through every self-heating CMC model with a thermal terminal, across four device classes: HICUM/L2 (HBT, flsh/rth), ASMHMET (GaN HEMT, shmod/rth0), BSIMBULK (bulk MOSFET, SHMOD/RTH0 per-width), BSIMCMG (FinFET, SHMOD/RTH0 geom-normalized). For each: thermal node reads 0 with SH off; $V(tnode)/P_{diss}$ is bias-independent and scales linearly with the rth param ($8\times$ sweep); exact = $rth/rth0/RTH0 \cdot W^{-1}$ to machine precision for the three direct/per-width models (2000/20/1000 K/W). One log-log plot: all four on their $P_{diss} \cdot rth$ lines over five decades of power. KLU: $1e12\Omega$ thermal-probe resistor keeps the node non-singular when SH off. Scope: BSIM-SOI needs internal P_{diss} (body/impact-ionization $\neq I_d \cdot V_{ds}$). Validation/example (no source change); both solvers, 12 checks	doc	cmcsselfheat

#	What it delivered	Doc	Examples
168	RF noise figure of an LNA — lifts device noise (E-165) to a circuit figure of merit: the noise figure of a common-emitter HICUM/L2 LNA, extracted from <code>.noise</code> as $NF=10 \cdot \log_{10}(inoise^2/4kT \cdot R_s)$ ($T_0=290$ K). Noise match: $NF(R_s)$ U-curve with optimum source resistance ($R_{s_opt} \approx 200 \Omega$, $NF_min \approx 0.85$ dB); voltage-noise + current-noise asymptotes cross at the minimum; extracted $in \approx 2.59$ pA/$\sqrt{\text{Hz}}$ = base shot $\sqrt{(2q \cdot I_B)} = 2.53$ (3%). Friis cascade $F_{total} = F_1 + (F_2 - 1)/G_{av1}$: high first-stage gain $\rightarrow F_{total} \rightarrow F_1$ (first-stage dominance, +0.01 dB, matches to 0.001 dB); low gain $\rightarrow$ 2nd stage adds measurable 0.05 dB (still exact). Validation/example (no source change); both solvers (<code>.noise</code> under KLU via E-113), 6 checks	doc	noisefigure
169	ngspice interactive command-line syntax highlighting — the interactive prompt colors the command line as you type: command word green when it is a recognized command (looked up in the real <code>cp_coms</code> table, so color matches whether it runs), red when it can't become any command, neutral while still a valid prefix (<code>plo</code>$\rightarrow$neutral, <code>plot</code>$\rightarrow$green, <code>plt</code>$\rightarrow$red); numbers yellow, "strings" magenta, - options cyan. Live via GNU readline <code>rl_redisplay_function</code> override (only when the line fits one row, else deferred so wraps aren't corrupted); on by default at a TTY, off via <code>set no_syntax_highlight/NO_COLOR</code>, never injects codes into piped output. New <code>synhl</code> command previews + batch-tests the engine. New <code>src/frontend/syntaxhl.c</code> + <code>src/main.c</code> hook; needs a readline build (shipped binaries are). 11 checks (engine + live pseudo-terminal)	doc	syntaxhl

#	What it delivered	Doc	Examples
170	ngspice semantic syntax highlighting — extends E-169 from <i>lexical</i> to <i>semantic</i> : checks whether the signals and expressions typed exist and parse. An unknown signal <code>v(typo)/i(nope)/@dev[bad]</code> turns red (read-only lookup in the current plot via <code>vec_fromplot</code>), a valid one stays default; inside an expression only the invalid signal reddens (<code>v(a)+v(zzz) → just v(zzz)</code>). A malformed expression (<code>v(a)*v(b)</code>) reddens whole (real parser <code>ft_getpnames_from_string</code> , muted, tree freed); a half-typed one (<code>v(bP, v(a)+)</code>) stays neutral. Error output red — <code>cp_err</code> wrapped (<code>funopen/fopencookie</code> , <code>cp_curerr</code> too so the per-command reset keeps it). Gotcha: parser's <code>PPerror</code> writes to raw <code>stderr</code> , so the check also mutes fd 2 across the parse (else <code>syntax error...</code> leaked onto the prompt). Signal-validity scoped to unambiguous <code>v()/i()/@</code> forms. Command execution unaffected. 19 checks (lexical + semantic via pseudo-terminal)	doc	syntaxhl
171	ngspice KLU/Sparse solver audit — KLU pole-zero fixed — <code>.pz</code> under KLU silently gave garbage for complex poles/zeros (series RLC: 4 bogus real poles vs $-5000 \pm j \cdot 999987$. 5; real-axis roots right → RC regression never caught it). Defect 1: <code>spDeterminant_KLU</code> mixed-pivot formula $(1/(U_x \cdot R_s), U_z \cdot R_s) + \text{reciprocal}$ correct only for real pivots (KLU <code>Udiag=actual</code> , Sparse <code>Diag=reciprocal</code>); real branch loop never ran, divided not multiplied, imag part uninitialized; permutation parity <code>#nonfixed/2</code> wrong for >2-cycles. Rewritten <code>det=sign(P) \cdot sign(Q) \cdot \prod(Udiag \cdot R_s)</code> , exact cycle parity → matches Sparse ~14 digits. Defect 2: PZ's <code>PivRel=0.0</code> unsanitized → <code>tol=0</code> → zero inductor diagonal accepted as pivot at <code>s=0</code> → spurious singular → fake root at origin. Both reorders now sanitize like Sparse. E-113 "finite-zero not supported" guard removed (root cause was defect 2). 6 root-set-parity checks + analytic anchors; regression 134/134	doc	klupz

#	What it delivered	Doc	Examples
172	ngspice KLU balanced pole-zero + full partial pivoting — closes E-171's scope items; no analysis remains Sparse-only under KLU. Balanced/differential .pz fold <code>col(b)+=col(a)</code> (<code>SMPcAddCol</code>) gained a KLU branch + union-pattern reservation in <code>CKTpzSetup</code> (KLU's CSC pattern is fixed at conversion; Sparse creates fill on the fly); <code>pzan.c</code> guards removed. And the out-of-range-PivRel fallback became full partial pivoting tol=1.0 : with pattern-only fixed ordering + tol=0.001 , extreme-	s	(~1e19+) factorizations picked catastrophically-cancelling pivots — exact-rational det +4.36e11 vs pivot product -2.05e12 — minting spurious far-field roots; full pivoting is KLU's only value-adaptive lever (Sparse re-runs Markowitz per trial). Also cured the twin-T conjugate-pair stall (6/6 roots). 9 parity checks; harness balanced-pz skip removed; regression 134/134
173	ngspice eigenvalue-based pole-zero (options pzeig) — alternative to the fragile Muller-on-determinant driver. Affine pencil G, C extracted densely from two <code>CKTpzLoads</code> (new <code>SMPdenseExtractReal</code> , affinity check → clean fallback); shift-invert: factor $(G+\sigma C)$ once, $M=(G+\sigma C)^{-1}C$ via n sparse solves, roots $s=\sigma-1/\mu$ (infinite eigenvalues → $\mu=0$, dropped); new self-contained balance/Hessenberg/Francis-QR eigensolver (<code>maths/dense/eig.c</code> , no LAPACK). Bandpass: Muller's iteration-limit warning gone, same roots; 10-pole ladder == analytic tridiagonal formula; twin-T 6/6 both solvers; imaginary zeros exact; balanced OK. Default stays Muller; cap 2000 unknowns. 13 checks; regression 136/136	doc	pzeig
174	ngspice help all crash fix — interactive <code>help all/help montecarlo</code> crashed with <code>tvprintf failed / fatal exit(-1)</code> : ngspice uses each command's help text as a <code>printf</code> format (<code>out_printf(co_help, cp_program)</code>), only one arg (<code>cp_program→one %s</code>). The E-151 <code>montecarlo</code> help had a literal <code>Wilson 95% CI</code> ; the <code>%</code> is an invalid specifier → fatal. Fixed by escaping <code>%%</code> in both command tables. New <code>helpcmd_examples</code> guard: runtime <code>pty help all</code> no-crash + static scan of <code>commands.c</code> for any bare- <code>%/multi-%s</code> help string. Regression 137/137	doc	helpcmd

#	What it delivered	Doc	Examples
175	ngspice RF-suite audit — dropped parametric term in periodic small-signal analyses — LPTV conversion matrices used the column frequency $j\omega_m \cdot C_{\{n-m\}}$, dropping $\dot{C} \cdot \delta v$: pumped-varactor conversion sidebands exactly ω_{in}/ω_{out} ($3\times/5\times/7\times/9\times$) too small in pac/psp/pnoise/pxf/qpac/qpnoise/qpxf/phasenoise; LTI unaffected (why all prior checks passed). HB residual correctly keeps ω_m (chain rule $dQ/dt=C \cdot \dot{v}$) $\rightarrow$ mode-flag fix, HB byte-identical, linear PAC control identical pre/post. Truth = transient+Fourier on OSDI varactor; fixed qpac within 2% on all sidebands. Clean: .sp/Touchstone 17/17, HB & QPSS analytic anchors exact. 5 checks $\times$ both solvers; regression 138/138	doc	rfconv
176	ngspice driven-mode PSS shooting — the oscillator-style PSS hunted the fundamental even on driven circuits via a breakpoint grid $\propto$ steady_coeff (0.2 ps spacing $\rightarrow$ 9.6M steps by $2\mu s$; residual floored, never converged). Driven mode (auto-detected from time-varying sources): exact source period, no hunt/grid ($8e-9$ in 17 cycles), max-step clamp T/psspoints (coarse-orbit trap: swing 0.1651 vs 0.15718 analytic), FFT strongest-line relaunch disabled, autonomous path untouched. rc_pss 4min $\rightarrow$ 0.05s exact-f, psp 406s $\rightarrow$ 0.9s; rfpss+rfanalyses join every sweep both solvers; direct .pac varactor vs truth <1%. 6 checks $\times$ both solvers; regression 145/145	doc	pssdriven
177	ngspice pnoise folding referee + folded-flicker fix — 3-layer referee (from-scratch Python LPTV matrix, TRNOISE Monte-Carlo ratio, LTI/white limits): white folding measured-correct to 6 digits. BUG: stationary pnoise/qpnoise/phasenoise evaluated ALL folded sidebands at the output frequency; flicker/noise_table must use the source-side $ f+k \cdot f_0 $ — pre-fix $\equiv$ wrong-model digit-for-digit (21% over, unbounded f^2/f_0), post-fix $\equiv$ correct model. phasenoise $1/f^3$ region gains most. Cyclo frequency-flat assumption documented. 5 checks $\times$ both solvers; regression 146/146	doc	pnoisefold

#	What it delivered	Doc	Examples
178	ngspice exact separable cyclostationary folding + HB DC-source fix — removes the cyclo mode's frequency-flat limitation: per-generator envelope harmonics B _q via load POLARIZATION (5 DEVnoise sweeps/sample, no device-API change) fold colored noise from its SOURCE frequency $ f+q\cdot f_0 $, shapes MEASURED pointwise (noise_table works); flat generators keep the exact any-rank identity; 2-D port to qpnoise ... cyclo. Physics: flicker sees $\frac{1}{2}m^2$, white sees $\frac{1}{2}m^2$ — old law 23% high on sin -modulated flicker ($8/\pi^2$), 34% on the conversion circuit. BONUS BUG (4th accidental-correctness): both HB Newtons double-subtracted DC sources — every DC bias voltage converged to exactly 2× (flicker $\propto I^{AF}$ off 2^{AF}); fixed → qpnoise cyclo $\equiv$ pnoise cyclo digit-for-digit across machineries. 6 checks × both solvers; regression 147/147	doc	cyclofold
179	ngspice standard-analyses audit — referee'd .disto (Volterra $\equiv$ analytic kernels $\leq 0.06\%$ incl. freq-dependent loads; junction-cap $\equiv$ HB to 6 digits), .noise integrals (kT/C + flicker log-law), .sens DC $\equiv$ FD at nonlinear OP, nonlinear-OP .pz (E-62 'quirk' = input convention). FIXED: .tf i(v _m) output impedance always 1e20 (35-year Berkeley SPICE3 sign clamp); KLU AC .sens truncated to 1 freq (loop-var clobber in the E-114 block); .meas DERIV{ACTIVE} unimplemented stub → implemented (+INTEGRAL alias). 8 checks × both solvers; regression 148/148	doc	stdaudit
180	ngspice checkpoint/restart under KLU — E-131's guard masked an option-ordering bug: loadstate ran CKTsetup before the task's .option klu reached the circuit (Sparse matrix built; resume dispatch flipped mode; NIiter deref'd NULL SMPkluMatrix). Fix = copy solver selection + E-152 knobs before setup; guards removed. Bonus: cross-solver restore (file is solver-agnostic); all 4 save×load combos $\equiv$ uninterrupted run. Nothing Sparse-only remains. 22/22; regression 148/148	doc	checkpoint

#	What it delivered	Doc	Examples
181	ngspice core-numerics audit + .options ordfix — Gear/BDF coefficients certified EXACTLY (accepted trajectories satisfy the exact variable-step BDF-k formula $\leq 1.3e-13$, orders 1–6 — first direct verification; 3–6 dormant ~30 yrs). ordfix=K fixed-order verification mode (3 measurement traps documented: LTE preference inversion at loose tol, order-1 starter $O(h^2)$ floor, high-order ramp instability). Solver precision $\equiv$ conditioning theory; DC aid paths agree; xmu/lvltim=1 work. 8 checks $\times$ both solvers; regression 149/149	doc	corenum
182	ngspice pyplot autoscale by default — generated scripts rely on matplotlib autoscaling + <code>tight_layout()</code> ; <code>set_xlim/set_ylim</code> emitted only for explicit user <code>xlimit/ylim</code> (user-limit flags captured before plotit frees the statics). 9 checks $\times$ both solvers; regression 149/149	doc	pyplot
183	ngspice pyplot: distinct default names, deck-folder output, linewidth, backend — window-mode no-name plots raced on <code>pyplot.py</code> (background viewer) $\rightarrow$ both showed the 2nd; now <code>pyplot/pyplot-2/...</code> distinct. Artifacts written next to the <code>.cir</code> (ngdirname). <code>set pyplot_linewidth=<w>; set pyplot_backend=<name></code> (matplotlib.use). 13 checks $\times$ both solvers; features/ bundle; regression 149/149	doc	pyplot
184	ngspice sweep progress bar reaches 100% — the E-129 bar froze below 100% at end of run: the throttled (0.25 s) "Reference value" line skips the final sweep point. <code>outp_finish_reference</code> (from <code>fileEnd/plotEnd</code>) reprints the bar full at 100% using the sweep's true endpoint; one-shot, silent for non-swept runs. <code>verify tightened $\geq 90\% \rightarrow == 100\%$</code> ; 22/22; regression 149/149	doc	progressbar
185	openvaf-r autodiff: hypot & atan2 derivative fixes — audit of <code>mir_autodiff</code> (AC/Jacobian) vs analytic <code>f(V)</code> caught 2 builtins right in <code>VALUE</code> , wrong in <code>DERIVATIVE</code> (accidental-correctness, 1st on compiler side). hypot: $(x'+y')/(2 \cdot \text{hypot}) \rightarrow (x \cdot x' + y \cdot y')/\text{hypot}$ (28% off, only right at $x=0.5$). atan2: common factor needed reciprocal $1/(x^2+y^2)$ + 2nd-arg factor needed $-x$ (wrong magnitude+sign). <code>builder.rs</code> . 9 checks + 15-builtin regression; 150/150	doc	vafautodiff

#	What it delivered	Doc	Examples
186	openvaf-r autodiff: real-modulo (%) derivative fix — same AC-conductance referee (E-185) caught a 3rd builtin right in VALUE, wrong in DERIVATIVE. Real modulo $x \% c = x - \text{floor}(x/c) \cdot c$ is a slope-1 sawtooth ($d/dx = 1$), but Frem was grouped with floor/ceil/integer ops → derivative 0 in BOTH the live-derivative gate (<code>lib.rs</code>) AND the chain rule (<code>builder.rs</code>): a modulo term got the right DC value but a ZERO AC/Jacobian. Fixed rule $d/du(x \% c) = x' - \text{floor}(x/c) \cdot c'$; floor/ceil correctly untouched. 14 checks (+variable-divisor probe); 19 unit tests unchanged; 150/150	doc	vafautodiff
187	openvaf-r math-identity simplifier: invalid inverse-function cancellations — the algebraic simplifier (<code>mir_opt/simplify.rs</code>) collapsed <code>asin(sin x)</code>, <code>acos(cos x)</code>, <code>atan(tan x)</code>, <code>acosh(cosh x)</code> and <code>sqrt(x*x)/sqrt(x**2)</code> to the raw inner <code>x</code> — valid only inside each outer function's principal range, so wrong for ordinary finite inputs (e.g. <code>asin(sin 3)=0.1416</code> not 3; <code>sqrt((-3)^2)=3</code> not <code>-3</code>). Corrupts the DC value itself, not just the derivative. Removed those six; kept the whole-real-line-valid identities (<code>tan(atan)</code>, <code>ln(exp)</code>, <code>asinh(sinh)</code>, <code>atanh(tanh)</code>, ...); Inf/NaN-only rewrites left as finite-math. 12 checks; mir unit tests unchanged; 151/151	doc	mathident
188	ngspice Monte Carlo warm-start (<code>montecarlo ... -warm</code>) — the MC loop cold-solved every sample's DC bias point, re-running the full gmin/source-stepping homotopy (~52 Newton iters) although consecutive samples barely move the operating point. <code>-warm</code> preloads the previous converged solution and warm-starts Newton (<code>MODEINITFLOAT</code>, <code>dcop.c</code>), converging in ~4 iters; a bad guess fails the first Newton and falls back to the cold homotopy, so the yield is unchanged (exact at tight <code>reltol</code>; agrees to convergence tolerance at default). ~13× fewer iters/sample; Sparse+KLU; composes with <code>-lhs</code>. 5 checks; 152/152	doc	warmstart

#	What it delivered	Doc	Examples
189	ngspice sweep waveform overlay (sweep ... -overlay) — the E-146 sweep command recorded only each run's <i>last</i> value into its summary transfer curve; overlaying the full per-point waveforms meant setplot-ing each run by hand. -overlay (alias -ov) captures every point's whole output waveform, resamples them onto a common grid (runs land on different adaptive time/freq grids — sw_interp binary-search linear interp), and builds one sweepwave plot with a <out>_<val> vector per point, left current — the HSPICE .step overlay in one step. Front-end only (com_sweep.c), solver-independent; ignored gracefully for a scalar op; summary path byte-identical without the flag. Overlaid RC step responses match $1 - \exp(-t/RC)$ to $<1e-4$. 5 checks; 153/153	doc	sweepwave
190	ngspice nested multi-knob sweep (sweep ... -vs ...) — the E-146 sweep stepped only one knob. -vs <knob> <spec> (alias -family) adds outer knobs whose cartesian product forms a curve <i>family</i> : the inner knob is the x-axis, each outer combination gives one curve per output (<out>_<knob>_<val>), the parametric family of .dc ... SWEEP Spec parser factored into sw_parse_spec (shared by inner+outer). Correct set-ordering: a .param knob re-sources (alterparam+reset) only when its value changes (outer boundaries), with in-place alter/altermod re-applied after each reset. Single knob reduces exactly to E-146 (name <out>, same messages); -overlay composes (waveform name carries every knob value). RC families match $1 - \exp(-T/RC)$ to $<5e-6$, incl. a .param-outer ordering check. 5 checks; 154/154	doc	nestedsweep

#	What it delivered	Doc	Examples
191	ngspice .ac lin 2 / .sp lin 2 off-by-one fix — found auditing the output path (a ragged wrdata test wrote a too-short AC vector). The AC (acan.c) and S-parameter (span.c) linear-sweep step was guarded by <code>numberSteps - 1 > 1</code> (true only for $N \geq 3$), so exactly lin 2 fell into the single-point patch (<code>freqDelta=0</code>, which <code>goto endsweeps</code> after one point) and produced ONE frequency point — the <code>fstop</code> point silently dropped. <code>dec/oct</code> and <code>lin 1/3/4/...</code> were fine; only $N=2$. Fixed to <code>numberSteps > 1</code> (the form <code>noisean.c</code> already used), preserving the genuine lin 1 single-point patch. Both lin 2 endpoints are genuine solved points matching $1/(1+j\omega RC)$ to $<1e-6$; .sp lin 2 gives analytic series-R S-params. Both solvers. The wider output-path audit found the writers clean (Touchstone RI/MA/DB·S/Y/Z round-trip w/ Rbase normalization, wrdata complex/ singlescale/ragged, save filtering, rawfile write/load bit-exact). 5 checks; 155/155	doc	aclin2
192	ngspice auto-checkpoint on interrupt (set <code>autosave=<file></code>) — E-131 <code>savestate/loadstate</code> had to be run by hand; interrupting a long transient lost its progress. <code>set autosave=<file></code> makes an interrupted transient auto-write an E-131 checkpoint. SAFE hook point: NOT the signal handler (<code>ft_sigintr</code> only sets <code>ft_intrpt</code>); at the next accepted timepoint <code>dctran's IFpauseTest→OUTstopnow</code> returns <code>E_PAUSE</code>, which unwinds on the MAIN thread to <code>dosim's err==1</code> "interrupted" branch (<code>runcoms.c</code>), where E-192 writes the checkpoint with ordinary buffered I/O. Reuses the <code>savestate</code> writer (factored into <code>ckt_write_checkpoint</code>), so the <code>autosave</code> file is byte-identical to a manual <code>savestate</code> at the same pause; <code>loadstate</code> resumes it. Opt-in (<code>no var → unchanged</code>), transient-only (<code>MODETRAN+CKTtime>0</code> guard). Interactive-mode only (the handler isn't installed under <code>-b</code>). <code>SIGINT</code>, <code>stop when ...</code>, and a Verilog-A <code>stop</code> task all funnel through the same branch. Verified incl. a real Ctrl-C (<code>SIGINT</code>) driven through a PTY. 5 checks; 156/156	doc	autosave

#	What it delivered	Doc	Examples
193	ngspice .pnoise honors sqrnoise (noise-density units fix) — found auditing the RF/PSS suite. The manual (§.noise) defines <code>onoise_spectrum/inoise_spectrum</code> as $V/\sqrt{\text{Hz}}$ by default, switched to squared V^2/Hz by set <code>sqrnoise</code> ; <code>.noise</code> obeys this but <code>.pnoise</code> always emitted V^2/Hz and IGNORED <code>sqrnoise</code> , so the same-named vectors differed by a <i>square</i> across the two analyses and <code>sqrnoise</code> silently did nothing for <code>pnoise</code> . FIX: <code>pnoise_sweep(dcpss.c)</code> reads <code>sqrnoise</code> via <code>cp_getvar</code> (like <code>noisean.c</code>) and applies <code>sqrt</code> when unset (default $V/\sqrt{\text{Hz}}$); set <code>sqrnoise</code> $\rightarrow V^2/\text{Hz}$. Both emit paths (E-178 cyclo folding + standard adjoint folding) covered. Now <code>.pnoise == .noise</code> by default (matches to 0.00). Scope: <code>.pnoise</code> only (two-tone <code>qpnoise</code> 's single-point diagnostic already prints both forms). Examples updated: <code>rc_pnoise</code> $\rightarrow$ default $V/\sqrt{\text{Hz}}$; <code>rc_cyclo/cyclofold/pnoisefold</code> set <code>sqrnoise</code> (power-density relations), two LTI-limit <code>pnoise == .noise²</code> checks became direct equalities. Audit also verified PAC ± 1 conversion sidebands + HB vs clean transient DFT (both correct) and documented the <code>.pac</code> trailing <code>maxsideband</code> arg. 4 checks; 157/157	doc	pnoiseunits
194	ngspice particle swarm optimization (<code>optimize -method pso</code>) — the built-in optimizer (E-130 Nelder-Mead, E-143 Levenberg-Marquardt) had only LOCAL methods that descend into whichever basin the start sits in. <code>-method pso</code> adds a GLOBAL, population-based, derivative-free search (<code>com_optimize.c</code>): <i>N</i> particles fly through the normalized $[0,1]^{np}$ box, each pulled toward its own (<code>pbest</code>) and the swarm's (<code>gbest</code>) best with the Clerc-Kennedy constriction (<code>chi=0.72984</code> , <code>phi=2.05</code> , ‘	v	<code><=0.5</code>); particle 0 at the init point, the rest random. Reuses <code>opt_</code> <code>evalso</code> it works for scalar-minimizeAND-targetleast-squares (unlike LM). <code>-.swarmsize (auto10+4*np, cap 60)</code> , <code>-seed (self-contained splitmix64 PRNG, reproducible)</code> , <code>shared-maxiter/-tol(gbest-stagnation)</code> . Existing <code>nm/lm</code> dispatch unchanged. Verify: <code>onsin(p)+sin(10p/3)</code> from the trapping <code>p=2.7</code> corner PSO finds the global (<code>p=5.146, -1.900</code>) while NM sticks at a local (<code>-1.20</code>); reproducible seed; robust across seeds; LS mode recovers <code>p</code> to $1e-12$ residual; 2-D multimodal (<code>-3.799</code>). 6 checks; existing <code>optimize</code> (20 checks) unchanged; 158/158

#	What it delivered	Doc	Examples
195	ngspice differential evolution (optimize -method de) — a second GLOBAL optimizer method after PSO (E-194), same population/derivative-free family (com_optimize.c). DE/rand/1/bin: keeps a population in $[0, 1]^{np}$, builds each trial from a scaled DIFFERENCE of random members $v = a + F*(b-c)$ ($F=0.8$) binomially crossed with the target ($CR=0.9$, one dim always taken), greedy selection (keep iff no worse). The difference vector self-scales to the population spread (broad while far apart, fine as it converges), so DE is robust on rugged/discontinuous/poorly-scaled landscapes without step tuning. Reuses opt_eval → scalar -minimize AND -target least-squares. Shares PSO's splitmix64 PRNG (-seed reproducible), -swarmsize (auto $10+4*np$ cap 60, clamped ≥ 5 for the 4 distinct members), -maxiter/-tol. nm/lm/pso dispatch unchanged. Verify: on $\sin(p)+\sin(10p/3)$ from the $p=2.7$ corner DE finds the global ($p=5.14574$ to 5 digits, -1.8996) while NM sticks at a local (-1.20); reproducible + robust seeds; LS mode (resid $4e-13$); 2-D (-3.799); DE and PSO agree on the global while NM does not. 7 checks; existing optimize (20) + pssopt (6) unchanged; 159/159	doc	deopt
196	ngspice simulated annealing (optimize -method sa) — the classic single-walker GLOBAL optimizer, third after PSO/DE, completing the method set (com_optimize.c). A single point proposes a random neighbour $clamp01(x + step*U(-1,1))$ and applies the Metropolis rule: accept if better, else accept with prob $\exp(-Dcost/T)$ -- so it climbs out of local minima while temperature T is high, then settles as T is cooled geometrically ($T = \alpha$, α so T drops ~ 4 decades over -maxiter levels, $L=8+4np$ moves each). ONE eval per step (no population) -> lightest global method, best when each analysis is expensive. Auto-scaled: $T0 = \text{mean}$	Dcost	of 12 random probes; $step = 0.30*\sqrt{T/T0}+0.02$ (wide hot, fine cold). Returns the best point ever visited. Reuses opt_eval -> scalar -minimize AND -target least-squares. Shares the splitmix64 PRNG (-seed reproducible); -maxiter = cooling levels. nm/lm/pso/de dispatch unchanged. A single walker refines more loosely than a population -> reaches the global BASIN not machine precision (a short nm/lm polish sharpens). Verify: on $\sin(p)+\sin(10p/3)$ from the $p=2.7$ corner SA finds the global basin ($f<-1.85$,

#	What it delivered	Doc	Examples
197	ngspice 100-parameter optimization — raised the optimize static limits OPT_MAXP 16→128 (params) and OPT_MAXT 64→128 (least-squares targets), plus the auto-population cap 60→256 (com_optimize.c), so a large problem fits one call. Testbed: 100 independent 1 mA→R_i nodes ($v(n_i)=1e-3 \cdot R_i$) fitted to a 100-point voltage ramp 0.1..9.9 V — a well-posed separable 100-D least-squares. LM solves it to machine precision (sum-sq resid 2.5e-29, rms 5e-16, 619 evals, ~2 s), all 100 params + 100 targets honored. The GLOBAL methods now function at scale too: classic DE CR=0.9 mutates ~every coordinate, so at high-D a ~n-coordinate trial is almost always worse than its parent and rejected → DE freezes at its start. Fix: adaptive crossover CR = min(0.9, 15/n) caps the expected mutated-coordinate count ~15 for large n (unchanged CR=0.9 for n≤16), and DE descends again (1240→882 at 100-D over 35 gens). PSO/DE stagnation counter also gains dimension-scaled patience 8+n/4 (unchanged 8 for n≤3) since high-D runs plateau between improvements. DE at 100-D is a genuine hard global search (slow, intrinsic — not a defect); LM is the tool that nails a well-posed high-D fit. Verify: LM machine-precision fit (both caps proven); DE substantial 100-D descent (self-calibrated ≥15%); -seed bit-reproducible at 100-D; 129-param over-cap reported cleanly. 4 checks; existing optimize (39)+psoopt (6)+deopt (7)+saopt (7) unchanged; 161/161	doc	opt100

#	What it delivered	Doc	Examples
198	ngspice stb stability / loop-gain analysis (<code>com_stb.c</code>, front-end command) — the loop-gain measurement ngspice lacked (Spectre stb / Middlebrook–Tian double injection). User breaks the loop with a probe pair <code>Vstb A B</code> (series 0 V, +node=driver A, –node=load B) + <code>Istb 0 B</code> (shunt 0 A, ground→load), both bias-transparent. <code>stb Vstb Istb <ac-sweep></code> reads the probe's nodes via <code>CKTinst2Node</code>, runs two AC sweeps by alter-ing the probe AC magnitudes: voltage injection $T_v = -v(A)/v(B)$, current injection $T_i = -i(Vstb)/(i(Vstb)+1)$, combined $T = (T_v \cdot T_i - 1)/(T_v + T_i + 2)$ — coded in the singularity-free form $T = -(T_v \cdot a + a + 1)/(T_v \cdot a + T_v + a + 2)$, $a = i(Vstb)$, exact in the clean-break limit $a \rightarrow -1$ ($T_i \rightarrow \infty$, $T \rightarrow T_v$; the $(a+1)$ denominator would otherwise divide by zero). Load-corrected $\Rightarrow$ probe-location-independent (a single injection is not). Stores complex loopgain vs frequency in an <code>stb</code> plot; reports $PM = 180 + \angle T$ at	T	$=1$ and $GM = -$

#	What it delivered	Doc	Examples
199	N-port Touchstone device (examples/ nport_examples/snp2va.py) — instantiate a measured/simulated S- parameter block (.sNp: filter, cable, package, amplifier) as a circuit element. No native n-port existed (.sp "ports" are tagged VSRCs for MEASURING S-params; rdsnp/wrsnp are reader/writer commands). Rather than a native convolution device, snp2va.py converts .sNp → a Verilog-A n-port realized with laplace_nd, so OpenVAF's OSDI laplace machinery (E-31 complex poles) gives AC AND transient with no convolution code. Pipeline: parse Touchstone (any N; S/Y/Z; MA/DB/RI; Hz-GHz; R z0) → S(f) → Y(f)=(1/z0)(I-S)(I+S)⁻¹ → common- pole vector fit (Gustavsen: classic form, freq- normalized — REQUIRED else ill- conditioned — real-valued pole relocation via poly-roots) → emit I(p_i) <+ Σ_j [laplace_nd(V(p_j), num_ij, den) + e_ij·ddt(V(p_j))]. KEY: Y is IMPROPER for shunt-C (Y~sC), which laplace_nd can't represent (→63% AC err); split the e·s term out as explicit ddt and give laplace only the strictly-proper (pole) part → 1e-6. HARDENING: automatic order selection CLIMBS to capture high- order/delay-dominated responses (a delay is uniformly bad at low orders — must not quit early) but stops at the KNEE once the error nears a floor, so noisy data is fit at its noise floor not over-fitted into an unstable model; breaks on instability and returns the best STABLE fit; RHP poles reflected → BIBO- stable; passivity checked+reported. Pure Python stdlib, no numpy — Householder- QR least-squares, complex Gauss-Jordan inverse, Durand-Kerner polynomial roots all reimplemented. Verify: device matches original R-L-C resonator AC 5e-6 (incl. transmission peak) + transient 5e-3 (one model, both analyses); a 5-pole LC ladder (order selection scales past 2 poles); 3-port star matches both coupled outputs 4e-9; noisy fit picks the true order, reports non- passive, stays bounded (stability enforced). Stress-tested to near machine precision on resonators/notches/high-Q/ladders (delay lines degrade gracefully: AC accurate over band, transient edges ring — inherent to rational fitting). 6 checks. 163/163	doc	nport

#	What it delivered	Doc	Examples
200	built-in pre_snp command — folds the E-199 Touchstone→Verilog-A converter into ngspice itself as a C command (no external script, no Python). Inside <code>.control</code>, <code>pre_snp <file.sNp> [module]</code> parses the <code>.sNp</code>, common-pole vector-fits it, writes the <code>.va</code>, and invokes openvaf-r to compile the <code>.osdi</code> — a one-line analog of <code>pre_osdi</code>. <code>snp2va.c</code> is a faithful C port of <code>snp2va.py</code> (double <code>_Complex</code>, typedef'd to avoid ngspice's <code>complex.h</code> macros): Householder-QR least-squares, Gauss-Jordan complex inverse, Durand-Kerner roots, automatic order selection, RHP-pole reflection — checked numerically identical to the Python (resonator 4e−6, ladder 7e−7, 3-port 3e−8). Registered as command <code>snp</code> so <code>pre_snp</code> triggers ngspice's <code>pre_</code> mechanism (runs before circuit parsing); on top of that a two-pass execution of the collected <code>pre_</code> commands forces every <code>pre_snp</code> to run before every other <code>pre_</code> command, so the <code>.osdi</code> it generates already exists when <code>pre_osdi</code> loads it — even with the pre_osdi line written first (ordering guarantee, not a convention). <code>find_openvaf()</code>: <code>openvaf var (spinit/CLI — a .control set runs too late) → OPENVAF env → \$SPICE_LIB_DIR/openvaf-r → PATH</code>; clean error listing all four if none resolve. <code>snp/pre_snp</code> added to <code>inpcom.c</code>'s case-preservation list beside <code>osdi/pre_osdi</code> (keep the path's case). Verify: <code>pre_snp</code> writes+compiles inside ngspice; device matches an R-L-C resonator AC 5e−6 (incl. transmission peak) + transient 5e−3; a 3-port star matches both coupled outputs 4e−9; with <code>pre_osdi</code> before <code>pre_snp</code> the model still loads. 5 checks. 164/164	doc	presnp

#	What it delivered	Doc	Examples
201	pre_snp scalability — a stress test (N-port RLC $\rightarrow$ S-params $\rightarrow$ round-trip) showed the E-200 converter struggled from ~8 ports up (8-port ~190s): its pole solve stacked all N^2 elements into one dense least-squares ($O(N^4)$ mem, $O(N^6)$ compute, ~8TB@N=100) and the emitted model gave each element its own <code>laplace_nd</code> bank ($O(N^2 \cdot N_p)$ state). Four fixes in <code>snp2va.c</code>: (1) Fast Vector Fitting — the block-arrow pole-ID solved by Householder-reducing each element block ($2N_s \times (N_p + 2)$) alone and stacking only the shared-pole coupling into one $((2N_s - N_p - 2) \cdot N_e) \times N_p$ solve $\rightarrow$ mem $O(N^4) \rightarrow O(N^2)$, compute $O(N^6) \rightarrow O(N^2 \cdot N_s \cdot N_p^2)$; (2) reciprocity — symmetric $Y \Rightarrow$ fit upper triangle + mirror (~2$\times$); (3) iteration cap — relocate until poles settle ($<1e-4$, was fixed 12); (4) shared-pole realization — filter each input port ONCE (real pole $\rightarrow V/(s-p)$, conj pair $\rightarrow$ real $(s-\sigma)/D + \omega/D$ bases, residue=real weight), model $O(N \cdot N_p)$ not $O(N^2 \cdot N_p)$ <code>laplace_nd</code>/state. Plus 3 latent bugs the stress test exposed: order-selection double-free when relocated poles lose conjugate-pair adjacency ($\rightarrow$ <code>canon_poles</code> snaps near-real + rebuilds exact adjacent pairs); a 256-entry pv[16*16] stack array hard-capping ports at 16 ($\rightarrow$ heap, cap 512); an OpenVAF crash on integer <code>laplace coeff '{1}'</code> when assigned to a var ($\rightarrow$ force <code>'{1.0}'</code> real literal). Fit N=8 220s $\rightarrow$ 3.6s, N=64 47s, N=100 2.75min @9.6e-4; DC/AC/tran preserved (N=16 1.3%); new limit = OpenVAF compile (~N=20-24 end-to-end). Verify: 8-port coupled ladder converts+compiles fast (65 <code>laplace_nd</code>) + AC match; E-200 order/realization guards still pass. 9 checks. 164/164	doc	presnp

#	What it delivered	Doc	Examples
202	.sp S-parameter scalability ($O(N!) \rightarrow O(N^3)$ inverse) — producing an N-port Touchstone via ngspice .sp was pathologically slow (8-port ~13s, 10-port ~18min, $\sim \times N$ per added port). Profiling: ~99% in CKTspCalcSMatrix, which inverts the $N \times N$ port matrix every frequency (S/Y/Z). cinverse (maths/dense/dense.c) used the adjugate/determinant method — cdet is a recursive cofactor expansion $O(N!)$, adjugate does N^2 of them $\rightarrow O(N \cdot N!)$ (the $\times 7/\times 8/\times 9$-per-port $6! \rightarrow 7! \rightarrow 8!$ blowup). Fix: Gauss-Jordan + partial pivoting, $O(N^3)$, drop-in same result; zero-fills a singular matrix so the old never-NULL contract holds (callers cktspdum.c$\times 3$ + dcps.c). .sp $N=8$ 13s$\rightarrow$0.3s, $N=10$ 18min$\rightarrow$0.02s ($\sim 50000\times$), $N=32$ 0.09s; S-params unchanged (vs closed form $2e-7$). Also speeds .psp/PSS (same inverse). Verify: 12-port .sp extracts in 0.02s + 12$\times$12 S-matrix matches closed form $2e-7$. 2 checks. 165/165	doc	spscale

#	What it delivered	Doc	Examples
213	openvaf-r crash hardening: four compiler panics — fuzzing openvaf-r with malformed Verilog-A found four distinct panics: rather than printing a diagnostic the compiler aborted with "OpenVAF encountered a problem and has crashed!" (exit 101) and directed the user to file a bug report -- for what is often just a typo. Every one is reachable from ordinary source mistakes; the headline case is a module that is merely missing its endmodule, one of the most common Verilog-A editing errors. Bug 1 -- EOF span (syntax/parsing/tree_builder.rs): a parse error landing at end of file built its span as <code>TextRange::at(self.text_pos, <length of the LAST token>).text_pos</code> is already the end of the source, so adding the previous token's length produced a span running past the end of the file -- 6..12 for the 6-byte input "module". Mapping that span back to its source file then failed <code>assert!(range.end() <= self.range.end())</code> in <code>FileSpan::with_subrange</code> (preprocessor/sourcemap.rs), crashing the compiler <i>while trying to print the error</i> ("subrange 6..12 -> 6..12 must fit into the total range 0..6"). Fixed by using an empty range at EOF -- exactly what the adjacent <code>expected_at</code> field already does. Additionally <code>find_ctx_range</code> (syntax/lib.rs) maps a position through half-open <code>[start, end)</code> ranges that by construction never cover the EOF position itself (<code>pos == last_range.end()</code> compares Less against every range); it <code>.expect()</code>ed and panicked, and now clamps to the nearest range. Triggers fixed: missing endmodule; bare module; module header then EOF; unclosed analog begin; unterminated string. Bug 2 -- real literal (lexer/lib.rs): the lexer consumed e/E as an exponent marker without checking that an exponent follows (the result of <code>eat_float_exponent()</code>, which reports whether a digit was seen, was discarded), so 1e became a Float token whose text does not parse as f64 and panicked in <code>ast::StdRealNumber::value()</code>'s <code>src.parse().unwrap()</code>. Fixed so an e only joins the number when a digit (optionally signed) follows; a bare 1e now lexes as 1 plus identifier e and surfaces as an ordinary parse error -- the same approach <code>based_literal_body</code> already takes for a malformed 8'squark. Valid exponents (1.5e3, 2e-3, 1E6) untouched. Triggers: 1e⁶¹, 1e+, 99e, 1.5e, parameter real p=2e. Bug 3 -- preprocessor EOF (preprocessor/parser.	doc	vaocrash

#	What it delivered	Doc	Examples
214	whole-array type coercion: a recurring crash class, fixed at its root -- some bugs are found once; this one was found FOUR times, in four corners of the compiler, and fixed four times, because each fix landed on the symptom rather than the cause. The symptom is always the same: an INTEGER Value reaches a FLOAT MIR op (feq/fmul/fsub/fdiv), and mir_opt::const_eval::eval_binary has no (Int, Float) case for it, so the compiler panics -- "invalid operation fdiv Int(1) Float(Ieee64(1.0))", exit 101, with the usual invitation to file a bug report. If constant propagation happens NOT to fold the expression, the very same defect instead survives to codegen and reaches LLVM as <code>i32 = fadd .., ConstantFP:f64,</code> aborting with "LLVM ERROR: Cannot select" -- one bug, two faces, depending on an optimization that has nothing to do with it. Nothing exotic triggers it: the trigger is writing {1} where a real is expected, which is simply how a unity coefficient or a small selector is naturally written. The four instances: (1) a case over an INTEGER ARRAY -- the array element type was hardcoded real, so an feq landed on i32 (E-33); (2) <code>laplace_/zi_ integer-LITERAL</code> coefficients, <code>laplace_nd(V(in), {1}, ..)</code> (b77266ec); (3) <code>laplace_/zi_ integer</code> ARRAY-VARIABLE coefficients (c55812d6, fixed here) -- a good illustration of how a symptom-level fix propagates the problem, since the fix for (2) cast integer literals but EXPLICITLY EXCLUDED the array-variable path with the recorded reasoning "a whole-array variable reference is already real by its declaration, so the var-ref path needs no cast", which is false for an integer array whose element reads are i32 and hit exactly the mixed-type MIR that fix set out to prevent; (4) an integer case ITEM against a REAL array discriminant (8d0ab057, fixed here) -- the comparison opcode is chosen from the DISCRIMINANT's type (Feq for a real array) but the item's integer elements lower to i32. The root -- a cast that was silently dead: type inference is not at fault; it DOES record the coercion (expect() inserts <code>casts.insert(item, Array{Real})</code>). The problem is where that cast lands and who reads it. It is recorded on the array EXPRESSION, but every whole-array consumer goes through hir_lower's <code>lower_array_elems_impl</code>, and that function DECOMPOSES the array and lowers each	doc	arraycast

#	What it delivered	Doc	Examples
212	Crash hardening: seven user-triggerable crashes — a static-analysis pass (clang analyzer) plus argument fuzzing of every frontend command and of malformed/degenerate netlists surfaced seven reproducible, user-triggerable crashes (SIGSEGV/SIGABRT) in stock ngspice, all in user-input handling (command parsers, the .op output path, netlist recursion) and none in the numerical core (the device models, OSDI loader, Sparse 1.3, KLU, and analysis drivers were audited and found clean). Each is a missing edge-guard. (1) <code>iplot (breakp.c)</code>: <code>iplot -w/-d</code> with the flag as the trailing token consumes its value word (<code>wl=wl->wl_next</code>), guards it with <code>if(wl)</code> (proving <code>wl</code> can be NULL), then the loop-bottom <code>wl=wl->wl_next</code> dereferences NULL; fixed by guarding the advance. (2) <code>altermod (device.c)</code>: <code>com_alter</code> guards <code>if(!wl)</code> but its sibling <code>com_altermod</code> doesn't, so a bare <code>altermod</code> reaches <code>com_alter_common</code> with an empty list <code>→ wl_nthelem(100, NULL)</code> returns NULL <code>→ eq(wlin->wl_word, ...)</code> derefs; fixed by guarding the empty <code>wl_head</code>. (3) <code>measure (vectors.c)</code>: <code>meas ... FIND/WHEN</code> with a missing operand leaves <code>meas->m_vec</code> NULL, which flows into <code>com_measure_when</code> <code>→ vec_get(NULL) → copy(NULL)=NULL</code> <code>→ strchr(NULL, ' ')</code>; fixed by making <code>vec_get</code> NULL-safe (returns NULL, "no such vector"), the common sink for every trigger across <code>tran/dc/ac/sp</code>. (4) empty <code>.op (dotcards.c)</code>: an empty or all-commented-out deck + <code>.op</code> makes an <code>op</code> plot with no data vectors (<code>pl_dvecs</code> NULL), so <code>assert(pl_dvecs!=NULL)</code> aborted (SIGABRT; the autotools build defines no NDEBUG) and would segfault under -DNDEBUG; fixed by guarding <code>if(plot_cur && plot_cur->pl_dvecs)</code> and skipping the empty printout (the solver had handled the empty matrix fine — this was purely output formatting). (5) KLU pole-zero (<code>cktpzset.c</code>): a <code>bsearch</code> miss for the drive pointer was reported with a <code>printf</code> then dereferenced anyway (<code>job->PZdrive_pptr = matched->CSC_Complex</code>); latent (SMPmakeElt keeps the element in the bind table so the miss can't occur today) but the guard was broken — fixed by adding the else/NULL path matching <code>cktsetup.c</code>. (6) <code>.include recursion (inpcom.c)</code>: <code>inp_read</code> tracked <code>call_depth</code> but never limited it, so a file that <code>.includes</code>⁶³ itself — directly, via an <code>A→B→A</code> cycle, or via a <code>.lib</code> section that includes its own file	doc	crashfix

#	What it delivered	Doc	Examples
211	Code-analysis bug fixes (XSPICE DC-op else + LTSPICE table free) — a static-analysis pass over the ngspice tree (clang's analyzer run on every project-modified file, each finding then confirmed by reading the code, false positives discarded) surfaced two real defects. Bug 1 (dcop.c): DCop() chooses between the event-driven XSPICE solver (EVTOP) and the analog solver (CKTOP) with a braceless else whose body was originally converged = CKTOP(...) — so CKTOP ran only when there were no event-driven instances. E-188 (DC warm-start) inserted its preload code between the else and CKTOP, so the else then covered only its first statement(wsize = SMPmatSize(...)) and CKTOP ran unconditionally: for a deck with event-driven code-model instances (A-devices) BOTH EVTOP and CKTOP ran, CKTOP re-solving and overwriting the event-driven DC result, and wsize was read uninitialised at the warm-start guard/snapshot on the EVTOP path (undefined behaviour, masked today only because DC warm-start is off by default). Fixed by hoisting wsize = SMPmatSize(...) above the branch (always initialised; the snapshot needs it on both paths) and adding braces so the else covers the warm-start preload + CKTOP, restoring the original exactly-one-solve semantics; the #ifdef XSPICE braces vanish together when XSPICE is off, so the non-XSPICE build is unchanged. Bug 2 (inpcom.c): inp_compat() converts an LTSPICE E ... table=(...) controlled source using an uninitialised char *ckt_array[100]; the LTSPICE branch sets only ckt_array[1], and a single-pair table (table=(x0,y0), ipairs==1) then tfree(ckt_array[2]) — a free of a garbage stack pointer (that slot is written only on the other, non-LTSPICE, branch). Fixed by zero-initialising the array; ngspice's tfree(NULL) is a safe no-op (txfree guards if (ptr)), so an unwritten slot's tfree does nothing instead of freeing garbage. Verify (examples/codeanalysis, 5 checks x both solvers): a mixed-signal DC op (analog divider + adc_bridge, an event-driven instance) solves v(a)=0.5 without error; a single-pair LTSPICE E ... table=(0.5,3) imports and yields the constant 3 V; a plain analog DC op is unchanged. No functional change to correct analog decks -- the fixes remove undefined behaviour and a redundant analog solve on the XSPICE / LTSPICE-import paths. 5	doc	codeanalysis

#	What it delivered	Doc	Examples
210	PSS polish: .pss dot-card in batch + complex spectrum vectors — two usability fixes to the E-117 shooting-method periodic steady state (pss), in the spirit of E-209 (hb). (1) A top-level .pss card by itself silently no-op'd in batch mode (ngspice -b deck.cir with just .pss ... reported "no simulations run"), unlike .tran/.ac/.hb which auto-run — a deck had to wrap it in .control ... runendc. .pss is now dispatched to the pss command via the deck→control mechanism (the E-146 .sweep / E-162 .hb pattern, in src/frontend/inp.c: a top-level .pss ... line is stripped of its leading . and appended to the post-parse control list, executing as pss ... once the circuit is built); a boundary check keeps it from matching a longer name and it never matches .psp. Decks that already wrap .pss in .control ... run keep working (one PSS run, unchanged). (2) The pss analysis published two plots — pss1 (time-domain periodic waveform) and pss2 (frequency-domain spectrum) — but pss2 stored magnitude only (IF_REAL): the DFT computed each harmonic's phase and threw it away, so vp(out) was unavailable and print out gave a bare magnitude, inconsistent with AC (complex node vectors) and with the hb command's E-209 vectors. pss2 now publishes complex node vectors (IF_COMPLEX): each harmonic is stored as $\text{mag} \cdot e^{j\phi}$ (the DFT already returns ϕ in degrees), built into IFcomplex rows and emitted with OUTpData (as PAC/PXF/PSP do) instead of the real-only CKTdump — so mag(out) reproduces the old magnitude spectrum exactly and vp(out) now recovers the phase; pss1 (time-domain waveform) is unchanged (real). Backward-compat: because pss2 node vectors are now complex, print out/wrdata out emit re,im columns (exactly as AC does), not a bare magnitude — decks parsing the magnitude directly should use mag(out); the bundled rc_pss.cir was migrated from print b to print mag(b). Numerical PSS core, retained periodic operating point, and everything built on it (PAC/Pnoise/PXF/PSP) untouched. Verify (examples/rfpss): a top-level .pss card (no .control) auto-runs in batch and reaches convergence; the frequency-domain spectrum resolves as complex — both vm(out) and vp(out) return, with a driven diode rectifier's harmonics showing the expected non-trivial phase; the	doc	rfpss

#	What it delivered	Doc	Examples
209	hb publishes its spectrum as nutmeg vectors — the E-134 harmonic-balance command (<code>hb <f0> <K></code>) used to only <i>print</i> a spectrum table; <code>HBanalyze()</code> freed the converged two-sided Fourier solution V_r/V_i as soon as the table was written, so the result could not be <code>plot/print/wrdata</code>-ed without scraping <code>stdout</code> (the <code>amp2n2222</code> HB study had to parse the table in Python). Now, after convergence, a fresh hb plot is created and left current, holding a real scale hbfrequency ($0, f_0, 2 \cdot f_0, \dots, K \cdot f_0$, $K+1$ points) and one complex vector per node (named by the node, e.g. <code>out</code>, <code>q1#branch</code>) carrying the single-sided amplitude — so <code>mag(out)/vp(out)</code> reproduce the printed ‘	V	/phase columns exactly, and every accessor works (<code>v(out)</code>, <code>vm</code>, <code>vdb</code>, <code>vp</code>, <code>mag</code>, <code>real</code>, ...). Implementation: <code>HBanalyze()</code> (<code>spicelib/analysis/dcpss.c</code>) gains an optional out-parameter struct <code>hbspectrum*out</code> (declared <code>incktdefs.h</code>) that, on convergence, transfers ownership of $V_r/V_i(+N, K, f_0)$ to the caller instead of freeing them; <code>com_hb()</code> (<code>frontend/com_hb.c</code>) receives it and, in a new <code>hb_publish()</code>, builds the plot with the frontend vector API (<code>plot_alloc/plot_new/plot_setcur</code>, <code>dvec_alloc/vec_new</code>— the same idiom <code>stb/eyeuse</code>), naming node vectors from <code>CKTnames(a#branchname</code> is typed <code>SV_CURRENT</code>, else <code>SV_VOLTAGE</code>), with the single-sided $\times 2$ ($k > 0$) / $\times 1$ (DC) scaling that matches the table, then frees the transferred arrays. Publishing from the frontend (not the analysis layer) means no analysis JOB is needed, so a bare <code>hbin</code>. <code>control(whereCKTcurJob</code> may be unset) publishes cleanly. The printed table is kept (backward-compatible), and the <code>hbdot-card</code> publishes the same vectors. Numerical HB core untouched. Verify (<code>examples/hb</code>): after <code>hb 100meg 5on</code> a diode rectifier, the <code>publishedhbfrequency+</code> node vectors exist and <code>print mag(n)</code> equals the printed table's

#	What it delivered	Doc	Examples
208	pyplot -eye — one-line matplotlib eye diagrams — a convenience bridge between the E-207 eye analysis and the E-94/95/98/99/182/183 pyplot matplotlib back-end. <code>pyplot [name] -eye <expr> -ui <T> [-tstart t0] [-threshold vth] [-window frac]</code> runs the eye analysis and renders it in one command. The raw plot <code>eye_wave vs eye_t</code> connects the folded samples in file order (a scribble); <code>pyplot -eye</code> instead draws them as a 2-D histogram (<code>hist2d</code>, <code>LogNorm</code>, <code>turbo</code>, empty cells masked via <code>cmin=1</code>) so the overlapping traces accumulate into a persistence eye, annotated with the decision threshold (dashed), the eye-centre sampling instant at 1 UI (dotted), a vertical eye-height double-arrow, a horizontal eye-width double-arrow along the threshold, and a title carrying UI / eye height / eye width (% UI) / RMS jitter. Implementation: <code>com_pyplot()</code> detects the <code>-eye</code> marker (a single bare token before it is the output base name; default <code>eye</code>), runs <code>com_eye()</code> on the trailing tokens — which folds the waveform and leaves <code>eye_wave/eye_t</code> + the scalar metrics in a fresh current eye plot — then calls the new <code>ft_pyplot_eye()</code> (<code>plotting/pyplot.c</code>), which reads those back, writes <code><name>.data/<name>.py</code>, and launches Python with the SAME mechanism as <code>ft_pyplot()</code>, honouring every <code>pyplot_*</code> setting (<code>pyplot_terminal=png/svg/pdf</code> → headless Agg hardcopy run synchronously, else an interactive window in the background; <code>pyplot_python</code>, <code>pyplot_backend</code>, <code>pyplot_figsize</code>, <code>pyplot_style</code> — a dark sheet flips the annotation colours to stay legible). No numerical-core changes; the eye math is entirely E-207's, <code>ft_pyplot()</code> and the <code>gnuplot/asciiplot</code> back-ends are untouched. Verify (<code>examples/pyplot</code>): a self-contained data eye — a pseudo-random NRZ bit stream (PWL) through a bandwidth-limiting RC channel ($\tau \sim 0.5$ UI) — is rendered with <code>pyplot eyefig -eye v(rx) -ui 0.5n</code>: the eye analysis runs and reports its metrics, <code>eyefig.png</code> is a valid PNG, the generated script is a <code>hist2d</code> persistence eye referencing the metrics, and the no-name form defaults the base to <code>eye.png</code>. 4 checks (17 total in the pyplot suite). 170/170 (extends the pyplot suite, no new folder)	doc	pyplot

#	What it delivered	Doc	Examples
207	eye diagram / jitter analysis (eye) — the core signal-integrity measurement for serial links / SerDes, which ngspice lacked. <code>eye <expr> -ui <T> [-tstart t0] [-threshold vth] [-window frac]</code> is a front-end post-processing command (<code>com_eye.c</code>, in the <code>fft/linearize/meas</code> family) over a transient result vector. It reads the waveform and its time scale via <code>ft_evaluate</code>; auto-detects the two logic rails from the 20th/80th percentiles (<code>eye_level0/eye_level1</code>, amplitude) and the decision threshold (default midpoint); finds every threshold crossing by linear interpolation; estimates the UI phase as the circular mean of the crossings mod UI; computes each crossing's TIE (time-interval error, relative to the ideal UI grid) → <code>eye_jitter_rms</code> (std of TIE) and <code>eye_jitter_pp</code>; measures <code>eye_height</code> (vertical opening at the sampling instant UI/2 from the transitions: $\min(\text{high samples}) - \max(\text{low samples})$ in a $\pm \text{window} \cdot \text{UI}$ slice) and <code>eye_width</code> ($\text{UI} - \text{jitter_pp}$), plus <code>eye_width_ber12 = UI - 14.069 \cdot \text{jitter_rms}</code> (the eye width at BER $1e-12$ from the $\pm 7.035\sigma$ Gaussian random-jitter tail); and folds the waveform modulo $2 \cdot \text{UI}$ into the <code>eye_wave</code> vs <code>eye_t</code> vectors (created with <code>dvec_alloc/vec_new</code>), whose scatter plot is the eye diagram (<code>plot eye_wave vs eye_t</code>). Solver-independent; works on built-in-device, OSDI, and behavioral signals alike; publishes 11 permanent result vectors. Verify (examples/eye): an ideal 0/1 clock recovers rails [0,1] / amplitude 1 with jitter $\sim 1e-24$ s (machine zero) and a full-UI open eye; a clock with a known injected Gaussian jitter ($\sigma=20$ ps) is recovered to a few percent (≈ 18.6 ps rms, 99 ps pp) with <code>eye_width = UI - jitter_pp</code>; the folded eye vectors are published and <code>wrdata eye_wave</code> writes them without crashing (<code>eye_wave/eye_t</code> live in their own fresh eye plot). 7 checks. 170/170	doc	eye

#	What it delivered	Doc	Examples
206	design centering / yield optimization (optimize -center) — the capstone that fuses two whole subsystems: the optimizer (Nelder-Mead / LM / PSO / DE / SA, E-130–197) and the Monte-Carlo yield suite (agauss/mccorr sampling + montecarlo specs, E-149–151). The OUTER optimizer searches the nominal design point (–dparam/–param/–mparam knobs feeding the agauss centres); at each candidate the INNER loop runs –samples N Monte-Carlo draws — each reset re-samples the deck's process variation around the current centre — evaluates every –spec <metric> [–max hi] [–min lo], and reduces to the worst-case Cpk and the pass-fraction yield. The fusion is a two-line change in opt_eval: the in-place-knob application was factored into opt_apply_inplace (so the MC loop re-applies the centre after each reset re-sources the deck), and a new opt_eval_center runs the inner MC and returns –min(Cpk); the methods (nm/ps/de/sa) are unchanged. Cpk is the objective (yield reported) because raw yield is a granular step function that stalls a simplex, while $Cpk = \min(\text{gap to each active limit}) / (3\sigma)$ is continuous and monotone in yield; a fixed inner seed gives every candidate the same process draws (common random numbers), and with –lhs the stratified sample-mean ≈ 0 so the optimum lands on the analytic centre. Reuses the E-149/151 sampling (mc_lhs_config/mc_sss_off/setseed); lm is rejected for –center (Cpk is not a least-squares residual), default is Nelder-Mead; publishes dcenter_yield/dcenter_cpk vectors. com_optimize.c only. Verify (examples/dcenter): output $\sim N(xc, 0.5)$ with spec [4,6] centres xc from an off-centre 4.0 to ≈ 5.00 ($Cpk \approx 0.667 = 1/(3 \cdot 0.5)$, yield 50%→96%); a montecarlo baseline confirms the improvement; two design params centre independently on their own spec windows (xa→5 on [4,6], xb→10 on [9,11]). 5 checks. 169/169	doc	dcenter

#	What it delivered	Doc	Examples
205	pre_snp low-rank residue factorization — the E-200/201 N-port Touchstone device (snp2va.c) emitted an $O(N^2)$-term, $O(N \cdot N_p)$-laplace_nd-filter shared-pole realization; the vector fit scales to $N=100$ but OpenVAF's compile of that model grows super-linearly and caps the practical port count at ~ 20–24 ($N=32 \sim 80$ s). When ports couple through a few shared modes (multi-port filters, cavities, packages with a shared plane), each pole's $N \times N$ residue matrix has rank r much smaller than N. E-205 exploits that: per channel (conductance d, capacitance e, and each pole section) the emitter builds the $N \times N$ real weight matrix and, via a new in-C one-sided Jacobi SVD (jacobi_svd), picks the cheaper of a dense emit (a term per significant entry, one laplace_nd per input port) or a low-rank emit $W=U \cdot V^T$ (rank kept above $\text{tol_rank}=1e-7$, so a clean singular-value gap compresses fully while a slowly-decaying channel stays dense). The key is input combining: because laplace is linear, $\sum_j V[j][m] \cdot \text{laplace}(V(p_j)) = \text{laplace}(\sum_j V[j][m] \cdot V(p_j))$, so the low-rank form filters the r combined inputs $u_m = \sum_j V[j][m] \cdot V(p_j)$ once (r filters, not N) and distributes them to the outputs via U — collapsing filters $O(N \cdot N_p) \rightarrow O(r \cdot N_p)$ (the piece that dominates compile) and coupling terms $O(N^2) \rightarrow O(N \cdot r)$. A full-rank block keeps the dense form on every channel and is emitted bit-identically (the E-201 PSD projection of e is preserved, so the low-rank transient stays stable). Env var PRE_SNP_DENSE forces the old dense realization (escape hatch / A-B test). Measured on a 3-shared-mode block: $N=16$ 16.9 s $\rightarrow$ 1.5 s (96 $\rightarrow$ 8 filters), $N=24$ 42 s $\rightarrow$ 5.7 s (144 $\rightarrow$ 10), $N=32$ 81 s $\rightarrow$ 11.6 s (192 $\rightarrow$ 8) — super-linear to $\sim$linear, ~ 7–$14\times$, response exact (AC $\sim 4e-7$, transient $\sim 3e-5$ vs a forced-dense build of the same fit). Verify (examples/lowrank): a 12-port/3-mode block emits far fewer filters low-rank than dense (7 vs 73); its AC and transient equal the forced-dense build and the transient stays bounded; a full-rank 8-port ladder gets no compression with bit-identical AC. 5 checks. 168/168	doc	lowrank

#	What it delivered	Doc	Examples
204	auto-triggering DC convergence aids (.option convhelp) — ngspice's CKTop cascade already auto-escalated through gmin stepping and source stepping, but the two strongest aids — the globalized damped-Newton line search (E-111) and pseudo-transient continuation (E-127) — only fired when the user hand-set .option linesearch/.option ptcont, and nothing reported which aid rescued a hard operating point. The gap analysis flagged exactly this ("auto-triggering heuristics... without the user asking, and folding them into the robustness presets"). .option convhelp makes the whole cascade one automatic ladder — Newton(+line search) → gmin step → source step → pseudo-transient → optran — and prints a one-line note naming the aid that produced the point. Under convhelp the op-point Newton (and its fallback homotopies, which call NIiter) run with line search enabled; since E-111's line search reduces to a plain full Newton step whenever the full step already reduces the weighted residual, easy points take the same iterates. The E-127 gate becomes <code>if (ckt->CKTptcont ckt->CKTconvhelp)</code> so pseudo-transient continuation is tried automatically. CKTop was refactored to a single <code>done: exit</code> that saves/restores the caller's line-search flag (never leaks into the following transient) and emits the aid note. Off by default (default path byte-identical, no new output); <code>errpreset=conservative</code> enables it via the E-110 <code>TSKtolGiven</code> given-flag mechanism (so an explicit .option convhelp=0 still overrides regardless of .options order). Files: <code>optdefs.h</code> (<code>OPT_CONVHELP + ERRP_CONVHELP</code>), <code>tskdefs.h</code> (<code>TSKconvhelp</code>), <code>cktdefs.h</code> (<code>CKTconvhelp</code>), <code>cktsopt.c</code> (setter + <code>errpreset</code> table + <code>OPTtbl</code> keyword), <code>ckntask.c/cktdojob.c</code> (copy + default off), <code>cktop.c</code> (the ladder + reporting). Verify (examples/convhelp): option accepted; result-neutral + silent on diode/BJT/two-diode/res-net; the stiff behavioral-exp deck (plain Newton → spurious 70.5 V) reaches the correct 0.837922 V via auto-fired pseudo-transient continuation and changes the outcome vs plain; <code>errpreset=conservative</code> enables the ladder; explicit <code>convhelp=0</code> overrides. Both linear solvers. 35 checks. 167/167	doc	convhelp

#	What it delivered	Doc	Examples
203	.meas ac gain / phase margin (+ batch vdb/vp fix) — audit: ngspice .measure already has TRIG/TARG (VAL/RISE/FALL/CROSS/LAST/TD/AT), FIND...WHEN, AVG/RMS/INTEG/DERIV, MIN/MAX/PP, ERR, param=; the one gap was AC margins. Manual FIND vdb WHEN vp=-180 CANNOT work — vp() wraps to (-180,180] so phase never equals -180°. Added phase_margin(180 + phase at 0 dB gain crossover) and gain_margin(-gain(dB) at -180° phase crossover), computed on the UNWRAPPED phase (undo $\pm 360^\circ$ atan2 jumps), each reporting its crossover freq (log-interp); no finite -180° crossover $\Rightarrow$ reported as infinite GM. measure_margin/get_measure2 in com_measure2.c. NOTE: phase honors runtime cx_degrees (default radians), so margins compute degrees explicitly. Also fixed gettok_iv (misc/string.c) truncating vdb(out) $\rightarrow$ vd (broke only after first suffix char on n_paren==0) $\rightarrow$ batch .meas ac vdb(out) "can't parse 'vd'" / "no data saved"; now stops once parens close; strip_ac_vec saves base v(out). Verify: closed-form loops — 2-pole PM 84.89° (=analytic) + GM infinite; 3-pole PM -42.75° + GM -15.92 dB (exact); batch vdb dot-card. 5 checks. 166/166	doc	acmargin
162	ngspice .hb dot-card for harmonic balance — gives the E-134 hb command dot-card parity with the PSS family. A top-level .hb <f0> <K> [points] [maxiter] runs HB straight from the deck (no .control), dispatching to com_hb via the E-146 deck $\rightarrow$ control bridge (inp.c strips the . and appends hb ... to the control list; boundary-checked). Was unimplemented dot command (only a disabled #ifdef WITH_HB stub existed). Verified: batch-mode spectrum bit-for-bit identical to the hb command, threads args, coexists with .control, both solvers	doc	hb

#	What it delivered	Doc	Examples
163	ngspice .qpss / .hbosc / .phasenoise dot-cards — completes the HB family's netlist dot-card parity (E-162). .qpss <expr> <f1> <f2> ... (two-tone QPSS, E-133/136), .hbosc <oscnode> <K> ... (autonomous-oscillator HB, E-140), .phasenoise <fstart> <fstop> ... (E-140) run from the deck; .hbosc+.phasenoise compose in order (bridge preserves order) so an oscillator phase-noise run needs no .control. Same one-branch inp.c bridge as .hb/.sweep, per-card boundary checks (.hbosc not caught by .hb). Verified: .qpss spectrum bit-for-bit == qpss command; .hbosc+.phasenoise reproduce f0 + phase-noise exactly; both solvers	doc	qpss

Part II — The Enhancements, One by One

The complete engineering record: for every enhancement, what was broken or missing, how it was fixed, and how the fix was verified. These are the detailed documents the handbook's matrix rows and the top-level README summaries point at.

Enhancement-1 — **absdelay** support for OSDI/Verilog-A in OpenVAF + ngspice

This document describes, step by step, every source change made to turn the original OpenVAF and ngspice-46 sources into version 2. It was produced by diffing `original/` against `version2/`.

The single goal of version 2 is to make the Verilog-A **absdelay()** operator work end-to-end through the OSDI flow — compile a model that uses **absdelay** with `openvaf-r`, and simulate it in ngspice for DC, AC, and transient analysis.

Nothing outside the OSDI subsystem of ngspice is touched, and on the OpenVAF side only the **absdelay** lowering + OSDI code-gen path is changed.

1. The idea (how **absdelay** is modeled)

`absdelay(V(in), td)` returns the input delayed by `td` seconds. A delay is not expressible as a local algebraic/reactive stamp, so version 2 splits each **absdelay** call into two synthetic implicit equations that the compiler emits and the simulator closes:

```
eq_y (AbsDelayInput) : V(y_synth) = y_expr          ← stamped by the model
eq_z (AbsDelayOutput): V(z)          = delayed(V(y_synth), td) ← stamped by ngspice
```

- The model (OpenVAF output) owns `eq_y`: it forces a synthetic node `y_synth` to equal the input expression, and it stores the current `td`.
- The simulator (ngspice) owns `eq_z`: it keeps a waveform history of `V(y_synth)` and stamps the row that ties output node `z` to the delayed value (interpolated from history in transient, $e^{-j\omega t}$ in AC, pass-through in DC).

To connect the two, OpenVAF exports a small descriptor per delay slot and ngspice reads it at `.osdi` load time:

```
OsdiAbsDelayInfo { uint32 y_node; uint32 z_node; uint32 td_offset; }
```

plus two module-level arrays: `OSDI_ABSDELAY_COUNTS[i]` (number of delay slots in descriptor `i`) and `OSDI_ABSDELAY_INFOS[]` (the flattened slot descriptors). This contract is the heart of version 2.

2. OpenVAF compiler changes

8 source files. The job: lower **absdelay** to the two-equation form, carry the `td` value into instance data, and export the slot descriptors in the `.osdi`.

2.1 `openvaf/hir_lower/src/lib.rs` — new IR vocabulary

- New `PlaceKind::AbsDelayTime(u32)` — a real-typed place that holds the current `td` for slot `i` (so it can be written into instance data and read back by the simulator).
- New `ImplicitEquationKind::AbsDelayInput(u32)` and `AbsDelayOutput(u32)` — the `eq_y` / `eq_z` synthetic-node equations.
- New field on `HirInterpreter`: `absdelay_equations: Vec<(ImplicitEquation, ImplicitEquation)>` — records the (`eq_y`, `eq_z`) pair for every slot, in order.

2.2 `openvaf/hir_lower/src/ctx.rs`

- `PlaceKind::AbsDelayTime(_)` initializes to `F_ZERO` (default value before the model body assigns it).

2.3 `openvaf/hir_lower/src/expr.rs` — the actual lowering (core change) The original code for `absdelay` was a disabled `/* TODO */` block that tried to approximate the delay with a reactive (`ddt`-style, `res/3`) term and then fell through to an “unsupported” arm. Version 2 replaces it with the two-node form:

1. lower `y_expr = args[0]` and `td = args[1]` (clamp to `tdmax` if a third arg is present);
2. allocate `eq_y = AbsDelayInput(idx)` and `eq_z = AbsDelayOutput(idx)` and push (`eq_y`, `eq_z`) onto `absdelay_equations`;
3. emit the resistive residual `y_expr - V(y_synth) = 0` on `eq_y`;
4. store `td` into the `AbsDelayTime(idx)` place;
5. return `V(z)` as the value of the `absdelay` expression — `eq_z`’s row is left for the simulator to stamp.

2.4 `openvaf/sim_back/src/topology.rs` — keep equation indices aligned The DAE builder used to filter_map away equations with dead/collapsed unknowns. That would renumber equations and break the slot→node mapping. Version 2 changes it to map so every implicit equation keeps a slot in the unknowns vector; dead/collapsed ones get a `Contribution::default()` (zero-contribution) placeholder. The `absdelay` output rows are exactly these “empty” rows — `ngspice` fills them in. This guarantees `ImplicitEquation` indices stay valid as node indices.

2.5 `openvaf/sim_back/src/context.rs`

- Treat `PlaceKind::AbsDelayTime(_)` as a real output place (so it participates in output/eval-slot handling like other stored quantities).

2.6 `openvaf/osdi/src/inst_data.rs` — carry `td` into instance data

- New `delay_times: Vec<EvalOutputSlot>` on the instance-data layout, one slot per `absdelay`, populated from the `AbsDelayTime(i)` outputs.
- New `store_delay_times(...)` — writes each `td` slot during eval.
- New `delay_time_offset(i, target_data)` — returns the byte offset of slot `i`’s `td` inside the instance struct (this becomes `td_offset` in the descriptor).

2.7 `openvaf/osdi/src/eval.rs`

- Call `inst_data.store_delay_times(instance, &builder)` inside the generated `eval` so `td` is live in instance memory whenever the simulator reads it.

2.8 `openvaf/osdi/src/lib.rs` — export the descriptors

- Define the `OsdiAbsDelayInfo { y_node, z_node, td_offset }` LLVM struct type.
- For each module, for each (`eq_y`, `eq_z`) slot, resolve `y_node/z_node` from the DAE unknowns (`SimUnknownKind::Implicit(eq) → node index`) and `td_offset` from `delay_time_offset(i)`, and build a const struct.
- If any module uses `absdelay`, export the global arrays **`OSDI_ABSDELAY_COUNTS`** (per-descriptor slot count) and **`OSDI_ABSDELAY_INFOS`** (flattened slot descriptors).

3. `ngspice` simulator changes

9 OSDI files (8 edited + 1 new). The job: read the descriptors, allocate the delay-row matrix entries and waveform history, and stamp `eq_z` for DC, AC, and transient. The core analysis engine (`dctran.c`, `acan.c`, `optran.c`, ...) is unchanged — all support lives in the OSDI device layer.

3.1 `src/include/ngspice/osdiitf.h` — registry entry

- Added `uint32_t num_absdelays` and `const void *absdelay_infos` to `OsdiRegistryEntry`, filled at load time from the exported symbols.

3.2 `src/osdi/osdidefs.h` — data structures

- New `OsdiAbsDelayInfo { y_node, z_node, td_offset }` (mirrors the OpenVAF export).
- Extended `OsdiExtraInstData` with:
 - `delay_hist[][] + delay_hist_cap` — per-slot waveform history of $V(y_{\text{synth}})$, indexed by accepted timepoint;
 - `delay_jac_y[], delay_jac_z[]` — active matrix-entry pointers for the (z, y_{synth}) and (z, z) stamps;
 - `delay_jac_{y,z}_csc[], delay_jac_{y,z}_cx[]` — KLU-only saved real and complex pointers (used by the AC fix in §3.8).

3.3 `src/osdi/osdiregistry.c` — read the descriptors

- At `.osdi` load, look up `OSDI_ABSDELAY_COUNTS` and `OSDI_ABSDELAY_INFOS`; for each descriptor compute its slice into the flattened infos array (12 bytes/slot) and store `num_absdelays + absdelay_infos` on the entry.

3.4 `src/osdi/osdiext.h` and `src/osdi/osdiinit.c` — register accept hook

- Declare `OSDIaccept` and wire it as `OSDIinfo->DEVaccept = OSDIaccept`; so ngspice calls it after each accepted transient timepoint.

3.5 `src/osdi/osdisetup.c` — allocate entries + KLU binding

- In `OSDIsetup`: when `num_absdelays > 0`, allocate the history/pointer arrays and create the two matrix entries per slot with `SMPmakeElt(matrix, z_row, y_col)` and `SMPmakeElt(matrix, z_row, z_row)`.
- In `OSDIbindCSC` (KLU): rebind each delay pointer from the temporary COO buffer to the live KLU matrix, saving both the real (CSC) and complex (CSC_Complex) addresses (`delay_jac*_csc / delay_jac*_cx`).
- In `OSDIupdateCSC` (KLU real↔complex switch): repoint the active `delay_jac_*` pointers to the complex array for AC and back to real for DC/tran — see §3.8.

3.6 `src/osdi/osdiload.c` — DC and transient stamping (largest change) New static helpers + hooks in `OSDIload`:

- `absdelay_ensure_timepoints / absdelay_grow_hist` — lazily allocate and grow `CKTtimePoints` and the per-slot history (ngspice only allocates `CKTtimePoints` for LTRA otherwise).
- `absdelay_lookup` — interpolate the delayed $V(y_{\text{synth}})$ at $t - td$ from the accepted history (binary search), returning the value and a Jacobian α .
- `absdelay_stamp_dc` — steady state: a delay is an ideal wire, so stamp `jac[z,y]+=1, jac[z,z]+=-1` ($V(z)=V(y_{\text{synth}})$), keeping the matrix non-singular.
- `absdelay_stamp_tran` — transient: initialize history on the first step, then stamp the interpolated delayed value into `eq_z`. Sub-femtosecond delays (**`td < 1e-15`**) are treated as DC pass-through so the epsilon delays that appear in photonic `S11/S22` terms don't collapse the timestep.
- `OSDIload` calls `absdelay_stamp_tran` (transient) or `absdelay_stamp_dc` (DC) after the normal device load.

3.7 `src/osdi/osdiaccept.c` — new file, commit history

- `OSDIaccept` runs after each accepted transient timepoint and writes the converged $V(y_{\text{synth}})$ into `delay_hist[k][CKTtimeIndex]`, growing the history if `CKTtimePoints` grew. This is the data `absdelay_lookup` reads next step.

3.8 `src/osdi/osdiacld.c` — AC stamping

- For each slot, stamp the complex delay row: $(z, y_{\text{synth}}) += e^{-j\omega\tau} = \cos(\omega\tau) - j \cdot \sin(\omega\tau)$ and $(z, z) += -1$. Magnitude is 1 (lossless delay); only the phase rotates with frequency, which is the physical group delay.
-

4. Bug fixes made while bringing the three analyses up

Two defects were found and fixed during development; both are folded into the files above.

4.1 Transient “timestep too small” from epsilon delays Some models emit `absdelay` terms with $\sim 1e-20$ s delays (S11/S22 phase derivatives). Feeding those into the integrator drove the timestep toward zero. Fix: in `absdelay_stamp_tran`, any `td < 1e-15` is stamped as a DC pass-through ($V(z)=V(y_{\text{synth}})$) instead of a true delay (`osdiload.c` §3.6).

4.2 AC singular matrix under KLU KLU keeps two arrays per matrix entry — real (CSC) for DC/tran and complex (CSC_Complex) for AC — and swaps the active pointer on every DC↔AC transition. The `absdelay` delay-row pointers were only ever bound to the real array, so during the AC complex solve the delay rows were stamped into the unused real array → empty complex rows → singular matrix. (The legacy SPARSE solver was unaffected because it stores `[real, imag]` adjacently in one element.)

Fix (`osdidefs.h` + `osdisetup.c`): save both CSC and CSC_Complex for each delay entry in `OSDIbindCSC`, and in `OSDIupdateCSC` switch the active `delay_jac_*` pointer to the complex array for AC and back to real for DC/tran — mirroring exactly how ngspice already handles the regular device Jacobian. After the fix, KLU and SPARSE AC results are bit-identical.

5. Summary of changed files

OpenVAF (**version2/OpenVAF-master**)

file	lines	what
<code>openvaf/hir_lower/src/lib.rs</code>	12	IR kinds: <code>AbsDelayTime</code> , <code>AbsDelayInput/Output</code> , <code>absdelay_equations</code>
<code>openvaf/hir_lower/src/ctx.rs</code>	1	default <code>td</code> place to 0
<code>openvaf/hir_lower/src/expr.rs</code>	58	lower absdelay to the two-node DAE form
<code>openvaf/sim_back/src/topology.rs</code>	20	keep equation indices aligned (placeholder rows)
<code>openvaf/sim_back/src/context.rs</code>	5	<code>AbsDelayTime</code> is an output place
<code>openvaf/osdi/src/inst_data.rs</code>	31	store <code>td</code> in instance data; offset accessor

file	lines	what
openvaf/osdi/src/eval.rs	1	call store_delay_times in eval
openvaf/osdi/src/lib.rs	62	export OSDI_ABSDELAY_COUNTS / OSDI_ABSDELAY_INFOS

ngspice (**version2/ngspice-46/src**)

file	lines	what
include/ngspice/osdiitf.h	4	num_absdelays, absdelay_infos on registry entry
osdi/osdidefs.h	29	OsdAbsDelayInfo, history + matrix-pointer fields
osdi/osdiregistry.c	29	read the exported symbols at load time
osdi/osdiext.h	1	declare OSDIaccept
osdi/osdiinit.c	1	register DEVaccept = OSDIaccept
osdi/osdisetup.c	90	allocate entries/history; KLU bind + real/complex switch
osdi/osdiload.c	226	DC pass-through + transient history stamping
osdi/osdiaccept.c	86	new — commit converged $V(y_{\text{synth}})$ to history
osdi/osdiacld.c	46	AC $e^{-j\omega t}$ stamping

The core ngspice analysis files (acan.c, dctran.c, optran.c, span.c, noisean.c) are unchanged.

6. Build & verify

```
# compiler
cd ./OpenVAF-master && ./configure && ./build.sh --release
# -> target/release/openvaf-r (also copied to ./bin/openvaf-r)

# simulator
cd ./ngspice-46/build && make -C src -j4
# -> src/ngspice (also copied to ./bin/ngspice)

# end-to-end checks
cd ./absdelay_examples
bash examples/run_examples.sh # DC/AC/tran, KLU vs SPARSE agree
bash benchmark/run_benchmark.sh # KLU vs SPARSE timing sweep
```

Enhancement-2 — Indirect Branch Assignment support in OpenVAF

This document describes every source-code change made to OpenVAF in the `version3/` directory, on top of `version2/` (Enhancement-1, `absdelay`), to implement Verilog-AMS indirect branch assignment. It was produced by diffing `version2/OpenVAF-master` against `version3/OpenVAF-master`.

The goal of Enhancement-2 is to make the Verilog-AMS construct

```
<lhs_access>(<branch>) : <rhs_access>(<branch>) == <expr> ;
```

compile and simulate correctly end-to-end (DC, AC, transient) through the OSDI flow. The canonical motivating example, taken directly from the LRM (Accellera Std VAMS-2023, p. 114), is an ideal op-amp:

```
module opamp(out, pin, nin);
    inout out, pin, nin;
    electrical out, pin, nin;
    analog
        V(out):V(pin,nin) == 0;
endmodule
```

Before this change, OpenVAF's parser rejected the `:` token here:

```
error: unexpected token ':'; expected ';'
--> opamp.va:9:15
  |
9 |         V(out):V(pin,nin) == 0;
  |                   ^
```

Unlike Enhancement-1, this change is entirely confined to OpenVAF (`parser` → `AST` → `HIR` → `HIR lowering`). No ngspice/OSDI changes were needed — see §5.

1. The idea (how indirect branch assignment is modeled)

Indirect branch assignment introduces one new free unknown u per statement, plus one new implicit equation that solves for it:

Quantity	Meaning
u	a fresh DAE unknown, contributed into the LHS branch exactly like a normal <code><+ u></code> contribution
residual	$\text{constraint_lhs} - \text{constraint_rhs} = 0$, where $\text{constraint_lhs} == \text{constraint_rhs}$ is the RHS of the statement

For `V(out):V(pin,nin) == 0;`:

- u is contributed into the `out` branch as a voltage contribution (`V(out) <+ u`), because the LHS access function is `V`. This reuses OpenVAF's existing voltage-source stamping path, which already augments the DAE with the auxiliary current unknown a voltage source needs (the same mechanism behind plain `V(br) <+ expr;`) — nothing new had to be built for that half.
- The residual equation enforces $V(\text{pin}, \text{nin}) - 0 = 0$, i.e. it solves u (the now-ideal output voltage) for whatever value makes the input pins equal — exactly the nullor behavior of an ideal op-amp.

This is conceptually the same “new unknown + new implicit equation” DAE pattern Enhancement-1 used for `absdelay` (see `version2/.../hir_lower/src/expr.rs`, `BuiltIn::absdelay`), except:

- Enhancement-1 needed two implicit equations per call (`AbsDelayInput`, `AbsDelayOutput`) plus a runtime/OSDI contract because the second equation was stamped by ngspice (it needs a time-domain delay history).
- Enhancement-2 needs only one implicit equation per statement, and it is stamped entirely by OpenVAF at compile time — both halves (the branch contribution and the residual) are ordinary algebraic MIR, so no new OSDI export or ngspice-side code was required.

2. OpenVAF changes

9 source files touched, all under `openvaf/{parser,syntax,hir_def,hir_ty,hir,hir_lower}`.

2.1 **`openvaf/parser/src/grammar/stmts.rs`** — `accept : assign_or_expr()` previously only accepted `<+` (contribute) or `=` (assign) after the LHS expression. Added `T![:]` to the accepted `TokenSet`; the `:` token already existed in the lexer (used for ternary `?:`, case labels, array ranges), so no lexer change was needed.

```
if p.eat_ts(TokenSet::new(&[T![<+], T![=], T![:]])) {
```

2.2 **`openvaf/syntax/veriloga.ungram + src/ast/{node_ext,expr_ext}.rs`** — new **`AssignOp`** variant

- `veriloga.ungram`: Assign rule's operator alternatives extended to (`'<+' | '=' | ':'`).
- `node_ext.rs`: new `AssignOp::IndirectBranch` variant; `Assign::op()` recognizes `T![:]`.
- `expr_ext.rs`: the parallel (currently unused outside this enum) `AssignmentOp` got a matching `IndirectBranch` arm in `op_details()`, for consistency.

The generated AST node `Assign { syntax }` carries no per-operator fields (the operator is read directly off the raw syntax node), so no AST codegen regeneration was required — only the two hand-written extension files above.

2.3 **`openvaf/hir_def/src/body/lower.rs, src/expr.rs`** — no change `hir_def::Stmt::Assignment { dst, val, assignment_kind }` already carries `assignment_kind: ast::AssignOp` opaquely, so it picks up the new variant for free. For the indirect form, `dst` is the LHS access expression (`V(out)`) and `val` is the entire RHS expression after `:` — i.e. the equality expression `V(pin,nin) == 0`, parsed as one ordinary `== BinaryOp` (the expression grammar already understood `==`; nothing new needed there).

2.4 **`openvaf/hir_ty/src/inference.rs`** — validate & decompose the constraint This is where most of the new logic lives.

- New `InferenceResult::indirect_branch_constraints: AHashMap<StmtId, (ExprId, ExprId)>` — for an indirect-branch `StmtId`, the decomposed (`lhs`, `rhs`) operand `ExprIds` of its `==` constraint.
- `Ctx::infe_stmt`'s `Stmt::Assignment` arm now special-cases `assignment_kind == AssignOp::IndirectBranch`: it still calls `infe_assignment_dst` (unchanged `dst`-resolution logic — `V()/I()` branch access detection is identical to the `Contribute` path), but routes `val` through a new method instead of the generic `infe_assignment`:

```
fn infe_indirect_branch_constraint(&mut self, stmt: StmtId, val: ExprId,
    ↪ dst_ty: Option<Type>)
```

This requires `val` to be a top-level `Expr::BinaryOp { op: Some(BinaryOp::EqualityTest), lhs, rhs }`. If it isn't, a new diagnostic `InferenceDiagnostic::IndirectAssignRequiresEquality`

{ e } is emitted (e.g. `V(out):V(pin,nin) <+ 0;` or `V(out):1;` are rejected with a clear error instead of a generic type mismatch or a panic). If it is, both operands are type-checked against `dst_ty` (`Type::Real`, same rule as a normal contribution) and recorded in `indirect_branch_constraints`.

Crucially, the constraint operands are inferred individually — `val` itself (the whole `==` expression, whose value type is boolean-like) is never type-checked against the branch-access `Real` type, which is what a naive reuse of `inference_assignment` would have done and incorrectly rejected.

- The `(dst, assignment_kind)` operator-compatibility match gained an arm so `(AssignDst::Var | AssignDst::FunVar, AssignOp::IndirectBranch)` is rejected with the existing `InvalidAssignDst` diagnostic (you can't use `:` on a plain variable), grouped with the existing `Contribute` rejection.

2.5 `openvaf/hir_ty/src/diagnostics.rs` — error reporting

- `InvalidAssignDst` report: added `AssignOp::IndirectBranch` arms to both the primary-message match and the `maybe_different_operand` hint match (e.g. "found a branch access, perhaps you meant an indirect branch assignment (:)").
- New report arm for `IndirectAssignRequiresEquality`: *"invalid indirect branch assignment ... help: indirect branch assignment requires a constraint of the form <access> == <expr>"*.

2.6 `openvaf/hir/src/body.rs` — new `Stmt::IndirectContribute`

- New HIR statement variant:

```
Stmt::IndirectContribute {
    kind: ContributeKind,
    branch: BranchWrite,
    constraint_lhs: ExprId,
    constraint_rhs: ExprId,
}
```

- `BodyRef::get_stmt`: when `inference.indirect_branch_constraints` has an entry for the statement, emits `Stmt::IndirectContribute` (mapping `AssignDst::Flow/Potential` to `ContributeKind::Flow/Potential`, exactly like the existing `Contribute` path); otherwise falls through to the unchanged `Stmt::Assignment/Stmt::Contribute` logic.

2.7 `openvaf/hir_lower/src/lib.rs` — new implicit equation kind

- New `ImplicitEquationKind::IndirectBranch(u32)` — one per indirect branch assignment slot; its residual row enforces `constraint_lhs == constraint_rhs`.
- New `HirInterner::indirect_branch_equations`: `Vec<ImplicitEquation>` — slot index $\rightarrow$ implicit equation, parallel to `Enhancement-1's absdelay_equations` but a 1-tuple per slot instead of a pair (only one equation is needed here).

2.8 `openvaf/hir_lower/src/stmt.rs` — lowering

- Refactored `contribute()` (used for plain `V/I(br) <+ expr;`) into a thin wrapper around a new value-based primitive:

```
fn contribute_value(&mut self, voltage_src: bool, write: BranchWrite, rhs:
    ↪ Value, is_zero: bool)
```

`contribute_value` contains the exact same branch-stamping logic `contribute()` had (unnamed-branch resolution, `IsVoltageSrc` place, collapse-hint call, accumulating the `Contribute` place) but takes an already-lowered MIR `Value` instead of an `ExprId`. `contribute`

) now just lowers its rhs expression and calls `contribute_value` — behavior for ordinary `<+` statements is unchanged (verified by the unmodified `absdelay/regression` test suite, see §7).

- New `indirect_contribute()`:

```
fn indirect_contribute(&mut self, voltage_src: bool, branch: BranchWrite,
                      constraint_lhs: ExprId, constraint_rhs: ExprId)
```

1. Allocates one implicit equation: `(eq, u) = self.ctx.implicit_equation(ImplicitEquationKind::IndirectBranch(idx))`.
 2. Contributes `u` into `branch` via `contribute_value(voltage_src, branch, u, false)` — i.e. literally `V(branch) <+ u` or `I(branch) <+ u`, reusing the exact same backend stamping path (and its automatic auxiliary-unknown augmentation for voltage contributions) as a normal contribution.
 3. Lowers `constraint_lhs` and `constraint_rhs`, builds `residual = lhs - rhs`, and calls `self.ctx.def_resist_residual(residual, eq)` — the same `ImplicitEquation/residual` primitives Enhancement-1 used for `eq_y` in `absdelay`.
- `lower_stmt` dispatches the new `Stmt::IndirectContribute` to `indirect_contribute`, alongside the existing `Stmt::Contribute` arm.

3. Diff summary (version2 → version3)

File	Kind of change
openvaf/parser/src/grammar/stmts.rs	accept : as a 3rd assignment operator
openvaf/syntax/veriloga.ungram	grammar: Assign op alternatives
openvaf/syntax/src/ast/node_ext.rs	new <code>AssignOp::IndirectBranch</code> + <code>Assign::op()</code>
openvaf/syntax/src/ast/expr_ext.rs	matching <code>AsssignmentOp::IndirectBranch</code>
openvaf/hir_ty/src/inference.rs	new constraint map, new inference method, new diagnostic, operator-compatible match
openvaf/hir_ty/src/diagnostics.rs	error report text for the two new/extended diagnostics
openvaf/hir/src/body.rs	new <code>Stmt::IndirectContribute</code> + dispatch in <code>get_stmt</code>
openvaf/hir_lower/src/lib.rs	new <code>ImplicitEquationKind::IndirectBranch</code> , new internal field
openvaf/hir_lower/src/stmt.rs	<code>contribute()</code> refactor + new <code>contribute_value/indirect_contribute</code>

No changes to `openvaf/hir_def`, `openvaf/mir*`, `openvaf/sim_back`, or `openvaf/osdi` — see next section for why.

4. New diagnostics

Diagnostic	When
InvalidAssignDst (extended)	: used on a non-branch-access LHS (e.g. a plain variable)
IndirectAssignRequiresEquation (new)	if <code>lhs</code> is not a top-level <code>==</code> expression (e.g. <code>V(out):1;</code> or <code>V(out):V(pin,nin) <+ 0;</code>)

Both produce a normal compiler error with a source span and a help note, not a panic.

5. Why no backend/OSDI changes were needed

Both halves of an indirect branch assignment lower to mechanisms the backend (`sim_back`) and the OSDI exporter already handle generically:

1. The branch contribution (`contribute_value`) produces exactly the same `PlaceKind::Contribute / PlaceKind::IsVoltageSrc` outputs as a literal `V(br) <+ expr;` would — `sim_back::topology` doesn't know or care whether the contributed value came from a lowered expression or a fresh implicit unknown.
2. The constraint residual uses the exact same `ImplicitEquation / PlaceKind::ImplicitResidual` mechanism as `absdelay`'s `eq_y` — already wired all the way through `sim_back/src/topology/linalize.rs` and the OSDI Jacobian/residual export.

This was confirmed empirically: the generated MIR for the op-amp example (`openvaf/test_data/mir/indirect_branch.mir`) shows the implicit unknown flowing straight into the `Contribute` place and the residual into the `ImplicitResidual` place with no special-casing required, and the compiled `.osdi` simulates correctly (§7) without touching a single line in `sim_back` or `osdi`.

6. Build

```
cd version3/OpenVAF-master
cargo clean
LLVM_SYS_181_PREFIX=/opt/homebrew/opt/llvm@18 \
  cargo build --release --bin openvaf-r --features openvaf-driver/llvm18
cp target/release/openvaf-r ../bin/macos/apple-silicon/openvaf-r

(--features openvaf-driver/llvm18 is required — mir_llvm/openvaf-driver have no default LLVM version feature, see their Cargo.toml.)
```

7. Testing & verification

7.1 Compiler unit / snapshot tests

- `cargo test -p syntax -p hir_def -p hir_ty -p hir -p hir_lower` — all existing fast tests pass unchanged (no regression in plain `<+/=` contributions, `absdelay`, case statements, etc).
- New MIR snapshot test `openvaf/test_data/mir/indirect_branch.va / .mir`, exercising the op-amp example end-to-end through `hir_lower::MirBuilder`. The generated MIR confirms the unknown is contributed to the `out` branch and the residual is `V(pin,nin) - 0`.

7.2 End-to-end compile `version3/opamp.va` (the LRM example, using ``include "disciplines.vams"` — resolved by OpenVAF's built-in `stdlib`, no `-I` needed) compiles to a working `.osdi` with zero errors/warnings.

7.3 ngspice simulation — new feature `version3/examples/indirect_assignment_examples/` — a unity-gain buffer built from the op-amp (nin tied to out), simulated with `version3/bin/.../ngspice`:

- DC sweep $-2V \dots 2V$: $V(out) == V(pin)$ exactly.
- AC 1kHz...1GHz: flat 0 dB gain, 0° phase (ideal, infinite bandwidth).
- Transient 1MHz sine: $V(out)$ tracks $V(pin)$ with no lag.

PNG plots of all three are in that folder (`dc.png`, `ac.png`, `tran.png`).

7.4 Regression — Enhancement-1 (**absdelay**) `version3/examples/absdelay_examples/` recompiled (`absdelay.va` $\rightarrow$ `.osdi` with the new `openvaf-r`) and re-run (DC/AC/transient, KLU vs SPARSE): DC and transient bit-identical between solvers, AC differs only by $\sim 2e-15$ (floating-point roundoff) — matching the documented Enhancement-1 baseline.

This confirms indirect branch assignment support introduced no regressions in the existing `absdelay`-based models or the broader OSDI/ngspice pipeline.

Enhancement-3 — Vectored/Bus-Style Net Declarations in OpenVAF

This document describes every source-code change made to OpenVAF in the `version4/` directory, on top of `version3/` (Enhancement-2, indirect branch assignment), to implement Verilog-AMS vectored net declarations — bus syntax for nets and ports, with bit-select access in branch declarations and `V()/I()` branch-access calls.

The goal is to make constructs like

```
module bus_buffer(in, out);
    input in;
    output [0:3] out;
    electrical in;
    electrical [0:3] out;
    analog begin
        V(out[0]) <+ 0.25 * V(in);
        V(out[1]) <+ 0.50 * V(in);
        V(out[2]) <+ 0.75 * V(in);
        V(out[3]) <+ 1.00 * V(in);
    end
endmodule
```

(the non-ANSI port style above — bare names in the module header, direction and width declared separately in the body — is the common Verilog-A idiom and is fully supported; see §2.8 for the fix this required.)

compile and simulate correctly end-to-end (DC, transient) through the OSDI flow, with branch (`bus[2]`, `bus[0]`) `br`;-style bit-select endpoints also supported.

Scope, confirmed up front with the user:

- Declaration: `<discipline> [msb:lsb] name, ...;` for both `NetDecl` and `PortDecl` (body and module-head forms).
- Bit-select `bus[i]` valid in `BranchDecl` endpoints and `V()/I()` branch-access arguments. `i` must be a compile-time-constant integer (a literal, optionally unary-negated), range-checked against the bus's declared `[msb:lsb]`.
- Part-select `bus[hi:lo]` is only legal in the declaration's own width clause — never as a branch endpoint or call argument. This OpenVAF subset compiles one standalone compact model at a time (no submodule instantiation), and a branch endpoint is exactly one node, so there is no "vector branch expansion" to support.
- A bare bus name used without an index anywhere a node is expected produces a clean diagnostic, never a panic.

1. The idea (how bus declarations are modeled)

Unlike Enhancement-1 (`absdelay`) and Enhancement-2 (indirect branch assignment), this feature needs no new DAE machinery — no implicit equations, no new MIR places. A bus is purely a name-resolution-time construct:

A declaration `electrical [3:0] bus;` is expanded, at item-tree lowering time, into four independent scalar **Net/Node** entries, with synthesized names `"bus[3]"`, `"bus[2]"`, `"bus[1]"`, `"bus[0]"`. OpenVAF's `Name` type is just a `SmolStr` wrapper with no identifier-syntax restriction, and node/port lookup throughout the compiler is plain `Name` equality — so synthesizing non-identifier-shaped names like `"bus[3]"` is safe and requires no change to the lookup data structures themselves.

A new HIR expression node `Expr::BitSelect { base, index }` resolves `bus[i]`: it constant-folds `i`, range-checks it against the bus's recorded `[msb:lsb]`, synthesizes the same `"bus[i]"` name, and resolves it exactly like an ordinary `Expr::Path` would — producing `Ty::Node`. Because everything downstream of `Ty::Node` (branch resolution, MIR lowering, OSDI export) is already agnostic to *how* a `NodeId` was produced, this requires zero changes in `hir_lower`, `mir*`, `sim_back`, or `osdi` — only one match-arm addition in `hir::body::get_expr`. This is a stronger form of the “no backend changes needed” result Enhancement-2 documented.

`BranchDecl` endpoints (`branch (bus[2], gnd) br;`) are handled the same way, but one layer earlier: since `hir_def::BranchKind` is built directly from `ast::BranchKind` during item-tree lowering (before any HIR body/type inference exists), a bit-select endpoint there is constant-folded and resolved to a synthesized `hir_def::Path` (`Path::new_ident("bus[2]")`) right at that point, reusing the exact same `Path` type every other branch endpoint already uses.

2. OpenVAF changes

2.1 `openvaf/syntax/veriloga.ungram` — grammar

- `NetDecl/PortDecl` gained an optional `width: Range?` field, reusing the pre-existing `Range` rule (`'[' Expr ':' Expr ']'`) already used by parameter `from/exclude` constraints.
- New `Expr` alternative, parallel to `PathExpr`:
`BitSelectExpr = base: Path '[' index: Expr ']'`

2.2 `sourcegen/src/ast/src.rs` — manual `SyntaxKind` registration `BIT_SELECT_EXPR` had to be added to the hand-maintained `nodes: &[...]` list in `KINDS_SRC` (the `SyntaxKind` enum and `tokens/src/parser/generated.rs` are *not* auto-derived from the ungram's node list — only the `ast` struct/accessor codegen is). Regenerated via `cargo test -p sourcegen`.

2.3 Parser (`openvaf/parser/src/grammar/`)

- `items.rs`: new `pub(super) fn width_range(p)` parses `'[' expr ':' expr ']'` into a `RANGE` node — a small dedicated helper (the existing `range_or_expr`, used by parameter constraints, also has to disambiguate parenthesized sub-expressions and was left untouched).
- `items/module.rs`: `net_decl/port_decl` (covering both module-head and body port-decl forms) call `width_range(p)` when the next token is `[`, right after the discipline/net_type prefix and before the name list.
- `expressions.rs`: `atom_expr`'s `IDENT | ROOT_KW` arm gained a third case (alongside the existing call/bare-path cases): if the next token is `[`, parse `'[' expr ']'` into `BIT_SELECT_EXPR`. This single insertion point covers `V()/I()` call arguments and `BranchDecl` arguments for free, since both already parse arguments via the shared `expr()/atom_expr()` path.

2.4 `openvaf/syntax/src/ast/node_ext.rs` — AST extensions

- `Expr::as_bit_select(&self) -> Option<(Path, Expr)>`, mirroring the existing `as_path()`.
- New `BranchEndpoint` enum (`Plain(Path) / BitSelect(Path, Expr)`). `BranchKind::NodeGnd/Nodes` now hold `BranchEndpoint` instead of a bare `Path`, so a branch can mix a bit-select and a plain endpoint (e.g. `branch (bus[2], gnd) br;`).
- `openvaf/syntax/src/validation.rs`'s `validate_branch_decl` (which rejects non-identifier branch arguments with a clean `IllegalBranchNodeExpr` diagnostic) gained `ast::Expr::BitSelectExpr` as an accepted endpoint shape, alongside the existing plain-path and port-flow cases.

2.5 `openvaf/hir_def` — item-tree expansion and the new `Expr::BitSelect`

- `expr.rs`: new `Expr::BitSelect { base: Path, index: ExprId }` HIR variant, with a `walk_child_exprs` arm visiting `index`.
- `body/lower.rs`: a `collect_expr` arm lowers `ast::Expr::BitSelectExpr` into `Expr::BitSelect`.
- `item_tree.rs`:
 - Module gained `pub buses: Vec<BusDecl>`.
 - New `BusDecl { base_name, msb, lsb, ast_id }`, with `min_max()`, `contains_bit(bit)`, and `bit_name(bit)` helpers.
 - New `ItemTreeDiagnostic::NonConstantBusWidth { ast_id }` for a `[msb:lsb]` clause that doesn't constant-fold.
- `item_tree/lower.rs`:
 - `fold_width_range(range: &ast::Range) -> Option<(i32, i32)>` — a tiny AST-level constant folder (via `ast::Expr::as_constexprval()`, the existing literal/negated-literal evaluator used elsewhere for attribute values), restricted to integer literals.
 - `expand_bus_names(...)` — for each declared name, if a width clause is present and folds successfully, registers a `BusDecl` and emits one `(name, name_idx)` per bit (`"bus[3]" .. "bus[0]"`, ascending, declaration `[msb:lsb]` direction only affects range checks); on a non-constant width, pushes `NonConstantBusWidth` and falls back to an ordinary scalar declaration so compilation proceeds.
 - `lower_net_decl/lower_port_decl` now call `expand_bus_names` instead of iterating `decl.names()` directly; the rest of the per-name `Net/Port/Node` insertion logic — including the existing “merge into an already-declared node of the same name” path that already produces a clean duplicate-declaration diagnostic — is completely unchanged, since it already only operates on `Names`.
 - `lower_branch` gained `resolve_branch_endpoint(ast::BranchEndpoint) -> Option<Path>`: a plain endpoint resolves via the existing `Path::resolve`; a bit-select endpoint constant-folds its index (via `as_constexprval`) and synthesizes `Path::new_ident("base[idx]")` — the exact same path the corresponding bus bit was declared under. A non-constant index simply fails to resolve, degrading the branch to `BranchKind::Missing` (the same graceful fallback any other unresolvable branch endpoint already takes — no panic).

2.6 `openvaf/hir_ty` — bit-select inference and diagnostics

- `inference.rs`:
 - `Ctx` gained an `owner: DefWithBodyId` field (set in `infe_body_query`) and a `find_bus(&self, name) -> Option<BusDecl>` helper that looks up the owning module's `BusDecl` registry (only meaningful inside a module body; returns `None` otherwise).
 - The existing `Expr::Path { port: false }` arm gained a bare-bus check: if the path is a single identifier matching a known bus's base name, a new `BareBusReference` diagnostic is emitted instead of attempting ordinary resolution (which would otherwise produce a confusing generic “unresolved identifier” error).
 - New `infe_bit_select(stmt, expr, base, index) -> Option<Ty>`: resolves `base` against the bus registry (falling through to ordinary path resolution — and its existing diagnostics — if it isn't a known bus, so a genuine typo still gets a normal error rather than a bus-specific one); type-checks `index` via the normal expression inference path, then separately requires it to constant-fold to `Expr::Literal(Literal::Int)` (optionally wrapped in a single `UnaryOp::Neg`); range-checks the result against the bus's `[msb:lsb]`; and finally resolves the synthesized `"base[idx]"` name through the same `resolve_path` machinery as ordinary identifiers, producing `Ty::Node`.
 - `infe_expr`'s `match` gained an `Expr::BitSelect { ref base, index }` arm dispatching to `infe_bit_select`.

- Four new InferenceDiagnostic variants: InvalidBusReference, NonConstantBitSelectIndex, BitSelectOutOfRange, BareBusReference.
- `diagnostics.rs`: report text for all four, in the same style as Enhancement-2's IndirectAssignRequiresEqual (primary label + source span + help note, never a panic).

2.7 `openvaf/hir_def/src/item_tree/lower.rs` — non-ANSI bus ports A module header may declare a port by bare name only (`module m(in, out);`), with its direction/discipline/width given separately in the body (`output [3:0] out;`) — standard non-ANSI Verilog-A port style, and already supported for scalar ports before this enhancement (the body declaration's exact-name match finds and fills in the header's placeholder Node). For a *bus* port declared this way, the header only ever creates one placeholder node under the bare base name ("out"), but the body's width clause then needs to expand it into four nodes ("out[0]".."out[3]") — a mismatch the original per-name exact-match lookup couldn't bridge, leaving the header's "out" placeholder dangling and unresolved (manifesting as `error[L016]: no direction declared for port 'out'`).

Fix: `expand_bus_names` now marks the *first* synthesized bit of every bus with its original base name. A new `find_node_for_decl` helper tries an exact-name match first (the existing behavior), and — only for that first bit — falls back to finding a still-unresolved (`decls.is_empty()`) node under the base name and renaming it in place to the first bit's name before attaching the declaration. This reuses the header's placeholder Node (and its already-recorded position/identity) for bit 0, rather than discarding it; bits 1..N are still created as brand new nodes, same as before.

This is also why `openvaf/hir_def/src/data.rs`'s `ModuleData::module_data_query` changed: it used to classify a module's nodes into ports vs. internal nodes by slicing `nodes[0..num_ports]/nodes[num_ports..]`, where `num_ports` is captured right after the header's port list is lowered — *before* the body runs. A bus port expanding from 1 header placeholder to 4 nodes during body processing therefore grew nodes past the already-frozen `num_ports` cutoff, so its extra bits were silently misclassified as internal nodes instead of OSDI terminals (`error: too many nodes connected to instance from ngspice`, since the compiled model under-reported its terminal count). The fix replaces the positional slice with a filter on `Node::is_port`, which is correct regardless of when or where in nodes a port node was created or expanded.

2.8 `openvaf/hir/src/body.rs` — the only downstream change `BodyRef::get_expr`'s match gained `hir_def::Expr::BitSelect { .. }` as a second pattern on the existing `hir_def::Expr::Path { .. }` $\Rightarrow$ `Expr::Read(self.resolve_path(expr))` arm. `resolve_path` only consults `inference.expr_types`, not the raw `hir_def::Expr` shape, so this one line is sufficient — confirmed by the MIR evidence in §6 below.

3. Diff summary (version3 $\rightarrow$ version4)

File	Kind of change
<code>openvaf/syntax/veriloga.ungram</code>	grammar: <code>BitSelectExpr, width:Range?</code> on <code>NetDecl/PortDecl</code>
<code>sourcegen/src/ast/src.rs</code>	register <code>BIT_SELECT_EXPR</code> in the manual <code>SyntaxKind</code> list
<code>openvaf/parser/src/grammar/items.rs</code>	new <code>width_range</code> helper
<code>openvaf/parser/src/grammar/items/module.rs</code>	wire <code>width_range</code> into <code>net_decl/port_decl</code>

File	Kind of change
openvaf/parser/ src/grammar/ expressions.rs	parse base[index] as BIT_SELECT_EXPR in atom_expr
openvaf/syntax/ src/ast/node_ext. rs	as_bit_select, BranchEndpoint, updated BranchKind
openvaf/syntax/ src/ast.rs	export BranchEndpoint
openvaf/syntax/ src/validation.rs	accept bit-select as a valid BranchDecl endpoint shape
openvaf/hir_def/ src/expr.rs	new Expr::BitSelect HIR variant
openvaf/hir_def/ src/body/lower.rs	lower ast::Expr::BitSelectExpr
openvaf/hir_def/ src/item_tree.rs	Module::buses, BusDecl, ItemTreeDiagnostic::NonConstantBusWidth
openvaf/hir_def/ src/item_tree/ lower.rs	fold_width_range, expand_bus_names, bus-aware lower_net_decl/lower_port_decl/lower_branch, find_node_for_decl (non-ANSI bus port merge, §2.7)
openvaf/hir_def/ src/data.rs	module_data_query: classify ports/internal nodes by is_port, not a num_ports positional cutoff (§2.7)
openvaf/hir_def/ src/lib.rs	export BusDecl
openvaf/hir_ty/ src/inference.rs	find_bus, infere_bit_select, bare-bus check, 4 new diagnostics
openvaf/hir_ty/ src/diagnostics. rs	report text for the 4 new diagnostics
openvaf/hir/src/ body.rs	one match-arm addition in get_expr

No changes to openvaf/hir_lower, openvaf/mir*, openvaf/sim_back, or openvaf/osdi — see §6.

4. New diagnostics

Diagnostic	When
ItemTreeDiagnostic::NonConstantBusWidth	a [msb:lsb] width clause doesn't constant-fold to two integer literals; the declaration falls back to an ordinary scalar net/port
InferenceDiagnostic::InvalidBusReference	a bit-select base isn't a simple identifier
InferenceDiagnostic::NonConstantBitSelectIndex	a bit-select index isn't a constant integer literal (optionally negated), e.g. V(bus[i]) for a variable i
InferenceDiagnostic::BitSelectOutOfRange	a bit-select index is outside the bus's declared [msb:lsb]
InferenceDiagnostic::BareBusReference	a bus is referenced by its base name with no bit-select, e.g. V(bus, n)

All five produce a normal compiler error with a source span and a help note, never a panic. A bare bus name used as a BranchDecl endpoint (branch (bus, gnd) br;) is a known, narrower gap: it degrades gracefully to BranchKind::Missing (no crash, the branch just doesn't resolve) rather than emitting one of the diagnostics above — surfacing a dedicated hir_def-level diagnostic for that specific case was left out of scope here in favor of the higher-value V()/I()-argument and declaration-time checks, which cover the common cases.

5. Why almost no backend/OSDI changes were needed

A bus bit's Net/Port/Node entry is, by the time it reaches hir_lower/mir_build/sim_back/osdi, indistinguishable from any other scalar net or port declared the ordinary way — same NodeId, same PlaceKind::Contribute/IsVoltageSrc stamping, same OSDI terminal/node export. The *only* place that needed to know about buses at all was name resolution (hir_def item-tree lowering and hir_ty inference); once a bit-select expression resolves to a NodeId, every later stage treats it exactly like V(some_ordinary_net).

This was confirmed empirically: the new MIR snapshot test (§6) and a direct --dump-unopt-mir/--dump-mir run on the bus example (§7.2) both show the synthesized node names (bus[0]..bus[3]) flowing through ordinary branch resolution, DAE/Jacobian stamping, and OSDI export with no special-casing — and the compiled .osdi simulates correctly in ngspice (§7.3) without touching a single line in sim_back or osdi.

6. Build

```
cd version4/OpenVAF-master
cargo test -p sourcegen                                # regenerate SyntaxKind + AST from
→ the ungram
LLVM_SYS_181_PREFIX=/opt/homebrew/opt/llvm@18 \
  cargo build --release --bin openvaf-r --features openvaf-driver/llvm18
cp target/release/openvaf-r ../bin/macos/apple-silicon/openvaf-r
```

7. Testing & verification

7.1 Compiler unit / snapshot tests cargo test -p syntax -p hir_def -p hir_ty -p hir -p hir_lower — all existing tests pass unchanged (0 failures), confirming no regression in plain scalar net/port declarations, absdelay, indirect branch assignment, or any other existing construct (every pre-existing .va fixture has no width clause, so width is simply None and behaves exactly as before).

New .mir snapshot fixture openvaf/test_data/mir/bus_basic.va/.mir:

```
module bus_test(p, n);
  inout p, n;
  electrical p, n;
  electrical [3:0] bus;
  branch (bus[2], bus[0]) br_a;
  analog begin
    V(bus[1], n) <+ 1.0;
    I(br_a) <+ V(bus[3]) * 2;
  end
endmodule
```

This exercises declaration, `branch(bus[2], bus[0])`, and `V()/I()` bit-select end-to-end through `hir_lower::MirBuilder`. A direct `openvaf-r --dump-unopt-mir/--dump-mir` run on the same file shows the "Partially optimized MIR (with DAE)" output stamping `bus[0]..bus[3]` as ordinary nodes (visible in the literal table: `'bus[0]'`, `'bus[1]'`, `'bus[2]'`, `'bus[3]'`, `'flow(bus[1],n)'`), feeding the Jacobian/residual rows exactly like any scalar net — no special-casing artifacts.

New ui diagnostic fixtures under `openvaf/test_data/ui/`: `bus_bare_reference.va` (`V(bus, n)`), `bus_out_of_range.va` (`V(bus[10], n)` against a `[3:0]` bus), `bus_nonconstant_index.va` (`V(bus[i], n)` for a real variable `i`) — each produces a clear, correctly-spanned error (e.g. bus `'bus'` requires a bit-select `[i]`, bus bit-select index out of range, bus bit-select index must be a constant), verified by hand against the generated `.log` files.

7.2 End-to-end compile `version4/examples/bus_examples/bus_buffer.va` — a 4-tap fractional voltage buffer using a vectored port, declared non-ANSI style (bare names in the module header, direction/width given in the body — see §2.7):

```
module bus_buffer(in, out);
    input in;
    output [0:3] out;
    electrical in;
    electrical [0:3] out;
    parameter real gain = 1.0 from (0:inf);
    analog begin
        V(out[0]) <+ 0.25 * gain * V(in);
        V(out[1]) <+ 0.50 * gain * V(in);
        V(out[2]) <+ 0.75 * gain * V(in);
        V(out[3]) <+ 1.00 * gain * V(in);
    end
endmodule
```

compiles to a working `.osdi` with zero errors/warnings. The bus port expands to 5 OSDI terminals in declaration order (`in`, `out[0]`, `out[1]`, `out[2]`, `out[3]`), connected positionally in the SPICE netlist exactly like any other multi-terminal device.

7.3 ngspice simulation — new feature `version4/examples/bus_examples/` (`dc_sim.cir`, `ac_sim.cir`, `tran_sim.cir`), simulated with `version4/bin/.../ngspice`:

- DC sweep $-2V \dots 2V$: each tap tracks its exact fraction of the input (e.g. at $V_{in} = 2.0V$: $out_0 = 0.5V$, $out_1 = 1.0V$, $out_2 = 1.5V$, $out_3 = 2.0V$ — exactly $0.25 \times / 0.5 \times / 0.75 \times / 1.0 \times$ gain).
- AC 1kHz...1GHz: each tap shows a flat gain of -12.04 dB / -6.02 dB / -2.50 dB / 0 dB respectively (exactly $20 \times \log_{10}(\text{tap fraction})$) and 0° phase across the entire sweep — as expected for a purely algebraic model with no reactive elements.
- Transient 1kHz sine input: all four taps track the input instantaneously (purely resistive/algebraic model, no reactive elements), with the same exact fractional relationship at every sample.

Raw results saved in `dc.txt/ac.txt/tran.txt`, plotted in `dc.png/ac.png/tran.png`, in that directory (see `examples/bus_examples/README.md`).

7.4 Regression — `absdelay` (Enhancement-1) and indirect branch assignment (Enhancement-2) Both `version4/examples/absdelay_examples/absdelay.va` and `version4/examples/indirect_assignment_examples/opamp.va` were recompiled with the new `openvaf-r` and re-simulated (DC) against the documented Enhancement-2 baselines:

- `absdelay` 5-stage delay line DC sweep: bit-identical to `examples/absdelay_examples/examples/dc_sparse.txt`.

- opamp indirect-branch-assignment unity-gain buffer DC sweep: bit-identical to examples/
indirect_assignment_examples/dc.txt.

This confirms vectored net declaration support introduced no regressions in either prior enhancement or the broader OSDI/ngspice pipeline.

Enhancement-4 — Laplace-Domain Transfer-Function Operators in OpenVAF

This document describes every source-code change made to OpenVAF in the `version5/` directory, on top of `version4/` (Enhancement-3, vectored/bus net declarations), to implement:

1. Verilog-A's four Laplace transform filter analog operators (§1-8 below).
2. Array-variable declarations (`real [msb:lsb] x;`), added as a follow-up once it became clear `laplace_*`'s array-literal *arguments* ('`{...}`') were a different feature from array *variables* — see §9.
3. Array variables as **laplace_* num/den** arguments — letting a bare array-variable reference (`coeffs`) stand in for an array literal in a `laplace_*` call, so Parts 1 and 2 compose — see §17.
4. A large-integer-literal crash fix, found and fixed while building a real 5th-order Bessel filter example whose coefficients include large bare-integer-shaped literals — see §22.

Part 1: Laplace transform filter operators

The four operators:

```
laplace_nd(in, num_coeffs, den_coeffs)    // num/den given as polynomial
↳ coefficients
laplace_np(in, num_coeffs, den_poles)     // num as coefficients, den as poles
↳ (roots)
laplace_zd(in, num_zeros, den_coeffs)     // num as zeros (roots), den as
↳ coefficients
laplace_zp(in, num_zeros, den_poles)      // both num/den given as roots
```

The goal is to make constructs like

```
module laplace_lpf(in, out);
  input in;
  output out;
  electrical in, out;
  parameter real tau = 1e-6 from (0:inf);
  analog begin
    V(out) <+ laplace_nd(V(in), '{1.0}', '{1.0, tau}');
  end
endmodule
```

(a first-order RC-style low-pass filter, $H(s) = 1/(1+\tau s)$) compile and simulate correctly end-to-end (DC, transient) through the OSDI flow, for all four `laplace_*` forms, with coefficient/root lists given as '`{...}`' array-literal arguments.

Scope, confirmed up front with the user:

- All four forms — `laplace_nd`, `laplace_np`, `laplace_zd`, `laplace_zp`.
- `num/den` (or `zero/pole`) arguments are compile-time-constant real array literals ('`{...}`'), as already enforced by pre-existing `hir_ty` validation (see §1.3) — this was *not* new validation written for this enhancement.
- The optional trailing tolerance (`laplace_*_tol`) / `nature` argument is accepted for signature compatibility but has no effect: the realization built here is an exact algebraic transformation of the transfer function into an ODE system, not an approximation that would need an error tolerance to converge.
- `zi_nd/zi_np/zi_zd/zi_zp` (the z-domain/discrete-time analogues) remain out of scope and still report `is_unsupported`.

1. What was actually there already (and what wasn't)

Before writing any new logic, the existing scaffolding for `laplace_*` turned out to be front-end-complete but parser/lowering-incomplete — a more interesting starting state than Enhancement-3's bus feature, which started from nothing:

1. `BuiltIn::laplace_nd/np/zd/zp` already existed as recognized keywords, with `hir_ty` signatures (`LAPLACE_FILTER`, taking two `ArrayAnyLength{ty: Real}` arguments plus optional tolerance/nature) already wired up — but marked `is_unsupported()`, so any use was rejected with a hard compile error ("function ... is currently not supported by OpenVAF") before reaching `hir_lower` at all.
2. The array-literal expression syntax ('{a, b, c}', `ARRAY_EXPR` in the ungram/SyntaxKind/AST layers) was fully scaffolded but never parsed: `openvaf/parser/src/grammar/expressions.rs` had a complete `array_expr` parser function sitting commented out behind a `// TODO properly implement arrays` marker, with `atom_expr` never dispatching to it. So '{1.0, 2.0}' was a syntax error everywhere, independent of `laplace_*`.
3. `hir_ty::inference::infere_array` (the array-literal type-checker) had a latent bug: it returned `Ty::Val(ty)` — the *element* type — instead of `Ty::Val(Type::Array { ty, len })`, so even with parsing fixed, a 3-element real array literal would type-check as a bare `real`, not an array, failing every `ArrayAnyLength` signature match.
4. `openvaf/hir_lower/src/expr.rs::lower_array` was `todo!("arrays")` — arrays were never lowered as general MIR values (and still aren't; they remain a syntax-only construct consumed positionally by builtins like `laplace_*/noise_table`, never materialized as a runtime array value — see §2.2).

So this enhancement closes three latent gaps (#2-#4) in addition to the actual new feature (the transfer-function-to-DAE lowering, #5 below).

2. The idea (how Laplace operators are modeled)

2.1 No new "Laplace" DAE primitive — reuse of `idt`'s machinery Unlike `absdelay` (Enhancement-1), which needed a genuinely new simulator-side mechanism (history-based delay lookup, `PlaceKind: AbsDelayTime`, a dedicated `OsdiAbsDelayInfo` struct), a Laplace transfer function $H(s) = \text{num}(s) / \text{den}(s)$ can be realized exactly as an ordinary linear ODE system — no approximation, no new backend concept.

The standard controllable canonical form state-space realization is used: for a proper rational $H(s)$ of order $n = \text{deg}(\text{den})$, with n state variables $x_0 \dots x_{n-1}$:

$$\begin{aligned} dx_0/dt &= x_1 \\ dx_1/dt &= x_2 \\ &\dots \\ dx_{n-2}/dt &= x_{n-1} \\ dx_{n-1}/dt &= u - (a_{\text{bar}_0} * x_0 + a_{\text{bar}_1} * x_1 + \dots + a_{\text{bar}_{n-1}} * x_{n-1}) \end{aligned}$$

$$y = c_0 * x_0 + c_1 * x_1 + \dots + c_{n-1} * x_{n-1} + d * u$$

where u is the operator's input expression, $a_{\text{bar}_i} = \text{den}[i] / \text{den}[n]$ is the monic-normalized denominator, $d = \text{num}[n] / \text{den}[n]$ is the direct feedthrough term (only present when num is exactly proper, i.e. $\text{deg}(\text{num}) == \text{deg}(\text{den})$), and $c_i = (\text{num}[i] - d * \text{den}[i]) / \text{den}[n]$.

Each state x_i is exactly the same shape as the existing `idt()` builtin's implicit equation: a fresh implicit unknown (`ctx. implicit_equation`) with a *reactive* residual of x_i (contributing $d(x_i)/dt$ to KCL) and a *resistive* residual of $-(\text{RHS of } dx_i/dt)$, so that the simulator's standard $d(\text{react})/dt + \text{resist} = 0$ stamping produces exactly $dx_i/dt = \text{RHS}$. This is the same mechanism `lower_integral`

already uses for `idt(arg)` (`react = val`, `resist = -arg` there). The output `y` is then a purely algebraic linear combination of state values (plus `d*u`) — it needs no implicit equation of its own.

Because this reduces entirely to ordinary implicit-equation residuals (the same primitive `idt/absdelay` already use), no changes were needed in `sim_back` or `osdi` — exactly the same “no backend changes” result Enhancement-3 reported for bus nets, but here it falls out of reusing an *existing* DAE primitive rather than needing none at all.

2.2 Root-to-polynomial expansion for `np/zd/zp` `laplace_np/laplace_zd/laplace_zp` give one or both of `num/den` as a list of roots (poles/zeros) rather than polynomial coefficients. These are expanded into ascending-power coefficients at MIR-build time by repeated synthetic polynomial multiplication by `(s - root)`:

```
poly := [1]                                // the empty product
for each root r:
    poly := poly * (s - r)                  // convolution, fully unrolled
```

Since the *number* of roots/coefficients is always known at compile time (it’s the literal length of an `{...}` array — no constant-folding of the array *length* is needed, unlike the array *element values*, which are MIR Values computed normally via `lower_expr` and may depend on instance parameters), this unrolls into a small, fixed sequence of `fmul/fsub/fneg` MIR instructions per call site — no runtime loop, no new MIR opcode.

2.3 Why arrays are still not general MIR values `lower_array/Expr::Array` remain syntax-only: a `{...}` argument to `laplace_*` is never lowered as a single array Value. Instead, `array_elems()` reads the raw `hir::Expr::Array` node directly (bypassing `lower_expr` for the array as a whole) and lowers each element individually with the ordinary `lower_expr`. This mirrors how `noise_table_log({...})` and other pre-existing `ArrayAnyLength` consumers were always implicitly expected to work, and avoids having to invent a real “array value” MIR representation for a four-builtin, syntax-only feature.

3. OpenVAF changes

3.1 `openvaf/parser/src/grammar/expressions.rs` — enable array-literal parsing

- `atom_expr`’s `dispatch` gained `T!['{'"] | T!['{''] => array_expr(p)` (previously a commented-out TODO covering only `{}`).
- The `array_expr` parser function itself was uncommented and fixed: the original draft checked `p.at(T!['']')` for its loop-exit condition (a copy-paste bug from a `[...]` array sketch) instead of `T!['}']`, which would have looped past the actual closing brace. Now parses `('{' | '{') (Expr (',' Expr)*)? '}'` into an `ARRAY_EXPR` node, matching the pre-existing `BIT_SELECT_EXPR`-style structure already wired through the rest of the AST/HIR/type-inference stack (§3.2-3.3 confirm those layers were already complete).
- Both `{a, b, c}` and bare `{a, b, c}` are accepted as the opening delimiter, lexed as distinct tokens (`ArrStart` for `{`, `OpenBrace` for plain `{`) but parsed identically — `array_expr`’s opening bump uses `p.bump_ts(TokenSet::new(&T!['{'"], T!['{'']))` rather than a single fixed token. This matches a precedent already in the codebase: `openvaf/parser/src/grammar/items.rs`’s `constraint` parser (for parameter `from/exclude` array-style constraints) already accepted both spellings (`p.eat(T!['{'"])` || `p.eat(T!['{''])`) before this enhancement touched anything — `array_expr` simply needed to follow the same convention. Verified end-to-end: `laplace_nd(V(in), {1.0}, {1.0, tau})` (bare braces) and `laplace_nd(V(in), '{1.0}', '{1.0, tau})` (with leading `'`) compile to bit-identical MIR and produce identical simulated results.

This is a prerequisite for *any* array literal anywhere in the language, not just `laplace_*` arguments — `noise_table_log` and other pre-existing `ArrayAnyLength`-typed builtins benefit identically as a side effect.

3.2 `openvaf/hir_ty/src/inference.rs` — fix `infern_array`'s return type `infern_array` (the array-literal type-checker) returned `Ty::Val(ty)` — the unified *element* type — instead of `Ty::Val(Type::Array { ty: Box::new(ty), len })`. This made every non-empty array literal type-check as a bare scalar of its element type, so it could never satisfy an `ArrayAnyLength{ty: Real}` parameter requirement (only the *element* type matched, not “is an array”), failing overload resolution for every `laplace_*/noise_table_log` call with a literal array argument. Fixed to construct the correct `Type::Array` value, matching the `(Ty::Val(Type::Array{ty: ref ty1, ..}), TyRequirement::ArrayAnyLength{ty: ty2}) => equiv.compare_ty(ty1, ty2)` match arm in `hir_ty/src/types.rs` that was already correctly written to expect it.

3.3 `openvaf/hir_def/src/builtin.rs` / `sourcegen/src/hir_builtins.rs` — mark Laplace as supported `laplace_nd/laplace_np/laplace_zd/laplace_zp` removed from the `UNSUPPORTED` list in `sourcegen/src/hir_builtins.rs` (the source of truth; `openvaf/hir_def/src/builtin.rs` is regenerated from it via `cargo test -p sourcegen`, so it was *not* hand-edited directly). `zi_*` (z-domain) builtins were left in `UNSUPPORTED`, out of scope.

3.4 `openvaf/hir_lower/src/lib.rs` — new implicit-equation kind

- New `ImplicitEquationKind::LaplaceState(u32)` variant — purely a descriptive tag (mirroring `AbsDelayInput/AbsDelayOutput/IndirectBranch`) for debug/MIR-dump readability; nothing downstream pattern-matches on `ImplicitEquationKind` beyond `hir_lower` itself (confirmed by `grep` — only `lineralize.rs`'s unrelated `inline-ddt()` handling and `hir_lower/src/stmt.rs`'s `IndirectBranch` push do, and neither needed to change), so this needed no further wiring.

3.5 `openvaf/hir_lower/src/expr.rs` — the actual lowering

- `lower_builtin`'s `match` gained a `Builtin::laplace_nd | laplace_np | laplace_zd | laplace_zp => self.lower_laplace(builtin, args)` arm.
- `lower_laplace`: lowers the input expression, lowers each num/den (or zero/pole) array element via the ordinary `lower_expr`, and dispatches num/den through `laplace_roots_to_poly` for whichever of the two (if either) the specific builtin variant gives as roots (`laplace_zd/laplace_zp` for num, `laplace_np/laplace_zp` for den), then calls `laplace_state_space`.
- `array_elems`: reads `hir::Expr::Array` directly off `self.body` for a given `ExprId` (falling back to treating the expression as a single-element array if it somehow isn't a literal array — defensive, not expected to trigger given the type-level `ArrayAnyLength` requirement).
- `laplace_roots_to_poly`: the polynomial-from-roots expansion of §2.2.
- `laplace_state_space`: builds the controllable-canonical-form realization of §2.1 — normalized denominator, direct feedthrough, output coefficients, `n` implicit equations with paired reactive/resistive residuals, and the final algebraic output combination. Handles the degenerate `n == 0` case (constant-gain, dynamics-free) directly as `(num[0]/den[0]) * input`.

3.6 No changes needed in `sim_back` or `osdi` As in Enhancement-3, confirmed empirically (§5 below): once a Laplace call lowers to ordinary implicit-equation residuals, every later stage (topology linearization, Jacobian/residual stamping, OSDI export) treats the resulting states exactly like any other `idt()`-style implicit unknown.

4. Diff summary (version4 → version5)

File	Kind of change
openvaf/parser/ src/grammar/ expressions.rs	enable '{...}' array-literal parsing (array_expr, dispatch in atom_expr) — fixes a pre-existing dead/buggy stub, prerequisite for any array-literal use
openvaf/hir_ty/ src/inference.rs	fix infer_array to return Type::Array{..} instead of the bare element type — pre-existing latent type-inference bug
sourcegen/src/ hir_builtins.rs	remove laplace_nd/np/zd/zp from UNSUPPORTED (regenerates openvaf/ hir_def/src/builtin.rs)
openvaf/hir_ lower/src/lib.rs	new ImplicitEquationKind::LaplaceState
openvaf/hir_ lower/src/expr. rs	lower_laplace, array_elems, laplace_roots_to_poly, laplace_ state_space — the actual transfer-function-to-DAE lowering

No changes to openvaf/hir_def item-tree/lowering, openvaf/mir*, openvaf/sim_back, or openvaf/osdi.

5. Why almost no backend/OSDI changes were needed

Once laplace_state_space finishes, every state x_i is, from mir_opt/sim_back/osdi's point of view, indistinguishable from an `idt()` integrator's implicit unknown: same `ImplicitEquation` index space, same `PlaceKind::ImplicitResidual{equation, reactive}` stamping, same OSDI Jacobian-entry generation. The output value y is a plain algebraic MIR expression (sums and products of state/parameter values), so it needs no special handling at all — it flows into the branch contribution exactly like any other expression.

This was confirmed via `--dump-unopt-mir/--dump-mir` on the test fixtures in `examples/laplace_examples/`: the `Implicit equations` section of the HIR interner dump lists one `LaplaceState(i)` entry per state (one for a first-order `laplace_nd/laplace_np` filter, two for a second-order `laplace_zd/laplace_zp` filter), feeding ordinary reactive/resistive residuals with no special-casing artifacts anywhere in the dump.

6. Build

```
cd version5/OpenVAF-master
cargo test -p sourcegen                                # regenerate is_unsupported() from
↪ hir_builtins.rs
LLVM_SYS_181_PREFIX=/opt/homebrew/opt/llvm@18 \
  cargo build --release --bin openvaf-r --features openvaf-driver/llvm18
cp target/release/openvaf-r ../bin/macos/apple-silicon/openvaf-r
```

7. Testing & verification

7.1 Compiler unit / snapshot tests `cargo test -p syntax -p hir_def -p hir_ty -p hir -p hir_lower` — all existing tests pass unchanged (0 failures), including the Enhancement-2 (`mir::indirect_branch.va`) and Enhancement-3 (`mir::bus_basic.va`) MIR snapshot fixtures, confirming

the array-literal parser/type-inference fixes and the new Laplace lowering introduced no regression in any existing construct.

7.2 End-to-end compile `version5/examples/laplace_examples/`:

- `laplace_lpf.va` — `laplace_nd(V(in), '{1.0}', '{1.0, tau})`, a first-order RC-style low-pass filter (single bus-free port pair).
- `laplace_variants.va` — all four forms side by side: `laplace_nd/ laplace_np` both realizing the same $H(s) = 1/(1+\tau s)$ (coefficient form vs. pole form), and `laplace_zd/laplace_zp` both realizing the same second-order $H(s) = (s+2e6)/((s+1e6)(s+3e6))$ (zero-with-coefficient-denominator form vs. fully factored pole/zero form).
- `laplace_zd_only.va` — an isolated single-call fixture used to cross-check the multi-call comparison file's results column-by-column.

All compile to a working `.osdi` with zero errors/warnings. `--dump-unopt-mir/--dump-mir` confirm the expected implicit-equation counts: 1 state for each first-order filter, 2 states for each second-order filter (`openvaf-r --dump-mir laplace_zd_only.va` shows `0 : LaplaceState(0) / 1 : LaplaceState(1)`).

7.3 ngspice simulation — new feature `version5/examples/laplace_examples/` (`dc_sim.cir`, `ac_sim.cir`, `tran_sim.cir` for the primary `laplace_lpf` model, plus `dc_variants.cir/ dc_zd_only.cir` for the four-forms cross-check), simulated with `version5/bin/.../ngspice`. Raw results saved in `dc.txt/ac.txt/ tran.txt`, plotted via `plot_results.py` (matplotlib) into `dc.png/ac.png/tran.png`:

- DC (`dc.txt/dc.png`), `laplace_lpf` (first-order, $\tau=1e-6$): out tracks in exactly 1:1 across a -2V..2V sweep — correct, since $H(0) = 1/(1+0) = 1$.
- DC, `laplace_variants` (`dc_variants.txt`): at `in = 2.0`, `out_nd = out_np = 2.0` (both realize $H(0)=1$) and `out_zd = out_zp = 1.33333e-6` (both realize the second-order system's $H(0) = 2e6/3e12 = 6.667e-7$, so $2.0 * 6.667e-7 = 1.3333e-6$) — all four forms agree exactly on equivalent transfer functions, cross-validating the coefficient and root-expansion code paths against each other.
- AC (`ac.txt/ac.png`), `laplace_lpf` swept 1kHz..1GHz: a textbook single-pole Bode response — flat 0 dB / 0° well below the corner frequency $f_{pole} = 1/(2\pi\tau) = 159.2$ kHz, exactly -3.0 dB / -45.0° at f_{pole} (read directly off the AC sweep data, e.g. 158.5 kHz $\rightarrow$ -2.99 dB, -0.7833 rad = -44.88°), a -20 dB/decade rolloff above it, and phase asymptoting to -90° at high frequency — confirming the realization's frequency-domain behavior, not just its DC/step response, matches the analytic transfer function.
- Transient (`tran.txt/tran.png`), `laplace_lpf` step response: a 0 $\rightarrow$ 1V step at $t=0$ produces $V(out) = 0.632...$ at $t = \tau = 1\mu s$ — exactly $1 - e^{-1} = 0.63212...$, the analytic first-order step response, confirming the state-space realization's *dynamics* (not just its DC gain) are correct.

7.4 Regression — bus nets (Enhancement-3), `absdelay` (Enhancement-1), indirect branch assignment (Enhancement-2) `examples/bus_examples/bus_buffer.va`, `examples/absdelay_examples/absdelay.va`, and `examples/indirect_assignment_examples/opamp.va` all recompile cleanly with the new `openvaf-r` (zero errors/warnings), and their corresponding `hir_lower` MIR snapshot tests (`mir::bus_basic.va`, plus the full `-p hir_lower` suite) pass unchanged — confirming the parser/type-checker fixes needed for array literals (§3.1, §3.2) didn't disturb bit-select expressions, branch resolution, or any other prior-enhancement construct.

8. Known limitations

- Improper transfer functions are not diagnosed. If $\text{deg}(\text{num}) > \text{deg}(\text{den})$ (more numerator coefficients than the state-space realization can use), the extra high-order numerator terms are silently dropped rather than producing a diagnostic or supporting differentiation of the input. Enhancement-3 set a precedent of documenting this kind of narrower gap rather than blocking on it (its bare-bus-as-branch-endpoint case); the same call is made here given improper transfer functions are an unusual/degenerate modeling case. A `hir_ty`-level diagnostic (comparing `num/den` array-literal lengths, no constant-folding needed) would be the natural follow-up.
 - **zi_nd/zi_np/zi_zd/zi_zp** (z-domain/discrete-time filters) remain unimplemented (`is_` unsupported), as scoped up front.
-

Part 2: Array-variable declarations (`real [msb:lsb] x;`)

9. Motivation and prior state `laplace_*`'s `'{...}'` arguments (Part 1) are array-literal expressions — fixed-size constant lists, never assigned to or indexed by a runtime-variable index, and not a storage location. Verilog-A separately allows declaring an array variable — a named, indexable piece of mutable storage, e.g.:

```
real [0:4] coeffs;
analog begin
    coeffs[0] = 0.1;
    V(out) <+ coeffs[0] * V(in);
end
```

This was previously unsupported: `VarDecl`'s grammar rule (`openvaf/syntax/veriloga.ungram`) had no width/range field at all (unlike `NetDecl/PortDecl`, which already gained one in Enhancement-3), so `real [0:4] x;`/`real x[0:4];` were both syntax errors.

10. Design: same compile-time-constant-index expansion as bus nets Rather than building genuine runtime-indexable array storage (a much larger change — a new MIR memory/array primitive, since today's MIR only has scalar SSA values, confirmed by `hir_lower::lower_array` still being unimplemented, see §2.3 in Part 1), array variables are modeled exactly like Enhancement-3's bus nets: `real [msb:lsb] x;` is expanded at item-tree lowering time into independent scalar `Var` entries ("`x[4]`" .. "`x[0]`"), and `x[i]` reuses the already-generic `Expr::BitSelect` machinery from Enhancement-3 — `i` must be a compile-time-constant integer (a literal, optionally unary-negated), range-checked against the declared `[msb:lsb]`. This is a real, if narrower, capability than a general indexable array: a for-loop with a runtime-variable index into the array (`x[i]` for non-constant `i`) is not supported, same constraint bus-net bit-select already had.

The syntax chosen is `<type> [msb:lsb] name;` — the *prefix-range* form already established for `NetDecl/PortDecl` in this codebase — rather than chasing the LRM's postfix `name[size]` declarator form, for consistency with the existing bus-declaration convention.

11. Why this needed almost no new machinery The key discovery, made before writing any code: `Expr::BitSelect` resolution (`hir_ty::inference::inference::infer_bit_select`) was already fully generic past the point of finding a `BusDecl` and constant-folding the index — its *only* bus-specific step was the final match on what the synthesized "`x[i]`" name resolved to:

```
match self.resolve_path(stmt, expr, &synth_path)? {
    ScopeDefItem::NodeId(node) => Some(Ty::Node(node)),
    _ => None,    // a Var resolution was silently dropped here
}
```

Adding one arm — `ScopeDefItem::VarId(var) => Some(Ty::Var(self.db.var_data(var).ty.clone(), var))` — was the *entire* change needed for `x[i]` to work as a value (read) and as an assignment target (write):

- Reads: `infer_bit_select` returning `Ty::Var` flows into `hir::body::BodyRef::get_expr's` existing `Expr::BitSelect => Expr::Read (self.resolve_path(expr))` arm (added in Enhancement-3 §2.8) completely unchanged — `resolve_path` there already dispatches off `Ty::Var` identically for ordinary variable reads, producing `Ref::Variable(var)` and ultimately `ctx.read_variable(var)` in `hir_lower`.
- Writes: `hir_ty::inference::infer_assignment_dst` (which resolves an assignment statement's LHS expression, e.g. `x[i] = ...;`) calls the same generic `infer_expr` used for any expression, and already had a `Ty::Var(ty, var) => (AssignDst::Var(var), ty)` arm — so once `infer_bit_select` can produce `Ty::Var`, assignment-to-a-bit-select works immediately, with no further code. Same for `hir::body's as_assignment_lhs (Expr::Read(Ref::Variable(var)) => AssignmentLhs::Variable(var))`.

So unlike Part 1 (which needed a genuinely new lowering, `lower_laplace`), this feature needed no new `hir_lower/mir/sim_back/osdi` code at all — each expanded scalar `Var` is, from MIR onward, indistinguishable from a variable declared without a width clause.

12. OpenVAF changes

12.1 **openvaf/syntax/veriloga.ungram** — grammar `VarDecl` gained an optional width: `Range?` field, reusing the pre-existing `Range` rule, identical in shape to `NetDecl/PortDecl's`:

```
VarDecl =
  AttrList* Type width:Range? (Var (',' Var)*) ';'

```

12.2 **openvaf/parser/src/grammar/items.rs** — parser `var_decl` calls the existing `width_range(p)` helper (built for `NetDecl/PortDecl` in Enhancement-3) when the next token is `[`, right after the type and before the name list — the same insertion pattern `net_decl/port_decl` already use.

12.3 **openvaf/hir_def/src/item_tree.rs** — registry

- Module gained `pub var_arrays: Vec<BusDecl>`, kept as a separate list from buses (nets/ports) purely so the two declaration kinds stay distinguishable for diagnostics/debugging — `BusDecl` itself (base name, `msb`, `lsb`, `ast_id`) is reused verbatim, since an array variable is structurally identical to a bus: a base name plus an `[msb:lsb]` range expanding into independent scalar entries.
- New `ItemTreeDiagnostic::ArrayVarUnsupportedScope { ast_id }` for a width clause that appears somewhere array-variable resolution doesn't support (§12.4).

12.4 **openvaf/hir_def/src/item_tree/lower.rs** — expansion, and scope restriction `lower_var` (shared by module-body `VarDecls`, analog function-body `VarDecls`, and nested `begin..end-block VarDecls`) gained a `var_arrays: Option<&mut Vec<BusDecl>>` parameter:

- The module-body call site (inside `lower_module_items`) passes `Some(&mut var_arrays)`, an accumulator threaded through `lower_module` exactly like buses already is, attached to the final `Module { ..., var_arrays }`.
- The analog function-body and nested-block call sites pass `None` — array-variable bit-select resolution is module-body scope only (mirroring `find_bus's` existing `DefWithBodyId::ModuleId`-only lookup; a function has its own `DefWithBodyId::FunctionId` owner, so a registry keyed only on the module wouldn't be reachable from inside one anyway without further plumbing).

A width clause in either unsupported scope triggers `ArrayVarUnsupportedScope` and degrades gracefully to an ordinary scalar variable, never a panic.

When `var_arrays` is `Some` and the width constant-folds, each declared name expands into one scalar `Var` per bit (ascending `lo..=hi`, synthesized name `"x[i]"` via the same `bus_bit_name` helper Enhancement-3 added), and a `BusDecl` is recorded. A non-constant width falls back to `NonConstantBusWidth` (the same diagnostic bus nets already use) plus a scalar declaration, exactly mirroring `expand_bus_names`.

A default initializer on an array declaration (`real [0:4] x = 1.0;`) is silently ignored (not assigned to any bit) — see §13.

12.5 `openvaf/hir_ty/src/inference.rs` — resolution

- New `find_var_array(&self, name) -> Option<BusDecl>`, identical in shape to `find_bus` but querying `Module::var_arrays`.
- `infe_bit_select`: the base-name lookup is now `self.find_bus(&name).or_else(|| self.find_var_array(&name))` — a bit-select base can be *either* a bus or an array variable (the two can't collide on the same name, since they'd already conflict as ordinary duplicate declarations).
- The final resolution match gained `ScopeDefItem::VarId(var) => Some(Ty::Var(self.db.var_data(var).ty.clone(), var))` (§11) — the one line that makes everything else work.
- The bare-reference check (`Expr::Path` with no bit-select) now also checks `find_var_array`, so `V(out) <+ coeffs * V(in);` (forgetting the `[i]`) reuses the existing `BareBusReference` diagnostic rather than falling through to a generic "unresolved identifier" error. Its message text was generalized from "bus referenced..."/"select a single bit" to "bus/array referenced..."/"select a single element" to read sensibly for both cases (`openvaf/hir_ty/src/diagnostics.rs`).

13. New/changed diagnostics

Diagnostic	When
<code>ItemTreeDiagnostic</code> : a <code>[msb:lsb]</code> width clause on a <code>VarDecl</code> inside an <code>analog</code> function body	
:	or a nested <code>begin...end</code> block (only module body scope is supported); falls back to a scalar variable
<code>ArrayVarUnsupportedScope</code>	
<code>ItemTreeDiagnostic</code> : (reused) a width clause that doesn't constant-fold, on an array variable	
:	same as on a bus net
<code>NonConstantBusWidth</code>	
<code>InferenceDiagnostic</code> : (reused, generalized wording) an array variable referenced by its base name	
:	with no bit-select, e.g. <code>V(out) <+ coeffs * V(in);</code>
<code>BareBusReference</code>	
<code>InferenceDiagnostic</code> : (reused) <code>coeffs[10]</code> against a declared <code>[0:4]</code> array	
:	
<code>BitSelectOutOfRange</code>	
<code>InferenceDiagnostic</code> : (reused) <code>coeffs[i]</code> for a non-constant/variable <code>i</code>	
:	
<code>NonConstantBitSelectIndex</code>	

All confirmed via hand-written `.va` fixtures producing clean, correctly spanned errors, never a panic (see §15).

14. Diff summary (additive to Part 1's table)

File	Kind of change
openvaf/syntax/ veriloga.ungram	VarDecl gains width: Range?
openvaf/parser/ src/grammar/items. rs	var_decl calls the existing width_range helper
openvaf/hir_def/ src/item_tree.rs	Module::var_arrays, ArrayVarUnsupportedScope diagnostic
openvaf/hir_def/ src/item_tree/ lower.rs	lower_var gains array expansion + module-body-only scope restriction; lower_module/lower_module_items thread the new var_arrays accumulator
openvaf/hir_def/ src/item_tree/ diagnostics.rs	report text for ArrayVarUnsupportedScope
openvaf/hir_ty/ src/inference.rs	find_var_array; infer_bit_select checks both registries and resolves ScopeDefItem::VarId; bare-reference check covers array variables too
openvaf/hir_ty/ src/diagnostics.rs	generalized BareBusReference wording

No changes to openvaf/hir, openvaf/hir_lower, openvaf/mir*, openvaf/sim_back, or openvaf/osdi (§11).

15. Testing & verification version5/examples/array_var_examples/array_var_fir.va — a 5-tap weighted-sum model exercising declaration, indexed write, and indexed read end-to-end:

```

module array_var_fir(in, out);
  input in;
  output out;
  electrical in, out;
  real [0:4] coeffs;
  analog begin
    coeffs[0] = 0.1;
    coeffs[1] = 0.2;
    coeffs[2] = 0.3;
    coeffs[3] = 0.2;
    coeffs[4] = 0.2;
    V(out) <+ (coeffs[0]+coeffs[1]+coeffs[2]+coeffs[3]+coeffs[4]) * V(in);
  end
endmodule

```

(coefficients sum to 1.0, so V(out) should track V(in) exactly — a closed-form expected result, same spirit as examples/bus_examples/bus_buffer.va). Compiles with zero errors/warnings; dc_sim.cir DC sweep (dc.txt) confirms V(out) = V(in) exactly across a -2V..2V sweep.

Diagnostics verified by hand with isolated fixtures: coeffs[10] = 1.0; (out-of-range, against [0:4]), coeffs * V(in) (bare reference), and a real [0:2] tmp; declared inside an analog function (unsupported scope) — each produces a clean, correctly spanned error, never a panic.

Regression: cargo test -p syntax -p hir_def -p hir_ty -p hir -p hir_lower passes unchanged (only one snapshot intentionally updated — test_data/ui/bus_bare_reference.log's wording, for the generalized bare-reference message in §12.5). All Part 1 (Laplace), Enhancement-3

(bus nets), Enhancement-2 (indirect branch assignment), and Enhancement-1 (absdelay) examples recompile cleanly with the rebuilt `openvaf-r`.

16. Known limitations

- No runtime-variable indexing. `x[i]` for a non-constant `i` (e.g. a for loop accumulating into an array) is not supported — `i` must constant-fold, identical to bus-net bit-select's existing constraint. Genuine indexable array storage would need a new MIR memory/array primitive (`hir_lower::lower_array` is still todo!), out of scope here.
- Module-body scope only. Array variables inside `analog` function bodies or nested `begin..end` blocks degrade to scalars with a diagnostic (§12.4) rather than being supported — extending `find_var_array`'s lookup to function/block scope would need a registry keyed on more than just the owning module.
- Default initializers are silently dropped on array declarations (`real [0:4] x = 1.0;` doesn't assign to any bit) — there's no well-defined per-bit meaning for a single scalar initializer on an array declaration; a diagnostic rejecting the combination outright would be the natural follow-up.

Part 3: Array variables as `laplace_* num/den` arguments

17. Motivation Parts 1 and 2 were built independently and didn't initially compose: a `laplace_* num/den` argument had to be a literal `'{...}'/{...}'` array, and a bare reference to an array *variable* (`coeffs` for `real [0:n] coeffs;`) was rejected by the BareBusReference diagnostic (§3.5's array-literal-only `array_elems` never saw anything else). So

```
real [0:1] coeffs;
analog begin
    coeffs[0] = -2e6;
    coeffs[1] = 0.0;
    V(out) <+ laplace_zd(V(in), coeffs, '{3e12, 4e6, 1.0}');
end
```

failed to compile with `'coeffs'` requires a bit-select `[i]` — correct per the rules as they stood, but an unnecessary restriction: there's nothing about the state-space realization that requires `num/den`'s *structure* (element count) to come from literal syntax specifically, as opposed to a previously-declared array variable of fixed size. Only the *element count* needs to be known at compile time (to size the realization) — the element *values* were already lowered as ordinary runtime values either way (see §2.2/§2.3), so an array-variable element written to at runtime is no harder to support than an array-literal element referencing a parameter.

18. Design: treat a bare array-variable reference as an implicit array literal A bare `coeffs` in a `laplace_* num/den` position is now accepted as exactly equivalent to writing out `'{coeffs[0], coeffs[1], ..., coeffs[n]}'` by hand (ascending declared-index order) — no new syntax, no new MIR concept, just one more shape `inere_laplace_array_arg` recognizes for that specific argument position. A *part-select*-style mid-array variable (`coeffs[1:3]`) is not supported — only a bare reference to the *whole* declared array, matching the scope already established for everything else in this enhancement (no part-select anywhere, per Enhancement-3's original scope).

19. Why this needed real (if contained) plumbing, unlike Part 2 Part 2 needed almost no new machinery because `inere_bit_select` resolving to `Ty::Var` flowed transparently through every existing generic mechanism (ordinary reads, ordinary assignment-destination resolution). This case is different: the *bare* `coeffs` (no bit-select at all) was, by design, deliberately rejected everywhere via the

BareBusReference check inside `infe_expr`'s generic `Expr::Path` arm — and that check is unconditional, with no way to make it context-sensitive (e.g. "unless this is a laplace argument") from inside `infe_expr` itself. Three changes were needed:

1. **openvaf/hir_ty/src/inference.rs** — bypass generic argument inference for **laplace_***'s **num/den** positions. `infe_builtin` gained a `laplace_nd | laplace_np | laplace_zd | laplace_zp` arm that short-circuits to a new `infe_laplace` function — mirroring the pre-existing `ddx` special case (`ddx`'s second argument is also not an ordinary value-typed expression, and already bypasses the shared `resolve_function_args` machinery the same way). `resolve_function_args` (used by every other builtin/ function call) unconditionally calls `infe_expr` on each argument, which would hit the bare-reference check before a laplace-specific override could intervene — so `laplace_*` needed its own argument loop instead of reusing it.

`infe_laplace` type-checks `args[0]` (input, must be `Real`) normally, dispatches `args[1]/args[2]` through the new `infe_laplace_array_arg` helper, and type-checks an optional `args[3]` (tolerance/nature) normally — picking the matching `LAPLACE_NO_TOL/LAPLACE_TOL/LAPLACE_NATURE_TOL` signature constant (their identity isn't otherwise load-bearing anywhere in the codebase — confirmed by `grep` — so this is purely for diagnostic/ tooling accuracy).

`infe_laplace_array_arg(stmt, arg):`

- if `arg` is `Expr::Array(elems)` (the pre-existing literal-array case): delegates to the existing `infe_array` exactly as before.
 - if `arg` is a bare `Expr::Path` whose name matches a known `find_var_array` entry: resolves each `coeffs[bit]` (`lo..=hi`, via the same `Path::new_ident(bus.bit_name(bit))`) + `resolve_path` pattern `infe_bit_select` already uses) to a `VarId`, records the ordered list in a new `InferenceResult::array_var_refs: AHashMap<ExprId, Vec<VarId>>` field, and assigns the argument `Ty::Val(Type::Array{ty: Real, len})` directly — never calling `infe_expr`/triggering `BareBusReference` for this one expression.
 - anything else: falls back to ordinary `infe_expr`, so a genuine scalar argument or typo still gets its normal diagnostic (the usual "expected array" type mismatch).
2. **openvaf/hir_ty/src/validation/body.rs** — stop forcing **num/den** into **BodyCtx::Const**. The pre-existing laplace validation arm forced every argument past the input signal into a constant-expression context (`validate_const_expr`, which sets `BodyCtx::Const` and rejects ordinary variable references via `allow_var_ref`). This was already stricter than what Part 1's lowering actually needs — each array element was always lowered as an ordinary `lower_expr` value (supporting e.g. parameter references), never literally constant-folded — so it was leftover-strict scaffolding rather than a real requirement. The laplace arm was split off from `zi_*` (which keeps the original behavior, unaffected since it's still `is_unsupported` and never reaches this code in practice) and changed to only force-const the *optional trailing* tolerance/nature argument, validating `args[0..3]` (input, num, den) normally — which is what allows an ordinary (non-const) array-variable reference, and its `coeffs[i]` element reads, to pass body validation at all.
 3. **openvaf/hir/src/body.rs** + **openvaf/hir_lower/src/expr.rs** — surface the resolved **VarIds** to the lowering pass. `BodyRef` gained `array_var_ref(&self, expr) -> Option<Vec<Variable>>`, projecting `InferenceResult::array_var_refs` into the hir crate's public `Variable` wrapper type (the lowest layer that already has one, since `hir_ty` doesn't depend on it). `lower_laplace_array_arg` (renamed from the old array-literal-only `array_elems`) checks this first: if present, each element is read directly via the existing `ctx.read_variable(var)` (the same primitive an ordinary `coeffs[i]` read anywhere else in the model already uses) — no `ExprIds`, no `lower_expr` call, since there's no array-literal syntax node to walk for this case at all. Otherwise it falls back to the original array-literal-element path unchanged.

No changes were needed in `openvaf/hir_def` (item-tree/body lowering), `openvaf/mir*`, `openvaf/sim_back`, or `openvaf/osdi` — once the elements are `Values`, `laplace_roots_to_poly/laplace_`

state_space (§3.5) don't care whether they came from a literal or a variable.

20. Testing & verification The motivating example from §17 now compiles and simulates correctly. Two variants were checked:

- The exact mixed declaration above (`coeffs[0]=-2e6, coeffs[1]=0.0, laplace_zd(V(in), coeffs, '{3e12, 4e6, 1.0}')`) compiles with zero errors and simulates to $V(\text{out}) \approx 0$ at DC — correct, *given the literal contents written*: `coeffs` has two elements, so it's a two-root zero list $((s - (-2e6)) \cdot (s - 0) = s^2 + 2e6 \cdot s$, which is zero at $s=0$), not the single zero $(s+2e6)$ an inline comment in the source suggested. This is a property of what the model actually declares (an authoring mismatch between a 2-element array and a 1-root-numerator comment), not a compiler bug — confirmed by checking the equivalent literal-only form `laplace_zd(V(in), '{-2e6, 0.0}, ...)`, which produces the identical (correct, for *that* input) result.
- A corrected one-element version (`real [0:0] zero_coeffs; zero_coeffs[0] = -2e6;`, i.e. an actual single zero at $s=-2e6$) gives $V(\text{out}) = 1.3333e-6$ at $V(\text{in}) = 2.0$ — exactly $2.0 \cdot 2e6/3e12 = 2.0 \cdot 6.667e-7$, matching the originally-intended $H(s) = (s+2e6) / ((s+1e6)(s+3e6))$, $H(0) = 6.667e-7$.

Regression: `cargo test -p syntax -p hir_def -p hir_ty -p hir -p hir_lower` passes unchanged (0 failures, no snapshot updates needed this time). All Part 1/Part 2 fixtures (`laplace_lpf.va`, `laplace_variants.va`, `laplace_zd_only.va`, `array_var_fir.va`) and the bare-reference diagnostic *outside* a `laplace` call (`V(out) <+ coeffs * V(in);`, still correctly rejected) recompile/re-diagnose identically to before this addition.

21. Known limitations (additive to §16)

- Only a bare, whole-array reference is accepted as a `laplace_*` argument — `coeffs[1:3]` (part-select) or any expression more complex than a single identifier naming a known array variable is not recognized as the “array variable” shape and falls through to ordinary scalar inference (producing the standard “expected array” type mismatch rather than a tailored diagnostic).
- Mixing is still one-or-the-other per argument, not per-element — you can write `laplace_zd(in, coeffs, '{...}')` (one arg a variable, the other a literal, as in §17/§20) but not splice a variable element into the middle of a literal's element list directly; for that, index the variable explicitly inside the literal instead (`'{coeffs[0], 1.5, coeffs[2]}'` — already supported, since each literal element is just an ordinary expression, see §2.3).

Part 4: Fixed a large-integer-literal crash, verified with a real 5th-order Bessel filter

22. Motivation To exercise `laplace_nd` with a non-toy example, a genuine 5th-order analog Bessel low-pass filter was built: `scipy.signal.bessel(N=5, ...)` designs the filter, and the *exact same* coefficients (not hand-derived, avoiding any transcription mismatch) are written into a `laplace_nd` call, so the `ngspice/OpenVAF` simulation can be cross-checked against an independent analytical computation of the identical transfer function.

Some of a 5th-order Bessel filter's coefficients are large (e.g. 6134876650875544) and, written as bare digit strings with no `./exponent`, crashed the compiler outright:

```
Panic occurred in file 'openvaf/syntax/src/ast/expr_ext.rs' at line 340
called `Result::unwrap()` on an `Err` value: ParseIntError { kind: PosOverflow }
```

23. Root cause and fix `ast::IntNumber::value()` parsed the literal's text directly as `i32` and `unwrap()`ed the result — Verilog-A's integer type is 32-bit in this compiler (`hir_def::expr::Literal::Int(i32)`), used consistently across ~10 call sites: bit-select indices, bus/array widths, MIR's

iconst), so that part is by design. The bug is that any bare integer-shaped literal exceeding i32 range crashed the whole compiler, even when used in a Real-typed context (like a `laplace_nd` coefficient) where it isn't semantically an integer at all — it's just a real number the user didn't bother giving a decimal point.

Two designs were considered:

- Promote to i64. Rejected: `Literal::Int` is i32 everywhere downstream (bit-select indices, bus width folding, MIR's `iconst`), so this would only move the crash threshold from $\sim 2.1e9$ to $\sim 9.2e18$ — and the Bessel filter's own leading coefficient ($9.79e18$) already exceeds `i64::MAX`, so it would still crash. Widening `Literal::Int` itself to i64 to fully fix it would ripple through all ~ 10 call sites for a problem that isn't really about integer width at all.
- Fall back to a float literal on overflow (chosen). A bare digit string is always valid f64 syntax too, and f64's range ($\sim 1e308$) covers any realistic literal. This fixes the crash for literals of any size, not just those between i32 and `i64::MAX`.

Implemented at two call sites:

- `openvaf/syntax/src/ast/expr_ext.rs`: `IntNumber::value()` now returns `Option<i32>` (None on overflow, instead of panicking); new `IntNumber::value_as_f64()` is the fallback parse.
- `openvaf/hir_def/src/body/lower.rs` (`Literal::new`, the actual panic site): on None, produces `Literal::Float` instead of `Literal::Int`.
- The same `IntNumber::value()` call inside `expr_ext.rs`'s `as_constexprval` (used by bus-width/parameter-constraint constant folding) was fixed the same way, falling back to `ConstExprValue::Float`.

Because every consumer that actually needs integer semantics (bit-select index, bus/array width) already only matches `Literal::Int/ConstExprValue::Int` specifically, a `Literal::Float` for an oversized literal used in one of those positions automatically falls through to the existing "not a constant integer" diagnostics (`NonConstantBitSelectIndex`, `NonConstantBusWidth`) — no new diagnostic code was needed, and it's arguably the *correct* diagnosis anyway (nothing has 6×10^{15} bits).

24. Testing & verification

- The exact literals that crashed before now compile and simulate correctly (verified with a standalone fixture reproducing the crash case, giving the expected DC gain).
- A huge literal used where an integer is actually required still degrades gracefully to the pre-existing diagnostics, confirmed for both a bit-select index and a bus/array width — no panic, no behavior change from before this fix in the cases that were already erroring cleanly.
- New regression fixture `test_data/ui/huge_int_literal.va/.log`.
- `cargo test -p syntax -p hir_def -p hir_ty -p hir -p hir_lower` passes unchanged (10/10 ui tests, up from 9, with the new fixture).

25. The Bessel filter example `examples/bessel_filter_examples/`:

- `design_bessel.py` designs a 5th-order analog Bessel low-pass filter ($f_c = 1$ kHz) with `scipy.signal.bessel(N=5, Wn=2*pi*1000, analog=True, norm='phase')` and writes the exact coefficients into `bessel5.va`'s `laplace_nd(V(in), num, den)` call (confirmed via `--dump-mir`: exactly 5 LaplaceState implicit equations, matching the filter order).
- `compare_bessel.py` computes $H(j\omega)$ (AC) and the step response (transient) directly from the *same* b, a via `scipy.signal.freqs/ scipy.signal.step`, and overlays them against the `ngspice/OpenVAF` simulation (`ac_sim.cir/tran_sim.cir`).

Result: numerical-noise-level agreement — max AC gain error $5.6e-7$ dB, max AC phase error $7.2e-7^\circ$, max step-response error $6.6e-6$ V. The AC plot shows the classic Bessel shape (gentle magnitude rolloff,

near-linear phase out to -450° across 5 poles); the step response shows the characteristic small ($\sim 1\%$), non-ringing overshoot Bessel filters are chosen for.

26. Diff summary (additive to Parts 1-3's tables)

File	Kind of change
openvaf/syntax/src/ ast/expr_ext.rs	IntNumber::value() returns Option<i32> instead of panicking; new value_as_f64(); as_constexprval falls back to ConstExprValue::Float
openvaf/hir_def/ src/body/lower.rs	Literal::new falls back to Literal::Float on i32 overflow instead of panicking
openvaf/test_data/ ui/huge_int_ literal.va/.log	new regression fixture

No changes to openvaf/hir_ty, openvaf/hir_lower, openvaf/mir*, openvaf/sim_back, or openvaf/osdi — this was purely a parsing/lowering robustness fix, unrelated to the Laplace realization itself.

Enhancement-5 — Verilog-A Module Instantiation in OpenVAF

This document describes every source-code change made to OpenVAF in the `version6/` directory, on top of `version5/` (Enhancement-4, Laplace-domain transfer-function operators), to implement Verilog-A module instantiation — one module placing other modules as sub-circuit elements on its own nets, e.g.:

```
module resistor(p, n);
    inout p, n;
    electrical p, n;
    parameter real r = 1000;
    analog begin
        I(p, n) <+ V(p, n) / r;
    end
endmodule

module divider(in, out, gnd);
    inout in, out, gnd;
    electrical in, out, gnd;
    resistor #(.r(1e3)) r1(in, out);
    resistor #(2e3) r2(.p(out), .n(gnd));
    resistor rarr[0:1](out, gnd);
endmodule
```

Full feature scope, confirmed up front with the user ("full support and implementation", not a cut-down v1):

- Both positional (`resistor r1(in, out);`) and named (`resistor r1(.p(in), .n(out));`) port connections, including open ports (`inst(a, , c);` — a blank slot leaves that port internally floating).
- Both positional (`#(1e3, 2e3)`) and named (`#(.r(1e3))`) parameter overrides.
- Instance arrays (`resistor rarr[0:3](out, gnd);`), expanding to N independent instances at compile time.
- Arbitrary nesting depth (a module instantiating a module instantiating a module, ...).
- Cyclic instantiation is a hard compile error, diagnosed by name (not a stack overflow).

1. What was actually there already (and what wasn't)

Unlike Enhancement-4's `laplace_*` operators (front-end-complete but lowering-incomplete), module instantiation had zero support anywhere in the compiler before this enhancement:

- The grammar's `ModuleItem` production (`openvaf/syntax/verilog.ugram`) had exactly 7 variants (`BodyPortDecl`, `NetDecl`, `AnalogBehaviour`, `Function`, `BranchDecl`, `VarDecl`, `ParamDecl`, `AliasParam`) — no instantiation form, and no parser code path for it either.
- `hir::ScopeDef::ModuleInstance(Module)` existed but was dead code from a user's perspective: it's only ever produced for top-level module declarations (`ScopeDefItem::ModuleId` in the root scope), never for an actual instantiation statement, because no such statement could be parsed.
- Everything downstream of `hir_def` — `hir_ty`, `hir_lower`, `mir*`, `sim_back::DaeSystem`, `osdi::OsdiDescriptor` — is architected around exactly one flat module per compiled artifact: per-module-local node IDs (`LocalNodeId = Idx<Node>`), one `DaeSystem` built per module, one flat node/parameter/Jacobian table per OSDI descriptor. There is no composition/nesting primitive anywhere in that stack, and building one would mean risky, invasive surgery through the entire backend.
- No TODOs, comments, or partial scaffolding anywhere suggested hierarchy was ever planned.

1.1 Why this isn't solved with a CST/HIR-level splice The first design instinct — clone the sub-module's parsed syntax subtree, rename its identifiers, and splice it into the parent's tree before `hir_def` runs — does not work with this codebase's architecture. Every layer that consumes syntax (basedb::AstIdMap, hir_def::ItemTree, hir_def::body::Body::body_with_sourcemap_query, lint/diagnostic collection) independently re-derives its input from `db.parse(root_file)`, keyed purely by `FileId`, using `(TextRange, SyntaxKind)` pointers into *that one real parse*. There is no seam to hand a mutated/spliced tree to all of those consumers at once — a change made at one layer (e.g. after parsing) is invisible to every other layer, which independently re-parses the *original* file text.

The only mechanism consistent with this architecture is text-level elaboration: synthesize a fully flattened Verilog-A source for any module that contains instantiation statements — inlining each instantiated module's own declarations, alpha-renamed per instance, with ports bound to the caller's net expressions and parameters bound to the caller's override expressions — register that synthesized text as a new file, and feed *that* through the entirely unmodified `parse` → `hir_def` → `hir_ty` → `hir_lower` → `mir*` → `sim_back` → `osdi` pipeline. This is conceptually the same trick SPICE simulators use for `.subckt`/X-instance expansion.

Part 1: Grammar and parser (**openvaf/syntax, openvaf/parser, sourcegen**)

2. New grammar productions `openvaf/syntax/veriloga.ungram` gained a `ModuleItem` variant and five supporting rules:

```
Instantiation =
  AttrList* module:NameRef ParamOverrides?
  (InstanceUnit (',' InstanceUnit)*) ';'

ParamOverrides =
  '#' '(' (ParamAssign (',' ParamAssign)*)? ')'

ParamAssign =
  ('.' name:Name '(' val:Expr ')') | val:Expr

InstanceUnit =
  name:Name width:Range? PortConns

PortConns =
  '(' (PortConn (',' PortConn)*)? ')'

PortConn =
  ('.' name:Name '(' net:Expr ')') | net:Expr
```

`Range` (the `[msb:lsb]` clause) is reused verbatim from Enhancement-3/4's vectored-net support for instance-array ranges. `PortConn`/`ParamAssign` each produce a single AST node with two independently-optional fields (name, net/val) rather than a named/positional enum — a bare `PortConn` with neither field present represents an open/unconnected port (an empty slot between commas, e.g. `inst(a, , c)`).

New `SyntaxKinds` (`INSTANTIATION`, `PARAM_OVERRIDES`, `PARAM_ASSIGN`, `INSTANCE_UNIT`, `PORT_CONNS`, `PORT_CONN`) were registered in `sourcegen/src/ast/src.rs`'s `KindsSrc::nodes` list; running `cargo test -p sourcegen ast` regenerated `openvaf/tokens/src/parser/generated.rs` and `openvaf/syntax/src/ast/generated/{nodes,tokens}.rs` from the updated `ungram` (standard workflow in this codebase — the test writes the files and fails once, then passes on the immediate re-run).

3. Parser disambiguation `openvaf/parser/src/grammar/items/module.rs`'s `module_items()` dispatch previously sent every bare IDENT unconditionally to `net_decl::<false>` (a discipline-name-first net declaration, e.g. `electrical p, n;`). Module instantiation (`resistor r1(in, out);`) also starts with a bare IDENT, so a 2-token lookahead disambiguates the two, added as a new `is_instantiation` guard before the existing IDENT arm:

- # immediately after the first identifier is an unambiguous instantiation signal (`resistor #(...)` `r1(...)`) — net declarations never contain #.
- Otherwise: a second IDENT followed by `(` or `[` (instance ports, or an instance-array range) means instantiation; followed by `,`/`;` means an ordinary (possibly single-net) net declaration.

This is exact and requires no backtracking. `instantiation()`, `param_assign()`, `instance_unit()`, `port_conns()`, and `port_conn()` were added to `module.rs` mirroring the existing `branch_decl/net_decl` parsing style.

Part 2: `hir_def` item tree, name resolution, diagnostics

4. Item tree representation `openvaf/hir_def/src/item_tree.rs` gained `ModuleItem::Instantiation` and a new `Instantiation` item-tree node:

```
pub struct Instantiation {
    pub name: Name,           // instance name (or synthesized "r[2]" for an array
    ↪ element)
    pub unit_idx: usize,      // which comma-separated InstanceUnit this is
    pub array_index: Option<i32>,
    pub module: Name,         // referenced module name, unresolved
    pub ast_id: AstId<ast::Instantiation>,
}
```

One `Instantiation` entry is created per array element (mirroring how `Net/Var/Param` handle comma-separated declarations — all names from one source statement share an `ast_id`, disambiguated by index), reusing `bus_bit_name/fold_width_range` from Enhancement-4's bus-range folding for the `[msb:lsb]` instance-array expansion. Deliberately not captured here: port/parameter binding details — those are read straight from the AST (via `ast_id`) by name resolution and by the elaboration pass, exactly like how `Net/Port` only cache their discipline in the item tree and leave everything else to the AST.

5. Name resolution (`openvaf/hir_def/src/namer/collect.rs`) Module instantiation needed the collector to change from a single forward pass to two passes: originally, `collect_root_map()` called `collect_module()` once per module in file order, which both registered the module's name in the root scope *and* processed its internal items in the same step. A module instantiating another module declared later in the same file would then fail to resolve, purely because of a body/root collector ordering, not a Verilog-A semantic. Fixed by splitting `collect_module` into `predeclare_module` (registers every module's name and opens its scope) and `collect_module_items` (processes internal items, including instantiation resolution), and running all predeclarations before any item collection.

Each instantiation is resolved against the (now fully predeclared) root scope, producing a new `ScopeDefItem::InstantiationId(InstantiationId)` — deliberately not reusing `ScopeDef::ModuleInstance`, which represents a module *definition*; an instantiation is a *use*. `InstantiationId/InstantiationLoc` follow the exact `impl_intern!` pattern already used for `BranchId/FunctionId/etc`.

6. New diagnostics A DFS with a 3-color visiting-set over the file's instantiation graph detects cycles (`DefDiagnostic::CyclicInstantiation`) before any per-site resolution runs, so a self-/mutually-

recursive instantiation is a clean compile error rather than infinite recursion. Per-instantiation-site diagnostics (checked once per distinct `InstanceUnit`, not once per array element, to avoid N duplicate reports for an N-wide array) resolve against the target module's own item tree, no flattening required:

- `UnknownInstantiatedModule` — referenced module doesn't exist in the file.
- `InstancePortCountMismatch` — positional port list length $\neq$ target's port count.
- `UnknownInstancePort` — a named `.port(net)` names a port the target doesn't have.
- `UnknownInstanceParam` — a named `.param(value)` names a parameter the target doesn't have.
- `TooManyInstanceParams` — more positional overrides than the target has parameters.

All five are `hir_def::namer::diagnostics::DefDiagnostic` variants (the existing channel used for `AlreadyDeclared`), rendered through the same `Diagnostic/Report` machinery as every other compiler error.

Part 3: Elaboration — the text-flattening pass (`openvaf/hir/src/elaborate.rs`)

7. Where it runs `elaborate_instantiations(db: &mut CompilationDB)` runs once, as ordinary Rust code, at the very end of `CompilationDB::new` (`openvaf/hir/src/db.rs`) — not as a salsa query hooked into the preprocess/parse chain (that was the original plan; direct inspection of `AstIdMap/item_tree/body_with_sourcemap_query`, which all independently call `db.parse(root_file)`, showed there's no query-graph seam to intercept — see §1.1). Instead:

1. Cheap fast path: `db.item_tree(root_file)` is checked for any `ModuleItem::Instantiation` anywhere in the file; if none, return immediately. This keeps the pass a true no-op (one item-tree lookup) for the overwhelming majority of `.va` files, which never use the feature.
2. If `def_map.diagnostics` already contains a `CyclicInstantiation` diagnostic, skip elaboration entirely (a cyclic graph can't be flattened — recursing would never terminate) and let the normal diagnostic-printing path surface the cycle against the *original* file.
3. Otherwise, walk the root file's already-parsed `SourceFile::items()`; for each `ModuleDecl`, `flatten_top_level_module` produces its final text (verbatim, unless it directly contains an instantiation).
4. The concatenated result is registered as a new virtual file via `Vfs::add_virt_file`, the original file's `include_dirs/macro_flags/global_lint_overwrites` salsa inputs are copied onto the new `FileId` (they're per-`FileId` inputs, so the synthetic file needs its own copies to behave identically to the real one), and `CompilationDB::root_file` is redirected to it via a new `pub(crate) fn set_root_file`.

Everything from `hir_def::item_tree` onward — `hir_ty`, `hir_lower`, `mir*`, `sim_back`, `osdi`, `mir_llvm` — required zero changes: they simply compile what looks like an ordinary, hand-written flat module.

8. How flattening actually works `render_with_holes(text, holes, scope)` is the one primitive everything else is built from: it tokenizes text (via the existing lexer crate), rewrites `SimpleIdent` tokens using an exact-match `scope.subst: HashMap<String, String>`, and — for each `(byte_range, replacement)` in `holes` — emits the replacement verbatim instead of individually inspecting the tokens inside that range. That last property is what makes composing renamed scopes correct: an already-fully-resolved nested instance's text, or an override expression written in a caller's scope, is never accidentally re-renamed using the wrong module's `subst` map, because the tokenizer never looks inside a hole.

`scope: Scope` bundles two things: `subst` (plain whole-identifier renames) and `bus_ports: HashMap<Name, BTreeMap<i32, String>>` (bus-typed ports — see below). `render_with_holes` always starts by computing bus-port holes via `find_bus_port_holes` and merging them with whatever holes the caller already had, before doing the token pass.

Rendering one instantiated module (`render_instance_content`) builds a Scope covering every name the target module itself declares:

- Scalar ports are bound to the caller's connected net text (already resolved at the *caller's* level — see "resolve, then recurse" below), or, if left open, get a fresh internal net (`{prefix}open__{port}`, redeclared with the port's own discipline) — inserted into `scope.subst`.
- Bus-typed ports need a fundamentally different mechanism: an ordinary rename ($p \rightarrow \text{prefix_}p$) is one whole-token substitution, but a bus port's bit 0 and bit 1 need to become *different* identities, and the source text only ever contains the bare base identifier with the bit-select as separate tokens right after it — there's no single token to hang a per-bit answer on. These go into `scope.bus_ports[base]` instead: a per-bit map, each bit either the caller's binding (itself possibly *sliced* from a caller-scope bus of matching width — see below) or a freshly declared net if left open. `find_bus_port_holes` then scans for ident '[' int_literal ']' token sequences matching a `bus_ports` base name and replaces the whole sequence as one hole.
- Internal nets/vars/branches/functions/alias-params get a unique per-instance prefix (`{prefix}{name}`) in `scope.subst`.
- Internal bus/array nets and variables (Enhancement-3/4) are renamed by their *base* name only ($\text{bus} \rightarrow \{\text{prefix}\}\text{bus}$) in `scope.subst` — `bus[3]`-style bit-select syntax keeps the base identifier as its own token, so this one substitution correctly renames every bit without needing per-bit entries (this is the mechanism internal buses use, in contrast to bus *ports*, which can't use it — see above).
- Parameters are renamed like other locals; an override (if supplied) replaces only the *default-value* sub-expression inside the parameter's own (renamed) declaration — resolved once at declaration time, not patched at every reference site.

The target module's own port-direction (`inout p, n;`) and discipline (`electrical p, n;`, including a vectored `electrical [3:0] p;`) declarations are dropped, not merely renamed, when inlining an instance — a port's identity now refers to an already-declared net from an outer scope (or a freshly declared one), so re-declaring it inside the inlined body would collide with that outer declaration. `NetDecl/BodyPortDecl` items get filtered per-*base*-name (a single `electrical` line may legally mix port and non-port names/buses).

Resolve, then recurse: an instantiation's port/parameter argument expressions are written in the *instantiating* module's scope. Each recursive call receives already-fully-resolved (renamed) bindings — the resolution (`apply_rename/resolve_port_bindings` against the parent's own Scope) happens once, in the parent, immediately before recursing, never inside the child. This is what lets `#(.r(r_base*2))` (an override expression referencing the *parent's own* parameter) and arbitrarily deep nesting compose correctly without a second rename pass ever touching already-resolved text.

Bus/array slicing (`find_matching_caller_bus`) is the one heuristic layered on top of "resolve, then recurse": a port actual, or an instance-array port actual, is sliced per-bit/per-element only when it's a *bare identifier* naming a bus, in the instantiating module's own scope, whose bit width exactly matches what needs slicing:

- A bus-typed target port (`bus_resistor br(a, b);`, where `bus_resistor` declares input `[1:0] p;` `electrical [1:0] p;`) slices caller bus `a` bit `i` onto target port `p` bit `i` (`bind_port`, aligned by relative position, not absolute index — so a caller bus `[7:4]` correctly maps onto a target bus `[3:0]`).
- An instance array (`resistor rarr[0:3](p, gnd);`) slices a matching-width caller bus `p` onto array element `i`'s port (`expand_instantiation`'s per-element loop) — this is the array-level analogue of the same idea, reusing the identical `find_matching_caller_bus` width/name check.

Anything that doesn't match this heuristic (a plain scalar net, a mismatched width, a non-trivial expression) falls back to plain `bind/broadcast` — e.g. `resistor rarr[0:3](out, gnd);` where `out` is an ordinary scalar net still connects every array element to the same `out/gnd`, exactly like the original (pre-slicing) behavior.

Part 4: Verification

9. End-to-end result `examples/instantiation_examples/resistor_divider.va` exercises the full feature set in one file: nested instantiation (`divider` $\rightarrow$ `buffer` $\rightarrow$ `resistor`), named and positional ports, named and positional parameter overrides, and an instance array.

- MIR/DAE structural check: building `divider`'s `CompiledModule` produces a `DaeSystem` with exactly 3 unknowns (`in`, `out`, `gnd` — the module's 3 real ports; no spurious extra unknowns, confirming port binding merges node identities rather than adding equations) and 7 Jacobian entries, consistent with two parallel-resistor groups cross-coupling those 3 nodes.
- Real OSDI + ngspice DC sweep: `openvaf-r resistor_divider.va -o resistor_divider.osdi` (built with `--features llvm18` against a Homebrew LLVM 18 toolchain) produces a working `.osdi` shared library; loaded into ngspice (`examples/instantiation_examples/dc_sim.cir`), a `.dc Vin -2 2 0.5` sweep is cross-checked in `examples/instantiation_examples/compare_divider.py` against the analytically combined resistor network (`buffer` $\parallel$ `r1` between `in/out`; `r2` $\parallel$ `rarr[0]` $\parallel$ `rarr[1]` between `out/gnd`) — matches to $\sim 1e-9$, i.e. solver precision.
- Bus-port/array-slicing check (`openvaf/test_data/ui/instantiation_bus.va`): a module with a 2-bit bus port (`bus_resistor`, ports `p/n` each `[1:0]`) instantiated as `bus_resistor br(a, b);` where `a/b` are 2-bit buses in the caller, plus resistor `rarr[0:1](a, c);` (per-element array slicing onto the same bus `a`) and resistor `rbroadcast[0:1](c, d);` (plain scalars — confirms broadcast still applies where slicing doesn't). Compiled to a real `.osdi` and driven with ideal DC sources at `a[0]=1V/a[1]=2V`: with nothing else loading `b[0]/b[1]`, the zero-current KCL condition through `br`'s resistors forces `b[0]=a[0]` and `b[1]=a[1]` *exactly* — confirmed by ngspice (`v(b[0])=1.0, v(b[1])=2.0`), verifying the bus port bound bit `i` to bit `i`, not some other pairing.
- Regression: every pre-existing `.va` example from Enhancements 2-4 (`laplace_examples`, `bus_examples`, `array_var_examples`, `bessel_filter_examples`, `combined_examples`, `absdelay_examples`, `indirect_assignment_examples`), plus `resistor_divider.va`, still compile unchanged through the pipeline after the bus-port/slicing work. The full fast test suite (`cargo test --workspace`, minus two pre-existing failures confirmed to already fail identically and unmodified in `version5` — a stale MIR-value-numbering snapshot in `sim_back's dae/init/topology` unit tests, and a header-parsing quirk in `sourcegen's osdi::gen_osdi_structs` — neither touched by this enhancement) passes with zero new failures.
- Cross-``include` instantiation (`examples/instantiation_examples/resistor_divider_include.va + resistor_lib.va`): a module can instantiate a target declared in a *different* file, pulled in via ``include`, with no special-casing anywhere in the elaboration code. This falls directly out of where the pass sits in the pipeline (§7): ``include` is a preprocessor directive, resolved by the preprocessor crate *before* `db.parse` ever runs, so by the time elaboration inspects `parse.tree().items()`, the included file's modules are already merged into the same single parse tree as the including file's own modules — indistinguishable from a same-file declaration. The flattened output (and every DC/AC/transient result) is byte-for-byte/numerically identical to the equivalent same-file version.

New fixtures:

- `openvaf/test_data/item_tree/instantiation.{va,item_tree,def_map}` — positional/named ports, positional/named params, and an instance array, checked at the item-tree and name-resolution level.
- `openvaf/test_data/ui/instantiation_errors.{va,log}` — exercises all four new diagnostics in one file (unknown module, port-count mismatch, unknown param override, mutually-

recursive cycle).

- `openvaf/test_data/ui/instantiation_ok.{va,log}` — the nested/positional/ named/array-instance case, asserting zero diagnostics.
 - `openvaf/test_data/ui/instantiation_bus.va` — bus-typed ports and per-element array slicing (both the sliced and the broadcast case in one file), asserting zero diagnostics.
-

10. Known limitations

- No span provenance into inlined code. Diagnostics for code that originated inside an instantiated module point at the synthesized flattened file (`<name>__elaborated.va`), not the original module's source location. The codebase's `preprocessor::sourcemap` machinery (`CtxSpan/SourceContext`) solves exactly this problem for ``include/macro` expansion, but hooking into it would have meant operating at the token-stream level inside the `preprocess salsa` query — which, per §1.1/§7, isn't where this pass can run. Diagnostics are still fully readable, just not pointing at the file the user actually wrote for inlined content.
- Bus-typed ports and per-element instance-array slicing are supported, via one heuristic (`find_matching_caller_bus`, `openvaf/hir/src/ elaborate.rs`): a port actual is sliced only when it is a *bare identifier* naming a bus, in the instantiating module's own scope, whose bit width exactly matches what needs slicing — the target bus port's width (`bus_resistor br(a, b);`, wiring bit `i` of port `p/n` to bit `i` of caller-scope buses `a/b`), or the instance array's element count (`resistor rarr[0:3](p, gnd);` where `p` is itself a matching-width bus, wiring element `i` to `p[i]`). Anything else (a non-bus net, a mismatched width, a non-trivial expression) falls back to plain `bind/broadcast`, matching the original (v1) semantics — so `resistor rarr[0:1](out, gnd);` where `out` is an ordinary scalar net still connects every array element to the same `out/gnd`, as before. A bus port needed a different substitution mechanism than everything else in this pass, since a plain identifier-`rename` can't express "bit 0 and bit 1 become different identities" — `find_bus_port_holes` scans for `[' int_literal ']` token sequences matching a bus port's base name and replaces the whole sequence as one hole, reusing the existing hole-splicing machinery rather than teaching the core renamer a new substitution kind. Combining both at once (a bus-typed port *on* an array instance, e.g. `bus_resistor barr[0:3](p, q);`) is not sliced on the array dimension — only the per-bit port dimension is resolved.
- Escaped identifiers (`\foo`) inside an instantiated module aren't renamed by the token-level substitution (only `SimpleIdent` tokens are matched) — a very rare Verilog-A construct in practice.
- A module that is instantiated by another also continues to be compiled as its own standalone top-level OSDI descriptor if it appears at the top level of the file, unchanged from today's `collect_modules` behavior (every top-level module is always compiled). This is a real, useful capability worth calling out explicitly: a single `.osdi` file already exposes *every* top-level module in the source `.va` file as an independently `.model`-able SPICE device, so a netlist can compose two *separately compiled* OSDI descriptors from the same file (ordinary SPICE-level hierarchy) alongside Verilog-A-level `instantiate` (compile-time inlining within one descriptor) — two complementary mechanisms, not competing ones. See `examples/instantiation_examples/dc_sim_multi_model.cir` for a worked example (chaining standalone buffer and divider instances from `resistor_divider.osdi` in one netlist) — including a naming-collision gotcha (`resistor` collides with ngspice's own built-in `resistor .model` keyword, unrelated to OpenVAF/OSDI).

11. Diff summary

File	Kind of change
openvaf/syntax/veriloga.ungram	New Instantiation/ParamOverrides/ParamAssign/InstanceU
sourcegen/src/ast/src.rs	grammar rules Registers the 6 new SyntaxKinds consumed by codegen
openvaf/tokens/src/parser/generated.rs, openvaf/syntax/src/ast/generated/{nodes, tokens}.rs	Regenerated via cargo test -p sourcegen ast
openvaf/parser/src/grammar/items/module. rs	2-token lookahead disambiguation + instantiation/param/port parsing
openvaf/hir_def/src/item_tree.rs	Instantiation item-tree node, ModuleItem::Instantiation, NonConstantInstanceArrayWidth diagnostic
openvaf/hir_def/src/item_tree/{lower, diagnostics,pretty}.rs	Lowering, diagnostic rendering, pretty- printing for the new node
openvaf/hir_def/src/lib.rs	InstantiationId/InstantiationLoc interning
openvaf/hir_def/src/db.rs	New intern_instantiation salsa query
openvaf/hir_def/src/nameres.rs	ScopeDefItem::InstantiationId variant + all exhaustive-match sites
openvaf/hir_def/src/nameres/collect.rs	Two-pass module collection (predeclare, then collect items); instantiation resolution + 5 new diagnostics
openvaf/hir_def/src/nameres/diagnostics. rs	5 new DefDiagnostic variants + Report rendering, plus ReservedModuleName (§12)
openvaf/hir_ty/src/inference.rs	ScopeDefItem::InstantiationId treated as Ty::Scope (matches ModuleId/BlockId)
openvaf/hir/src/lib.rs	ScopeDefItem::InstantiationId in the "implementation detail" bucket of declarations()
openvaf/hir/src/db.rs	pub(crate) fn set_root_file; calls elaborate::elaborate_instantiations at the end of CompilationDB::new
openvaf/hir/src/elaborate.rs	New — the text-flattening elaboration pass (§7-8)
openvaf/hir/Cargo.toml	Adds lexer/tokens path dependencies (for token-level rename)
openvaf/basedb/src/lints.rs	New reserved_module_name lint (§12)
openvaf/test_data/item_tree/ instantiation.*, openvaf/test_data/ui/ instantiation.*, openvaf/test_data/ui/ reserved_module_name.*	New regression fixtures
openvaf/test_data/ui/{ddx, formatting, function}.log	Updated (UPDATE_EXPECT=1) — pre-existing fixtures name their module diode, now correctly flagged by §12's new lint
examples/instantiation_examples/	New end-to-end examples + ngspice cross- checks (§9, §11)

No changes to openvaf/mir*, openvaf/sim_back, openvaf/osdi, or openvaf/mir_llvm — by design,

per §1.1/§7.

12. `reserved_module_name` lint

The resistor/ngspice-built-in collision found while building §11's multi-model example turned out to be worth catching at *compile time*, not just documenting: a new lint, `reserved_module_name` (openvaf/basedb/src/lints.rs, doc id 18, warn by default), fires when a top-level module's name matches — case-insensitively — one of ngspice's built-in native SPICE device names (`RESERVED_NGSPICE_DEVICE_NAMES` in openvaf/hir_def/src/names/collect.rs, collected from every `.name = "..."` in ngspice-46/src/spicelib/devices/**/*.c; ~55 names — Resistor, Capacitor, Diode, BSIM4, Mos1, etc.). This was a natural fit for OpenVAF's existing lint framework (basedb::lints, same machinery `port_without_direction/trivial_probe` already use): fully suppressible (`-A reserved_module_name`) and promotable to a hard error (`-E reserved_module_name`) via the standard CLI flags, with no special-casing needed beyond registering the lint and implementing `Diagnostic::lint()` on the new `DefDiagnostic::ReservedModuleName` variant (openvaf/hir_def/src/names/diagnostics.rs) to return it.

The check runs once per module in `predeclare_module` (openvaf/hir_def/src/names/collect.rs) — the same place module names are first registered into the root scope — so it needs nothing beyond the name already available there, and fires for every top-level module regardless of whether it's ever instantiated.

Verified: `.model resmodel resistor (r=500)` in ngspice is empirically confirmed to silently fail to bind to the OSDI resistor module (falls through to ngspice's own built-in resistor parsing instead, producing an "unrecognized parameter" warning and wrong circuit behavior) — this isn't a hypothetical. Running the lint against the existing test suite surfaced the same collision in three *pre-existing*, unrelated fixtures that happened to name their module `diode` (openvaf/test_data/ui/{ddx, formatting, function}.va) and in this enhancement's own resistor-named examples — all expected `.log` snapshots were updated to include the new warning (`UPDATE_EXPECT=1`), zero other regressions.

Enhancement-6 — Missing Basic Verilog-A Features in OpenVAF-r

This document describes every source-code change made to OpenVAF-r in the `version7/` directory, on top of `version6/` (Enhancement-5, module instantiation), following a deep gap analysis of the compiler against the Verilog-A/Verilog-AMS LRM across five categories: operators, scale factors, keywords, math functions, and compiler directives.

0. Gap analysis summary

A systematic audit (five parallel research passes over the OpenVAF-r source) found:

Category	Verdict
Scale factors (T, G, M, K/k, m, u, n, p, f, a)	Complete — all 10 correct.
Math functions (ln, exp, sqrt, sin, ..., hypot, limit, clog2)	Complete.
Operators	Essentially complete for the analog-only LRM subset (<code>**</code> , <code>^~/~^</code> , <code><</>></code> , <code><+ all present</code>). <code>===/!==</code> , <code>{ }</code> concat/replication, reduction ops, <code>++/--</code> , compound assign are digital/SystemVerilog constructs with no semantic in a pure-analog compiler — correctly out of scope. <code><<</>>></code> were lexed but dead — fixed in this enhancement (§2).
Keywords	<code>generate/genvar</code> blocks entirely unimplemented (out of scope here — comparable in size to a whole enhancement on its own). <code>cross/above/timer</code> missing entirely (deferred — see §6). <code>zi_*</code> , <code>last_crossing</code> , <code>slew</code> , <code>transition</code> were parsed/type-checked but explicitly stubbed as unsupported — implemented in this enhancement (§4-5).
Compiler directives	Only 9 of ~17 standard directives worked; everything else hard-failed compilation with <code>MacroNotFound</code> instead of being ignored — fixed in this enhancement (§1).

Sections below cover what was actually implemented, in the order it was done.

1. Compiler directives (**openvaf/preprocessor**)

Real foundry/vendor `.va` files routinely carry boilerplate directives that this compiler previously treated as undefined macro calls, hard-failing the whole compilation. Added recognition (and correct token-consumption) for:

```
`undefineall, `celldefine, `endcelldefine, `default_discipline, `default_nettype,
`unconnected_drive, `nounconnected_drive, `timescale, `line, `pragma.
```

1.1 Design

- `openvaf/preprocessor/src/parser.rs`: new `CompilerDirective` enum variants, matched by literal text in `compiler_directive()`.
- Two new Parser helpers:
 - `skip_rest_of_line(&mut self, err)` — for directives whose argument grammar isn't worth parsing precisely (``timescale`, ``line`, ``pragma`, ``unconnected_drive`, ``default_`

nettype): scans the raw source for the next `\n` and discards every token up to it. Robust regardless of how the argument text happens to tokenize, since Verilog/Verilog-AMS directives with inline arguments are always terminated by end-of-line, not by `;`.

- `bump_directive_and_capture_ident(&mut self, err) -> Option<&str>` — for ``default_discipline` (captures the discipline name, then falls through to `skip_rest_of_line`).
- `openvaf/preprocessor/src/processor.rs`: Processor gained a `default_discipline: Option<&'a str>` field, set by the new directive but not yet wired into net-discipline resolution — doing that fully would mean threading it through `hir_def`'s name-resolution/discipline-inference code (every net currently requires an explicit discipline; see `hir_ty/src/validation.rs:699`, "no discipline for net"), which is a separate, larger feature. Recording it at least means the directive no longer breaks compilation.
- ``undefineall` clears `self.macros` for real.
- Everything else is a documented no-op (matches how real simulators treat directives they don't implement — ignorable, not fatal).

1.2 Tests `openvaf/preprocessor/src/tests.rs::misc_directives` exercises all ten new directives in one file, asserting zero diagnostics and that the expanded output contains only the real module body (golden file `test_data/misc_directives_expanded.va`).

2. `<<< / >>>` arithmetic shift operators

2.1 A pre-existing lexer bug While wiring these up, found that `openvaf/lexer/src/lib.rs`'s three-symbol match arms for `ShlA/ShrA` only called `self.bump()` once instead of twice, so `<<<` tokenized as a 2-character `ShlA` token followed by a stray `<` — a real, independent bug, not something introduced by this work:

```
// before (only consumes 2 of the 3 characters):
'<' if self.first() == '<' && self.second() == '<' => {
    self.bump();
    ShlA
}
// after:
'<' if self.first() == '<' && self.second() == '<' => {
    self.bump();
    self.bump();
    ShlA
}
```

Same fix applied to `ShrA`. Verified via a standalone lexer harness (`Literal / Whitespace / ShlA len=3 / ...` instead of `len=2 + stray Lt`).

2.2 Wiring

- `openvaf/parser/src/grammar/expressions.rs`: `<<</>>>` added to the Pratt-parser precedence table at the same tier as `<</>>`.
- `openvaf/syntax/src/ast/expr_ext.rs`: `new BinaryOp::ArithmeticLeftShift/ArithmeticRightShift` variants, `op_details()/Display` updated.
- `openvaf/syntax/veriloga.ungram`: `BinExpr` operator list updated (documentation; codegen for this node's accessors is hand-written, not ungram-generated).
- `sourcegen/src/mir_instructions.rs` + regenerated `openvaf/mir/src/instructions/generated.rs` / `openvaf/mir/src/builder/generated.rs`: new `Opcode::Iashr` (arithmetic

— sign-extending — right shift). `<<<` reuses `Opcode::Ishl`: per the LRM, arithmetic and logical left shift are identical (both always zero-fill); only right shift differs (logical zero-fills, arithmetic sign-extends).

- `openvaf/hir_ty/src/inference.rs`, `openvaf/hir_lower/src/expr.rs`: type inference and MIR lowering for the two new `BinaryOp` variants.
- `openvaf/mir_llvm/src/builder.rs`: `Opcode::Iashr` → `LLVMBuildAShr` (vs. `Opcode::Ishr` → `LLVMBuildLShr`).
- `openvaf/mir_interpret/src/lib.rs`, `openvaf/mir_opt/src/const_eval.rs`, `openvaf/mir_opt/src/simplify.rs`, `openvaf/mir_autodiff/src/lib.rs`, `openvaf/mir_autodiff/src/builder.rs`: the six other places that pattern-match `Opcode` exhaustively (or via `unreachable!()/matches!()`, which the compiler doesn't flag as non-exhaustive) needed the new variant added too — found by running actual `.va` files through each optimization pass, not just cargo check (several of these use wildcard `_ => unreachable!()` arms that compile fine but panic at runtime on an unhandled opcode).

2.3 Verification `4 <<< 1` constant-folds to 8 (same as `<<`); `-4 >>> 1` constant-folds to -2 (arithmetic sign-extending), confirmed via `--dump-mir` showing the correctly-folded `0x1.8p2 = 6.0` for `(4<<<1) + (-4>>>1)`.

3. `slew()` / `transition()` — compile-time rate-limited tracking loop

Previously lowered as a literal identity passthrough (`self.lower_expr(args[0])`) and gated behind `is_unsupported()` specifically to stop that silently-wrong stub from reaching users.

3.1 Numerical approach The LRM's `slew()` is an ideal (non-smooth, "bang-bang") rate limiter: track the input exactly whenever possible, else ramp at the bound. That's not directly expressible as a well-posed continuous residual — a naive $dy/dt = \text{clamp}(dx/dt, -\text{neg}, \text{pos})$ formulation leaves the DC operating point undetermined (any y satisfies $dy/dt = 0$ at DC). Instead:

$$dy/dt = \text{clamp}(K * (x - y), -\text{max_neg_rate}, \text{max_pos_rate}) \quad (K = 1e9, \text{ large})$$

a saturating tracking loop — well-posed at DC ($y = x$ uniquely, since K is large), and reproduces the rate-limited ramp whenever x moves faster than the bound allows, converging to the ideal limiter as $K \rightarrow \infty$. Implemented as an ordinary idt-style implicit equation (`ImplicitEquationKind::Slew(u32)`) — no simulator/OSDI changes needed, same category as `laplace_*`.

`transition(x, td, trise, tfall[, tol]) = slew(absdelay(x, td), 1/trise, 1/tfall)`: the delay stage reuses `absdelay`'s exact mechanism (factored out into a new shared `lower_delay` helper), the rate stage reuses the same tracking loop (`ImplicitEquationKind::Transition(u32)`). `trise/tfall` are transition *times* per the LRM; converting to a rate via $\text{rate} = 1/t$ assumes a unit-amplitude transition — exact for the common comparator-style 0/1 input, an approximation for arbitrary-amplitude inputs.

New code: `openvaf/hir_lower/src/expr.rs` (`lower_slew`, `lower_transition`, `lower_delay`, `lower_rate_limited_track`), new `ImplicitEquationKind::Slew/Transition` variants (`openvaf/hir_lower/src/lib.rs`), `hir/src/lib.rs` gained a pub use `hir_ty::types::Signature`; re-export (needed to name the signature type from `hir_lower`, which doesn't depend on `hir_ty` directly). `is_unsupported()` flag removed for both (`openvaf/hir_def/src/builtin.rs`).

3.2 Verification Real ngspice transient simulation (`slew(V(ctrl), 1e6, 2e6)` driven by a pulse source): measured rise rate over the ramp = exactly 1e6 V/s, fall rate = exactly 2e6 V/s (to 3+ significant figures), matching the specified bounds.

4. **zi_nd / zi_np / zi_zd / zi_zp** — bilinear-transform reuse of **laplace_***

Unlike `laplace_*`, a z-domain filter is inherently a sampled-data system — exact semantics need the simulator to hold the output between samples taken every T seconds, which needs dedicated per-timestep/ breakpoint support that doesn't exist in this codebase (and is a substantially different, simulator-runtime-level project — see §6).

Instead, this applies the standard bilinear (Tustin) transform, $z^{-1} = (1 - sT/2)/(1 + sT/2)$, to convert the z-domain transfer function into an equivalent continuous s-domain transfer function, then reuses the exact same `laplace_state_space` state-space realization already built for `laplace_*`. This exactly preserves the filter's pole/zero mapping and DC/near-DC behavior; it does not reproduce true zero-order-hold/aliasing behavior near the Nyquist rate ($1/T$) — a documented approximation, not full LRM fidelity.

4.1 Implementation `openvaf/hir_lower/src/expr.rs`:

- `lower_zi(kind, args)` — extracts num/den (roots or coefficients, same four-way nd/np/zd/zp convention as `laplace_*`, reusing `lower_laplace_array_arg/laplace_roots_to_poly` unchanged), then calls `bilinear_transform` on each and feeds the result into the existing `laplace_state_space`.
- `bilinear_transform(poly, n, half_t)` — for a degree- n z-domain polynomial $P(w)$ ($w = z^{-1}$, LRM's ascending-power convention), $P(w) * (1+x)^n = \sum_k p_k * (1-x)^k * (1+x)^{(n-k)}$, a degree- n polynomial in $x = s \cdot T/2$ whose coefficients are fixed, compile-time-known integer linear combinations of the p_k — the $(1-x)^k(1+x)^{(n-k)}$ binomial expansion depends only on $n/k/i$, not any runtime value, so those combination weights are plain f64 constants (`binomial_bilinear_weight`, computed via a multiplicative `binomial(n, k)` helper, no factorial overflow risk for realistic filter orders). The $x^i \rightarrow s^i$ conversion ($\times (T/2)^i$) uses runtime `Value` arithmetic since T may be an arbitrary expression, not just a literal.

4.2 Verification Two independent checks:

1. Hand-derived closed form: for `den=[1,-1]` (a z-domain pole at $z=1$, i.e. an ideal digital accumulator $H(z)=1/(1-z^{-1})$), the bilinear transform maps this to a pure continuous integrator (`den_s=[0,T]`) — an exact property of Tustin's method (a DC pole maps to a DC pole exactly). Derived by hand and matches what the code produces.
2. Real ngspice simulation: a first-order z-domain lowpass (`a1 = e^{(-T/\tau)}`, `b0 = 1-a1`, $\tau=10\mu\text{s}$, $T=1\mu\text{s}$) settled to the correct DC gain (1.0) with $\sim 61.4\%$ rise at $t=\tau$ (theoretical: 63.2% for an ideal RC — the small deviation is the expected/documented bilinear frequency-warping effect at a 10:1 $T:\tau$ ratio).

`is_unsupported()` flag removed for all four (`openvaf/hir_def/src/builtin.rs`).

5. **last_crossing(expr[, dir])** — OSDI ABI extension + ngspice-46 patch

Unlike `zi_*`, `last_crossing` genuinely needs the simulator's own accepted-timepoint history — the crossing time is a function of the whole past trajectory of `expr`, not derivable from its instantaneous value — so this does require extending the OSDI ABI and patching the bundled ngspice-46/ simulator, following the exact precedent set by `absdelay()`'s own OSDI extension (`OsdiAbsDelayInfo`, documented in `openvaf/osdi/header/osdi_0_4_enhancement1.h`).

5.1 ABI design — **openvaf/osdi/header/osdi_0_4_enhancement2.h** An additive, backward-compatible extension (old simulators ignore the new symbols; models without `last_crossing()` don't export them), mirroring `OsdAbsDelayInfo`'s (`y_node`, `z_node`, `offset`) shape:

```
typedef struct OsdLastCrossingInfo {
    uint32_t y_node;      /* synthetic input node y_synth (the watched expr) */
    uint32_t z_node;      /* crossing-time output node z */
    uint32_t dir_offset; /* byte offset of `dir` in per-instance OSDI data */
} OsdLastCrossingInfo;
```

exported as `OSDI_LAST_CROSSING_COUNTS[OSDI_NUM_DESCRIPTOR]` / `OSDI_LAST_CROSSING_INFOS[Σcounts]`, same indexing convention as `OSDI_ABSDELAY_*`.

Lowered the same way as `absdelay`: `eq_y(V(y_synth) = expr`, a normal compiler-stamped resistive residual so the simulator can read `expr`'s converged value each accepted timepoint) and `eq_z` (the crossing time, `V(z)`, stamped entirely by the simulator). Key simplification vs. **absdelay**: `V(z)` has zero Jacobian sensitivity to **`V(y_synth)`** almost everywhere (crossing time is locally constant in time between crossings, jumping only at a new crossing — a measure-zero event for Newton iteration), so unlike `absdelay` the simulator never needs a `J[z, y]` coupling term, only `J[z, z] = -1` and the crossing-time RHS.

5.2 Compiler side

- `openvaf/hir_lower/src/lib.rs`: `ImplicitEquationKind::LastCrossingInput/LastCrossingOutput`, `PlaceKind::LastCrossingDirection` (Real-typed, mirroring `AbsDelayTime`), `HirInterner::last_crossing_equations: Vec<(eq_y, eq_z)>`.
- `openvaf/hir_lower/src/expr.rs`: `lower_last_crossing` — same shape as the `absdelay` block, minus the `tdmax/interpolation` logic (not needed — the simulator side does the crossing search). Correctness bug found and fixed during testing: `dir` is `Val(Integer)` per the LRM signature but the storage place is Real-typed like all other simulator-read fields — needed an explicit `ifcast` before `def_place`, or the raw integer bit pattern gets reinterpreted as garbage double data on the C side (caught by the crossing time staying stuck at 0.0 in a real ngspice run — see §5.4).
- `openvaf/sim_back/src/context.rs`, `openvaf/hir_lower/src/ctx.rs`: `LastCrossingDirection` added to the same “always emit, exempt from dead-output collapsing” bucket as `AbsDelayTime`.
- `openvaf/osdi/src/inst_data.rs`, `openvaf/osdi/src/lib.rs`, `openvaf/osdi/src/eval.rs`: `last_crossing_dirs: Vec<EvalOutputSlot>` field and `store_last_crossing_dirs/last_crossing_dir_offset` methods, mirrored line-for-line from `delay_times/store_delay_times/delay_time_offset`; `OSDI_LAST_CROSSING_COUNTS/INFOS` export block mirrored from the `OSDI_ABSDELAY_*` block.

5.3 ngspice-46 side (**ngspice-46/src/osdi/**, **ngspice-46/src/include/ngspice/osdiitf.h**)

- `osdiitf.h`: `OsdRegistryEntry` gained `num_last_crossings/last_crossing_infos`, mirroring `num_absdelays/absdelay_infos`.
- `osdidefs.h`: `OsdLastCrossingInfo` struct; `OsdExtraInstData` gained `crossing_hist/crossing_hist_cap` (waveform history, same pattern as `delay_hist`), `crossing_time` (cached last-known crossing time per slot, persists across `accept()` calls, initialized to 0.0 = “no crossing observed yet”), and `crossing_jac_z/_csc/_cx` (single matrix pointer per slot — no `y` counterpart needed, since no `y`-coupling).
- `osdiregistry.c`: reads `OSDI_LAST_CROSSING_COUNTS/INFOS` at `.osdi` load time, mirroring the `OSDI_ABSDELAY_*` block exactly.
- `osdisetup.c`: allocates the single `J[z, z]` matrix entry per slot (`SMPmakeElt`) and the history buffer at instance setup; KLU CSC/complex rebinding mirrored from the `absdelay` blocks (both `OSDIbindCSC` and `OSDIupdateCSC`).

- `osdilo.c`: new `last_crossing_stamp()` — unlike `absdelay_stamp_dc/_tran`, valid in both DC/OP and TRAN modes unconditionally (no y-coupling means no DC-vs-TRAN distinction is needed): `J[z,z] += -1, RHS[z] += -crossing_time[k]`. Also calls the existing (now shared, not `absdelay`-only) `absdelay_ensure_timepoints(ckt)` when in TRAN mode — a real bug found during testing (§5.4): the shared `ckt->CKTtimePoints/ CKTtimeIndex` accepted-timepoint timeline was previously only initialized from inside `absdelay_stamp_tran`, so a circuit using `last_crossing` but *no* `absdelay` would never get that timeline set up at all, and `OSDIaccept` would silently no-op forever.
- `osdiaccept.c`: new `last_crossing_accept()` — commits `V(y_synth)` into `crossing_hist` at each accepted timepoint (same pattern as `delay_hist`), then checks whether the just-completed step contains a zero-crossing matching the requested direction (`dir>0`: rising, `dir<0`: falling, `dir==0`: either), and if so linearly interpolates the crossing time within that step and updates the cache. Left unchanged (not reset to 0) if no new qualifying crossing is found, so `V(z)` keeps returning the time of the *last* crossing, per the LRM.
- `osdiacld.c` (AC): `J[z,z] += -1`, no frequency-dependent term — the crossing time has no well-defined small-signal AC sensitivity.

5.4 Bugs found and fixed while verifying against real ngspice

1. **dir** type mismatch (compiler side, §5.2): fixed by adding the missing `ifcast`.
2. **CKTtimePoints** never initialized without **absdelay** (ngspice side, §5.3): fixed by calling `absdelay_ensure_timepoints(ckt)` from `last_crossing_stamp` too, guarded by `is_tran` (the call is idempotent).

Both were caught by running an actual transient simulation and observing the output stay stuck at `0.0` instead of tracking crossings — a reminder that cargo check/unit tests alone would not have caught either bug (both compile cleanly; both are runtime logic errors only visible under real simulation).

5.5 Verification Real ngspice-46 (built from `ngspice-46/build` in this directory, not the system-wide ngspice) transient simulation: a 100kHz sine wave (period 10μs) watched with `last_crossing(V(ctrl), 1)` (rising crossings only). Result: `V(a)` correctly stays `0.0` until the first rising crossing at `t=10μs`, then jumps to `1.0000148e-5` (0.015% from the theoretical `1.0e-5`) and holds constant until the next rising crossing at `t=20μs`, where it updates to `2.0000148e-5`. Also re-ran the pre-existing `examples/absdelay_examples/absdelay.va` 5-stage delay-line example through the patched simulator to confirm no regression in the shared C code paths (`osdilo.c/osdisetup.c/osdiaccept.c/osdiacld.c/osdiregistry.c` all touched by this change) — output still ramps up starting at exactly `t=10ns`, matching the expected `5×2ns` delay unchanged.

6. Known limitations / deferred work

- `cross(expr, dir[, tol[, accuracy]])`, `above(expr[, tol[, accuracy]])`, `timer(t0[, period])` remain unimplemented. Per the LRM these are valid only as arguments to `@(...)` event-control statements (unlike `last_crossing`, which is an ordinary value-returning expression) — so implementing them needs new `@()` event-control grammar/HIR semantics (currently only `@(initial_step)/@(final_step)` are supported) on top of further OSDI/ngspice breakpoint-scheduling support, a different and larger category of work than everything in this enhancement. The `last_crossing` infrastructure built here (§5) — waveform history, crossing detection/interpolation — is directly reusable for `cross`'s underlying detection logic; `timer` could likely reuse/extend the existing `bound_step/0sdiExtraInstData` mechanism for periodic breakpoint requests. Recommended as its own follow-up enhancement.
- **generate/genvar** blocks are entirely unimplemented (no grammar anywhere in `parser/src/grammar/`) — comparable in scope to the module instantiation work in Enhancement-5, and out

of scope for this pass.

- **`default\_discipline`** is parsed and captured (no longer breaks compilation) but not wired into net-discipline inference — every net still requires an explicit discipline declaration.
- Scale-factor/identifier disambiguation (1nsec lexing as 1n + sec instead of being rejected per the LRM) — a pre-existing minor edge case, noted but not fixed (out of scope, low real-world impact).

7. Diff summary

File	Kind of change
openvaf/preprocessor/src/parser.rs	New CompilerDirective variants, skip_rest_of_line/bump_directive_and_capture_ident helpers
openvaf/preprocessor/src/processor.rs	Dispatch for 10 new directives, default_discipline field
openvaf/preprocessor/src/tests.rs, test_data/misc_directives*	New regression fixture
openvaf/lexer/src/lib.rs	Bug fix: ShlA/ShrA three-symbol tokens now consume all 3 characters
openvaf/parser/src/grammar/expressions.rs	<<</>>> precedence
openvaf/syntax/src/ast/expr_ext.rs, openvaf/syntax/veriloga.ungram	BinaryOp::ArithmeticLeftShift/ArithmeticRightShift
sourcegen/src/mir_instructions.rs, openvaf/mir/src/instructions/generated.rs, openvaf/mir/src/builder/generated.rs	New Opcode::Iashr
openvaf/hir_ty/src/inference.rs, openvaf/hir_lower/src/expr.rs	Type-check + lowering for arithmetic shift, slew/transition/zi_*/last_crossing
openvaf/mir_llvm/src/builder.rs	Iashr → LLVMBuildAShr
openvaf/mir_interpret/src/lib.rs, openvaf/mir_opt/src/const_eval.rs, openvaf/mir_opt/src/simplify.rs, openvaf/mir_autodiff/src/lib.rs, openvaf/mir_autodiff/src/builder.rs	Iashr handling in constant-fold/const-eval/autodiff passes
openvaf/hir_lower/src/lib.rs	New ImplicitEquationKind/PlaceKind variants for slew/transition/last_crossing
openvaf/hir_lower/src/ctx.rs	LastCrossingDirection init value
openvaf/hir/src/lib.rs	Signature re-export, SLEW_*/TRANSITION_*/LAST_CROSSING_* signature re-exports
openvaf/sim_back/src/context.rs	LastCrossingDirection output-collapsing exemption
openvaf/hir_def/src/builtin.rs	is_unsupported() flag removed for slew, transition, zi_nd/np/zd/zp, last_crossing
openvaf/osdi/src/inst_data.rs, openvaf/osdi/src/lib.rs, openvaf/osdi/src/eval.rs	OsdiLastCrossingInfo export, mirrored from OsdiAbsDelayInfo
openvaf/osdi/header/osdi_0_4_enhancement2.h	New OSDI ABI extension document

File	Kind of change
ngspice-46/src/include/ngspice/osdiitf.h, ngspice-46/src/osdi/osdidefs.h, osdiregistry.c, osdisetup.c, osdiload.c, osdiaccept.c, osdiacld. c bin/macos/apple-silicon/ngspice	last_crossing runtime support (history, crossing detection, matrix stamping), mirrored from the absdelay implementation Rebuilt from the patched ngspice-46/ build tree

No changes to openvaf/osdi's `OsdiDescriptor` layout or `OSDI_DESCRIPTOR_SIZE` (binary-compatible, additive-only extension, same guarantee as Enhancement 1's `absdelay` extension).

Enhancement-7 — `@(initial_step)` event gating and variable persistence

While scoping `cross()/above()/timer()` (all three are only valid inside `@(...)` event-control statements, per the LRM), this enhancement found and fixed a foundational, pre-existing gap those operators — and a large fraction of real-world Verilog-A models — depend on: event-control statements didn't gate anything, and ordinary analog-block variables didn't persist their value across evaluations at all. `cross()/above()/timer()` and `generate/genvar` blocks are deferred to Enhancement-8 (see §5) now that this foundation is in place.

1. `@(initial_step)` / `@(final_step)` didn't gate execution at all

Root cause: `openvaf/hir_lower/src/stmt.rs`'s `Stmt::EventControl` lowering discarded the event field entirely and unconditionally lowered the body — `@(initial_step) foo();` behaved identically to bare `foo();`, every evaluation, forever. Verified with a real simulation before the fix (a reset statement that should only fire once kept firing on every timepoint).

Fix:

- New `ParamKind::IsInitialStep` — a simulator-provided boolean, true only on an instance's first-ever `eval()` call (`openvaf/hir_lower/src/lib.rs`).
- `Stmt::EventControl` now genuinely branches on it for `@(initial_step)` (`openvaf/hir_lower/src/stmt.rs`, using the same `make_cond` machinery `Stmt::If` already uses). `@(final_step)` fails safe (never fires) rather than firing every evaluation — true "about to finish" detection needs a dedicated simulator-lifecycle hook not built yet (see §5).
- ngspice-46 side: new `EVAL_FLAG_IS_INITIAL_STEP` bit (`osdidefs.h`), a per-instance `has_evaluated` flag (`OsdiExtraInstData`), set once in `osdiload.c`'s sequential (non-OMP) eval loop, reset at instance setup/temperature-update (`osdisetup.c`). No new OSDI ABI struct needed — just an additional input flag bit, same convention as the existing `CALC_*/ANALYSIS_*` flags.
- Verified via `--dump-mir` (real `br v18, block2, block3` conditional, not dead code) and a real ngspice run showing the flag set exactly once per instance.

2. Ordinary **real/integer** variables didn't persist across evaluations at all

Testing item 1 above surfaced a much deeper, pre-existing issue: an ordinary analog-block variable's value was reset to its declared default on *every* evaluation, not just the first — with zero event-control involved:

```
real accum;  
analog begin  
    accum = accum + 1.0;    // stayed flat forever; never accumulated  
    V(out) <+ accum;  
end
```

Root cause, in two parts:

- `openvaf/osdi/src/inst_data.rs/eval.rs` had two `todo!("hidden state")` panics and one `unreachable!()` on the read side — genuinely unimplemented scaffolding predating this enhancement (the `HiddenState(Variable)` parameter kind existed but nothing backed it with real storage).
- `openvaf/hir_lower/src/state.rs`'s `insert_var_init` unconditionally replaced every use of a variable's `HiddenState` parameter with its declared initializer expression — meaning even if the storage problem above were fixed, every read would still resolve to the init value, every single call.

Fix:

- `openvaf/osdi/src/inst_data.rs`: `new_hidden_state`: `Vec<(Variable, EvalOutputSlot)>` field, built from `HiddenState(var)` parameters with a live corresponding `Var(var)` output; `read_hidden_state/ store_hidden_state` (read at the start of `eval()`, store the final value at the end — the same OSDI-instance-memory slot serves as both "previous" input and "new" output, since instance memory is never reallocated between an instance's evaluations).
- `openvaf/osdi/src/eval.rs`: `ParamKind::HiddenState(var)` now reads via `read_hidden_state`; `store_hidden_state` called at the end of `eval()`.
- `openvaf/hir_lower/src/state.rs`: `insert_var_init` now applies the initializer only when `ParamKind::IsInitialStep` is true (via `make_select`, snapshotting pre-existing uses of the parameter *before* constructing the select, to avoid a self-referencing cycle), falling back to the genuine `HiddenState` read otherwise.
- `openvaf/sim_back/src/context.rs`: two-pass build. A variable's final value must be kept alive as a real output for `hidden_state` to have anything to store — but naively marking *every* `PlaceKind::Var` as always-alive broke `aggressive_dead_code_elimination`'s assumptions for genuinely-dead variables (19 pre-existing test failures, including a real `unwrap()` panic in `mir_opt/src/dead_code_aggressive.rs`). Fixed with a throwaway first build (baseline predicate) to discover exactly which `HiddenState(var)` parameters are genuinely live (not eliminated as dead), then a real second build that keeps alive only that discovered set (plus `op_vars`, as before). Fully reverted the regression; full existing test suite passes again (`cargo test -p sim_back` and friends).

Verified: the accumulator model above now genuinely accumulates across timepoints (1074, 1076, 1078, ..., incrementing correctly once per Newton iteration/evaluation as expected); full existing workspace test suite passes with zero regressions.

3. Known limitation: explicit `@(initial_step)` statements that write to a variable can crash the compiler

Two related crashes, both involving an *explicit* `@(initial_step)` statement writing to a variable (as opposed to relying on the variable's plain declared initializer, `real x = 5.0;`, which is unaffected — see `examples/initial_step_examples/`, built entirely around the safe form):

1. Self-referential + explicit double-init: a variable both explicitly written inside `@(initial_step)` *and* separately self-referentially read elsewhere crashes with `assertion failed: cx.func.validate()` (a dominance violation — `insert_var_init`'s `FunctionBuilder::edit`-based CFG insertion collides with the pre-existing branch structure from the `explicit @(initial_step)` statement):

```
real accum;
analog begin
    @(initial_step) accum = 0.0;    // now redundant -- see below
    accum = accum + 1.0;
    V(out) <+ accum;
end
```

2. Any explicit `@(initial_step)` write, even without self-reference: found while building this enhancement's examples — crashes with a *different* panic, `Option::unwrap()` on `None` in `mir_opt/src/dead_code_aggressive.rs:105`, reached via `sim_back::init::Builder::build_init_cache` — a separate init/operating- point-cache-building pass with its own similar keep-alive-predicate assumption that the `sim_back/src/context.rs` two-pass fix above didn't cover:

```
real marker;
analog begin
    @(initial_step) marker = 100.0;    // crashes even with no self-reference
    V(out) <+ marker + V(in);
end
```

Both patterns are now redundant given the fix above (variables already get their declared initializer applied automatically and only on the true first evaluation — no explicit `@(initial_step)` reset is needed for the common case anymore), so this is a narrow edge case, not a blocker for typical models using plain declared initializers. Root-causing and fixing the `FunctionBuilder::edit` interaction (crash 1) and extending the two-pass keep-alive fix to `sim_back::init` as well (crash 2) are noted here as follow-up work rather than silently left for someone to rediscover via a crash.

4. Examples

Two example folders (`.va`, `.osdi`, `DC/AC/transient.cir`, raw `wrdata` results, and PNG plots — all run against `version8`'s own `openvaf-r` and `ngspice-46` binaries, not system-wide ones) demonstrate the fixes above. Both deliberately use the *safe* (plain declared-initializer) form, per the known limitation in §3:

- **examples/initial_step_examples/** (`initial_step_demo.va`): `real accum = seed;` (a parameter), then `accum = accum + 1.0;`. DC/AC confirm `V(out) = accum + V(in)` has slope 1 and unity/0dB/0deg AC gain (accum itself has zero small-signal sensitivity to the input); transient shows the sine input riding on a slowly-rising, *persistent* baseline that starts near seed rather than resetting to seed every evaluation.
- **examples/variable_persistence_examples/** (`persist_demo.va`): the minimal case, `real accum; accum = accum + 1.0;`, no parameters, no event-control at all. Transient is the key plot — a clean, sustained linear ramp over 100μs, proving accum is never silently reset. DC/AC are included for completeness and show the honest (and expected) non-behavior: accum counts evaluations, not a real function of `V(in)`, so DC produces a roughly-monotonic curve driven by Newton-iteration count rather than a real transfer function, and AC gain is ~zero — the same “documented negative result” spirit as `examples/last_crossing_examples/`'s DC/AC plots in Enhancement-6.

5. Deferred to Enhancement-8

- **cross()/above()/timer()**: not started. These need new `@(...)` event-control grammar (currently only `@(initial_step)/@(final_step)` are parseable — see `openvaf/syntax/veriloga.ungram`'s `EventStmt` production) plus OSDI/ngspice breakpoint-scheduling support (predicting and forcing a timestep at the exact crossing time) analogous to but distinct from `last_crossing`'s history-based detection from Enhancement-6. The event-gating foundation built in this enhancement (`ParamKind::IsInitialStep`, the `make_cond`-based conditional lowering pattern, the `EVAL_FLAG_*` convention for simulator-to-model flags) is directly reusable for wiring up whatever new event kinds `cross/above/ timer` need.
- **generate/genvar** blocks: not started. No grammar exists anywhere in `openvaf/parser/src/grammar/`. Comparable in scope to Enhancement-5's module-instantiation work (likely another compile-time elaboration pass).

6. Diff summary

File	Kind of change
<code>openvaf/hir_lower/src/lib.rs</code>	New <code>ParamKind::IsInitialStep</code>
<code>openvaf/hir_lower/src/stmt.rs</code>	Real <code>@(initial_step)/@(final_step)</code> gating
<code>openvaf/hir_lower/src/state.rs</code>	<code>insert_var_init</code> gated by <code>IsInitialStep</code> , not unconditional
<code>openvaf/hir/src/lib.rs</code>	<code>Event/GlobalEvent</code> re-exports (needed to name them from <code>hir_lower</code>)

File	Kind of change
openvaf/osdi/src/inst_data.rs	hidden_state field, read_hidden_state/store_hidden_state
openvaf/osdi/src/eval.rs	IsInitialStep flag read, HiddenState real read/write wiring
openvaf/sim_back/src/context.rs	Two-pass build to correctly keep self-referential Var outputs alive
ngspice-46/src/osdi/osdidefs.h	EVAL_FLAG_IS_INITIAL_STEP, has_evaluated field
ngspice-46/src/osdi/osdiload.c	Sets the flag once per instance in the sequential eval loop
ngspice-46/src/osdi/osdisetup.c	Resets has_evaluated at instance setup/temperature update

No OSDI ABI struct/header extension was needed for this enhancement (unlike Enhancement-6's last_crossing) — the IsInitialStep mechanism reuses the existing eval-flags convention, and hidden_state reuses Enhancement-6's eval_outputs infrastructure. cross/above/timer will very likely need a real ABI extension, following the osdi_0_4_enhancementN.h convention.

Enhancement-8 — **generate for/genvar** and **cross()/above()/timer()**

Enhancement-7 (Enhancement-7.md §5) deferred two features to this enhancement: **generate/genvar** compile-time elaboration (Feature B), and the **cross()/above()/timer()** event-control functions (Feature A). Both are implemented. Feature B is built on the existing text-level elaboration pattern from Enhancement-5's module instantiation and is fully working with no known gaps. Feature A's event-detection logic is real, verified, and correct; it originally surfaced a significant *pre-existing* compiler bug (confirmed present in version8's unmodified baseline, not introduced by this enhancement) that blocked its most natural use case — persistent-variable writes inside the event body — which has since been found and fixed; see §2's "Known limitations" item 1 for the full root-cause writeup.

All work is in version9/ only; all simulation verification uses version9/ngspice-46's own locally built binary.

1. **generate for / genvar**

Scope: structural/declarative generation only — **generate for** with an ascending, compile-time-constant-foldable **genvar** loop, generating repeated **net/instance/variable/parameter** declarations. Per the Verilog-A LRM, **generate** may never emit a new **analog** block, so this scope boundary matches the language, not just an implementation shortcut. **generate if/generate case** are not implemented (noted as follow-up work, same convention as prior enhancements deferring sub-scope).

Design: text-level elaboration, mirroring Enhancement-5

- Grammar (`openvaf/syntax/veriloga.ungram`): new `ModuleItem` alternatives `GenvarDecl` (`genvar i, j;`) and `GenerateFor` (`generate for (i = 0; i < N; i = i + 1) begin : label ... end endgenerate`, body items restricted to structural `ModuleItems` via a new `GenerateBlock` production).
- Parser (`openvaf/parser/src/grammar/items/module.rs`): unambiguous keyword-dispatch on **genvar/generate** (simpler than `Instantiation`'s 2-token lookahead, since there's no bare-identifier ambiguity).
- Elaboration (`openvaf/hir/src/elaborate.rs`, new `elaborate_generates()`, run *before* `elaborate_instantiations()` so any instantiation statements inside a **generate for** body are already unrolled into concrete text by the time instantiation elaboration processes them): text-level splicing, not item-tree-level — for each **generate for** block, renders `N` concatenated copies of the block body's raw source text via the same `render_with_holes` machinery Enhancement-5 built, substituting the **genvar** identifier with its literal per-iteration integer value (whole-token substitution, plus a small integer-only constant-expression evaluator, `eval_int_expr`, to fold `node[i+1]`-style bit-select indices), and disambiguating every name declared inside the block by suffixing it with the **genvar** value (`r` → `r_0`, `r_1`, ...), exactly like an instance array's per-element naming. The loop bound/init/step must constant-fold to integer literals (`fold_int`, reusing the same literal-only evaluator `hir_def::item_tree::lower::fold_width_range` uses for `[msb:lsb]` instance-array ranges); a non-constant bound is a hard compile error (`NonConstantGenerateBound`).
- Rationale for text-level over structural: Enhancement-5 proved this needs zero downstream (`hir_def` onward) changes, and every `hir_def`-level consumer independently re-parses `db.parse(root_file)`, so a structural-only synthetic node would be invisible to them anyway.

Verification

- **--dump-mir** equivalence: `examples/generate_examples/resistor_ladder_generate.va` (a 4-element **generate for** resistor chain) vs. `resistor_ladder_manual.va` (the same chain, hand-written as 4 explicit instantiations) produce structurally identical "Optimized model setup

MIR” — same block/branch/phi shape (br v20, block4, block3, ...), differing only in SSA value numbers from the different intermediate filenames.

- ngspice DC sweep: both compiled with version9's own `openvaf-r` and run through version9's own `ngspice;examples/generate_examples/compare_ladder.py` cross-checks the two — bit-exact match (`max |generate - manual| = 0.0`) at every swept point, and both match an independent analytical resistor-divider computation to $2.068e-09$ (floating-point noise level).
- Regression: full existing example folders (`instantiation_examples`, `bus_examples`, etc.) still compile unchanged through the patched pipeline. Full cargo test suite (see §3) unaffected.

See `examples/generate_examples/README.md` for the full example writeup.

2. `cross()` / `above()` / `timer()`

Grammar Extended `EventStmt`'s existing token alternation (not a general callable builtin in `hir_def::builtin.rs`) so misuse outside event-control position (`x = cross(...)`) is statically impossible, matching how `initial_step/final_step` are handled today:

`EventStmt =`

```
AttrList* '@' '(' (('initial_step' | 'final_step') SimPhases? | condition: Expr) ')' Stmt
```

condition, when present, is always a `Call` per the LRM (these three names are only ever legal in this exact position) — recognized by bare function name in `hir_def::body::lower::collect_cross_above_timer`, *not* materialized as a general builtin/function-call expression. A malformed/unrecognized condition degrades to the event being dropped (body fires unconditionally) rather than a hard lowering error — there's no lowering-level diagnostic channel in this module yet (a real "not a valid event-control expression" diagnostic is noted as follow-up work, same as other soft-degradation conventions in this codebase).

Design: eval-granularity edge detection, not breakpoint-forcing The original plan called for exact-crossing-time detection via a real OSDI ABI extension and ngspice-side `CKTsetBreak` breakpoint scheduling (Feature A's Phase 7/8 in the approved plan), mirroring Enhancement-6's `last_crossing` ABI extension. This was descoped during implementation in favor of a much simpler design that turned out to need *zero* new OSDI ABI surface and *zero* ngspice-46 C changes:

- New `ParamKind::EventState(u32)` / `PlaceKind::EventState(u32)`: a persistent real storage slot, read-at-start/store-at-end-of-eval(), exactly like Enhancement-7's `HiddenState(Variable)` mechanism, but keyed by a compiler-assigned index instead of a source-level `Variable` (there's no user declaration to hang synthetic per-call-site state off of). Wired into `openvaf/osdi/src/inst_data.rs` (`event_state` field, `read_event_state/store_event_state`) and `openvaf/osdi/src/eval.rs` identically to `hidden_state`'s existing plumbing.
- `above(expr)`: fires when `expr` transitions from `<= 0` to `> 0` between the *previous evaluation* and the *current* one (`hir_lower/src/stmt.rs`'s `lower_above`) — `prev` is seeded to the current value on `ParamKind::IsInitialStep`, so the first evaluation never spuriously fires.
- `cross(expr, dir)`: fires on any evaluation-to-evaluation zero-crossing, filtered by `dir` (`< 0` falling-only, `> 0` rising-only, `== 0`/absent either) — `dir` is read as an ordinary runtime `Value`, not assumed constant, mirroring how `last_crossing`'s own `dir` argument is handled.
- `timer(t0, period)`: fires when `Abstime >=` a persisted "next fire time" (initially `t0`); reschedules by one period each firing (or to `INFINITY`, i.e. never again, for a one-shot timer with no period). No crossing-prediction math needed — `t0/period` arithmetic is ordinary MIR.
- All three combine boolean sub-conditions via `bool_and/bool_or` helpers built from `LoweringCtx::make_select` (the same machinery `BooleanAnd/BooleanOr` expression lowering already uses), since the MIR has no native `Band/Bor` opcode for pre-computed boolean values.
- Why this is a legitimate design, not a shortcut: Enhancement-7 already established (and verified) that this codebase's variable-persistence granularity is *per-eval()-call*, not per-accepted-timepoint — "the accumulator... incrementing correctly once per Newton iteration/evaluation

as expected” (Enhancement-7.md §2). EventState reuses that exact, already-verified granularity rather than introducing a different one.

Compile-time verification (**--dump-unopt-mir**) All three event kinds produce genuine, non-dead-code conditional branches. above:

```
v19 = phi [v16, block2], [v17, block3]    // gated "previous value"
v20 = f1e v19, v3                          // was_below = prev <= 0
v21 = fgt v16, v3                          // is_above = current > 0
br v20, block5, block6
```

cross additionally shows the dir-filtering rising/falling/either combination (fgt/flt/f1e/fge combined via nested branches); timer shows v21 = fge v20, v19 (fired) and v22 = fadd v19, v16 (reschedule).

ngspice verification examples/cross_examples/tran_above.cir: $V(in) = 2 \cdot \sin(2\pi \cdot 1\text{kHz} \cdot t)$, thresh = 1.0 — above fires once per cycle, exactly every ~1ms, at $in \approx \text{thresh}$:

```
above fired at t=8.428e-05 in=1.01028
above fired at t=0.00108428 in=1.01028
above fired at t=0.00208428 in=1.01028
```

examples/cross_examples/tran_cross.cir: same sine, thresh = 0.0, dir = 0.0 (either direction) — cross fires twice per cycle (rising and falling), alternating sign, exactly every ~0.5ms.

examples/timer_examples/tran_timer.cir: t0 = 2ms, period = 1ms — first firing at exactly t0, then every period after:

```
timer fired at t=0.0020028
timer fired at t=0.0030028
timer fired at t=0.0040028
timer fired at t=0.0050028
timer fired at t=0.0060028
```

Known limitations

1. No persistent state inside the event body — a chain of three pre-existing compiler bugs, not introduced by this enhancement, all now fixed. The natural way to use cross/above/timer is to accumulate a count or track state on each firing (count = count + 1.0; inside the event body). Confirmed to already crash on version8’s unmodified baseline with a plain if ($V(in) > 0.0$) count = 1.0; — no event-control involved at all; the trigger is any *single-branch* conditional write to a variable that’s also read elsewhere (an implicit “else: keep old value” phi) — if/else where *both* branches assign does not trigger it. Same class of bug as Enhancement-7.md §3’s two documented @(initial_step)-write crashes, now confirmed *general*. Three distinct, independently-diagnosed bugs were found and fixed chasing this:

- Fixed: mir_opt::simplify_cfg::SimplifyCfg::simplify_bb’s “remove CFG-unreachable block” cleanup unconditionally zap_inst-ed every instruction in such a block, unlike this file’s other phi-removal helpers (simplify_trivial_phis, simplify_duplicates_phis_naive), which call replace_uses first. A block can look CFG-unreachable at this point in the pipeline while one of its instructions’ results is still genuinely needed by a not-yet-inserted mir_autodiff derivative/Jacobian instruction (created *after* this pass runs, referencing existing values by raw operand before its own instructions are spliced into the layout). Fixed by skipping removal when any of the block’s instructions still have outstanding uses (sim_back’s compute_outputs/base_keep also needed a matching PlaceKind::EventState entry, already covered in the diff table above).

- Fixed: `mir::dominators::DominatorTree::compute_reverse_postorder` (post-dominance) seeded its walk from only `func.layout.last_block()`, silently treating it as the function's one universal exit. A function with more than one real exit block (e.g. two independent branch regions each ending their own control-flow path) left every block that couldn't reach *that specific* block with `rpo_number == UNDEF`, making `ipdom()` return `None` for perfectly well-defined nested branches — which `mir_opt::split_tainted::propagate_taint`'s branch-handling relies on to know where an op-dependent branch's control-dependence region ends, and which `sim_back::refresh_op_dependent_insts` calls right before every `Initialization::new`. Fixed by seeding the reverse-postorder walk from *every* real exit block at once (as if they fed into one virtual super-exit) and giving each of them beyond the first an explicit synthetic idom of the first (rather than computing one, which requires at least one reachable predecessor these true exits don't have by construction) — equivalent to a virtual multi-exit root without allocating a real block.
 - Fixed: `mir::Layout::merge_blocks(pred, succ)` keeps `pred` at its own layout position and discards `succ` entirely — it never moves `pred` to `succ`'s old position. `mir_opt::simplify_cfg`'s `merge_block_into_predecessor` uses this to fold a block into its unique predecessor whenever safe, including cases where the merged-away block (`succ`) happened to be `func.layout.last_block()` — the function's true, physically-last exit block. After such a merge, `last_block()` silently started pointing at whatever block was physically last *before* the removed block, which is not necessarily related to the true final block at all (confirmed via debug instrumentation: `bb=block1 pred=block4 merge`, with `block1` the actual last block, left `last_block()` pointing at an unrelated earlier block, `block8`). This directly broke `mir::cursor::CursorPosition::goto_exit()` (`self.goto_bottom(self.layout().last_block().unwrap())`), which `sim_back::dae::builder::ensure_optbarriers()` calls to append optbarrier-wrapped DAE residual/Jacobian instructions at "the" exit — it silently appended them into the wrong block, producing a value (`v31 = optbarrier v19`) used before its operand's defining block (`block4`) dominated the insertion point (`block8`), a genuine SSA dominance violation. This is the exact same class of "trust `last_block()` as the one universal exit" bug already fixed once above in `dominators.rs`, now confirmed to also afflict block-merging. Caught precisely (rather than as the late, opaque "internal error: entered unreachable code: attempted to read undefined value" LLVM-codegen panic release builds produce) by switching to a plain debug build, which activates `debug_assert!(cx.func.validate())` in `sim_back/src/lib.rs` and surfaces `mir::validation`'s real dominance checker's precise diagnostic ("`v19 doesn't dominate use (block4 !dom block8)`") immediately after `DaeSystem::new`, long before codegen. Fixed with a new `Layout::move_block_to_end(pred)` helper, called from `merge_block_into_predecessor` whenever the merged-away block was the layout's last block — `pred` (which now holds the merged-away block's terminator and contents) takes over the last-block position, so `last_block()` continues to mean "the function's true exit" for every downstream consumer (`goto_exit()` included) without those consumers needing to change.
 - Verified fixed: `verify_fix.va` (`examples/cross_examples/`) — a plain `if (V(in) > 0.0) count = 1.0; V(out) <+ count;` — now compiles and simulates correctly (ngspice transient run: `count/V(out)` latches to `1.0` the instant `V(in) > 0` and stays there even after `V(in)` falls back to `0`, exactly the correct persistent-write semantics). The `above_demo.va/cross_demo.va/timer_demo.va` examples have been upgraded from \$strobe-only reporting to real persistent-counter accumulation (`count = count + 1.0;` on every firing, exposed on `V(out)`) — see the updated DC/AC/transient plots and READMEs in `examples/cross_examples//examples/timer_examples/`. Full regression re-verified after the fix: all unit tests in the touched crates (`mir`, `mir_opt`, `mir_autodiff`, `mir_build`, `hir_lower`, `sim_back`) pass; every existing example folder (`cross_examples`, `timer_examples`, `generate_examples`, `absdelay_examples`) still compiles and simulates cleanly.
2. Detection is eval-granularity, not exact-time-forced. No breakpoint scheduling (`CKTsetBreak`) —

see "Design" above for why this is an intentional, documented scope reduction, not an oversight. Firing times land within one simulator timestep of the true crossing, not bit-exact (see the in values in the above/cross results above — close to but not exactly thresh/0).

3. Fixed: an unrelated, pre-existing ngspice-46 parser bug (confirmed on version8's baseline too, before this fix): the *first* param=value pair in a multi-parameter .model override list was silently ignored, falling back to the .va's declared default; subsequent ones applied correctly. Discovered while debugging what first looked like a timer() bug (t0 appeared stuck at its default) — turned out to be entirely unrelated to event-control, reproducible with a plain two-parameter module and no event-control at all. Root-caused to INPgetNetTok() (ngspice-46/src/spicelib/parser/inpgtok.c): its leading-garbage-skip loop treats (as a skippable boundary character, but its token-end scan loop did not treat (as a terminator (unlike the sibling function INPgetTok(), which is consistent on both loops). spicelib/parser/inpgmod.c's create_model() uses INPgetNetTok() to discard an OSDI model's device-type token (. model name devtype(p1=v1 p2=v2) — note no space before (, and this asymmetry made it swallow the (and the first parameter's name (devtype(p1) as one token, stopping only at the following =. The first parameter's value token was left with no name attached, silently dropped by create_model's unrecognized-bare-number fallback; every subsequent name=value pair re-synced correctly since the token boundaries realign after that. Fixed by adding the missing if (*point == '(') break; to INPgetNetTok()'s token-end scan loop. Verified against the entire existing example suite (every *_examples/ folder's .cir files) with no behavior change, and confirmed timer_demo/above_demo/cross_demo now correctly apply a *single* (non-repeated) first-parameter override — the repeat-first-param workaround in the example .cir files has been removed.

3. Verification summary

- Full cargo test --workspace (excluding veriloga*, which has an unrelated pre-existing llvm-sys feature-unification build issue in this environment, and the openvaf --test integration binary, which reads from external/vacask/devices, a git submodule absent in this non-git working copy): all green except the single pre-existing sourcegen::osdi::gen_osdi_structs failure, already documented as a pre-existing, unrelated header-parsing quirk in Enhancement-5.md's regression notes.
- sim_back's full 24-test suite (dae/init/topology) passes — including after the PlaceKind::EventState/compute_outputs keep-alive extension.
- Regression-compiled every pre-existing example folder (instantiation_examples, initial_step_examples, variable_persistence_examples, etc.) through the patched compiler — unchanged output.

4. Diff summary

File	Kind of change
openvaf/syntax/veriloga.ungram	GenvarDecl/GenerateFor/GenerateBlock productions; EventStmt's condition: Expr alternative
openvaf/parser/src/grammar/items/module.rs	genvar/generate keyword dispatch
openvaf/parser/src/grammar/stmts.rs	event_stmt falls through to expr(p) for non-keyword conditions
openvaf/hir_def/src/item_tree.rs, item_tree/lower.rs, item_tree/diagnostics.rs	ModuleItem::GenerateFor/Genvar variants, genvar constant-fold diagnostics

File	Kind of change
openvaf/hir_def/src/builtins.rs, openvaf/syntax/src/ast/generated/*.rs, openvaf/tokens/src/parser/generated.rs, sourcegen/src/ast/src.rs	Regenerated via <code>cargo test -p sourcegen ast</code> after the grammar changes above
openvaf/hir/src/elaborate.rs, openvaf/hir/src/db.rs	New <code>elaborate_generates()</code> , text-splicing generate for unrolling, hooked into <code>CompilationDB::new</code> before <code>elaborate_instantiations</code>
openvaf/hir_def/src/expr.rs	<code>Event::Cross/Above/Timer</code> variants + <code>walk_child_exprs</code>
openvaf/hir_def/src/body/lower.rs	<code>collect_cross_above_timer</code> : recognizes <code>cross/above/timer</code> calls by name
openvaf/hir_ty/src/inference.rs	Type-checks <code>Event</code> 's <code>exprs</code> as <code>Real</code>
openvaf/hir_ty/src/validation/body.rs	Validates <code>Event</code> 's <code>exprs</code>
openvaf/hir_lower/src/lib.rs	<code>ParamKind::EventState</code> , <code>PlaceKind::EventState</code> , <code>event_state_count</code> counter
openvaf/hir_lower/src/stmt.rs	<code>lower_above/lower_cross/lower_timer</code> , <code>bool_and/bool_or</code> helpers
openvaf/hir_lower/src/ctx.rs	<code>PlaceKind::EventState</code> 's default-init case
openvaf/osdi/src/inst_data.rs	<code>event_state</code> field, <code>read_event_state/store_event_state</code> (mirrors <code>hidden_state</code>)
openvaf/osdi/src/eval.rs	<code>EventState</code> read/store wiring
openvaf/sim_back/src/context.rs	<code>PlaceKind::EventState</code> added to both keep-alive predicates (<code>base_keep</code> , <code>compute_outputs</code>)
openvaf/mir_opt/src/simplify_cfg.rs	Compiler fix: <code>simplify_bb</code> 's unreachable-block removal now skips blocks whose instructions' results still have outstanding uses; <code>merge_block_into_predecessor</code> now preserves the layout's last-block invariant when the merged-away block was last (§2, limitation 1)
openvaf/mir/src/dominators.rs	Compiler fix: <code>compute_reverse_postorder/compute_domtree</code> now handle functions with more than one real exit block (§2, limitation 1)
openvaf/mir/src/layout.rs	Compiler fix: new <code>Layout::move_block_to_end</code> helper, used by <code>merge_block_into_predecessor</code> to keep <code>last_block()</code> meaning "the true function exit" after a block merge (§2, limitation 1)
ngspice-46/src/spicelib/parser/inpgtok.c	ngspice fix: <code>INPgetNetTok()</code> now terminates a token on (, fixing the first-.model-parameter-silently-ignored bug (§2, limitation 3)

No OSDI ABI header extension (`osdi_0_4_enhancement3.h`) and no ngspice-46 C-side changes were needed — see §2's "Design" for why.

Deferred to follow-up work: `generate_if/generate_case`; exact breakpoint-forcing for `cross/above/timer` (§2, limitation 2); a real diagnostic (rather than silent degradation) for malformed `@(...)` conditions.

Enhancement-9 — Verilog-A noise sources: completing **noise_table** / **noise_table_log**

This document describes the source-code changes made to OpenVAF-r in the `version10/` directory, on top of `version9/` (Enhancement-8, `generate for/genvar` and `cross()/above()/timer()`), to make the Verilog-A noise analog operators fully usable for ngspice `.noise` analysis.

All work is in `version10/` only; all simulation verification uses `version10/ngspice-46`'s own locally built binary and `version10/OpenVAF-master`'s own `openvaf-r`.

0. Scope, and the honest starting point

The requested feature was "noise sources". A gap analysis of the `version10` baseline (which is upstream OpenVAF-Reloaded plus Enhancements 1–8) showed that the noise-source family was two-thirds already functional and one-third crashing:

operator	baseline status
<code>white_noise(pwr [, "name"])</code>	already worked end-to-end — never touched by any prior enhancement; it is stock upstream functionality
<code>flicker_noise(pwr, exp [, "name"])</code>	already worked end-to-end
<code>noise_table(..) / noise_table_log(...)</code>	crashed the compiler — front-end recognised the builtins and type-checked all four call forms, but no data was ever read and the OSDI backend hit <code>unimplemented!("noise tables")</code>

This was verified empirically before writing any code:

- `white_noise`: a `white_noise(4kT/R)` thermal resistor compiled and simulated with the *unmodified* baseline, and its `.noise` output matched the closed-form `4kT` result to the last digit (see §4).
- `flicker_noise`: a `flicker_noise(pwr, 1)` element produced the correct $1/\sqrt{f}$ density rolloff on the unmodified baseline.
- `noise_table`: a one-line `I(a,b) <+ noise_table("f","g")` model crashed `openvaf-r` with index out of bounds: the `len` is `0` but the index is `0` in `sim_back/src/topology/lineralize.rs:45`.

Enhancement-9 therefore implements **noise_table/noise_table_log** end-to-end (the genuinely-missing noise source) and ships an analytically-verified example suite covering all four operators (`examples/noise_examples/`), including `white_noise/flicker_noise` as verified reference examples. No changes to `white_noise/flicker_noise` were needed or made; a regression run confirms their output is byte-identical before and after (see §4).

Four further unrelated defects found during this work are also fixed here: `localparam` being silently overridable, contrary to the LRM (§5); the electrical ground `gnd`; net-declaration ordering failing to parse (§6); a regression in which Enhancement-8's regeneration of `builtin.rs` re-disabled `slew/transition/last_crossing/zi_*` (implemented back in Enhancement-6) — now re-enabled (§7); and an uninitialized string variable crashing the compiler (§8). The `repeat` loop, previously unsupported entirely, is also implemented (§9), as is the `disable` early-exit statement (§10).

1. Why **noise_table** crashed: a three-part gap

`noise_table/noise_table_log` were "wired for show" in the baseline — every layer between the parser and codegen referenced them, but three links in the chain were stubs:

1. No data was ever read. `hir_lower`'s builtin-lowering created the callback with a hard-coded placeholder table and no arguments:

```
let noise_table = NoiseTable::new([(0.0, 0.0)], log, name, idx);
self.ctx.call1(CallBackKind::NoiseTable(Box::new(noise_table)), &[]) // <--
→ &[]
```

(`NoiseTable::new` even carried a `// TODO: read from disk.`) Neither the inline-array argument nor the file-name argument was consulted.

2. The empty arg list crashed topology construction. `sim_back::topology::lineralize`'s `build_analog_operators` unconditionally read `instr_args(operator_inst)[0]` for every analog operator. `ddt`, `white_noise` and `flicker_noise` all have at least one MIR value argument, so this never fired before — but `noise_table` legitimately has zero value args (its data lives in the callback), so `[0]` panicked.
3. The OSDI backend refused to emit code. Both frequency-dependent noise entry points in `osdi/src/load.rs` (`load_noise` and `load_noise_params`) had `NoiseSourceKind::NoiseTable { .. } => unimplemented!("noise tables")`.

2. The implementation

2.1 Reading the table data (`hir_lower/src/expr.rs`) The `noise_table/noise_table_log` lowering arm now gathers the real (frequency, power) pairs before building the callback:

```
let table_vals = self.noise_table_data(signature, args);
let noise_table = NoiseTable::new(table_vals, log, name, idx);
```

`noise_table_data` dispatches on the resolved call signature (the same four signatures `hir_ty` already defines — `NOISE_TABLE_INLINE`, `NOISE_TABLE_FILE`, and their `_NAME` variants):

- Inline array `noise_table({f0, p0, f1, p1, ...})`: the first argument is a real `Expr::Array`; each element is folded to a constant via a small `eval_const_real` helper (literal `Real/Integer`, with an optional leading unary `+/-`), then the flat list is paired up into `(f, p)` tuples.
- File `noise_table("path")`: the first argument is a string literal; a new `read_noise_table_file` helper resolves it relative to the compilation root file's directory and parses a two-column whitespace-separated `<frequency> <power>` file (blank lines and `#!/**` comment lines skipped).

Path resolution needed the on-disk directory of the root source file. Rather than widen `hir_lower`'s dependency surface (it depends on `hir`, but `basedb` is only a `hir_lower dev`-dependency), a small public accessor was added to the `hir` crate:

```
// openvaf/hir/src/db.rs
pub fn root_file_dir(&self) -> Option<VfsPath> {
    self.file_path(self.root_file).parent()
}
```

Missing/unreadable files and malformed lines degrade gracefully to an empty table (which the codegen renders as a constant-zero noise source) rather than crashing — matching the codebase's existing soft-degradation conventions for other builtins.

2.2 Fixing the topology crash (`sim_back/src/topology/lineralize.rs`) `arg0` is only ever consumed by the non-noise (`ddt`) branch, so it is now read defensively:

```
let arg0 =
    self.func.dfg.instr_args(operator_inst).first().copied().unwrap_or(F_ZERO);
```


This removes the panic for zero-argument noise operators without changing behaviour for `ddt/white_noise/flicker_noise` (all of which still supply `arg0`).

2.3 OSDI interpolation codegen (`osdi/src/load.rs`) The core of the enhancement. `load_noise(inst, model, freq, noise_dens)` is the per-frequency evaluator ngspice calls during `.noise`. For a table source it now emits, via a new `build_noise_table_interp` helper, LLVM IR that:

1. computes $\log_{10}(\text{freq})$ (the `llvm.log10.f64` intrinsic);
2. performs piecewise-linear interpolation of the tabulated power over $\log_{10}(\text{freq})$, fully unrolled into a chain of selects. Every segment's slope and intercept is a compile-time constant, so each segment costs exactly one `fmul`, one `fadd`, one `fcmp olt` and one `select`;
3. clamps to the first/last tabulated point outside `[x[0], x[n-1]]`.

The unrolled form walks segments from the top down; after the loop the surviving select is precisely the bracketing segment (the lowest-index segment whose upper bound exceeds $\log_{10}(\text{freq})$), and a final select clamps the below-range case to `y[0]`. Degenerate tables are handled (`n == 0` $\rightarrow$ constant `0`, `n == 1` $\rightarrow$ constant `y[0]`). The existing `pwr * factor2` scaling that `white_noise/flicker_noise` already apply is reused unchanged, so a `k * noise_table(...)` contribution scales correctly.

The second entry point, `load_noise_params(inst, model, power, exponent)`, is a white/flicker-oriented ABI slot (it exposes a single scalar power and a single frequency exponent). A frequency-dependent table has neither, and ngspice never calls this function (its OSDI noise path uses only `load_noise` — see §3), so the table case there returns `0.0` with an explanatory comment rather than fabricating a meaningless scalar. The real, frequency-aware evaluator is `load_noise`.

2.4 Interpolation semantics (why the `log` flag needs no run-time handling) `hir_lower::NoiseTable::new` stores the table keyed by $\log_{10}(\text{frequency})$ in both cases: for `noise_table` it applies `.log10()` to the (linear) frequency column; for `noise_table_log` it stores the column as-is (the user already supplied $\log_{10}(f)$). Because the stored key domain is identical, the run-time lookup key is always $\log_{10}(\text{freq})$ and the `log` flag is fully consumed at build time — the codegen ignores it. This is confirmed by the examples: `noise_table.txt` (linear 1, 100, 10000) and `noise_table_log.txt` (0, 2, 4) produce bit-identical spectra (§4). The interpolated quantity is the linear power spectral density; interpolation is linear-in-power over log-frequency.

3. ngspice-46: no changes required (and why)

`ngspice-46/src/osdi/osdinoise.c` already drives the OSDI noise ABI generically: in `N_CALC/N_DENS` it calls `descr->load_noise(inst, model, data->freq, noise_dens)` once per frequency and folds the result into the output-noise integral. It does not branch on `noise_source_type`, so `NOISE_TYPE_TABLE` sources flow through the exact same path as white/flicker ones — the table-specific behaviour lives entirely inside the OSDI-generated `load_noise` function produced in §2.3. `load_noise_params` is not referenced anywhere in ngspice-46. The full `.noise` verification in §4 uses the unmodified prebuilt `version10/ngspice-46/build/src/ngspice`. No OSDI ABI header change (`osdi_0_4*.h`) and no ngspice C change were needed.

4. Verification

All verification lives in `examples/noise_examples/` and is automated by `examples/noise_examples/verify_noise.py`, which compiles each model with `version10's openvaf-r`, runs the matching `.noise` deck through `version10's ngspice`, reads back the `onoise_spectrum` (output-noise voltage density, $V/\sqrt{\text{Hz}}$), and compares it point-by-point against an independent closed-form computation.

Measurement topology (identical for every deck): $V(\text{in}) - [r1 = 1 \text{ k}\Omega] - V(\text{out}) - [n1 = \text{OSDI noise device}] - \text{gnd}$, with `.temp 26.85` ($= 300.00 \text{ K}$) so ngspice's own `r1` thermal noise matches the models' `Temp=300`. Then $S_{\text{out}}(f) = (S_{\text{i,device}}(f) + 4kT/R1) \cdot Z_{\text{out}}^2$, $Z_{\text{out}} = R1 || R_{\text{device}}$.

```

white      : max relative error = 1.6e-07    PASS    (flat 4kT spectrum)
flicker    : max relative error = 1.5e-07    PASS    (1/f + white floor)
table      : max relative error = 2.5e-09    PASS
table_log  : max relative error = 2.5e-09    PASS
table_log vs table: max |diff| = 0.0         PASS    (bit-identical)

```

- **white_noise** — the same thermal-divider example run on the *unmodified* baseline and on the patched compiler produces byte-identical `onoise_total = 3.881810e-06`, `inoise_total = 4.269991e-06`, matching $\sqrt{(4kT \cdot (1/1k + 1/10k) \cdot Z_{out}^2)}$ analytically.
- **noise_table** — includes a non-trivial interpolated point: at $f = 1$ kHz ($\log_{10} f = 3$, midway between the $\log_{10} f = 2$ and $\log_{10} f = 4$ table nodes) the interpolated power is $1e-12 + (1e-16 - 1e-12) \cdot (3-2)/(4-2) = 5.0005e-13$, i.e. $onoise = \sqrt{(5.0005e-13) \cdot 1000} \approx 7.0714e-4$ V/ $\sqrt{\text{Hz}}$, which ngspice reproduces as $7.071414e-4$. End-of-range clamping is confirmed by sweeping down to 0.1 Hz (below the first node) and up to 10 kHz — both flatten to the endpoint values.

See `examples/noise_examples/README.md` and `examples/noise_examples/noise_spectra.png` for the full write-up and plots.

Regression

- `cargo test -p sim_back` — all 24 tests pass, including every noise topology test (`correlated_noise`, `conditional_noise`, `unused_noise`, `manual_correlated_noise`).
- `cargo test -p hir` and `cargo test -p hir_lower` — all pass (the new `root_file_dir` accessor and `noise_table_data` helpers included).
- Regression-recompiled the pre-existing example models (`cross_examples`, `laplace_examples`, `instantiation_examples`, `bus_examples`, `generate_examples`, `absdelay_examples`, `timer_examples`, `initial_step_examples`, `variable_persistence_examples`) through the patched compiler — 18/18 compile unchanged.
- `white_noise/flicker_noise.noise` output is byte-identical pre/post.

5. Follow-up fix: **localparam** is now non-overridable (LRM conformance)

While verifying the noise models (which use `localparam` for physical constants), a gap analysis of parameter handling surfaced a second, unrelated pre-existing defect: **localparam** was treated identically to **parameter**.

The defect The parser records an `is_local` flag for `localparam` declarations (`hir_def::item_tree`), but that flag was set and never read anywhere — it was not even carried into `hir_def::ParamData`. Consequently every `localparam` was exposed to the simulator as an externally-settable model/instance parameter and could be overridden from the `.model` card, which the Verilog-AMS LRM forbids (a `localparam` is a local constant, not overridable). Confirmed on the baseline: `.model m_lptest(G=0.01)` for a `localparam real G = 0.001`; changed the device's behaviour to $G = 0.01$.

The fix `is_local` is now threaded from `ParamData(hir_def) → hir::Parameter::is_local →` the OSDI setup functions. In `osdi::setup(setup_model/setup_instance)`, the runtime "given" flag fed to the parameter-initialization code is forced to a constant **false** for every **localparam**. The parameter-init MIR is left completely unchanged, so its `given ? override : default` selection always resolves to the default expression; the constant-false condition is folded by LLVM (robust), not by the MIR optimizer.

Why the fix lives in `osdi::setup`, not the param-init MIR The default-value store is physically emitted inside the *not-given* branch of the param-init `make_cond` (so a defaulted parameter is written only when

the simulator did not supply a value). Two more direct-looking approaches were tried and rejected because they corrupted derived localparams (`localparam G = 1/R`): forcing the `make_cond` condition to a MIR constant `false` triggered a constant-branch mis-fold that leaked the uninitialized override slot, and a dedicated straight-line param-init path evaluated the default outside the context where the dependency `R` resolves. Forcing the *runtime given input* `false` at the OSDI/LLVM boundary keeps the MIR pristine — so `1/R` still resolves correctly — while guaranteeing the default is always stored.

Verified behaviour (`examples/localparam_examples/`) `examples/localparam_examples/verify_localparam.py` (a `GAIN*G = GAIN/R` conductance in a divider) exercises every case end-to-end through ngspice:

<code>rdiv()</code>	<code>V(out)=0.333333</code>	PASS	defaults
<code>rdiv(R=2000)</code>	<code>V(out)=0.500000</code>	PASS	parameter <code>R</code> overridable
<code>rdiv(R=500)</code>	<code>V(out)=0.200000</code>	PASS	parameter <code>R</code> overridable
<code>rdiv(G=0.5)</code>	<code>V(out)=0.333333</code>	PASS	localparam <code>G</code> override ignored
<code>rdiv(GAIN=10)</code>	<code>V(out)=0.333333</code>	PASS	localparam <code>GAIN</code> override ignored
<code>rdiv(R=4000 G=9 GAIN=9)</code>	<code>V(out)=0.666667</code>	PASS	only <code>R</code> applies

A derived localparam (`G = 1/R`) is verified to still track its underlying parameter (`R=2000` $\rightarrow$ `V(out)=0.5`) while ignoring any direct override of `G`. Regular parameter overrides are unaffected, and the full unit-test suite and all pre-existing example models (18/18) compile and simulate unchanged.

6. Follow-up fix: **electrical ground gnd**; now parses

The Verilog-A ground net-type (which declares a node as the global `V = 0` reference, collapsed to the circuit ground) already worked, but the net-declaration parser only accepted the net-type before the discipline (`ground electrical gnd;`). The equally-valid discipline-first ordering `electrical ground gnd;` failed with unexpected token `NET_TYPE`; expected identifier, even though `port_decl` already accepted a net-type in that position.

`net_decl` (`openvaf/parser/src/grammar/items/module.rs`) now eats an optional `NET_TYPE` token after the discipline in the discipline-first branch — a one-line mirror of what `port_decl` does. All four natural orderings now parse to an identical device (`examples/ground_examples/verify_ground.py`, all end-to-end through ngspice):

<code>ground electrical gnd;</code>	<code>a=1.0 b=0.6667</code>	PASS	(already worked)
<code>electrical ground gnd;</code>	<code>a=1.0 b=0.6667</code>	PASS	(fixed here)
<code>electrical gnd; ground gnd;</code>	<code>a=1.0 b=0.6667</code>	PASS	
<code>ground gnd; electrical gnd;</code>	<code>a=1.0 b=0.6667</code>	PASS	

The internal ground node correctly acts as the `0 V` reference (`V(b) = 2000/3000`). A bare `ground gnd;` with no discipline remains a (correct) error. The parser test suite (15 tests) passes and the change is a strict superset — ordinary net declarations (`electrical a, b;`) are unaffected because `eat` consumes a `NET_TYPE` only when one is actually present.

7. Regression fix: **slew/transition/last_crossing/zi_*** re-enabled

These analog operators were implemented in Enhancement-6 (version7): their `BuiltIn::is_unsupported()` gate was removed and full lowering — plus, for `last_crossing`, ngspice runtime support — was added. They worked in version7 and version8.

Enhancement-8 (version9) silently regressed them. That enhancement regenerated `openvaf/hir_def/src/builtin.rs` (via `cargo test -p sourcegen ast`), and the regeneration re-added the `is_unsupported()` entries for `zi_nd/zi_np/zi_zd/zi_zp`, `last_crossing`, `slew`, and `transition` — shadowing the still-present, still-correct lowering with a hard “function is currently not supported” validation error. version10 inherited the regression. (Confirmed by diffing `is_unsupported()` across

versions: the seven entries are absent in v7/v8 and present in v9/v10, and the only difference is those seven lines; the lowering code was untouched.)

Enhancement-9 removes those seven entries again (with a comment noting the history so a future re-generation doesn't silently reintroduce them). No other change was needed — the lowering and last_crossing ngspice runtime from Enhancement-6 are intact. All four example folders now compile and simulate:

examples/slew_examples/slew_demo.va	compiles + tran OK
examples/transition_examples/transition_demo.va	compiles + tran OK
examples/last_crossing_examples/last_crossing_demo.va	compiles + DC/AC/tran OK
examples/zi_examples/zi_lpf.va	compiles + tran OK

After this fix, 39/39 example models across all folders compile, and the unit-test suite is unaffected.

8. Follow-up fix: uninitialized **string** variables no longer crash

real and integer analog-block variables were always fully supported, as was a string variable with an initializer (`string s = "x";`). But an uninitialized string `s`; crashed the compiler: the type-based default-value assignment for a variable declared without an initializer (`hir_def/src/body.rs`) only handled Real ($\rightarrow 0.0$) and Integer ($\rightarrow 0$) and fell through to `unreachable!("invalid var type")` for String:

```
let default_val = match db.var_data(var).ty {
  Type::Real    => Literal::Float(Ieee64::with_float(0.0)),
  Type::Integer => Literal::Int(0),
  Type::String  => Literal::String("").into(), // added in Enhancement-9
  _             => unreachable!("invalid var type (TODO arrays)"),
};
```

Enhancement-9 gives string variables the LRM-correct empty-string (`""`) default. `string s`; now behaves like the other types — verified end-to-end (`examples/vartype_examples/`): an uninitialized string mode defaults to `""`, the mode `== ""` test then assigns `"series"`, and the string-selected branch drives the output correctly (`V(b) = 2000/3000`), alongside real and integer variables in the same model.

9. New feature: **repeat** loop

The Verilog-AMS **repeat (count) statement** loop was entirely unsupported — `repeat` was not a recognized keyword, so `repeat (4) begin ... end` failed to parse. Enhancement-9 implements it across the full pipeline:

- Tokens/grammar/AST (`tokens/parser/generated.rs`, `syntax/veriloga.ungram`, `syntax/src/ast/generated/nodes.rs`): a `repeat` keyword (`REPEAT_KW`) and a `RepeatStmt` node (`repeat '(' count:Expr ')' body:Stmt`). The generated files were hand-edited rather than regenerated via `cargo test -p sourcegen ast`, to avoid re-triggering the kind of regeneration regression documented in §7.
- Parser (`parser/src/grammar/stmts.rs`): a `repeat_stmt` production, mirroring `while_stmt`.
- HIR (`hir_def::Stmt::Repeat`, `hir::Stmt::Repeat`, body lowering, pretty printer, `hir_ty` inference + validation): the count expression is type-checked/validated as an ordinary value and the body as a child statement.
- MIR lowering (`hir_lower/src/stmt.rs`, `lower_repeat`): the count is evaluated once, coerced to an integer (Verilog-AMS `real` $\rightarrow$ `integer` = round-to-nearest, via `FIcast`), and the body is run that many times as a counted loop. The integer counter is carried by a header `phi` spliced into the top of the loop-condition block (`FuncCursor::at_first_inst`), so no new `PlaceKind/persistent-state` surface was needed.

Verified behaviour ([examples/repeat_examples/](#))

```
count=0      -> 0 iters    V(b)=1.00000 (body never runs)
count=1..10  -> exact      integer counts run exactly
count=3.4    -> 3 iters    round down
count=3.6    -> 4 iters    round up
count=2.5    -> 3 iters    round half away from zero
```

Nested repeat loops multiply (repeat(P) repeat(Q) runs the inner body P*Q times), and repeat composes with for/while/if. The parser (15) and all touched-crate unit tests pass, and 40/40 example models compile — the shared token/AST/HIR changes are a strict superset (existing statements unaffected).

10. New feature: **disable** (loop break / early exit)

Verilog-A has no break/continue keywords — **disable <named_block>;** is the language's early-exit mechanism, and it was unsupported (the disable keyword tokenised but had no grammar/parser/lowering). Enhancement-9 implements it:

- Tokens/grammar/AST/parser: a DISABLE_STMT node (disable name ;) and a disable_stmt parser production (the DISABLE_KW keyword already existed).
- HIR: hir_def::Stmt::Disable { name }, plus the enclosing block's label is now recorded on Stmt::Block(name: Option<Name>) so a disable can be matched to it. Threaded through hir::Stmt, body lowering, pretty-printer, and hir_ty (validation; inference covers it via its catch-all).
- MIR lowering (hir_lower): a stack of enclosing named blocks and their MIR exit blocks is kept on LoweringCtx(disable_scopes). Entering a *named* begin : label ... end creates an exit block and pushes (label, exit); disable label branches to the matching exit and continues lowering in a fresh (dead) block, exactly like the existing \$fatal early-termination idiom. Un-named blocks are still lowered inline (no overhead).

Idioms and verification ([examples/disable_examples/](#)) Both loop-control idioms are built from disable and verified end-to-end:

```
break      (named block wraps the loop; disable it -> loop exits, code after runs)
  STOP=2/4/8 -> 2/4/8 iterations  V(b)=0.667 / 0.800 / 0.889  PASS
continue (loop *body* is the named block; disable it -> skip the iteration)
  8 iterations, 4 contribute      V(b)=0.800                      PASS
```

Disabling the whole analog block terminates it (contributions after the disable are skipped) — also verified. An unresolved block name degrades to a no-op. The parser (15) and touched-crate unit tests pass, and 41/41 example models compile.

11. Diff summary

File	Kind of change
openvaf/hir_lower/src/expr.rs	Read real table data (noise_table_data, read_noise_table_file, eval_const_real helpers) from an inline array or a file resolved relative to the root source file; pass it to NoiseTable::new instead of the [(0.0,0.0)] placeholder
openvaf/hir/src/db.rs	New CompilationDB::root_file_dir() accessor for resolving source-relative paths

File	Kind of change
openvaf/hir_lower/src/callbacks.rs	Replaced the stale <code>// TODO: read from disk with a doc comment</code> describing the now-populated <code>NoiseTable::new</code> contract
openvaf/sim_back/src/topology/lineralize.rs	Crash fix: read <code>arg0</code> defensively so zero-argument noise operators (<code>noise_table</code>) no longer panic (§1, item 2)
openvaf/osdi/src/load.rs	Codegen: <code>build_noise_table_interp</code> emits <code>log10</code> -domain piecewise-linear, endpoint-clamped interpolation for <code>load_noise</code> ; <code>load_noise_params</code> returns a documented zero fallback for tables (§2.3)
examples/noise_examples/	New analytically-verified example suite for all four noise operators (<code>.va</code> models, <code>.noise</code> decks, data files, <code>verify_noise.py</code> , <code>README.md</code> , <code>plot</code>)
openvaf/hir_def/src/data.rs	localparam fix: <code>ParamData</code> now carries <code>is_local</code> , populated from the item tree (§5)
openvaf/hir/src/lib.rs	localparam fix: new <code>Parameter::is_local()</code> accessor (§5)
openvaf/osdi/src/setup.rs	localparam fix: force the "given" flag to constant false for localparams in <code>setup_model/setup_instance</code> , so they are never externally overridable (§5)
examples/localparam_examples/	New end-to-end verification of localparam non-overridability, incl. derived-localparam tracking (<code>rdiv.va</code> , <code>verify_localparam.py</code> , <code>README.md</code>)
openvaf/parser/src/grammar/items/module.rs	ground fix: <code>net_decl</code> now accepts an optional net-type after the discipline, so <code>electrical ground gnd</code> ; parses like the other three orderings (§6)
examples/ground_examples/	New end-to-end verification of the ground net-type across all four declaration orderings (<code>rgnd.va</code> , <code>verify_ground.py</code> , <code>README.md</code>)
openvaf/hir_def/src/builtin.rs	regression fix: removed the <code>is_unsupported()</code> entries for <code>slew/transition/last_crossing/zi_*</code> that Enhancement-8's <code>builtin.rs</code> regeneration erroneously re-added, shadowing their Enhancement-6 implementation (§7)
openvaf/hir_def/src/body.rs	string-variable fix: an uninitialized string variable now defaults to <code>""</code> instead of hitting <code>unreachable!</code> and crashing (§8)
examples/vartype_examples/	New end-to-end verification of <code>real/integer/string</code> analog-block variables, incl. an uninitialized string (<code>vartypes.va</code> , <code>verify_vartypes.py</code> , <code>README.md</code>)
openvaf/tokens/src/parser/generated.rs, openvaf/syntax/veriloga.ungram, openvaf/syntax/src/ast/generated/nodes.rs	repeat loop: <code>REPEAT_KW/REPEAT_STMT</code> tokens + <code>RepeatStmt</code> AST node (§9)
openvaf/parser/src/grammar/stmts.rs	repeat loop: <code>repeat_stmt</code> parser production (§9)
openvaf/hir_def/src/expr.rs, body/lower.rs, body/pretty.rs, openvaf/hir/src/body.rs, openvaf/hir_ty/src/inference.rs, validation/body.rs	repeat loop: <code>Stmt::Repeat</code> through the HIR + type-check/validate (§9)

File	Kind of change
openvaf/hir_lower/src/stmt.rs	repeat loop: lower_repeat builds the counted loop with a header-phi integer counter (§9)
examples/repeat_examples/	New end-to-end verification of the repeat loop (repeat_demo.va, verify_repeat.py, README.md)
openvaf/tokens/src/parser/generated.rs, openvaf/syntax/src/ast/generated/nodes.rs, openvaf/parser/src/grammar/stmts.rs	disable : DISABLE_STMT token + DisableStmt AST node + disable_stmt parser (§10)
openvaf/hir_def/src/expr.rs, body/lower.rs, body/pretty.rs, openvaf/hir/src/body.rs, openvaf/hir_ty/src/validation/body.rs	disable : Stmt::Disable + block-label recording on Stmt::Block through the HIR (§10)
openvaf/hir_lower/src/ctx.rs, stmt.rs	disable : LoweringCtx::disable_scopes named-block-exit stack; branch to the matching exit on disable (§10)
examples/disable_examples/	New end-to-end verification of disable as break/continue (break_demo.va, continue_demo.va, verify_disable.py, README.md)

No OSDI ABI header extension and no **ngspice-46** C-side changes were needed — see §3.

12. Deferred to follow-up work

- **log**-power tables. The interpolation is linear-in-power; a `_log` variant that also interpolates the *power* column in log space (log-log interpolation) is not implemented, matching upstream's storage convention (`NoiseTable::new` never transforms the power column). Documented in §2.4.
- A real diagnostic for a missing/malformed `noise_table` file or an odd-length inline array (currently a graceful degrade to an empty/zero table), consistent with other soft-degradation sites in this codebase.
- **load_noise_params** for tables (only relevant to non-ngspice OSDI hosts that consume that ABI slot).

Enhancement-10 — Verilog-A statistical / random-number system functions

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory, on top of `version10/` (Enhancement-9, noise sources + repeat/disable/localparam/ground/string-var fixes), to implement the Verilog-AMS random-number and statistical-distribution system functions:

- `$random`, `$arandom`
- `$rdist_uniform`, `$rdist_normal`, `$rdist_exponential`, `$rdist_poisson`, `$rdist_chi_square`, `$rdist_t`, `$rdist_erlang` (real-valued)
- `$dist_uniform`, `$dist_normal`, `$dist_exponential`, `$dist_poisson`, `$dist_chi_square`, `$dist_t`, `$dist_erlang` (integer-valued)

All work is in `version11/` only; all simulation verification uses `version11/ngspice-46`'s own locally built binary and `version11/OpenVAF-master`'s own `openvaf-r`.

0. Scope and the starting point

A gap analysis of the `version11` baseline showed the random/statistical family was fully recognised by the front-end but rejected before codegen:

- `openvaf/hir_ty/src/builtin.rs` already declares complete, multi-form type signatures for every function (seeded / const-seed / named variants), so calls `type-check`.
- But `openvaf/hir_def/src/builtin.rs`'s `is_unsupported()` listed all of them, so `hir_ty/src/validation/body.rs` emitted **UnsupportedFunction** and the build stopped. No MIR lowering and no runtime existed.

So, exactly like Enhancement-9's `noise_table`, the front-end was complete and the back-end was missing. Enhancement-10 removes the `is_unsupported()` gate, adds MIR lowering, and adds a small deterministic RNG runtime to the OSDI standard library. `ngspice-46` needed no changes — the functions lower to an ordinary pure callback resolved generically in `general_callbacks`, exactly like `$simplparam`.

1. Semantics: deterministic, read-only seed (the key design decision)

The LRM nominally treats the seed as `inout`: each call advances it in place. In a circuit simulator that model is actively harmful. Analog-block variables in OpenVAF are persistent hidden state that is re-read at the start and stored at the end of every `eval()` call (`osdi/src/eval.rs` calls `store_hidden_state` unconditionally — once per Newton iteration, not once per accepted timepoint). An in-place-advancing seed would therefore produce a *different* random value on every Newton iteration, so any residual built from it would never converge, and the DC/transient problem would be ill-posed.

Enhancement-10 instead defines each draw as a pure, deterministic function of (**seed_value**, **call-site salt**, **parameters**), with no writeback:

- Reproducible — the same seed always yields the same value.
- Stable across the nonlinear solve — the value does not change between Newton iterations or accepted timepoints, so it is safe to use anywhere, including directly in a residual. This is what makes it usable in a SPICE engine.
- Per-instance variation — because the seed can be an instance parameter (or any expression), distinct instances draw distinct, independent samples. This is the dominant real use case: process/mismatch variation.
- Independent streams per call site — the compiler mixes a unique per-call-site *salt* (the call `ExprId`) into the generator state, so two textual calls with the *same* seed variable are decorrelated (e.g. `$rdist_normal(s,0,1)` – `$rdist_normal(s,0,1)` is not identically zero).

Documented deviation from the LRM: the seed is not advanced in place, so repeatedly calling the *same* call site with the *same* seed value (e.g. drawing in a loop without changing the seed) returns the same value. To draw a *sequence*, vary the seed (`$rdist_normal(seed+i, ...)`), or advance a seed variable

yourself). This is the standard, convergence-safe interpretation for a circuit simulator and is sufficient for statistical device modelling.

2. Front-end: ungating (**hir_def/src/builtin.rs**)

`is_unsupported()` previously returned `true` for `dist_*`, `rdist_*`, `random` and `arandom`. Those arms are removed (the file/string-I/O and node-alias builtins remain unsupported). A comment records that Enhancement-10 owns their lowering, mirroring the existing note about the Enhancement-6 analog operators. `validation/body.rs` then falls through to its `_ => ()` arm, so the calls are accepted and reach lowering.

3. MIR lowering (**hir_lower**)

3.1 The callback (callbacks.rs**)** A single new `CallbackKind::Rng(RngFun)` variant is added, where `RngFun` enumerates the nine distinct underlying generators (`Random`, `Uniform`, `UniformInt`, `Normal`, `Exponential`, `Poisson`, `ChiSquare`, `StudentT`, `Erlang`). It is a pure callback (`has_sideeffects: false`) whose MIR call arguments are (`seed: int`, `salt: int`, `params...: real`); `RngFun` maps to the runtime function name and its real-parameter count. Because the salt is passed as an *argument* rather than stored in the kind, all call sites of one `RngFun` share a single interned callback (efficient), while their distinct salt arguments keep them uncorrelated and un-CSE'd.

3.2 Dispatch (expr.rs**)** Each builtin lowers to `lower_rng`, which reads the seed value, appends the call `ExprId` as the salt constant, appends the (real) distribution parameters, and emits the callback. Return-type handling follows OpenVAF's signature table, which types every `$dist_*/$rdist_*` as `Real` and only `$random/$arandom` as `Integer`:

- `$random/$arandom` → `ficast` the real callback result to an integer MIR value.
- `$rdist_*` → return the real result directly.
- `$dist_*` → the LRM defines these as integer-valued, so round to the nearest integer (`rng_round_real = floor(x + 0.5)`) but keep the value `real` (its OpenVAF type is `Real`; returning an `int` MIR value here would be reinterpreted bit-for-bit and yield garbage — this was a real bug caught in testing).

Two robustness details:

- `lower_num_as_real` coerces integer distribution parameters to `real` using the *post-lowering* type (from `needs_cast`), because the parameter type is signature-dependent — at least one upstream const-seed signature even mixes a `Val(Real)` into an otherwise-integer form. Driving the coercion by the actual lowered type avoids a double cast.
- Autodiff already treats any non-`ddx` `Call` as zero-derivative (`mir_autodiff::is_zero_call`), so a random value feeding a residual is correctly constant with respect to the unknowns — no special-casing needed.

4. Runtime (**osdi/src/compilation_unit.rs**, **osdi/stdlib.c**)

`general_callbacks` resolves `CallbackKind::Rng(fun)` to the matching `osdi_rng_*` C function (built type (`i32, i32, double...`) → `double`, no prepended state). Because both `eval()` and the `setup/init` function call `general_callbacks`, the functions work in the analog block, in `@(initial_step)` and in instance/parameter initialisation.

`stdlib.c` gains the nine `osdi_rng_*` functions plus extern declarations for `sqrt/exp/cos` (the file is compiled `-DNO_STD`, so like the pre-existing `log` these resolve against the host `libm` at OSDI load — no `ngspice` change). The core generator is `splitmix64` seeded by an avalanche hash of (`seed, salt`); each uniform advances a *local* 64-bit state, so multi-uniform distributions (normal via Box-Muller, chi-

square/student-t as sums of squared normals, erlang as a sum of exponentials, poisson via Knuth) draw independent underlying uniforms within one call. Everything is a pure function of (seed, salt).

5. Build-system fix (**osdi/build.rs**) — necessary for the runtime to load

The versionN workflow copies the whole target/ directory from the previous version. `osdi/build.rs` located `stdlib.c` via `stdx::project_root()`, which bakes `CARGO_MANIFEST_DIR` at `stdx`'s compile time; the copied build-script binary therefore still pointed at an older checkout's `stdlib.c` (`.../version6/...` was observed), so edits to `version11/.../stdlib.c` were silently ignored and the new `osdi_rng_*` symbols were missing at load (`stdlib` function `osdi_rng_uniform` is missing). Fixed by reading the fresh, per-invocation `CARGO_MANIFEST_DIR` env var instead — it always points at the crate being built. This is a general robustness fix for the copy-the-target workflow, not specific to this feature.

6. Verification (**examples/rng_examples/**)

`rng_demo.va` fixes $V(p, n)$ to a single draw selected by a `dist` model parameter and stream-selected by seed. Since each draw is a stable, reproducible function of (seed, call site), instantiating N devices with seeds $1..N$ and running one `.op` yields N i.i.d. samples. `verify_rng.py` compiles with this version's `openvaf-r`, builds the deck, runs this version's `ngspice` as a subprocess (a bare heredoc misbehaves in some shells — a known project note), parses the node voltages and checks, for $N = 4000$ samples each:

check	result
<code>\$rdist_uniform(-2,6)</code> mean/std vs $(2, \sqrt{(64/12)})$	PASS
<code>\$rdist_normal(3,2)</code> mean/std	PASS
<code>\$rdist_exponential(4)</code> mean/std (= mean)	PASS
<code>\$rdist_poisson(5)</code> mean/std (var = mean)	PASS
<code>\$rdist_chi_square(4)</code> mean/std (= $k, \sqrt{(2k)}$)	PASS
<code>\$rdist_erlang(k=3, mean=6)</code> mean/std	PASS
<code>\$rdist_t(5)</code> mean ≈ 0 , std $\approx \sqrt{(5/3)}$	PASS
<code>\$random</code> sign balance, magnitude spread, integrality	PASS
<code>\$dist_uniform(0,6)</code> fair-die: faces $\{0..6\}$, mean ≈ 3	PASS
<code>\$dist_normal(5,2)</code> integer-valued, mean ≈ 5 , Sheppard-corrected std	PASS
reproducibility (same seeds $\rightarrow$ identical) & independence	PASS

24/24 checks PASS. Touched-crate unit tests pass (`sim_back` 24/24; `hir_ty`, `hir_lower`, `hir_def` data-tests unchanged).

`plot_mc.py` additionally exercises all three analysis types on a Monte-Carlo RC low-pass (`rc_mc.va`), whose gain/R/C are perturbed per instance by `$rdist_normal` (three independent streams from one seed via the call-site salt). A single `ngspice` job runs `.dc`, `.ac` and `.tran` over $N=30$ randomized instances and produces `mc_dc.png` (random DC gains, 1.00 ± 0.17), `mc_ac.png` (spread of Bode magnitudes / cutoffs about $f_c \approx 159$ Hz) and `mc_tran.png` (spread of 1 V step responses about $\tau = 1$ ms), each overlaid on the nominal response. This confirms the draws are stable within and across DC/AC/transient solves — the convergence property §1 is built around.

7. Diff summary

File	Kind of change
openvaf/ hir_def/ src/ builtin.rs	Removed the <code>is_unsupported()</code> entries for <code>random/arandom/dist_*/rdist_*</code> (they now lower), with an explanatory note (§2)
openvaf/ hir_lower/ src/ callbacks. rs	New <code>RngFun</code> enum (9 generators, <code>stdlib_name()/num_real_params()</code>) + pure <code>CallbackKind::Rng(RngFun)</code> variant and its <code>FunctionSignature</code> (§3.1)
openvaf/ hir_lower/ src/lib.rs	Re-export <code>RngFun</code>
openvaf/ hir_lower/ src/expr. rs	Lowering for all 16 builtins: <code>lower_rng</code> (seed + call-site salt + real params → callback), <code>lower_rng_seed</code> , <code>lower_num_as_real</code> , <code>rng_round_real</code> ; <code>Real/Integer</code> return handling (§1, §3.2)
openvaf/ osdi/src/ compilation_ unit.rs	<code>general_callbacks</code> resolves <code>CallbackKind::Rng(fun)</code> to the <code>osdi_rng_*</code> runtime function with the right LLVM type (§4)
openvaf/ osdi/ stdlib.c	Nine deterministic <code>osdi_rng_*</code> runtime functions (splitmix64 core, Box-Muller / Knuth / sums) + extern <code>sqrt/exp/cos</code> (§4)
openvaf/ osdi/ build.rs	Locate <code>stdlib.c</code> via the per-invocation <code>CARGO_MANIFEST_DIR</code> instead of <code>stdx::project_root()</code> , so a copied target/ cannot make the build read another checkout's <code>stdlib.c</code> (§5)
examples/ rng_ examples/	New analytically-verified example suite: <code>rng_demo.va</code> + <code>verify_rng.py</code> (all 16 functions, 24/24 moment/integrality checks) and <code>rc_mc.va</code> + <code>plot_mc.py</code> (DC/AC/transient Monte-Carlo → <code>mc_dc.png/mc_ac.png/mc_tran.png</code>), <code>README.md</code> (§6)

Enhancement-11 — Verilog-A file I/O and string-formatting system functions

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory, implementing the Verilog-AMS file I/O, file-descriptor, string-formatting and file-reading system functions on top of Enhancement-10 (random / statistical functions, same folder):

- Output: `$fopen`, `$fclose`, `$fdisplay`, `$fwrite`, `$fstrobe`, `$fmonitor`, `$fdebug`, `$fflush`, `$ftell`, `$fseek`, `$rewind`, `$feof`
- String / reading: `$swrite`, `$sformat`, `$sscanf`, `$fgets`, `$fscanf`, `$ferror`

All work is in `version11/` only; verification uses `version11/ngspice-46`'s own binary and `version11/OpenVAF-master`'s own `openvaf-r`.

The only file/string system functions still unsupported are connectivity aliasing (`$simprobe`, `$analog_node_alias`, `$analog_port_alias`) and the plusarg functions (`$test_plusargs`, `$value_plusargs`).

This document was originally written for the output-only set (§1–§5 below); §6 covers the string-formatting and reading functions added afterward.

0. Starting point

Like the random functions in Enhancement-10, the front-end already type-checked all of these (full signatures in `hir_ty/src/builtin.rs`), but they were gated in `hir_def/src/builtin.rs`'s `is_unsupported()` and rejected before codegen. The console `$display` family, by contrast, was fully working -- and since `$fdisplay/$fwrite/...` are exactly `$display/$write/...` aimed at a file, the implementation is mostly *reuse* of the existing formatting machinery plus a small runtime file-descriptor table.

1. Design

1.1 Descriptor table (`osdi/stdlib.c`) `$fopen` must return an *integer* descriptor, but a host `FILE*` is 64-bit, so the runtime keeps a small module-global table of open `FILE*`s and hands out 1-based indices into it (index 0 reserved, so a returned 0 means "failed", per the LRM). Every `$f*` descriptor function looks the `FILE*` back up by index. `FILE*` is treated opaquely as `void*`; `fopen/fputs/fclose/fflush/fseek/ftell/rewind/feof` are declared extern and resolve against the host libc at OSDI load -- exactly like the pre-existing `log`, so `ngspice` needs no changes.

1.2 Output path reuses `$display` `$fdisplay/$fwrite/$fstrobe/$fmonitor/$fdebug` share the entire `snprintf`-based formatting path with `$display` (`fmt::ins_display + osdi::print_callback`, handling `%g/%d/%h/%s/%b/engineering %r`, etc.). The only differences: the file variant takes the descriptor as an extra leading call argument, and the backend routes the formatted string to `osdi_fputs(fd, s)` instead of the console `osdi_log`. `$fwrite` omits the trailing newline; `$fstrobe/$fmonitor` are treated as `$fdisplay` (one write per evaluation).

1.3 Execution context (important semantic note) File-I/O writes that depend only on parameters/constants are placed by `sim_back` in the model's initialization code and run once per instance -- ideal for exporting model parameters and characterization tables. Writes that depend on node voltages / the operating point are a different matter: `sim_back` splits node-dependent code into the per-iteration `eval` path, which (a) would run on every Newton iteration and (b) separates such writes from the `$fopen/$fclose` pair (which are parameter-level and stay in `init`), so the descriptor is already closed by the time they run. Enhancement-11 therefore targets the sound use case -- parameter/setup-derived export -- and the example writes only parameter-derived data. (The descriptor table itself is not guarded against concurrent access, so I/O should stay in these setup contexts.)

2. Implementation

Layer	Change
Ungate	hir_def/src/builtin.rs: removed the implemented functions from <code>is_unsupported()</code> ; the reading/string ones stay gated (<code>\$0</code>).
Callbacks	hir_lower/src/callbacks.rs: added <code>to_file: bool</code> to <code>CallBackKind::Print</code> ; new <code>CallBackKind::Fopen</code> and <code>CallBackKind::FileOp(FileOp)</code> (<code>Close/Flush/FlushAll/Eof/Tell/Rewind/Seek</code>), all side-effecting.
Lowering	hir_lower/src/fmt.rs: <code>ins_display</code> takes an optional descriptor, passed as the second call arg and flagged <code>to_file</code> . hir_lower/src/expr.rs: arms for every file builtin (<code>\$fopen</code> synthesises a default "w" mode; <code>lower_file_op</code> for the descriptor ops).
Backend	osdi/src/compilation_unit.rs: <code>general_callbacks</code> resolves <code>Fopen/FileOp</code> to the <code>osdi_*</code> runtime functions; <code>print_callback</code> gained a <code>to_file</code> path that routes to <code>osdi_fputs</code> .
Runtime	osdi/stdlib.c: the descriptor table + <code>osdi_fopen/osdi_fputs/osdi_fclose/osdi_fflush/osdi_fflush_all/osdi_feof/osdi_ftell/osdi_fseek/osdi_frewind</code> , plus the extern libc declarations.

3. Two bugs found and fixed during bring-up

1. **fun shadowing in `print_callback`**. The formatted-text sink extracted the file descriptor with `LLVMGetParam(fun, 2)` at the *end* of the hand-built callback -- but `fun` had already been rebound to the `snprintf` intrinsic earlier in the function, so it was reading `snprintf`'s parameter, not the callback's descriptor. Every write went to garbage/fd 0 and silently produced an empty file. Fixed by capturing the descriptor param up front (`file_fd`), while `fun` still refers to the callback.
2. IPO mis-specialisation of the descriptor table. The `osdi_f*` functions and the descriptor table are all internal after linking, so LLVM's aggressive interprocedural passes "proved" the table contents at compile time -- folding `$fopen`'s returned index to a constant and rewriting `osdi_fputs` down to a fixed `table[0]` (always NULL), so no write reached the file. Fixed by marking the table `volatile` (forcing real runtime loads/stores) and the entry points `noinline`.

4. Verification (`examples/fileio_examples/`)

`fileio_demo.va` is a resistor that also exports a characterization report -- its parameters and a computed $I = V/R$ table -- to `fileio_out.txt` at initialization. `verify_fileio.py` runs a .op and checks every line against the closed-form values, exercising `%g/%d/%h/%s`, the newline-less `$fwrite` (fragment join), and `$ftell` (byte offset). `fileio_seek.va` separately checks `$rewind` and `$fseek` by overwriting a file in place (`0123456789 -> XY234**789`).

```
$ python3 verify_fileio.py
...
ALL PASS (9/9)
```

Regression: console `$display/$strobe` still format and print correctly (shared `print_callback`); Enhancement-10's `verify_rng.py` still passes 24/24; `sim_back` unit tests 24/24; `hir_*` data-tests unchanged.

6. String-formatting and file-reading functions

Added afterward: `$swrite`, `$sformat`, `$sscanf`, `$fgets`, `$fscanf`, `$ferror`. These all involve writing back into a *string/variable argument*, so the enabler is a small helper -- `hir::into_variable(expr)` (via a new `Ty::unwrap_var`) -- that recovers the destination `Variable` from a `Var(..)` argument, after which a normal `def_place(PlaceKind::Var(..), value)` stores the result.

6.1 String formatting (**\$swrite** / **\$sformat**) These are `$write/$display` that format into a string variable instead of a sink. The `Print` callback's `to_file: bool` was generalised to a `PrintDst` enum (`Console / File / String`); for `String`, `print_callback` returns the freshly `snprintf`-ed `char*` (rather than calling `osdi_log/osdi_fputs`), and the lowering stores it into the destination string variable. `$swrite` and `$sformat` lower identically (both format args `[1..]` -- `ins_display` already treats a leading format-string literal as the format and otherwise auto-formats, which is exactly the `$sformat` vs `$swrite` distinction).

6.2 Reading (**\$fgets**, **\$fscanf**, **\$sscanf**, **\$ferror**) Small `osdi_*` runtime helpers back these:

- `$fgets(str, fd) -> osdi_fgets(fd)` reads one line (returned as a `char*`, stored into `str`); the return count is `osdi_strlen` of it.
- `$ferror(fd, str) -> osdi_ferror_msg/osdi_ferror_code`.
- `$sscanf/$fscanf` share `lower_scanf`: `osdi_scanf_begin(input)` resets a module-global cursor (`$fscanf` first reads a line via `osdi_fgets`), then one `osdi_scan_int/_real/_str` per target variable pulls the next whitespace-delimited token (parsed by the variable's type, not by interpreting the C format string -- adequate for the usual whitespace-separated input), each stored via `def_place`; `osdi_scanf_count` returns the match count. The cursor/counter are volatile for the same IPO reason as §3.2.

An upstream signature bug had to be fixed: `$sscanf` and `$fscanf` were both mapped to `FDISPLAY_FUN` (leading `Val(Integer)`, return `Void`) -- wrong for `$sscanf` (whose first argument is the input *string*) and for the return type (both return the `Integer` match count). Split into `SSCANF_FUN` (`Val(String)` + `Integer` return) and `FSCANF_FUN` (`Val(Integer)` + `Integer` return).

6.3 Verification (**examples/stringio_examples/**) `stringio_demo.va` formats strings (`$sformat/$swrite`), parses a literal (`$sscanf`), round-trips a file read (`$fgets` and `$fscanf` over a file the model itself wrote) and queries `$ferror`, writing a report checked by `verify_stringio.py`: 6/6 PASS. Same setup-context caveat as the output functions (§1.3) applies.

5. Diff summary

File	Kind of change
<code>openvaf/hir_def/src/builtin.rs</code>	Removed the implemented file functions from <code>is_unsupported()</code> ; kept the reading/string ones gated, with a note (§0)
<code>openvaf/hir_lower/src/callbacks.rs</code>	<code>Print{ to_file }</code> ; new <code>Fopen</code> , <code>FileOp</code> (<code>FileOp</code>) callbacks + <code>FileOp</code> enum and their signatures (§2)
<code>openvaf/hir_lower/src/lib.rs</code>	Re-export <code>FileOp</code>
<code>openvaf/hir_lower/src/fmt.rs</code>	<code>ins_display</code> gained an optional descriptor argument and emits <code>Print{to_file}</code> (§1.2)
<code>openvaf/hir_lower/src/expr.rs</code>	Lowering for <code>\$fopen/\$fclose/\$fdisplay/\$fwrite/\$fstrobe/\$fmonitor/\$fdebug/\$fflush/\$ftell/\$fseek</code> lower_file_op helper (§2)
<code>openvaf/osdi/src/compilation_unit.rs</code>	<code>print_callback</code> file-routing path (<code>to_file</code> , descriptor captured up front) + <code>general_callbacks</code> arms for <code>Fopen/FileOp</code> (§2, §3)

File	Kind of change
openvaf/osdi/ stdlib.c	Runtime descriptor table + osdi_f* functions + extern libc decls; volatile table + noinline to defeat IPO mis-specialisation (§1.1, §3)
examples/ fileio_ examples/ §6 additions:	New verified example suite (fileio_demo.va, fileio_seek.va, verify_fileio.py, README.md) (§4)
openvaf/hir_ ty/src/types. rs	string-formatting / reading functions Ty::unwrap_var() accessor (§6)
openvaf/hir/ src/body.rs	into_variable(expr) -- recover a Variable from a Var(..) argument (§6)
openvaf/hir_ ty/src/ builtin.rs	Fixed \$sscanf/\$fscanf signatures: new SSCANF_FUN (Val(String)→Integer) / FSCANF_FUN (Val(Integer)→Integer), replacing the wrong shared FDISPLAY_FUN (§6.2)
openvaf/hir_ lower/src/ callbacks.rs	Generalised Print{ to_file } → Print{ dst: PrintDst }(Console/File/ String); new ScanBegin/Scan(ScanKind)/ScanCount/Fgets/StrLen/ErrorMsg/ErrorCode callbacks (§6)
openvaf/hir_ lower/src/ lib.rs	Re-export PrintDst, ScanKind
openvaf/hir_ lower/src/ fmt.rs	ins_display takes a PrintDst; returns the formatted string for String (§6.1)
openvaf/hir_ lower/src/ expr.rs	Lowering for \$swrite/\$sformat/\$fgets/\$ferror/\$sscanf/\$fscanf; lower_scanf helper (§6)
openvaf/hir_ def/src/ builtin.rs	Ungated the 6 string/reading functions (only \$simprobe/\$*_alias/plusargs remain)
openvaf/osdi/ src/ compilation_ unit.rs	print_callback PrintDst::String return path; general_callbacks arms for the scan/read helpers (§6)
openvaf/osdi/ stdlib.c	osdi_fgets/osdi_strlen/osdi_ferror_* + the volatile-cursor scanner (osdi_scanf_begin/osdi_scan_*/osdi_scanf_count) + extern libc decls (§6.2)
examples/ stringio_ examples/	New verified example suite (stringio_demo.va, verify_stringio.py, README.md) (§6.3)

Enhancement-12 — the last Verilog-A system functions: probe / alias / plusargs

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory, implementing the final group of previously-gated Verilog-AMS system functions, on top of Enhancement-11 (file I/O + string functions, same folder):

- `$simprobe`
- `$analog_node_alias`, `$analog_port_alias`
- `$test$plusargs`, `$value$plusargs`

After this, no system function is gated as unsupported in `hir_def::Builtin::is_unsupported()` -- it now always returns false.

All work is in `version11/` only; verification uses `version11/ngspice-46`'s own binary and `version11/OpenVAF-master`'s own `openvaf-r`.

1. Why these are "mechanism-unavailable" fallbacks, not full features

Unlike the I/O functions, these five have no underlying mechanism in the OSDI/ngspice target, so there is nothing to wire them to:

- `plusargs` (`$test$plusargs`, `$value$plusargs`) query simulator *command-line* `+args`. OSDI models are loaded objects, not invoked with a command line, and ngspice passes no `plusargs` to them -- so no `plusarg` is ever present.
- `$simprobe` reads an arbitrary named simulator quantity of another instance. OSDI exposes no such generic cross-instance probe API.
- `$analog_node_alias`/`$analog_port_alias` establish a runtime hierarchical *alias* for a node/port (a `connectmodule`/hierarchy feature). OSDI compact models have no runtime hierarchical-aliasing mechanism.

The LRM defines a well-behaved result for each when the mechanism is unavailable, and that is exactly what Enhancement-12 emits. So these functions now compile and run with predictable, defined semantics (a model that uses them is no longer rejected), which is the correct outcome for this target -- the same fallback other simulators give for OSDI models. There is deliberately no runtime callback and no ngspice change: each lowers to a compile-time constant.

2. Implementation

Ungated in `hir_def/src/builtin.rs` (the `is_unsupported()` match body is now just `_ => false`), and lowered in `hir_lower/src/expr.rs` to constants:

Function	Return type	Lowers to
<code>\$test\$plusargs(str)</code>	Bool	FALSE -- no plusarg present
<code>\$value\$plusargs(str, str)</code>	Bool	FALSE (OpenVAF's signature has no in-out value arg, so there is nothing to extract into)
<code>\$analog_node_alias(node, str)</code>	Integer	0 -- no alias created
<code>\$analog_port_alias(port, str)</code>	Integer	0
<code>\$simprobe(inst, quant)</code>	Real	0.0 -- probe unavailable
<code>\$simprobe(inst, quant, default)</code>	Real	the supplied default (via <code>lower_expr(args[2])</code>)

The node/port arguments are type-checked as Nodes during inference but are not lowered (the result is a constant), so referencing them adds nothing to the device topology.

Naming note: the plusarg functions use the IEEE \$-separated spelling \$test\$plusargs / \$value\$plusargs in Verilog-A source (registered that way in syntax/src/name.rs), not underscores.

3. Verification (**examples/alias_examples/**)

alias_demo.va calls all five (with and without a \$simprobe default) and writes the results to alias_out.txt; verify_alias.py runs a .op and checks them:

```
test_plusargs=0    value_plusargs=0
node_alias=0       port_alias=0
simprobe=0         simprobe_default=3.5
```

```
$ python3 verify_alias.py
```

```
...
```

```
ALL PASS (6/6)
```

Regression: sim_back unit tests 24/24; hir_* data-tests unchanged; Enhancement-10 verify_rng.py 24/24, Enhancement-11 verify_fileio.py 9/9 and verify_stringio.py 6/6 all still pass.

4. Diff summary

File	Kind of change
openvaf/hir_def/src/builtin.rs	Ungated the last five functions; is_unsupported() is now unconditionally false (§2)
openvaf/hir_lower/src/expr.rs	Constant lowering for \$test\$plusargs/\$value\$plusargs (→ FALSE), \$analog_node_alias/\$analog_port_alias (→ 0), \$simprobe (→ default or 0.0) (§2)
examples/alias_examples/	New verified example suite (alias_demo.va, verify_alias.py, README.md) (§3)

Enhancement-13 — `limexp()`: investigation, kept stateless

This document records an investigation into the Verilog-AMS `limexp()` limited exponential in OpenVAF-r (version11/), and the deliberate decision to keep its existing stateless implementation rather than adopt a stateful prev-iteration step-limiting one. No functional change was made (only an explanatory code comment was added at the `limexp` lowering site); this doc exists so the decision and its rationale are not lost.

1. Current (kept) implementation — stateless cutoff linearisation

`limexp(x)` is lowered in `hir_lower/src/expr.rs` to:

- `exp(x)` for $x \leq \ln(1e30)$;
- above that, the exponential is continued along its tangent line ($1e30 * (1 + (x - \ln(1e30)))$), so the value and derivative stay finite.

This is a pure function of the current argument, so it is exact and correct in every context (DC sweep, operating point, AC/noise, transient). It provides the practical benefit of `limexp` -- bounding the derivative and preventing overflow of the exponential nonlinearity -- and matches what a number of simulators ship. A diode $I(a, c) \leftarrow I_s * (\text{limexp}(V(a, c)/V_t) - 1)$ reproduces the analytic $I_s * (\exp(V/V_t) - 1)$ curve across the operating range.

2. Why the stateful step-limiting version was NOT adopted

The LRM describes `limexp` as keeping the argument's previous value and limiting its change between iterations (pnjlim-style). That version was implemented and tested, and then reverted because it produces incorrect DC values:

- The step-limited value is $\exp(x_{\text{lim}})$, where x_{lim} is derived from the *previous evaluation's* argument (held in a `per_eval()` `EventState` slot, gated by `EnableLim`).
- But the simulator judges convergence from the node voltages, not from `limexp's` internal x_{lim} . When the circuit converges while x_{lim} still lags x -- e.g. an ideal voltage source fixes the argument, so there is no Newton loop to unwind the limiting -- the returned current is $\exp(x_{\text{lim}}) \neq \exp(x)$. A diode I-V sweep came out wrong at roughly half the bias points, including high-current ones. A DC characterisation sweep is a first-class use case, so this is disqualifying.

The only way to step-limit while keeping the converged value exact is SPICE's limiting-RHS correction, $\text{lim_rhs} = J(x_{\text{lim}}) \cdot (x_{\text{lim}} - x)$ (it cancels at convergence). OpenVAF *has* this machinery (`sim_back/src/dae/builder.rs`, `build_lim_rhs`), but it only applies the correction to values registered as circuit unknowns (node voltages / branch currents) -- the same reason `$limit/start_limit` require their probe to be a bare voltage/current. `limexp's` argument is a derived quantity (e.g. V/V_t), which is not an unknown, so the correction is silently skipped and the DC value is wrong.

3. What a correct version would require (not done)

Extending `build_lim_rhs` (and the derivative tracking) to limit derived arguments -- applying the chain rule from the limited quantity back to the underlying node/branch unknowns so the RHS correction is emitted. That is real, higher-risk surgery in `sim_back's` DAE builder + autodiff, and was judged out of scope; the stateless version is correct in the meantime. Reusable primitives identified if it is ever pursued: `new_event_state()` (a `per_eval()`-persisted, zero-derivative scalar) and `ParamKind::EnableLim` (set by ngspice for DC/tran loads but not for small-signal AC/noise).

4. Diff summary

File	Kind of change
openvaf/ hir_lower/ src/expr.rs	Added an explanatory comment at the <code>limexp</code> arm documenting that it is intentionally stateless and why the stateful version is not correct here (§2). No behavioral change.

Enhancement-14 — Verilog-A array literals / aggregates

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory to implement array literals and aggregate operations on top of Enhancement-13. Before this, arrays existed only as Enhancement-3/4 "bus-style" declarations (`real [0:n] x;`) accessible one element at a time by a constant index (`x[0]`); a whole array could not be assigned, an array could not be a parameter, and a non-constant index was rejected.

Enhancement-14 adds three capabilities (all verified end-to-end through ngspice — see `examples/array_examples/`):

- A. Whole-array aggregate assignment — `c = '{v0, v1, ...}';` (and the brace-only `{...}` form) and array-to-array copy `c = d;`.
- B. Array-valued parameters — `parameter real [msb:lsb] c = '{...}';`, with a per-element default and per-element SPICE override (`c[0], c[1], ...`).
- C. Dynamic (non-constant) indexing — `c[i]` reads and writes with a runtime index, lowered to a select over the array's element variables.

All work is in `version11/` only; verification uses `version11/ngspice-46`'s own binary and `version11/OpenVAF-master`'s own `openvaf-r`. No OSDI ABI change and no ngspice change were needed — array parameters surface as ordinary scalar OSDI parameters (one per element), and everything else lowers to ordinary MIR.

0. Starting point and representation

Array variables/nets are already expanded into independent scalar entries at item-tree time: `real [0:2] c;` registers a `BusDecl` in `Module::var_arrays` and declares three scalar variables named `c[0]`, `c[1]`, `c[2]` (`hir_def::item_tree::lower::lower_var`). A constant bit-select `c[k]` resolves, via the synthesized path `c[k]`, to the corresponding scalar `VarId` (`hir_ty::inference::infe_bit_select`). Enhancement-14 builds entirely on this expansion: an aggregate operation is just a set of per-element scalar operations, and a dynamic index is a runtime select over the same elements.

A pre-existing infinite-loop bug was found and fixed while doing this: `Type::base_type` (`hir_def/src/types.rs`) looped while `let Type::Array{..} = self` (testing the unchanging `self` instead of the loop cursor `curr`), so any `base_type()` call on an array type hung the compiler. It is reached through `is_assignable_to/is_convertible_to`, so e.g. assigning an array literal to a scalar (`real x; x = '{...}';`) hung instead of reporting a type error. Now fixed (`= curr`), and such mismatches report a clean diagnostic.

A. Whole-array aggregate assignment

`Expr::Array` (an `'{...}'` literal) previously reached `hir_lower`'s `lower_array`, an unimplemented! ("arrays") — but only after the front-end had already rejected the bare array destination with a "requires a bit-select" error, so the `todo` was in practice unreachable. Enhancement-14 handles the whole statement in inference and desugars it to per-element assignments.

- **`hir_ty::inference`** — `try_infe_array_assignment` runs before ordinary assignment inference. If the destination is a bare reference to a `var_array`, it resolves the destination element `VarIds` (declaration order, `msb→lsb`, the order a literal fills) and type-checks the right-hand side:
 - an array literal `'{...}'` (each element inferred and cast to the element type; length checked against the destination),
 - or a bare reference to another array variable (`c = d`, element types and lengths checked).

The decomposition is recorded in `InferenceResult::array_assignments` as an `ArrayAssign::Literal, Copy`. Length/element-type mismatches reuse the existing `TypeMismatch` diagnos-

tic (rendered as e.g. *"expected real[0:3] value but found real[0:2] value"*), so no new diagnostic wiring was needed.

- **hir** — a new elaborated `Stmt::ArrayAssignment { assigns: Vec<ArrayAssignElem> }` is produced by `get_stmt` when a statement has an `array_assignments` entry; each element is `ArrayAssignElem::{{Val { dst, val }, Copy { dst, src }}}`.
- **hir_lower::stmt** — lowers `Stmt::ArrayAssignment` as its per-element scalar `def_places` (a `lower_expr` for a literal element, a `read_variable` for a copy source). Per-element `int→real` casts are applied by the usual `needs_cast` path.

B. Array-valued parameters

- **parser** (`grammar/items.rs::parameter_decl`) — accepts an optional `[msb:lsb]` width clause after the parameter type, mirroring `var_decl`. The AST grammar (`verilog.grammar`, generated `nodes.rs`) gains `ParamDecl::width()`.
- **hir_def::item_tree** — `Param` gains `array_index: Option<u32>` and `Module` gains `param_arrays: Vec<BusDecl>`. `lower_param` (at module scope) expands an array parameter into one scalar `Param` per element, named `c[lo]..c[hi]`, each tagged with its declaration-order position, and registers a `param_arrays BusDecl` so `c[i]` bit-selects resolve to the element parameters. (A width clause in an unsupported scope, or a non-constant width, degrades to a scalar parameter, as for array variables.)
- **hir_def::body** — the parameter default body query gives each element parameter *its own* default: the `array_index`-th element of the shared `'{...}'` literal (rather than the whole literal).
- **hir_ty::inference** — `find_param_array` (analogous to `find_var_array`) lets `inference_bit_select` resolve `c[k]` to the element `ParamId` (typed as `Ty::Param`), and lets the bare-reference guard reject an un-indexed array parameter.

Because each element is an ordinary parameter, it becomes an ordinary OSDI parameter automatically. `ngspice` reads its default and overrides it per-element by name — `.model m array_demo(w[2]=0.9)` sets only `w[2]`, leaving the others at their literal defaults. (Confirmed: `ngspice` accepts the bracketed parameter name.)

C. Dynamic (non-constant) indexing

A non-constant index can't resolve to one element, so it lowers to a runtime select over the elements. This applies to array variables (mutable state); array parameters are constant tables read by a constant index (a parameter table is indexed dynamically by first copying it into an array variable — see `examples/array_examples/array_demo.va`).

- **hir_ty::inference** — `inference_bit_select`, on a non-constant index of a variable array, records `dynamic_index_refs[expr] = DynArrayIndex { elems, index, lsb }` (element `VarIds` in ascending bit order) and types the read as the element type. A dynamic `write c[i] = v` is caught by `try_inference_dynamic_index_assignment` and recorded in `dynamic_index_assignments`. A literal index that failed to fold (e.g. an oversized integer) is *not* treated as dynamic — it stays a "non-constant bit-select index" diagnostic (`is_literal_index` guard), preserving existing behaviour and the huge_int_literal regression test.
- **hir** — exposes `dynamic_index(expr)` for reads and emits a new `Stmt::DynArrayAssignment { elems, index, lsb, value }` for writes.
- **hir_lower** — a dynamic read (`lower_dynamic_index_read`, intercepted at the top of `lower_expr`) builds `res = elems[0]; res = (i == lsb+k) ? elems[k] : res` for each `k`. A dynamic write updates every element conditionally: `elems[k] = (i == lsb+k) ? v : elems[k]`. Both use `Opcode::Ieq + make_select`.

Verification

- `examples/array_examples/verify_array.py` — array-parameter defaults, full and partial per-element override, aggregate assignment, array copy, and dynamic read/write all match their closed-form gains through ngspice (ALL PASS).
- The `hir_def/hir_ty/hir/hir_lower/parser/syntax` unit-test suites pass with no regressions (including the `huge_int_literal` bit-select regression test). The pre-existing stale `sim_back DAE/topology` snapshot failures are unchanged (identical 9-pass/15-fail before and after Enhancement-14) and are unrelated to arrays.
- Every prior `*_examples/` folder still compiles and simulates with unchanged results.

Known limitations

- Dynamic indexing of an array parameter is not supported directly (copy the parameter into an array variable and index that). A non-constant index on a parameter array reports the ordinary "non-constant bit-select index" error.
- Arrays remain one-dimensional (matching the existing bus/array-variable model); nested `'{{...}}'` aggregates are not supported.
- The array-literal fill order follows the declared `[msb:lsb]` direction.

Enhancement-15 — Verilog-A multi-dimensional arrays

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory to generalise the 1-D array support of Enhancement-14 to multi-dimensional arrays (real [0:1][0:2] m; ... m[i][j]). It covers array variables and array-valued parameters, constant and dynamic (runtime) indexing, and nested aggregate literals — verified end-to-end through ngspice (see `examples/mdarray_examples/`).

All work is in `version11/`; verification uses `version11/ngspice-46`'s own binary and `version11/OpenVAF-master`'s own `openvaf-r`. No OSDI ABI change and no ngspice change were needed — a multi-dim array parameter surfaces as one ordinary scalar OSDI parameter per element (named `w[i][j]`), which ngspice overrides individually, and everything else lowers to ordinary MIR.

0. Representation

The design keeps every array (net, variable, parameter) expanded into independent scalar elements, exactly as the 1-D bus/array machinery already did; a multi-dim array simply has more elements, named by their full index tuple (`m[i][j]`). Two representation changes carry the extra dimensions:

- **Expr::BitSelect** (`hir_def`) changes index: `ExprId` → `indices: Vec<ExprId>`. A single bit-select node now carries every `[...]` clause of an access (`m[i][j]` → `indices = [i, j]`), since an array's base is always its name. 1-D is the common case (`indices.len() == 1`).
- **BusDecl** (`hir_def::item_tree`) gains `dims: Vec<(i32, i32)>` (one (msb, lsb) per dimension, outermost first). The existing `msb/lsb` fields are kept (`== dims[0]`) so all 1-D net/array code is untouched. New helpers: `ndim`, `elem_count`, `contains`, `elem_name` (synthesizes `m[i][j]`), and `index_tuples` (every index tuple in declaration order — each dimension `msb`→`lsb`, outermost slowest, the order a nested literal fills).

1. Parsing

- **grammar/expressions.rs** — the postfix `[...]` after an identifier now *loops*, collecting `m[i][j]....` into one `BIT_SELECT_EXPR` with several index children.
- **grammar/items.rs** — `var_decl` and `parameter_decl` loop the `[msb:lsb]` width clause, so a declaration can carry several dimensions.
- AST (`veriloga.ungram`, generated `nodes.rs`) — `BitSelectExpr::indices()`, `VarDecl::widths()`, `ParamDecl::widths()` (the plural `AstChildren` accessors).
- Consumers of the old single accessors are updated: `hir_def::body::lower` (collects all indices), `body::pretty`, and `hir::elaborate's generate` for genvar bit-select folding (folds every index sub-expression).

2. Declaration & element expansion

`fold_width_ranges` folds all width clauses into `dims`. `lower_var / lower_param` build one `BusDecl` with those `dims` and declare one scalar element per `index_tuples()` entry, named via `elem_name(x[i]` for 1-D — byte-identical to before — or `x[i][j]` for multi-dim). An array parameter's elements each carry an `array_index` = their flat declaration-order position.

3. Constant indexing

`inere_bit_select` (`hir_ty`) is rewritten for N indices: it checks the index count against `ndim` (new `WrongArrayDimensions` diagnostic), constant-folds every index, range-checks the tuple (`contains`), and resolves the synthesized `elem_name` path to the element `VarId/ParamId/NodeId`. A vectored net still requires exactly one constant index.

4. Dynamic (runtime) indexing

When any index is non-constant (a genuine runtime value, not an unfoldable literal — the `is_literal_index` guard is preserved), a variable array access is recorded for a runtime element select:

- `DynArrayIndex` (`hir_ty`) now stores the flat element `VarIds` (declaration order), the per-dimension `dims`, and one index expression per dimension. `inference_dynamic_bit_select` builds it for reads; `try_inference_dynamic_index_assignment` for writes (`m[i][j] = v`). Mixed constant/dynamic indices (`m[0][j]`) are supported.
- **`hir_lower`** — `lower_flat_array_index` computes the runtime flat position $\sum_k \text{pos}_k \cdot \text{stride}_k$ (`pos_k = \text{idx} - \text{msb}` ascending / `msb - \text{idx}` descending; `stride_k` = product of later dimension sizes), matching `index_tuples` ordering. A read selects `elems[flat]`; a write conditionally updates every element (`elems[k] = (flat == k) ? v : elems[k]`). Both reuse `make_select`.

Array *parameters* are constant tables and stay constant-indexed (a non-constant index on a parameter array is the ordinary "non-constant bit-select index" error); copy a parameter into a variable array to index it dynamically.

5. Nested aggregate literals

- Assignment — `try_inference_array_assignment` flattens the RHS literal (`flatten_array_literal: '{a,b},{c,d}' → [a,b,c,d]`, row-major) and zips the leaves against the destination's flat element list. Length/element-type mismatches reuse the existing `TypeMismatch` diagnostic. Array-to-array copy (`m = d`) and 1-D flat literals are unchanged special cases of the same code.
- Parameter defaults — the parameter body query flattens the (possibly nested) default literal at the AST level and gives each element parameter the leaf at its `array_index`.

Verification

- `examples/mdarray_examples/verify_mdarray.py` — 2-D array parameter defaults, per-element 2-D override (`w[1][1]=0.9`), nested-literal aggregate assignment, and dynamic 2-D read/write all match their closed-form gains through `ngspice` (ALL PASS).
- The `hir_def/hir_ty/hir/hir_lower/parser/syntax` unit-test suites pass with no regressions. Every prior `*_examples/` folder still compiles and simulates unchanged — including the 1-D `array_examples` (Enhancement-14), vectored-net `bus_examples`, generate `genvar` bit-select folding, and the `laplace` array-coefficient forms. The pre-existing stale `sim_back` snapshot failures (9 pass / 15 fail) are unchanged and unrelated.

Known limitations

- Aggregate-literal shape is validated by total element count, not per-level shape, so a ragged literal with the right leaf count is accepted.
- A whole multi-dim array is not itself a first-class value (no slicing, no passing a whole array to a function) — access is always down to a scalar element, as for 1-D arrays.

Enhancement-16 — Verilog-A `$table_model`

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory to implement the Verilog-AMS `$table_model` lookup-table system function (1-D), which previously carried an explicit `// TODO TABLE_MODEL` in the builtin signature table and was rejected as *"not found in the current scope"*.

`$table_model(x, <data>[, "control"])` interpolates a value from a tabulated grid. Unlike the E-9 `noise_table` (which feeds only the noise PSD path), `$table_model` is used in the main device equations, so the interpolation must be differentiable — its slope becomes the Jacobian entry. It is verified end-to-end through ngspice (see `examples/table_model_examples/`).

All work is in `version11/`; verification uses `version11/ngspice-46`'s own binary and `version11/OpenVAF-master`'s own `openvaf-r`. No OSDI ABI change and no ngspice change were needed — the table is read at compile time and lowered to ordinary differentiable MIR arithmetic.

Design: lower to differentiable MIR (no special autodiff support)

The key decision is to lower `$table_model` to plain MIR arithmetic — a piecewise-linear interpolation built as a select chain over the compile-time grid — rather than to a special callback. Because `mir_autodiff` differentiates ordinary arithmetic, this gives the correct per-segment slope dy/dx as the Jacobian entry for free, exactly as the Enhancement-14/15 dynamic-index select chains do. `$table_model` is therefore an ordinary pure function (not an analog operator with state/time), classified `is_analog_operator() == false`.

1. Wiring the builtin

- **`hir_def::builtin`** — `BuiltIn::table_model = 111` added; the pre-existing `sysfun::table_model` name (`$table_model`, already tokenized) is registered: `dst.insert(sysfun::table_model, BuiltIn::table_model.into())`. It is a pure function, so the classification predicates leave it at their `_ => false` defaults.
- **`hir_ty::builtin`** — the `TODO TABLE_MODEL` is replaced by a `TABLE_MODEL` signature group (four 1-D variants: inline-array vs data-file $\times$ with/without a control string). The generated `BUILTIN_INFO` table gains the `TABLE_MODEL` entry (length 111 $\rightarrow$ 112).

2. Reading the table (compile time)

`table_model_data` gathers the (x, y) points — from an inline real array of flat $\{x_0, y_0, x_1, y_1, \dots\}$ pairs (const-folded via `eval_const_real`) or from a two-column data file (reusing E-9's `read_noise_table_file`; blank/comment lines skipped, path relative to the source file). Points are sorted by x and duplicate abscissae dropped so the segments are well-formed.

3. Lowering the interpolation (**`lower_table_model`**)

For a grid of N points, each segment i contributes the linear value $y_i + (x - x_i) \cdot \text{slope}_i$ (slope folded to a constant at compile time). These are combined into a select chain: segment i becomes active once $x \geq x_i$ (segment 0 is the default and also covers x below the grid; the last segment covers x above it). This yields linear interpolation inside the grid and, by default, linear extrapolation from the end segments.

The control string selects extrapolation: it defaults to constant (the value is clamped to the endpoint outside the grid, via two extra selects), or linear when the string contains `L`. The comparison-driven selects make the result piecewise-linear and differentiable; `mir_autodiff` follows the active branch, so the Jacobian entry is the local segment slope (zero in a clamped region — hence prefer `L` where the operating point may leave the grid).

Verification

- `examples/table_model_examples/verify_table.py` exercises the value and its derivative across DC, AC and transient:
 - DC — an inline-array transfer function `V(out) = $table_model(V(in), '{...})` matches a reference piecewise-linear interpolation bit-exactly; and a file-based nonlinear resistor `I(p,n) = $table_model(V(p,n), "diode_iv.tbl", "1L")` driven through a series resistor converges to the analytic nonlinear DC operating point ($\sim 5e-10$ V) — only possible because the interpolation supplies the correct dI/dV to the Jacobian;
 - AC — that same slope is the small-signal conductance: it matches the analytic table slope exactly at every bias point (error $\sim 1e-18$ S);
 - Transient — the table is re-evaluated each timestep and `V(out)` tracks `table(V(in))` instantaneously ($\sim 2e-8$ over the sweep).

All three work identically because `$table_model` lowers to plain differentiable MIR: the value flows into the residual and its autodiff derivative into the Jacobian, which DC Newton, the AC linearization, and each transient step all use.

- The `hir_def/hir_ty/hir/hir_lower` unit-test suites pass with no regressions; `noise_examples` (the closest existing code) and every other prior example still pass. The pre-existing stale `sim_back` snapshot failures are unchanged and unrelated.

Known limitations / future work

- 1-D only, linear interpolation. Multi-dimensional tables (bilinear/trilinear) and higher-degree (spline) interpolation are the natural next step — the same way Enhancement-14's 1-D arrays were generalised to N-D in Enhancement-15.
- The control string is interpreted leniently (degree is always 1; presence of `L` $\Rightarrow$ linear extrapolation on both ends, otherwise constant), rather than the full per-dimension LRM control-string grammar.
- The data-file format is the simple two-column form shared with `noise_table`, not the full Verilog-AMS `$table_model` file grammar.

Enhancement-17 — multi-dimensional `$table_model`

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory to generalise the 1-D `$table_model` of Enhancement-16 to multiple dimensions (2-D and 3-D): `$table_model(x1, x2[, x3], "grid_file" [, "control"])` interpolates a value from an N-dimensional tabulated grid.

Like the 1-D case, the interpolation must be differentiable (it is used in the main device equations, so every partial derivative becomes a Jacobian entry). It is verified end-to-end through ngspice — see `examples/mdtable_examples/`.

All work is in `version11/`; verification uses `version11/ngspice-46`'s own binary and `version11/OpenVAF-master`'s own `openvaf-r`. No OSDI ABI change and no ngspice change were needed.

Design: N-D multilinear = recursive 1-D interpolation

Multilinear interpolation is separable, so an N-D interpolation is built from 1-D interpolations recursively: peel the outermost axis, interpolate each of its grid slices over the remaining N-1 axes (yielding one runtime value per outer-axis grid line), then interpolate those values along the outer axis. The base case (one axis left) is an ordinary 1-D interpolation of the tensor's constant values. Because each step is the same differentiable piecewise-linear 1-D kernel, the whole thing is differentiable in every coordinate — `mir_autodiff` supplies all N partial derivatives (e.g. a table-based MOSFET's `gm` and `gds`) for free, with no special support.

The shared 1-D kernel, `interp_1d_values`, was generalised from Enhancement-16's 1-D lowering to interpolate a list of runtime **Values** (the sub-results of the recursion) rather than only compile-time constants, using an `fdiv` for the segment slope so the pure-1-D path stays bit-identical to Enhancement-16.

1. Signatures

`hir_ty/builtin.rs` adds four file-based variants to the `TABLE_MODEL` group (alongside the existing 1-D ones): `TABLE_MODEL_2D_FILE`, `..._2D_FILE_CTRL`, `TABLE_MODEL_3D_FILE`, `..._3D_FILE_CTRL` — one `Val(Real)` coordinate per dimension, a `Literal(String)` grid-file name, and an optional control string. The fixed-signature type checker resolves the right variant by argument count and types (a 2-D call `(Real, Real, String)` vs a 1-D file+control `(Real, String, String)` differ in the second argument).

2. Grid data file

Multi-dimensional data is a self-describing grid file (`read_table_grid_nd` over `read_table_tokens`): whitespace-separated tokens — `ndim`, then `ndim` axis sizes, then each axis's ascending coordinates, then `prod(sizes)` values in row-major order (outermost axis slowest) — with `blank/#/*` lines ignored. 1-D still accepts an inline `{x0,y0,...}` array or a two-column data file.

3. Lowering (`lower_table_model`, `interp_nd`, `interp_1d_values`)

`lower_table_model` maps the resolved signature to `(ndim, is_file, has_ctrl)`, reads the grid into per-axis coordinate vectors and a row-major value tensor, lowers each coordinate expression, and calls `interp_nd`. `interp_nd` performs the recursive peel-and-blend described above; `interp_1d_values` is the differentiable piecewise-linear select chain (segment `i` active once `x >= grid[i]`; constant or linear extrapolation outside). Everything lowers to ordinary MIR (`fsub/fdiv/fmul/fadd + make_select`), so it is differentiable and works identically in DC, AC and transient.

Verification

- `examples/mdtable_examples/verify_mdtable.py` — a table-based MOSFET $I(V_{gs}, V_{ds})$:
 - DC — the drain current over a (V_{gs}, V_{ds}) scan matches a reference bilinear interpolation of the same grid to machine precision ($\sim 1e-19$ A);
 - AC — both partial derivatives, $g_m = dI_d/dV_{gs}$ and $g_{ds} = dI_d/dV_{ds}$, match the bilinear surface gradient ($\sim 1e-16$ S), i.e. the 2-D Jacobian is exact.
- Ad-hoc checks confirm bilinear reproduces $x \cdot y$ and trilinear reproduces $x \cdot y \cdot z$ exactly (multilinear functions), with both/all partial derivatives correct.
- The 1-D `table_model_examples` (Enhancement-16) still passes unchanged (DC/AC/transient), confirming the shared-kernel refactor did not regress 1-D.
- The `hir_def/hir_ty/hir/hir_lower` unit-test suites pass with no regressions; the pre-existing stale `sim_back` snapshot failures are unchanged and unrelated.

Known limitations / future work

- Dimensionality is 1-D, 2-D, 3-D (the practical range for compact-model tables). A truly variadic (arbitrary- N) form would need a special-cased builtin rather than fixed per-dimension signatures; the `interp_nd` lowering is already general in N .
- Interpolation is multilinear (degree 1). Higher-degree (spline) interpolation remains future work.
- Multi-dimensional data must come from a grid file (an inline array is 1-D only); axis coordinates must be ascending.

Enhancement-18 — array declaration syntax + arrays in analog functions

This document describes two related source-code changes made to OpenVAF-r in the `version11/` directory:

1. Standard array declaration syntax — the LRM *name-then-range* form `real x[0:n];` (and multi-dimensional `real m[0:1][0:2];`), complementing the existing *range-then-name* form `real [0:n] x;`.
2. Array support in **analog functions** — array local variables and whole array arguments, previously rejected with “*array-variable declarations are only supported at module body scope*”.

Both are verified end-to-end through ngspice — see `examples/funcarray_examples/`. All work is in `version11/`; no OSDI ABI change and no ngspice change were needed.

Part 1 — `real x[0:n]` declaration syntax

Verilog-AMS declares unpacked arrays as `real coeffs[0:4];` (dimensions *after* the name), whereas OpenVAF previously only accepted `real [0:4] coeffs;` (dimensions *before* the name). Enhancement-18 accepts both, per-variable.

- Parser (`grammar/items.rs::var`) — after a variable name, parses zero or more `[msb:lsb]` clauses (`x[0:n]`, `m[0:1][0:2]`). AST (`Var::widths()`, `ungram`) exposes them.
- `item_tree::lower_var` — each variable’s dimensions are its own name-then-range widths if present, else the shared declaration-level range-then-name widths. So `real g, w[0:1], k;` declares a scalar, a 1-D array, and a scalar. Everything downstream (the Enhancement-14/15 element expansion) is unchanged.

Part 2 — arrays in analog functions

Array variables were expanded into scalar element variables (`x[0]`, `x[1]`, ...) only at *module body* scope. Enhancement-18 extends this to `analog` function bodies and adds whole-array argument passing.

- Function-scope array locals — `item_tree::Function` gains a `var_arrays` list; `lower_fun` passes it to `lower_var` (previously `None`), so array locals and array-typed argument declarations expand into element variables inside the function. `hir_ty::find_var_array` now also resolves arrays for a `DefWithBodyId::FunctionId` owner, so `x[i]` bit-selects (constant *and* dynamic) work in a function body exactly as at module scope.
- Array-typed arguments — `function_data_query` types an argument whose name matches a function `var_array` as `Type::Array{elem, len}` (its declaration expanded to element variables, so it has no single scalar declaration).
- Whole-array call binding — at a call `f(c, ...)`, `inference_user_fun_call` pre-resolves a bare array actual passed to an array formal into its element `VarIds` (recorded in `array_var_refs`, matching the laplace-argument mechanism), and `inference_expr` returns the array type for such a pre-resolved reference instead of the “requires a bit-select” error. In lowering, `lower_user_fun_impl` binds the callee’s element variables (resolved via `hir_def::function_array_arg_vars`, which resolves the `v[i]` names in the function’s body scope) from the caller’s array elements — pass-by-value of the elements. Because the callee body is lowered inline as ordinary MIR, the Jacobian flows through the function automatically.

Array arguments are input (element pass-by-value); array output writeback and array return values are not supported.

Verification

- `examples/funcarray_examples/verify_funcarray.py` — a polynomial stage $V(\text{out}) = 0.5 \cdot V(\text{in}) + 0.3 \cdot V(\text{in})^2$ evaluated inside an array-argument function `polyeval(x, a)` (

Horner's rule, indexing the array argument with a loop variable), with `coeffs` declared `real coeffs[0:3]`; and passed by name:

- DC — $V(\text{out})$ matches the closed-form polynomial ($\sim 1\text{e-}16$);
- AC — the small-signal gain matches $\text{poly}'(\text{bias}) = 0.5 + 0.6 \cdot \text{bias}$ ($\sim 1\text{e-}16$), i.e. the derivative flows through the array-argument function into the Jacobian.
- Ad-hoc checks: `real x[0:n]/real m[0:1][0:2]/mixed real g, w[0:1], k`; declarations simulate correctly; array locals with dynamic indexing inside a function work; a `dot(c, w)` array-argument function returns the expected value.
- The `hir_def/hir_ty/hir/hir_lower/parser/syntax` unit-test suites pass with no regressions; every prior example folder still compiles and simulates.

Known limitations / future work

- Array output arguments (writing a whole array back to the caller) and array return values are not supported — array arguments are input-only.
- A width clause inside a *nested* `begin...end` block (rather than at the function or module top level) still degrades to a scalar with a diagnostic.

Enhancement-19 — **do ... while** loop

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory to implement the Verilog-AMS **do ... while** loop, previously a parse error (`do` was lexed as an identifier). It is the one loop construct OpenVAF didn't support — `for`, `while`, and `repeat` already worked.

`do <statement> while (<condition>);` executes its body once before the condition is first tested (a post-test loop), so the body always runs at least once. Verified end-to-end through ngspice — see `examples/dowhile_examples/`.

Changes

The feature threads a new statement kind through the whole front-end, mirroring `while` at each stage:

- Token / syntax kind (`tokens/src/parser/generated.rs`) — a `do` keyword (`D0_KW`) and a `D0_WHILE_STMT` node kind, wired into `is_keyword`, `from_keyword` ("do"), the display table, and the `T!` macro. (`SyntaxKind` is `#[repr(u16)]` with a bounds-checked `from_u16`, so adding variants rennumbers consistently.)
- Parser (`grammar/stmts.rs`) — `D0_KW` added to the statement token sets and dispatch; `do_stmt` parses `do <stmt> while ( <expr> ) ;`.
- AST (`veriloga.ungram`, generated `nodes.rs`) — a `DoWhileStmt` node (`body(): Stmt`, `condition(): Expr`) added to the `Stmt` enum, its `can_cast`, `cast`, `syntax`, and `From` impls.
- HIR — `hir_def::Stmt::DoWhile { cond, body }` (with the `expr/stmt` walkers and `body/lower` mapping the AST node), the pretty printer, `hir_ty` condition inference and validation, and the elaborated `hir::Stmt::DoWhile` produced by `get_stmt`.
- Lowering (`hir_lower/stmt.rs`) — `lower_do_while`: like `lower_loop`, but the body block is entered unconditionally and the condition is tested at the end of each iteration. The body block is the loop header (both the entry edge and the back-edge from the condition), so it is sealed last for correct SSA.

Verification

- `examples/dowhile_examples/verify_dowhile.py` — a `do` loop reports its iteration count as a gain; across the loop count `n` (overridden per `.model`) the count equals `max(n, 1)`, and in particular `n = 0` still runs the body once (`count = 1`), the defining post-test behaviour. ALL PASS.
- Ad-hoc: a `do`-loop that iterates a fixed number of times, one whose condition is initially false (body runs exactly once), and a single-statement body (no `begin ... end`) all simulate correctly.
- The `tokens/parser/syntax/hir_def/hir_ty/hir/hir_lower` unit-test suites pass with no regressions; every prior example folder still compiles and simulates with unchanged results.

Enhancement-20 — array output/inout arguments to analog functions

This document describes the source-code change made to OpenVAF-r in the `version11/` directory to support **output** and **inout** array arguments to `analog` functions, completing Enhancement-18's array-argument support (which was input-only: a whole array could be passed *into* a function, but a function could not write one back).

Before this, an array output/inout argument compiled but silently did nothing on return — the function computed into its element variables, but they were never copied back to the caller's array (a correctness gap). Now the writeback happens, so `fill(r)` and `scale_in_place(r)` actually update `r`. Verified end-to-end through ngspice — see `examples/arrayout_examples/`.

The change

Enhancement-18 already lowered a whole-array argument by binding the callee's element variables (`v[i]`) from the caller's array elements at function entry (input semantics), and its inference (`pre_resolve_array_call_arg`) already resolved the caller's array to its element `VarIds` (`array_var_refs`) for *any* array argument, input or output. The only missing piece was the exit writeback, which the Enhancement-18 writeback loop explicitly skipped for array arguments.

`lower_user_fun_impl` (`hir_lower/src/expr.rs`) now, for an output (or inout) array argument, copies each of the function's element variables back to the corresponding caller array element after the body has run:

```
for each element i: caller_array[i] = read(callee_element_var[i])
```

reusing the same `FunctionArg::array_elems` (callee element `VarIds`, via `hir_def::function_array_arg_vars`) and `Body::array_var_ref` (caller element variables) machinery introduced in Enhancement-18. Because the callee body is lowered inline, these are ordinary variable defs in the same MIR. inout arguments get both the entry bind (from Enhancement-18) and this exit writeback; input arguments are unaffected.

Verification

- `examples/arrayout_examples/verify_arrayout.py` — `make_taps` fills a geometric tap array via an output array argument, `normalize` scales it in place via an inout array argument, and the gain is the sum of the normalized taps. That sum is 1 for any `ratio` (swept, overridden per `.model`), so $V(\text{out}) = V(\text{in})$ — which holds only if both writebacks reach the caller's array. ALL PASS.
- Ad-hoc: a pure output array argument (`fill(r)` sets `r = {3,4}`) and an inout array argument (`scale2(r)` doubles `r` in place) both produce the expected caller-side values; Enhancement-18 input array arguments are unchanged (`funcarray_examples` still passes).
- The `hir_lower/hir_ty/hir` unit-test suites pass with no regressions; every prior example folder still compiles and simulates unchanged.

Known limitations

- The array actual passed to an output/inout argument must be a writable array variable (a bare array reference). Passing a non-writable array expression is a silent no-op on writeback rather than a diagnosed error.
- Array return values (a function whose return type is an array) remain unsupported; use an output argument instead.

Enhancement-21 — Verilog-AMS **paramset** blocks

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory to support Verilog-AMS **paramset** blocks — named, instantiable models that specialise an existing behavioural module by binding some of its parameters. `paramset` was previously a hard parse error (`paramset` was lexed as an identifier and the top-level parser rejected it).

Syntax and semantics

```
paramset <name> <target_module>;
    parameter real <p> = <default>;    // the paramset's own (card) parameters
    .<target_param> = <expr>;          // bind a target-module parameter
endparamset
```

`<name>` becomes its own OSDI model — usable as `.model foo <name>` in a netlist — with the same terminals and analog behaviour as `<target_module>`. Each bound target parameter takes the value of its `<expr>` (which may reference the paramset's own parameters and constants) and is no longer settable from the model card; unbound target parameters remain settable (pass-through). This is the Verilog-AMS way of shipping a *model library*: one behavioural module plus several named, pre-configured variants.

Implementation: the "twin module"

The key idea is that a `paramset` is lowered into a synthetic twin module that *reuses the target module's declaration verbatim* rather than duplicating any behaviour. Concretely, in `hir_def`'s item-tree lowering (`lower_paramset`), a `paramset` produces a new `Module` that:

1. shares the target module's `ast_id` — so its ports and analog body (and branches, functions, nets) are exactly the target's, resolved through the twin's own scope;
2. carries the paramset's own parameters as its (card) parameters; and
3. replaces each bound target parameter with a fresh parameter that is marked `localparam` and whose initializer is the override expression.

Because item identity is `(scope, item-tree-index)`, the twin gets its own module scope and re-interns the shared item-tree entries as fresh parameters, variables, etc., under that scope — so the twin is a fully independent module that happens to share the target's syntax. Everything downstream (name resolution, type inference, MIR lowering, autodiff, OSDI descriptor emission) then treats the twin as an ordinary module, so DC, AC (Jacobian), noise, and transient all work with no further changes.

Modelling a bound parameter as a `localparam` whose value is the override expression is what makes this cheap: the existing `localparam` machinery already (a) computes the value inline from the analog body, referencing other parameters, and (b) keeps it off the model card. No change to parameter initialization, the OSDI ABI, or ngspice was required.

Files changed

- `tokens` (`tokens/src/parser/generated.rs`) — `paramset/endparamset` keywords and the `PARAMSET_DECL / PARAMSET_OVERRIDE` syntax kinds.
- `parser` (`parser/src/grammar.rs`, `parser/src/grammar/items.rs`) — parse a top-level `paramset ... endparamset` (name, target, parameter declarations, and `.<param> = <expr>;` overrides).
- `syntax` (`syntax/src/ast/generated/nodes.rs`) — `ParamsetDecl / ParamsetOverride` AST nodes and the `Item::ParamsetDecl` variant.
- `basedb` (`basedb/src/ast_id_map.rs`) — give `ParamsetOverride` nodes `ast-ids` so their override expressions are addressable.
- `hir_def`:

- `item_tree.rs` — `Param::override_expr` field; the `UnknownParamsetTarget` diagnostic.
- `item_tree/lower.rs` — `lower_paramset` (the twin-module construction).
- `item_tree/diagnostics.rs` — render the unknown-target diagnostic.
- `data.rs` — an overridden parameter reports `is_local = true`.
- `body.rs` — an overridden parameter's default is its override expression, lowered in the twin's scope.

Verification

`examples/paramset_examples/verify_paramset.py` — one module conductor = $g_0 \cdot (1 + k \cdot V)$ and three paramsets (`res_1k`, `res_kohm`, `varistor`). ALL PASS:

- constant bindings take effect (`res_1k = 1 kΩ`);
- a binding computed from a card parameter takes effect (`res_kohm:  $g_0 = 1 / (kohm \cdot 1000)$`);
- an unbound parameter stays settable while a bound one is driven by the paramset (`varistor:  $g_0$  pass-through,  $k = kv$`);
- a bound parameter is not settable from the card (`k=9` is ignored);
- the derivative flows through the paramset — the AC conductance $gm = g_0 \cdot (1 + 2 \cdot k \cdot V)$ is exact (autodiff Jacobian on the shared body);
- the base module conductor still works independently.

Every prior example folder still passes (notably `instantiation_examples`, which exercises the same item-tree/parameter machinery), and the `hir_def/hir_ty` unit-test suites pass with no regressions.

Known limitations

- The target module must be declared in the same file.
- Multiple paramsets sharing one name with instance-based selection, and `aliasparam`/statement-based selection blocks, are not supported — each paramset maps to exactly one model.
- An override naming a parameter the target module does not declare is silently ignored rather than diagnosed.

Enhancement-22 — natural cubic-spline `$table_model`

This document describes the source-code change made to OpenVAF-r in the `version11/` directory to add natural cubic-spline interpolation to `$table_model`, extending the piecewise-linear interpolation of Enhancement-16 (1-D) and the multilinear interpolation of Enhancement-17 (2-D/3-D). A cubic spline is C^1 — its derivative is continuous — so a table-based compact model's gm/gds are smooth, unlike the staircase derivative produced by linear interpolation.

Selecting cubic interpolation

The interpolation degree is chosen entirely by the `$table_model` control string — no new builtin signature and no OSDI/ngspice change. A 3 anywhere in the control string selects natural cubic spline; otherwise interpolation stays multilinear. Extrapolation is unchanged (L = linear, else clamp). Following Enhancement-16/17's simplification, a control code applies to all axes (per-axis degree is future work). So this is a **hir_lower**-only change (`expr.rs`).

How it lowers to differentiable MIR

The key observation is that a natural cubic spline's per-point second derivatives (the "moments" M) solve a tridiagonal system that is linear in the grid values y and depends only on the grid:

$M = L \cdot y$ with L ($n \times n$) fixed by the grid alone (natural BC: $M[0]=M[n-1]=0$)

`natural_cubic_spline_moment_matrix(grid)` precomputes L at compile time (building the interior tridiagonal system and inverting it with a small dense Gauss–Jordan). At lowering time each moment $M[i] = \sum_j L[i][j] \cdot \text{vals}[j]$ is then just a constant-weighted sum of the (possibly runtime) grid values — no runtime linear solve. The per-interval cubic

$$S(x) = M[i] \cdot a^3/(6h) + M[i+1] \cdot b^3/(6h) + (v[i]/h - M[i] \cdot h/6) \cdot a + (v[i+1]/h - M[i+1] \cdot h/6) \cdot b$$
with $a = x[i+1]-x$, $b = x-x[i]$, $h = x[i+1]-x[i]$

is emitted as ordinary MIR and selected per interval with the same $x \geq \text{grid}[i]$ select chain the linear kernel uses, so `mir_autodiff` supplies the exact, continuous Jacobian for free. Extrapolation mirrors the linear kernel: clamp to the endpoint value, or (with L) continue the spline's end tangent (also a constant-weighted combination of the values).

For N-D, the existing recursive-1-D scheme (`interp_nd`) is reused with the cubic kernel at each level; because a natural spline interpolates exactly at nodes, recursive-1-D natural spline equals the exact tensor-product natural spline.

Functions (`hir_lower/src/expr.rs`)

- `natural_cubic_spline_moment_matrix(grid)` $\rightarrow$ `Vec<Vec<f64>>` — compile-time L .
- `weighted_sum(w, vals)` — $\sum w_j \cdot \text{vals}_j$ in MIR (constant weights, runtime values).
- `interp_1d_spline(x, grid, vals, linear_extrap)` — the cubic kernel (falls back to `interp_1d_values` for fewer than 3 points).
- `interp_nd` gains a cubic flag dispatching to the spline or linear kernel; `lower_table_model` derives cubic from the control string ('3').

Verification

`examples/cubic_table_examples/verify_cubic_table.py` (ALL PASS), each check contrasting cubic with linear on the same data:

- accuracy — cubic tracks $\sin(V)$ ~46× better than linear at off-grid points;

- smoothness — across a grid node the cubic gm matches $|\cos(V)|$ on both sides (continuous), while the linear gm jumps $\sim 10\times$ more — the defining benefit of splines, and a direct test that the autodiff Jacobian through the cubic MIR is continuous;
- exactness — a natural cubic spline reproduces straight-line data exactly (all moments zero);
- N-D — 2-D tensor-product cubic reproduces $\sin(x) \cdot \cos(y)$ accurately.

The existing linear `table_model_examples/mdtable_examples` still pass unchanged (the linear path is untouched), and every other example folder still passes.

Known limitations

- Natural boundary conditions only (zero end curvature); other end conditions (clamped, not-a-knot) are future work.
- A control code applies cubic to all axes; per-axis interpolation degree is future work.
- The moment matrix is dense ($O(n^2)$ weighted-sum terms per moment); fine for the modest grids typical of compact-model tables, heavier for very large grids.

Enhancement-23 — array return values from analog functions

This document describes the source-code changes made to OpenVAF-r in the `version11/` directory to support array return values from analog functions (analog function `real[0:n] f;`), completing the array-in-functions arc: Enhancement-18 (array arguments, input) → Enhancement-20 (array output/inout arguments) → Enhancement-23 (array return values).

Before this, analog function `real[0:n] f;` was a parse/resolve error (the array dimensions after the return type were unexpected, and a call `c = f(...)` reported the call as "not found").

Syntax and semantics

```
analog function real[0:n] f;    // the return type carries array dimensions
    ...
    f[i] = ...;                // the body writes the return array's elements
endfunction
...
real c[0:n];
c = f(args);                   // the whole returned array is copied into c
```

An array-returning call is valid only as the entire right-hand side of an array assignment (`c = f(...)`); the destination is a writable array variable of the same length.

Implementation: the return array as a function `var_array`

The design reuses the Enhancement-18/20 array machinery. The return array is modelled as a function-scoped array variable named after the function: its element variables `f[0]`, `f[1]`, ... are registered in the function's `var_arrays` (exactly like an array argument's element variables), so inside the body `f[i] = ...` resolves and writes them via the existing bit-select / dynamic-index paths. `function_array_arg_vars(_, f, f.name)` then resolves those element `VarIds` at the call site.

At the call site `c = f(args)`, inference recognises the right-hand side as a call to an array-returning function and records an `ArrayAssign::ReturnCall`. Lowering then:

1. lowers the call — which inlines the function body, writing the return element variables; and
2. copies each return element variable into the destination array element.

Because the body is inlined as ordinary MIR (as for every user function), the Jacobian flows through the array return automatically — the AC conductance of a table/polynomial device defined this way is exact. No OSDI ABI change and no ngspice change.

Files changed

- `parser(grammar/items/module.rs)` — parse the `[msb:lsb]` return dimensions after the type in `func_decl`.
- `syntax(ast/generated/nodes.rs)` — `Function::widths()` accessor.
- `hir_def`:
 - `item_tree.rs` — `Function::ret_dims` field.
 - `item_tree/lower.rs` — `lower_fun` expands the return array into element variables (a `var_array` named after the function) and records `ret_dims`.
 - `data.rs` — `FunctionData::ret_len`.
 - `body.rs` / `nameres.rs` — tolerate the synthetic return element variables, which have no `ast::Var` node (they carry the function's `ast-id` as a placeholder): `var_body` falls back to the type default and `nameres` uses the erased id, rather than casting to `ast::Var`.
- `basedb(ast_id_map.rs)` — `AstId::from_erased`.

- `hir_ty (inference.rs)` — `ArrayAssign::ReturnCall`; the array-returning-call case in `try_infer_array_assignment` (with a length-mismatch diagnostic).
- `hir (lib.rs, body.rs)` — `Function::returns_array / return_array_elems`; the `Stmt::ArrayReturnAssignment` HIR statement.
- `hir_lower (stmt.rs)` — `lower ArrayReturnAssignment` (inline the call, then copy the return elements into the destination).

Verification

`examples/arrayret_examples/verify_arrayret.py` (ALL PASS) — a cubic polynomial device $I = c_0 + c_1 \cdot V + c_2 \cdot V^2 + c_3 \cdot V^3$ built two ways: `polyret` (a function returns the power array $\{1, V, V^2, V^3\}$, summed at the call site) and `polyret_arg` (the returned array is fed straight into an array-argument function, composing E-23 with E-18). For both, across a bias sweep, the DC current matches the closed form ($\sim 1e-9$) and the AC conductance matches the exact derivative $gm = c_1 + 2 \cdot c_2 \cdot V + 3 \cdot c_3 \cdot V^2$ ($\sim 1e-9$) — the autodiff Jacobian flows through the array return.

A length mismatch (`real c[0:1]; c = f3(...)` where `f3` returns 3 elements) is a clean compile-time type error, not a crash. Every prior example folder still passes (notably `funcarray`, `arrayout`, `array`, `mdarray`), and the `hir_def/hir_ty/hir_lower` unit-test suites pass with no regressions.

Known limitations

- An array-returning call is only valid as the entire RHS of an array assignment, not as a sub-expression.
- Multi-dimensional return arrays parse and expand (via the N-D `var_array` machinery) but are exercised less than the 1-D case.

Enhancement-24 — **\$discontinuity(n)** simulator support

This document describes the changes made to OpenVAF-r *and* ngspice-46 in the version11/ directory to give real effect to **\$discontinuity(n)** (for $n \geq 0$). Previously **\$discontinuity** compiled but was a no-op except for the internal **\$discontinuity(-1)** used by device limiting (which sets the OSDI LIM eval flag); every other form hit a `// TODO implement support for discontinuity?` branch and generated nothing.

Semantics

\$discontinuity(n) ($n \geq 0$) announces a discontinuity of degree n in the branch constitutive relations at the current point. A transient simulator should limit the timestep there — placing a fine timepoint and not extrapolating a large step across the event — rather than trusting its truncation-error estimate through the discontinuity. It affects only timestep control, never the computed solution.

Implementation: via the **bound_step** output

The natural vehicle would be an OSDI eval return flag (like **\$finish/\$stop**), but that path is not honoured by ngspice's timestep control (ngspice acts on FATAL/LIM/STOP during load, but nothing routes a flag into OSDItrunc). The **bound_step** eval output, on the other hand, is honoured: OSDItrunc already clamps the next timestep to it (it defaults to +INFINITY).

So **\$discontinuity(n)** is lowered exactly like **\$bound_step**, but writing a negative sentinel (**-1.0**) to `PlaceKind::BoundStep` instead of a real bound. OSDItrunc interprets a negative **bound_step** not as a literal step limit but as "a discontinuity occurred here": it clamps the next timestep to the last accepted step (`CKTdeltaOld[0]`), so the step cannot grow across the event. A positive **bound_step** keeps its original meaning (an explicit **\$bound_step** bound). Because **bound_step** defaults to +INFINITY and real bounds are positive times, a negative value is an unambiguous sentinel.

This required no OSDI ABI change (the **bound_step** slot already exists) and only a few lines in each of OpenVAF and ngspice.

Files changed

- OpenVAF `hir_lower/src/expr.rs` — **\$discontinuity(n)** for $n \geq 0$ now `def_place(PlaceKind::BoundStep, -1.0)` (the **\$discontinuity(-1)**-inside-limit case is unchanged).
- ngspice `src/osdi/osditrunc.c` — a negative **bound_step** clamps the next timestep to `CKTdeltaOld[0]` (the last accepted step) rather than being used as a literal bound.

Verification

`examples/discontinuity_examples/verify_discontinuity.py` (ALL PASS) — a conductance switch ($I = g \cdot V(a,b)$, g jumps at $V(a,b)=v_{th}$) that announces **\$discontinuity(0)** while in the switched region:

- timestep limiting — the same transient produces $\sim 590\times$ more (finer) timepoints with the announcement on than off, i.e. the discontinuity actually limits the timestep;
- solution unchanged — the DC operating point is identical either way (the announcement changes timestep control, never the computed result).

\$bound_step (positive values) is unaffected by the sentinel change, and every prior example folder still passes.

Known limitations

- The degree n is treated uniformly (any $n \geq 0 \Rightarrow$ "limit the step here"); ngspice's OSDI timestep control has no finer degree-specific hook.
- `$discontinuity` and `$bound_step` share the `bound_step` output slot within one evaluation (last-writer-wins); a negative value is the discontinuity sentinel, a positive value an explicit bound.
- Requires the accompanying ngspice rebuild; a stock ngspice ignores the sentinel (a negative `bound_step` there would be a no-op or, in an unpatched OSDI `trunc`, wrongly clamp to a negative step — so the two must be built together).

Enhancement-25 — `$simparam$str(name)` support

This document describes the changes made to OpenVAF-r *and* ngspice-46 in the version11/ directory to make `$simparam$str(name)` work. It returns a *string* simulator parameter (the string counterpart of the numeric `$simparam`). Previously it was completely unusable, due to three independent defects — two in OpenVAF, one in ngspice.

The three defects

1. Wrong return type (OpenVAF, `hir_ty/src/builtin.rs`). The builtin signature declared `SIMPARAM_STR(...) -> Real`, so any use in a string context (assigning to a string variable, comparing to a string literal, `%s` in `$strobe`) was a type mismatch: expected string value but found real value. Fixed to `-> String`.
2. Bugged runtime lookup (OpenVAF, `osdi/stdlib.c`). `simparam_str` iterated the *numeric* name list (`params->names[i]`) and, on a match, returned the *name* (`params->names_str[i]`) instead of the *value* — and, because the string list is shorter than the numeric one, read past the end of `names_str`. Fixed to walk the NULL-terminated `names_str` list and return `vals_str[i]`.
3. No string parameters exposed (ngspice, `src/osdi/osdiload.c`). `get_simparams` set `sim_params_str = {NULL}` and `vals_str = NULL`, so even a correct lookup had nothing to find. Now it exposes two string parameters and fills their values each call.

What ngspice exposes

- **"analysis_name"** — "dc" / "ac" / "tran" / "noise", derived from CKTmode with the same convention as `analysis()` (noise → ac → tran → dc).
- **"simulator"** — the constant "ngspice".

An unknown name raises a fatal "unknown *simparam_str*(*matchingthenumeric*'simparam' with no default).

Files changed

- OpenVAF `hir_ty/src/builtin.rs` — `SIMPARAM_STR` return type `Real` → `String`.
- OpenVAF `osdi/stdlib.c` — `simparam_str` walks `names_str` and returns `vals_str`.
- ngspice `src/osdi/osdiload.c` — `get_simparams` exposes "analysis_name" and "simulator" (the second enhancement to also touch the ngspice source, after Enhancement-24). No OSDI ABI change.

The OpenVAF lowering (`CallbackKind::SimParamStr` → the `simparam_str` runtime, returning a string pointer) already existed and is unchanged.

Verification

`examples/simparamstr_examples/verify_simparamstr.py` (ALL PASS) — a model that sets its conductance from `$simparam$str("analysis_name")` (read into a string variable and compared): `g_dc` in `dc/op`, `g_ac` in `ac`, `g_tran` in `tran`. Running each analysis and checking the terminal current confirms the correct string is returned in each case. String comparison (`== "tran"`) and graceful handling of an unknown name were also checked ad hoc. The numeric `$simparam(name[, default])` is unchanged and still works, and every prior example folder still passes.

Known limitations

- Only "analysis_name" and "simulator" are provided; other Verilog-AMS string parameters (e.g. "cwd", hierarchical "path"/"instance") are future work.

- The LRM variadic numeric form `$simparam("a","b")` (match any) is still single-argument (unrelated to this enhancement).
- Requires the accompanying ngspice build.

Enhancement-26 — `ac_stim(...)` baseline (crash fix + correct large-signal semantics)

This document describes the change made to OpenVAF-r in the `version11/` directory for `ac_stim([name][, mag][, phase])`, the Verilog-AMS small-signal AC stimulus source. This is the baseline step: it eliminates a compiler crash and gives `ac_stim` its correct large-signal value. Full small-signal AC injection is a larger follow-up (scoped below).

The bug

`ac_stim`'s signatures were declared (so it type-checked), and the lowering had a `no_equations` fallback returning 0. But any contributing use of `ac_stim(I(a,b) <+ ... + ac_stim(...))`, in the normal has-equations path) matched no dedicated arm and fell through to the builtin match's `_ => unreachable!()`, panicking the compiler with `internal error: entered unreachable code`.

The fix

`hir_lower/src/expr.rs` now has a dedicated `BuiltIn::ac_stim => F_ZERO` arm. Per the Verilog-AMS LRM, `ac_stim` evaluates to 0 in the large-signal (DC and transient) domain and injects `mag/phase` only during small-signal (AC) analysis. Returning `F_ZERO` is therefore the correct large-signal value, and it stops the crash — a model using `ac_stim` (in any of its four signature forms) now compiles and simulates. One-line change; no OSDI ABI change, no ngspice change.

Verification

`examples/acstim_examples/verify_acstim.py` (ALL PASS):

- the model compiles (it previously crashed `openvaf-r`);
- DC and transient currents equal $g \cdot V(a, b)$ and are identical with the `ac_stim` terms included vs excluded — i.e. `ac_stim` correctly contributes 0 in the large-signal domain.

Every prior example folder still passes.

Known limitation — AC injection (the follow-up)

The one thing this baseline does not do is the actual small-signal AC injection: during AC analysis, `ac_stim(name, mag, phase)` should inject `mag/phase` into the AC right-hand side (it currently contributes 0 there too, so a testbench using `ac_stim` as its AC source sees no excitation).

Implementing that is a substantial, ABI-touching subsystem — essentially the *noise* infrastructure rebuilt for AC:

- `hir_lower` — a new `CallbackKind::AcStim` producing a recognized small-signal source value;
- `sim_back` — recognise it in `topology/linearize` (like `Noise::new`), mark it small-signal, and carry a new `ac_sources` list (node pair + complex value) through `dae/builder`;
- OSDI (ABI addition) — a new descriptor entry (a `load_spice_rhs_ac` function or `ac_source_infos` + offsets) with codegen in `load.rs/inst_data.rs/metadata.rs/eval.rs` and `osdi_0_4.h`;
- ngspice — `OSDIacLoad` stamps the complex `CKTrhs/CKTirhs` via the node mapping, with `OSDIsetup/osdidefs.h/osdi.h` support.

Because that modifies the AC load/descriptor path shared by every OSDI device, it warrants its own focused, per-layer-tested enhancement rather than being folded in here.

Enhancement-27 — `idtmod(...)` modulo-integrator fix

This document describes the change made to OpenVAF-r in the `version11/` directory to fix `idtmod(expr, ic, modulus[, offset[, ...]])`, the Verilog-AMS modulo time-integrator (the standard VCO/PLL phase integrator). `idtmod` compiled and its integration was correct for the first period, but the modulo wrap was broken — and there was a second, unrelated argument bug.

The two bugs

1. The modulo wrap diverged. `idtmod` lowers to an implicit DAE equation whose reactive residual is the integrated state $q = \text{val}$ (so $d(\text{val})/dt = \text{expr}$). The old lowering (`lower_integral`) detected $\text{val} > \text{modulus}$ and set the residual to $[\text{val} - \text{min}, F_ZERO]$ — forcing $\text{val} = \text{min}$ and zeroing the reactive residual. But the transient integrator computes the branch current from the reactive residual's history: at the wrap step it becomes $\approx (0 - q_{\{n-1\}})/dt$ with $q_{\{n-1\}} \approx \text{modulus}$, an enormous term the resistive part then “cancels” by driving $\text{val} \approx q_{\{n-1\}}/dt$. So the state either got stuck (a VCO froze at ~ 0.297) or blew up (a sawtooth shot to ~ -799) at the first wrap. Only the first period — before any wrap — worked.
2. The offset argument was wrong. For `idtmod(expr, ic, modulus, offset)` the offset was read from `args[2]` (which is the *modulus*) instead of `args[3]`, so `IdtKind::ModulusOffset` used the wrong offset entirely.

The fix

Integrate the DAE state unbounded — plain integration in the residual, exactly like `idt`, with no discontinuity for the transient integrator to trip over — and apply the modulo wrap only to the returned value:

```
result = offset + floor_mod(val - offset, modulus),    floor_mod(x, m) = x - m·floor(x/m) ∈ [
```

so the returned value lands in $[\text{offset}, \text{offset} + \text{modulus})$ (correct even when the integral goes negative), and the offset now reads `args[3]`. Because the DAE state stays smooth, the transient integrator never sees the wrap discontinuity, so it no longer diverges. `hir_lower/src/expr.rs` only — no OSDI ABI change, no ngspice change.

Verification

`examples/idtmod_examples/verify_idtmod.py` (ALL PASS):

- VCO — a modulo-1 phase drives $\sin(2\pi \cdot \text{phase})$; the output tracks $\sin(2\pi \cdot \text{freq} \cdot t)$ to $\sim 1e-4$ across three periods (it used to freeze after one);
- sawtooth — `idtmod(1, 0, 2, off)` produces a clean sawtooth whose value stays in $[\text{off}, \text{off} + 2)$ and wraps correctly for both `off=0` and `off=5` ($\sim 1e-16$ error away from the wraps) — exercising the fixed offset argument.

Plain `idt` (no modulus) is unchanged and still exact, and every prior example folder still passes.

Known limitations

- The DAE state integrates unbounded, so over very long runs (many millions of wraps) the phase magnitude grows and the wrapped output loses a little floating-point resolution. A bounded-state modulo integrator would need simulator-side breakpoint/state-reset support that OSDI does not expose; the unbounded-state form is correct and, crucially, does not diverge.
- The `tol/nature` argument forms are accepted but tolerance is not separately modelled (unchanged from before).

Enhancement-28 — `idt(...)` initial-condition fix

This document describes the change made to OpenVAF-r in the `version11/` directory to fix the initial condition of `idt(expr, ic[, ...])` in transient analysis. `idt` integrated correctly, and its IC was honoured at the DC operating point, but the IC was silently lost in transient.

The bug

`idt(expr, ic)` is an ideal integrator: its value is `ic` at $t=0$ and then integrates `expr`. Observed behaviour:

- `.op` of `idt(1, 3)` returned 3 — the IC was applied at DC;
- but a transient of `idt(1, 3)` started at 3 for the very first sample and then immediately dropped to 0 and integrated from there, giving $v = t$ instead of $v = 3 + t$;
- `idt(0, 7)` (zero integrand — should hold at 7) drifted from 7 to 0.

The LRM says the IC is “*used as the starting value for transient analysis*,” so this is a genuine bug.

Root cause `idt` lowers to an implicit DAE equation $\text{resist} + d/dt(\text{react}) = 0$ whose unknown is `val` and whose reactive residual `react` is the integrator’s stored charge Q . `EnableIntegration` is false during the DC operating point and true during transient. The old lowering used, for the IC (integration-disabled) phase:

```
[ val - ic, F_ZERO ]      // resist = val - ic (pins val = ic); react = 0
```

So at the DC operating point `val = ic` but the stored charge $Q = 0$. When transient integration turned on (`react = val`), the integrator’s history had $Q = 0$, so it restarted the integral from 0 — the IC was applied at DC but never carried into transient.

The fix

Store the charge as `ic` during the IC/DC phase:

```
[ val - ic, ic ]          // resist = val - ic; react (charge) = ic
```

Now the DC operating point has `val = ic` and $Q = ic$, so when transient integration turns on it continues from $Q = ic$ (the state is continuous at the DC→transient boundary because `val = ic` there). One-value change in `hir_lower/src/expr.rs` (`lower_integral`); no OSDI ABI change, no ngspice change.

Verification

`idtic_examples/verify_idtic.py` (ALL PASS), an ideal integrator $v = ic + \text{rate} \cdot t$:

- the DC operating point equals `ic`;
- the transient ramp starts from `ic`: `idt(1, 3)` gives $3 + t$ ($\sim 1e-4$) — it used to give t ;
- with `rate = 0` the integrator holds at `ic`: `idt(0, 7)` stays at 7 (was drifting to 0).

`idtmod` (Enhancement-27) still works, and every prior example folder still passes.

Known limitations

- This fixes the DC→transient IC carry for `idt(expr, ic)` (and the same IC phase in `idtmod`). The mid-transient **assert** reset form `idt(expr, ic, assert)` — a runtime reset of the integrator to `ic` whenever `assert != 0` — is a separate, harder problem: it forces a discontinuous state jump the transient integrator cannot follow smoothly (like the raw `idtmod` wrap), and remains unaddressed.

- `idt(expr)` with no IC still has an unconstrained DC operating point (nothing pins the integration constant), unchanged.

Enhancement-29 — port-branch flow access **I(<port>)**

This document describes the change made to OpenVAF-r in the `version11/` directory to make the port-branch flow probe **I(<port>)** functional. The front-end already parsed, type-checked and lowered **I(<p>)**, so models *compiled*, but the value was always 0 at run time — it was an unfinished (`// TODO?`) stub.

What **I(<port>)** is

I(<p>) — the *port branch* — is the current flowing into the module through terminal *p*. It is most commonly used to build current-controlled sources (CCCS / CCVS) and to monitor terminal currents:

```
I(out, com) <+ k * I(<in>);    // CCCS: output current = k * (current into port in)
```

By Kirchhoff's current law the current entering the module at port *p* equals the net device current flowing out of node *p*, i.e. the sum of every branch contribution at that node.

The bug

I(<p>) lowers to `ParamKind::Current(CurrentKind::Port(node))`. That parameter was never wired into the DAE system, so it read as 0:

- **sim_back/src/dae/builder.rs** — in `build_input_unknown_pairs` the port case was literally:

```
CurrentKind::Port(_) => {  
    // TODO?  
}
```


so a port flow was never registered as a model input / DAE unknown.
- **osdi/src/eval.rs** — the eval reader hard-coded it to zero:

```
ParamKind::Current(CurrentKind::Port(_)) => cx.const_real(0.0),
```

A model like `I(out) <+ 10*I(<in>)` therefore produced `i(out) = 0` regardless of the current actually flowing into `in`.

The fix

Port flow is given a real DAE unknown with a defining equation, reusing the exact machinery that already backs named/unnamed branch-current probes. Three small changes, all confined to `sim_back` + `osdi`:

1. **sim_back/src/dae/builder.rs** — new `build_port_flow_equations()`, called at the top of `finish()` (before derivatives/Jacobian are computed). For every probed port flow **I(<p>)** it synthesises the equation

$$\text{residual}[\text{Current}(\text{Port}(p))] = \text{residual}[\text{KCL}(p)] - I(<p>) \quad \Rightarrow \quad I(<p>) = \text{residual}[\text{KCL}(p)]$$

by mirroring node *p*'s resistive and reactive Kirchhoff residual into the port-current row and subtracting the unknown. Because the reactive part is mirrored too, the solved value includes displacement (capacitive) current for free. The mirrored **I(<p>)** value is the very same parameter the model reads, so any source built on **I(<p>)** sees the correct current. A port branch has no `branch(...)` object, so it never passes through `build_branch` / `add_source_equation`; this is why the equation has to be synthesised here.

2. **sim_back/src/dae/builder.rs** — `build_input_unknown_pairs()`: the `CurrentKind::Port` special case is removed, so port currents are registered as ordinary model inputs like any other branch current.

3. **osdi/src/eval.rs**: the hard-coded `const_real(0.0)` arm is removed, so `ParamKind::Current(CurrentKind::Port)` falls through to the generic `get_prev_solve(SimUnknownKind::Current(kind))` path and reads its solved value.

The OSDI descriptor side needed nothing new — `osdi/src/metadata.rs` already named the unknown flow(<node>).

Verification — **portflow_examples/**

`portflow_demo.va` is a CCCS $I(\text{out}, \text{com}) = k \cdot I(\text{<in>})$ whose input terminal is an `rin || cin` load, so $I(\text{<in>}) = V(\text{in}, \text{com}) / r_{in} + c_{in} \cdot d/dt V(\text{in}, \text{com})$. `verify_portflow.py` (ALL PASS) checks, end-to-end through version11's own `openvaf-r + ngspice`:

1. resistive (DC) — $i(\text{vout}) = -k \cdot v_{in} / r_{in}$ (e.g. -20 mA for $k=10$, $v_{in}=2$, $r_{in}=1k$); this was 0 before the fix;
2. gain scaling — $i(\text{vout}) / i(\text{vin}) == k$ for $k \in \{1, 5, 25, 100\}$;
3. reactive (AC) — with `cin`, the port flow carries the in-phase ($1/r_{in}$) and quadrature ($w \cdot c_{in}$) parts, and $|i(\text{vout})| = k \cdot |i(\text{<in>})|$ — proving displacement current flows through the probe.

The implementation is completely general: resistive, reactive, and mixed resistive+reactive port loads all work.

Gotcha (ngspice, not OpenVAF)

Do not name a Verilog-A module `cccs`, `vccs`, or `vcvs`. Those names collide with ngspice's built-in controlled-source device types; ngspice then null-derefs in `create_model` when the `.model` card is parsed. This is unrelated to port flow (any OSDI module so named crashes) — just choose a different module name (the demo uses `portflow_cccs`).

Scope

- Pure `sim_back` + `osdi` change; the parser / type system / HIR lowering of `I(<p>)` were already in place.
- Vector (whole-bus) branches remain out of scope: `branch(bus, gnd)` is diagnosed (requires a bit-select); explicit per-bit `branch(bus[i], gnd)` works (E-3/E-4).

Enhancement-30 — variadic `analysis(arg1, arg2, ...)`

This document describes the change made to OpenVAF-r in the `version11/` directory to support the multi-argument list form of the Verilog-AMS `analysis()` system function.

What `analysis()` is

`analysis()` (Verilog-AMS LRM 4.7.1) queries the analysis currently being run. It takes a list of analysis-name strings and returns true (1) if the current analysis matches any of them:

```
if (analysis("ic", "dc"))      ...    // static / DC-like phases
if (analysis("ac", "noise"))   ...    // small-signal phases
```

Recognised names are "ac", "dc", "tran", "ic", "static", "noise", "nodeset". Several can hold at once — e.g. an operating point sets both "static" and "dc".

The bug

The single-argument form already worked end-to-end (the stdlib `analysis()` reads `sim_info->flags` and ngspice sets those flags correctly for op/dc/ac/tran/noise). But the builtin was declared with exactly one signature:

```
ANALYSIS = const {
    fn ANALYSIS_SIG(Val(String)) -> Integer;
}
```

so `max_args = 1`, and any list form was rejected at compile time:

```
error: invalid argument count: expected 1 arguments but found 2
|     if (analysis("ac", "tran")) ...
```

Only `analysis("ac")` compiled — you had to chain `analysis("ac") || analysis("tran")` by hand, which is not what the LRM prescribes.

The fix

Two small changes:

1. **hir_ty/src/builtin.rs** — `analysis` is redefined as a varargs builtin (like `$display/$limit`): one mandatory `String` argument, no upper bound.

```
const ANALYSIS: BuiltinInfo = BuiltinInfo::varargs(
    &[SignatureData { args: Cow::Borrowed(&[Val(String)]), return_ty:
        ↪ Type::Integer }],
    false,    // pure / read-only, no side effects
);
```

The generic vararg path (`max_args == None`) accepts the extra arguments; the now-unused `ANALYSIS_SIG` signature index is removed.

2. **hir_lower/src/expr.rs** — the lowering emits the `Analysis` callback for each argument and bitwise-ORs the results:

```
BuiltIn::analysis => {
    let mut acc: Option<Value> = None;
    for &arg in args {
        let name = self.lower_expr(arg);
        let hit = self.ctx.call1(CallBackKind::Analysis, &[name]);
```

```

    acc = Some(match acc { None => hit, Some(prev) =>
↪   self.ctx.ins().ior(prev, hit) });
    }
    acc.unwrap() // min_args == 1 guarantees ≥1 argument
}

```

OR (not a sum) matters: at an operating point both "static" and "dc" return 1, so a sum would yield 2 — the bitwise OR clamps the result to a proper 0/1.

No OSDI ABI change and no ngspice change — the `stdlib analysis()` and ngspice's flag-setting were already correct; only the compiler front-end arity was wrong.

Verification — **analysis_examples/**

`analysis_demo.va` is a conductance that is `g_static` at the DC operating point and `g_dynamic` for the dynamic analyses, selected by a single list-form call `if (analysis("ac", "tran", "noise")) g = g_dynamic;`. `verify_analysis.py` (ALL PASS) checks, end-to-end through version11's own `openvaf-r + ngspice`:

1. the multi-argument list form compiles (it used to be a hard error);
2. DC → `g_static` (none of `ac/tran/noise` match);
3. AC → `g_dynamic` (matches "ac");
4. TRAN → `g_dynamic` (matches "tran");
5. OR, not sum — `analysis("static","dc","ic")` at `.op` returns exactly 1, even though both "static" and "dc" are set.

Single-argument `analysis()` is unchanged and still works (regression-checked against `simparamstr_examples/`).

Enhancement-31 — complex poles/zeros in laplace/zi root forms

This document describes the change made to OpenVAF-r in the `version11/` directory to support complex conjugate poles and zeros in the root-based forms of the Laplace and z-domain filter operators.

What the root forms are

The Verilog-AMS filter operators come in four forms each, differing in how the numerator and denominator are given:

suffix	numerator	denominator
<code>_nd</code>	polynomial coefficients	polynomial coefficients
<code>_np</code>	polynomial coefficients	poles (roots)
<code>_zd</code>	zeros (roots)	polynomial coefficients
<code>_zp</code>	zeros (roots)	poles (roots)

for both `laplace_*` and `zi_*`. Per the LRM the pole/zero vectors hold (real, imaginary) pairs: element $2k$ is the real part and $2k+1$ the imaginary part of root k . A complex conjugate pair is `{re, +im, re, -im}`; a real root is `{re, 0}`.

The bug

`laplace_np/laplace_zd/laplace_zp` and `zi_np/zi_zd/zi_zp` all lowered their root vectors through a single helper, `laplace_roots_to_poly`, which expanded the array as a list of individual real roots $\prod_k (s - r_k)$ — ignoring the (real, imaginary) pairing entirely. Consequences:

- a vector like `{-1e6, -3e6}` was read as two real poles ($-1e6$ and $-3e6$) instead of the LRM's one complex pole ($-1e6 - 3e6j$);
- complex conjugate poles/zeros could not be expressed at all, so every resonant / underdamped second-order section was impossible. A $Q=5$ resonant low-pass built with `laplace_np` and the correct complex pole pair produced -242 dB of garbage (the imaginary parts were treated as extra real roots, one of them in the right-half-plane).

The fix

`laplace_roots_to_poly([hir_lower/src/expr.rs])` now consumes the vector as (real, imaginary) pairs and forms $\prod_k (s - (re_k + j \cdot im_k))$ with full complex arithmetic — each polynomial coefficient carried as a (re, im) pair of MIR Values — and returns only the real coefficients:

```
// coefficients as (re, im) Values, ascending powers; start at 1 + 0j
let mut re = vec![F_ONE];
let mut im = vec![F_ZERO];
let mut k = 0;
while k < roots.len() {
    let rr = roots[k];
    let ri = if k + 1 < roots.len() { roots[k + 1] } else { F_ZERO };
    k += 2;
    // new = s*P - (rr + j*ri)*P    (complex multiply, then subtract from the
    //   ↪ shift)
    ...
}
re // imaginary parts cancel for conjugate-paired (physical) inputs
```

For a physical, real-coefficient transfer function the roots come in conjugate pairs, so the imaginary parts of the product cancel to zero; taking the real part both realises that and harmlessly drops floating round-off. A trailing unpaired element (odd-length vector) is treated as a purely real root, so a lone real pole/zero may still be written `{r}` as well as `{r, 0}` (keeps single-real-root models working).

This one helper is shared by all six root-based forms — both `laplace_*` and `zi_*` — so the single change covers every one of them. No OSDI/ngspice change; the roots are turned into ordinary polynomial coefficients exactly as before, just correctly.

Behaviour change Because the vector is now interpreted as pairs, an even-length vector means half as many roots as it used to. Existing models that relied on the old real-list convention must switch to the paired form: `{-1e6, -3e6}` (two real poles) becomes `{-1e6, 0, -3e6, 0}`. The shipped `laplace_examples/laplace_variants.va` was updated accordingly (single real roots such as `{-2e6}` are unaffected thanks to the lone-element leniency).

Verification — `complexpole_examples/`

`complexpole_demo.va` builds two second-order sections that require complex roots and cross-checks the root form against the equivalent `laplace_nd` polynomial baseline. `verify_complexpole.py` (ALL PASS) checks, end-to-end through version11's own `openvaf-r + ngspice`:

1. a resonant low-pass via `laplace_np` (complex conjugate poles) matches `laplace_nd` to `0.00` dB across the sweep;
2. the complex poles produce a real resonant peak of `+18.06` dB at exactly 1 MHz (textbook $20 \cdot \log_{10}(Q=8)$) — impossible with real-only roots;
3. a notch via `laplace_zd` (imaginary-axis complex zeros at $\pm j \cdot \omega_0$) matches `laplace_nd` to `0.00` dB, with a deep null (`-290` dB) at ω_0 .

Additionally spot-checked: `laplace_zp` (complex poles *and* zeros) and `zi_np` (complex poles in the z^{-1} domain, matched to the equivalent `zi_nd` real polynomial). Existing `nd` and single-real-root examples (`laplace_lpf`, `zi_lpf`, `laplace_zd_only`, `laplace_mixed_var_literal`) are unchanged and regression-checked.

Enhancement-32 — integer persistent/event-state variables

This document describes the changes made to OpenVAF-r and ngspice in the `version11/` directory to fix a compiler crash on integer persistent state and to let integer operating-point variables be recorded in ngspice output vectors.

The bug

A Verilog-A variable holds *persistent state* when its value must survive from one evaluation to the next — either because it is read before it is written (a running peak, an accumulator) or because it is only updated inside an event block (`@(cross)`, `@(above)`, `@(timer)`, `@(initial_step)`). Enhancement-7/8 implemented this via per-variable *persistent eval-output slots* in the OSDI instance data, read at the start of `eval()` (`ParamKind::HiddenState/EventState`) and stored back at the end.

Real-typed persistent variables worked. But any integer persistent variable crashed the compiler:

```
integer m;  
if (V(a,c) > 1.0) m = m + 1;    // read-before-write -> persistent
```

```
LLVM ERROR: Cannot select: 0x...: f64 = add ...  
... f64,ch = load<(load (s64) from %ir.18)> ...
```

(or a segfault, depending on surrounding code). Found during a deep-dive TODO sweep: the two `todo! ("hidden state/event state")` stubs in `osdi/src/inst_data.rs` sit on exactly this hole.

Root cause `OsdiInstanceData::new` created every hidden-state slot with a hardcoded **f64** type:

```
Some((var, eval_outputs.insert_full(val, ty_f64).0))    // hidden_state
```

The slot's recorded type drives *both* the end-of-eval store and the start-of-eval `read_hidden_state` load. For an integer variable the state was therefore loaded back as a double and fed straight into integer MIR ops — malformed LLVM IR (`f64 = add`), which aborts instruction selection. Worse, `eval_outputs` is keyed by MIR value and `insert_full` overwrites on duplicate keys, so the hardcoded `f64` could also clobber the correctly-typed slot entry that the opvar path had already inserted for the same value.

The fix

OpenVAF (`osdi/src/inst_data.rs`)

1. Type the hidden-state slot from the variable — exactly like the opvar path does:

```
let ty = lltype(&var.ty(db), cx);  
Some((var, eval_outputs.insert_full(val, ty).0))
```

Event-state slots keep `f64` (their values are always the crossing expression / next-fire time — genuinely real).

2. Retire the two `todo! ("hidden state/event state")` stubs in `load_eval_output / nth_opvar_ptr`. They are unreachable today (`EvalOutput::new` only produces `EvalOutput::Param` for `Param/ParamSysFun/Temperature` kinds), but they now resolve the parameter to its persistent eval-output slot instead of panicking — a real implementation rather than a stub, in case a future path produces state-typed `EvalOutput::Params`.

ngspice (`src/frontend/outitf.c`) With the compiler fixed, exposing the counter as an operating-point variable (`(*desc="..."*) integer n;`) compiled and read correctly at end of run, but per-timestep recording (`save @n1[n] + tran`) printed `OUTpData: unsupported data type` once per timestep and produced no vector:

- `getSpecial()` masked the vector type with `IF_REAL | IF_COMPLEX`, turning an integer instance-param's `IF_INTEGER` into 0 so no branch matched. The mask now preserves `IF_INTEGER`.
- Both per-point writers (the in-memory plot path and the rawfile path) now record `IF_INTEGER` values via `plotAddRealValue/fileAddRealValue((double) val.iValue)` — integer opvars land in output vectors as reals, like every other plot vector.

Verification — `intstate_examples/`

`intstate_demo.va` packs the three cases that used to crash plus a real-typed regression control: an integer `@(cross)` edge counter `n` (opvar), an integer `@(initial_step)` flag `started` (opvar), and the real running-peak `vpeak` (E-7 behaviour). `verify_intstate.py` (ALL PASS), driving a 2 V / 1 kHz sine for 5 cycles:

1. it compiles (integer persistent state used to abort the compiler), and the run emits no unsupported data type;
2. final `n == 5` — exactly the number of upward 1 V crossings;
3. the recorded per-timepoint waveform of `n` is a clean staircase `0,1,2,3,4,5`, stepping at the analytic crossing times $\text{asin}(v_{th}/A)/2\pi f + k/f$ (max error $< 10 \mu\text{s} = \text{one timestep}$);
4. `started` reads 1;
5. `vpeak` reads the sine amplitude (real persistent state unchanged).

Regressions: the E-7 (`variable_persistence_examples`) and E-8 (`cross_examples/timer_examples`) transient decks re-run cleanly with the new toolchain, and the full pre-fix crash matrix (9 models: integer/real $\times$ persistent/event/plain $\times$ exposed/hidden) now compiles.

Enhancement-33 — array **case** + array-literal function arguments

This document describes the changes made to OpenVAF-r in the `version11/` directory to retire the compiler's last `todo!()` hard-panic stubs and fix the array-expression bugs found underneath them. Purely front-end (`hir_ty` + `hir_lower`); no OSDI/ngspice change.

The bugs

A deep-dive TODO sweep (the one that produced Enhancement-32) left two `todo!()` stubs in `hir_lower`. Probing their reachability uncovered four related defects:

1. **case** over an array crashed the compiler. Type inference happily accepted an array-typed discriminant (demanding array-typed items), but lowering hit `Type::Array { .. } => todo!()` in `lower_case`:

```
case ('{1.0, 2.0})
  '{1.0, 2.0}: g = 1e-3;    // -> "not yet implemented" panic
```

2. Array-literal function arguments silently bound nothing. A whole-array function argument (Enhancement-18) only handled bare array-*variable* references; a literal compiled cleanly but left every element 0:

```
g = sum2('{1.0, 2.0});    // returned 0 instead of 3 – silent wrong answer
```

3. Whole-array variables were typed **real** regardless of their element type. `infer_array_arg` hardcoded `Type::Real`, so an *integer*-array case discriminant compared its i32 elements with `feq` — a second panic (`invalid int operation feq in const-eval`).
4. Array literals were silently accepted as array *output* arguments (the Enhancement-20 writeback just skipped), while scalar output arguments have always properly required a variable.

The fixes

- Element-wise array **case** (`hir_lower/src/stmt.rs`): the discriminant and each case item are lowered to their element `Values` and the per-element equalities (`feq/ieq/beq/seq` by element type) are AND-combined into the single branch condition — an arm matches iff all elements are equal. The scalar path is unchanged (it is just the one-element case). Works for array literals and whole-array variables, real and integer, in both positions; mismatched lengths stay a clean type error.
- Shared array-expression helpers: the laplace-specific helpers were the general mechanism all along, so they are renamed and reused — `lower_laplace_array_arg` → `lower_array_elems` (`hir_lower`) and `infer_laplace_array_arg` → `infer_array_arg` (`hir_ty`). The case inference arm now runs discriminant/items through `infer_array_arg`, which also registers whole-array variable references (previously `case (x)` on an array variable was rejected with “requires a bit-select”).
- Array-literal function arguments (`hir_lower/src/expr.rs`, `lower_user_fun_impl`): the input-binding now uses `lower_array_elems`, which handles literals (each element lowered) and variable references (each element read) identically — `sum2('{1.0, 2.0})` returns 3.
- True element types (`hir_ty/src/inference.rs`, `infer_array_arg`): a whole-array variable is typed from its actual element variables instead of hardcoded `real`, so integer arrays compare with `ieq` and mixed-type discriminant/item combinations are diagnosed instead of miscompiled.
- Array output arguments require a variable (`hir_ty/src/inference.rs`): an array literal passed to an output/inout array formal is now rejected with the same “expected ... variable reference” diagnostic scalars get, instead of silently skipping the writeback.

- The stubs are retired: `lower_case`'s `todo!()` is replaced by the working array path, and `lower_array`'s `todo!("arrays")` becomes a documented unreachable! — every context that accepts an array value consumes its elements directly (aggregate assignment, `laplace/zi`, `$table_model`, function arguments, `case`), and inference rejects arrays everywhere else, so no array expression can reach the generic scalar lowering path.

Verification — **arraycase_examples/**

`arraycase_demo.va` classifies the input voltage into a 2-bit integer state vector and selects the conductance with one element-wise array case (`case (st) '{0,0}: ... '{1,0}: ... '{1,1}: ...`), scaled by a helper summing an array-literal argument (`sum2('{0.25, 0.75}) == 1.0`). `verify_arraycase.py` (ALL PASS) checks, end-to-end through version11's own `openvaf-r + ngspice`:

1. it compiles (array case used to panic the compiler);
2. all three case regions select the correct conductance in a DC sweep — which simultaneously proves the literal function argument reads 1.0, not 0;
3. a real-array case (literal discriminant vs literal item) compiles and matches;
4. an array literal passed to an array output argument is rejected with a proper type error.

Regressions: every array-consuming feature re-verified — `funcarray` (E-18), `arrayout` (E-20), `arrayret` (E-23), `array/mdarray` (E-14/15), `cubic_table/table_model` (E-16/22), `complexpole` (E-31, exercises the renamed `lower_array_elems` via `laplace`) — ALL PASS, plus `zi_lpf` and a scalar case model behave unchanged.

Enhancement-34 — `{...}` concatenation & `{n{...}}` replication

This document describes the changes made to OpenVAF-r in the `version11/` directory to implement the Verilog-AMS concatenation and replication operators properly. Purely front-end (parser → syntax → hir_def → hir → hir_ty → hir_lower); no OSDI/ngspice change.

The two brace constructs

Verilog-AMS has two distinct brace constructs that OpenVAF previously conflated:

syntax	construct	OpenVAF before E-34
<code>'{...}</code>	array aggregate literal (assignment patterns, incl. nested <code>'{'{...},...}</code> for N-D arrays)	supported (E-4/14/15)
<code>{...}</code>	concatenation operator; <code>{n{...}}</code> replication	parsed as just another spelling of the aggregate literal

Because `{...}` was treated as an aggregate literal: whole arrays could not appear inside it (`{p, q}` → "requires a bit-select"), the replication form `{n{...}}` did not parse at all (unexpected token `'{'`), and string operands produced a useless *string array* instead of the LRM's concatenated string.

What Enhancement-34 implements

`{...}` is now the real concatenation operator:

- numeric concatenation flattens its operands in order — scalars, whole-array variables, aggregate literals, nested concatenations — into one flat 1-D array value: `w = {half1, {3{k2}}, 3.0*k2};`
- replication `{n{...}}` repeats the flattened operand list `n` times, where `n` is a positive compile-time integer literal (diagnosed otherwise);
- string concatenation: if any operand is a string, all must be, and the result is a runtime-concatenated string — `{"volt","age"} == "voltage"`, including replication (`{2{"ab"}} == "abab"`). Lowered through the proven `$swrite/$sformat` machinery (a `PrintDst::String` print callback with a `"%s%s..."` format, so operand data is never interpreted as a format string);
- usable everywhere an array value is consumed: array assignment, whole-array function arguments, `laplace_*/zi_*` coefficient vectors, case discriminants/items;
- element typing: the result is `real` if any operand is real (integer *scalars* are cast, exactly like aggregate-literal elements; an integer *array* mixed into a real concatenation is a type error, since array elements have no per-element cast machinery).

`'{...}` aggregates are completely untouched.

Implementation sketch

- parser/syntax: `'{...}` and `{...}` now produce different nodes (`ARRAY_EXPR` vs new `CONCAT_EXPR/REPLICATION_EXPR`; the replication node's children are `[count, elem0, elem1, ...]`).
- hir_def/hir: new `Expr::Concat { rep: Option<ExprId>, elems }`.
- hir_ty: `infe_concat` types the expression (string mode vs flattened array, constant replication count, casts, diagnostics — new `InvalidReplicationCount/EmptyConcat` diagnostics); array assignment gains a `concat` case that expands into per-destination-element sources (`ArrayAssign::Concat`, each element either a scalar expression or a source array-element variable), which the existing mixed `Val/Copy ArrayAssignElem` machinery consumes unchanged.
- hir_lower: `lower_array_elems` (the shared array-expression flattener from E-33) flattens `concat` operands recursively and repeats the list for replication, which makes `concat` work in every array-

consuming context for free; a string-typed concat lowers via a new `lower_string_concat` helper on the E-11 formatting machinery.

Behaviour change `{...}` no longer denotes an aggregate literal. For the common flat scalar list ($x = \{1.0, 2.0\}$) concatenation produces the identical result, so nothing breaks; but a *nested* bare-brace literal ($\{\{1,2\}, \{3,4\}\}$), previously a 2-D aggregate spelling, now flattens (concatenation semantics — and 2-D destinations require `'{'. .}', '{'. .}'`, the LRM aggregate form, as all shipped examples already use).

Verification — `concat_examples/`

`concat_demo.va` assembles a 6-tap coefficient vector by concatenation + replication ($w = \{\text{half1}, \{3\{k2\}\}, 3.0*k2\}$), feeds a concat-built vector with integer scalars to an averaging function (`avg4({1, 3, 2.0, 2.0})` — casts), and gates the output branch on a runtime string concatenation (`{"con", "cat"} == "concat"`). `verify_concat.py` (ALL PASS):

1. it compiles (array operands / replication / string concat all used to fail);
2. the DC conductance equals $2 \cdot (3 \cdot k1 + 6 \cdot k2)$ exactly for two parameter sets — proving the concatenated coefficient vector, the replication, the integer casts and the string gate all evaluate correctly;
3. `{0{...}}` and a non-literal replication count are clean diagnostics.

Additionally exercised during development: `concat` as `laplace_nd` coefficient vectors, as a case discriminant, string replication, and replication of a mixed scalar/array list ($\{2\{p, 10.0\}\}$). Regressions: 15 example verify suites (all array-family + string/file I/O + analysis/portflow/intstate) ALL PASS, and `laplace_variants/zi_lpf/bessel5` compile unchanged.

Enhancement-35 — lexer hang on // comment at end-of-file

This document describes the change made to OpenVAF-r in the `version11/` directory to fix a compiler infinite loop on a `//` line comment terminated by end-of-file instead of a newline. One-line fix in the lexer; no OSDI/ngspice change.

The bug

Both Verilog-A comment forms were already fully supported — `//` line comments and `/* ... */` block comments, in every position (own line, trailing after code or compiler directives, mid-expression, multi-line, containing code-like text). But a `//` comment as the last line of a file with no trailing newline hung the compiler forever:

```
module m(a,c);
...
endmodule
// final comment<EOF>      <- no trailing newline: openvaf-r spins at 100% CPU
```

Files without trailing newlines are extremely common (editors, generators, truncated copies), and a hang is the worst failure mode — it looks like the tool froze, gives no diagnostic, and stalls any build/CI pipeline that invokes the compiler. The bug was pre-existing (reproduced with the CI-built binary); it was found while torture-testing comment support.

Root cause `line_comment` in `openvaf/lexer/src/lib.rs` scanned with:

```
loop {
  match self.first() {
    '\n' => break,
    '\\\n' if self.second() == '\n' => break,
    _ => self.bump(),
  };
}
```

At end of input the cursor's `first()` returns the `EOF_CHAR` sentinel forever while `bump()` no-ops — so a comment not terminated by `'\n'` never exits the loop. Its sibling `block_comment` drives on while `let Some(c) = self.bump()`, which terminates at EOF naturally — which is why an *unterminated* `/*` was always a clean unexpected EOF, expected `*/` error rather than a hang.

The fix

Add the missing end-of-file break:

```
'\n' => break,
 '\\\n' if self.second() == '\n' => break,
_ if self.is_eof() => break,      // Enhancement-35
_ => { self.bump(); }
```

An audit of every other scan loop in the lexer (whitespace, identifiers, digits, strings, block comments, `eat_while`) confirmed `line_comment` was the only EOF-unsafe loop: the rest either drive on `bump()`'s Option, check `is_eof()`, or break on the sentinel by predicate.

Verification — `comment_examples/`

`comment_demo.va` is a comment-torture model (line/block/multi-line/ mid-expression/trailing comments, code-like text inside comments). `verify_comment.py` (ALL PASS) checks, end-to-end through `version11`'s own `openvaf-r + ngspice`:

1. the hang reproducer — a file whose final bytes are exactly `// eof comment` with no trailing newline — compiles within a watchdog timeout (a regression trips the 20 s watchdog instead of hanging CI);
2. the torture model compiles and simulates with the exact expected current ($I = -2 \cdot 10^{-3} \cdot V$; a commented-out `I(a,c) <+ 999.0;` is ignored);
3. an unterminated `/*` at EOF stays a clean unexpected EOF error;
4. a `//` comment ending in a lone backslash at EOF (the escaped-newline lookahead touching end of input) also terminates.

Regressions: lexer unit tests 8/8 pass; all 73 version11 example models recompile; spot verify suites (concat, arraycase, array, stringio 6/6, fileio 9/9) ALL PASS.

Enhancement-36 — probe-only branches / ideal ammeter + flow-only signal flow

This document describes the change made to OpenVAF-r in the `version11/` directory to give probe-only branches their LRM-mandated 0V-source (ideal ammeter) semantics — which simultaneously completes support for flow-only signal-flow disciplines. One new pass in the DAE builder (`sim_back`); no OSDI/ngspice change.

The bug

Verilog-AMS distinguishes conservative disciplines (`electrical`: potential + flow, KCL/KVL) from signal-flow disciplines (`voltage`: potential only; `current`: flow only). Probing the flow of a branch that is never *contributed* to is a fundamental idiom:

```
branch (p, n) sense;
V(out) <+ rtz * I(sense);      // ideal ammeter + transimpedance readout
I(onp, com) <+ k * I(sense);   // CCCS on a sense branch (current mirror)
```

Per the LRM, such a probe-only branch behaves as a short — a potential source of 0 V — whose current is the probed value. In OpenVAF it instead read 0 and conducted nothing: the “ammeter” was an *open circuit*, silently breaking the surrounding series path. The same mechanism underlies flow-only (current discipline) signal-flow nets — an input port only *probes* its net’s flow — so entire current-signal chains produced 0.

Root cause The topology only materialises branches that are contributed to (it is keyed off the `IsVoltageSrc` outputs, which are created per contribution). A branch that is merely probed never passes through `build_branch/add_source_equation`, so its current unknown never exists and the OSDI eval falls back to the “always zero” path — the same failure family as Enhancement-29’s port-flow stub.

The fix

A new `build_probe_only_branches()` pass in `sim_back/src/dae/builder.rs`, run at the start of `finish()` right after Enhancement-29’s `build_port_flow_equations` (its direct template). For every live `ParamKind::Current` (named or unnamed branch; port flows excluded — E-29 owns those) whose `Current unknown` was not materialised by the contribution-driven pass, it synthesises exactly the system `add_source_equation` builds for a voltage branch with a zero source expression:

```
residual[Current(br)] = -V(hi,lo)      (nature Potential -> equation V(hi,lo) = 0)
residual[KCL(hi)]      += I(br)
residual[KCL(lo)]      -= I(br)
```

The branch current `I(br)` is the very parameter the model reads, so probes and any sources built on them see the solved through-current; the Jacobian entries come out of the ordinary derivative machinery since the pass runs before `sim_unknown_reads/auto_diff`. One caveat inherent to the semantics: putting *several* probe-only branches in parallel across the same node pair is degenerate (parallel ideal 0V sources), exactly as paralleling ideal voltage sources is.

Verification — `signalflow_examples/`

`signalflow_demo.va` packs all four system styles: a conservative ideal ammeter with transimpedance readout, a CCCS current mirror on a probe-only sense branch, a potential-only (`voltage` discipline) gain chain, and a flow-only (`current` discipline) source → gain → converter chain. `verify_signalflow.py` (ALL PASS):

1. the ammeter shorts (the series 2 V / 1 kΩ loop conducts its full 2 mA — it used to be open) and reads it (`v(out) = 2 V` exactly);

2. it reads displacement current in AC (series 1 nF at $\omega = 10^6$: $v(\text{out}) = j \cdot 1 \text{ V}$ exactly — the 0V source is fully linear and reactive-aware);
3. the current mirror senses 3 mA and outputs 6 mA;
4. the potential-only signal-flow chain gives $1.5 \times 3 \times 2 = 9 \text{ V}$ (this style already worked and is regression-locked here);
5. the flow-only chain gives $1 \text{ mA} \times 5 \times 1 \text{ k}\Omega = 5 \text{ V}$ (used to be 0), with the probed signal net sitting at exactly 0 V — textbook signal-flow semantics (the probe shorts the net and takes its total inflow as the signal).

Regressions: all 32 version11 example verify suites ALL PASS; the `sim_back` snapshot tests remain at their pre-existing 9-pass/15-fail baseline (no new failures).

System-style support after E-36

style	status
conservative (electrical)	full (unchanged)
signal-flow, potential-only (voltage)	full (already worked; now regression-locked)
signal-flow, flow-only (current)	full (new)
probe-only branches / ideal ammeters / CCCS-on-sense	full (new)

Enhancement-37 — operator-correctness audit + fixes

This document describes the changes made to OpenVAF-r in the `version11/` directory after a systematic operator-correctness audit covering the arithmetic, relational, logical, bitwise/shift, ternary and concatenation operator families. Three real defects were found and fixed; everything else checked out exactly correct. Purely front-end/middle-end (`hir_lower`, `mir_opt`, `hir_ty`); no OSDI/ngspice change.

The audit

`operator_examples/operator_audit.va` holds five self-checking modules — one per operator family. Every individual check that fails adds a distinct power of two to a score emitted on a signal-flow output, so `v(out) == 0` means every check in that family passes and any nonzero value is a bitmask pinpointing exactly which check failed. 60+ checks cover:

- integer arithmetic: `+` `-` `*` `/` `%` incl. truncation toward zero (`-7/3 == -2`) and modulus sign rules (`-7%3 == -1`, `7%-3 == 1`), `**`, unary minus, precedence;
- real arithmetic: `+` `-` `*` `/` `%` (fmod semantics, sign of first operand), `**` incl. negative exponents and negative base with odd integer exponent;
- relational/logical: `<` `<=` `>` `>=` `==` `!=` on integers and reals, 0/1 result values, `&&` `||` `!`, mixed expressions;
- bitwise/shifts: `&` `|` `^` `^^` `^~` `~`, `<<` `>>` `<<<` `>>>` incl. sign-extension vs zero-fill, shift-by-zero, involution (`~~x == x`), `&` `|` precedence;
- ternary: integer/real/string results, nested, comparisons inside conditions;
- concatenation/replication (regression-locking Enhancement-34).

The three defects found → fixed

1. `~x` (bitwise NOT) was lowered as arithmetic negation `hir_lower` mapped `UnaryOp::BitNegate` to `ineg (-x)`, so `~12` evaluated to `-12` instead of `-13` (`~x = -x - 1`). Silent wrong answers in any model using bit manipulation. Fixed to the `inot` opcode (whose `const-fold`, `!val`, was already correct — only the lowering picked the wrong instruction).

2. constant folding of `>>` sign-extended The MIR distinguishes `Ishr` (logical `>>`, zero-fill) from `Iashr` (arithmetic `>>>`, sign-extending), and the LLVM runtime path emits the correct `LShr/AShr` pair — but the constant folder in `mir_opt/src/const_eval.rs` computed both with Rust's `>>` on a signed `i32`, which sign-extends. So `-16 >> 2` folded to `-4` instead of the zero-filled `1073741820` (`0x3FFFFFFC`), and — worse — a constant-foldable `>>` disagreed with the same expression computed from runtime values. Fixed by folding through `u32`: `((lhs as u32) >> rhs) as i32`.

3. the ternary operator rejected string operands `cond ? "a" : "b"` was a type error (`SignatureData::SELECT` only allowed bool/real/integer branch pairs). Per the LRM the conditional operator supports string operands. Fixed by appending a `STR_BIN_OP` signature `((String, String) → String)` to the `SELECT` list — appended last, keeping the existing signature indices stable. The lowering needed no change (the existing `phi` handles string values).

Verification — `operator_examples/`

`verify_operators.py` (ALL PASS):

1. the audit file compiles (string ternaries used to be a type error);
2. all five family scores read exactly 0 (60+ individual operator checks, any failure would name itself in the bitmask);
3. the three formerly-broken cases asserted directly: `~12 == -13`, `-16 >> 2 == 1073741820`, `(1>0) ? "yes" : "no" == "yes"`.

Regressions: all 33 version11 example verify suites ALL PASS and all 75 example models recompile after the fixes (the `~/>>/ternary` changes touch shared lowering and const-eval paths, so the full sweep matters).

Enhancement-38 — operator-precedence audit + fixes

This document describes the changes made to OpenVAF-r in the `version11/` directory after a systematic operator-precedence audit against the Verilog-AMS precedence table (LRM Table 4-2). One observable defect and one harmless deviation were found and fixed — both in the parser's Pratt binding-power table (`parser/src/grammar/expressions.rs`); nothing else in the pipeline changes.

The audit

The parser's binding-power table was compared entry-by-entry against the LRM table (highest → lowest):

```
unary + - ! ~ > ** > * / % > + - > << >> <<< >>>
> < <= > >= > == != > & > ^ ~^ ^~ > | > && > || > ?:
```

with associativity rules from LRM 4.1.3: *all operators associate left to right except the conditional* (right to left). The structural review confirmed:

- left associativity for all binary operators — `expr_bp(p, op_bp + 1)` parses each right operand one level tighter, including `2**3**2 == (2**3)**2 == 64`;
- right associativity for `?:` — the else-branch reparses a full expression, so `a ? b : c ? d : e == a ? b : (c ? d : e)`;
- unary above `**` — prefix operators take an *atom* operand, so `-2**2 == (-2)**2 == 4` (the classic Verilog difference from C/Python).

`precedence_examples/precedence_audit.va` then verifies all of it empirically: 28 checks covering every adjacent level pair, each failing check adding a distinct power of two to a score on a signal-flow output (`v(out) == 0` ⇔ all pass; any nonzero value is a bitmask naming the failure).

The defects found → fixed

1. `%` bound tighter than `*` and `/` (observable) The LRM puts `*`, `/`, `%` on one left-associative level; the table gave `%` a higher binding power. Consequence: `a * b % c` parsed as `a * (b % c)` —

```
6*7%4 ==> 6*(7%4) = 18      (LRM: (6*7)%4 = 2)
42/5%3 ==> 42/(5%3) = 21     (LRM: (42/5)%3 = 2)
```

Silent wrong answers in any model mixing multiplication/division with modulus. Fixed: `%` now shares level 12 with `*` and `/`.

2. `~^/^~` bound tighter than `^` (harmless, fixed for exactness) The LRM puts `^`, `~^`, `^~` on one level. The split was provably unobservable: xor is associative/commutative and each xnor contributes exactly one global inversion, so every grouping of an xor/xnor chain yields the same value. Fixed anyway so the table is LRM-exact.

Everything else in the table was already correct.

Verification — `precedence_examples/`

`verify_precedence.py` (ALL PASS):

1. all 28 precedence/associativity checks read exactly 0 — including the `*` / `%` level (4 groupings), `**` vs `*` and unary, `**` left-associativity, shifts below `+` `-`, relational below shifts, `==` below relational, the `& ^ |` ladder, `&&/||` levels, ternary lowest + right-associative, and unary-tightest cases (`!0+1 == 2`, `~0+1 == 0`);
2. the marquee fix case asserted directly: `6*7%4 == 2` (was 18).

Regressions: no example in either tree contains a `*//`-then-`%` grouping (grep-audited, so the behaviour change affects no shipped model), and all 34 version11 example verify suites ALL PASS — including the Enhancement-37 operator audit, which now doubles as a semantics regression lock for this change.

Enhancement-39 — derived natures & deriving natures from disciplines

This document describes the changes made to OpenVAF-r in the `version11/` directory to make derived natures — `nature X : Parent`; and `nature X : discipline.potential / discipline.flow`; (LRM 3.4.1.3) — actually work. Three small fixes across parser, syntax validation and the OSDI nature-descriptor builder; the substantial inheritance machinery already existed.

The bugs

A derived nature inherits every attribute (`units`, `access`, `abstol`, `ddt_nature/idt_nature`) it does not override. OpenVAF's `hir_ty::NatureTy` implements all of this — parent resolution, base-nature chains, `units/ddt/idt` inheritance, attribute lookup through parents, and access-function compatibility by units. None of it was reachable:

1. The parent link was silently always **None**. The parser emitted a `NAME_REF` node for the `: parent` clause, but the AST accessor (`NatureDecl::parent()`) looks for a `Path` child — so `lower_nature_path` never saw a parent. Consequences: a derived nature without its own `units/access` never inherited them, so the parent's access function was rejected on its disciplines ("illegal access of branch") — making the canonical nature `TightCurrent : Current; abstol = 1e-15; pattern unusable`.
2. `nature X : electrical.flow`; did not parse at all ("unexpected token '.'") — same root cause, since a bare name-ref cannot carry a qualifier — and behind the parse there was a second gate: the syntax validation whitelisted only `ddt_nature/idt_nature` as qualified nature-path segments, rejecting `potential/flow` ("illegal nature identifier").
3. Discipline-qualified `ddt_nature/idt_nature` attribute values hard-panicked the OSDI nature-descriptor builder ("Nature's ddt must be a nature reference." in `osdi/src/ndatable.rs`): the `OsdiNature` descriptor encodes `ddt/idt` as bare nature indices, and the builder refused to resolve a discipline-qualified reference to its underlying nature.

The fixes

- **`parser/src/grammar/items.rs`** — the parent of a nature is parsed as a path (`Current` or `electrical.flow`), producing the `Path` node the AST and the (already-complete) item-tree lowering expect. This single change lights up the whole dormant inheritance machinery.
- **`syntax/src/validation.rs`** — `check_nature_path` also accepts `potential/flow` as the qualified segment.
- **`osdi/src/ndatable.rs`** — a new `resolve_nature_index` resolves discipline-qualified `ddt_nature/idt_nature` references through the discipline to the underlying nature's index instead of panicking.

Verification — `derivednature_examples/`

`derivednature_demo.va` packs the full feature matrix (5 modules): `derive-from-nature` with inherited `units+access`, `derive-from-discipline` (`electrical.flow/electrical.potential`), `derived nature` with its own access name, a two-level derivation chain, and a discipline-qualified `ddt_nature` attribute. `verify_derivednature.py` (ALL PASS):

1. the matrix compiles (three of the five constructs used to fail or crash);
2. runtime conductances are exact for every module — proving the inherited access functions genuinely resolve end-to-end through ngspice (`dn_nature 1 mS` via inherited `I`, `dn_discipline 2 mS`, `dn_access 5 mS` via its own `I2`, `dn_chain 3 mS` through two levels);
3. the `ddt_nature = electrical.potential` module's OSDI descriptor builds and loads (used to panic the compiler).

Regressions: all 35 version11 example verify suites ALL PASS and all 77 example models recompile (the parser change makes every file reparse).

Enhancement-40 — N-dimensional `$table_model`

This document describes the changes made to OpenVAF-r in the `version11/` directory to lift `$table_model`'s 3-dimension cap: lookup tables of any dimension now work. Purely front-end (`hir_ty + hir_lower`); no OSDI/ngspice change.

The gap

The question "are multi-dimensional tables supported?" verified out as: 1-D (linear + cubic, inline + file), 2-D (bilinear + bicubic) and 3-D (trilinear) all exact — but a 4-D call failed at the *signature* level:

```
error: invalid argument count: expected at most 2 arguments but found 6
```

The interesting part: the grid-file reader (`read_table_grid_nd`) and the recursive multilinear interpolation (`interp_nd`) from Enhancement-17 were already fully dimension-general — only the declared signature list (explicit 1-D/2-D/3-D variants) and the signature-matched dispatch capped it. The LRM sets no dimension bound.

The fix

1. **`hir_ty`** — `TABLE_MODEL` becomes a varargs builtin (the listed 1-3-D signatures kept verbatim), and a dedicated `BuiltIn::table_model` arm in `infern_builtin` owns *every* call:
 - the 1-D inline-array form and small file forms resolve against the listed signatures unchanged;
 - N-D file forms (≥ 4 non-array arguments) get the exact signature `[Real × ndim, Literal(String)(, Literal(String))]` synthesised from the argument shapes — the trailing string literals are the data-file name and the optional control string, everything before them a coordinate.

Owning all counts matters twice over: the generic varargs fallthrough *resizes* the listed signatures to the call's arity, which truncates longer signatures and made 2-argument calls ambiguous; and a 5-argument call is ambiguous *by arity alone* (3-D + file + ctrl vs 4-D + file) — the shape scan disambiguates it.

2. **`hir_lower`** — `lower_table_model` dispatches on argument shapes instead of the resolved signature (array second argument $\rightarrow$ 1-D inline; otherwise the first string literal marks the file, its index the dimension). `read_table_grid_nd + interp_nd` were already general; all partial derivatives continue to flow through `mir_autodiff` into the Jacobian.

The self-describing grid-file format is unchanged: `ndim`, axis sizes, each axis's ascending coordinates, then row-major values (first axis slowest).

Verification — `ndtable_examples/`

The demo grids hold multilinear functions, which multilinear interpolation reproduces *exactly* at any off-grid point — so every check asserts analytic equality. `verify_ndtable.py` (ALL PASS):

1. a 4-D table compiles (used to error at the cap) and evaluates exactly at two off-grid points (`f4(1.5, 0.25, 0.75, 0.4) = 7.9625`, second point parameter-swept);
2. 4-D without a control string — the ambiguous 5-argument arity — resolves and evaluates exactly;
3. a 5-D table with control string evaluates exactly (6.5625);
4. regression lock: the 1-D inline form still interpolates exactly.

Also exercised during development: 3-D exact (9.36), mixed per-dimension control strings ("3,1L"). Regressions: all 36 version11 example verify suites ALL PASS — including `table_model/mdtable/cubic_table`, which lock the 1-3-D behaviour this change re-routes through the new dispatch.

Enhancement-41 — implicit nets in instance connections

This document describes the change made to OpenVAF-r in the `version11/` directory to support implicit nets: a plain identifier used in a module-instance port connection that names nothing declared in the enclosing module is implicitly declared as a scalar net. One-file change in the Enhancement-5 elaboration pass (`hir/src/elaborate.rs`); no OSDI/ngspice change.

Semantics (and the Verilog-A subtlety)

Per the LRM's structural-connection semantics, undeclared interconnect names in instance connections are implicit nets — the idiom every netlist-style module relies on:

```
module top(in, out);
  inout in, out; electrical in, out;
  res2 r1(in, mid);           // `mid` never declared -> implicit net
  res2 r2(mid, out);
endmodule
```

In full Verilog-AMS the implicit net's discipline comes from the ``default_discipline` directive — but the Verilog-A appendix excludes that directive ("not supported in Verilog-A"; every net's discipline must be defined). The two statements reconcile through discipline resolution: the implicit net takes its discipline from the connected port — exactly what resolution produces for the common case of joining compatible ports. That is what Enhancement-41 implements:

- the implicit net's discipline = the discipline of the target module's port it is connected to (fallback electrical if the port has none);
- two connections implying conflicting disciplines for the same implicit net are a hard error (implicit net 'mid' is connected to ports of conflicting disciplines 'electrical' and 'thermal' — declare it explicitly);
- ``default_discipline` remains ignored (the preprocessor has always parsed and discarded it), per the appendix;
- implicit declaration is structural-only: an undeclared identifier inside `V()`/`I()` access functions remains a clean scope error.

Before this change, every internal wiring node had to be declared manually — error: 'mid' was not found in the current scope.

Implementation

Implicit nets are synthesised in the module-instantiation elaboration pass (Enhancement-5's compile-time flattening, `hir/src/elaborate.rs`):

- while binding an instantiation's port connections, a `PortBinding::Scalar` whose text is a plain identifier (`as_plain_ident`) naming nothing in the parent module's declarations (declared names: net/port base names, bus/array base names, parameters, variables, branches, functions, instance names) is recognised as implicit;
- the net becomes a local of the parent module, so it takes the parent's instance prefix like every other local — two flattened instances of the same submodule therefore keep their internal implicit nets distinct (no accidental cross-instance shorts);
- its declaration (`electrical top__mid; // implicit net`) is emitted exactly once, prepended to the parent's rendered body so it precedes every use;
- discipline conflicts collect into a hard anyhow error at the end of elaboration.

Both positional (`r1(in, mid)`) and named (`.n(mid)`) connection forms work; the same implicit net may appear in any number of connections.

Verification — **implicitnet_examples/**

`implicitnet_demo.va` chains two `ser2k` submodules — each with its own implicit internal net `w` — through an implicit top-level `mid`, mixing positional and named forms. `verify_implicitnet.py` (ALL PASS):

1. it compiles (used to error);
2. the DC resistance is exactly $4\text{ k}\Omega$ — proving `mid` joined the instances *and* the two nested `w` nets stayed distinct after flattening (a cross-instance short would read $2\text{ k}\Omega$);
3. conflicting-discipline connections are rejected with a clear message;
4. an undeclared identifier inside `V()` access remains an error.

Regressions: all 37 version11 example verify suites ALL PASS, all 79 example models recompile, and the instantiation/generate decks (the heavy users of the elaboration pass) run unchanged.

Enhancement-42 — correlated (same-named) noise sources

This document describes the changes made to OpenVAF-r and ngspice-46 in the `version11/` directory to implement noise-source correlation by name: noise functions that carry the same name argument model the *same* physical source and must sum coherently (as amplitudes) at the output, per Verilog-AMS LRM 4.6.4.

The gap

The LRM defines the name argument of `white_noise` / `flicker_noise` / `noise_table` as source *identity*: two noise functions with identical names are the same source — perfectly correlated — so their contributions to the output add as complex amplitudes,

$$\text{onoise}^2 = \left| \sum_k f_k \cdot \sqrt{\text{pwr}_k} \cdot T_k \right|^2 \quad (\text{same-named group})$$

not as powers,

$$\text{onoise}^2 = \sum_k f_k^2 \cdot \text{pwr}_k \cdot |T_k|^2 \quad (\text{independent sources})$$

where f_k is the (signed) contribution factor and T_k the complex transfer from the source's branch to the noise output.

Before this enhancement the name was used only to *label* the per-source output vectors. Every source was independent — a two-fold same-named source pair read $\sqrt{2} \times$ instead of $2 \times$, and even a negated contribution of the same source (`<+ -white_noise(S, "n")`, anti-phase, should cancel) *added* power.

The three `// TODO` noise markers in the OSDI crate (`metadata.rs`, `load.rs`) turned out to be stale — the code below them is complete — and the `lineralize.rs` “complex noise power” note is an optimization idea, not a gap. Correlation was the real leftover.

The fix (two-sided)

OpenVAF — **`openvaf/osdi/src/load.rs`** (`load_noise` codegen): the contribution factor used to be folded into the loaded power as `pwr * fac2`, destroying its sign. It is now folded as `pwr * fac * |fac|` — identical magnitude, but the loaded power carries the factor's sign. ngspice's noise analysis takes `fabs()` of each source's power before using it, so nothing changes for independent sources; the sign is what enables anti-phase cancellation inside a correlated group. (Supporting fix: `openvaf/mir_llvm/src/intrinsics.rs` never registered `llvm.fabs.f64` — one `ifn!` line.)

ngspice — **`src/osdi/osdinoise.c`** (`N_DENS`): the per-source loop first records each source's complex transfer $T_k = (\text{CKTrhs}[n1] - \text{CKTrhs}[n2]) + j(\text{CKTirhs}[n1] - \text{CKTirhs}[n2])$ (the adjoint solution), then walks the sources grouping same-named ones within the instance (`strcmp` on the OSDI descriptor's `noise_sources[].name`) and sums signed amplitudes $\text{sign}(\text{pwr}_k) \cdot \sqrt{|\text{pwr}_k|} \cdot T_k$ coherently before squaring:

- a uniquely-named source reduces *exactly* to the classic $|\text{pwr}| \cdot |T|^2$;
- the group's power is assigned to the group's first source (its `onoise_<inst>_<name>` vector shows the group total; members show 0), so the per-source reporting, log-slope integration and total-noise machinery are untouched;
- grouping is per-instance by construction (the loop runs inside one instance's descriptor), so identical names in *different* instances — or in the RFSPIICE S-parameter noise path, which keeps the independent per-source form — remain uncorrelated, as they must.

What now works (**`noisecorr_examples/`**, all exact)

With PSD $1\text{e-}12$ sources on a unity-transfer series chain ($\sqrt{\text{PSD}}$ -PSD amplitude $1\text{e-}6$ V/ $\sqrt{\text{Hz}}$ each):

case	onoise before	onoise after
same name twice	1.414e-6	2e-6 (amplitudes add)
distinct names	1.414e-6	1.414e-6 (unchanged)
same name, one negated	1.414e-6	0 (anti-phase cancels)
$2 * \text{wn}(S, "n") + \text{wn}(S, "n")$	2.236e-6	3e-6 (linear weights)
same name, two instances	2.828e-6	2.828e-6 (unchanged)
white + flicker, one name	1.414e-6 @1Hz	2e-6 (kind-agnostic)

verify_noisecorr.py: 7/7 PASS. Full version11 regression: 38/38 suites PASS (including the Enhancement-9 noise_table suite, byte-identical results — all its sources are uniquely named).

Notes

- Unnamed noise functions get compiler-synthesised unique names, so they can never group accidentally.
- Correlation is *total* (same source). Partial correlation is out of scope — the LRM models it by composing shared and private named sources, which this enhancement makes work.

Enhancement-43 — variable initializers, completed

This document describes the changes made to OpenVAF-r in the `version11/` directory to complete declaration initializers (`real x = 2.5;`). All changes are in `hir_def`; no OSDI/ngspice change.

What already worked (verified, kept as regression checks)

Scalar module-level initializers (`real/integer/string`), initializers that are constant expressions over parameters (`real y = 2.0*p + 1.0;`, re-evaluated against model-card overrides), named-block locals, multiple declarators, implicit `real`→`integer` rounding, and — the semantic core — LRM init-once behavior: each live unwritten variable read becomes a `ParamKind::HiddenState` input that `hir_lower/state.rs` seeds from the variable's initializer body gated on `IsInitialStep`, so event-updated state *starts* at the initializer and is never re-initialized per evaluation (`integer cnt = 10; +@(cross) cnt = cnt + 1;` counts 10, 11, ...). Initializers referencing other *variables* are correctly rejected ("constant expressions must not contain variable references").

The three defects

1. Array declaration initializers were rejected. `real x[0:2] = '{1.0, 2.0, 4.0}';` — an array variable expands into per-element scalar variables (E-14), and *each* element's body picked up the whole `'{...}'` aggregate, so the type check failed "expected real value but found `real[0:3]` value" once per element (the mysterious triplicated diagnostic).
2. Untyped analog-function arguments crashed the compiler. In analog function `real f; input v; ... endfunction` with no `real v;` declaration, `FunctionArg::ty` returned `Type::Err(declarations.first().map_or(Type::Err, ...))`, which flowed into inference and hit `unreachable!("unknown cast found Real -> Err")` (`hir_lower/ctx.rs`) at the first cast involving the argument. (This masqueraded as a "function-local initializer crash" during probing — the initializer was innocent.)
3. Wrong-arity initializers crashed the compiler — variables *and* parameters. `real x[0:2] = '{1.0, 2.0}';` (or the array-parameter equivalent, a *pre-existing* E-14 latent bug) produced `Expr::Missing` for the uncovered elements with no diagnostic, and lowering died with "invalid HIR: Missing".

The fix (`hir_def` only)

- `item_tree.rs` / `item_tree/lower.rs`: `Var` gains `array_index: Option<u32>` — each expanded element records its flat declaration-order (row-major) position, exactly mirroring `Param::array_index` (E-14/15).
- `body.rs`: an element variable's body takes the matching leaf of the shared `'{...}'` literal (`flatten + nth(pos)`, the same split the param path uses). N-D nesting comes free — the `flatten` is row-major, matching the element expansion order; integer leaves in a real array get the normal implicit cast; leaves may be constant expressions over parameters and are re-evaluated on overrides. Everything downstream (hidden-state seeding, init-once gating, function-local expansion) works unchanged because each element is just a scalar variable with its own initializer body.
- `item_tree.rs` (`FunctionArg::ty`): an argument with no type declaration defaults to **real**, matching the untyped-return default (analog function `f;` is a real function per the LRM).
- `item_tree/lower.rs`: both array-expansion sites (variables and parameters) count the literal's leaves against the declared element count and emit a new named diagnostic — array initializer for 'x' has N element(s) but the array has M — covering too-few, too-many, and scalar-on-array forms.

What now works (`varinit_examples/`, all exact)

case	result
<code>real v1[0:2] = '{1.0, 2.0, 4.0}; + integer iv[0:1] = '{8, 16};</code>	elements read exactly
<code>real m[0:1][0:2] = '{' {...}', '{...}'; (2-D), 3-D with integer leaves</code>	row-major leaves, casts applied
<code>real pd[0:1] = '{s, 3*s};</code>	leaves track s model-card override
<code>real acc[0:1] = '{100.0, 0.0}; + @(cross) update</code> <code>function-local real w[0:2] = '{...} + scalar init + untyped</code> input v <code>'{1.0, 2.0} on [0:2] (var or param), '{...x4}, scalar-on-array</code>	starts at 100, init-once evaluates exactly (was ICE) clean named diagnostic (was ICE)

verify_varinit.py: 12/12 PASS. Regression: all 39 example verify suites ALL PASS; hir_def/hir_ty/hir_lower/hir/syntax/parser crate tests pass.

Notes

- The stale `lower_var` comment claiming array initializers are "silently ignored" (Enhancement-4 era) was removed — they are now real.
- parameter `real [0:2] c = '{1.0, 2.0};` crashing was pre-existing (E-14); the new arity diagnostic covers it through the shared code path.

Enhancement-44 — paramset hierarchical system parameters

This document describes the changes made to OpenVAF-r in the `version11/` directory to support hierarchical system parameters in paramsets (`.$mfactor = 8;`, LRM 6.4 — the canonical “quad device” idiom). No OSDI/ngspice change.

Baseline (probed first, all correct)

The six hierarchical system parameters (`$mfactor`, `$xposition`, `$yposition`, `$angle`, `$hflip`, `$vflip`) were already fully working at the instance level:

- readable in expressions with exact LRM defaults (`1 / 0 / 0 / 0 / 1 / 1`);
- settable per-instance in ngspice — `m=` for `$mfactor` (the OSDI layer’s standard alias) and `_xposition=...` for the rest (ngspice rewrites the `$` prefix to `_`, since `$` starts a netlist comment);
- `$mfactor` semantics exact: flow contributions $\times m$, potential contributions invariant, noise PSD $\times m$, correct across E-5-flattened sub-instances.

The gap: a paramset could not set them — `.$mfactor = 8;` was a parse error (“unexpected token system function identifier”), yet per LRM 6.4 paramsets may specify hierarchical system parameters alongside parameter bindings.

Semantics

The paramset value composes with the instance-level value, per the LRM’s hierarchy rules:

- `$mfactor`, `$hflip`, `$vflip` — multiplicative (multiplicities and flips multiply down the hierarchy): `m=3` on a `.$mfactor = 8` paramset gives an effective 24;
- `$xposition`, `$yposition`, `$angle` — additive (offsets accumulate).

The composed value must be seen by *every* consumer: explicit `$mfactor` reads in the body, the DAE builder’s automatic flow scaling, its `sqrt(mfactor)` noise-factor scaling, and all derivative code. Override expressions are `paramset_constant_expressions` and may reference the paramset’s own card parameters (`.$mfactor = nf;`), including their model-card overrides.

Implementation

- **`parser/grammar/items.rs`** — `paramset_override` accepts a `SYSFUN` token as the overridden name, wrapped in the same `NAME_REF` node (plus a `NameRef::sysfun_token()` accessor in syntax).
- **`hir_def/item_tree/lower.rs`** (`lower_paramset`) — each HSP override becomes a hidden real localparam named `$paramset$<name>` in the E-21 twin module, with the override expression as its `override_expr` (the ordinary E-21 localparam-with-override machinery evaluates it in twin scope, so card-parameter references and model-card overrides just work). The `$`-spelling collides with no user identifier and deliberately differs from the OSDI built-in `$mfactor` instance parameter — so ngspice’s `m=` alias keeps pointing at the *instance* value. Unknown system functions (`.$vt = 1;`) get a named diagnostic (`ItemTreeDiagnostic::InvalidParamsetSysParam`). The synthetic param’s `ast_id` is a placeholder (E-23 `from_erased` trick); the override branch of `param_body_with_sourcemap` was reordered to not dereference the `ast::Param` node it doesn’t have.
- **`hir_lower/state.rs`** — new `HirInterner::insert_paramset_sys_fun_overrides`, modeled on the E-7 `insert_var_init` use-rewrite: for each override, read the hidden localparam (`ParamKind::Param`), build `fmul/fadd` with the `ParamKind::ParamSysFun` value, and rewrite every pre-existing use of the system parameter with the composed value.
- **`sim_back/lib.rs`** — the pass runs after **`DaeSystem::new`** (so the automatic `mfactor` flow/noise scaling and the derivative code exist and get rewritten) and before the post-derivative optimization; the twin’s hidden localparams are recognized by their `$paramset$` name prefix. One supporting

fix: the pass creates new MIR values, so `output_values` is re-grown — `BitSet::contains` in the `init-cache` dead-code pass indexes by value and panicked otherwise.

What now works (**paramsethsp_examples/**, all exact)

case	result
<code>paramset quad rbase; .r = 2e3; .</code>	250 Ω effective (−4 mA at 1 V)
<code>\$mfactor = 8;</code>	
<code>quad + netlist m=3</code>	effective 24 → −12 mA
all six HSPs in one paramset, read back	8254.89 composed exactly
<code>+ instance m=3 _xposition=5 _hflip=-1</code>	24755.09 ($\times$ m, + positions, $\times$ flips)
<code>.\$mfactor = nf (card param)</code>	−5 mA at default <code>nf=5</code> , −2 mA with <code>nf=2</code>
noise through paramset multiplicity	onoise 5e-4 = netlist <code>m=4</code> exactly
<code>.\$vt = 1;</code>	”\$vt’ is not a hierarchical system parameter”

`verify_paramsethsp.py`: 8/8 PASS. Regression: all 40 example verify suites ALL PASS; `sim_back/hir_def/hir_lower/parser/syntax` crate tests pass.

Notes

- The `sim_back` MIR snapshot files (`test_data/dae/topology/init`) were stale from earlier enhancements (the suite hadn’t been run since ~E-36’s DAE builder changes) and failed identically on the committed pre-E-44 sources; they were refreshed with `UPDATE_EXPECT=1` against the current (verified bit-identical at E-42) behavior.
- Paramsets *selecting* on hierarchical system parameters (LRM’s restriction-checking use) remain out of scope; only value specification is implemented.

Enhancement-45 — net initialization + net attribute access

This document describes the changes made to OpenVAF-r and ngspice-46 in the `version11/` directory to implement two LRM features that were both completely missing: net nodeset initializers (LRM 3.6.3.2) and net/branch attribute access (LRM 5.5.3).

Half A — accessing net and branch attributes (LRM 5.5.3)

```
I(a,b) <+ c*ddt(V(a,b), a.potential.abstol); // the LRM's twocap example
x = n1.flow.abstol;
y = br.potential.abstol; // branch form (LRM prose)
```

Every spelling failed with “expected a scope but found node/branch/nature” — the E-39 “scaffolded-but-unwired” pattern at yet another boundary: `nameres` had `ResolvedPath:: {Flow, Potential}Attribute` arms for branches, but only in the cross-scope `resolve_names_in` traversal; the module-body `resolve_path` entry rejected any non-scope qualifier first, so even the branch form was unreachable from where models actually use it.

Fix (front-end only):

- `hir_def/nameres.rs`: the module-body entry gains `NodeId/BranchId` arms for 3-segment `.potential/.flow` attribute paths, with two new `ResolvedPath` variants (`NetPotentialAttribute/NetFlowAttribute`).
- `hir_ty/lower.rs`: `net_nature_attr` — `net` → `discipline` → `potential/flow nature` → `attribute`, through `NatureTy::lookup_attr` (E-39’s inheritance-aware lookup, so attributes of derived natures resolve).
- `hir_ty/inference.rs`: the new variants type as `Ty::NatureAttr`; lowering needed nothing — `Ref::NatureAttr` reads already lower to the attribute’s constant value body.

Unknown attributes get the existing clean diagnostic (“‘nonsense’ was not found in ‘Voltage’”).

Half B — net discipline initial (nodeset) values (LRM 3.6.3.2)

```
electrical a = 5.0; // nodeset for a's potential
electrical [0:2] b = '{0.5, -1.0, 2.0}'; // per-bit leaves
```

Previously a parse error. The initializer is a constant used as a nodeset value — an initial Newton-Raphson guess, not an enforced constraint.

Pipeline (both stacks):

- parser: each net declarator accepts an optional `= expr`; the expression is a direct `NET_DECL` child after its `NAME` (new `NetDecl::declarators()` AST accessor pairs them).
- `hir_def`: `Net/NodeData` carry the const-folded value (`OrderedFloat`); bus declarators split the `{...}` literal into per-bit leaves in ascending bit order (matching the bus expansion). Non-constant initializers get a new named diagnostic (`NonConstantNodeset`) — parameter-dependent nodesets are out of scope.
- OSDI ABI: `OsdNode` gains a `double nodeset` field (`NAN = none`), filled for Kirchhoff-law unknowns from the net’s value. Because this changes the node-array stride — which the `OSDI_DESCRIPTOR_SIZE` mechanism does not cover — the OSDI version is bumped to 0.5, and ngspice’s loader now requires ≥ 0.5 , rejecting stale `.osdi` files with “Recompile the model with the matching `openvaf-r`” instead of misreading them.
- ngspice (`osdisetup.c`, landing exactly on a pre-existing `// TODO nodeset?` marker): at instance setup, internal nodes created with a non-NAN nodeset get `node->nodeset/nsGiven` set; connected terminal nodes get the same treatment unless the netlist already gave one (`.nodeset` wins, matching the LRM’s hierarchical-declaration-wins spirit).

- elaboration (hir/elaborate.rs): the E-5 flattening re-renders net declarations when filtering out port names — it now preserves each surviving net's initializer text, so submodule nets keep their nodesets.

What now works (**netinit_examples/**, all verified)

Bistable $x = \tanh(5x)$ (solutions $0, \pm 0.999909$), the nodeset selecting the branch:

case	result
no initializer	0 (trivial solution)
electrical m = 1.0; /= -1.0; (internal)	+0.999909 / -0.999909
port initializer electrical q = 1.0;	+0.999909 on the terminal
netlist .nodeset v(q)=-1 vs port initializer	netlist wins (-0.999909)
bus '{0.5, -1.0, 2.0}	per-bit: weighted sum 90.99
initializer in flattened submodule	preserved (-0.999909)
1e6·q.potential.abstol + 1e12·q.flow.abstol + 1e6·br.potential.abstol	3.0 exactly
electrical m = 2*p; / a.potential.nonsense	clean named diagnostics

verify_netinit.py: 11/11 PASS. Regression: all 41 example verify suites ALL PASS; crate tests (hir_def/hir/hir_ty/hir_lower/sim_back/osdi/ parser/syntax) 57/57.

Notes

- Breaking ABI note: .osdi files compiled before E-45 must be recompiled (clean version error, not silent corruption). All committed example .osdi binaries and prebuilts are regenerated.
- Null bus-literal slots ('{2.3, 4.5, , 6.0}', LRM) remain a parse error — use an explicit value per element; extra/missing leaves simply leave the remaining bits without a nodeset.
- Nature-qualified spellings (Voltage.abstol) stay rejected: the LRM BNF (Syntax 5-4) allows net qualifiers only; the abstol-for-ddt() use passes the nature identifier directly, which already worked.

Enhancement-46 — escaped identifiers + integer literal bases

This document describes the changes made to OpenVAF-r in the `version11/` directory to implement integer literal bases (LRM A.8.7) and complete escaped identifiers (LRM A.9.3). Front-end only; no OSDI/ngspice change.

Integer literal bases (were entirely missing, plus a crash)

'h1F, 'o17, 'b1010, 'd42, sized 8'hFF, signed 8'shFF — every based form was "encountered unexpected token": the lexer contained only a commented-out sketch of based-number tokenization. And the LRM-legal underscore separator (`1_000_00`) crashed the compiler: `eat_decimal_digits` accepted `_` into the token, but `IntNumber::value_as_f64` parsed the raw text ("IntNumber token must be valid float syntax too: `ParseFloatError`").

Fix (`lexer, syntax/ast/expr_ext.rs`):

- The lexer tokenizes `[size]'[s]<base><digits>` — the unsized form from the `'` dispatch (alongside the `'{` aggregate arm), the sized form as a continuation of `number()`. Digits are validated per base while lexing (`'b12` ends the token at the 2 → ordinary parse error), a digit is required after the base (a bare `'h` never becomes a silently-zero literal), and `x/z` digits — not meaningful in the analog subset — are not consumed.
- Value parsing (`parse_based_int`): strip `_`, radix-parse, mask to the declared size (clamped 1..=32), sign-extend from the size's MSB under `s`, wrap to the 32-bit integer type (`'hFFFFFFFF = -1`). `_` separators are stripped in all number forms (plain ints, `1_234.5` reals, SI-scaled).

Escaped identifiers (half-wired)

The lexer already emitted `EscapedIdent` tokens (backslash through the last non-whitespace character) and mapped them to `IDENT` — but two defects made the feature unusable:

1. **Name::resolve** stripped the last character along with the backslash (`raw[1..len-1]`, written for a token shape that included the terminating whitespace). `\foo` resolved to `"fo"`, so the escaped and plain spellings of a name never matched — and the compiler's own `std.va` def-map snapshot had baked in `logi` for the escaped `\logic` discipline, hiding the bug. Fixed to strip only the backslash; the stale snapshots were refreshed.
2. The E-5 elaboration re-rendered substituted names unescaped. An escaped net inside a flattened submodule (`\n-1` → instance-prefixed `u1_n-1`) was emitted raw into the synthesized text, which no longer lexed ("unexpected token '-"). A `render_name` helper now re-escapes any substitution value that isn't a plain identifier (`\u1_n-1`), applied in `render_with_holes` (which also gained the `EscapedIdent` substitution arm — escaped *uses* keyed by their resolved name) and the net-declarator re-render path.

Keyword spellings (`\module`) work as names — `EscapedIdent` never goes through keyword mapping.

What now works (**`escid_examples/`**, all exact)

case	result
'h1F + 'o17 + 'b1010 + 'd42	31+15+10+42
8'hFF / 8'shFF / 4'sb1000	255 / -1 / -8 (size mask + sign extension)
'hFFFFFFFF	-1 (32-bit wrap)
1_000_00, 16'hAB_CD, 1_234.5	separators stripped everywhere
whole-module sum	0.1443252345 V exactly
\2wire, \value#, \r+val	nets/vars/params with specials
\mid ≡ mid	one net (LRM equivalence)

case	result
escaped net in flattened submodule \module	re-escaped, 2k series exact keyword spelling as a name
'h, 'sh, 'b12, 8'squark	clean parse errors, no crash, no silent 0

verify_escid.py: 12/12 PASS. Regression: all 42 example verify suites ALL PASS; 65/65 crate tests (lexer/tokens/syntax/parser/hir_def/hir/ hir_ty/hir_lower/sim_back) after refreshing the snapshots that had encoded the \logic→logi bug.

Notes

- Sized literals wider than 32 bits clamp to the 32-bit integer type.
- The 'd form takes decimal digits only; x/z digits are rejected by construction (analog subset).

Enhancement-47 — ‘default_transition + transition() fixes

This document describes the changes made to OpenVAF-r in the version11/ directory to implement the ``default_transition` compiler directive and fix two pre-existing `transition()` defects the work exposed. Front-end + lowering only; no OSDI/ngspice change.

The directive (was a hard error)

``default_transition <time>` sets the default rise/fall time used by `transition()` filters that omit those arguments (LRM; 0 = instantaneous when no directive is given). Previously it hard-errored as an undeclared macro — the only directive from the E-47 probe's list of nine that didn't work (define/undef/ifdef/ifndef/else/elsif/endif/include were all verified exact).

Implementation:

- preprocessor: a `DefaultTransition` directive kind (next to `DefaultDiscipline`), a capture-number helper, and an SI-suffix-aware value parser (1u, 10n, 1e-6, _ separators). The value rides on the Preprocess result (last directive wins — file-level granularity; real models declare at most one, and a directive inside a false ``ifdef` is never processed, so conditional guards behave correctly).
- `hir/db.rs:CompilationDB::default_transition()` accessor.
- `hir_lower`: the no-args and delay-only `transition()` forms ramp with the directive's time (rate = 1/t) instead of returning the input directly; explicit rise/fall arguments are untouched.

Pre-existing defect #1: the TRANSITION signature table (compiler crash)

Every entry was one argument short: `TRANSITION_DELAY_RISE` claimed 2 arguments (it takes 3), `_FALLT` 3 (takes 4), `_TOL` 4 (takes 5). So a 3-argument `transition(s, td, trise)` resolved to `_FALLT` and the lowering read `args[3]` out of bounds — compiler crash (confirmed on the released E-46 binary); 4-argument calls only worked *by accident* (they resolved to `_TOL`, whose lowering happened to read the right indices); the true 5-argument `tol` form did not resolve at all. The table now declares the correct arities and all five forms work.

Pre-existing defect #2: singular DC operating point

`lower_rate_limited_track` (shared by `slew` and `transition`) implements $dy/dt = \text{clamp}(K \cdot (x - y), -\text{fall}, \text{rise})$. When the clamp saturates, the residual's derivative w.r.t. y is zero — and since the reactive part doesn't count in DC, the Jacobian diagonal vanished and the operating point went singular whenever the input started more than rate/K away from the state (e.g. a timer-driven comparator already high at $t=0$): `gmin/source` stepping failed and the transient produced garbage unless `uic` was given. Per the LRM a transition/slew filter is a static identity in DC, so the residual now selects on the integration-enable parameter: $y - x$ in DC (diagonal 1, never singular, exact LRM semantics), the rate-limited form in transient. AC also becomes an exact unity transfer (previously a spurious pole at $K = 1\text{e}9$ rad/s).

What now works (`defaulttransition_examples/`, all verified)

case	result
bare <code>transition(s)</code> + <code>`default_transition 1u</code>	1 μs ramp, half-cross at 0.5 μs , clean DC (no <code>uic</code>)
<code>transition(s, 0.2u)</code> + directive	delay then default ramp (half-cross at flip + 0.7 μs)
explicit <code>transition(s, 0, 2u)</code>	2 μs ramp — explicit args win
all five arities in one contribution	weighted plateau 0.875 exact (3-arg used to crash)
no directive	instantaneous, unchanged

case	result
directive inside false `ifdef	ignored

verify_defaulttransition.py: 8/8 PASS. Regression: all 43 example verify suites ALL PASS; 71/71 crate tests.

Notes

- Positional granularity is file-level (the last directive processed wins), matching real usage; the LRM's mid-file re-declaration subtlety is out of scope.
- The absdelay history's startup value (input nonzero at t=0 reads 0 for the first td) is a pre-existing E-24 behavior surfaced while testing, not changed here.

Enhancement-48 — string literal escape sequences

This document describes the change made to OpenVAF-r in the `version11/` directory to complete string literal escape handling per LRM 2.7.1. One function in syntax; no OSDI/ngspice change.

The defects

The LRM escape set for string literals is `\n`, `\t`, `\\`, `\"`, and `\ddd` (a character given by one to three octal digits). The probe found:

escape	before
<code>\n, \t, \\, \"</code>	worked
<code>\ddd octal</code>	unsupported — printed literally
<code>\\n</code> (literal backslash, then n)	corrupted — printed backslash + real newline

Root cause: `StrLit::unescape_value` chained sequential `str::replace` calls, with `\n` replaced *before* `\\` — the classic overlapping-escape bug (the `\n` inside `\\n` was consumed first, orphaning the leading backslash) — and simply had no `\ddd` handling.

The fix

`unescape_value` is now a single left-to-right pass:

- `\n, \t, \\, \"` — the LRM basics;
- `\ddd` — one to three octal digits, greedy, out-of-range values degrade to the replacement character;
- backslash before a (possibly CRLF) newline keeps the newline — the pre-existing line-continuation extension, behavior unchanged;
- any other escape (and a trailing backslash) is preserved verbatim.

The audit confirmed every string consumer already routes through this one function — `$strobe/$display/$swrite` format strings, string literal values and comparisons, `(* desc= *)/units` attribute strings, `@(initial_step("phase"))` phase names, and lint-attribute names — so the fix applies uniformly with no second unescape path to drift.

What now works (**stresc_examples/**, all verified)

case	result
<code>"\101\102\103"</code> in <code>\$strobe</code>	prints ABC
<code>"\060\61"</code> (3- and 2-digit forms)	prints 01
<code>"a\\nb"</code>	prints <code>a\nb</code> — literal backslash + n (the old bug printed a newline)
<code>"\101\102" == "AB"</code>	true — octal and plain spellings are the same string
<code>"x\\ny" == "x\\ny", "a\\qb" == "a\\qb"</code>	overlap-safe, unknown escapes consistent
<code>\n/\t/\\/"</code> rendering	unchanged, exact

`verify_stresc.py`: 8/8 PASS. Regression: all 44 example verify suites ALL PASS; `syntax/hir_def/basedb/hir_lower/sim_back` crate tests 52/52.

Notes

- %% in format strings is printf-level, not a string escape — untouched.
- Multi-line strings remain tolerated with the backslash-newline continuation keeping the newline, exactly as before this change.

Enhancement-49 — \$root + hierarchical names, transition() input

This document describes the changes made to OpenVAF-r in the `version11/` directory for three related front-end defects: hierarchical references into flattened instances, nested named-block paths, and `transition()`'s input type (plus a builtin signature audit). No OSDI/ngspice change.

1. Hierarchical references into flattened instances (LRM 6.6)

The probe found `$root`-anchored paths and single-level block paths working, but every reference into an instance failed — after the E-5 elaboration flattens `rdiv u1(a, c);` into prefixed locals (`u1__m`, `u1__r`), parent-side references were never rewritten:

```
analog V(a,c) <+ 0.0*V(u1.m);    // error: 'u1' was not found in the current scope
x = u1.r;                        // same
```

Fix (`hir/elaborate.rs`): every rendering scope now carries an instance-chain map — all chains reachable from that scope ("`u1`", "`u1.u2`", "`u1[2]`", ...) mapped to their composed flattening prefixes, built recursively from the item tree. A token-level scanner (`find_instance_path_holes`, the same hole mechanism as the bus-port substitution) rewrites `chain.member` occurrences to `render_name(prefix + member)`:

- deep chains compose (`u1.u2.x` → `u1__u2__x`); instance-array elements work (`u1[2].m` → `u1_2__m`, disambiguated from bus selects by chain lookup);
- the top module's scope adds `<top>...` alias entries and the scanner strips a leading `$root.`, so `$root.hier.u1.u2.m`, `hier.u1.u2.m` and `u1.u2.m` all resolve identically (per the LRM, `$root.<top>.x` ≡ `x` from `<top>`);
- bus selects after the member stay in place (`u1.b[2]` → `u1__b[2]`); escaped member names re-escape through E-46's `render_name`;
- a member followed by a further `.` (a named block *inside* the child) is left untouched — out of scope, cleanly diagnosed by name resolution.

2. Nested named-block paths

`outer.inner.w` failed with "'w' was not found in 'inner'" while single-level `blk.v` worked. Root cause in `hir_def/namer.rs::resolve_names_in`: the traversal correctly redirects `current_map` into each nested block's def map, but the final name lookup used `self.scopes` — probing the original map's scope ids against the wrong arena. One-token fix (`current_map.scopes[scope]`), plus the write-up of why only multi-segment paths hit it (single-segment callers had already switched maps).

3. transition() input type (user-reported) + builtin audit

The LRM's canonical comparator failed to compile:

```
error: type mismatch: expected integer value ... but found real variable reference
  | V(cout) <+ transition(vcout, td, tr, tf);
```

The TRANSITION signature table typed the input `Val(Integer)` (and the lowering `ifcast`-ed it), but per LRM 4.5.7 the filter's input is a real expression — smoothing piecewise-constant real waveforms is its whole purpose. All five signatures now take `Val(Real)` (integer inputs promote implicitly; the manual cast is gone). The follow-up audit of every builtin signature table (`noise`, `ddt/idt/idtmod`, `laplace/zi`, `absdelay/slew`, `last_crossing`, `file I/O`, `simparam/simprobe`, `random/arandom`, `rdist/dist`, `plusargs`, `discontinuity`, `bound_step`, `ddx`, `table_model`, `display`, and the event kinds) found exactly one more defect: `DIST_2_ARG_CONST_SEED` typed its middle argument `Val(Real)` while its three siblings say `Val(Integer)` — fixed.

What now works (**hiername_examples/**, all exact)

case	result
u1.u2.r (deep hierarchical parameter)	reads the child's default exactly
V(u1.u2.m) / V(\$root.hier.u1.u2.m) / V(hier.u1.u2.m)	all three spellings identical (sum 557 exact)
outer.inner.w, \$root.blocks.outer.inner.w	3.75 exact (was an error)
real vcout; transition(vcout, td, tr, tf)	the LRM comparator compiles and switches

verify_hiername.py: 4/4 PASS. Regression: all 45 example verify suites ALL PASS; 57/57 crate tests.

Notes

- Upward references (a child naming its parent's objects) and hierarchical references *out of* the compiled top module remain out of scope — they have no meaning for a self-contained OSDI model.
- Named blocks inside child instances (u1.blk.v) are not rewritten; they surface as ordinary resolution errors.

Enhancement-50 — domain binding validation

This document describes the change made to OpenVAF-r in the `version11/` directory to enforce the one missing rule of discipline domain bindings (LRM 3.6.2.2). One validation check in `syntax`; no OSDI/ngspice change.

Probe verdict (domain was substantially implemented)

- `domain continuous`; / `domain discrete`; parse and are stored on `DisciplineData` (the std header's `ddiscrete/logic` disciplines exercise `domain discrete` in every compilation);
- nature-bound disciplines default to the continuous domain;
- the domain participates in discipline-compatibility checks, with domainless disciplines treated permissively (LRM 3.6.2.3);
- discrete-domain nets are rejected in analog accesses ("illegal access of branch");
- a custom continuous discipline with natures compiles and simulates exactly.

The gap and the fix

LRM 3.6.2.2: *"It is an error for a discipline to have a domain binding of discrete if it has nature bindings."*
Previously accepted silently:

```
discipline bad
  domain discrete;
  potential Voltage;  // ← no diagnostic
enddiscipline
```

`validate_discipline_decl` now tracks the `domain discrete` binding and the first (unqualified) `potential/flow` nature binding, and emits a two-label error — "the domain is bound discrete here" / "... but a nature is bound here" — with a help note citing the rule and both remedies (drop the natures for a digital discipline, or bind `domain continuous`). Qualified attribute *overwrites* (`potential.abstol = ...`) are not nature bindings and don't trigger it.

Verification

`domainbind_examples/verify_domainbind.py`: 5/5 PASS (continuous discipline simulates at -1 mA exactly; natureless discrete accepted; the error case diagnosed; discrete-net analog access still clean).
Regression: all 47 example verify suites ALL PASS; `syntax/basedb/hir_def/hir_ty` crate tests 24/24.

Enhancement-51 — full ac_stim AC injection

This document describes the changes made to OpenVAF-r and ngspice-46 in the `version11/` directory to complete **ac_stim** (LRM 4.6.3) — the small-signal AC stimulus source, deferred since Enhancement-26 (which fixed the compiler crash and the correct large-signal 0, but injected nothing).

Semantics (LRM 4.6.3)

`ac_stim([analysis_name][, mag[, phase]])` — zero during large-signal analyses and on small-signal analyses whose name doesn't match `analysis_name` (default "ac"); when the analysis matches, a source with magnitude `mag` (default 1) and phase `phase` (radians, default 0).

Implementation (the noise-source mold)

- `hir_lower`: `ac_stim` lowers to a dedicated `CallbackKind::AcStim` callback per call site (exactly like `white_noise`), carrying the analysis name and `[mag, phase]` values; the named signatures take the analysis name as a string literal per the LRM BNF.
- `sim_back`: the callback rides the existing noise extraction — the linearizer classifies it into the small-signal network (so the branch exists with its factor while the large-signal residual stays 0), and `NoiseSourceKind::AcStim { mag, phase }` flows through the shared `NoiseSource` records with `hi/lo/factor`. `mfactor` scales the stimulus *linearly* for current sources and leaves voltage stimuli invariant — deterministic-signal laws, unlike noise's `sqrt(m)`.
- OSDI (ABI 0.6): the sources are partitioned out of the noise arrays (which stay aligned with `load_noise`'s slots via filtered indexing) into appended descriptor fields — `num_ac_stim_src`, `ac_stim_sources({analysis, nodes})`, and a generated `load_ac_stim(inst, model, dst)` that fills `[re, im]` pairs = `factor·mag·cos/sin(phase)` from eval-cached values (op-dependent magnitudes work). `ngspice`'s descriptor view also gained the previously undeclared tail fields it now needs to reach the appended ones. Breaking: OSDI version bumped to 0.6; the loader rejects older `.osdi` files with the recompile message.
- `ngspice (osdiacld.c)`: after loading the AC Jacobian, each instance's active sources (analysis name "ac") add `-re/-im` at node 1 and `+re/+im` at node 2 of the complex RHS — the sign follows the residual convention $(G+j\omega C)x = -\text{residual_stim}$, calibrated so $V(\text{out}) \leftarrow +ac_stim()$ reads exactly $+1\angle 0$.

What now works (**acstim_examples/**, all exact)

case	result
<code>V(out) <+ ac_stim();</code>	AC $v(\text{out}) = 1\angle 0$ exactly
<code>ac_stim("ac", 2.0, $\pi/2$)</code>	$j2$ (phase in radians)
<code>ac_stim("sp")</code>	inactive in AC (0), per LRM name matching
<code>I(out) <+ ac_stim(); into 1k</code>	-1000 (contribution sign convention)
internal stimulus + RC lowpass	$0.5-0.5j$ at f_c ($0.7071\angle -45^\circ$), 0.01 at $100\cdot f_c$ — an embedded AC test bench measuring the model's own transfer
large-signal invariance	DC/transient unchanged (the E-26 checks still pass)
<code>m=3</code> on a current stimulus	$\times 3$ linearly

`verify_acstim.py`: 9/9 PASS (2 original + 7 new). Regression: all 47 example verify suites ALL PASS

(noise + correlated-noise suites confirm the partition is airtight); crate tests 28/28.

Notes

- ngspice's only OSDI small-signal analysis is AC, so "ac" is the only active name; other names are stored and stay inactive (LRM-conformant).
- Noise analysis is unaffected: ac_stim sources never enter the noise descriptor arrays or osdinoise.c's loops.

Enhancement-52 — `idt()` assert/reset forms

This document describes the change made to OpenVAF-r in the `version11/` directory to fix the **`idt(expr, ic, assert[, tol|nature])`** reset forms — the E-28 leftover and the last open item of the integrator family. `hir_lower-only`; no OSDI/ngspice change.

The defect

Per the LRM, while `assert` is nonzero the integral output is reset to `ic` and held; integration resumes from `ic` when `assert` returns to zero. The previous lowering selected between "integrate" (`resist = -expr, react = val`) and "pin" (`resist = val - ic, react = ic`) — so at the reset onset the reactive residual (the integrator's stored charge) jumped from the integrated value to `ic`. The transient integrator's d/dt term turned that jump into a one-step impulse — the exact failure mode Enhancement-27 documented for `idtmod`'s state wrap. Externally-driven resets mostly survived (the impulse settled between measurement points), but a self-referential reset (`idt(1.0, 0.0, V(out) > 1.0)`) fed the impulse back into its own condition: the probe's 1 V/s ramp rang chaotically, went negative, and spiked to ~400 V.

The fix (smooth charge + bounded stiff decay)

- The reactive residual is always **`val`** — the charge never jumps, at either the reset onset or the release.
- Reset is a stiff first-order decay: `resist = K · (val - ic)` with $K = 1e5$ ($\tau = 10\ \mu\text{s}$), so the output is driven to `ic` fast but continuously.
- A conditional **`bound_step`** ($2/K = 20\ \mu\text{s}$) is emitted only while the decay is active ($|val - ic| > 1e-6 \cdot (1 + |ic|)$) — keeping the trapezoidal method in its deadbeat region so the stiff mode cannot ring — and released once settled, so an `assert` held for seconds simulates at full speed. (An unbounded $\tau = 1\ \text{ns}$ decay was tried first: trap's ± 1 amplification of stiff modes made the oscillator undershoot to -0.25 and stretched its period 40%.)
- The DC/IC phase keeps the E-28 behavior: `val` pinned to `ic` algebraically, with the charge ($= val = ic$) handing over continuously into transient.

What now works (**`idtassert_examples/`**, all exact)

case	result
external reset pulse	ramp 0.5→1.5, held at 0.5, resumed to 1.5
op-dependent integrand + reset at the op + tol form	0.25 held, then 2 V/s
self-referential <code>V(out) > 1</code> reset	bounded at exactly 1.0 (was ~400)
relaxation oscillator (<code>idt</code> + hysteretic cross reset)	peaks 1.0, valleys at <code>ic</code> , period exactly 1 s, no undershoot

`verify_idtassert.py`: 9/9 PASS. Regression: all 47 example verify suites ALL PASS; 28/28 crate tests.

Notes

- A non-hysteretic self-reset (`assert = output > threshold`) settles at the threshold (a sliding equilibrium) rather than oscillating — correct continuous dynamics; oscillators need hysteresis, as demonstrated.
- The nature-typed 4th-argument form compiles and behaves as the tol form (tolerances remain accepted-but-unused, the project-wide convention).

Enhancement-53 — `@(final_step)` + analysis-phase lists on step events

This document describes the changes made to OpenVAF-r and ngspice-46 in the `version11/` directory to implement the `@(final_step)` event and the analysis-phase lists on both step events (`@(initial_step("tran","ac"))`), LRM 5.10.2) — the last visibly missing Verilog-A (LRM Annex C) feature.

What was broken

The probe (`@(initial_step)`), phase-qualified variants, `@(final_step)` through `tran` and `op`) confirmed two documented gaps:

1. **`@(final_step)`** never fired. Enhancement-7 implemented `@(initial_step)` via a one-shot `EVAL_FLAG_IS_INITIAL_STEP` but left `final_step` as a documented fail-safe no-op — firing needs “the analysis is over” knowledge the per-iteration eval loop doesn’t have (before E-7 it fired on *every* evaluation).
2. Analysis-phase lists were silently dropped. The AST/HIR always carried `Event::Global { phases: Vec<String> }`, but `lower_event_control` matched `{ kind, .. }` — `@(initial_step("ac"))` fired during a transient run and `@(initial_step("tran"))` fired during an op (the recurring “scaffolded-but-unwired at a boundary” pattern).

The fix

`@(final_step)` — one dedicated post-analysis evaluation

- **`hir_lower`**: new `ParamKind::IsFinalStep` (sibling of `IsInitialStep`, same `op_dependent` classification); `lower_event_control` gates the body on it.
- **`osdi/src/eval.rs`**: `EVAL_FLAG_IS_FINAL_STEP = 1 << 21` (next additive bit above E-7’s 1 << 20, still clear of the core ABI’s flag space; not an ABI change — no descriptor/stride is touched).
- **ngspice**: new `OSDIfinalStep(CKTcircuit *)` in `osdi/osdiload.c` (declared in `ngspice/osdiitf.h`), called once at the successful end of each analysis — `dctran.c` (last accepted transient point), `dcop.c` (an op is both the first and last point of its analysis), `dctrcurv.c` (last DC sweep point), `acan.c` and `noisean.c` (end of the frequency sweep). It issues one dedicated `eval()` per OSDI instance with `EVAL_FLAG_IS_FINAL_STEP` set, computed at the converged final solution (`CKTrhsOld`); the results are deliberately not loaded into the matrix/RHS — the analysis is over, the call exists so `@(final_step)` bodies (*strobe*/fdisplay summaries, cleanup assignments) run exactly once. The `ANALYSIS_*` flags are set from `CKTmode` with `OSDIload`’s mapping so phase-qualified `@(final_step("tran"))` matches.

Phase lists — reuse the **`analysis()`** matcher `lower_event_control` now lowers `@(step("a", "b"))` as `step_flag & (analysis("a") | analysis("b") != 0)` using the same per-name `CallbackKind::Analysis` callback that `analysis(...)` itself uses (Enhancement-30’s OR-shape, factored as `lower_phase_filter`). An empty list fires in every analysis, as before.

The AC/noise op-phase mapping gap `@(initial_step("ac"))` still didn’t fire in an AC run: an AC job’s first model evaluation happens during its DC operating-point phase, where `OSDIload` set only `ANALYSIS_DC|ANALYSIS_STATIC` — so the one-shot fired with no “ac” match. ngspice already treats `tran`’s op phase as part of `tran` (`MODETRANOP` → `ANALYSIS_TRAN`), but `CKTmode` cannot distinguish an AC job’s op from a standalone op. `OSDIload` now consults the running job’s type (`ft_sim->analyses[ckt->CKTcurJob->JOBtype]->name`) and adds the `ANALYSIS_AC` (or `ANALYSIS_NOISE`) name bit only during that op — *not* the reactive `CALC_*` bits `is_ac` carries, which would wrongly enable `ddt`/integration during an operating point. This also makes `analysis("ac")` hold through the whole AC analysis per LRM 4.6.1.

What now works (**finalstep_examples/**, 23 checks, all exact)

analysis	behavior
tran 2 μs	final fires exactly once at $t = t_{\text{stop}}$ and sees the converged solution ($V = 1.0$ at two full sine periods, $1e-6$); final_tran fires; final_ac/final_dc silent; initial_ac silent; multi-phase ("ac", "tran") fires
op	single point = first and last: initial, initial_dc, final, final_dc fire once each; tran/ac-qualified silent
ac sweep	final + final_ac fire once after the sweep; initial_ac + multi-phase fire (op-phase mapping fix); tran/dc-qualified silent
dc sweep 0→2 V	final fires once and sees the last sweep point ($V = 2.0$ exact); final_dc fires
noise sweep	initial + final fire once each
peak track- ing	the LRM's classic use case: a variable accumulated across the whole transient is reported once at final_step ($v_{\text{peak}} = 1.5, 1\%$)

verify_finalstep.py: 23/23 PASS. Regression: all 48 example verify suites ALL PASS; crate tests (hir_lower, sim_back, osdi) all pass.

Notes

- Not an OSDI ABI change: the flag is an additive bit in the existing flags word (same convention as E-7); no descriptor layout moves, so existing .osdi files stay loadable (they just never see the new bit).
- With autostop, final_step fires at the autostop point (the last point the analysis actually solved), which is the LRM-correct reading of "the last point of the analysis".
- Interrupted/failed analyses do not fire final_step — only the successful completion paths call OSDIfinalStep.
- The final-step evaluation may update hidden/event state slots; every analysis re-initializes them (osdisetup.c resets), so subsequent analyses are unaffected.

Enhancement-54 — correct + node-free noise factors

This document describes the changes made to OpenVAF-r and ngspice-46 in the `version11/` directory for the noise "extra node" subsystem — the last survivor of the old candidate list (`linearize.rs`'s "TODO: complex noise power"). The probe turned an optimization task into a correctness one: the extra-node path's noise was silently lost end-to-end.

What the probe found

Four shapes were tested against analytic PSDs (device + surrounding-resistor floors):

shape	before E-54
<code>white_noise(pwr)</code> direct	correct (Linear path)
<code>gm * white_noise(pwr)</code> (op-dependent factor)	extra internal unknown and NO noise output
<code>ddt(cc * white_noise(pwr))</code> (induced-gate idiom)	extra internal unknown and NO noise output
one wave into two branches (correlation network)	extra internal unknown and NO noise output

The onoise spectrum in all three broken cases equaled exactly the surrounding resistors' floor — the device contribution had vanished.

The two correctness defects

1. **build_implicit_equation** never called **add_noise** (`sim_back/src/dae/builder.rs`). Noise attached to an implicit-equation contribution (the `Evaluation::Equation` path: `NoiseSrc` unknowns and `ddt` correlation networks) was dropped before reaching the DAE's `noise_sources` — no OSDI descriptor entry, nothing for the simulator to evaluate. Every other `add_contribution` site pairs with `add_noise`.
2. The late-created `react` `optbarrier` was unregistered. When `build_analog_operators` moves a `ddt()` into a contribution's reactive dimension, `update_optbarrier` creates (or rewrites) the `react` barrier — but it was never inserted into `topology.contributes`, so `prune_small_signal`'s `as_contribution` lookup could not find it: the noise wave's replayed coupling twin was built and then dropped, leaving a hole in the Jacobian (zero transferred noise). The refreshed `psp103_topology.snap` shows the impact on a flagship model: its `react_small_signal` couplings went from `F_ZERO` to real values.

With just these two fixes the Equation path produces exact PSDs (verified against closed-form analytics for all shapes above).

The optimization: factors instead of extra unknowns

3. Op-dependent factors stay Linear — `determine_evaluation` barred `fmul/fdiv` with op-dependent operands from the Linear path. For a `ddt` chain that check is correct ($g(v) \cdot ddt(q) \neq ddt(g(v) \cdot q)$), but for a NOISE wave the right question is *wave-derivation*: only a product of two wave-derived values is nonlinear in the wave; an op-dependent factor is replayed into a per-instance value evaluated at the operating point. The check now discriminates by `val_visisted` for noise (`gm * white_noise` → factor `gm`, no extra unknown) and keeps op-dependence for `ddt`.
4. One **ddt()** in a noise chain becomes a $j\omega$ factor (the old TODO's "complex noise power"). The factor generalizes to `re + jω·im`:
 - `determine_evaluation` admits `TimeDerivative` calls in noise chains (not for `ac_stim` — its injection is a complex RHS pair, not a power), with a pre-scan rejecting nested `ddt` ($(j\omega)^2$

- needs an ω^2 real part) and post-ddt values feeding phis (replay complexity bound) — those fall back to the (now-correct) Equation path;
 - `create_dimension` runs a parallel react replay (`val_map_react`): `ddt(x)` moves the argument's replayed flat component into the react component; `fadd/fsub/fneg/fmul/fdiv/optbarrier` propagate both;
 - `Noise/NoiseSource` carry `factor_react` through the dae builder (`mfactor` scaling — both components scale identically since they multiply the same wave — and switch-branch `phi` joins);
 - OSDI 0.7 (ABI change): `load_noise(inst, model, freq, dst)` fills `[flat, react]` signed power pairs per source (stride 2): `dst[2i] = fac·|fac|·pwr(f)`, `dst[2i+1] = fac_react·|fac_react|·pwr(f)` — the same E-42 sign-carrying fold, and the frequency shaping (`flicker pwr/f^exp, tables`) applies to both;
 - ngspice **osdinoise.c** groups same-named sources as complex amplitudes $(a + j\omega \cdot b) \cdot T_j$ (T = per-source complex transfer): exact for a single source (the two components are in quadrature: $(a^2 + \omega^2 b^2) |T|^2$) and for coherent groups, including anti-phase cancellation; the RFSPICE SP path uses $|a|^2 + \omega^2 |b|^2$;
 - registry gate raised to ≥ 0.7 .
5. Operator-ordering hazard fixed — `analog_operator_evaluations` visited callbacks in registration order, and `determine_evaluation` mutates the function for Linear results (the dimension replay runs inside it). A shared-`FuncRef` `ddt` evaluated *between* two noise operators detached the second wave's path to its contribution mid-flight, misclassifying it as Dead (source silently lost — caught by the anti-phase cancellation test). All noise operators are now processed before any `ddt` operator: noise replays only zero the waves, which is exactly what the subsequent `ddt` pass should see.

What now works (**noisejw_examples/**, 18 checks, all exact)

See the README: plain control; `gm*white_noise` and `ddt(cc*white_noise)` node-free (were one extra unknown each) with exact flat/ ω^2 -shaped PSDs; `ddt(k*flicker_noise)` composing $\omega^2 \cdot kf/f^{\text{exp}}$; coherent same-named flat+`ddt` mixing ($x^2 + \omega^2 \tau^2$); exact anti-phase cancellation; the formerly-lost two-branch correlation network (coherent cross-branch sum incl. all resistor floors); and `m=4` `mfactor` scaling of the $j\omega$ case.

`verify_noisejw.py`: 18/18 PASS. Regression: all 50 example verify suites ALL PASS; crate tests pass (sim_back topology/dae snapshots refreshed — `factor_react` field + the noise-first ordering + psp103's restored react couplings).

Notes

- OSDI ABI 0.6 $\rightarrow$ 0.7: `load_noise`'s `dst` stride changed; ngspice rejects older `.osdi` files — recompile (committed `.osdi` artifacts must be regenerated at fold time, per the E-45 rule).
- Real models work around the old limitations with manual noise networks (e.g. HiSIM2's internal node `n` with `I(n) <+ V(n) + white_noise(...)` coupled via `ci·V(n)` and `ddt(V(n)·sigrat)`); those still compile unchanged — but the direct idiom (`ddt(sigrat·white_noise(...))`) is now both correct and free.
- `ac_stim` through `ddt` stays on the Equation path (its injection is a complex RHS pair, not a power); nested `ddt(ddt(noise))` and op-dependent conditionals around post-ddt values also fall back — correct, just not node-free.
- The env-gated debug dumps added during this work were kept: `OPENVAF_DAE_DEBUG=1` (openvaf: unknowns/residuals/jacobian/noise of each compiled module) and `OSDI_NOISE_DEBUG=1` (ngspice: per-source loaded powers and complex transfers per frequency).

Enhancement-55 — simulation-control tasks + **\$discontinuity** rejection

This document describes the changes made to OpenVAF-r and ngspice-46 in the `version11/` directory for the remaining simulator-behavior audit items: the simulation-control system tasks (`$finish`, `$stop`, `$fatal`), the `$discontinuity` eval-return-flag path, and two documented decisions (`Opcode::Exit`, `is_voltage_src`).

What the probe found

task	before E-55
<code>\$finish</code> in tran	ignored entirely (ran to tstop; the FINISH return flag was never checked in the load path)
<code>\$stop</code> in tran	timestep collapse: <code>E_PAUSE</code> returned mid-Newton-iteration was treated as a step failure → rejection loop
<code>\$fatal</code> under an op-dependent condition	silently deleted at compile time (never fired at all)
<code>\$fatal</code> 's <code>E_PANIC</code>	swallowed by the transient's nonconvergence retry (step ground down)
<code>\$discontinuity(n>=0)</code>	E-24's sentinel clamps the NEXT step only; the event step itself extrapolates across the jump

The fixes

1. Deferred **\$finish/\$stop** (ngspice) Acting mid-Newton-iteration is what broke `$stop` — the correct point is the accepted-point boundary. `OsdiExtraInstData` gains `point_eval_flags`: the eval-return flags OR-ed over all Newton iterations of the current timepoint attempt (an event may fire on an intermediate iteration only; `eval_flags` holds just the last eval's). Reset on `MODEINITJCT|MODEINITPRED|MODEINITTRAN` — a rejected attempt's requests are discarded with it. A new `OSDIpendingRequests(ckt)` (declared in `osdiitf.h`, codes `OSDI_REQ_FINISH/STOP`) reports them; the mid-NR return `E_PAUSE` on `STOP` is removed.

- `dctran.c`: once a point is accepted and output, `FINISH` → Note, fire `@(final_step)` (LRM: `$finish` completes the analysis — E-53 synergy), end the plot, return OK; `STOP` → Note, `E_PAUSE` (resumable).
- `dctrcurv.c`: same per accepted sweep point (`FINISH` exits through the normal restore/endplot path via an `osdi_finish` label).

2. **\$fatal** fixes (a) Compile-time silent deletion. `$fatal`'s lowering emits a Fatal print + `SetRetFlag(Abort)` + `exit`. Neither call takes an op-dependent argument, so the init/eval split classified them op-independent and hoisted them to instance-init — where the op-dependent branch is rewritten to its else edge, leaving them in an unreachable block: deleted from both functions. The taint propagation *should* have control-tainted the branch arm, but the shared post-dominator tree roots at the exit sink (`ipdom(branch)` = the exit arm itself), so `taint_block(arm, end=arm)` inserted nothing — and `compute_postdom_frontiers` walks up to the same bogus `ipdom`, so the frontier row was empty too. Rather than re-rooting the shared `pdom` machinery (consumed by ADCE, the topology pass, ...), the fix in `context.rs::refresh_op_dependent_insts` computes the needed control dependence directly: for every op-dependent branch, blocks reachable from exactly one of its two arms are controlled by it (exact for the structured CFGs the lowering emits — and always exact for early-exit arms, which never reconverge). Side-effecting callbacks (`SetRetFlag`, prints) in op-controlled blocks are marked op-dependent and stay in eval. Parameter-only `$fatal` still hoists to init and validates at setup (instance rejected), as designed.

(b) **E_PANIC** swallowed. `NIiter` propagates the load's `E_PANIC`, but `dctran`'s `if (converged != 0)` treated it as nonconvergence and retried with a smaller step. An explicit check now aborts the transient

with a clear error. (dctrcurv/dcop already propagate errors out.)

3. **\$discontinuity(n>=0)** step rejection The lowering additionally raises a new return flag (RetFlag::Discont → stdlib set_ret_flag_discont → EVAL_RET_FLAG_DISCONT = 16, additive — not an OSDI ABI break; both osdi_0_4.h and ngspice's osdi.h define it). OSDItrunc checks the converged attempt's point_eval_flags, edge-triggered and once per onset: only when the flag is NEW versus the last *accepted* point (prev_point_eval_flags, latched in OSDIaccept) and the per-point retry latch is clear (and CKTdelta > 20*CKTdelmin) does it request CKTdelta/8 — the integrator rejects the too-large event step and retries. The edge/one-shot condition matters: a model announcing a discontinuity over a whole REGION (every eval while a condition holds — the E-24 example does exactly this) must not have every step of the region rejected down to the delta floor (the first cut of this fix, without the latch, made the E-24 regression suite crawl indefinitely). The E-24 sentinel (clamp the *next* step) is kept unchanged.

4. Documented decisions

- **Opcode::Exit => todo!()** in mir_llvm/builder.rs was dead code, not a missing feature: InstructionData::Exit is intercepted by build_inst's instruction-data match (which emits the function's ret) before the value-producing opcode dispatch can see it. Replaced with unreachable! and a comment.
- **is_voltage_src** OSDI exposure (the dae-builder TODO for switch branches): the static metadata is already correct (Switch residuals emit NATREF_NONE), ngspice derives nothing from residual natures at runtime, and no OSDI consumer exists for a per-iteration switch-state export. Deferred until a consumer exists — an eval-output + descriptor array would be an ABI change with zero users today.

What now works (**simctrl_examples/**, 17 checks)

See the README: \$finish ends the transient exactly at the requesting point with @(final_step) firing there; \$stop pauses cleanly; \$fatal prints and aborts (tran) or rejects the instance at setup (parameter validation); a DC-sweep \$finish ends the sweep at the requesting value; and the \$discontinuity A/B twins show the event step $\geq 4\times$ smaller with a sharper, no-later jump.

verify_simctrl.py: 17/17 PASS. Regression: all 51 example verify suites ALL PASS; 28/28 crate tests.

Notes

- Requests latch per instance; any OSDI instance requesting FINISH/STOP is honored (multiple instances OR together).
- \$finish in a lone .op has nothing to cut short (the analysis is one point); it completes normally. \$stop there is likewise a no-op.
- With autostop semantics unchanged; \$finish takes the same clean exit path as reaching the stop time.
- The env-gated OPENVAF_TAINT_DEBUG=1 dump (op-dependence statistics at the init/eval split) was kept from the investigation.

Enhancement-56 — VA_TEST end-to-end sweep: CMC default-range idiom + noise crash

This document describes the changes made to OpenVAF-r and ngspice-46 in the version11/ directory as the outcome of the first end-to-end sweep of the VA_TEST corpus: all 92 standalone industry models compiled and run through ngspice op/AC/tran/noise with a generated bias bench, checking for crashes, aborted analyses, and NaNs (the corpus had previously only been compile-tested).

Sweep-harness lesson

ngspice control scripts continue past aborted analyses, so echo markers after an analysis prove nothing — the first sweep's "92/92 clean" was a false positive. Failure detection must be data-based ("No. of Data Rows" per completed analysis, "simulation(s) aborted", return codes).

The fixes

1. Parameter DEFAULTS exempt from range validation (compiler) CMC-standard models declare a default outside the parameter's own range as the "feature disabled" state and expect the range to bind only user-given values:

```
parameter real CORECOVERY = 0.0 from (0.0:1.0];    // diode_cmc
parameter real Fb          = 0.0 from (0.0:inf);    // FBH-HBT
```

insert_param_init (hir_lower/src/parameters.rs) lowered the default and then range-checked it exactly like a given value, so the stock CMC models (diode_cmc, bsimcmg-110, fbh_hbt, psphv fragments, hisim family, ...) were rejected at setup with "Parameter ... is out of bounds". The not-given branch no longer checks; the given branch still validates both from ranges and exclude constraints (verified: the excluded bound, an exclude-list value, and a beyond-range value are all still rejected when given).

2. Setup rejection diagnostics (ngspice) A \$fatal/\$finish raised during setup is a model rejecting its parameters/configuration (HiSIM validates \$port_connected combinations against COSUBNODE/COBCNODE and calls \$finish(0) by design). ngspice surfaced that as E_PANIC's baffling "impossible error - can't occur". osdisetup.c::handle_init_info now reports *"a Verilog-A device rejected its configuration during setup (finishraised) * rightnexttothemodel's own 'write' message, which was always printed.*

3. Noise-analysis singular-matrix crash (ngspice) noisean.c ignored NIacIter's return value in the frequency loop: on a singular AC matrix the factorization fails, and the noise adjoint solve (SMPcaSolveTransposed → spSolveTransposed) then hit Assertion failed: IS_FACTORED(Matrix) — a hard SIGABRT (reproduced with hisimsoi rejecting an all-terminals bench). The return is now checked: *"AC solution failed at ... Hz; aborting the noise analysis."* The noise analysis additionally honors E-55's deferred \$finish/\$stop raised during its internal operating point (a model that "wanted out" no longer keeps evaluating in a degenerate state).

Sweep verdicts (everything else triaged, not defects)

model	symptom	verdict
hisimhv ×2, hisimsoi- 140	setup rejected	model's own \$finish config guard: 6 nodes connected needs COSUBNODE/COBCNODE=1 — clean diagnostics now
hisimsoi ×3	noise SIGABRT	crash fixed (above); the underlying singularity is the same config guard

model	symptom	verdict
vbic_4T_ et_cf	singular cx/si in AC	the model contributes nothing to those nodes at RCX=0/RS=0 (defaults) by design; commercial simulators' node-gmin covers it — use .option rshunt=1e12
EPFL_ HEMT	op noncon- vergence	bench sensitivity: converges fine at a realistic bias (Vd=0.5, Vg=0.3: Id=127 mA, all analyses pass)
FBH_HBT	tran "timestep too small"	the model divides by its own disabled-default ($\text{ddt}(V(nii)/(2\pi \cdot Fb))$ with Fb=0 → infinite capacitance); give fb>0 — reachable at all only thanks to fix 1

Final sweep result: 83/92 module-runs fully green (op+AC+tran; the setup-rejected CMC models — diode_cmc, bsimcmg-110, psphv, fbh_hbt's op/AC — are all cured); the 9 remaining are exactly the by-design/bench cases tabulated above. All 88 noise runs are crash-free (the three previous SIGABRTs now abort cleanly with diagnostics).

What now works (**paramrange_examples/**, 13 checks)

See the README: out-of-range defaults accepted with exact solutions, given values still validated (three rejection forms), the hisimsoi noise-crash reproducer aborts cleanly, and the stock CMC diode_cmc runs op/AC/noise at defaults with a positive noise spectrum.

verify_paramrange.py: 13/13 PASS. Regression: all 52 example verify suites ALL PASS; 28/28 crate tests.

Notes

- The default-exemption matches industry practice (and the LRM's intent that constraints describe *valid user input*); a model wanting a hard default-check can still assert in its analog block.
- The sweep harness (e56_sweep.py, e56_noise_sweep.py) generates the bench from the OSDI descriptor (terminal count/names via a dlopen dumper); models with thermal terminals get non-sense biases by construction — physics-corrected re-runs were used for the triage verdicts above.

Enhancement-57 — physics-accuracy validation suite

This enhancement adds a permanent, quantitative physics regression suite (`physcheck_examples/`) checking that industry compact models compiled by OpenVAF-r reproduce analytic device laws in ngspice. Unlike E-56's smoke sweep (does it run?), this validates *the numbers*. No toolchain defects were found — the enhancement is the validation deliverable itself, guarding lowering, autodiff, AC, and noise against future regressions.

What is verified (all against closed-form physics)

model	law	result
r2_cmc	thermal noise $\equiv$ built-in ngspice resistor ($4kT/R$)	identical to $< 10^{-6}$
diode_cmc	junction law, 60 mV/decade in the ideal region	$n = 1.004\text{--}1.009$
MEXTRAM bjt505	Gummel slope + β	$n = 1.012\text{--}1.017$, β 73–159
PSP103	AC gm/gds $\equiv$ numeric d/dV of DC (autodiff Jacobian vs residual)	$3\text{--}6 \times 10^{-5}$ relative (finite-difference floor)
JUNCAP200C(V)	grading law self-consistency	1.2×10^{-4}

Notes from the probe

- diode_cmc below ~ 0.9 V is dominated by its recombination/TAT components at default lifetimes (apparent ideality 4–7) — that is the model's physics, not an artifact; the ideal region emerges at 0.96–1.0 V. A naive "check $n \approx 1$ at 0.3 V" would be wrong.
- The gds check in deep saturation is limited by finite-difference noise on a 35 nS conductance (12-digit prints); the triode-region check pins it tightly instead, and the saturation bound is set accordingly.
- The r2_cmc noise identity is constants-free: the same circuit is run with the Verilog-A resistor and ngspice's built-in resistor, and the spectra must be equal — whatever $k \cdot T$ ngspice uses.

`verify_physcheck.py`: 13/13 PASS. `plot_physcheck.py` renders the five laws as PNG plots (`physcheck_examples/plots/`): the diode I–V with its local-ideality panel, the Gummel plot, the PSP g_m AC-vs-numeric overlay, the JUNCAP C(V) fit, and the coinciding thermal-noise spectra. Regression: all 53 example verify suites ALL PASS (no compiler/ngspice source changes in this enhancement).

Enhancement-58 — **defparam** hierarchical parameter override

This document describes the changes made to OpenVAF-r in the `version11/` directory to implement **defparam** (legacy Verilog-2001 compile-time parameter override, LRM 2.6). Front-end only — no OSDI/ngspice change.

What the probe found

defparam did not parse at all. It was listed as a reserved word (`syntax::name`), but had no keyword token and no grammar rule, so `defparam u1.r = 2e3;` tokenised **defparam** as an ordinary identifier and tripped the net-declaration path — surfacing after elaboration as the misleading '**defparam**' was not found in the current scope. The idiomatic instance overrides `#(.r(2e3))` and `#(2e3)` already worked; **defparam** is the deprecated hierarchical form.

The implementation

A **defparam** is a compile-time hierarchical override, so it is handled entirely in the E-5 elaboration pass (the same source-to-source stage that flattens instances) and never reaches the semantic compiler stages.

1. Token (**tokens**) **DEFPARAM_KW** added to the `SyntaxKind` enum, `from_keyword`, `is_keyword`, the `Display` table, and the `T![defparam]` macro; plus a **DEFPARAM** node kind. (**defparam** was already reserved, so making it a keyword breaks no valid model.)
2. Parser (**parser**) A **DEFPARAM_KW** arm in `module_items` dispatches to `defparam_decl`, which parses `defparam <path> = <expr> [, <path> = <expr>]* ;` into a **DEFPARAM** node, reusing the existing `path()` and `expr()` grammar. **DEFPARAM** is deliberately not a member of the typed `ast::ModuleItem` enum — so `module_items()` (the typed iterator every later stage walks) simply skips it, and no `hir_def` / `hir_ty` / lowering change is needed. The node exists only for the elaboration pass to read.
3. Elaboration (**hir/elaborate.rs**)
 - **collect_defparams** runs at the start of `render_items` for every module (top and inlined instances), scanning that module's **DEFPARAM** syntax children. Each target path is resolved to its final flattened name through the very same instance-chain rewrite E-49 uses for hierarchical references (`find_instance_path_holes`): `u1.r` → `u1__r`, `u1.u2.r` → `u1__u2__r`, a single-segment same-module target to itself. The map is keyed by that flattened name (`u1__u2__r` → "4e3"), which is exactly the name the target parameter receives when its instance is inlined, so depth doesn't matter. The value expression is rename-applied (it may reference the enclosing module's own parameters).
 - The **ParamDecl** arm of `render_items`, for each parameter, computes its flattened name and — if a **defparam** targets it — rewrites its default via the existing `render_with_holes` hole mechanism. **defparam** is checked before the instance `#(...)` binding, giving it the higher precedence LRM 2.6 requires.
 - Any **defparam** whose target never matched a rendered parameter is a hard error (**defparam** target(s) did not resolve to any parameter: `u1.typo`), reported with the original source path.

What now works (**defparam_examples/**, 4 checks, all exact)

case	result
<code>defparam u1.r = 2e3</code> (1k default)	2 kΩ → I = 0.5 mA

case	result
deep defparam u1.u2.r = 4e3 + multi-assign defparam up. r=2e3, up.g=1e-3 overriding #(.r(5e3))	2k (not 5k) + g + 4k → I = 1.75 mA
defparam u1.r = 2.0*scale (scale = 3k)	6 kΩ → I = 1/6 mA
defparam u1.typo = 5e3	compile error naming u1.typo

verify_defparam.py: 4/4 PASS. Regression: all 54 example verify suites ALL PASS; crate tests (parser, syntax, hir, hir_lower, sim_back, osdi) all pass; the VA_TEST corpus still compiles.

Notes

- Machine-portability fallout (found by the pre-fold path check): the elaboration pass named its synthetic virtual file after the VFS's canonicalized ABSOLUTE root path, and that name is embedded in the compiled .osdi as source provenance — any model using instantiation (which defparam requires) leaked the build machine's layout into the artifact. The synthetic name is now /<basename>__elaborated.va (the VFS requires a leading /). Relatedly, the macOS linker embeds the -o output path AS GIVEN into the dylib, so every verify script now passes a relative -o (they already run with cwd at the example dir).
- Scope: defparam targeting a same-module parameter in a module that contains *no* instantiation is not rewritten (the elaboration pass only runs when the file has instantiations); the parameter's declared default applies there. The hierarchical instance-override use case — the actual reason defparam exists — always involves instantiation and is fully supported.
- Precedence: defparam > instance #(. . .) > declared default, as the LRM specifies. A model-card (SPICE .model) parameter value still overrides all of these at simulation time, unchanged.
- This reuses the E-49 hierarchical-path resolver and the E-21/#() parameter-hole mechanism wholesale; the only genuinely new machinery is the token, the one parser rule, and the collection/application glue — "scaffolded-but-unwired at the node-kind boundary" turned into a small, self-contained enhancement.

Enhancement-59 — LRM-corner probe follow-up: event OR lists, **\$realtime**, port concatenation, recursion diagnostics

This document describes the changes made to OpenVAF-r in the `version11/` directory following a fresh LRM-corner probe battery (16 never-exercised Annex-C constructs). Front-end only — no OSDI/ngspice change.

The probe battery

16 small models covering LRM corners no previous enhancement had exercised. 12 were validated as already correct (several at runtime, exact): away-from-zero real→integer rounding, `$vt(300)`, alias-param + `$param_given` through the alias, string-parameter case + `.model` string override, `above()` in DC, integer min/max/abs, integer function output args, grounded (single-node) branch declarations, parameter-dependent `laplace_nd` coefficients, `$mfactor` reads, `branch.potential.abstol`, and `ddx` w.r.t. a flow probe.

4 gaps were found and fixed:

gap	before
event OR lists <code>@(a or b)</code> (LRM 5.10.3)	parse error at or
<code>\$realtime</code> (LRM 9.7.2)	unknown system function
port-connection net concat <code>u1({a,c})</code> (LRM 6.5)	whole <code>{a,c}</code> text bound to every bit → "expected value but found net reference"
analog-function recursion	direct: misleading "expected a function but found variable"; mutual (f1 → f2 → f1): compiler stack overflow

1. Event OR lists

`@(cross(...) or cross(...)),@(initial_step or timer(t))` — any mix of members in one list.

- tokens: new `OR_KW` keyword (or was already reserved).
- parser (`grammar/stmts.rs`): `event_stmt` loops over or-separated units; each unit is either an `initial_step/final_step` (with optional phase strings) or an event expression.
- `hir_def`: new `Event::Or(Box<[Event]>)` variant; `collect_event_stmt` segments the syntax children on `OR_KW` tokens and builds one `Event` per unit (a malformed unit degrades the whole event to an unconditional body, as before). The child-expr walker recurses through `Or` via a `&mut dyn FnMut` inner `fn` (a generic closure would monomorphize infinitely).
- `hir_lower(stmt.rs)`: `lower_event_fired` lowers each member to its "fired" boolean and folds them with the file's `bool_or` helper (make_select-based). A raw `ior` instruction is wrong here: the members are `i1` values that can const-fold, and MIR const-eval has no `Bool` arm for `ior` — it ICEs (`invalid operation ior Bool(true) Bool(true)`).

Runtime check: with a crossing pair OR'd, the counter equals the sum of the two single-event counters exactly, and `@(initial_step or timer(0.55u))` fires exactly twice in a 1 μ s transient.

2. **\$realtime**

In the analog context `$realtime` is the simulation time in seconds — identical to `$abstime` (the timescale-scaled digital flavor is meaningless in a continuous-time solver). New builtin lowered to the same `ParamKind::Abstime`. Verified `$realtime - $abstime  $\equiv$  0` through a transient. (Gotcha re-confirmed: `hir_ty/src/builtin/generated.rs` holds a `BUILTIN_INFO` array whose length is hardcoded — appending a builtin without bumping `112usize` → `113usize` is an index-out-of-bounds ICE. Signature tables remain the recurring defect source.)

3. Net concatenation in port connections

`pleaf u1({a, c}); onto inout [1:0] p` — handled entirely in the E-5 elaboration pass. `PortConn::net()` already returned an `ast::Expr`, and E-34 made `{...}` a real `ConcatExpr`, so the pieces existed; `new bind_port_actual` dispatches a `concat actual` to `bind_port_concat`:

- each element contributes one bit (scalar net, bit-select, ...) or — when it names a same-scope bus used whole — all of that bus's bits in *its* declared `msb→lsb` order;
- the flattened bit list maps onto the port in the port's declared `msb→lsb` order (leftmost element = port `msb`), so `[1:0]` and `[0:1]` declaration styles both connect as written;
- a bit-count/width mismatch (or a multi-element concat on a scalar port) is a hard error, collected in a new `port_conn_errors` sink (rendering has no error channel) and bailed after rendering, like `implicit_conflicts`.

Works positionally and named (`.p({b[1], b[0]})`), nested through instance levels. Runtime check: two concat-connected 1 kΩ paths in parallel + 1 kΩ series → exactly 2 V / 1.5 kΩ = 1.3333 mA.

4. Recursion diagnostics

The LRM forbids analog-function recursion; the compiler now says so.

- Direct (fact calling fact): inside a function its own name resolves to the return variable, so the call landed in the “expected a function but found variable ‘fact’” fallback. `inere_fun_call` now recognises `owner == called name` and emits a dedicated `RecursiveFunctionCall` diagnostic: *“analog function ‘fact’ cannot call itself: recursion is not allowed”* with an LRM 4.7 help note.
- Mutual (`f1→f2→f1`): this crashed the compiler (stack overflow in the recursive inliner) — found by a sanity check while testing the direct case. Fix: a call-graph cycle check in `BodyValidationDiagnostic::collect` for function bodies — DFS from each user-function call through the callees’ own (independently inferred, so salsa-cycle-free) `resolved_calls`; a path back to the owner reports the full chain: `info: call cycle: f1 -> f2 -> f1`. Legitimate diamond call chains (`f→g→h, f→h`) are unaffected.

Examples (`lrmcorner_examples/`, 9 checks, ALL PASS)

`verify_lrmcorner.py`: [1] OR-list counter $\equiv$ sum of single-event counters + `initial_step` or timer fires twice; [2] `$realtime` tracks `$abstime` exactly; [3] port-concat op current exact (both concat forms); [4] direct and mutual recursion are clean errors (the mutual one names the cycle); [5] concat width mismatch rejected; [6] `lrmpin_demo.va` — self-checking bitmask module (E-37 technique) pinning 8 runtime-checkable validated corners, score 255/255; [7] compile-only pins (gnd branch, ddx-flow, above-DC, laplace-param, integer fn outputs).

Note: the corner-pin’s `$vt(300)` check uses a 1e-7 tolerance — the compiler’s internal `$vt` uses newer CODATA constants than the LRM-1998 values in the shipped `constants.vams` (difference $\approx 3e-8$). Not a defect.

Regression

All version11 example verify suites pass; crate tests (tokens, parser, syntax, `hir_def`, `hir`, `hir_ty`, `hir_lower`, `sim_back`, `osdi`) pass.

Enhancement-60 — multiple analog blocks: validation deliverable

This document records the Enhancement-60 probe of multiple **analog** / **analog initial** blocks per module (Verilog-AMS LRM 6.2: several blocks “shall behave as if they combine into a single analog block in the order they appear”). No defects were found — the feature is fully supported by construction — so, like Enhancement-57, the deliverable is the validation itself: a probe battery, a pinned example suite, and this write-up. No compiler or ngspice source changes.

Why it works by construction

`hir_def/src/body.rs` collects a module’s behavioural body as

```
body.entry_stmts = if initial {
    ast.analog_initial_behaviour().map(|stmt| ctx.collect_stmt(stmt)).collect()
} else {
    ast.analog_behaviour().map(|stmt| ctx.collect_stmt(stmt)).collect()
};
```

— an iterator over every analog (respectively analog initial) block of the module, collected into `entry_stmts` in document order. That is literally the LRM’s as-if-concatenated semantics; every downstream stage (type inference, lowering, autodiff, the DAE builder, OSDI) already consumes `entry_stmts` as a list and sees one concatenated body. There is no single-block assumption anywhere to fix.

The probe battery (9 corners, zero defects)

corner	result
contributions to one branch from several blocks	accumulate exactly (1+2 mS → 3 mS)
module variable written in block 1, read in block 3	shared, in order
execution order (<code>\$strobe</code> in consecutive blocks)	source order preserved
<code>analog</code> function declared between two blocks	callable from later blocks
module declarations (parameter/branch/var) after a block that uses them	resolve (module-scope, order-free)
events split across blocks (<code>cross</code> in one, <code>final_step</code> in another; <code>initial_step</code> counters)	all fire, once each
<code>ddt()</code> of a charge computed in an earlier block	correct reactive residual
multi-block module instantiated (E-5 elaboration re-render)	both blocks survive flattening — series current exact to 12 digits
two <code>analog initial</code> blocks	compose in order (<code>g = 1m; g = g + 1m</code> → exactly 2 mS)

Two behaviors verified as *correctly rejected* / *consistent* rather than gaps:

- duplicate named child blocks (`begin : work` in two analog blocks): clean 'work' was already declared in this scope error — named blocks live in the module namespace, so hierarchical references (`inst.work.t`) must stay unambiguous;
- `V(a,c) <+ in one block` and `I(a,c) <+ in another` compiles — identical to the single-block switch-branch behavior (LRM value-retention rules), i.e. multi-block introduces no inconsistency.

Examples (**multianalog_examples/**, 6 checks, ALL PASS)

`verify_multianalog.py`: [1] three-block accumulation to exactly 3 mS + cross event in the middle block fires; [2] strobos print in source order + ordered `analog initial` composition (2 mS exact); [3]

a multi-block module survives instance flattening (series current exact to 12 digits); [4] the duplicate-named-block diagnostic.

Notes

- Verilog-A 1.0 originally allowed only one analog block per module; Verilog-AMS 2.2+ relaxed this, and openvaf-r follows the relaxed (current LRM) rule. Accepting the multi-block form is a superset of Annex C either way — no legal single-block model changes meaning.
- Regression: no compiler/ngspice source changes in this enhancement; the Enhancement-59 full regression (55 suites, crate tests, 92/92 corpus) stands, plus the new suite's 6 checks.

Enhancement-61 — operator-argument audit: **slew** sign-convention fix

This document describes the changes made to OpenVAF-r in the version11/ directory following a full-argument-form audit of the analog operators (LRM 4.5) and events (LRM 5.10). Front-end only — no OSDI/ngspice change.

The audit

22 probe forms covering every optional-argument spelling: `cross` 3/4-arg (`time_tol`, `expr_tol`), `above` 2/3-arg, `timer` 2/3-arg (`period` + `tolerance`), `absdelay` 3-arg (`maxdelay`), `transition` 5-arg (`trailing tolerance`), `slew` 2/3-arg, `ddt` 2-arg (`abstol`), `idt` 4-arg, `idtmod` 5-arg, `last_crossing` 1/2-arg, `laplace_*/zi_*` trailing tolerance + full `zi` arg lists (`T`, `τ`, `t0`), `$bound_step`, `$limit` (built-in "pnjlim" string, user functions, user functions with extra arguments), and `ac_stim` with magnitude and phase.

One real defect found (and fixed); everything else verified working at runtime, not just parsing:

verified working	evidence
<code>timer(start, period[, tol])</code>	fires exactly 5× at 0.1/0.3/0.5/0.7/0.9 μs
<code>\$bound_step(5n)</code>	eval count 120 → 416 over a 1 μs transient
<code>\$limit "pnjlim" + user fn (+ extra args)</code>	stiff 5 V/1 Ω diode converges directly (raw diode needs gmin stepping); exact op 0.9345 V
<code>ac_stim("ac", 2.0, π/2)</code>	V = j1000 exactly (magnitude AND phase honored)
<code>ddt(x, abstol)</code>	−j2πfC to 10 digits at AC
<code>idt(x, ic, assert, abstol), idtmod(x, ic, mod, off, abstol)</code>	+j1e-6 exactly at ω = 1000
<code>toleranced cross/above</code>	fire correctly (tolerances are step-control hints)
<code>transition 4/5-arg</code>	linear ramp, exact midpoint (first runtime pin ever)
<code>absdelay(x, td, maxdelay)</code>	1.5e-5 max error vs the analytic delayed sine
<code>laplace_*/zi_*</code> trailing ε	accepted; realization exact (documented no-op)

(Also confirmed: an empty {} vector for "no zeros" is rejected — correct, Verilog-A has no empty array-literal syntax; E-34's EmptyConcat diagnostic stands.)

The defect: **slew** ignored its input

Before: `slew(V(in), 1e6, -1e6)` — the LRM-conformant spelling (4.5.15: *max_pos_slew_rate shall be greater than zero, max_neg_slew_rate shall be less than zero*) — produced an output that ignored the input entirely: it ramped at +1e6 V/s from t = 0 (while the input was still 0) and sailed unboundedly past the target (2.0 V at 2 μs for a 1.0 V input).

Root cause: `lower_slew` fed the bounds to the shared `lower_rate_limited_track` loop (`dy/dt = clamp(K·(x-y), lo, hi)`) after `fneg(neg_max)` — assuming a *positive magnitude* third argument. An LRM-conformant negative value was double-negated into a positive lower clamp bound (+1e6), so the clamp forced `dy/dt = +1e6` regardless of the input: a positive-feedback runaway disguised as a ramp. (`transition`, which converts rise/fall *times* to always-positive rates before calling the same helper, was unaffected — which is why it worked while `slew` didn't.)

Fix (`hir_lower/src/expr.rs`): bound with `|max_pos| / -|max_neg|` via a new `lower_fabs` helper (`neg/lt/select` — MIR has no `fabs` instruction). Exact for LRM-conformant inputs and tolerant of the legacy positive-magnitude spelling; the single-rate form keeps its LRM "absolute value bounds both directions" behavior.

After: the output holds at the input, ramps at exactly the bound when the input outruns it (asymmetric rates verified: rise 1e6, fall 0.25e6 — both edges exact), and stops at the target.

Examples (**opargs_examples/**, 16 checks, ALL PASS)

`verify_opargs.py`: [1] the fixed `slew` defect (holds before the step, rate-limited rise, stops at the target, asymmetric fall); [2] tolerated events + timer period (5 fires exact); [3] `$limit` `pnjlim` + user fn converge directly to the exact op with no `gmin` fallback; [4] `$bound_step` honored; [5] `ac_stim` magnitude and phase exact; [6] `ddt/idt/idtmod` trailing tolerances numerically exact at AC; [7] transition ramp semantics. Items [1] and [7] are the first runtime verification the `slew/transition/absdelay` operator family has had (the old `slew_examples/transition_examples/absdelay_examples` folders have no verify scripts).

Regression

All version11 example verify suites pass; crate tests (`hir_lower`, `sim_back`, `osdi`, `mir_autodiff`) pass; VA_TEST corpus compiles 92/92.

Enhancement-62 — ngspice analysis coverage for OSDI devices: **.dc @inst[param]** sweeps + **.disto** warning

This document describes the changes made to ngspice-46 in the version11/ directory following an analysis-coverage probe of OSDI (Verilog-A) devices across every ngspice analysis beyond op/dc/ac/tran/noise. ngspice-only — no compiler/OSDI ABI change.

The probe (built-in-twin comparisons, E-57 technique)

analysis	verdict	evidence
.tf	exact	OSDI divider: $TF = 0.75$, $Z_{out} = 750\ \Omega$, $Z_{in} = 2\ k\Omega$ (with a parallel built-in twin)
.pz (linear)	exact	OSDI RC pole $-1e6\ rad/s$, bit-identical to the built-in RC
.pz (nonlinear bias)	parity	"input signal is shorted" — but the <i>built-in diode circuit fails identically</i> : an ngspice pz quirk, not an OSDI gap
.sens (DC)	exact	dV/dR matches the analytic divider derivative ($-1.875e-4 / +6.25e-5$)
.sens (AC)	exact	$dV/dacmag = 0.5 - 0.5j$ at the pole frequency
.dc temp sweep	exact	\$temperature-dependent R: all points match $1/R(T)$, $^{\circ}C \rightarrow K$ included
alter / altermod / instance-line / print @n1[r]	works	via (* type="instance" *) (the OSDI instance-kind convention)
.dc @n1[r] param sweep	GAP 1	fatal error — sweep code hardcodes Vsource/Isorce/Resistor/temp
.disto	GAP 2	silent zeros: OSDI diode measured 0.0 where the identical built-in diode measured $1.8e-6$

Gap 1 fixed: generic **.dc @inst[param]** sweeps

dctrcurv.c recognized only voltage sources, current sources, resistors, and temp as sweep variables. A new **PARAM_CODE** sweep type accepts @inst[param]:

- DCTfindInstParam resolves the instance by case-insensitive name walk (the sweep name is a raw token, not an interned IFuid, so the device hash can't be used) and looks the parameter up in the device's own instanceParms table (settable, real-valued);
- DCTsetInstParam sets each sweep value through the generic DEVparam interface and refreshes with DEVtemperature — for OSDI devices exactly the alter path (setup_model + setup_instance re-run);
- integrated into every arm of the sweep machinery: stop criterion, nested-level reset, per-step increment, plot scale (param-sweep), XSPICE event step, and end-of-sweep restore (the saved value is restored; the parameter necessarily stays marked "given");
- the previous value is captured with DEVask before the sweep.

Works for any device type: OSDI instance-kind parameters (.dc @n1[r] 1k 4k 1k — $I = 1/R$ exact at every point), built-in devices (.dc @r1[resistance]), and nested with other sweep variables (.dc @n1[r] 1k 2k 1k V1 1 2 1 — inner level resets correctly).

New struct fields TRCVvParmId / TRCVvNow in trcvdefs.h (devices have no generic readback field to consult mid-sweep).

Gap 2 fixed: **.disto** warns instead of silently lying

Distortion analysis needs per-device Taylor coefficients (DEVdisto); the OSDI ABI only exposes first derivatives, so OSDI nonlinearities are invisible to **.disto** — and ngspice skipped such devices without a word. A **.disto** run over an OSDI diode reported exactly 0.0 distortion while the identical built-in-diode circuit reported 1.8e-6.

CKTdisto.c (D_SETUP) now prints a prominent warning for every OSDI device type present in the circuit (recognized by the DEVpublic.registry_entry marker that only OSDI device types carry):

```
Warning: Verilog-A (OSDI) device type 'odio' has no distortion model;  
        .disto results will NOT include its nonlinearities.
```

The analysis still runs (results remain valid for circuits whose nonlinear devices are all built-ins and whose OSDI content is linear). Implementing real OSDI distortion would require higher-order derivatives through the OSDI ABI — an order of magnitude beyond this enhancement's scope, now documented instead of silent.

Examples (**analyses_examples/**, 19 checks, ALL PASS)

verify_analyses.py: [1] **.tf** exact; [2] **.pz** exact + OSDI $\equiv$ built-in; [3] **.sens** DC + AC exact; [4] temp sweep exact; [5] new @inst[param] sweeps (OSDI, built-in, nested); [6] alter/altermod/instance-line/readback (including "altermod must not override a given instance value"); [7] new **.disto** warning + analysis completion.

The folder doubles as a tutorial: nine standalone, commented decks (tf.cir, pz.cir, sens_dc.cir, sens_ac.cir, temp_sweep.cir, param_sweep.cir, nested_sweep.cir, alter.cir, disto.cir — each runnable with ngspice -b <deck>.cir and stating its expected numbers), a walk-through README, and plot_analyses.py, which renders the sweep results to plots/: param_sweep.png (the new @n1[r] sweep vs analytic 1/R), nested_sweep.png (the $I = V/r$ curve family), temp_sweep.png (1/R(T) overlay), and ac_lowpass.png (RC Bode plot with the **.pz** pole at -3 dB and the **.sens** ac point at -45° marked).

Regression

All version11 example verify suites pass (ngspice rebuilt); no compiler change, so crate tests and the corpus stand as of Enhancement-61.

Enhancement-63 — RF analyses with OSDI devices: `.sp` / transient noise / PSS + `span.c` NaN fix

This document describes Enhancement-63: round 2 of the analysis-coverage work (Enhancement-62), probing the RF-flavored ngspice analyses — S-parameters, transient noise, and periodic steady state — with OSDI (Verilog-A) devices against built-in twins. One stock-ngspice defect found and fixed (`span.c`); ngspice-only, no compiler/ABI change.

`.sp` — S-parameter analysis: exact

- A series 100 Ω OSDI resistor between 50 Ω ports gives the textbook $S_{11} = R/(R+2Z_0) = 0.5$ and $S_{21} = 2Z_0/(R+2Z_0) = 0.5$, exactly.
- A frequency-dependent OSDI $R + C$ shunt two-port is bit-identical to the built-in R/C twin across three decades (1 MHz – 100 MHz).
- Arbitrary port count: `span.c` allocates every matrix $\text{CKTportCount} \times \text{CKTportCount}$, so `.sp` is fully N-port — a 3-port direct junction reproduces the textbook $S_{ii} = -1/3$ / $S_{ij} = +2/3$, and 3-/4-port OSDI resistor stars give the analytic 1/3 and 1/4 exactly (bit-identical to built-in twins). Only the `donoise` noise-parameter block (NF/SOpt/Rn — inherently two-port concepts) is restricted to exactly 2 ports.

`.sp ... donoise` — noise figure: OSDI exact, and a stock NaN fixed

The OSDI noise pipeline (E-42/E-54) reaches S-parameter noise figures: a Verilog-A resistor with explicit `white_noise(4kT/R)` gives $NF = 10 \cdot \log_{10}(1 + R/Z_0) = 4.7712$ dB exactly.

The defect (stock ngspice, exposed by OSDI parity testing): the same topology with a *built-in* resistor returned NaN for NF/SOpt/NFmin. `span.c`'s noise-parameter extraction computes $Y_{\text{sopt.re}} = \sqrt{SQR(Y_{\text{cor.re}}) + G_u/R_n}$, where G_u — the uncorrelated noise conductance — is analytically zero for a fully-correlated single-noise-source topology. Floating-point rounding can land the `sqrt` argument at $-1e-18$: `sqrt(negative)` $\rightarrow$ NaN. (The OSDI twin's rounding happened to stay ≥ 0 , which is exactly how the parity test caught it.)

Fix: clamp the argument to its physical range (≥ 0). After the fix the built-in single-R case reads 4.7712125 dB — $10 \cdot \log_{10}(3)$ to eight digits — and the multi-source regression case (series 100 + shunt 200) is unchanged at 7.2016 dB.

Transient noise: correct propagation, documented parity

A `TRN0ISE` source driving an OSDI divider propagates correctly (deterministic mean exact, noise σ as expected). Device-*internal* noise (`white_noise/flicker_noise`) does not enter `.tran` — the same is true of every built-in device's noise model; transient noise in ngspice comes only from `TRN0ISE` sources. Parity, not a gap.

PSS — periodic steady state: OSDI devices are full citizens

`.pss` is experimental and compile-time optional (`--enable-pss`; the suite auto-detects and SKIPs without it, and the README shows the build recipe — a `build-pss/` tree keeps it out of the default build).

- Linear OSDI RC (1k/1n, driven at 1 MHz): shooting converges (16 iterations), predicted fundamental within 0.1 ppm, and the fundamental harmonic equals the analytic $|H| = 1/\sqrt{1+(\omega RC)^2} = 0.1571767$ — matching the built-in twin to 7 digits.
- Nonlinear OSDI diode (mild drive): converges in 2 shooting iterations — the built-in diode twin actually *wanders longer* (its frequency estimate jumped to ~2 MHz before settling).
- Strongly nonlinear rectifiers defeat the shooting method for built-ins and OSDI alike (frequency-estimation wander) — a stock characteristic of the experimental PSS core, not an OSDI limitation.

Examples (**rfanalyses_examples/**, 15 checks, ALL PASS)

`verify_rfanalyses.py`: [1] `.sp` resistive exact; [2] `.sp` RC bit-identical to built-in; [2b] N-port S-matrices (3-port junction textbook values, 3-/4-port OSDI stars exact + bit-identical to built-in); [3] donoise — OSDI NF exact, built-in NF finite and analytic (the fix), 2-resistor regression unchanged; [4] TRNOISE through OSDI (mean exact, σ in band); [5] PSS — linear exact + twin parity

- nonlinear convergence (SKIPs cleanly without `--enable-pss`).

Regression

All version11 example verify suites pass with the rebuilt ngspice; no compiler change (crate tests / corpus stand as of Enhancement-61/62).

Enhancement-64 — Touchstone export: auto-**Rbase**, N-port **wrsnp**, 1-port **.sp**

This document describes Enhancement-64: making S-parameter results exportable to industry-standard Touchstone v1 files from the **.sp** analysis. Follows directly from Enhancement-63's findings. ngspice-only — no compiler/OSDI change.

What was broken

1. **wrs2p** was unusable out of the box. It required a vector named **Rbase** (the reference resistance for the Touchstone option line `# Hz S RI R <Rbase>`) that no code ever created — every call died with `Error: No Rbase vector given` unless the user knew the undocumented `let Rbase = 50` incantation, and a wrong value silently mislabeled the file header. The reference impedance is *already known* from the ports' `z0`.
2. 2 ports only, though the **.sp** analysis itself is fully N-port (Enhancement-63 verified 3-/4-port S-matrices analytically exact).
3. 1-port **.sp** was rejected outright ("we need at least two!") — and the check was hiding a real crash: with it lifted, ngspice died with `malloc: can't allocate -8 bytes`.

The fixes

Auto-Rbase** (**span.c**, **cktspdum.c**, **postcoms.c**)** The **.sp** analysis now publishes **Rbase** as a vector in its plot — one more UID after the Z-matrix block, with the per-point value read straight from port 1's `VSRCportZ0` (the `refPortY0` global is only set on the noise path, so it is not used). `com_write_sparam` reads it robustly (the **sp** plot is complex data; the old code blind-dereferenced `v_realddata[0]`) and a user-defined `let Rbase = ...` still overrides.

N-port writer + **wrsnp (**postcoms.c**, **commands.c**)** New `spar_write_np()` writes Touchstone v1 for any port count directly from the `S_i_j` vectors: `# Hz S RI R <Rbase>` option line; for $N \geq 3$ the matrix is row-major with at most four complex pairs per data line and every matrix row starting on a new line (per the Touchstone 1.x spec); a 1-port is one pair per line. The port count is auto-detected by probing `S_n_n` vectors. The classic 2-port `S11 S21 S12 S22` column order is preserved through the original `spar_write()` path. A new **wrsnp** command is registered; **wrs2p** transparently dispatches to the N-port writer when the plot isn't 2-port.

1-port **.sp (**span.c**, **maths/dense/dense.c**)** The "at least two ports" hard error was over-strict — a 1-port **.sp** is a plain reflection measurement and the matrix machinery is N-general. The real bug it hid: the complex-matrix `cadjoint()` had no 1×1 base case, so its cofactor loop allocated 0×0 and then negative-sized minors (`cdet` had the base case; `cadjoint` didn't). The adjugate of `[a]` is `[1]` (the empty minor's determinant is 1 by convention), which makes `cinverse` of a 1×1 equal $1/a$. With both fixed, a $100\ \Omega$ load on a $50\ \Omega$ port gives $S_{11} = 1/3$ exactly in a proper **.s1p**.

Examples (**touchstone_examples/**, 11 checks, ALL PASS)

`verify_touchstone.py`: [1] **wrs2p** with no manual **Rbase** (header `R 50`; the file's `S21` pairs equal the plot's `S_2_1` — also pins the 2-port column order); [1b] **wrsnp** on a 2-port produces a byte-identical file to **wrs2p** (same handler — both commands cover every port count); [2] `let Rbase = 75` override honored; [3] 1-port **.s1p**, $S_{11} = 1/3$ exactly (pins the `cadjoint` crash fix); [4] **.s3p** — 3 frequency blocks $\times$ 9 pairs of exactly $1/3$ with each matrix row on its own line; [5] **.s5p** — 25 pairs of exactly $1/5$ with rows wrapping at 4 pairs per line.

Notes

- Found while testing: **.sp** `lin 2 fstart fstop` produces only one frequency point in stock ngspice (`lin 3/lin 5` behave); worked around in the suite with `lin 3`, not fixed (separate stock

quirk).

- All values written are from OSDI (Verilog-A) resistor networks with analytically known S-matrices — the star topologies from Enhancement-63's N-port verification.

Regression

All version11 example verify suites pass with the rebuilt ngspice; no compiler change.

Enhancement-65 — preprocessor audit: macro-recursion guard

This document describes the changes made to OpenVAF-r in the `version11/` directory following a systematic audit of the Verilog-A compiler directives — the preprocessor predates every enhancement in this series and had never been probed. Front-end only — no OSDI/ngspice change.

The audit (22 probe forms)

Everything probed works, and the 18 compiling probes were verified numerically exact at runtime (each engineered to produce exactly 1 mA — compiling proves nothing about correct expansion):

feature	status
<code>define</code> (simple, 1-arg, multi-arg with nested parens in actuals)	exact
macro using another macro; macro call as another macro's argument	exact
<code>ifdef/else</code> , <code>ifndef</code> , <code>elsif</code> chains, nested conditionals	exact (right branch, dead branch dead)
conditionals inside a module body	exact
<code>undef</code> + redefinition; <code>resetall</code>	exact
backslash-continued (multi-line) definitions; trailing <code>//</code> comments	exact
macro calls spanning lines	exact
nested <code>include</code> chains (file → file → file)	exact
undefined macro use; unbalanced <code>ifdef</code>	clean, located errors
macro redefinition	warning (macro was overwritten)

One defect: recursive macro expansion crashed the compiler — a stack overflow for both the direct form (``define LOOP (`LOOP + 1)`) and the mutual form (``A uses `B uses `A`).

The fix — the fifth "scaffolded-but-unwired" find

The preprocessor's diagnostic enum already had a `MacroRecursion` variant with a rendered message ("macro 'X' was called recursively") — but nothing ever emitted it: `call_macro` carried a literal `// TODO track recursion`, and the report builder in `basedb/diagnostics/preprocessor_error.rs` was a `literaltodo!()` that would have panicked had the diagnostic ever been created. (This is the same defect pattern as E-39 derived natures, E-45 nature attributes, and E-46's based-literal sketch: complete machinery on one side of a boundary, never wired on the other.)

Two changes:

1. **`preprocessor/src/processor.rs`**: an `expansion_stack: Vec<&str>` on the `Processor`; `call_macro` reports `MacroRecursion` when the called name is already on the stack. Crucially, the name is pushed around the body expansion only, after arguments are built: a nested call of the same macro inside an *argument* — ``define QUAD(x) (`TWICE(`TWICE(x)))` — is finite and legal (argument tokens belong to the caller's expansion), and the first draft of the guard wrongly rejected exactly that. The final scoping catches direct bodies, mutual cycles through any depth, and recursion smuggled through arguments' *expansions*, while leaving argument-level self-nesting alone.
2. **`basedb/src/diagnostics/preprocessor_error.rs`**: the `todo!()` replaced with a real report — primary label at the recursive call site plus a help note.

Both forms now produce clean, source-located errors; the legitimate QUAD nesting compiles and evaluates exactly.

Examples (**preproc_examples/**, 5 checks, ALL PASS)

`verify_preproc.py`: [1] an 8-way self-checking macro tour (one branch per feature, total exactly 8 mS, with a not-taken `ifdef` that would add 100 S if it leaked); [2] a two-deep `include` chain; [3] direct recursion = clean error naming the macro; [4] mutual recursion likewise; [5] the `QUAD(x) = TWICE(TWICE(x))` false-positive regression pin, exact at runtime.

Regression

All version11 example verify suites pass; crate tests (preprocessor, basedb, parser, syntax, hir, hir_lower, sim_back, osdi) pass; the VA_TEST corpus compiles 92/92.

Enhancement-66 — Monte Carlo with OSDI devices: validation deliverable

This document records the Enhancement-66 probe of Monte Carlo (statistical) simulation over OSDI parameters — the remaining workflow gap after parameter sweeps (E-62) and `alter`. No defects were found — ngspice's statistical machinery fully reaches OSDI parameters — so, like Enhancements 57 and 60, the deliverable is the validation itself: a probe battery, a pinned example suite with distribution plots, and this write-up. No compiler or ngspice source changes.

Both standard MC idioms work

1. The **reset** idiom. `.param rr = agauss(1k, 100, 3)` feeding a `.model card (r={rr})` or an instance line (`N1 a 0 mm r={rr}`), requires the `(* type="instance" *)` param kind from E-62): each reset re-throws the dice and re-runs the OSDI model/instance setup. Verified over 200-run ensembles: parameter readback (`@n1[r]`) gives mean $\approx 1\text{ k}\Omega$ and $\sigma \approx 33.3\ \Omega$ ($= \text{avar}/\text{sig}$), all draws within $\pm 5\sigma$, and the output-current spread matches $\sigma_I \approx \sigma_R/R^2$.
2. The **alter** loop. Control-language random vectors (`sgauss(0)` standard normal, `sunif(0)` uniform on $[-1, 1]$ — not $[0, 1]$) with `alter @n1[r] = value` per run: no netlist re-parse, and `setseed N` makes the entire run sequence bit-reproducible (two seeded 200-run passes produce identical statistics to the last digit).

Also verified: `aunif(nom, avar)` bounds respected with a uniform-like spread ($\sigma \approx 2 \cdot \text{avar} / \sqrt{12}$); single-draw seeded reset reproducibility; and a nonlinear MC — an OSDI diode with `is_={agauss(1e-15, 2e-16, 3)}` whose op-point spread matches the analytic sensitivity $\sigma_V \approx v_t \cdot \sigma_{Is}/Is$ (1.54 mV measured vs 1.73 mV analytic, $N = 100$).

The documented gotcha

Every textual occurrence of a random-valued `{param}` draws independently: an OSDI device and a built-in resistor written with the same `{rr}` differ run by run (pinned: $\max |\Delta I| = 9.2\text{e-}5 \neq 0$ over 25 runs). Matched/correlated devices therefore need the `alter` idiom, where one control-language value is explicitly assigned to each instance. (This is stock `.param` semantics, not an OSDI behavior — but it is exactly the kind of trap a mismatch analysis falls into, so the suite pins it.)

Examples (`montecarlo_examples/`, 10 checks, ALL PASS)

`verify_mc.py`: [1] reset-idiom MC on a model-card param (mean and σ vs analytic); [2] instance-line param with `@n1[r]` readback ($\pm 5\sigma$ bounds); [3] seeded `alter+sgauss` loop — σ analytic + two passes bit-identical; [4] `aunif` bounds and spread; [5] seeded single-draw reproducibility; [6] the independent-draws gotcha; [7] nonlinear diode MC vs the analytic sensitivity.

`plot_mc.py` renders `plots/mc_distributions.png`: 500-run gaussian and uniform MC histograms sitting on the analytic transformed densities of $I = 1V/R$ (including the $1/i^2$ tilt of the uniform case).

Notes

- `wrdata` is unusable for control-created vectors: it pairs each vector with the *current plot's* scale, which after the last op has length 1 — a 500-long MC vector writes as one row. The plot script parses `print <vec>`'s indexed table instead.
- Regression: no compiler/ngspice source changes in this enhancement; the Enhancement-65 full regression (61 suites, crate tests, 92/92 corpus) stands, plus this suite's 10 checks.

Enhancement-67 — generate audit: genvar-substitution fix, nested loops, **generate if/case**

This document describes the changes made to OpenVAF-r in the `version11/` directory following a systematic audit of the `generate/genvar` machinery (built in Enhancement-5, never audited since). Front-end only — no OSDI/ngspice change.

The audit (13 probe forms)

Already correct: the basic labelled `generate for` ladder, non-trivial bounds (`start  $\neq$  0`, `step 2`, `<=`), per-iteration net declarations, genvar reuse across regions, bit-select indices with genvar arithmetic (`n[i+1]`), and the descending-loop / non-constant-bound rejections.

One defect and three gaps:

finding	before
DEFECT: genvar in an ordinary expression (<code>#(.r(1e3*(i+1)))</code>)	substituted through the identifier-renaming path, which re-escaped the numeral into a broken escaped identifier: <code>1e3*(\0 +1) $\rightarrow$ "error: '0' was not found"</code> . Bit-selects only worked because they had a dedicated pre-fold.
nested <code>generate for</code> loops	parse error (the block grammar had no <code>for arm</code>)
anonymous <code>begin</code> (no <code>: label</code>)	parse error — the label was mandatory (1364-2005 allows anonymous blocks)
<code>generate if / generate case</code>	mis-parsed into a broken <code>GENERATE_FOR</code> with misleading errors ("generate for: missing loop condition" for an <code>if</code>)

The implementation

Grammar (**parser**), AST (**syntax**), tokens — New `GENERATE_IF / GENERATE_CASE / GENERATE_CASE_ARM` node kinds with typed AST nodes and `ModuleItem` variants (full pipeline threading, the E-34 recipe). After `generate`, the parser dispatches on `for/if/case`; the same tail rules are reused for nested constructs inside a `generate` block (which have no `generate/endgenerate` of their own). `begin's : label` is now optional. `generate if` supports `else` and `else if` chains; `generate case` supports multi-value arms and `default`.

The unelaborated-generate safety net in `hir_def`'s item-tree lowering covers the new node kinds too (a `generate` construct that survives to `hir_def` is a diagnostic, never a miscompile).

Elaboration (**hir/elaborate.rs**) — recursive per-item rendering `render_generate_for` now recurses through a shared `render_generate_block(block, env, suffix, scope)`:

- **env** holds *every* genvar in scope (outer loops included), so inner bounds may reference outer genvars and all folds see the full picture;
- **suffix** accumulates per-iteration name disambiguators (`_0_2` two loops deep) — each block suffixes its *own* directly-declared names, so nested declarations flatten collision-free;
- the genvar fix: bit-select indices keep their whole-index constant fold (the bus machinery requires literal indices), and every *remaining* bare genvar identifier becomes a literal-value hole — genvars never pass through identifier renaming (whose escaping was the defect) anymore;
- **generate if/case** fold their condition/discriminant with a new `eval_cond_expr` (comparisons, `&&/||/!`, non-zero test) and emit only the chosen branch. Conditions must be elaboration-time constants (integer literals and genvars) — e.g. triangle structures `if (j <= i)`. A condition on a module parameter is a clear, honest error: parameters bind at *simulation* time under OSDI (model cards!), so they fundamentally cannot shape generated structure. The non-constant-bound message explains the same reasoning.

What now works (all verified numerically exact)

nested 2×3 loops (6 mS), genvar param-override expressions ($1k||2k||3k = 11/6$ mS), triangle generate if on genvars (6 mS), if/else (1.5 mS), generate case with a multi-value arm and default (2.75 mS), anonymous blocks with per-iteration nets (1 mS), $i = 2; i \leq 6; i = i + 2$ (3 mS) — plus the original E-5 ladder pinned bit-identical to its hand-written twin.

Examples (**generate_examples/**, 9 checks, ALL PASS)

The folder finally gets a verify script: `verify_generate.py` — [1] the E-5 ladder twin regression; [2] the seven Enhancement-67 feature checks above; [3] the honest parameter-condition error naming the OSDI reason.

Regression

All version11 example verify suites pass; crate tests pass; the VA_TEST corpus compiles 92/92.

Enhancement-68 — enabling openvaf's own integration test suite

This document describes Enhancement-68: bringing the fork's dormant integration test suite — 28 tests over real compact models (BSIM3/4/6, BSIMBULK, BSIMCMG/IMG/SOI, HiSIM family, MEXTRAM, PSP102/103, HICUM, EKV, ASMHMET, diode_cmc, plus the live `$limit/noise` numeric tests) — to life for the first time in this project. Test-infrastructure only: the shipped compiler binary is unchanged (every edit is in test harness code, test-only loaders, or snapshot data).

Why it never ran

1. The upstream vacask test legs need the **external/vacask** git submodule (codeberg.org/arpadbuermen/VACASK), which was never initialized — and the mini_harness panicked on the missing directory, taking the whole suite down ("reading test data must succeed: NotFound").
2. The remaining tests are gated behind `RUN_DEV_TESTS=1`.
3. Once those hurdles were cleared, the suite's own OSDI loader was frozen at ABI 0.4 ("invalid version v0.7") — this project's three ABI bumps (0.5: `0sdiNode.nodeset`, E-45; 0.6: the `ac_stim` descriptor tail, E-51; 0.7: the stride-2 signed-pair `load_noise` convention, E-54) had never been ported to the test-side structs.

The fixes (all test-side)

- **tests/load/osdi_0_4.rs**: synced to the authoritative OSDI 0.7 layout (mirroring ngspice's completed `osdi.h`): `0sdiNode.nodeset`, the new `0sdiAcStimSource` struct, and the descriptor tail (`num_ac_stim_src/ac_stim_sources/load_ac_stim`). **tests/load/mod.rs**: version gate 0.4 → 0.7 (the loader matches exactly that layout).
- **tests/mock_sim/mod.rs**: the noise buffer is stride-2 per source since OSDI 0.7 (`[flat, jw-routed]` signed pairs); `read_noise(i)` reads `dense[2i]`. The noise test's expected values were already correct — the test model's factors are all positive, so the flat densities are numerically identical under the new convention.
- **lib/mini_harness**: a missing test-data directory now skips its tests with a note instead of panicking the harness (general robustness; the suite is fully self-contained after the VACASK removal below).
- 13 snapshot files regenerated (`test_data/osdi/*.snap`) and reviewed line by line: every diff is a known feature of this project — `flow(<port>)` DAE unknowns (E-29 port-flow probes), `flow(a, b)` probe-only branches (E-36), implicit-equation nodes, and their Jacobian entries. No parameter metadata or physics changes.

The VACASK legs were removed outright

VACASK is AGPL-3.0, so its device library cannot be vendored here — and rather than carrying half-alive test legs that depend on cloning an external repository, the 34 upstream `vacask/vacask_spice/vacask_spice_sn` harness entries (and their 34 stale snapshots) were removed. The suite is now fully self-contained: the 26 in-tree `integration_tests/` models plus the two live numeric tests. (During development the VACASK legs were run once against a temporary clone — they passed — before removal.)

Bonus: the whole dormant dev-test surface is green

`RUN_DEV_TESTS=1` also un-ignores long-dormant harness tests in other crates — `parser` (41), `syntax` (40), `hir_lower` (30) — all passing, so the project's regression now runs them too.

Running

```
RUN_DEV_TESTS=1 cargo test --release -p openvaf \
    --features openvaf/llvm18 --test integration
```

28 passed; \$limit and noise are live numeric tests against the mock simulator, the rest are descriptor snapshots + operating-point sanity checks per model.

Regression

All example verify suites pass (the integration suite is now part of the local regression runner); crate tests pass with and without RUN_DEV_TESTS; the VA_TEST corpus compiles 92/92. The compiler binary is byte-unchanged by this enhancement.

Enhancement-69 — operating-point variables end-to-end: validation deliverable

This document records the Enhancement-69 audit of operating-point variable (opvar) access — Verilog-A module variables carrying a (`* desc="..." *`) attribute, exposed through the OSDI descriptor and read back from ngspice as `@instance[variable]`. No defects were found — the surface is fully working — so, like Enhancements 57, 60, and 66, the deliverable is the validation itself. No compiler or ngspice source changes.

What was probed (all exact)

- **.op** access (`print @n1[ids]`): real and integer opvars exact (1 mA / 1 mS / region flag 2); a module variable *without* a desc attribute is correctly not exposed — no such parameter, a clean error rather than a silent zero or crash.
- **.tran** per-point recording (`.save @n1[ids] @n1[region]`): 508-point vectors for the real *and the integer* opvar — the Enhancement-32 `outitf.c` IF_INTEGER fix finally gets a regression pin. **.meas MAX** hits the exact 1 mA sine peak; AVG of the region flag lands at 1.5 (above threshold exactly half the period).
- **.dc** sweeps: per-point recording; `ids == V/1k` exact at every point and the integer region flag steps exactly where $V > 0.5$.
- **.ac**: opvars record (op value per frequency point) and two instances of one model stay distinct (`@n1 = 1 mA` vs `@n2 = 0.5 mA` with $r=2k$).
- **.meas WHEN @n1[ids]=0.5m RISE=1**: 83.3334 ns against the analytic $\text{asin}(0.5)/2\pi \mu\text{s}$ — measurement logic works on opvar vectors exactly as on node voltages.
- String opvars: displayed by `show <inst>`; the *vector* path (`print @n1[strvar]`) is inherently numeric and fails with the `clear can not handle string value` message — pinned as the expected behavior (ngspice vectors cannot hold strings), not a defect.

Examples (**opvar_examples/**, 11 checks, ALL PASS)

`verify_opvar.py` covers the six paths above with exact expected values; `opvar_demo.va` (real + integer + hidden variables) and `opvar_str.va` (string opvar) are the two models.

Regression

No compiler/ngspice source changes; the Enhancement-68 regression state stands, plus this suite's 11 checks.

Enhancement-70 — behavioral-loop audit: precise loop diagnostics

This document describes Enhancement-70: a systematic audit of the runtime loop statements inside analog blocks (`for`, `while`, `do-while`, `repeat`) — the simulation-time cousins of Enhancement-67's `generate` audit. Front-end only — no OSDI/ngspice change.

The audit (14 probe forms, all values verified numerically exact)

feature	verdict
<code>for / while / do-while (E-19) / repeat(n)</code>	exact (sum 1..4 → 10 mS four ways)
nested loops	exact (3×4 → 12 mS)
parameter-dependent trip counts	exact — and a model-card override (n=25) changes the count at simulation time, the precise complement of E-67's <code>generate</code> restriction (structure binds at compile time, behavior at simulation time)
solution-dependent while condition (<code>while (acc < V(a, c)*5)</code>)	converges, exact
loops over arrays (E-14)	exact
iterative algorithms (Newton <code>sqrt(16)</code> , 20 iterations)	exactly 4
contributions inside loops	accumulate (3 iterations of <code>I <+ V·1m</code> → 3 mS)
loops inside analog functions	exact (iterative 5!)
<code>while (0)</code>	body never executes
<code>break</code>	correctly rejected (not Verilog-A)
analog operator (<code>ddt</code>) inside a loop body	correctly rejected per LRM 4.5.1 — but see below

The one defect: a misleading diagnostic (fixed)

An analog operator inside a loop body was rejected — correctly — but the message read *"analog operator 'ddt' is not allowed in conditions"*: the validator lumped loop bodies into the conditional context (`BodyCtx::Conditional`), pointing users at the wrong construct entirely.

Fix (**hir_ty/src/validation/body.rs + validation.rs**): loops now enter their own **BodyCtx::Loop** context (same restrictions, precise reporting) via a parametrized `validate_condition_in`. The error now reads:

```
error: analog operator 'ddt' is not allowed in loops
  = help: analog operators are not allowed inside looping statements (LRM 4.5.1)
  = help: hoist the operator out of the loop, or unroll the loop with `generate`
```

while the `if/case` message is unchanged.

Examples (**analogloop_examples/**, 12 checks, ALL PASS)

`verify_analogloop.py`: [1] nine exact-conductance checks covering every loop statement, nesting, arrays, Newton iteration, contributions-in-loops, and function-body loops; [2] the parameter-bound over-

ride pin (default 10, model-card 25); [3] the fixed diagnostic (names "loops", cites LRM 4.5.1, and no longer says "conditions").

Regression

All 64 example verify suites pass; the E-68 integration suite (now part of the regression runner) passes 28/28; crate tests (including the dev-gated sets) pass; the VA_TEST corpus compiles 92/92.

Enhancement-71 — display-task audit: format flags/width for every conversion + the %b segfault

This document describes Enhancement-71: a systematic audit of the display tasks (\$strobe, \$display, \$write, \$monitor, \$debug) and the full format-specifier surface. Front-end + OSDI codegen — no ngspice change.

The audit

Two defects found and fixed; everything else verified against exact printf output.

Defect 1: flags and width rejected for every non-real conversion %5d, %-8d, %+d, %08d, % d, %#o — all standard display syntax — failed to compile with *“failed to parse format specifier; unexpected character d”*. Only real conversions (%10.3e, %.2f) accepted a [flags] [width] [.precision] prefix: the inference-side parser (hir_ty/inference/fmt_parser.rs) only *terminated* on e/f/g/r, and the lowering-side translator (hir_lower/fmt.rs) had the same real-only assumption baked into its catch-all branch.

Fix, both layers: the parser now terminates on every conversion character and reports it (ParseResult : conversion), so inference types the argument from the conversion (d/h/o/b/c → integer, s → string, e/f/g/r → real) with flags/width/precision legal everywhere; the lowering translator collects the general prefix verbatim (consuming one extra integer argument per dynamic *) and re-emits it in the generated C format with the conversion translated (%h→%x, %b→%s over a pre-formatted binary string, %r→engineering-notation f%c pair).

Defect 2: %b crashed the simulator (pre-existing) The OSDI print codegen (osdi/src/compilation_unit.rs) called fmt_binary(val) to build the binary string and remembered the result for free() — but never passed it to **snprintf**. The matching %s consumed a garbage pointer: any model printing %b segfaulted ngspice. Latent since the print machinery was built (the in-tree users never exercised %b at runtime). One line: push the formatted string as an argument before remembering it for free.

Verified printf-exact (18 checks)

%5d → [42], %-5d → [42], %05d → [00042], %+d → [+42], %8s/%-8s, %4h → [ff], %8b → [101], dynamic %*d, %10.3e → [1.235e+03], %#o → [010], all base conversions (hex=ff HEX=FF oct=10 bin=101 chr=A), %, %m (module path), escape sequences, bare-argument defaults (%g/%d/%s inferred), and all five display kinds print (\$write joins lines; \$monitor/\$debug fire — their values reflect the evaluation they run in, per LRM).

Examples (display_examples/, 18 checks, ALL PASS)

verify_display.py + display_fmt.va (the format tour) + display_kinds.va (kinds, %m, escapes).

Regression

All 65 example verify suites pass; the integration suite 28/28; crate tests pass; the VA_TEST corpus compiles 92/92; zero-warning build maintained.

Enhancement-72 — Touchstone round 2: MA/DB formats, frequency units, Y/Z export, and the **rdsnp** reader

This document describes Enhancement-72, completing the Touchstone arc begun in Enhancement-64. ngspice-only — no compiler/OSDI change.

Writer options (**wrsnp** <file> [**ri**|**ma**|**db**] [**s**|**y**|**z**] [**hz**|**khz**|**mhz**|**ghz**])

The generalized writer (`spar_write_np`) now covers the full Touchstone v1 option surface, any combination, any order after the file name:

- formats: RI (real/imaginary, the default), MA (magnitude / angle in degrees), DB ($20 \cdot \log_{10}$ magnitude / angle);
- parameters: S (default), **Y** and **Z** — exported from the `sp` plot's $Y_{i,j}/Z_{i,j}$ vectors, normalized to Rbase ($Y \cdot R$, Z/R) as the Touchstone v1 spec requires;
- frequency units: Hz (default), kHz, MHz, GHz — the frequency column is scaled and the option line says so (`# GHz S DB R 50`).

The bare 2-port default (`wrs2p file / wrsnp file`) still goes through the byte-identical classic path; every optioned case (and every $N \neq 2$) uses the generalized writer, which also handles the Touchstone 2-port column order (S11 S21 S12 S22 on one line) and the $N \geq 3$ row-major / 4-pairs-per-line layout from E-64.

The reader: **rdsnp** <file> [**nports**]

Reads a Touchstone v1 file into a new plot (`Touchstone import <file>`) holding a real frequency scale in Hz plus complex $S_{i,j}$ (or $Y_{i,j}/Z_{i,j}$) vectors that match the `.sp` plot's conventions — so imported measurement data compares 1:1 against simulated vectors (`let err = maximum(mag(S_2_1 - {sp1}.s21sim))`).

- parses the `#` option line (any token order): frequency unit, parameter type, format, `R n` — missing option line assumes `Hz S RI R 50` with a warning;
- converts MA/DB back to real/imaginary and de-normalizes Y/Z back to absolute values;
- handles the 2-port column order and the general row-major layout;
- port count from the `.sNp` extension, or explicitly (`rdsnp file 3`);
- publishes Rbase in the imported plot so it round-trips back through `wrsnp`;
- a number count that doesn't divide into frequency blocks is a clear error naming the suspected wrong port count.

Verified (touchstone_examples grows to 17 checks, ALL PASS)

The five E-64 sections are unchanged; round 2 adds: [6] option lines and math for MA (= polar of the RI values), DB (= $20 \cdot \log_{10}$ magnitude) with GHz scaling, and Y/Z headers; [7] a full write-MA $\rightarrow$ **rdsnp** $\rightarrow$ compare round-trip — $\max |S_{21}(\text{read}) - S_{21}(\text{simulated})| = 4e-8$, below the file's own 6-digit precision; [8] a hand-written measurement-style MA/MHz file read back with exact values ($0.5 \angle -90^\circ \rightarrow -0.5j$) through the 2-port column order and unit scaling.

Regression

All 65 example verify suites pass with the rebuilt ngspice; the openvaf integration suite 28/28; no compiler change.

Enhancement-73 — the user handbook: 72 enhancements consolidated into **docs/handbook/**

This document describes Enhancement-73: a documentation deliverable — the repository's first consolidated user guide. No compiler or ngspice source changes.

Why

After 72 enhancements the top-level README had grown to ~950 lines of chronological, per-enhancement narrative — the story of the project, but not a usable reference: to learn whether arrays work, a user had to know that Enhancements 14, 15, 18, 20, 23, 33, 43 exist and read all seven. The handbook reorganizes the same verified material by what the user wants to do.

The handbook (**docs/handbook/**, six files)

- README.md — orientation: the chapter map, how features are verified (every claim is pinned by a committed verify script), the reference PDFs.
- 01-getting-started.md — prebuilt binaries; building from source (LLVM 18 pin, the ngspice configure line); a complete first-model walkthrough (`myres.va` → `openvaf-r` → `pre_osdi deck` → `v(mid)`); the example-suite and `_setup.py` conventions; `VA_TEST` and the integration suite.
- 02-verilog-a-language.md — the feature matrix, 13 sections organized by LRM area (hierarchy/elaboration, nets/disciplines/natures, types/arrays, parameters, operators/literals, analog operators, contributions/probes, events, analog functions, statements/loops, system tasks, preprocessor, attributes). Every row links its enhancement doc and names the example folder that proves it.
- 03-ngspice-workflows.md — simulator-side recipes: model cards vs instance lines, `opvar` access, `alter/.dc @inst[param]` sweeps, the analysis-coverage table, S-parameters + the full Touchstone import/export surface, both Monte Carlo idioms, debugging aids.
- 04-limitations-and-gotchas.md — the honest map: Verilog-A (Annex C) vs AMS scope, fallback-semantics builtins, the compile-time/simulation-time binding model, documented design decisions (stateless `limexp`, non-advancing RNG seeds, `.disto`), ngspice control-language traps, and assorted pinned edges.
- 05-enhancement-index.md — E-1...E-73 in one table: one line each, linking the `enhancements_doc/` write-up and the example folder.

The PDF edition (**docs/Ngspice-OpenVAF-Handbook.pdf**)

The handbook and the complete text of all 73 enhancement write-ups are also compiled into a single 202-page PDF with a linked table of contents: Part I = the five handbook chapters, Part II = `enhancements_doc/ Enhancement-1 ... 73` in order, each starting on a fresh page. The committed generator (`docs/handbook/build_pdf.py`, `pandoc + xelatex`) demotes headings under the two Part headings, rewrites cross-document links into internal PDF anchors (links to example folders/README sections become absolute GitHub URLs so they stay clickable), strips the historical "(versionN)" workflow tags from the enhancement titles, assigns proportional column widths to the wide feature-matrix tables via a `pandoc` Lua filter (the `gfm` reader emits none, which overflows the page), and maps the handful of math glyphs missing from STIX Two Text ($\rightarrow \geq \approx \equiv \angle \sqrt{\dots}$) onto math-mode equivalents — the final build has zero missing-glyph warnings and zero undefined link references. Building the PDF also surfaced (and fixed) a stale intra-doc anchor in `enhancements_doc/Enhancement-2.md` (§6 pointing at a heading numbered 5 — broken on GitHub too).

The README consolidation

With the handbook in place, the top-level README's ~850 lines of per-enhancement narrative sections (E-1 ... E-72) were replaced by the handbook's one-line index table (Doc + Examples links, re-anchored to the repo root) plus a prominent pointer to the handbook and its PDF edition — the README drops from 965 to ~210 lines and now reads: header / Precursors / the enhancements index / `VA_TEST` /

Prebuilt Binaries / CI. The detailed narratives remain where they always lived in full, enhancements_ doc/, and the result plots remain in their example folders.

Verified

- Link integrity: a checker walks every markdown link and #anchor in the six files (GitHub slug rules, code spans excluded) — 205 links, 0 failures — and every examples/<name> mention in code spans is a real committed folder.
- Link integrity, README: the same checker logic over the rewritten README — 110 relative links, 0 failures.
- Command truth: the getting-started model and deck were run verbatim with the repo binaries (`v ( mid) = 1.000000e+00` exactly as printed in the guide); the chapter-3 command spellings (`alter @n1[r], .dc @n1[r] ... V1 ...`, `agauss/sgauss/setseed`, `portnum 1 z0 50`, `wrsnp/rdsnp`) were cross-checked against the committed tutorial decks.
- One matrix claim corrected by testing: `casex` — written from memory as supported — was probe-compiled and is not implemented; the matrix and the limitations chapter now say so explicitly (plain case, including over strings and arrays, is supported). The project's own audit lesson — *verify, don't assume* — applied to its documentation.

Regression

No compiler or ngspice source changes; the Enhancement-72 regression state stands (66/66 example suites, integration 28/28, corpus 92/92).

Enhancement-74 — performance benchmark: OSDI vs built-in twins + flagship compile times

This document describes Enhancement-74: a tracked performance baseline for the toolchain — the first quantitative answer to “how fast is compiled Verilog-A?” in this project. Like Enhancements 57/60/66/69, the deliverable is a permanent, rerunnable measurement suite; no compiler or ngspice source changes.

The twin methodology

Timing two different circuits proves nothing, so every simulation benchmark is a twin pair: the same equations once as ngspice built-in devices and once as compiled Verilog-A, with matching model cards — ngspice then does the same numerical work (same Newton iterations, same timestep trajectory) and the waveforms must agree, which every run checks alongside the timing:

- RC ladder (`rcseg.va` vs built-in `r+c`, 200 segments, 100 306 timepoints): waveforms match to 8.9×10^{-32} V — the integrator followed the *identical* trajectory (same point count), so the ratio is pure per-evaluation OSDI overhead;
- rectifier bank (`vadiode.va` mirroring the built-in diode's DC core + junction `gmin` + depletion/diffusion charge, 50 cells): agreement 2.8×10^{-5} V through a strongly nonlinear transient;
- BSIM4 stage (VA-Models `bsim4.va` 4.8 vs the hand-coded C `level=14 version=4.8`, default cards): different codebases — a *correspondence* pin (op drain currents within 3.9%) plus a flagship compact-model throughput comparison.

Reference numbers (Apple Silicon; RESULTS.md has the full tables)

- Twin throughput: RC ladder ratio 0.99 — compiled Verilog-A through OSDI matches hand-coded C outright on identical physics; rectifier 1.26×; BSIM4 stage 1.26× (~185 000 timepoints/s).
- Scaling (RC ladder $N = 10 \dots 500$): the OSDI/built-in ratio stays 1.00–1.07 across a 50× size range — the overhead does not grow with circuit size.
- Compile time (median of 3): all 12 flagship industry models — BSIM4/6/BULK/CMG/SOI, PSP 102/103, HiCUM L2, MEXTRAM 505, EKV3, ASM-HEMT, `diode_cmc` — compile in 0.4–3.8 s each, ~21 s total (lines counted through their `include` bodies).

A benchmarking trap worth recording

The first $N=500$ OSDI ladder run died on ngspice's protective plot-memory guard (“memory required is more than memory available”): batch-mode `tran` stores every node vector by default, and ~500 vectors × 25k points tripped the estimate against momentary free RAM. The fix is benchmarking hygiene that also purifies the measurement: `save v(<probe>)` before the `tran` — output storage drops from N vectors to one, and the RC-ladder ratio improved from 1.08 to 0.99 (the “overhead” had been partly plot storage, not device evaluation). The guard's message printing literal “(Id Bytes)” (a `%Id` format specifier that macOS `printf` doesn't know) is noted as a stock-ngspice cosmetic quirk.

A second trap: provenance in the committed artifacts

The `verify` script originally passed absolute HERE-joined paths to `openvaf-r`, which embeds the source path (LLVM `comp_dir`) into the `.osdi` — silently re-polluting the committed artifacts every `verify` run. Worse, the standing fold check missed it: macOS **strings** without `-a` skips the section carrying the path (a raw `grep -a` on the binary catches it). Fixed by having `compile_va` relativize every path against the example dir; the fold check is upgraded to raw-byte `grep`.

Examples (`benchmark_examples/`, 8 checks, ALL PASS)

`run_benchmark.py` (the full benchmark: compile times, twin trio, scaling sweep → RESULTS.md, results.json, plots/*.png), `bench_common.py` (deck generators + timing harness), `plot_`

`benchmark.py`, `verify_benchmark.py`. The verify checks pin only what is deterministic or generously bounded, so the suite never flakes on a slow machine: the twin waveform matches ($< 10^{-9}$ / $< 10^{-3}$ / $< 10\%$), the artifacts' presence, and catastrophic-regression limits (OSDI/built-in $< 25\times$, flagship compile < 60 s). The compile section skips gracefully without the `VA_TEST/` corpus (the physcheck precedent).

Regression

No compiler or ngspice source changes; all 67 example verify suites pass (this suite included), the integration suite 28/28, and the `VA_TEST` corpus compiles 92/92.

Enhancement-75 — dynamic physics validation: the reactive paths cross-checked

This document describes Enhancement-75: the dynamic companion of Enhancement-57's physics suite. Where physcheck validated the static laws of compiled industry models (DC curves, autodiff Jacobians, noise), this suite validates the charges — the reactive paths of the toolchain (ddt () lowering, the reactive autodiff Jacobian, the $j\omega$ AC stamping, and the transient integrator) against physics that must hold *across analyses*. Like E-57, the deliverable is the guard itself: no toolchain defects were found, and no compiler or ngspice sources changed.

The four laws (9 checks, ALL PASS)

[1] One charge model, two code paths. PSP103's gate capacitance from AC ($\text{Im}(i_g)/\omega$ — the $j\omega$ reactive Jacobian) equals the capacitance from a slow transient ramp ($i_g/(dV_g/dt)$ — the integrator on the same charge model) at five biases across the accumulation-to-inversion transition: worst relative difference 6.3×10^{-4} over a $4\times$ capacitance swing ($305 \rightarrow 1235$ fF), plus the physically-required monotone rise.

[2] Charge conservation. Over a closed gate-bias loop ($0 \rightarrow 1.2 \rightarrow 0$ V triangle) the net gate charge integrates to 7.8×10^{-5} of the one-way charge: PSP103's charge model is conservative, and the integrator preserves that property through 12 000 accepted timepoints.

[3] Junction charge extraction. The transient integral of diode_cmc's current over a slow reverse ramp, with the static $I(V)$ leakage subtracted point-wise (the physical charge-extraction technique), equals $\int C(V)dV$ of the AC-measured capacitance over the same interval to 0.5%; $C(V)$ decreases monotonically under reverse bias. The subtraction is the instructive part: the raw integral is $200\times$ too large — reverse DC leakage ($\sim$ nA) dominates the ~ 20 pA displacement current over a $200 \mu\text{s}$ ramp. A dynamic check that forgets the static component measures the wrong physics.

[4] Linear response. A PSP103 common-source stage ($1 \text{ k}\Omega$ load) driven with a 1 mV transient sine reproduces the .ac prediction — obtained from the same operating point — to $1.5 \times 10^{-7}/1.7 \times 10^{-6}$ in magnitude and $0.000^\circ/0.001^\circ$ in phase at 1 MHz and 10 MHz (quadrature demodulation of the steady state, four whole cycles after ten settling cycles): the reactive matrix built by the transient integrator is the same matrix the AC analysis factorizes.

Why these four

Together they close the loop E-57 opened: static Jacobian $\equiv$ DC curves (E-57 [4]) and now reactive Jacobian $\equiv$ transient charge $\equiv$ AC imaginary part, on flagship corpus models rather than toy circuits. A regression anywhere in the ddt ()/charge pipeline — lowering, the reactive dimension of autodiff, OSDI's load_jacobian_react/load_spice_rhs, the integrator interface — lands in one of these numbers.

Examples (dynphys_examples/, 9 checks, ALL PASS)

verify_dynphys.py (corpus-based, skips gracefully without VA_TEST/; compiles with relative paths per the E-74 provenance rule) + plot_dynphys.py rendering the committed plots: the $C_{gg}(V)$ transition curve (transient-continuous, AC points on it), the antisymmetric closed-loop gate current, and the steady-state sine segment.

Regression

No compiler or ngspice source changes; all 68 example verify suites pass (this suite included), the integration suite 28/28, the VA_TEST corpus compiles 92/92.

Enhancement-76 — multi-module **.osdi** libraries: audit + three registration fixes

This document describes Enhancement-76: an audit of the multi-module packaging surface — one **.va** file holding several modules, compiled into one **.osdi**, loaded by one **pre_osdi**. The machinery is plural by design on both sides (**openvaf-r** exports an **OSDI_DESCRIPTOR**s array with a count; **ngspice**'s registry iterates it), and the audit confirmed the happy path end-to-end — then found three registration defects, all fixed on the **ngspice** side (**dev.c** + one stock parser fix in **inpgmod.c**). No compiler changes.

Working by construction (probed, exact values)

- Any number of modules per library: three device types from one **pre_osdi**, simulating side by side (currents sum exactly);
- hierarchy inside a multi-module file: a module that instantiates another — the flattened parent (top) and the standalone child (leaf) coexist as independent device types with independent parameters;
- **paramset** blocks mixed with plain modules (the E-21 twin-module machinery composes with real multi-module files);
- case-insensitive model-card types (**.model m RES_A** finds **res_a**);
- a duplicated module name *within one file* is a clean compiler error ("already declared in this scope").

The three fixes

(1) Silent cross-library shadowing. **osdi_add_device()** appended every descriptor to the device table unconditionally; the model-card lookup scans front-to-back, so a module name duplicated across two loaded libraries silently resolved to the first registration — loading an updated model library gave the stale device with no hint. Now each registration checks the existing table (case-insensitively, matching the lookup) and a duplicate is skipped with a warning naming the device: **Warning(osdi): device "dup" is already registered**; keeping the existing device and ignoring this one. First-wins stays the semantics — but loud.

(2) Silent double-load. **load_osdi()** now keeps the list of loaded paths; **pre_osdi** of an already-loaded file prints **Note(osdi): "<path>" is already loaded**; skipping instead of re-registering (which, after fix 1, would have produced one warning per module).

(3) A stock **ngspice** segfault, found through the OSDI door. The Enhancement-29 gotcha — "a module named like a built-in (**vcvs**, **cccs**) segfaults **create_model**" — turned out to be a stock defect reachable with no OSDI at all: **find_model_parameter()** dereferences ***(device->numModelParms)**, which is **NULL** for every built-in that takes no model cards (**VCVS**, **CCCS**, **CCVS**, ...). Any referenced **.model m vcvs()** card crashed — e.g. an ordinary MOS instance **m1 a b c d mm** with that card. One **NULL** guard fixes the class; both the OSDI shape (now: duplicate warning + the **inp2n** "Expected OSDI device" located error) and the pure stock shape (located "model type mismatch" error) terminate cleanly. The E-29 gotcha is retired.

A debugging note for the record: the crash was initially masked in the probe pipeline by **ngspice ... | grep | head** — **head**'s exit status hides the **SIGSEGV**. Same family as E-74's **strings-without--a** lesson: exit codes must be read from the process itself.

Examples (**multimod_examples/**, 13 checks, ALL PASS)

verify_multimod.py + five multi-module fixtures (**trio.va**, **hier.va**, **psmix.va**, **dup1.va/dup2.va**, **vcvs.va**) covering the five by-construction behaviors and all three fixes, including the two crash-regression pins.

Regression

All 69 example verify suites pass with the rebuilt ngspice; the integration suite 28/28; the VA_TEST corpus compiles 92/92 (compiler unchanged).

Enhancement-77 — ngspice zero-warning build: 33 → 0

This document describes Enhancement-77: the ngspice counterpart of the openvaf-r warning cleanup released with Enhancement-66 (44 → 0). A full clean rebuild of ngspice-46 on macOS/clang emitted 33 warnings; all are fixed at the source (five tracked-file changes) or traced to stale generated files and eliminated. The final clean rebuild is warning-free, and behavior is unchanged (full regression green). No compiler changes.

The inventory and the fixes

- (1) **'ALIGN' macro redefined** × ~8 — **src/osdi/osdidefs.h**. The OSDI glue's ALIGN(pow) collides with the macOS SDK's unrelated ALIGN from <sys/param.h> in every OSDI translation unit that pulls both. Renamed to OSDI_ALIGN (definition + the one use on OsdIExtraInstData).
- (2) **'NODEV' macro redefined** — **src/frontend/display.c**. Same collision class: the display driver's local NODEV vs the SDK's device number NODEV. One #undef NODEV above the local definition.
- (3) **-Wformat** × 4 — **src/frontend/outitf.c**. The plot-memory guard printed sizes with %Id — a Windows-only length modifier that is undefined behavior elsewhere, and the reason Enhancement-74/76 saw the garbled "memory required (Id Bytes)" message with no numbers in it. Replaced with the portable %zu (the arguments are size_t), so the guard now reports actual byte counts on every platform.
- (4) **-Wimplicit-const-int-float-conversion** × 3 — **src/maths/sparse/spfactor.c**. The Markowitz-product overflow guards compare a double against LARGEST_LONG_INTEGER (= LONG_MAX), which is not exactly representable as a double; the implicit conversion rounds it to 2⁶³. Made the conversion explicit ((double)LARGEST_LONG_INTEGER) — identical semantics, intentional and visible.
- (5) **ld: -undefined suppress is deprecated** × 14. Two independent sources, one real:
 - **src/xspice/icm/makedefs.in** (tracked — the real fix): the XSPIICE codemodel (.cm) link rule hardcodes the pre-macOS-10.3 idiom -bundle -flat_namespace -undefined suppress, which the modern linker deprecates loudly on every one of the codemodel links — including in the CI macOS builds. Replaced with -bundle -undefined dynamic_lookup, the modern spelling for plugins whose host symbols resolve at load time (the codemodels' exact situation).
 - Stale generated **configure** (local-only, nothing to commit): the local build tree's configure predated libtool 2.4.7's fix for the MACOSX_DEPLOYMENT_TARGET default ({VAR-10.0} matched the 10.[012] → suppress branch when the variable is simply unset). Regenerated with ./autogen.sh (libtool 2.5.4), whose logic falls through to dynamic_lookup; CI already regenerates on every build and never had this half.

Verification

- Full make clean rebuild: 0 warnings (was 33), build exit 0.
- The E-74/E-76 plot-memory message now prints real numbers (memory required (NNN Bytes)).
- Full regression: all 69 example verify suites pass with the zero-warning ngspice; the integration suite 28/28; the VA_TEST corpus compiles 92/92 (compiler untouched).

Scope note

This pass covers everything clang emits on the macOS build — the platform this project develops on and one of the five CI targets. If the Linux/gcc or Windows/MinGW CI logs show additional platform-specific warnings, they are a follow-up round in the same mold (the E-66 openvaf cleanup and this one give the recipe).

Enhancement-78 — **casex** / **casez**: the don't-care case statements

This document describes Enhancement-78: the casex/casez case-statement variants — the one “not implemented” row the Enhancement-73 handbook audit left on the language matrix, now flipped to supported. Front-end only — no OSDI/ngspice change.

Semantics in the 2-state analog world

Verilog-A values are 2-state, so the don't-cares live in the literals: x/X, z/Z and ? digits of a based literal written directly as a casex/casez item form a comparison mask — the arm matches when the discriminant equals the item on every *care* bit:

```
casex (sel)
    4'b1xxx: grant = 8;    // matches 8..15: bits 2..0 are don't-cares
    4'b01xx: grant = 4;    // the classic priority-encoder idiom
    ...
endcase
```

casex treats x, z and ? as don't-cares; casez only z/? (per the LRM). Don't-care digits are legal in the power-of-two bases ('b, 'o, 'h), each masking its digit's bit positions.

Implementation (the E-19/E-34 pipeline-threading recipe)

- lexer: the based-literal digit runs admit x/X/z/Z/? in binary, octal and hex (not decimal), for both the sized and unsized forms;
- tokens/parser: new CASEX_KW/CASEZ_KW keywords; all three keywords share the one CASE_STMT grammar rule — the keyword token on the node carries the flavor;
- syntax: parse_based_int_masked decodes a based literal to (value, x_mask, z_mask) (don't-care digits contribute zero value bits); the old parse_based_int delegates to it;
- hir_def: Stmt::Case gains a CaseKind and each arm a parallel CaseMask { care, had_x } list, computed at collection from item literals; every don't-care literal *not* consumed as a casex/casez item lands in a stray_dontcare_literals list on the body;
- hir_ty validation: three new diagnostics (below);
- hir_lower: an item with a partial mask compares (discr & care) == (item & care) — two iands feeding the existing Ieq; full-mask items take the unchanged equality path, as do plain case, arrays, strings, and reals.

The three diagnostics

- “don't-care digits are only allowed in casex/casez items” — a 'b1x0 spelled anywhere else (an assignment, a plain-case item, a nested expression) is rejected instead of silently reading its x-bits as zeros;
- “'x' digits are not don't-cares in casez” — with the casex hint;
- “casex/casez requires an integer discriminant” — bit masks are meaningless on real/string discriminants.

Verified (**casexz_examples**/, 7 checks, ALL PASS)

[1] an 8-way self-checking bitmask module (the E-37 audit technique) pins the semantics at exactly 63/63: casex x/z/? masking, casez z/?-only, fully-specified mismatch falling to default, first-match-wins arm order, and plain case unchanged; [2] a casex priority encoder whose 4-bit request word comes from the model card — the highest set bit wins at all nine probed values including the default arm; [3] all three diagnostics as clean, located compile errors.

Regression

Zero-warning build maintained; all 70 example verify suites pass, the integration suite 28/28, the VA_TEST corpus compiles 92/92.

Enhancement-79 — benchmark round 2: ring oscillator, small-signal throughput, KLU vs SPARSE

This document describes Enhancement-79, extending Enhancement-74's performance baseline with three sections — multi-device nonlinear circuits, the small-signal analyses, and the solver comparison. Like round 1, everything is a twin (same physics, built-in vs compiled Verilog-A) with correctness pinned alongside every timing. No compiler or ngspice source changes.

[D] 9-stage BSIM4 ring oscillator

The missing regime in round 1: evaluation-dominated, multi-device, nonlinear. 18 BSIM4 devices (9 CMOS inverters, `type=1/type=-1` VA cards vs built-in `level=14` with matching W/L), kick-started with `uic`, 200 ns transient:

- both implementations oscillate, at 1.077 GHz (built-in) vs 1.065 GHz (OSDI) — a 1.1% correspondence between two independent codebases of BSIM4 4.8, pinned as a verify check;
- wall-time ratio 1.91× — the honest number this section exists for: with 18 compact-model evaluations per Newton iteration and a tiny matrix, per-instance call overhead compounds beyond the single-device 1.26×. This is the workload where OSDI overhead is most visible.

[E] `.ac` and `.noise` throughput

N=200 ladders: the RC twin from round 1 for `.ac` (300 pts/dec over nine decades), and a new `nres.va` — a noisy resistor whose $4kT/R$ spectrum is identical to the built-in resistor's (the E-57 identity) — for `.noise`:

- `.ac` ratio 0.91×, output identical to the last bit (max diff 0.0);
- `.noise` ratio 1.19×, spectra matching to 7.6×10^{-15} over 601 points — the adjoint solves do the same work on both sides;
- both runs are sub-0.1 s: the small-signal analyses are factorization- dominated, so OSDI evaluation overhead barely registers.

[F] KLU vs SPARSE 1.3

`.options klu` after the title line (the E-1 recipe), both device kinds, on the N=500 ladder and the ring oscillator — and an honest wash: speedups 0.91–1.00×. These matrices (a tridiagonal ladder, an 18-device ring) are too small and too regular for KLU's AMD/BTF ordering to pay for itself; the Enhancement-1 mesh benchmark remains the reference for where KLU wins (large, denser matrices). The valuable pin that stays: KLU and SPARSE waveforms agree to $\sim 10^{-15}$ — solver independence as a regression check, run whenever the binary has KLU and skipped gracefully otherwise (the section auto-detects; local and CI binaries both carry KLU).

Examples (**benchmark_examples/**, grows to 12 checks, ALL PASS)

`run_benchmark.py` gains sections [D]/[E]/[F] (`ro_deck`, `ac_ladder_deck`, `noise_ladder_deck`, `with_klu`, `osc_freq` in `bench_common.py`; `nres.va` new); `RESULTS.md`, `results.json` and the plots regenerated (new `klu_vs_sparse.png`; the throughput bars now carry all six benches). The three new verify checks: ring oscillators oscillate with corresponding frequencies ($< 10\%$), the noise-spectrum identity ($< 10^{-4}$), and $KLU \equiv SPARSE$ waveforms ($< 10^{-6}$).

Regression

No compiler or ngspice source changes; all 70 example verify suites pass (this suite's 12 included), the integration suite 28/28.

Enhancement-80 — temperature physics validation + the **dtemp** alias

This document describes Enhancement-80: physcheck round 3, the thermal axis. E-57 validated the static laws of compiled industry models, E-75 the charges; this suite validates temperature — the \$temperature/\$vt plumbing, the OSDI instance temperature offset, and the classic junction/MOS thermal laws, quantitatively. The audit found one ngspice defect (fixed): OSDI instances rejected the conventional dtemp instance parameter.

The fix: **dtemp** on OSDI instance lines

The OSDI glue has full instance-temperature-offset plumbing (OsdIExtraInstData.dt, honored by the temperature update), but the synthetic instance parameter was registered under the name **dt** — while every built-in ngspice device (R, C, diode, MOS, ...) spells it **dtemp**. n1 a b mod dtemp=10 failed with *"unknown parameter (dtemp)"*. One alias IFparm entry in osdiinit.c registers both spellings; the identity is pinned to 12 digits: dtemp=10 at temp=17 $\equiv$ a plain instance at temp=27 (and the historic dt still works).

The laws (**tempphys_examples/**, 11 checks, ALL PASS)

1. **\$vt** tracks kT/q at all 21 points of a .dc temp -50..150 sweep — worst 1.0×10^{-7} , the documented E-59 constants-vintage residual.
2. The **dtemp** identity (above) — pinning the OsdIExtraInstData plumbing end-to-end through the temperature update.
3. Thermal noise $\propto T$: the nres $4kT/R$ twin's output noise power ratio between 127 °C and 27 °C equals T_2/T_1 to 3.5×10^{-14} , and the OSDI $\equiv$ built-in identity holds at the hot temperature (3×10^{-7}). A units trap worth recording: ngspice noise spectra are amplitude densities ($V/\sqrt{\text{Hz}}$) — square before comparing against power laws.
4. MEXTRAM 505 junction laws: dV_{BE}/dT at 1 mA lands at $-1.3 \dots -1.5$ mV/K (the textbook window) across $-25 \dots 125$ °C, and the Arrhenius activation energy extracted from $I_C(T)$ at fixed V_{BE} is pair-consistent to 0.9% with an E_g estimate of 1.25 eV — silicon, recovered through the whole toolchain from a compiled compact model.
5. PSP103's zero-temperature-coefficient point: $dI_d/dT > 0$ in weak inversion (the V_{th} drop wins) and < 0 in strong inversion (mobility wins) — the sign flip that makes ZTC biasing possible.
6. The CMC default-off idiom, thermally pinned: diode_cmc's *default* card shows $|dV_f/dT| < 0.5$ mV/K and a near-zero activation energy — corpus defaults are placeholders, not silicon (the E-56 lesson; a naive "check the diode tempco at defaults" would chase a phantom defect).

plot_tempphys.py renders the three-panel figure: the kT/q line, the MEXTRAM $V_{BE}(T)$ slope (at 100 μA — the default card's tiny I_S puts 1 mA into high injection where the cold-temperature op wobbles), and the PSP103 ZTC crossover.

Regression

ngspice rebuilt warning-free with the one-line alias; all 71 example verify suites pass, the integration suite 28/28, the VA_TEST corpus compiles 92/92 (compiler unchanged).

Enhancement-81 — session-lifecycle + memory audit: two resource fixes

This document describes Enhancement-81: an audit of interactive ngspice workflows with OSDI devices — re-sourcing, circuit removal, long reset/alter loops, plot management — probed for correctness and bounded memory, plus two quality-of-life fixes in the resource machinery that the user's "approaching max data size" question motivated.

The audit (all healthy)

- Re-sourcing the same deck is idempotent (identical currents), and `remcirc` + a new deck resolves cleanly to the new circuit — the OSDI registry (E-76's loud, deduplicated version) composes with circuit reloads.
- The E-66 Monte-Carlo **reset** idiom is leak-free in practice: 100 `reset+op` iterations grow the ngspice program size by ~6 kB per iteration (pinned < 20 kB) with the solution exact throughout — the OSDI setup/teardown cycle does not accumulate instance state.
- Plot accumulation behaves as documented (one plot per analysis — the E-66 trap for big loops) and `destroy all` genuinely frees the memory: the plot numbering restarts at `tran1`.

Fix 1: the "approaching max data size" warning (**resource.c**)

`ft_ckptspace()` warns when the process RSS exceeds 95% of (RSS + free RAM). Two behaviors improved:

- it now honors **set no_mem_check** — the same opt-out the fatal output-size estimate in `out.tif.c` already respects, so one variable governs both memory guards;
- it warns once per excursion above the threshold (re-arming only after usage drops clearly below 90%) instead of repeating on every check through a long analysis.

The warning itself requires ~95% RAM pressure and is deliberately not triggered by the bounded verify suite; the opt-out path is exercised and the latch logic is single-page code review.

Fix 2: the **pre_osdi** reload note gets instructions (**dev.c**)

E-76's already-loaded note now reads: *"already loaded; skipping (restart ngspice to load a recompiled file)"*. The suite pins the behavior it warns about: overwriting a loaded `.osdi` and re-issuing `pre_osdi` on the same path keeps the old model for the rest of the session — the same effective behavior as pre-E-76 (where the duplicate registration was silently shadowed by the first), but now with the remedy stated.

Examples (**lifecycle_examples/**, 9 checks, ALL PASS)

`verify_lifecycle.py` (self-contained decks) + `lres.va`. Checks: the re-source identity, `remcirc`/new-deck resolution, the 100-reset exactness and memory bound, plot accumulation + `destroy all`, the reload note's hint + the pinned stale-model behavior, and normal simulation under `no_mem_check`.

Regression

ngspice rebuilt warning-free (`dev.c` + `resource.c`); all 72 example verify suites pass, the integration suite 28/28, corpus 92/92 (compiler unchanged).

Enhancement-82 — provenance and compliance documentation

This document describes Enhancement-82: a documentation round in the Enhancement-73 mold, adding the project's provenance and compliance references. No compiler or ngspice source changes.

The change reports (**docs/change_log/**)

Two full change reports document every modification this project applied to each tool, with its reason — the complete provenance from the pristine original/ snapshots to the current trees:

- **ngspice_changes_full-report.md** — organized by subsystem: the OSDI ABI history (0.4 → 0.7 with why each bump happened), the OSDI runtime file by file (including the new `osdiaccept.c`), the analyses, the frontend (the Touchstone subsystem, opvar recording, the memory warnings), parser/registry fixes, maths, and the build system. 34 substantive files; the ~100 regenerated `Makefile.ins` documented as a class. Closes with the per-enhancement index (21 enhancements touched ngspice).
- **openvaf_changes_full-report.md** — organized by pipeline stage: lexer → tokens/grammar (with the sourcegen templates) → preprocessor → syntax/AST → the elaboration pass → `hir_def` → `hir_ty` → `hir_lower` → the MIR core and optimizations → `sim_back` → OSDI codegen and its C stdlib → driver/libraries → the test-suite enablement. ~120 files; `test_data/` and the integration `.osdi` objects documented as reviewed classes. Closes with the index of ~55 compiler-touching enhancements.

Both are coverage-verified by script — every substantive changed file from the tree diffs must be named in its report — and ship PDF editions regenerated by the committed `docs/change_log/build_pdfs.py`.

The LRM compliance document (**docs/compliance/**)

OpenVAF_Verilog-A_LRM_Compliance.md — a clause-by-clause account of Verilog-A coverage against the Verilog-AMS LRM 2023 (Annex C): lexical conventions, data types, the expression system, analog behavior, hierarchy, system tasks, and compiler directives, with status marks (✓ full / △ documented deviation / × out-of-scope AMS), Verilog-A code examples throughout, and a closing summary table mapping every LRM area to the verify suite that pins it. The documented deviations are stated honestly with their rationale: stateless `limexp`, range-exempt parameter defaults (the CMC convention), deterministic RNG semantics, compile-time-bound generate structure, and the mechanism-unavailable fallback group. PDF edition via the committed `docs/compliance/build_pdf.py`.

A discipline note that earned its keep: every code snippet was cross-checked against the committed example suites' exact spelling — five snippets written from memory did not match verified reality and were replaced before committing.

Maintenance rules (standing)

- Any future fold that touches ngspice or compiler sources updates the matching change report (subsystem entry + index row) and regenerates its PDF in the same commit.
- The compliance document is updated when language coverage changes.

Verification

Scripted checks gate all three documents: file-coverage cross-checks against the tree diffs, relative-link resolution (including through the `docs/change_log/` move, which re-anchored 310+ links), summary-table suite names verified to exist, and warning-free PDF generation. No toolchain changes — the Enhancement-81 regression state (72/72 suites, integration 28/28, corpus 92/92) stands.

Enhancement-83 — the transistor-level μ A741: a complete model-to-circuit demo

This document describes Enhancement-83: the classic op-amp built entirely from a Verilog-A BJT and characterized across DC, AC, and transient — a complete “write a compact model, build a real circuit, measure datasheet figures” workflow demonstration. No compiler or ngspice source changes; the deliverable is the example (E-62 tutorial precedent).

The model (**bjt741.va**)

A compact Gummel-Poon-flavored BJT in ~70 lines of Verilog-A: Ebers-Moll transport with forward Early modulation, β -based ideal base currents, standard depletion charges (via an analog function with the fc linearization) plus diffusion charges on both junctions, `limexp` and the simulator’s `gmin` for convergence robustness. One module serves NPN and PNP through polarity reflection (`type = ±1`, the corpus convention). Pinned at a Gummel point: $i_B = i_C, \text{fwd}/\beta$ to 4×10^{-7} relative, and the PNP is the *exact* polarity mirror of the NPN.

The circuit (**ua741.subckt**)

The textbook Fairchild topology, 20 transistors: Q1–Q4 input (npn emitter followers into pnp common-base) with the Q5–Q7 active-load mirror, Q8/Q9 common-mode bias mirror, Q10/Q11 Widlar source ($R4 = 5 \text{ k}\Omega \rightarrow$ the famous $\sim 19 \mu\text{A}$ tail), Q12/Q13 master bias, the Q16/Q17 Miller second stage with $C_c = 30 \text{ pF}$, and the class-AB Q14/Q20 output behind a $2 \times V_{BE}$ stack. Protection devices omitted (behavioral demo, not production silicon). The operating point converged first try with textbook internal bias voltages.

The measurements (11 checks, ALL PASS)

Figure	This model	Real 741 (typ.)
Open-loop DC gain	104.4 dB	$\sim 106 \text{ dB}$
Dominant pole	5.6 Hz	$\sim 5 \text{ Hz}$
Unity-gain bandwidth	0.754 MHz	$\sim 1 \text{ MHz}$
Phase margin (follower)	87°	$65\text{--}80^\circ$
Slew rate	$0.54 \text{ V}/\mu\text{s}$, asymmetric like the original	$0.5 \text{ V}/\mu\text{s}$
Input offset	-0.39 mV	$< 1 \text{ mV}$
Output swing ($\pm 15 \text{ V}$, $2 \text{ k}\Omega$)	$-13.1 \dots +14.0 \text{ V}$	$\pm 13\text{--}14 \text{ V}$

The centerpiece check is the Miller single-pole identity: $A_{ol} \cdot f_{-3\text{dB}} = 0.78 \text{ MHz}$ vs the measured 0.75 MHz crossover (3%) — the 30 pF compensation doing its textbook job, verified across two independent analyses. The figures *emerge from the topology*: the Widlar tail and C_c produce the slew rate; nothing is programmed in. Even the fine structure is authentic — the input-stage charge kick at the slew edge and the Miller RHP-zero wiggle near 10 MHz appear because the transistors are really there.

Two measurement crafts worth recording: the open-loop AC bias network (feedback through 1 TH, injection through 1 MF) must have its corner far below the amplifier’s dominant pole or it masquerades as extra low-frequency gain (an early 1 MH/1 F attempt inflated A_{ol} to 140 dB); and phase margin in the inverting-injection configuration is cleanly computed as 180° minus the phase *lag accumulated from the flat band*, sidestepping wrap/convention pitfalls.

Files (**opamp741_examples/**)

`bjt741.va`, `ua741.subckt`, `run_opamp741.py` (characterization $\rightarrow$ `results/summary.txt`), `plot_opamp741.py` (the four-panel figure), `verify_opamp741.py` (11 checks in generous windows — this is a demonstrative model, not silicon), README with the reference table.

Regression

No compiler or ngspice source changes; all 73 example verify suites pass (this suite included), the integration suite 28/28.

Enhancement-84 — the LRM example sweep and its six defect fixes

This document describes Enhancement-84: every code example in the Verilog-AMS LRM 2023 PDF extracted, classified, and compiled against `openvaf-r (lrm_examples/)`, and the six compiler defects that sweep exposed, fixed. The suite is both the test that found the bugs and the regression net that pins the fixes.

Part 1 — the sweep (`lrm_examples/`)

The 442-page standard's examples are identified by *font* (LRM code is typeset in Courier New), not text heuristics, so the extraction is exhaustive: 231 candidate blocks. `extract_lrm_examples.py` also normalizes the PDF's typography (en-dashes for minus signs, curly quotes — the page-114 diode example contains a literal en-dash inside `limexp(...) - 1`), merges same-baseline column fragments, and joins blocks across page breaks by construct balance.

`curate_suite.py` holds an explicit per-block disposition table (each manual entry carries a content fingerprint so a re-extraction that rennumbers blocks fails loudly) and produces:

Directory	Count	Verified by <code>verify_lrm.py</code> as
<code>va/</code>	37	compile cleanly
<code>limitations/</code>	22	rejected with the exact pinned diagnostic, no crash
<code>ams/</code>	21	mixed-signal language — out of Verilog-A scope, stored not compiled
<code>findings/</code>	6	micro-repros: fixed defects must compile, open gaps keep their pin
<code>fragments/</code>	146	reference; double as a no-crash fuzz corpus

Examples stay verbatim wherever possible; unavoidable adaptations are annotated in-line (`[lrm_examples patch]` one-liners, `[lrm_examples context]` stubs for modules the LRM references but never defines, such as `vertNPN` or the Annex E oscillator primitives). Files whose examples omit port directions — the LRM does this on at least five pages — compile under `-W port_without_direction` rather than being edited.

Two errata in the standard itself: page 265's `twoclk` declares `vout_q1b` where the port is named `vout_q2` (a typo), and the port-direction omissions above.

Part 2 — the defects (six fixed, two documented)

F1 — named port branches crashed the compiler. `branch (<p> probe_p;` is plain Verilog-A (LRM 3.7.2), used by the page-62 current-probe example; probing `I(probe_p)` panicked in `BranchWrite::nodes(unreachable! on BranchKind::PortFlow)`. Fix: `hir_lower/src/expr.rs` routes a flow probe of a `PortFlow`-kind named `branch` through the same `CurrentKind::Port` param as a direct `I(<p>)`, so E-29's `build_port_flow_equations` gives it its defining equation. Runtime-verified in `ngspice`: the probe reads +5 mA where the source branch reads −5 mA. Contributing to a port branch (illegal per LRM) now gets a real diagnostic (`ContributeToPortFlow` in `hir_ty`), with the declaration site labeled, instead of the same panic.

F2 — garbage input crashed the parser. Non-Verilog text (the LRM's attribute-section pseudo-code, its Annex B keyword table) tripped `bump_ts` assertions in `port_decl/func_arg` (module-head port parsing reaches `port_decl` for anything that is not a plain name). Fix: `expect_ts_r` — a diagnostic plus one token of forced progress — at both sites (`parser/src/grammar/items/module.rs`). All 146 extracted LRM fragments now compile-attempt without a single panic or hang. *A first attempt gated elaboration on a clean parse instead — wrong: generate-for unrolling legitimately operates on parse trees that carry recoverable errors (that is how E-8/E-67 textual elaboration works), and the gate broke ten valid suite files. Reverted.*

F3 — instantiating an undefined module compiled silently. The E-5 flattener's `expand_instantiation` returned empty text for an unknown target: a typo'd module name became an invisible open circuit. Fix: a collected hard error (same channel as E-41's conflict errors) naming the instance and module. Two tailored variants: when the "module" is a known discipline, the input was really a name-then-range net declaration (`electrical out[0:2];` parses as an instantiation!) and the message says exactly that with the supported spelling; when it names a paramset whose own target failed to resolve (page 158 targets SPICE's `nmos3`), the message says the paramset was dropped, not that the name is undefined. Valid paramset instantiation from VA source (twin module, E-21) is unaffected — probed explicitly.

F5 — `$port_connected` failed on unconnected ports of flattened instances. Flattening renamed an open port to a synthesized local net (`clk1__open__vout_qbar`), which then failed `hir_ty`'s port-reference check — the builtin broke in exactly the case it exists to detect (the page-265 clock example). Fix: connectivity is decided where it is known — `render_instance_content` rewrites `$port_connected(<rendered-arg>)` to a literal `(1)/(0)` per instance (`resolve_port_connected`, keyed by the already-renamed argument). Top-level modules keep the native OSDI connected-flag path. Nested flattening composes level-by-level.

F7 — dead analog operators aborted codegen (found *through* F5: its `(0)` literals create constant-false branches). A `transition()` inside `if ((0))` survived const-folding as a detached-but-interned op whose state setup read optimized-away values: first a `split_tainted` panic on a layout-detached branch, then, one layer deeper, an "attempted to read undefined value" abort in LLVM codegen. Two fixes: literal `if` conditions lower only the taken branch (`hir_lower/src/stmt.rs` — the dead arm never reaches MIR at all), and `split_tainted` tolerates detached branch instructions (`mir_opt/src/split_tainted.rs`).

F8 — `openvaf-r` exited 0 on hard errors. The driver's `Err` arm printed the error chain and fell through to a success exit — every elaboration failure looked like a successful compile to shell scripts (the quirk first noticed during E-58). Fix: `exit(DATA_ERROR)` (`openvaf-driver/src/main.rs`). Combined with F3, this unmasked thirteen suite files whose original probe "compiles" were silent drops or error exits — all reclassified with honest pins.

F4 (open) — `__FILE__`/`__LINE__` predefined macros are not implemented. F6 (open) — part-selects in instance connections (`adc2 hi (out[3:2], in);`) don't parse. Both pinned in `findings/`.

Verification

- `lrm_examples/verify_lrm.py`: 7/7 — 37 compiles, 22 limitation pins, 6 finding pins, manifest/tree consistency.
- F1 runtime check in `ngspice` (named-port-branch probe vs. source branch current, sign-exact).
- Full regression: all version11 verify suites + the 28 `openvaf` integration tests.

Gotchas recorded

- Elaboration-time textual passes MUST tolerate parse errors (see F2's reverted first attempt) — rowan error nodes preserve text, and the generate/instantiation renderers rely on that.
- The suite's original probe classified 13 files as "compiling" that never did: 7 exited 0 through the F8 hole (generate bails, implicit- net conflicts), 6 silently dropped unknown instances through the F3 hole. A green exit from a tool with F8-class bugs is not evidence.
- `$port_connected` had to be resolved *during* flattening; both "fix the validation" and "keep the port" approaches founder on the fact that after inlining there is no port left to ask about.

Enhancement-85 — `__FILE__`/LINE and connection part-selects

This document describes Enhancement-85: the last two open findings of E-84's LRM example sweep, implemented. With F4 and F6 closed, all eight defects the sweep exposed are fixed, and the sweep's findings directory contains only regression pins.

F4 — the predefined source-location macros (LRM 10)

`__FILE__` and `__LINE__` were "macro has not been declared" errors. They cannot be ordinary preprocessor macros in this compiler: preprocessor tokens are (kind, span) pairs into *existing* source text, so there is nowhere for a synthesized literal to live. The fix is a textual pre-pass (`expand_source_location_macros` in `hir/src/elaborate.rs`, run before the generate/instantiation passes), the same mechanism E-58 established:

- `__FILE__` → a string literal holding the root file's basename — not the full path, for the same provenance reason E-58 names its synthetic files by basename: the expansion is baked into the compiled `.osdi`, and an absolute path would leak the build machine's layout (the repo's standing machine-portability rule).
- `__LINE__` → the 1-based line of the occurrence in the user's file. Replacements are inline, so every later line number stays exact. The scanner skips string literals and comments.
- A use inside a `define` body expands at the definition site (textual semantics; C's "line of use" would need real preprocessor tokens). Documented, and pinned by the verify suite.

Runtime-verified in ngspice: `$strobe("%s:%0d", FILE, __LINE__)` prints `srcloc.va:18` for the direct use and `srcloc.va:11` for the `define`-site use (`filemacro_examples`, 5/5).

F6 — part-selects in instance connections (LRM 6, pages 163–164)

`adc2 hi (out[3:2], in);` was a parse error. Three small pieces:

- Parser: the bit-select bracket accepts an optional `: expr`; the colon token stays in the CST and is what distinguishes a part-select from E-15's multi-dimensional `[i][j]` indexing — no new node kind.
- Guard: part-selects are only legal in port connections (which elaboration consumes textually and never body-lowers), so body lowering collects any that reach it into `stray_part_selects` — E-78's stray-don't-care pattern — and `hir_ty` reports a dedicated diagnostic pointing at the connection form.
- Elaboration: `bind_port` recognizes a constant base `[msb:lsb]` actual and slices those bits of the caller's bus onto the port, ascending-to-ascending (the same bit-order convention as the existing full-bus slicing); a width-1 slice onto a scalar port degrades to the bit-select it denotes; width mismatches fall through to the ordinary path and its standard diagnostics. Positional and named (`.i(v[1:0])`) forms both work. Bounds are integer literals (parameter-dependent selects are the same class as parameter-dependent bus widths — documented limitation).

Runtime-verified in ngspice (`partselect_examples`, 5/5): a 4-bit bus with $V(v[k]) = k$ volts wired via `v[3:2]` (positional), `.i(v[1:0])` (named), and `v[2:2]` (width-1) produces exactly $8\text{ V} / 2\text{ V} / 2\text{ V}$ — the outputs $2 \cdot V(\text{msb}) + V(\text{lsb})$ pin the per-bit routing.

Suite graduations

- `micro_file_line.va`, `micro_partselect.va`: findings → must-compile regression pins.
- `lrm_p163_1.va` (binary ADC tree, positional part-selects): compiles verbatim.
- `lrm_p164_1.va` (named part-selects): compiles with the 1-bit adc context stub the LRM references but never defines.
- LRM suite counts: 39 compile / 20 limitations / 21 AMS; verify 7/7. All eight sweep defects (F1–F8) now fixed.

Verification

- filemacro_examples 5/5, partselect_examples 5/5 (both with ngspice runtime pins), lrm_examples 7/7.
- Full regression: all version11 verify suites + 28 integration tests; parser/hir_def/hir_ty dev+snapshot tests green.

Gotchas recorded

- Preprocessor tokens cannot carry synthesized text — anything that must *invent* source (like `__FILE__`'s literal) has to happen at the textual pre-pass layer where a virtual file can hold it.
- The CST tolerates extra tokens: the part-select colon rides inside the existing `BIT_SELECT_EXPR` node, avoiding the full new-SyntaxKind pipeline threading; the AST accessor (`is_part_select`) just looks for the token. Consumers that iterate `indices()` MUST check it — a two-expr part-select is otherwise indistinguishable from a 2-D index.

Enhancement-86 — hierarchical branch probes

This document describes Enhancement-86: the LRM page-119 hierarchical branch reference forms — `V(top.a1.b)`, `V(top.d1.branch(a, b))`, and `I(top.d1.branch(<p>))` — implemented end-to-end, plus the two pre-existing DAE defects and two elaboration bugs the work uncovered and fixed. The second scope item the user requested, named part-select connections (`.out(out[3:2])`), had already shipped in E-85; this enhancement adds the missing runtime pin for the output-direction named form (a 2-bit output bus sliced onto a caller bus, 13 V exact).

The three probe forms

Named branch of an instance (`V(top.a1.b)`): free once absolute paths resolve — branches rename like every other child item (`a1__b`), so E-49's member rewrite produces exactly the right name. The missing piece was scope: E-49's absolute aliases (`<top>.<chain>`, `$root.`) existed only for the top module's own body text. `ElabCtx` now carries the top's unambiguous absolute map (`abs_prefixes`) and every inlined child's scope merges it, so SIBLING bodies resolve absolute references too.

Unnamed branch (`V(top.d1.branch(va, vb)) / branch(a)`): after flattening, the child's unnamed branch (`va, vb`) IS the flattened node pair (`d1__va, d1__vb`) — the reference expands textually to that pair, so `V(...)` and `I(...)` of it are exactly the child's branch quantities. The path grammar swallows the `.branch(...)` tail into the path node (`paths.rs`) so the enclosing item's CST stays whole — the E-49 hole scanner then rewrites the reference (a parse-shredded item was the original failure mode); an unresolvable chain fails name resolution as a normal error.

Port branch (`I(top.d1.branch(<p>))`): the child's current into its port is not a node quantity, so flattening synthesizes a 0V ammeter: the child's body sees a fresh internal net, and a named branch from the caller's net to it carries exactly the instance current (positive into the child, matching E-29's `I(<p>)` sign). A pre-scan over every module's text (`find_port_branch_probes`) records which instances need one, and a child's own branch (`<p> pb`; declarations alias the same ammeter — fixing a blind spot where port branches inside flattened children were broken (bound to a top port: silently read the NODE total; bound to an internal net: hard error)).

Modules holding absolute references anchored at another module (the LRM's monitor idiom) are hierarchy-bound: their standalone flattened copies are omitted (a comment marks the omission) since only inlined copies can resolve.

The two DAE defects (pre-existing, exposed by the ammeter)

1. Voltage-source branches feeding internal nodes were open circuits at DC. The small-signal (noise/ac_stim) pruner registers a node's drives keyed on the branch's LIVE voltage unknown — a pure `V(a, f) <+ expr` whose `V(a, f)` is never read registered nothing, so an internal node fed only by such a source plus linear conduction was classified as a zero-DC small-signal node and its conduction silently moved to the AC-only residual. Any voltage-capable branch now disqualifies its nodes from pruning regardless of liveness (`small_signal_network.rs`). `V(port, internal) <+ 3.0` with a 1k to ground now conducts exactly 2 mA.

Behavior change: a collapsible branch whose current the model references (`MVSG_CMC`'s access resistances carry noise on `flow(d, drc)`) now stays a real 0V source instead of collapsing — electrically identical, two extra matrix rows, and the branch-current references stay meaningful (the `mvsg_cmc` descriptor snapshot updated accordingly).

2. A probed `V(x, y) <+ 0` branch was collapse-hinted away. Node collapse eliminated the very unknown `I(branch)` reads — the probe silently returned zero. `NodeCollapse::new` now skips hint pairs whose branch current is an actual DAE unknown (the unprobed collapse idiom is untouched), and `hint()` tolerates suppressed pairs (it used to `unwrap_index` — a panic). Pinned by the permanent `vsrc_internal_node sim_back` snapshot test.

Also fixed

- **ground gnd;** inside flattened children lost its keyword during net-declaration re-rendering (`a1_gnd;` — a parse error); the net-type token is preserved now.

Verification

- `hierbranch_examples` 6/6: all three probe forms runtime-exact (1.34 V / 2.5 V / 5 V), the discriminating check that the port-branch probe reads the instance's 5 mA and not the 10 mA node total, total source current pinning both DAE fixes, and the unresolvable-chain diagnostic.
- `partselect_examples` extended to 6/6 (output-direction named slice, 13 V exact).
- LRM suite: `lrm_p119_1` graduates (40 compile / 19 limitations / 21 AMS, verify 7/7).
- Full regression: all version11 verify suites + 28 integration tests; `sim_back/mir_opt/hir_lower` snapshot tests green (including the new `vsrc_internal_node` pin).

Gotchas recorded

- The E-49 hole scanner operates per-ITEM on the CST: a construct that shreds the parse (the branch keyword in path position) fragments the item and the scanner sees truncated text — grammar must swallow such tails even when elaboration consumes them.
- Absolute-path references from siblings need the top's alias map merged into every child render — per-child chain maps only cover the child's own subtree.
- ngspice caches nothing across processes, but STALE .osdi artifacts during compiler debugging absolutely do (two rounds of confusion in this enhancement — recompile before every ngspice probe).

Enhancement-87 — block-scoped parameters

This document describes Enhancement-87: block-scoped parameters (LRM 6.3, the page-112 example) — `parameter/localparam` declared inside a named `begin: label` block. The investigation found the feature already works end-to-end; the deliverable is a runtime-verified example suite plus a fix for the one genuine rough edge — the LRM's own `#(.myscope.p2(4))` // error case, which produced a confusing parser cascade instead of a targeted diagnostic.

The feature already works

A named block may declare its own parameters and localparams; they are compile-time constants local to the block, referenced hierarchically (`label.name`) from the enclosing analog code, and derivable from the module's parameters. All of this is supported by the existing `hir_def` name-resolution machinery (`block_def_map`, `block ScopeIds`):

```
analog begin
  begin: s
    parameter real g2 = gain * gain; // block param, from a module param
    localparam real base = 1.0;      // block localparam
    real contrib = g2 + base;         // block var uses both
  end
  vout = s.contrib;                  // hierarchical read
end
```

Runtime-verified in ngspice (`blockparam_examples`, 6/6): with the module parameters at their defaults $v_{out} = \text{gain}^2 + 1 + \text{offset} \cdot 10 = 10$ V, and a model-card override of the module parameters flows into the block-scoped parameters that depend on them (`gain=3`, `offset=0.2` → 12 V). Nested blocks work too — an inner block parameter derived from an outer block parameter resolves correctly (7 V).

The fix: the illegal hierarchical override

The LRM page-112 example deliberately includes an illegal case:

```
example #(.p1(4))      inst1(); // allowed -- p1 is module-level
example #(.myscope.p2(4)) inst2(); // error -- p2 is block-scoped
```

The `#()` instance parameter override binds to a module's *parameter ports*, which are only its module-level parameters; a block-scoped parameter is local to its block and cannot be reached this way (LRM 6.3.2). `openvaf-r` previously choked on the dotted name `.myscope.p2` with a bare unexpected token `' ) '` and a spurious cascade (a false `'myscope'` was already declared on the block below).

Two small changes fix this:

- Parser (`param_assign`): consume the extra `.segments` of a hierarchical override target so the parse stays clean (no cascade), the extra names collected as additional `Name` children of the `PARAM_ASSIGN` node.
- Elaboration (`resolve_param_bindings`): a named override with more than one segment is reported with a targeted diagnostic naming the offending path and the target module — collected and bailed like the E-84 unknown-module and E-59 port-concat errors (the illegal override is flattened away during elaboration, so the check must live where the instantiation is semantically processed, not in CST validation).

The page-112 example now yields exactly one clear error:

```
error: instance parameter override '.myscope.p2' targets a
hierarchical/block-scoped parameter, which cannot be overridden this way;
only a module-level parameter of 'example' may be named in an instance
parameter assignment
```

Scope decision: overriding block-scoped parameters

A block-scoped parameter (as opposed to `localparam`) is nominally overridable, but the LRM only demonstrates that the `#()` route is illegal. `openvaf-r` treats block-scoped parameters as hierarchically-visible compile-time constants: they take their value from their initializer (which may reference the enclosing module's parameters, so a module-parameter override propagates into them), and they are not overridable through an external mechanism — neither `#()` (now a targeted error) nor `defparam` (`defparam inst.s.p2 = ...` reports “did not resolve to any parameter”, since block-scoped parameters are not hoisted to module-level `defparam` targets). This is a documented scope decision, consistent with the `localparam`-like treatment the LRM's own example implies, and mirrors the parameter-shaped-generate decision of E-67.

Verification

- `blockparam_examples` 6/6: compile, defaults, module-override propagation into block params, nested-block derivation, and the targeted rejection of a block-scoped `#()` override (no cascade, no crash).
- LRM suite: `lrm_p112_1`'s pin updated from the old parser cascade (`unexpected token`) to the clean diagnostic; the file stays a correct limitation (it contains the LRM's own intentional error). Suite 7/7, counts unchanged (40 / 19 / 21).
- Full regression: all version11 verify suites + 28 integration tests; parser/syntax snapshot tests green.

Gotchas recorded

- A semantic error about a construct that elaboration *flattens away* (an instance parameter override) cannot be surfaced from CST validation on the original file — after elaboration, `root_file` is the synthesized flattened file, and the original's validation errors are not re-collected. Such checks belong in the elaboration pass, alongside the E-84/E-59 collected-error vecs.
- The feature “already works” was only visible by probing the *legal* construct in isolation; the LRM example's bundled `// error` case made the whole file look unsupported. Verify the minimal legal form before assuming a pinned limitation reflects a missing feature.

Enhancement-88 — the legacy **generate** statement

This document describes Enhancement-88: support for the obsolete Verilog-A 1.0 **generate** statement (LRM Annex C.4), the last unsupported *analog-block* looping construct — **generate** <id> (<start>, <end> [, <incr>]) <body>.

What it is

Unlike the modern module-level **generate for** (Enhancement-8, which produces structural items), the legacy **generate** is a behavioral statement inside an analog block that unrolls its body at compile time, substituting the index with each successive constant value. It is the form the LRM's own page-438 flash-ADC example uses:

```
analog begin
  thresh = fullscale/2.0;
  sample = V(in);
  generate i (bits-1, 0) begin           // i = 7, 6, ..., 0 (descending)
    V(out[i]) <+ transition(sample > thresh, dly, ttime);
    if (sample > thresh) sample = sample - thresh;
    sample = 2.0*sample;
  end
end
```

The unroll must be a compile-time replication (not a runtime loop): `out[i]` selects a bus *node*, so the index has to become a literal `out[7]`, `out[6]`, ... — a runtime index cannot select a node. The body is also stateful across iterations (`sample` is halved each step), so the unroll order is load-bearing.

Implementation

A self-contained textual pre-pass, `elaborate_legacy_generate(hir/src/elaborate.rs)`, run before the module-level **generate** pass and before name resolution (the index is not a declared variable, so the substitution has to happen on source text). This mirrors the E-8/E-67 **generate-for** machinery and the E-85 `__FILE__` textual pre-pass, and avoids threading a new statement kind through parser → `hir_def` → `hir_lower` for an obsolete construct.

- Scan: tokenize; find **generate** followed by an identifier that is not **for/if/case** (those are the module-level regions, left untouched) — unambiguous since the legacy form always names an index.
- Bounds: <start>, <end>, optional <incr> are evaluated by a small constant-integer token evaluator (decimal literals, + - * /, unary -, parens). Direction defaults to ± 1 from the bound ordering.
- Body: a balanced **begin ... end** or a single statement to the next ;.
- Unroll: per iteration, the index is substituted by its literal (whole-identifier), and each bit-select [`<expr>`] whose contents then constant-fold is rewritten to [`<literal>`] (a bus bit-select requires a literal index — `out[7+1]` is rejected, so `out[i+1]` must fold to `out[8]`). The iteration blocks are wrapped in one outer **begin ... end** so the whole expansion is a single statement, valid whether the **generate** sat inside an **analog begin ... end** or was the direct body of **analog**. A fixpoint loop handles nested legacy generates.
- Bounds must be elaboration-time constants. A parameter bound (**generate** `i (bits-1, 0)`) is a targeted error — a runtime-bindable parameter cannot shape a compile-time unroll, the same scope decision as **generate for/generate if** (Enhancement-67).

Verification

- `legacygen_examples 6/6`: the LRM flash-ADC (constant width) compiles and, for a 0.7 V DC input, produces the exact 4-bit code 1011 (per-bit runtime pins in ngspice — proving index sub-

stitution, descending order, and the stateful accumulation); a parameter-bound legacy generate is rejected with the targeted diagnostic.

- Probed additionally: ascending direction, analog-direct (no enclosing begin) body, single-statement body, and [k+1] bracket folding to a literal.
- LRM suite: lrm_p438_1 pin updated from the old parse error to the new bound-constant diagnostic — it stays a limitation because it uses *both* a parameter bound and a parameter bus width (7/7, 40/19/21).
- Modern generate for/if/case untouched (generate suite 9/9); full regression + 28 integration tests; parser/hir snapshot tests green.

Gotchas recorded

- A bus bit-select requires a literal integer, not a constant expression (out [7+1] is a compile error), so index substitution must be followed by bracket-content constant folding.
- Wrapping the unrolled iterations in a single outer begin ... end is what makes the expansion valid in both analog-block contexts (nested block vs. analog's single-statement body) — emitting bare adjacent blocks breaks the analog <stmt> form.

Enhancement-89 — name-then-range ports and the Annex E primitives

This document describes Enhancement-89, two related pieces of LRM coverage: the name-then-range form of vectored net/port declarations, and a small Annex E SPICE-compatibility primitives library.

Part 1 — name-then-range net/port declarations

The LRM allows a vectored net or port to be declared in two orders. The *range-then-name* form (Enhancement-3) was already supported:

```
output [0:3] out;
electrical [0:3] out;
```

Enhancement-89 adds the *name-then-range* (unpacked-array) form the LRM's page-45 example uses:

```
input in[0:2];          // port direction
output out[0:2];
electrical in[0:2];     // net / discipline
electrical out[0:2];
```

It is purely a syntactic alternative, so it is handled by a textual pre-pass (`normalize_name_range_decls`, `hir/src/elaborate.rs`) that rewrites `<head> <name>[range]` to `<head> [range] <name>` before parsing — reusing all of the existing bus/port machinery unchanged (parser, item tree, node expansion). The disambiguation from an instance array is exact: an instantiation always has a (port list) after the range, so a `<head> <name>[range]` followed by `;` is a declaration, not an instance. Scope: single-name 1-D declarations (the LRM form); a multi-name or multi-dimensional name-then-range declaration is left as-is. Complements E-3 (range-then-name buses) and E-18 (name-then-range variables).

Runtime-verified (`arrayport_examples`, 7/7): a name-then-range output bus drives per-bit taps to their exact fractions, a name-then-range input port compiles, and the form is confirmed equivalent to its range-then-name twin.

Part 2 — the Annex E SPICE-compatibility primitives

LRM Annex E defines the SPICE primitives a Verilog-AMS simulator provides for interoperability. Several LRM examples instantiate them (`spice_nmos/spice_pmos`, `resistor`, `capacitor`, ...). This enhancement provides them as a small, reusable Verilog-A library (`examples/annexe_examples/annex_e_primitives.va`):

- linear one-ports `resistor` / `capacitor` / `inductor`;
- independent sources `vsource` / `isource`;
- square-law (SPICE level-1) `spice_nmos` / `spice_pmos` (3-terminal).

They are ordinary modules meant to be instantiated and flattened (Enhancement-5) into a top module — the names `resistor/capacitor/inductor` deliberately match ngspice's built-in device names, so they may never be registered directly as a `.model` device (`openvaf-r` warns, L018).

Runtime-verified (`annexe_examples`, 6/6): an RC lowpass (DC pass-through

- the RC charging time constant to 63.2 %), and a CMOS inverter (rail-to-rail from the two square-law MOS primitives).

LRM suite

Two examples graduate with the primitives provided as context: the page-152 transmission gate (`spice_nmos/spice_pmos`) and the page-153 instance-parameter forms (`mosp/spice_pmos`). The page-45 exam-

ple's name-then-range parse is fixed but it stays a limitation on its undefined gen/sink modules and a multi-dimensional parameter-array literal. Suite now 42 compile / 17 limitations / 21 AMS (7/7).

Verification

- arrayport_examples 7/7, annexe_examples 6/6 (both with ngspice runtime pins); LRM suite 7/7.
- Full regression + 28 integration tests; parser/hir snapshot tests green.
- Confirmed the name-then-range pre-pass leaves instance arrays and variable/parameter declarations untouched (the head-keyword exclusion and the (-after-range instance rule).

Gotchas recorded

- A multi-bit *input* bus port read (input [0:2] in; ... V(in[k])) has a pre-existing OSDI terminal-mapping oddity (independent of E-89 — the range-then-name twin behaves identically); the demo therefore drives a bus *output* and reads a scalar input, the shape E-3's own bus example uses.
- Annex E primitive modules must stay sub-modules; naming a top-level OSDI device resistor collides with ngspice's built-in (the E-29 create_model gotcha).

Enhancement-90 — multi-bit input bus port bit reads

This document describes Enhancement-90, a compiler fix for reading an individual bit of a multi-bit input bus port declared in the non-ANSI (Verilog-2001) header style. It is the follow-up to the terminal-mapping oddity noted while finishing Enhancement-89.

The symptom

A module that reads one bit of a multi-bit *input* bus port returned the wrong value (often 0) when the bus was not the last port in the header:

```
module m(in, y);
    input  [0:2] in;      // 3-bit input bus, declared in the body
    electrical [0:2] in;
    output y; electrical y;
    analog V(y) <+ V(in[1]); // reads the middle bit
endmodule
```

Driving the three bus terminals with 1 V, 2 V and 3 V, $V(y)$ did not read 2 V. A scalar input port worked, and the Enhancement-89 *name-then-range* spelling (`input in[0:2];`) behaved identically to the *range-then-name* spelling — so this was not an E-89 regression but an older defect in E-3 territory (vectored ports).

Root cause — scrambled terminal order

The bug was purely in node ordering, not the physics. The DAE for the module above is correct: four unknowns, with the residual at y depending on $V(in[1])$. But the OSDI descriptor presented the terminals out of order.

OpenVAF builds a module's node list in two steps for a non-ANSI header:

1. the header (`module m(in, y);`) creates one placeholder node per port name, in header order: in , then y ;
2. the body declarations fill in each port's direction, discipline and width. Expanding the bus input `[0:2] in` renamed the in placeholder to $in[0]$ (its first bit) but appended the remaining bits $in[1]$, $in[2]$ at the end of the node list — *after* the y placeholder.

The resulting node order was therefore

```
[ in[0], y, in[1], in[2] ]      // wrong
```

instead of the header-port order

```
[ in[0], in[1], in[2], y ]      // right
```

Because the OSDI ABI maps a netlist instance's terminals positionally to the first `num_terminals` nodes, this scramble mis-wired the bus bits: netlist terminal 2 landed on y , terminal 3 on $in[1]$, and so on. The scramble only appeared when a multi-bit bus port was *not* the last port (when it is last, the appended bits happen to fall in the right place), which is why it had gone unnoticed. A true ANSI header (`module m(inout [0:2] in, inout y);`) was already correct, because there the bus is expanded in place while the header is processed.

The fix

`hir_def/src/item_tree/lower.rs` now pre-scans the body port declarations for each port's width *before* creating the header placeholders (`prescan_body_port_widths`). When `lower_module_ports` meets a non-ANSI header port name that a body declaration gives a width to, it expands the bus into its bits *there*, in header-port order, so the bits stay contiguous:

```
[ in[0], in[1], in[2], y ]
```

The `BusDecl` itself is still registered by the body declaration, so bit selects resolve exactly as before; only the node-creation order changes. Buses that are already the last port produce the identical node list, so existing models (the E-3 `bus_buffer`, scalar-in/bus-out shapes) are unaffected. Fixing the order at node creation means the DAE unknowns, the Jacobian, and the OSDI terminal array all follow correctly with no downstream changes.

Verification

- `busport_examples` (6/6, ngspice runtime pins): a bus-first model reads each bit distinctly (`o0=in[0]=1 V`, `o1=in[1]=2 V`, `o2=in[2]=3 V`); a model with the bus in the *middle* of the header reads its bits correctly ($q = V(in[2]) - V(in[0]) + V(p) = 2.5 V$); and the E-89 name-then-range spelling of the same bus reads identically.
- A permanent DAE snapshot regression guard (`sim_back bus_input_port_order`) pins the header-order node layout for a bus-not-last module.
- Full regression: 81/81 verify suites, 28/28 integration tests, and the parser/hir/sim_back snapshot suites all green.

Gotcha recorded

- The scramble is invisible when the bus is the last port and invisible in a true ANSI header — it only bites a non-ANSI header where a multi-bit bus port is followed by another port. Diagnosing it required dumping the OSDI node array order, not just the DAE physics (which was correct).

Enhancement-91 — multi-name name-then-range declarations and parameter-dependent widths

Enhancement-91 adds two related pieces of net/port/array declaration coverage, both handled by textual pre-passes in `hir/src/elaborate.rs` (the E-85/88/89/90 pattern), so the existing bus (E-3) and array (E-14/15) machinery is reused unchanged.

Part 1 — multi-name name-then-range declarations

Enhancement-89 added the *name-then-range* form of a vectored net/port (`input in[0:2];`) but only for a single name with a 1-D range. Enhancement-91 completes it for a comma-separated list, with a per-name range:

```
input a[0:1], b[0:3], c;           // two buses of different widths + a scalar
electrical a[0:1], b[0:3], c;
```

The normaliser (`normalize_name_range_decls`) now parses the whole name list and emits one *range-then-name* declaration per name, sharing the head (`input [0:1] a; input [0:3] b; input c;`) — a form the parser already accepts. The single-name path is unchanged, and the instance-array disambiguation is the same as E-89 (an instance has a `(port list)` after the range; a declaration ends in `,` or `;`).

A multi-dimensional name (`in[0:2][0:1]`) is still left untouched: multi-dimensional vectored ports are unsupported in *both* declaration orders (the range-then-name form does not parse either), so the existing diagnostic fires rather than a silent miscompile.

Runtime-verified: two input buses of different widths plus a scalar, each bit read exactly (`a[1]=2 V`, `b[2]=5 V`, `c=9 V`).

Part 2 — parameter-dependent declaration widths

A net/port/array range whose bounds reference a parameter now folds to a literal range using that parameter's elaboration-time value:

```
parameter integer N = 4;
electrical [0:N-1] out;           // -> electrical [0:3] out;
real w[0:N-1];                   // -> real w[0:3];
```

The pre-pass (`fold_parameter_widths`) collects each module's constant-integer parameter defaults (resolving one default through another by a fixpoint, and skipping `from/exclude` constraints) and rewrites every declaration range `[msb:lsb]` that references a parameter. Only declaration ranges (which contain a `:`) are touched; bit-selects (`x[i]`) and already-literal ranges are left alone. The shared constant-integer evaluator gained an optional parameter map — with an empty map it is exactly the Enhancement-88 literal-only evaluator, so legacy-generate bounds are unaffected.

Scope (a structural parameter). The width is fixed at the parameter's default. A model card or instance that overrides the parameter does not resize the bus or array — OSDI has a single fixed node count per module, so a width parameter is structural (the same decision as generate bounds, Enhancement-67/88). To get a different width, change the default and recompile.

Runtime-verified:

- a parameter-sized array with a runtime loop over the parameter (E-70): `real w[0:N-1]` with the harmonic sum $\sum \text{vref}/(k+1)$ — 2.08333 at `N=4` and 2.71786 at `N=8`, so the array width tracks the default;
- a parameter-width node bus `electrical [0:bits-1] out` — four terminals at `bits=4`, six at `bits=6`.

What this does *not* cover (documented limitations)

- Analog-block genvar for-loops — the LRM ADC pattern (p091/117/134) puts for (i=0;i<bits;. . .) V(out[i]) <+ . . . *inside* the analog block, indexing a bus by a genvar. Unrolling an analog-block genvar loop is a separate feature; those examples now fold their widths but still fail on the loop.
- Parameter-valued bus bit-selects outside declarations (n [N], p169) — folding a parameter into an arbitrary bit-select is unsafe for variable-array indexing, so only declaration ranges are folded.
- \$table_model with runtime array data (LRM p274) is deferred to a later enhancement — it is a scattered-data, runtime-variable table that the compile-time regular-grid machinery (E-16/17/40) cannot express.

Verification

- paramwidth_examples 11/11 (param-sized array, param-width bus at two widths, multi-name buses, multi-dim rejection) with ngspice runtime pins.
- Full regression: 82 verify suites + 28 integration tests; hir snapshot tests green; the E-3/E-14/E-67/E-89 suites unaffected.
- LRM suite 7/7: the five parameter-width examples (p091/117/134/169) stay documented limitations but are re-pinned to their post-width-fold blocker (analog-block genvar loop / parameter bus bit-select).

Enhancement-92 — freezing structural (width) parameters

Enhancement-92 closes a safety gap in Enhancement-91's parameter-dependent declaration widths: a parameter that shapes a width is now frozen to a `localparam`, so a netlist override can no longer desync the frozen structure from behavioural code.

The problem

Enhancement-91 folds a parameter-dependent width to a literal at elaboration, using the parameter's default. But it left the parameter declaration overridable. When the *same* parameter also drives behavioural code, an override desynced the two:

```
module wsum(out);
    parameter integer N = 4;
    output out; electrical out;
    real w[0:N-1];           // sized [0:3] at the default (E-91)
    integer k;
    analog begin : b
        real acc; acc = 0.0;
        for (k = 0; k < N; k = k + 1) begin // loop bound follows the override
            w[k] = 1.0/(k+1); acc = acc + w[k];
        end
        V(out) <+ acc;
    end
endmodule
```

With `.model ws wsum N=8`, the array `w` stayed sized at 4 (frozen from the default) while the runtime loop ran to 8 — a silent out-of-bounds write to `w[4..7]`, giving a garbage result (~6.08 instead of the correct 2.0833). ngspice accepted the override because `N` was still a settable OSDI parameter, even though structurally it could not take effect (the OSDI descriptor has one fixed node/array count per module).

The fix

The width-fold pre-pass (`fold_parameter_widths`, `hir/src/elaborate.rs`) now records every parameter that shaped a declaration width and rewrites its declaration from parameter to `localparam` (`freeze_width_parameters`):

- a multi-parameter declaration is split so only the structural names freeze — `parameter integer bits = 4; parameter real gain = 2.0;` with a bits-width bus becomes `localparam integer bits = 4; parameter real gain = 2.0;`, leaving `gain` overridable;
- a range constraint (`parameter integer width = 8 from [2:24];`) is dropped from the frozen name — a `localparam` cannot carry one — but preserved on parameters that stay overridable.

A `localparam` is a compile-time constant, so it is not exported as an OSDI parameter: the value is fixed at the default everywhere (structure *and* behaviour stay consistent), and a netlist attempt to set it is simply ignored rather than corrupting the model. To use a different width, change the default and recompile.

Verification

`paramfreeze_examples` (4/4, ngspice runtime pins):

- `wsum` default (`N=4`) gives the correct harmonic sum 2.08333;
- `.model ws wsum N=8` now stays **2.08333** — the override is ignored and the model keeps its default (before this fix it produced the corrupted ~6.08);

- a non-width parameter (gain) stays overridable: `.model m mp gain=10` scales the outputs (`out[0]=1.0`, `out[3]=4.0`), confirming the multi-parameter declaration was split and only the structural name froze.

Full regression: 84 verify suites + 28 integration tests + hir snapshot tests green. The Enhancement-91 paramwidth_examples suite still passes 11/11 with the freeze active.

Enhancement-93 — warning when a fixed (localparam) parameter is set

Enhancement-92 froze structural width parameters to `localparam` so a netlist override could no longer corrupt the model. But the override was still silently ignored — `openvaf` exports every `localparam` as an OSDI parameter (its slot stores the computed value), so `ngspice` recognised the name, accepted the assignment, and dropped it with no feedback. Enhancement-93 makes that override an explicit warning.

The change (`openvaf` + `ngspice`)

`openvaf` marks each non-settable parameter in the OSDI descriptor. A new additive flag bit, `PARA_FLAG_FIXED` (`1 << 2`, a free bit of the parameter `flags` field), is set for every `localparam` — which includes the structural width parameters frozen by Enhancement-92. Being additive, it does not change the descriptor layout or the ABI; a simulator that does not know the bit simply ignores it.

`ngspice` checks the flag where a model/instance parameter is applied (`OSDIparam/OSDIiParam`, `src/osdi/osdiparam.c`): if the target parameter carries `PARA_FLAG_FIXED`, it emits

Warning: parameter 'N' is a fixed (localparam) value and cannot be set from the netlist; ignored.

and skips the write (which was overwritten by the parameter-init default anyway). Ordinary parameters are unaffected.

This turns the silent no-op into clear feedback, and it generalises correctly to *all* localparams, not just frozen width parameters — a `localparam` is non-overridable by definition (LRM), so setting one from the netlist is always a user error worth reporting.

Verification

`paramnonset_examples` (7/7, `ngspice` runtime pins):

- overriding the frozen width parameter `N(.model ws wsum N=8)` now warns and keeps the default (`v(s)=2.08333`, the value that Enhancement-92 already made correct);
- overriding a hand-written `localparam` (`scale`) also warns and keeps its default;
- overriding an ordinary parameter (`gain`) does not warn and does take effect (`v(s)=6.25`).

Full regression: 84 verify suites + 28 integration tests (the OSDI descriptor snapshots were regenerated — only `hisimsofb`, which exports a `localparam`, gained the flag). The additive flag keeps every other descriptor byte-identical.

Enhancement-94 — the **pyplot** command (matplotlib backend)

Enhancement-94 adds a new ngspice interactive command, **pyplot**, that renders simulated vectors with matplotlib — a Python counterpart to the existing **gnuplot** command.

Usage

```
pyplot <file> <plot expressions...>
```

Like **gnuplot**, the first word is the output file base name and the rest are ordinary plot expressions (the same syntax **plot** accepts). **pyplot** writes a **<file>.data** table and a **<file>.py** matplotlib script and runs Python on it:

```
.control
run
pyplot rc v(out) v(in)          ; opens an interactive matplotlib window
.endc
```

Two set variables control it:

- **pyplot_terminal=png** — render headless (matplotlib's Agg backend) to **<file>.png** instead of opening a window. Ideal for batch/**-b** runs and CI.
- **pyplot_python=<interp>** — the Python interpreter to run (default **python3**).

The generated script honours the current plot's title, axis labels, log scales (**ac/loglog** etc. → **set_xscale('log')**), grid, axis limits, a legend, and the marker/step style implied by **plot**'s **point/comb** modes.

Implementation

Modelled directly on the **gnuplot** backend:

- **src/frontend/plotting/pyplot.c** — **ft_pyplot()** writes the data table (an (x, y) column pair per vector, real part for complex data) and emits the matplotlib script, then **system()**s **python3 <file>.py** (synchronously for a PNG, in the background for an interactive window). It mirrors **ft_gnuplot()**'s structure and reuses the same **plotit()** vector-expression machinery.
- **src/frontend/plotting/plotit.c** — a new "pyplot" device arm alongside "gnuplot"/"writesimple".
- **src/frontend/com_pyplot.c** — the **com_pyplot()** command wrapper (mirrors **com_gnuplot()**).
- **src/frontend/commands.c** — **pyplot** registered in both command tables.
- Makefiles updated for the two new source files.

Because **pyplot** goes through **plotit()**, it accepts the full range of plot expressions, vector arithmetic, and ranges that **plot/gnuplot** do.

Verification

pyplot_examples (6/6): an OpenVAF/OSDI model (**rcload.va**) is simulated and

- a transient **pyplot rc v(out) v(in)** writes **rc.py**, **rc.data** and a valid **rc.png** (checked by PNG magic bytes and size); the generated script is confirmed to load matplotlib and plot both vectors;
- an AC sweep **pyplot acmag db(v(out))** renders a log-x-axis PNG.

Full regression: 86 verify suites + 28 integration tests. **pyplot** is purely additive (a new command + backend); nothing else changed. matplotlib must be installed for the Python side (as **gnuplot** must be for **gnuplot**).

Enhancement-95 — optional file name for **pyplot**

A small refinement to Enhancement-94's `pyplot` command: the output file base name is now optional, so you can plot directly without inventing a name.

Before / after

Enhancement-94 required the base name as the first argument (like `gnuplot`), so `pyplot v(out)` silently did nothing — the single word `v(out)` was taken as the file name, leaving no plot arguments. Now:

```
pyplot v(out) v(in)      ; no file name -> writes pyplot.py / pyplot.png
pyplot out in           ; bare node names -> also defaults to "pyplot"
pyplot myplot v(out)    ; an explicit base name still works -> myplot.png
```

How the first word is classified

`com_pyplot()` treats the first word as a file name only if it is not itself a plot expression — that is, it contains no `(` (as in `v(out)`, `db(...)`) and does not name an existing vector (as a bare node name like `out` would, checked with `ngspice's vec_get()`). Otherwise the base name defaults to `pyplot` and every word is a plot argument.

This resolves the common cases unambiguously: a leading `v(...)/i(...)` expression or a bare node name defaults the name; a plain word that is not a vector is used as the name. Two supporting fixes: the command's minimum argument count was lowered from 2 to 1 (so a single-vector `pyplot v(out)` reaches the handler), and the help text is now `[file] plotargs`.

Verification

`pyplot_examples (7/7)`: the Enhancement-94 checks plus a new one confirming `pyplot v(out)` with no file name renders the default `pyplot.png`. Full regression: 86 verify suites + 28 integration tests. `ngspice-only` and additive.

Enhancement-96 — module-level **generate for** without **generate/endgenerate**

A parser fix: a `generate for/if/case` written at module scope without the optional `generate/endgenerate` keywords is now parsed and elaborated. The LRM makes those keywords optional, and the nested form already supported it — only the top module scope did not.

The bug

```
module m(bus);
    inout [0:3] bus; electrical [0:3] bus;
    ground gnd; electrical gnd;
    genvar i;
    for (i = 0; i < 4; i = i + 1) begin          // no `generate` wrapper
        gcell c (bus[i], gnd);
    end
endmodule
```

`module_items` handled a `generate ... endgenerate` region but had no arm for a bare **for/if/case**, so the loop fell through to error recovery. In two module shapes the failure surfaced as unexpected token 'for':

- a module with a header bus port, and
- a module with no analog block.

Worse, when an analog block followed the loop, error recovery could resync on it and silently drop the entire `generate-for` (its instances vanished from the flattened netlist), and some shapes even hit a compiler panic. The `generate-wrapped` form (`generate for ... endgenerate`) always worked, which masked how common the bare form is (the LRM's own page-169 example uses it).

The fix

`parser/src/grammar/items/module.rs` — `module_items` now has `FOR_KW`, `IF_KW`, and `CASE_KW` arms that call `generate_for_tail/generate_if_tail/generate_case_tail` with `top = false` (no `endgenerate` of their own), producing the same `GENERATE_FOR/GENERATE_IF/GENERATE_CASE` nodes the nested and `generate-wrapped` forms produce. The existing `generate` elaboration (Enhancement-8/67) then unrolls them unchanged. One arm each; the `generate-wrapped` form is untouched.

Verification

`baregenerate_examples` (4/4, ngspice runtime pins): two modules using a bare module-level `generate-for` —

- `busgen` (header bus port, no analog block): each bus bit is tied to ground through a conductance scaled by the bit index; the four branch currents come out distinct and index-scaled, proving the loop was applied, not dropped;
- `divgen` (no analog block): four parallel two-section dividers give $i(v_p) = -2 \text{ mA}$ (it would be 0 if the loop had been dropped).

Full regression: 87 verify suites + 28 integration tests; `parser/hir` snapshot tests and the Enhancement-8/67 `generate` suite unchanged. Three LRM examples that used the bare form (`p169_1/p169_2 for`, `p172_1 if`) now parse and are re-pinned from the old unexpected token / bit-select errors to their honest `generate-elaboration` diagnostic (the loop bound / condition depends on a parameter — the Enhancement-67 scope boundary).

Enhancement-97 — clean diagnostic for a contribution to a ground branch

A crash fix: contributing to a branch that is entirely the **ground** reference — `V(gnd) <+ ..., V(gnd, gnd) <+ ..., I(gnd) <+ ...` — panicked the compiler with an internal error ("please open an issue"). It is now a clean, located diagnostic.

The bug

ground is the fixed 0 reference and has no DAE unknown, so during lowering its node resolves to None. A contribution whose branch is *only* ground reduces both endpoints to None, and `lower_contribute_unnamed_branch` (`hir_lower/src/stmt.rs`) hit its `(None, None) => unreachable!()` arm — an ICE — instead of any error:

```
module m(a);
  inout a; electrical a;
  ground gnd; electrical gnd;
  analog V(gnd) <+ 0.0;      // panic
endmodule
```

The fix

The check belongs in type validation, before lowering. `hir_ty`'s `ExprValidator` (`validation/body.rs`) already rejects a contribution to a port branch (`ContributeToPortFlow`); a sibling `ContributeToGround` check now fires for a *write* whose branch is all-ground — the single-node access (`V(gnd)`, where the node's `is_gnd` flag is set) and the two-node access (`V(gnd, gnd)`, both nodes ground). It reports a proper error, so lowering is never reached and the `unreachable!()` stays unreachable:

```
error: contribution to a ground node
|   V(gnd) <+ 0.0;
|   ^^^^^^ this branch is entirely 'ground'
= help: the potential of 'ground' is fixed at 0, so there is no unknown to
       contribute to; contribute to a real node instead
```

Probing `V(gnd)` (a read) is untouched, and a real node-to-ground branch (`V(a, gnd) <+ ..., V(a) <+ ...`) is unaffected — only an all-ground *write* is rejected.

Verification

- New UI snapshot test `contribute_to_ground` pins the three ground-branch errors (`V(gnd)`, `V(gnd, gnd)`, `I(gnd)`) and confirms the node-to-ground contribution on the same module is accepted.
- `groundcontrib_examples` (5/5): a valid node-to-ground source drives its node (`v(p) = 1.5`); `V(gnd) <+ 0` is rejected with the ground diagnostic and no ICE banner; `V(a, gnd) <+ 1` is not a false positive.
- Full regression: 87 verify suites + 28 integration tests; `hir` snapshot tests green.

Found while confirming Enhancement-96 (a malformed test drove the ground node); independent of the generate work.

Enhancement-98 — **pyplot** multi-panel subplots and style sheets

Two additions to the Enhancement-94/95 **pyplot** command: stacked subplots and matplotlib style sheets, both driven by set variables.

set pyplot_subplots=N — stacked panels

By default **pyplot** draws every trace on one axis. Setting **pyplot_subplots=N** lays the traces out as stacked subplots sharing the x-axis, **N** traces per panel:

```
set pyplot_subplots=1
pyplot v(in) v(a) v(b)          ; three stacked panels, one trace each
```

```
set pyplot_subplots=2
pyplot v(a) v(b) v(c) v(d)      ; two panels, two traces each
```

The number of panels is $\text{ceil}(\text{numtraces} / N)$; the x-label appears on the bottom panel, the title becomes the figure's subtitle, and each panel gets its own grid, log scaling, limits, and legend. **0** (or unset) is the original single-axis behaviour.

Note on **vs**. In **ngspice**, **vs** already selects the x-axis vector (**plot y vs x**), and **pyplot** goes through the same **plotit()** expression parser — so **vs** is *not* a panel separator. Multi-panel layout is chosen with **pyplot_subplots** instead.

set pyplot_style=<name> — matplotlib style sheets

pyplot_style applies a matplotlib style sheet to the whole figure, e.g. **ggplot**, **bmh**, **seaborn-v0_8**. The short name **dark** aliases matplotlib's **dark_background**:

```
set pyplot_style=dark
pyplot v(out)          ; dark theme
```

An unrecognised style name is ignored (wrapped in **try/except**) rather than aborting the plot.

Implementation

All in **ft_pyplot()** (**src/frontend/plotting/pyplot.c**), which already has the resolved trace list from **plotit()**:

- the figure is now always **plt.subplots(nrows, 1, sharex=True, squeeze=False)** (a uniform 2-D axes), with **nrows** computed from **pyplot_subplots**; each trace is assigned to axes **[i // N, 0]**;
- per-axis cosmetics (**ylabel/grid/log/limits/legend**) are emitted in a **for _ax in axes[:, 0]** loop; the x-label goes on axes **[-1, 0]** and the title on **fig.suptitle**;
- if **pyplot_style** is set, **plt.style.use(...)** is emitted before the figure (guarded).

Verification

pyplotpanel_examples (5/5): a multi-RC transient rendered with **pyplot_subplots=1** (three stacked panels), **pyplot_subplots=2** (two panels for four traces), the default single axis, and **pyplot_style=dark** — the generated scripts are checked for the right **plt.subplots(n, 1, ...)** and **plt.style.use('dark_background')**, and every case produces a valid PNG. Full regression: 89 verify suites + 28 integration tests. Purely additive.

Enhancement-99 — **pyplot** vector export formats (SVG/PDF) and figure size

Two more additions to the Enhancement-94/95/98 **pyplot** command: vector export formats (SVG and PDF alongside the existing PNG) and a figure size control, both driven by set variables.

set pyplot_terminal=svg|pdf — vector output

Enhancement-94 added `set pyplot_terminal=png` for headless (matplotlib Agg) rendering to `<file>.png`. Enhancement-99 extends the same mechanism to the two vector formats matplotlib writes natively:

```
set pyplot_terminal=svg
pyplot out v(in) v(out)          ; writes out.svg
```

```
set pyplot_terminal=pdf
pyplot out v(in) v(out)          ; writes out.pdf
```

Any of `png`, `svg`, `pdf` (or the `.../quit` spellings) selects headless rendering; the file extension follows the format. Without `pyplot_terminal`, **pyplot** still opens an interactive matplotlib window as before.

set pyplot_figsize="W,H" — figure size

`pyplot_figsize` sets the figure size in inches, passed straight to matplotlib's `plt.subplots(..., figsize=(W, H))`:

```
set pyplot_terminal=svg
set pyplot_figsize="8,3"
pyplot out v(in) v(out)          ; an 8x3-inch out.svg
```

Quote the value. ngspice's `set` truncates an unquoted value at the first comma (`set pyplot_figsize=8,3` stores just 8), so the pair must be quoted: `set pyplot_figsize="8,3"`. A space or `x` separator is also accepted inside the quotes (`"8 3"`, `"8x3"`). An unparseable or non-positive value is ignored and matplotlib's default size is used.

The figure size applies to the single-axis and the Enhancement-98 multi-panel (`pyplot_subplots`) layouts alike.

Implementation

All in `ft_pyplot()` (`src/frontend/plotting/pyplot.c`):

- `pyplot_terminal` is parsed into a format string `fmt` (`png/svg/pdf`) and a `hardcopy` flag; `hardcopy` gates the `matplotlib.use('Agg')` import, the `fig.savefig(<file> + '<fmt>')`, `dpi=100` call (matplotlib picks the writer from the extension), and the synchronous (non-background) run;
- `pyplot_figsize` is read with `sscanf(..., "%lf%*[ ,xX]%lf", &figw, &figh)`; when two positive values parse, the `plt.subplots(...)` call gains a `figsize=(figw, figh)` argument.

Verification

`pyplotexport_examples` (5/5): an RC transient rendered with `pyplot_terminal=svg` + `pyplot_figsize="8,3"`, `pyplot_terminal=pdf`, and `pyplot_terminal=png` — the outputs are checked by magic bytes (`<?xml, %PDF, \x89PNG`), and the generated scripts are checked for `figsize=(8, 3)` when set and no `figsize=` when unset. Full regression: 90 verify suites + 28 integration tests. Purely additive.

Enhancement-100 — the first hundred: a milestone audit & retrospective

Enhancement-100 is a checkpoint, not a feature. Where Enhancements 1–99 each added or fixed something in the compiler or the simulator, E-100 adds no compiler or simulator code at all. Instead it does three things: it re-verifies the whole tree from a clean state, it audits the repository for provenance and link hygiene, and it steps back to look at the arc of the first hundred enhancements. Every number below was measured on the tree as it stands today (2026-07-08), not estimated.

By the numbers

Metric	Value
Feature / fix enhancements	99 (E-1 ... E-99)
GitHub releases	95 (a few early enhancements shipped grouped)
Committed example suites	90 (<code>verify_*.py</code>)
Pinned assertions across those suites	635
OpenVAF integration tests	28
Committed <code>.osdi</code> models under <code>examples/</code>	231
Enhancements that touched <code>openvaf-r</code>	78
Enhancements that touched <code>ngspice</code>	27
Commits on <code>main</code>	306
Span	2026-06-26 → 2026-07-08 (~12 days)

(78 + 27 exceeds 99 because a handful — e.g. the OSDI-parameter warning of [E-93](#) — changed both sides in one fold.)

Audit results

Regression (today, from the committed binaries). All 90 verify suites pass, 0 fail, and the OpenVAF integration suite is 28/28. This is the same suite set that has gated every fold; running it end-to-end at the hundred-mark confirms no suite has silently rotted.

Provenance. Zero git-tracked files under `examples/` contain an absolute path (`/Users/...`, `/home/...`, `/private/tmp/...`). The 231 committed `.osdi` binaries carry only bare or repo-relative names in their baked-in provenance strings, per the standing portability rule — a machine that clones the repo sees nothing about the machine that built it. (Absolute paths do exist in *untracked* local build scratch — `.build/`, `benchmark/cir/`, `__pycache__` — but those are gitignored and never leave the build host.)

Links. All 216 relative navigation links in the README resolve, every one of the 99 `enhancements_doc/Enhancement-N.md` files is present on disk, and a sweep of ~991 relative links across all 109 Markdown files found no broken file references. (The handful the sweep flagged were Verilog signal lists such as `V(out, gnd)` inside inline code — matched by the link regex, not real links.)

The arc, by theme

The hundred enhancements fall into a few broad families:

- Core Verilog-A language. The bulk of the early and middle work: transport delay ([E-1](#)), indirect branch assignment ([E-2](#)), bus nets and ports ([E-3](#)), `laplace_*/zi_*` filters ([E-4](#)), module hierarchy ([E-5](#)), arrays and multi-dimensional arrays (E-14/15), `$table_model` and its multi-dimensional and cubic-spline forms (E-16/17/22/40), `defparam`, `paramset`, generate constructs, concatenation, the integrator family, and a long tail of system functions and operators.

- Correctness & ICE fixes. Compiler crashes and wrong answers rooted out one at a time — the comment-at-EOF lexer hang (E-35), the bus-port node-ordering bug (E-90), the contribute-to-ground panic (E-97), integer-state slot types (E-32), and many more.
- ngspice simulator features. The `.dc @inst[param]` sweeps, RF and Touchstone I/O (E-62/63/64/72), Monte-Carlo, lifecycle/leak fixes, the zero-warning rebuild (E-77), and the whole `pyplot` command family (E-94/95/98/99).
- Systematic audits. Deliberate probe-sweeps over a construct class that then fix whatever real gap surfaces: operators (E-37/38/61), the preprocessor (E-65), generate (E-67), display tasks (E-71), opvars (E-69), the 231-example LRM-2023 sweep (E-84).
- Validation, benchmarks & docs. OSDI-vs-built-in benchmarks (E-74/79), static and dynamic physics cross-checks (E-57/75/80), the user handbook (E-73), and the change-log reports.

Recurring engineering lessons

A few patterns showed up often enough to be worth naming:

1. "Scaffolded but unwired." More than once, a feature's machinery was already present in the tree but never connected at some node-kind or signature boundary, so it silently did nothing — the macro-recursion guard (E-65), derived natures (E-39), and others. The fix was usually small; *finding* it was the work.
2. Signature/builtin tables are a defect magnet. The argument-type tables for built-in functions were behind a recurring class of bugs (E-33, E-40, E-47, E-49) — an off-by-one or a too-eager `resize()` there corrupts a whole family of calls.
3. Textual pre-passes are a clean extension seam. Several late language features (name-then-range decls, legacy/bare generate, source-location macros) were implemented as ordered normalization passes in `hir/src/elaborate.rs` rather than deep parser surgery (E-85/88/89/91/96).
4. The OSDI ABI is a living contract. Runtime features repeatedly required bumping the OSDI descriptor ABI (0.4 → 0.7) and regenerating `.osdi` in lockstep with ngspice's loader (E-45/51/54/93) — a reminder to keep the two halves of the toolchain versioned together.
5. Every enhancement ships a pinned example. The single most valuable habit is that each enhancement lands with a `verify_*.py` suite that becomes a permanent regression pin. The 90-green run above is the compounding interest on that discipline.

What's next

The most concrete deferred item is `$table_model` with runtime array data arguments (the grid/values passed as array variables rather than inline or from a file), explicitly held back from E-91. Beyond that, the audit-and-fix loop still has surface area: the LRM sweep graduated 231 examples, but AMS-level constructs remain out of scope by design.

The first hundred are done and green. Onward.

Enhancement-101 — `$clog2` correctness (arity + value)

A probe sweep over under-exercised Verilog-A constructs (the E-59/E-84 methodology) turned up a two-layer bug in the `$clog2` system function. Both layers are fixed here.

What `$clog2` should do

`$clog2(n)` is the IEEE-1800 system function that returns the ceiling of the base-2 logarithm of its single argument — the number of bits needed to index n distinct values. Per the standard, `$clog2(0)` is 0. So `$clog2(1) = 0`, `$clog2(16) = 4`, `$clog2(17) = 5`, `$clog2(1024) = 10`.

The two bugs

1. Wrong arity — every call rejected. `$clog2` was aliased to a 2-argument integer signature (`INT_MATH_2`), so *any* call was rejected at type-check time:

```
error: invalid argument count: expected 2 arguments but found 1
    b = $clog2(W);
      ~~~~~^ expected 2 arguments
```

The builtin id, the lowering (`hir_lower`), and the MIR `Clog2` opcode were all present and expected a single argument — only the `hir_ty` signature table was wrong. This is the recurring “scaffolded but mis-wired at the signature table” pattern. The fix adds a 1-arg integer signature `INT_MATH_1` and points `CLOG2` at it (`openvaf/hir_ty/src/builtin.rs`).

2. Wrong value — off by one on powers of two. With the arity fixed, a second bug surfaced (it had been fully masked, since no call ever compiled): all three backends computed

$$clog2(n) = 32 - clz(n) = bit_width(n) = floor(\log_2 n) + 1$$

That is the *bit width* of n , which equals $ceil(\log_2 n)$ only when n is not a power of two. For exact powers it is one too large:

n	correct <code>\$clog2</code>	old result
1	0	1
16	4	5
1024	10	11
33	6	6 ✓

The correct identity is $ceil(\log_2 n) = bit_width(n-1)$ for $n \geq 2$, and 0 for $n \leq 1$. The fix applies this in all three places that implement the `Clog2` opcode:

- `openvaf/mir_interpret/src/lib.rs` (interpreter),
- `openvaf/mir_opt/src/const_eval.rs` (constant folding — the literal path),
- `openvaf/mir_llvm/src/builder.rs` (LLVM codegen — the runtime, parameter-dependent path). Here the `ctlz` intrinsic is now emitted with `is_zero_poison = false` so that `ctlz(0) = 32` is well defined (making the $n = 1$ case, where $n-1 = 0$, evaluate to 0), with a select guarding $n < 1 \rightarrow 0$.

Verification

`clog2_examples (13/13)`: `clog2_demo.va` exposes `$clog2` results as operating-point variables and is read back in ngspice via `.op`. The constant-folded literals `$clog2(1,2,3,4,7,8,16,17,1024)` and the runtime parameter path `$clog2(N)` for $N \in \{16, 33, 1\}$ all match $ceil(\log_2 n)$ exactly — in particular the powers-of-two cases that the old code got wrong.

The wider probe battery (~35 constructs: severity tasks \$info/\$warning/ \$error, ddx, \$limit, hypot/atan2/limexp, aliasparam, timer/ cross/above events, named-block disable, switch branches, \$temperature/ \$vt, analog-function recursion, ...) otherwise compiled or produced the correct diagnostic — \$clog2 was the one real defect found. Full regression: all verify suites plus the OpenVAF integration tests remain green.

Enhancement-102 — name-then-range array-valued parameters

Enhancement-101's probe sweep surfaced a small asymmetry in how array dimensions may be written on a parameter declaration. This enhancement closes it.

The gap

Verilog-AMS lets an unpacked-array dimension be written either *before* the name (type-then-range) or *after* it (name-then-range). `openvaf-r` accepted both forms for local variables, nets, and ports (Enhancement-18/89/91), but for parameters only the type-then-range form parsed:

```
parameter real [0:2] c = '{1.0, 2.0, 3.0}; // worked
parameter real c[0:2] = '{1.0, 2.0, 3.0}; // error: unexpected token '['
```

Since the more common spelling in practice is name-then-range (the dimensions sit next to the name they belong to), the rejection was a real, if minor, gap.

The fix

The array-variable grammar (`var()`) already accepted name-then-range dimensions; the parameter grammar (`parameter()`) did not — it expected = immediately after the name. The fix mirrors `var()`:

- Parser (`openvaf/parser/src/grammar/items.rs`): `parameter()` now consumes a `[msb:lsb]` run after the name before the `=`, so the dimensions become `Range` children of the `PARAM` node. `Param` gains a `widths()` accessor (`veriloga.ungram + generated AST`).
- Item tree (`openvaf/hir_def/src/item_tree/lower.rs`): `lower_param` now resolves the width set per name — the shared decl-level dims (type-then-range) if present, otherwise this name's own dims (name-then-range). Everything downstream — element expansion, per-element OSDI parameters, the initializer-length diagnostic (Enhancement-43) — is the existing Enhancement-14/15 machinery, reused unchanged.

Because the width is now resolved per name, a multi-name declaration may mix widths, and multi-dimensional names work too:

```
parameter real a[0:1] = '{10.0, 20.0}, b[0:2] = '{30.0, 40.0, 50.0}, g = 7.0;
parameter real m[0:1][0:1] = '{ '{1.0, 2.0}, '{3.0, 4.0}};
localparam integer w[0:1] = '{3, 5};
```

Verification

`paramarray_examples` (11/11): `paramarray_demo.va` declares single-name, multi-name (mixed widths), multi-dimensional, and integer-`localparam` name-then-range arrays, exposing element values as operating-point variables read back in `ngspice`. It checks that (a) the file compiles, (b) every element default resolves to its initializer value, (c) a per-element `.model` override (`c[1]=99`) changes only that element (the OSDI per-element parameters are intact), and (d) the type-then-range form still compiles. Full regression: all verify suites plus the OpenVAF integration tests remain green.

Enhancement-103 — `ceil()` of a runtime argument (missing LLVM intrinsic)

A wider probe sweep (the E-101 methodology, extended with a runtime-value battery) turned up a compiler crash: `ceil()` applied to any non-constant argument aborts code generation.

The bug

The LLVM code generator lowers `ceil(x)` to the `llvm.ceil.f64` intrinsic, but that intrinsic was never registered in the compiler's intrinsic table (`openvaf/mir_llvm/src/intrinsics.rs`). When codegen asked for it, the lookup returned `None` and the compiler panicked:

```
internal error: entered unreachable code: intrinsic llvm.ceil.f64 not found
```

Its sibling `llvm.floor.f64` was registered, so `floor(x)` compiled fine — a textbook “scaffolded but unwired” gap. The crash was fully masked whenever the argument was a compile-time constant (`ceil(2.1)`), because that folds to a value long before codegen; it only fired for a genuinely runtime argument, e.g. `ceil` of a parameter or a node voltage:

```
parameter real a = 2.1;
real y;
analog y = ceil(a);    // crashed the compiler (llvm.ceil.f64 not found)
```

The fix

One line — register the missing intrinsic next to `floor`:

```
ifn!("llvm.floor.f64", fn(t_f64) -> t_f64);
ifn!("llvm.ceil.f64",  fn(t_f64) -> t_f64);    // Enhancement-103
```

An audit of every intrinsic name requested by the code generator against the registered table confirmed `llvm.ceil.f64` was the *only* missing one — all other math builtins (`sqrt`, `exp`, `ln`, `log10`, `pow`, `sin/cos/tan`, the hyperbolic and inverse-hyperbolic family, `hypot`, `atan2`, `fabs`, `ctlz`, `lround`) resolve.

Verification

`ceil_examples (8/8)`: `ceil_demo.va` takes `ceil` of module parameters (the runtime path that used to crash) and reads the results back in `ngspice` — `ceil(2.1)=3`, `ceil(2.0)=2`, `ceil(-2.7)=-2`, `ceil(-0.5)=0`, `ceil(5.0)=5`, with `floor` cross-checked on the same inputs. The wider runtime-value battery behind this enhancement (37 math / integer / bit expressions checked against analytic values) otherwise matched exactly — `ceil` was the one real defect found. Full regression: all verify suites plus the OpenVAF integration tests remain green.

Enhancement-104 — `$rtoi` / `$itor` real↔integer conversion functions

A second, wider gap-hunt round (static-value + dynamic transient batteries) confirmed the math builtins, integer/bit operators, and the time-domain operators (`ddt`, `idt`, `absdelay`) all return correct values. What it *did* surface is a missing pair of standard conversion functions.

The gap

Verilog / Verilog-AMS provides two explicit conversion system functions:

- `$rtoi(real)` → integer, converting by truncating toward zero;
- `$itor(integer)` → real.

openvaf-r supported the *implicit* conversions (assigning a real to an integer variable, and vice versa), but the explicit functions were unknown:

```
error: '$rtoi' was not found in the current scope
```

They matter because `$rtoi` is not interchangeable with the implicit cast: an implicit real→integer assignment rounds to nearest, whereas `$rtoi` truncates. So `$rtoi(3.9)` is 3 (not 4) and `$rtoi(-3.9)` is -3 (not -4). Compact models that want truncation had no correct way to express it.

The implementation

Two new builtins wired through the standard path — the interned names (`syntax/src/name.rs`), the `BuiltIn` enum + scope registration (`hir_def/src/builtin.rs`), the type signatures (`hir_ty`), and the MIR lowering (`hir_lower/src/expr.rs`):

- `$itor` lowers to the existing integer→real cast (`ifcast`) — exact.
- `$rtoi` must truncate toward zero, which the rounding `FIcast` cast does not do. It lowers to `ficast( (x < 0) ? ceil(x) : floor(x) )`: `floor/ceil` select the toward-zero neighbor, producing an exact integer-valued real that the final cast carries through unchanged. This reuses only existing MIR ops, so both the runtime (LLVM) and the constant-folding paths compute it.

Verification

`convert_examples(9/9)`: `convert_demo.va` takes `$rtoi` of module parameters (the runtime path) and of a `localparam` (the constant-folding path), and `$itor` of an integer. It checks the toward-zero truncation on positive and negative inputs — `$rtoi(3.9)=3`, `$rtoi(-3.9)=-3`, `$rtoi(-3.2)=-3`, `$rtoi(5.0)=5` (a rounding cast would give 4/-4), that `$rtoi(9.6)` in a `localparam` const-folds to 9, and that `$itor(7)*0.5 = 3.5` (proving `$itor` yields a real, not an integer). The gap-hunt batteries behind this enhancement (static values across `trig` / `conversion` / `based literals`, and a transient `ddt`/`idt`/`absdelay` battery) otherwise matched their analytic values exactly. Full regression: all verify suites plus the OpenVAF integration tests remain green.

Enhancement-105 — `$sscanf` / `$fscanf` honour the format conversion base

Gap-hunt round 3 extended the runtime-value batteries into string and file I/O. A `$sscanf` value battery — parsing known strings and checking the parsed results — turned up a real bug: the `scanf` functions ignore the format string's conversion base.

The bug

`$sscanf` / `$fscanf` parse each field by the destination variable's *type* (integer via `strtol`, real via `strtod`, string as the next token) and never look at the format string. For integers the runtime used `strtol(p, &end, 0)` — base 0, which auto-detects the base from the *input's* own prefix. So the conversion character had no effect:

call	expected	got (before)
<code>\$sscanf("ff", "%h", x)</code>	255	0 (needs "0xff")
<code>\$sscanf("17", "%o", x)</code>	15	17 (parsed as decimal)
<code>\$sscanf("1010", "%b", x)</code>	10	1010 (parsed as decimal)
<code>\$sscanf("42", "%d", x)</code>	42	42 ✓

Only the default (decimal / real / string) cases happened to work.

The fix

The lowering now reads the format string (a compile-time literal in practice) and picks the integer base per field from its conversion character, then dispatches to a base-specific runtime scanner:

- **hir_lower** (expr.rs): `lower_scanf` receives the format expression, extracts the ordered conversion characters (`scanf_conversion_chars`), and maps each integer field to `ScanKind::IntHex` (`%h/%H/%x/%X`), `IntOct` (`%o/%O`), `IntBin` (`%b/%B`), or the default `Int`. A non-literal format falls back to the previous type-driven behavior.
- **hir_lower** (callbacks.rs) + **osdi** (compilation_unit.rs): the new `ScanKind` variants map to new runtime symbols `osdi_scan_hex/oct/bin`.
- **osdi/stdlib.c**: the three new scanners call `strtol` with base 16 / 8 / 2, so the format's base is honoured (e.g. "ff" → 255 without a 0x prefix). `osdi_scan_int` keeps its base-0 auto-detect for `%d` and untyped fields, so existing behavior is unchanged. The C spelling `%x` is accepted as an alias for the Verilog `%h`.

Verification

`sscanf_examples` (7/7): a device parses fixed input strings and exposes the parsed values as operating-point variables. It checks `%h ff→255, %o 17→15, %b 1010→10, %d 42→42` (each wrong under the old base-0 behavior), a repeated conversion (`%h %h → 160, 255`), and a mixed integer/real parse (`%d %g → 7, 8.5`) with the correct match count. The wider gap-hunt batteries behind this (string/file-function compile probes, and a `$sscanf` value battery over `%d/%g/%e/%h/%o/multi-field`) otherwise matched their expected results. Full regression: all verify suites plus the OpenVAF integration tests remain green.

Enhancement-106 — string relational comparison (<, <=, >, >=)

Gap-hunt round 4 extended the runtime-value batteries into output formatting, `ddx`, `$table_model` interpolation and extrapolation, `laplace` AC response, and integer/division edge cases — all of which checked out exactly. The one genuine inconsistency it surfaced is in the string comparison operators.

The gap

String equality (`==` / `!=`) works, but the relational operators (`<`, `<=`, `>`, `>=`) rejected string operands:

```
if (mode < "low") ...    // error: typed mismatch invalid function arguments
```

Equality already carried a string signature (`STR_EQ`); the relational operators were still numeric-only. Since ordering strings lexicographically is a well-defined, commonly-useful operation (comparing corner/mode names, sorting), this was a real hole in the comparison surface.

The implementation

The relational operators now accept two strings and compare them lexicographically, reusing the same lightweight `stdlib`-callback pattern as the file/string helpers (no new MIR opcode):

- **hir_ty**: the relational operators' signature set becomes `RELATIONAL_COMPARISON = [INT, REAL, STR]` (the string signature appended last so the int/real indices stay stable); `STR_REL` names the string branch.
- **hir_lower**: when the resolved signature is `STR_REL`, `a <op> b` lowers to `strcmp(a, b) <op> 0` via a new `StrCmp` callback and an integer compare against zero.
- **osdi** (`compilation_unit.rs` + `stdlib.c`): the `StrCmp` callback binds to a new `osdi_strcmp` runtime function (`strcmp`, `NULL` treated as empty).

Numeric relational comparison and string equality are untouched.

Verification

`strcmp_examples` (8/8): a device evaluates `"abc" < "abd"` (`=1`), `"abd" > "abc"` (`=1`), `"abc" <= "abc"` (`=1`), `"abc" >= "abd"` (`=0`), `"abc" < "abc"` (`=0`), `"abc" == "abc"` (`=1`, equality still works), and uses a string relational as an `if` condition (`"high" < "low" → true`). The wider gap-hunt batteries behind this enhancement — `$strobe` output formatting across all conversions, `ddx` partial derivatives, `$table_model` interpolation and extrapolation, and the `laplace_nd` AC transfer function — matched their analytic expectations exactly. Full regression: all verify suites plus the OpenVAF integration tests remain green.

Enhancement-107 — the **\$fgetc** file-input function

Gap-hunt round 5 pushed the runtime-value batteries into 2-D `$table_model` bilinear interpolation, `white_noise` output spectra, file write/read round-trips, and file positioning (`$ftell/$fseek/$rewind`) — all of which matched their analytic expectations exactly. The compiler is robust across that surface. The one clear in-scope gap it confirmed is a missing member of the file I/O family.

The gap

Enhancement-11 gave `openvaf-r` a full file I/O family — `$fopen`, `$fclose`, `$fgets`, `$fscanf`, `$fdisplay/$fwrite`, `$feof`, `$ferror`, `$rewind`, `$fseek`, `$ftell`, `$fflush` — but not **`$fgetc`**, the standard IEEE-1364 single-character read:

error: '`$fgetc`' was not found in the current scope

`$fgetc(fd)` reads one character and returns its integer code, or `-1` (EOF) at end of file or on a bad descriptor.

The implementation

`$fgetc` is another `fd → int` file operation, so it slots straight into the existing `FileOp` machinery — no new callback shape:

- **syntax/hir_def**: the interned name, the `BuiltIn::fgetc` enum entry (= 115), and its scope registration.
- **hir_ty**: the signature is `BASIC_IO (integer → integer, impure — like $ftell/$feof)`, added to the builtin table.
- **hir_lower**: `BuiltIn::fgetc` lowers to `FileOp::Getc`, whose runtime symbol is `osdi_fgetc`; the OSDI backend already binds `FileOp` variants generically by name.
- **osdi/stdlib.c**: `osdi_fgetc` looks up the descriptor and returns `fgetc(f)` (`-1` for a NULL descriptor).

Verification

`fgetc_examples (6/6)`: a device reads a fixed text file character by character — the first two characters come back as their ASCII codes, and a `while` loop over `$fgetc` counts and sums the remaining characters, terminating on the `-1` EOF sentinel. The wider gap-hunt batteries behind this enhancement — 2-D bilinear `$table_model` (exact on a linear surface), `white_noise` output spectrum (matching $\sqrt{S \cdot R}$ to the loading), `$fdisplay/$fscanf` round-trips across `%d/%g/%h/%o/%b`, and `$ftell/$fseek/$rewind` positioning — all matched their expected values. Full regression: all verify suites plus the OpenVAF integration tests remain green.

Enhancement-108 — the **\$ungetc** file-input function

Gap-hunt round 6 pushed the runtime-value batteries into `$random/$dist_*` statistics, module instance arrays, and the `idt(ddt(x))` integrator $\leftrightarrow$ differentiator round-trip — all of which behaved correctly:

- `$dist_uniform/$rdist_*` stay in range, are deterministic (same seed $\rightarrow$ same value), and change with the seed; distinct call sites draw from independent streams (the Enhancement-10/66 Monte-Carlo design).
- `rsub xarr[0:2] (p, n)` correctly instantiates three parallel devices (measured $G = 3$ vs $G = 1$ for a single instance).
- `idt(ddt(V))` reconstructs V to machine precision (max error 0.0).

The one clear in-scope gap it surfaced is the companion to Enhancement-107's `$fgetc`.

The gap

`$fgetc` (Enhancement-107) reads one character; its natural partner `$ungetc(c, fd)` — push a character back so the next read returns it — was missing ('`$ungetc`' was not found). It is the standard IEEE-1364 one-character look-ahead used by hand-written parsers.

The implementation

`$ungetc` is a two-argument file operation, so it extends the existing `FileOp` machinery (the same path as `$fseek`, which already takes three arguments):

- **syntax/hir_def**: the interned name, `BuiltIn::ungetc (= 116)`, and its scope registration.
- **hir_ty**: a two-integer signature `UNGETC(integer, integer)  $\rightarrow$  integer (impure)`, plus its `BUILTIN_INFO` entry.
- **hir_lower**: `BuiltIn::ungetc` lowers to `FileOp::Ungetc (num_args = 2)`, runtime symbol `osdi_ungetc`.
- **osdi/stdlib.c**: `osdi_ungetc(c, fd)` looks up the descriptor and calls `ungetc(c, f)`; the argument order matches the Verilog `$ungetc(c, fd)` call (c first).

Verification

`ungetc_examples (6/6)`: a device reads a character, `$ungetc`s it, and confirms the next `$fgetc` returns the same character (and that `$ungetc` returns the pushed character). It then performs the classic one-character look-ahead — accumulating the file's leading decimal digits into an integer ("`4271;...`" $\rightarrow$ `4271`) and `$ungetc`-ing the first non-digit (`;`) so it stays in the stream. The wider gap-hunt batteries behind this enhancement (`$random/$dist_*` statistics, instance arrays, `idt(ddt(x))` round-trip) all behaved correctly. Full regression: all verify suites plus the OpenVAF integration tests remain green.

Enhancement-109 — **noise_table** / **noise_table_log** interpolation per the LRM

Gap-hunt round 7 turned to event runtime semantics and noise-spectrum shapes. The event battery checked out exactly — a `timer(0, 100μs)` fires 16 times in a 1.55 ms transient, `cross(V, +1)` counts the two rising crossings of a 1 kHz sine, and `last_crossing` reports the 1.0000 ms crossing time — and `flicker_noise` reproduces its 1/f law to 7 digits. The noise-table battery, however, found a real LRM violation in *both* table forms.

The bugs

The LRM is precise about the two interpolation rules:

- **noise_table** (LRM 4.6.4.3): “*performs piecewise linear interpolation*” of the tabulated power over frequency, clamping to the endpoint powers outside the tabulated range.
- **noise_table_log** (LRM 4.6.4.4): same (*f*, *p*) input as `noise_table` (frequencies in Hz), but interpolated log-log: $P = 10^{(\log_{10} p_1 + (\log_{10} p_2 - \log_{10} p_1) \cdot (\log_{10} f - \log_{10} f_1) / (\log_{10} f_2 - \log_{10} f_1))}$.

`openvaf-r` (from Enhancement-9) violated both. `NoiseTable::new` log10-ed the frequency axis of a plain `noise_table` and the runtime interpolator keyed on `log10(freq)` — so the power was interpolated linearly over log-frequency (a lin-log hybrid). And `noise_table_log` stored its pairs raw against that same `log10(freq)` key — effectively expecting the *input* frequencies to already be log10 values, and interpolating the raw power rather than its logarithm. With a table {10 Hz: 1e-12, 1 kHz: 3e-12}:

form	at 100 Hz, LRM	measured (before)
<code>noise_table</code>	1.182e-12 (linear in <i>f</i>)	2e-12 (linear in log <i>f</i>)
<code>noise_table_log</code> ({10: 1e-12, 1k: 1e-16})	1e-14 (log-log)	1e-12 — flat (mis-keyed input clamped to the first point)

The fix

Two files, keeping the existing unrolled piecewise-linear runtime interpolator and changing only the coordinate systems:

- **hir_lower/callbacks.rs** (`NoiseTable::new`): a plain `noise_table` now stores (*f*, *p*) raw; `noise_table_log` stores (`log10 f`, `log10 p`).
- **osdi/load.rs** (`build_noise_table_interp`): gains the log flag — the lookup key is the raw frequency for the linear form and `log10(freq)` for the log form, and the log form’s interpolated `log10(P)` is mapped back with $10^v = \exp(v \cdot \ln 10)$. Endpoint clamping is unchanged in both forms.

The Enhancement-9 `noise_examples` suite — which had pinned the nonconformant behaviour (its reference used log-*f* interpolation, and its `_log` deck fed a log10-frequency column) — was updated to the LRM laws: the `_log` data file now carries frequencies in Hz, and the references implement linear-in-*f* and log-log respectively (all four spectra pass at ≤4e-9 relative error, and the two forms agree at the tabulated nodes).

Verification

`noisetable_examples` (9/9): a 1 mS device injects each table form, and the `.noise` output spectrum is checked against the closed-form laws — the linear form at 10/100/1000 Hz (at 100 Hz the LRM’s 1.1818e-12, not the 2e-12 the old lin-log gave), the log-log form at the same points (1e-14 at the log midpoint), and endpoint clamping a decade outside the range on both sides. Full regression: all verify suites (including the corrected `noise_examples`) plus the OpenVAF integration tests remain green.

Enhancement-110 — **errpreset**: coordinated accuracy presets for ngspice

A SPICE-class simulator's accuracy and convergence are governed by roughly eight interacting options — `reltol`, `abstol`, `vntol`, `chgtol`, `trtol`, the `gmin`- and source-stepping counts, and the DC iteration limit `itl1`. Setting them well means understanding how they trade off, and setting them *consistently* is easy to get wrong. Commercial tools solve this with a single knob: Spectre's `errpreset=conservative|moderate|liberal` selects one validated, coordinated combination. This enhancement adds the same control to ngspice.

```
.option errpreset=conservative ; accurate / robust, slower
.option errpreset=moderate    ; ngspice's historical defaults (backward compatible)
.option errpreset=liberal     ; fast, looser accuracy
```

What each preset sets

`moderate` is exactly ngspice's existing default set, so the feature is fully backward-compatible — a netlist that does not mention `errpreset` behaves precisely as before. `conservative` tightens every tolerance, tightens the LTE control (`trtol`), and adds DC-robustness (more `gmin`/source steps, a higher iteration limit). `liberal` loosens the tolerances and the LTE control for speed.

option	conservative	moderate (= default)	liberal
<code>reltol</code>	1e-4	1e-3	1e-2
<code>abstol</code>	1e-13	1e-12	1e-10
<code>vntol</code>	1e-7	1e-6	1e-4
<code>chgtol</code>	1e-15	1e-14	1e-12
<code>trtol</code>	1	7	20
<code>srcsteps</code> / <code>gminsteps</code>	10 / 10	1 / 1	1 / 1
<code>itl1</code>	200	100	100

The integration method (`trapezoidal`) and `maxord(2)` are not changed by the preset: switching to Gear would trade accuracy for stability, which does not belong in a "conservative = more accurate" knob. The presets vary only the tolerances, the LTE tightness, and DC-convergence aids. (The values live in one function and are easy to retune later.)

Explicit options always win — regardless of order

The subtle part is coexistence with individual options. `.option errpreset=liberal reltol=1e-4` must keep the tight `reltol` — and so must `.option reltol=1e-4 errpreset=liberal`, even though `.options` are parsed in text order. This is handled without any deferred-apply machinery:

- each individual tolerance case records a bit in a new `TSKtolGiven` mask when the user sets that option explicitly;
- `errpreset` writes only the fields whose bit is clear.

So in either order the explicit value survives: if the explicit option is parsed first, its bit is set and `errpreset` skips that field; if `errpreset` is parsed first, it sets the field and the later explicit option simply overwrites it. The verify suite checks both orders give an identical result.

Files changed

All changes are purely additive (82 insertions, 0 deletions) and confined to ngspice's option-handling layer — no numerical-core or device code is touched.

File	What changed
ngspice-46/ src/include/ ngspice/ optdefs.h	Added the OPT_ERRPRESET option code to the enum, and eight ERRP_* bit constants (one per tolerance field the preset touches) used to track which options the user set explicitly.
ngspice-46/ src/include/ ngspice/ tskdefs.h	Added unsigned int TSKtolGiven to the task struct — the bitmask of explicitly-given tolerance options (the override guard).
ngspice-46/ src/ spicelib/ analysis/ cktsopt.c	Added the static helper ckt_apply_errpreset() (the conservative/moderate/liberal value table + the "only write un-given fields" logic); added the case OPT_ERRPRESET that maps the string to a preset (prefix match on cons/mod/lib, warns on anything else); set the ERRP_* given-bit in each of the eight individual option cases (RELTOL, ABSTOL, VNTOL, TRTOL, CHGTOL, ITL1, SRCSTEPS, GMINSTEPS); and added the { "errpreset", OPT_ERRPRESET, IF_SET IF_STRING, ... } entry to the OPTtbl keyword table so .option errpreset=... is recognized.
ngspice-46/ src/ spicelib/ analysis/ cktntask.c	Initialised TSKtolGiven = 0 in both task-creation paths (the copy-from-defaults path and the hard-coded-defaults path) — a fresh task has no user-given options yet.

Verification

[examples/errpreset_examples/](#) (9/9), driving an adaptive-stepping RC transient through the committed ngspice:

- the three presets order the accepted time-point count conservative (323) > moderate (165) ≥ liberal (145) — tighter tolerances resolve the pulse edges more finely;
- **moderate** reproduces the no-errpreset default exactly (165 = 165) — backward compatibility;
- an explicit reltol=1e-4 overrides the preset and gives the same result in either **.options** order (153 = 153), and differs from plain liberal (145);
- a preset can be loosened by an explicit value (errpreset=conservative reltol=1e-2 drops from 323 to 169);
- an unknown preset warns (unknown errpreset '...') and the run still completes.

Regression: a representative slice of the existing suites (operator, simctrl, noise, clog2, ceil) pass unchanged against the rebuilt ngspice, confirming the additive option handling changes nothing for netlists that do not use errpreset.

Enhancement-111 — globalized (damped) Newton via an Armijo line search

ngspice solves each DC operating point with Newton's method, which converges only *locally*: from a poor starting point the full Newton step can overshoot and the iteration stall or diverge. Commercial simulators guard against this with a globalized Newton — a line search that shrinks the step to guarantee progress on a *merit function*, the residual norm $\|F\|$. ngspice had every other convergence aid (per-device junction limiting, node damping, gmin/source-stepping homotopy) but not a principled residual-based line search, because it had no way to compute $\|F\|$ mid-solve: its convergence test is purely iterate-based ($|\Delta x| < \text{tol}$).

This enhancement adds both: the residual merit, and an Armijo backtracking line search built on it.

```
.option linesearch      ; enable the line search (OFF by default)
```

The residual merit ngspice lacked

In modified nodal analysis, after CKTload builds the Jacobian G and the right-hand side b at the point x , the Newton (KCL) residual is exactly $F(x) = G \cdot x - b$ — the net current mismatch at every node. It is computed here on the just-loaded, unfactored matrix via ngspice's own sparse matrix-vector product `SMPmultiply` (which requires an unfactored matrix). This is a genuinely new, reusable capability: the tolerance-weighted $\|F\|$ that a globalized Newton — or future work like pseudo-transient continuation — needs.

The line search

When enabled and the iteration is not converging, the full Newton step $x_k \rightarrow x_{full}$ is damped by the largest $\lambda \in \{1, \frac{1}{2}, \frac{1}{4}, \dots, 1/64\}$ giving a sufficient decrease of $\|F\|$ (the Armijo condition, $c = 1e-4$). Each trial re-loads the devices at $x_k + \lambda \cdot d$ and re-evaluates $\|F\|$. At or near a solution the full step already reduces $\|F\|$, so $\lambda = 1$ is accepted on the first trial and the run is result-neutral; backtracking only engages on genuine overshoot.

Three subtleties had to be solved to make this correct (the full reasoning is in the [implementation write-up](#)):

- The residual is only computable after ordering. `SMPmultiply` reads the matrix's external-ordering maps, which are populated by the *first* factorization — so the merit is gated to `iterno > 1` (before that it would read garbage and crash). The merit is built by `SMPmultiply`, ngspice's sparse matrix-vector product. Under the default Sparse 1.3 solver this works directly; under KLU (`.option klu`) it originally segfaulted, fixed in [Enhancement-112](#) — the line search now runs identically under both linear solvers (merit sequence numerically identical between them).
- SPICE device limiting is stateful. Junction-voltage limiting is referenced to the previous iterate stored in `CKTstate0`; naive trial re-loads drift that reference and destroy the iteration. The trial loads are made state-neutral by restoring `CKTstate0` to the x_k state (`OldCKTstate0`) before every trial and after the search, so the trials leave no trace but the chosen step.
- DC is a multi-phase state machine. The operating point runs `MODEINITJCT` $\rightarrow$ `MODEINITFIX` $\rightarrow$ `MODEINITFLOAT`, and $\|F\|$ is only a consistent function of x in the final `MODEINITFLOAT` Newton phase — so the line search is gated to that phase.

Files changed

Additive (117 insertions), confined to the option layer and the Newton iteration; no device code touched.

File	What changed
ngspice-46/src/ include/ ngspice/ optdefs.h	OPT_LINESEARCH option code
ngspice-46/src/ include/ ngspice/ tskdefs.h	TSKlinesearch task flag
ngspice-46/src/ include/ ngspice/ cktdefs.h	CKTlinesearch flag; CKTlsMerit (the residual $\ F\ $); CKTlsXk/CKTlsD/CKTlsBufSz (line-search scratch)
ngspice-46/src/ spicelib/ analysis/ cktsopt.c	OPT_LINESEARCH setter + "linesearch" keyword-table entry
ngspice-46/src/ spicelib/ analysis/ cktdojob.c	copy TSKlinesearch $\rightarrow$ CKTlinesearch
ngspice-46/src/ spicelib/ analysis/ cktntask.c	TSKlinesearch default (off) in both task paths
ngspice-46/src/ spicelib/ analysis/ cktdest.c	free CKTlsXk/CKTlsD
ngspice-46/src/ maths/ni/ niiter.c	the residual-merit computation (before factorization) and the Armijo backtracking line search (after the solve), with the state-neutral trial re-loads and the MODEINITFLOAT gate

Verification

[examples/linesearch_examples/](#) (9/9): a battery of nonlinear DC circuits (BJT, BJT+diode, two-diode divider, bistable latch) run with the option OFF and ON — the option is accepted, each still converges, and (the load-bearing property) the converged node voltages are identical ON vs OFF.

The backtracking path itself ($\lambda < 1$), which ngspice's robust DC init rarely triggers, was validated separately with a temporary hook that forces a damped step on every iteration: at $\lambda = 0.5, 0.25$ and even 0.1 the same roots are reached — BJT 2.442076, two-diode 0.679789 — and the bistable latch converges to the same basin as the full-step run under all damping levels. So both the full-step and the damped-step paths are verified to reach the correct answer.

Regression: the existing verify suites pass unchanged against the rebuilt ngspice (the feature is compiled in but off by default).

Honest scope. This is safe, principled, result-neutral infrastructure. Its practical *benefit* — converging circuits that otherwise fail — could not be demonstrated on constructible small circuits, because ngspice's multi-phase init, limiting, and homotopy already resolve their difficulty before the FLOAT-phase Newton where the line search acts; backtracking would only bite on large or pathological circuits. The durable win is the residual-merit machinery and a correct, verified globalized-Newton implemen-

tation to build on.

Enhancement-112 — KLU support for the globalized-Newton line search

Enhancement-111 added `.option linesearch`, a globalized (damped) Newton whose merit function is the true KCL residual $\|F\| = \|G \cdot x - b\|$, computed mid-solve with ngspice's sparse matrix-vector product `SMPmultiply`. That merit was only ever exercised under the Sparse 1.3 solver. Under the KLU solver (`.option klu`), enabling the line search crashed ngspice with a segfault:

```
.option klu
.option linesearch
→ SIGSEGV in klu_matrix_vector_multiply
```

This enhancement fixes that: the line search now runs correctly under both linear solvers.

Root cause — an unfinished KLU code path

`SMPmultiply` dispatches to a KLU-specific matrix-vector product when KLU is the active solver (`src/maths/KLU/klusmp.c`):

```
klu_matrix_vector_multiply(Ap_CSR, Ai_CSR, Ax_CSR, RHS, Solution,
                           NULL, NULL,          /* IntToExtRowMap, IntToExtColMap */
                           N, Common);
```

It passes **NULL** for the internal↔external ordering maps. But `KLU_matrix_vector_multiply` (`src/maths/KLU/klu_multiply.c`) dereferenced them unconditionally:

```
pExtOrder = &IntToExtColMap[n];          /* &NULL[n] = n·sizeof(int) */
... Solution[(pExtOrder--)] ...          /* reads address 0x14 for n=5 → SIGSEGV
↪ */
```

`&NULL[5]` is `0x14` — exactly the faulting address. The line search is the only caller of `SMPmultiply` under KLU, so this dead path had never been exercised before and its NULL-map case was never implemented.

The fix

Passing **NULL** for the ordering maps is `SMPmultiply`'s way of saying *"the matrix and the RHS/Solution vectors are already in the same order"* — the KLU CSC arrays and the circuit vectors share the external node ordering. So a NULL map means the identity ordering. `KLU_matrix_vector_multiply` now treats it as such:

```
ei = (IntToExtColMap != NULL) ? IntToExtColMap[i + 1] : (i + 1);
```

applied symmetrically to the gather (`Solution`) and scatter (`RHS`) loops, in both the real and complex builds. The `i + 1` is the one bookkeeping subtlety: the KLU CSR is 0-based while ngspice's RHS/Solution vectors are 1-based (index 0 is the grounded reference), so internal index `i` maps to external index `i + 1`.

Normal KLU factor/solve is untouched — `SMPmultiply` is called *only* by the line search, so nothing else changes.

Verification

Built against the repository ngspice (KLU enabled) and checked end-to-end:

- No crash. `.option klu + .option linesearch` now converges the BJT test to $v(c) = 2.442076$ V (was SIGSEGV).
- Numerically correct, not just non-crashing. The residual merit sequence under KLU is identical to Sparse 1.3 at every iteration — $28.15 \rightarrow 21.20 \rightarrow 9.75 \rightarrow 1.54 \rightarrow 0.032 \rightarrow 1.3e-5$ —

matching to ~11 significant digits (the last-digit differences are summation-order rounding). A wrong matrix-vector product would have produced a different merit.

- Result-neutral across the [linesearch battery](#) (BJT, BJT+diode, two-diode divider, bistable latch): `klu+linesearch == sparse+linesearch == klu without linesearch`.
- Backtracking path ($\lambda < 1$), forced with a temporary damped-step hook, reaches the correct root under KLU at $\lambda = 0.5, 0.25$ and 0.1 — matching Sparse.
- Full example suite, run under both solvers, is 101/101 OK; the `linesearch` example is now 17/17 under KLU (it was crashing, 9/17, before).
- No regression to normal KLU operation on non-line-search circuits.

Files changed

File	Change
<code>ngspice-46/src/maths/KLU/klu_multiply.c</code>	<code>KLU_matrix_vector_multiply</code> treats <code>NULL IntToExt{Row,Col}Map</code> as the identity ordering (<code>ext = i + 1</code>) instead of dereferencing <code>NULL</code> ; applies to the gather and scatter loops in both the real and complex builds

This also corrects the record for Enhancement-111: its line search is now genuinely verified under both KLU and Sparse 1.3 (the earlier "both solvers" note reflected a mislabeled test in which both runs were in fact Sparse).

Enhancement-113 — KLU support for noise and pole-zero analysis

Two analyses used to refuse the KLU solver outright:

```
.option klu
  noise ... → Error: Noise simulation is not (yet) supported with 'option KLU'. (noisea
  pz      ... → Error: Pole/zero analysis is not (yet) supported with 'option KLU'. (pzan.
```

so every `.noise` / `.pz` deck had to fall back to Sparse 1.3. This enhancement makes noise work under KLU, and pole-zero work under KLU for the common single-ended case.

Noise — the real reason it was disabled

Noise uses the adjoint method: it solves the *transposed* system $A^T \cdot x = e$ for the output port, then propagates each internal noise source through x . The transposed complex solve is `SMPcaSolve`. Its Sparse branch correctly calls `spSolveTransposed`, but its KLU branch called the *non-transposed* `klu_z_solve` — so for any asymmetric matrix (every circuit with a transistor or controlled source) it silently produced the wrong noise. A symmetric RC divider can't reveal it ($A^T = A$); a VCCS makes it obvious ($9.1e-9$ wrong vs $2.0e-7$ correct). That silent-wrong result is why noise was disabled under KLU rather than merely slow.

Fix (`src/maths/KLU/klusmp.c`): `SMPcaSolve`'s KLU branch now uses the transposed solve `klu_z_tsolve(..., conj_solve = 0, ...)`, matching `spSolveTransposed`. The noise guard in `noisean.c` is removed.

Verified: KLU noise now matches Sparse exactly — resistor thermal noise, asymmetric VCCS circuits, real OSDI device models (induced-gate-noise `noisejw`: $1.02622732e-08$ under both), and the integrated `onoise/inoise` totals. `.sp` S-parameters share the same `SMPcaSolve` adjoint and are corrected too.

Pole-zero — single-ended works; balanced-output stays on Sparse

The pole-zero path was already almost fully wired for KLU (`cktpzset.c` binding, `spDeterminant_KLU`, complex LU). The blanket guard in `pzan.c` is removed, and single-ended pole-zero — a grounded output reference, the overwhelmingly common case — now runs under KLU and matches Sparse across poles, zeros and current-mode (RC pole -1000 , high-pass zero at the origin, active 2-pole circuits, all identical to Sparse).

The one exception kept behind a targeted guard is a non-grounded output reference (balanced/differential output). Its zeros phase folds columns (`SMPcAddCol/SMPcZeroCol` in `CKTpzLoad`) at solve time — which Sparse survives via dynamic Markowitz re-ordering on every factorization, but KLU cannot, because its symbolic ordering is fixed once at `klu_analyze`. (Enabling KLU partial pivoting lets it factor, but then the `pz` determinant — which the whole method depends on — comes out wrong, because the pivoted+scaled factorization isn't unwound by `spDeterminant_KLU`.) That case now returns a clear error directing to `.option sparse`, instead of the earlier blanket refusal.

Files changed

File	Change
<code>ngspice-46/src/maths/KLU/klusmp.c</code>	<code>SMPcaSolve</code> KLU branch: transposed solve <code>klu_z_tsolve</code> (was <code>klu_z_solve</code>) — the adjoint fix that makes noise/S-parameters correct under KLU

File	Change
ngspice-46/src/spicelib/analysis/noisean.c	remove the KLU refusal guard
ngspice-46/src/spicelib/analysis/pzan.c	remove the blanket KLU refusal guard; add a targeted guard for balanced-output (non-grounded reference) pole-zero
examples/_setup.py	dual-solver harness: noise and single-ended pole-zero now run (and pass) under KLU; only balanced-output pole-zero and AC sensitivity are skipped as Sparse-only

Scope and remaining KLU limitations

Under KLU: noise ✓, single-ended pole-zero ✓ (+ latent .sp adjoint fix). Still Sparse-only: balanced-output pole-zero (above) and AC sensitivity (.sens ... ac; DC .sens is fine) — a separate analysis with its own KLU gap, out of scope here. The full example suite is 101/101 under both solvers; this moved 10 examples from KLU-skipped to KLU-passing (the noise + single-ended-pz suite), leaving only the analyses example skipped under KLU for its AC-sensitivity sub-test.

Enhancement-114 — KLU support for sensitivity analysis

Sensitivity analysis (`.sens <output>` for DC, `.sens <output> ac ...` for AC) was the last analysis that could not run under the KLU solver. It did not merely refuse — it crashed with a segfault:

```
.option klu
.sens v(out) ac lin 1 159155 159155 → Segmentation fault (exit 139)
.sens v(out)                        → Segmentation fault (exit 139)
```

so every `.sens` deck had to fall back to Sparse 1.3.

Why it crashed

Sensitivity (`src/spicelib/analysis/cktsens.c`) builds a second matrix, `delta_Y`, that holds the perturbation $\partial Y / \partial p$ of each parameter. It is created with

```
delta_Y = TALLOC(SMPmatrix, 1);
SMPnewMatrix(delta_Y, size);
```

`SMPnewMatrix` allocates a plain Sparse 1.3 matrix — it leaves `delta_Y->CKTkluMODE = 0` and never allocates a `SMPkluMatrix`. That is correct: `delta_Y` is only ever *multiplied* (`SMPmultiply` → `spMultiply`) against the solution vector, never factored, and the per-device `DEVbindCSC` callbacks always bind their matrix pointers into `ckt->CKTmatrix` (the main solver matrix), never into `delta_Y`. So `delta_Y` must stay Sparse regardless of which solver the main matrix uses.

But two KLU-only setup blocks in `cktsens.c` were gated on the main matrix's flag:

```
if (ckt->CKTmatrix->CKTkluMODE) /* true whenever .option klu is set */
{
    SMPconvertC00toCSC(delta_Y); /* delta_Y->SMPkluMatrix is NULL ...
→ */
    ... delta_Y->SMPkluMatrix->KLumatrixAx ... /* ... → NULL dereference → SIGSEGV
→ */
}
```

Under `.option klu` the main matrix's `CKTkluMODE` is 1, so these blocks ran and dereferenced `delta_Y->SMPkluMatrix` — which is NULL because `delta_Y` is a Sparse matrix. Under the default Sparse solver the flag is 0, the blocks were skipped, and everything worked — which is exactly the behavior `delta_Y` needs in both cases.

The fix

Gate the two `delta_Y` KLU blocks on **`delta_Y`**'s own `CKTkluMODE` (which is always 0) instead of the main matrix's, so `delta_Y` is treated as the Sparse matrix it actually is under KLU too — identical to how it already behaves under the default Sparse solver.

```
-      if (ckt->CKTmatrix->CKTkluMODE)
+      if (delta_Y->CKTkluMODE) /* delta_Y is Sparse under KLU too */
...
-      if (ckt->CKTkluMODE)
+      if (delta_Y->CKTkluMODE) /* delta_Y is Sparse under KLU too */
```

The main `Y` matrix stays KLU and is factored/solved by KLU as usual (`NIIsenReload` / `NIacIter`); only the auxiliary perturbation matrix is Sparse. This is a two-line behavioral change (plus an explanatory comment); no KLU plumbing is added because none is needed.

Verification

KLU sensitivity now matches Sparse exactly, and no longer crashes:

Case	SPARSE	KLU
Built-in RC, AC sens v1_acmag	4.999998e-01, -5.000000e-01	identical
OSDI ores+ocap, AC sens v1_acmag	4.999998e-01, -5.000000e-01	identical
Resistor-divider DC sens (r1, r2, v1)	-1.111111e-04 / 2.222220e-04 / 3.333333e-01	identical

Both DC and AC sensitivity are covered (they share the `delta_Y` block that was crashing). Verified against built-in devices and real OSDI device models.

Files changed

File	Change
<code>ngspice-46/src/spicelib/analysis/cktsens.c</code>	gate the two <code>delta_Y</code> KLU setup blocks on <code>delta_Y->CKTklumODE (0)</code> instead of the main matrix's flag, so the auxiliary perturbation matrix stays Sparse under KLU — fixes the NULL-deref segfault in DC and AC <code>.sens</code>
<code>examples/_setup.py</code>	dual-solver harness: sensitivity (<code>.sens ... ac</code>) no longer triggers a KLU skip. Unskipping the <code>analyses</code> example surfaced a <i>separate</i> pre-existing gap — <code>.disto</code> has no KLU wiring — so the skip trigger is switched from <code>.sens ...ac</code> to <code>.disto</code> . Also fixes a harness bug found while validating: the solver-card injector wrote <code>.option</code> into the real deck files and never restored them (every sweep permanently polluted committed decks); decks are now restored at process exit

Scope — sensitivity done; one adjacent gap surfaced

With this fix, KLU runs DC, DC-sweep, AC, transient, noise, S-parameters, single-ended pole-zero, and DC/AC sensitivity — matching Sparse 1.3. Two analyses remain Sparse-only under KLU:

- Balanced-output pole-zero — a pz card whose output reference node is not ground; the fixed KLU symbolic ordering cannot handle the zeroed columns (see [Enhancement-113](#)).
- Distortion (`.disto`) — surfaced while validating this fix. Unskipping the `analyses` example (previously KLU-skipped for its `.sens ...ac` sub-test) let its `.disto` sub-test run under KLU, where it silently produces no output: the distortion path (`cktdisto.c`) has no KLU binding at all — the only analysis driver that ever referenced KLU was `cktsens.c` (now fixed here). This is a distinct, pre-existing gap unrelated to sensitivity and out of scope for E-114; the harness now skips the `analyses` example under KLU on its `.disto` card instead of its `.sens ...ac` card (a candidate for a future enhancement).

Fixing the deck-injector restore bug also un-masked a third KLU numerical discrepancy: the `hierbranch` example's hierarchical branch-*current* probes read 0 under KLU (node voltages are correct). It had been passing under KLU only because a stale, never-restored `.option sparse` in its deck made the "klu" child silently run Sparse — a false pass. It is deterministic, independent of this sensitivity fix (the DC KLU solve is untouched), and now correctly marked `KLU_XFAIL` alongside the two previously known numerical differences — the stiff `opamp741` transient (convergence) and the degenerate `groundcontrib` single-node topology (wrong DC). The full example suite is 101/101 under both solvers (`sparse=PASS, klu ∈ {PASS, SKIP, XFAIL}`).

Enhancement-115 — KLU support for distortion analysis (**.disto**)

Distortion (**.disto**) was the last analysis that produced wrong results under the KLU solver — not a crash and not a refusal, but silent zero output: every **.disto** run under **.option klu** completed with no distortion data, so distortion always had to fall back to Sparse 1.3. (This gap was surfaced while validating [Enhancement-114](#), when un-skipping the analyses example let its **.disto** sub-test run under KLU.)

Why it produced nothing

Distortion is a complex analysis: a Volterra-series method that solves the small-signal system at the harmonic and intermodulation frequencies (f_1 , $2 \cdot f_1$, $3 \cdot f_1$, and — with a second tone — $f_1 \pm f_2$, $2 \cdot f_1 \pm f_2$, ...). Each solve goes through **NIdeIter** → **SMPcSolve**, the *complex* matrix solve — the same machinery AC uses via **NIacIter**.

Under KLU the matrix is stored once and re-used. AC (**acan.c**) prepares it for complex solves by, before its frequency loop, calling **DEVbindCSCComplex** for every device (which re-points the device matrix stamps at the complex KLU storage) and setting **KLUmatrixIsComplex**. Sparse 1.3 needs no such step — it carries real and imaginary parts together intrinsically.

distoan.c had no KLU code at all. So under **.option klu** the KLU matrix stayed in real mode, the complex solves in **NIdeIter** ran against an unconverted matrix, and every harmonic came back zero — a silent wrong answer. (**cktsens.c**, fixed in E-114, had been the *only* analysis driver that referenced KLU; distortion was the last one still unwired.)

The fix

Mirror exactly what **acan.c** does, in **src/spicelib/analysis/distoan.c**:

- Before the harmonic solve loop, once the operating point and matrix are set up, convert the KLU matrix to complex mode:

```
#ifdef KLU
    if (ckt->CKTmatrix->CKTkluMODE)
        if (!ckt->CKTmatrix->SMPkluMatrix->KLUmatrixIsComplex) {
            for (i = 0; i < DEVmaxnum; i++)
                if (DEVICES[i] && DEVICES[i]->DEVbindCSCComplex &&
                    ↪ ckt->CKThead[i])
                    DEVICES[i]->DEVbindCSCComplex(ckt->CKThead[i], ckt);
            ckt->CKTmatrix->SMPkluMatrix->KLUmatrixIsComplex =
            ↪ KLUmatrixComplex;
        }
#endif
```

The whole distortion loop is complex, so this one-time conversion suffices.

- On exit (successful completion), convert back to real with **DEVbindCSCComplexToReal / KLUmatrixReal**, exactly as **acan.c** does after AC, so a subsequent real analysis (**.op/.dc/.tran**) in the same interactive session finds the matrix in the expected state.
- Add **#include "ngspice/devdefs.h"** for **DEVICES / DEVbindCSCComplex**.

No changes to the distortion math, the RHS setup (**CKTdisto**), or the Sparse path.

Verification

KLU distortion now matches Sparse bit-for-bit:

Case	KLU vs Sparse
Built-in diode, single-tone (2nd + 3rd harmonic, every output vector)	identical
Built-in diode, two-tone intermodulation (Df2wanted)	identical
OSDI device model <code>.disto</code> completes (was 0 output rows under KLU, now full)	✓
<code>.disto</code> then <code>.tran</code> in one KLU session (matrix reconverted to real)	both complete, correct

The analyses example now runs fully under KLU (`sparse=PASS`, `klu=PASS`) — its distortion sub-test previously forced a KLU skip. The full example suite is 101/101 under both solvers.

Files changed

File	Change
<code>ngspice-46/src/spicelib/analysis/distoan.c</code>	add the KLU real↔complex matrix conversion around the distortion solve loop (<code>DEVbindCSCComplex</code> before, <code>DEVbindCSCComplexToReal</code> after), mirroring <code>acan.c</code> ; <code>+devdefs.h</code> include
<code>examples/_setup.py</code>	dual-solver harness: <code>.disto</code> is no longer a KLU-skip trigger, so the analyses example runs its distortion sub-test under KLU

Scope — KLU is now analysis-complete but for one case

With distortion fixed, KLU runs DC, DC-sweep, AC, transient, noise, S-parameters, single-ended pole-zero, sensitivity, and distortion — matching Sparse 1.3 across the entire example suite. The only analysis still Sparse-only under KLU is balanced-output pole-zero (a `pz` card whose output reference node is not ground; KLU's fixed symbolic ordering cannot fold the zeroed columns — see [Enhancement-113](#)). The three known KLU *numerical* differences are unrelated and unchanged: the stiff `opamp741` transient (convergence), the degenerate `groundcontrib` single-node topology (wrong DC), and `hierbranch`'s hierarchical branch-current probes (see [Enhancement-114](#)).

Enhancement-116 — KLU wrong-DC fix for decoupled OSDI nodes (2 of 3 XFAILs)

Three examples produced a different, wrong result under KLU than under Sparse 1.3 — genuine numerical discrepancies (not refusals), tracked as KLU_XFAIL in the [dual-solver harness](#):

- `groundcontrib` — a node-to-ground voltage contribution read $v(p)=0$ under KLU instead of 1.5.
- `hierbranch` — hierarchical branch-current probes read 0 under KLU (the node voltages were correct).
- `opamp741` — the transistor-level μ A741 transient diverges at the slew edge under KLU.

This enhancement fixes the first two (which turned out to share one root cause); `opamp741` is a separate KLU factorization-robustness limit, discussed at the end.

Root cause (`groundcontrib` + `hierbranch`)

Both models use a voltage contribution ($V(a,b) \leftarrow \text{expr}$), which OpenVAF lowers to a synthesized branch-current unknown plus a branch equation. The key detail is the ground reference: a contribution such as $V(p, \text{gnd}) \leftarrow 1.5$, where `gnd` is an explicit ground net, has the branch equation $V(p) - V(\text{gnd}) = 1.5$. Because `gnd` is ground, OpenVAF correctly omits the $\partial/\partial V(\text{gnd})$ partial — so the `gnd` net appears in no Jacobian entry at all.

But `OSDIsetup` (ngspice side) still allocated `gnd` its own solver row (`node_mapping` gave it a real matrix node). With no Jacobian coupling, that row and column are entirely zero:

$$\begin{bmatrix} 1/R_l & 1 \end{bmatrix} \begin{bmatrix} V(p) \end{bmatrix} = \begin{bmatrix} 0 \end{bmatrix} \quad \text{row/col for gnd is all-zero} \\ \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} i \end{bmatrix} = \begin{bmatrix} 1.5 \end{bmatrix} \quad + \quad \rightarrow \text{structurally singular under KLU}$$

Sparse 1.3 tolerates the decoupled all-zero row (it solves to $V(\text{gnd})=0$, the rest is unaffected). KLU's factorization sees a structurally singular matrix and returns a wrong solution ($v(p)=0$). This is not specific to a "degenerate single-node" topology as previously thought — it affects *any* OSDI voltage-branch contribution against a ground reference (verified with series resistors and multiple instances). `hierbranch` hits the same issue through its top-module ground `gnd` and the synthesized ammeter branches its current probes read.

The fix

In `src/osdi/osdisetup.c`, when allocating an instance's internal nodes, first build the set of node indices that appear in any Jacobian entry (as equation row or unknown column). An internal node that appears in none is structurally decoupled from the matrix; instead of allocating it a solver row, map it to ground (node 0):

```
/* build "used" set from descr->jacobian_entries (mapped through node_mapping) */
for (i = connected_terminals; i < num_nodes; i++) {
    if (!node_used[i]) { node_ids[i] = 0; continue; } /* decoupled -> ground */
    ... CKTmkVolt / CKTmkCur as before ...
}
```

This matches how OpenVAF already treats such a net (no coupling), removes the all-zero row/column, and fixes both solvers (Sparse is unchanged in result; KLU now matches it). Terminals are never grounded — only internal nodes.

Verified: `groundcontrib` $\rightarrow v(p)=1.5$ under KLU (was 0); `hierbranch` $\rightarrow$ 6/6 checks under KLU including the branch-current probes (was 4/6). The full example suite is 101/101 under both solvers with no regressions — the node-allocation change touches every OSDI model, and every one still agrees across solvers.

opamp741 — a genuine KLU robustness limit (unchanged)

The stiff transistor-level μ A741 follower, driven with a large square wave, slews its output. At the slew edge ($t \approx 2.03 \mu\text{s}$) output-stage transistors switch off, their transconductances collapse, and the Jacobian becomes ill-conditioned. KLU declares the matrix singular (nodes `x1.o1/o2/b34/cm`) and the timestep collapses, where Sparse's dynamic Markowitz threshold pivoting re-orders and completes the run (KLU aborts at ~133 rows, Sparse at 4058).

This is the classic KLU-vs-Sparse tradeoff: KLU's fill-reducing symbolic ordering is computed once and `klu_factor` can only pivot *within* it, whereas Sparse re-orders every factorization. It was confirmed not fixable with the available knobs — full partial pivoting (`tol = 1.0`), disabling BTF and re-analyzing a fresh ordering, and the `gmin`-loading path (identical to Sparse — both skip absent diagonals) all left the abort unchanged. A real fix would require a hybrid solver that falls back to Sparse when KLU's factorization fails, i.e. maintaining both matrix representations in sync — a large architectural change disproportionate to one stiff example. `opamp741` therefore remains the single documented `KLU_XFAIL`.

Files changed

File	Change
<code>ngspice-46/src/osdi/osdisetup.c</code>	ground-tie internal OSDI nodes that appear in no Jacobian entry (decoupled ground-reference nets), instead of giving them an all-zero solver row that makes the KLU matrix structurally singular
<code>examples/_setup.py</code>	<code>KLU_XFAIL</code> reduced from <code>{opamp741, groundcontrib, hierbranch}</code> to <code>{opamp741}</code>

Scope

KLU now matches Sparse across the suite for DC, DC-sweep, AC, transient, noise, S-parameters, single-ended pole-zero, sensitivity, and distortion, plus the two formerly-wrong OSDI voltage-branch cases. The remaining differences under KLU are: balanced-output pole-zero (Sparse-only by construction — see [Enhancement-113](#)) and the **opamp741** stiff transient (robustness, above). Sparse 1.3 remains the default and runs everything.

Enhancement-117 — Periodic steady state (PSS) productionized

Periodic steady state analysis (`.pss`) computes a circuit's periodic operating point directly — via a shooting method that iterates the fundamental frequency and the initial state until the waveform repeats — instead of integrating transients until they settle. It is the entry point of the RF periodic suite (the periodic small-signal analyses PAC / pnoise / PXF build on a converged PSS).

ngspice already contained a working shooting-method PSS (`dcpsp.c`), but it was experimental in two disqualifying ways, so it had never really shipped:

1. Gated behind **--enable-pss**. The standard configure omits it, so `.pss` was an *"unimplemented dot command"* in every shipped binary. It could only be exercised by specially rebuilding ngspice.
2. Deafening output. When enabled, a single `.pss` run printed ~230 lines of shooting-loop trace to stderr (per-iteration frequency estimates, residuals, breakpoint bookkeeping, delta control) — unusable in production.

This enhancement turns PSS into a first-class, shipped analysis.

1. Built by default

`configure.ac`: the PSS feature flag is flipped from opt-in to default-on (`if test "x$enable_pss" != xno and AM_CONDITIONAL([PSS_WANTED], ...!= xno)`), the help text becomes `--disable-pss ... Default=enabled`, and `configure` is regenerated (`autoconf 2.73`). A plain `./configure` now defines `WITH_PSS`, so `.pss` works in the shipped binary; `--disable-pss` still omits it.

2. Quiet by default, full trace on demand

`dcpsp.c`: the shooting-loop diagnostics are routed through a new gate

```
#define PSSDBG(...) do { if (ft_ngdebug) fprintf(stderr, __VA_ARGS__); } while(0)
```

69 debug prints become `PSSDBG(...)`; the converged summary and genuine errors (*"Convergence reached ... fundamental frequency is F Hz"*, *"PSS analysis aborted"*, transient failures) stay always-on. A normal `.pss` run drops from 232 to 31 lines of output; `set ngdebug` restores the full ~217-line trace for debugging.

3. Sparse-domain (KLU guard)

PSS's shooting loop does not converge under the KLU solver — it stalls at the first shooting cycle and spins indefinitely — even though a plain `.tran` on the same circuit runs fine under KLU. The failure is specific to the periodic breakpoint/timestep machinery's interaction with KLU, not KLU transients in general. Rather than let a `.option klu + .pss` deck hang, `DCpsp` now fails fast:

```
Error: periodic steady state analysis (.pss) is not supported with 'option KLU';  
use 'option sparse' (the default solver) for .pss.
```

PSS is therefore a Sparse-1.3-domain analysis, consistent with the other Sparse-only cases (balanced-output pole-zero). Fixing KLU-PSS convergence is left as a separate future investigation.

Verification

A 1 MHz-driven RC low-pass ($R=1k$, $C=1n$): PSS converges to the drive frequency and its fundamental harmonic equals the analytic AC response $|H(1MHz)| = 1/\sqrt{1 + (2\pi fRC)^2} = 0.15714$.

```
Convergence reached ... (iteration n° 22) ... fundamental frequency is 999999.8976 Hz  
harmonic 1: 9.999999e+05 Hz 1.571762e-01 (analytic 0.157136)
```

examples/rfpss_examples/ carries the deck (rc_pss.cir), a README documenting the .pss Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff [uic] syntax, and verify_rcpss.py, which checks the converged frequency, the fundamental magnitude, and that the default output carries no shooting-loop trace. (PSS is a shooting method — it simulates many drive periods — so the example runs ~1–2 minutes and is deliberately single-solver.)

Files changed

File	Change
ngspice-46/configure.ac (+ regenerated configure)	PSS built by default; --disable-pss opts out
ngspice-46/src/spicelib/analysis/dcpss.c	PSSDBG gate for the shooting-loop trace (232 → 31 lines by default); fail-fast KLU guard directing .pss to .option sparse
examples/rfpss_examples/	new: rc_pss.cir, verify_rcpss.py, README.md

Scope

PSS ships and is verified under Sparse 1.3. It is the foundation for the periodic small-signal analyses (PAC / pnoise / PXF), which remain future work, as does harmonic balance (HB) and KLU-PSS convergence.

Enhancement-118 — PSS runs under the KLU solver

Enhancement-117 shipped periodic steady state (`.pss`) but had to guard it to the Sparse solver: under `.option klu` the shooting loop *hung* — it stalled and spun indefinitely, even though a plain `.tran` on the same circuit runs fine under KLU. This enhancement finds and fixes the cause, so PSS runs correctly under both linear solvers, and removes the guard.

Diagnosis

It was never a crash or a wrong answer — it was a timestep explosion. By instrumenting the PSS transient loop and comparing the per-step `delta` (timestep) trajectory of the two solvers on the same RC deck:

- Both start identically (stabilization delta doubling $2e-10 \rightarrow 4e-10 \rightarrow 8e-10 \rightarrow \dots$).
- At step 5 they diverge: Sparse keeps doubling cleanly (up to $4e-7$), but KLU's `CKTtrunc` (local truncation error) returns a *slightly smaller* step and never fully recovers — the timestep collapses to $\sim 1e-13$ and stays there.
- Result: KLU takes ~ 21 million timesteps to Sparse's ~ 9.6 million — so a `.pss` that finishes in ~ 2 minutes under Sparse never finishes under KLU.

The root cause is the linear-solver re-factorization. `NIiter` normally reuses `klu_refactor` — which keeps the pivot ordering from the last full factorization and only recomputes the numerical values — for speed. Across PSS's extremely fine, breakpoint-dense shooting timesteps, that reused ordering accumulates just enough numerical error to inflate the truncation error, which shrinks every subsequent step. Sparse's own factorization is accurate enough not to trigger this, and a plain `.tran` under KLU is unaffected because its steps are coarse enough that the tiny refactor error never matters.

The fix

In `dcps.c`, under KLU, force a full re-factorization every PSS timestep by setting `NISHOULDREORDER` before the Newton iteration:

```
/* Enhancement-118: under KLU, force a full factorization every PSS timestep. */
if (ckt->CKTmatrix->CKTkluMODE)
    ckt->CKTniState |= NISHOULDREORDER;
converged = NIiter(ckt, ckt->CKTtranMaxIter);
```

`NISHOULDREORDER` makes `NIiter` take the accurate `SMPreorder` (`klu_factor`, re-pivoted) path instead of `klu_refactor`, so the truncation error — and hence the timestep — matches Sparse. The E-117 KLU refusal guard is removed. The change is gated on `CKTkluMODE`, so the Sparse path and plain `.tran` are untouched.

Verification

The 1 MHz-driven RC low-pass ($R=1k$, $C=1n$) under both solvers:

Solver	Converges	Fundamental frequency	Fundamental magnitude	Time
Sparse (unchanged)	iter 22	999999.8976 Hz	0.1571762	~ 2 min
KLU (was hanging)	iter 22	999999.8976 Hz	0.1571762	~ 3.7 min

KLU now converges to the identical result as Sparse (analytic $|H(1MHz)| = 0.157136$). It is $\sim 1.8\times$ slower (a full factor every step vs. a refactor), which is an acceptable cost for a specialized analysis that previously did not complete at all. `examples/rfpss_examples/verify_rcpss.py` now runs the deck under both solvers and asserts they agree.

Files changed

File	Change
ngspice-46/src/ spicelib/analysis/ dcpss.c	force a full KLU re-factorization each PSS timestep (NISHOULDREORDER); remove the E-117 Sparse-only guard
examples/rfpss_ examples/verify_ rcpss.py	run the PSS deck under both Sparse and KLU and assert the converged frequency and fundamental agree

Scope

PSS now runs under both Sparse 1.3 and KLU, closing the last KLU gap opened by E-117. It remains a brute-force shooting method and the foundation for the RF periodic small-signal suite (PAC / pnoise / PXF, future work).

Enhancement-119 — retain the PSS periodic operating point

This is the first step toward the RF periodic small-signal suite (PAC → pnoise → PXF). Those analyses linearize the circuit around the *periodic* operating point produced by **PSS** — the device Jacobians $G(t) = \partial I / \partial V$ and $C(t) = \partial Q / \partial V$ sampled along the steady-state waveform — and solve a harmonic conversion matrix. Before any of that can be built, the periodic operating point has to exist past the analysis. Today it does not.

The gap

PSS's shooting method already samples the node voltages at P equally-spaced time points across one converged period — it needs them for the DFT that produces the harmonic output. But it then frees those samples (`dcps.c`), and it never captured the device states (the charges/fluxes in `CKTstate0`) at all. The reactive Jacobian $C(t)$ a small-signal analysis needs is a function of those states, so voltages alone are not enough.

What E-119 does

Retain the converged periodic operating point on the PSS job (`PSSan`) instead of discarding it:

- capture `CKTstate0` (all `CKTnumStates` device states) at each of the P samples, alongside the node voltages that were already being captured;
- transfer ownership of the sample arrays to the job (`PSSopVoltages`, `PSSopStates`, `PSSopTimes`) with the metadata a consumer needs (`PSSopPoints`, `PSSopMsize`, `PSSopNumStates`, `PSSopFreq`), rather than freeing them;
- keep the memory bookkeeping correct: the previous run's operating point is freed before a new one is retained, the abort paths free the new states array, and on the frequency-relaunch path the stale samples are still freed (the relaunched run retains the correct ones).

A self-check reports the osc-node's voltage swing straight from the retained samples, so the retained data can be validated before any consumer exists.

Verification

The 1 MHz-driven RC low-pass ($R=1k$, $C=1n$):

Convergence reached ... 999999.8976 Hz

PSS periodic operating point retained: 1024 samples $\times$ 3 unknowns $\times$ 2 states at $f = 999999.8976$

retained op-point self-check: osc-node swing $[-0.157176, 0.157175]$ over the period

- 1024 samples $\times$ 3 unknowns $\times$ 2 states — the right shape (P points; nodes a , b plus the $V1$ branch current; $C1$'s charge + current).
- osc-node swing $[-0.157176, 0.157175]$ — the retained samples hold the *actual* periodic waveform: the peak equals the fundamental amplitude $|H(1\text{MHz})| = 0.157136$, computed directly from the retained data, independent of the DFT output.
- PSS still converges to the identical result under both solvers — no regression.

The per-sample state capture is a straight copy sized by `CKTnumStates`, so it scales automatically with the circuit's reactive content (here $C1 \rightarrow 2$ states; more reactive elements simply retain more).

Files changed

File	Change
ngspice-46/src/ include/ngspice/ pssdefs.h	add the retained periodic-operating-point fields to PSSan (PSSopVoltages/PSSopStates/PSSopTimes + dims + frequency)
ngspice-46/src/ spicelib/analysis/ dcpss.c	capture device states per sample; retain the operating point on the job instead of freeing it; self-check the retained data
examples/rfpss_ examples/verify_ rcpss.py	assert the operating point is retained with the right dimensions and that the retained samples reproduce the fundamental amplitude

Scope

E-119 only captures and retains the periodic operating point; nothing consumes it yet. Next: E-120 walks the retained samples, CKTloads each to build the periodic Jacobians $G(t)$, $C(t)$ and DFTs them to harmonics; then E-121 assembles and solves the harmonic conversion matrix and wires up the .pac command.

Enhancement-120 — periodic small-signal Jacobian harmonics

Second step toward periodic AC (`.pac`). PAC/pnoise/PXF solve a harmonic conversion matrix whose blocks are the harmonics G_k , C_k of the periodically time-varying device Jacobian — the conductance $G(t) = \partial I / \partial V$ and capacitance $C(t) = \partial Q / \partial V$ swept along the PSS steady-state waveform. E-119 retained the periodic operating point; E-120 turns it into those Jacobian harmonics.

What it does

For each of the P retained samples the routine (`pss_jacobian_report` in `dcps.c`):

1. restores that instant's bias — the node voltages and device states from the E-119 retained operating point;
2. re-linearizes the devices at that bias (`CKTload` with `MODEINITSMSIG`);
3. stamps $G + jC$ into the complex matrix (`CKTAcLoad` at $\omega = 1$, so the imaginary part is exactly C);
4. reads the (`osc, osc`) diagonal — its real part is the node's conductance $g(t)$, its imaginary part the capacitance $c(t)$.

The DFT of $g(t)$ and $c(t)$ across the period gives the periodic Jacobian's harmonics: flat (DC only) for a linear circuit, rich for a pumped nonlinear one. The `osc`-node diagonal is reported as a verifiable slice of the full G_k/C_k that the PAC conversion matrix (E-121) will be assembled from.

One subtlety worth recording After PSS the matrix is in real mode. `CKTAcLoad`'s `SMPcClear` only clears the imaginary part when the matrix is flagged complex, so on the first attempt $C(t)$ accumulated across samples ($\approx s \cdot C1$, whose DFT is a ramp: $DC \approx (P/2) \cdot C1$ with $1/k$ harmonics) while $G(t)$ was already correct. The fix is to put the matrix in complex mode first — `spSetComplex` for Sparse, the `KLUmatrixIsComplex + DEVbindCSCComplex` path for KLU (as `acan.c` does) — so each sample's stamp starts from a cleared complex matrix.

Verification

The 1 MHz-driven RC low-pass ($R1=1k$, $C1=1n$), `osc` node `b`:

periodic small-signal Jacobian at `osc` node (1024 samples, 10 harmonics):

$G(t)$: DC = 0.001 S, $|G1| = 4.24e-20$, $|G2| = 2.15e-19$, $|G3| = 1.35e-19$

$C(t)$: DC = 1e-09 F, $|C1| = 2.32e-25$, $|C2| = 1.72e-25$, $|C3| = 2.21e-25$

- $G(t)$ DC = 0.001 S = $1/R1$, $C(t)$ DC = 1e-9 F = $C1$ — the extracted Jacobian equals the *exact* known small-signal values.
- All harmonics ≈ 0 — the Jacobian is correctly time-invariant for a linear circuit (no spurious harmonics).

Because the routine re-linearizes at each sampled bias, a nonlinear device whose slope varies with its operating point (a diode's $g = (I_s/V_t) \cdot \exp(V/V_t)$, a switching mixer) yields genuinely time-varying $g(t)$, $c(t)$ and hence non-trivial harmonics — the very content PAC converts between sidebands. The linear check pins the extraction to exact analytic values; the harmonic machinery that carries the time-variation is the same code path.

`verify_rcpss.py` asserts the extracted $G(t)/C(t)$ DC values equal $1/R1$ and $C1$ and that their harmonics vanish.

Files changed

File	Change
ngspice-46/ src/ spicelib/ analysis/ dcpss.c	pss_jacobian_report: walk the retained operating point, re-linearize + AC-stamp per sample, read the osc-node $G(t)/C(t)$, DFT to harmonics; complex-mode setup so the imaginary stamp does not accumulate
examples/ rfpss_ examples/ verify_ rcpss.py	assert $G(t)$ DC == $1/R1$, $C(t)$ DC == $C1$, harmonics ≈ 0

Scope

E-120 reports the osc-node diagonal of the periodic Jacobian as a verified slice. E-121 generalizes to all matrix entries, assembles the $(2K+1)N$ harmonic conversion matrix, sweeps the input frequency, extracts the output sidebands (conversion gains), and wires up the `.pac` command.

Enhancement-121 — the PAC conversion-matrix engine

The third and central step of the RF periodic small-signal suite. **PSS** finds the periodic operating point, **E-119** retains it, and **E-120** turns it into the harmonics G_k, C_k of the periodically time-varying device Jacobian. **E-121** assembles those harmonics into the harmonic conversion matrix and solves it — the numerical heart of periodic AC (PAC), periodic noise, and PXF.

The idea

A circuit linearized about its PSS steady state has a T -periodic Jacobian ($T = 1/f_0$). A small tone injected at frequency f_{in} therefore does not produce a response only at f_{in} : the periodic time-variation mixes it up and down by every multiple of the fundamental, so the response appears at all sidebands $f_{in} + k \cdot f_0$ ($k = -M \dots M$). Writing the response as that sideband sum and balancing harmonics gives a block linear system — the conversion matrix — whose block (n, m) is

$$H_{\{nm\}} = G_{\{n-m\}} + j \cdot \omega_m \cdot C_{\{n-m\}}, \quad \omega_m = 2\pi \cdot (f_{in} + m \cdot f_0)$$

built from the Jacobian harmonics G_k, C_k . It has size $(2M+1) \cdot N$ ($N = \text{circuit unknowns}$). Solving $H \cdot X = B$ for a stimulus B injected at one sideband yields the responses X at all sidebands — the conversion gains that PAC reports.

What E-121 does

In `dcps.s.c`, after the retained operating point exists:

1. Samples the full Jacobian. Extends **E-120** from the osc-node diagonal to *every* structural nonzero of the matrix: it enumerates the nonzeros (`SMPfindElt`, which is KLU-aware and non-creating), then at each of the P retained samples restores that instant's bias, re-linearizes (`CKTload` with `MODEINITSMSIG`) and stamps $G + jC$ (`CKTAcLoad` at $\omega = 1$), reading each entry's real part $G(t)$ and imaginary part $C(t)$.
2. Extracts the harmonics. A complex DFT of each entry's $G(t), C(t)$ over the period gives G_k, C_k for $k = 0 \dots 2M$ ($G_{\{-k\}} = \text{conj}(G_k)$ since the time series is real).
3. Assembles the conversion matrix $H_{\{nm\}} = G_{\{n-m\}} + j \cdot \omega_m \cdot C_{\{n-m\}}$ — a $(2M+1)N$ complex block matrix, block-Toeplitz in the harmonic index with the per-sideband ω_m on the reactive term.
4. Solves it. A unit current is injected at the osc node in the 0-th sideband and the system is solved by a dense complex LU (`pss_csolve`, partial pivoting). The block count is small for the circuits PAC targets, so a direct dense factor is exact and simple; a production PAC on large circuits would assemble this as a sparse block system.
5. Reports the sideband responses at $-1 / 0 / +1$ — the conversion gains.

Verification

The 1 MHz-driven RC low-pass ($R = 1k, C = 1n$), probed at $f_{in} = f_0/2 = 500$ kHz:

PAC conversion matrix: $f_{in} = 500000$ Hz, 7 sidebands, unit I at osc node
sideband -1 (-500000 Hz): $|V| = 3.1688e-14$
sideband $+0$ (500000 Hz): $|V| = 303.314$
sideband $+1$ ($1.5e+06$ Hz): $|V| = 1.23414e-14$
expected sideband-0 driving-point $|Z| = 303.314$ Ohm (linear, from G_0/C_0)

For a linear circuit the periodic Jacobian is time-invariant, so the off-diagonal harmonic blocks G_k, C_k ($k \neq 0$) vanish and H is block-diagonal — its 0-block is exactly the ordinary AC matrix at f_{in} . The engine therefore reproduces the AC driving-point impedance at f_{in} ,

$$|Z(f_{in})| = 1 / |1/R + j \cdot 2\pi \cdot f_{in} \cdot C| = 1 / |1e-3 + j \cdot 3.1416e-3| = 303.31 \, \Omega,$$

to six figures (sideband 0 = 303.314 Ω), while the ± 1 sidebands come back at $\sim 3\text{e-}14$ — floating-point zero relative to 303 (ratio $\sim 1\text{e-}16$): no spurious conversion for a circuit that does not mix. The linear case pins the whole pipeline — nonzero-enumeration, per-sample re-linearization, harmonic DFT, block assembly, and the complex solve — to an exact analytic value; a pumped nonlinear circuit (a mixer, a switched-capacitor stage) fills the off-diagonal blocks and produces genuine conversion between sidebands through the *same* code path.

`verify_rcpss.py` asserts the reported `f_in`, that all three sidebands are present, that the reported and analytic driving-point $|Z|$ agree, that sideband 0 equals that impedance, and that the ± 1 conversion is $< 1\text{e-}6$ of sideband 0.

Files changed

File	Change
<code>ngspice-46/src/spicelib/analysis/dcpss.c</code>	<code>pss_csolve</code> (dense complex LU) and <code>pss_pac_report</code> — sample the full periodic Jacobian, DFT to harmonics, assemble the $(2M+1)N$ conversion matrix, inject a sideband-0 stimulus, solve, report the sideband gains; called from the retained-operating-point block
<code>examples/rfpss_examples/verify_rcpss.py</code>	assert the PAC sideband-0 response equals the analytic AC driving-point impedance and that the ± 1 sidebands carry no converted energy

Scope

E-121 delivers the conversion-matrix engine and verifies it against the exact linear driving-point response. It runs automatically off the retained PSS operating point and reports the sideband conversion gains. The remaining PAC work is user-facing wrapping — a dedicated `.pac` command that sweeps `f_in` and writes the conversion gains as an output vector, and reusing the same conversion matrix for periodic noise (fold device-noise sidebands through $H^\top$) and PXF — all of which now stand on this solve.

Enhancement-122 — the `.pac` command (periodic AC)

The user-facing periodic-AC analysis, built on the conversion-matrix engine of [E-121](#). Where E-121 solved the harmonic conversion matrix at a single probe frequency and reported it to stderr, `.pac` sweeps a small-signal input frequency and writes the response as a proper complex output vector — a first-class analysis you drive from a netlist and read back with `print/plot`.

The command

```
.pac Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff \  
    <DEC|OCT|LIN> Npts Fstart Fstop
```

The first seven fields are exactly the `.pss` parameters — `.pac` runs a periodic steady state to find the operating point it linearizes around — followed by an `.ac`-style frequency sweep (Npts per decade/octave, or total for linear). It reuses the PSS analysis internally: the parser sets the PSS parameters plus the `pac_*` sweep parameters and a `PSSdoPAC` flag, and `DCpss`, after retaining the periodic operating point, runs the sweep.

What it does

`dcpss.c` factors the E-121 engine into two reusable pieces:

- **`pac_extract_harmonics`** — walk the retained operating point, sample every Jacobian nonzero's $G(t)$, $C(t)$ over the period, and complex-DFT them to the harmonics G_k , C_k . Done once (the harmonics do not depend on the input frequency).
- **`pac_solve_at`** — at a given input frequency f_{in} , assemble the conversion matrix $H_{\{nm\}} = G_{\{n-m\}} + j \cdot \omega_m \cdot C_{\{n-m\}}$ (only the $\omega_m = 2\pi(f_{in} + m \cdot f_0)$ terms change per frequency), inject a unit current at the osc node in the 0-th sideband, and solve by dense complex LU.

`pac_sweep` then extracts the harmonics once and loops the input frequency (lin/dec/oct), solving at each point and emitting the 0-th-sideband node responses as a complex "PAC Analysis" plot versus frequency (through the same `OUTpBeginPlot/OUTpData` path as `.ac`, so `print/plot/wrdata` all work).

Verification

The 1 MHz-driven RC low-pass ($R=1k$, $C=1n$), swept 10 kHz–1 MHz (10 points/decade):

Index	frequency	mag(b)
0	1.000000e+04	9.980319e+02
...		
10	1.000000e+05	8.467330e+02
...		
20	1.000000e+06	1.571767e+02

For this linear circuit the periodic Jacobian is time-invariant, so the conversion matrix is block-diagonal and its 0-block is the ordinary AC matrix at f_{in} . The PAC sideband-0 response at the osc node b therefore equals the AC driving-point impedance across the whole sweep,

$$|Z(f)| = 1 / |1/R + j \cdot 2\pi \cdot f \cdot C|,$$

and it does — to a worst-case relative error of $1.6e-7$ over all 21 points (998.0Ω at 10 kHz $\rightarrow$ 157.2Ω at 1 MHz, matching the analytic values to six-plus figures). `verify_rcpac.py` parses the swept output vector and asserts every point against $|Z(f)|$.

A pumped nonlinear circuit fills the off-diagonal harmonic blocks, so its sideband-0 response is the genuine periodic-AC response (which differs from a plain `.ac` because it accounts for the periodic operating point) — through the same code path.

Files changed

File	Change
ngspice-46/src/ include/ngspice/ pssdefs.h	PSSan: add PSSdoPAC + PACfStart/PACfStop/PACpoints/PACstepType; param enum PAC_*
ngspice-46/src/ spicelib/analysis/ psssetp.c	PSSsetParm cases + PSSparms entries for the pac_* parameters
ngspice-46/src/ spicelib/parser/ inp2dot.c	dot_pac — parse the .pac card onto the PSS analysis with the sweep params + PSSdoPAC; dispatch .pac
ngspice-46/src/ spicelib/analysis/ dcpss.c	factor the E-121 engine into pac_extract_harmonics + pac_solve_at; add pac_sweep (frequency sweep → complex "PAC Analysis" plot); call it from DCpss when PSSdoPAC
examples/rfpss_ examples/rc_pac. cir, verify_rcpac. py	.pac example + a check that the swept sideband-0 response equals the analytic AC driving-point impedance

Scope

E-122 delivers a working .pac command with correct, verified periodic-AC output. The present stimulus is a unit current at the osc (probe) node and the output is the 0-th sideband; the natural extensions are a source-referenced stimulus (drive a named small-signal source, as .ac does) and multi-sideband conversion-gain output (emit every sideband $f_{in} + k \cdot f_0$ as its own vector). Both reuse the same conversion-matrix solve, which is also the substrate for periodic noise (fold device-noise sidebands through H^T) and PXF.

Enhancement-123 — finish **.pac**: source stimulus + conversion sidebands

E-122 landed a working **.pac** command, but two pieces were deliberately deferred: the stimulus was a fixed unit current at the osc node, and the output was only the 0-th sideband. E-123 completes both — **.pac** now drives a netlist-referenced small-signal source and emits the full set of conversion sidebands — so it is a general periodic-AC analysis.

1. Source-referenced stimulus

.ac takes its small-signal drive from sources carrying an AC <mag> <phase> spec; **.pac** now does the same. During harmonic extraction the engine calls **CKTAcLoad**, which clears **CKTrhs/CKTirhs** and lets every device stamp — a source with an AC spec stamps its (bias-independent) value there. **pac_extract_harmonics** captures that vector as the sideband-0 stimulus **B_0**; **pac_solve_at** uses it when present, falling back to the unit-current-at-osc-node probe otherwise.

The consequence is the physically meaningful one: with a source the PAC result is a transfer / conversion gain (output per unit source), not a driving-point impedance. For the RC driven by **V1 ... AC 1**, the sideband-0 response at **b** is the low-pass transfer $|H(f)| = 1/\sqrt{1 + (2\pi fRC)^2}$ — 0.998 at 10 kHz down to 0.157 at 1 MHz — versus E-122's driving-point $998 \rightarrow 157 \Omega$.

2. Multi-sideband conversion-gain output

A new optional trailing field selects how many conversion sidebands to emit:

```
.pac Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff \  
    <DEC|OCT|LIN> Npts Fstart Fstop [maxsideband]
```

With **maxsideband = Ksb**, **pac_sweep** builds an expanded output name list and writes, at every swept input frequency, the response at each sideband $f_{in} + k \cdot f_0$ for $k = -Ksb \dots Ksb$ as its own complex vector. Sideband 0 keeps the plain node name (so **plot b** still gives the fundamental), and the conversion sidebands are named <node>_usb<k> (upper, $k > 0$) and <node>_lsb<k> (lower, $k < 0$) — e.g. **b_usb1**, **b_lsb1** — created as vector UIDs with **IFnewUid**. **Ksb** defaults to 0 (sideband 0 only, exactly E-122) and is clamped to the number of harmonics the conversion matrix carries.

Verification

rc_pac_src.cir drives **V1 a 0 DC 0 AC 1 SIN(0 1 1meg)** and asks for two sidebands each side (**.pac ... dec 10 10k 1meg 2**):

PAC sweep: dec from 10000 to 1e+06 Hz (10 pts/decade) around $f_0 = 1e+06$ Hz;
stimulus: netlist AC source; 5 sidebands

```
mag(b):          9.980e-01 (10 kHz) ... 1.572e-01 (1 MHz)      <- low-pass transfer  
mag(b_usb1): ~2e-17                                           <- no conversion (linear)  
mag(b_lsb1): ~1.7e-17
```

- The source-referenced sideband-0 response equals the analytic AC transfer $1/\sqrt{1+(2\pi fRC)^2}$ across the sweep (worst-case rel. err < $2e-2$) — and is a thousand-fold different from E-122's unit-current driving-point impedance, confirming the stimulus is the netlist source.
- The **b_usb1** / **b_lsb1** conversion sidebands exist as named vectors and come back at floating-point zero — a linear circuit does not mix, exactly as it must. A pumped nonlinear circuit fills them with real conversion gain through the same path.

verify_rcpac.py runs both the E-122 (unit-current) and E-123 (source + sidebands) decks under **Sparse** and asserts all of the above.

Files changed

File	Change
ngspice-46/src/ include/ngspice/ pssdefs.h	PSSan.PACmaxSideband + PAC_MAXSB param id
ngspice-46/src/ spicelib/analysis/ psssetp.c	pac_maxsb setter + IFparm entry
ngspice-46/src/ spicelib/parser/ inp2dot.c	dot_pac parses the optional trailing maxsideband
ngspice-46/src/ spicelib/analysis/ dcpss.c	capture the source AC RHS B_0 in pac_extract_harmonics; pac_solve_at gains a use_src stimulus selector; pac_sweep emits $2 \cdot Ksb + 1$ sidebands as named vectors
examples/rfpss_ examples/rc_pac_src. cir, verify_rcpac.py	source-referenced + multi-sideband example and checks

Scope

.pac is now a complete periodic-AC analysis: PSS operating point $\rightarrow$ periodic Jacobian harmonics $\rightarrow$ conversion matrix $\rightarrow$ swept solve, driven by a netlist source and reporting every conversion sideband. The same conversion-matrix solve is the substrate for periodic noise (fold device-noise sidebands through H^T) and PXF (periodic transfer function), the natural next analyses.

Enhancement-124 — periodic noise (**.pnoise**)

The second RF periodic small-signal analysis on the conversion-matrix engine. Where **.pac** propagates a deterministic small signal, **.pnoise** propagates noise: each device's noise sources are converted between sidebands by the periodic operating point, and the output noise at a given frequency is the sum of every source folded through the harmonic conversion matrix. This is the analysis that gives a mixer its noise figure or an oscillator its phase-noise skirt.

The idea, and how it reuses ngspice's noise machinery

Ordinary **.noise** computes, per frequency, the adjoint transimpedance — solve $Y^T x = e_{out}$, leaving in **CKTrhs/CKTirhs** the transfer from every node to the output — and then each device's noise routine adds $S \cdot |\Delta \text{transimpedance}|^2$ to the output noise (S = the device's noise PSD). The device routines (**NevalSrc**, **OSDI load_noise**) read that transimpedance straight from **CKTrhs/CKTirhs**.

.pnoise keeps *exactly* those device routines and only changes the transfer they see. It solves the adjoint of the conversion matrix, $H^T \Psi = e_{\{out,0\}}$, whose sideband- k block Ψ_k is the transfer from a noise injection at sideband k to the output at sideband 0. Then, per frequency:

$$\text{onoise}(f) = \sum_k \sum_{\text{devices}} S \cdot |\Psi_k(p) - \Psi_k(n)|^2$$

is computed by loading Ψ_k into **CKTrhs/CKTirhs** and calling the device noise routines once per sideband $k = -M \dots M$, accumulating — each device folds its own noise correctly, with no per-device special-casing, so **.pnoise** covers resistors, **OSDI/Verilog-A** devices, transistors, everything **.noise** does. A minimal local **NOISEAN** job (with **NStpsSm = 0**, **prtSummary** off) gives those routines the context they expect while the summary/integration side effects stay dormant. The input-referred spectrum divides by the source→output conversion gain (the **E-123** source-referenced solve at sideband 0).

The command

```
.pnoise Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff \  
      OutNode InSrc <DEC|OCT|LIN> Npts Fstart Fstop
```

The first seven fields are the **.pss** parameters, then the output node, the input source (for the input-referred spectrum), and an **.ac**-style sweep. It reuses the PSS analysis via a **PSSdoPnoise** flag and produces a **PNoise Analysis** plot with **onoise_spectrum** and **inoise_spectrum** vectors.

Verification

The 1 MHz-driven RC low-pass ($R=1k$, $C=1n$), output **b**, swept 10 kHz–1 MHz:

Index	frequency	onoise_spectrum
0	1.000000e+04	1.651088e-17
...		
10	1.000000e+05	1.188432e-17
...		
20	1.000000e+06	4.095038e-19

Because the circuit is linear, the periodic Jacobian is time-invariant: the conversion matrix is block-diagonal, only sideband 0 contributes, and **.pnoise** must reduce to ordinary **.noise**. It does — the output noise density equals the analytic thermal-noise result

$$S_{out}(f) = 4 \cdot k \cdot T \cdot R_1 / (1 + (2\pi \cdot f \cdot R_1 \cdot C_1)^2) \quad [V^2/Hz]$$

to a worst-case relative error of $8e-7$ across the sweep ($1.65e-17$ at 10 kHz down to $4.10e-19$ at 1 MHz), and it matches a plain **.noise** run of the same network to every printed digit. A pumped nonlinear circuit fills the off-diagonal conversion blocks, so its noise genuinely folds between sidebands (up/down-

conversion, phase noise) through the same code path. `verify_rcpnoise.py` asserts `pnoise` against both the analytic law and the `.noise` reference.

Files changed

File	Change
<code>ngspice-46/src/include/ngspice/pssdefs.h</code>	PSSan: PSSdoPnoise, PnOutNode, PnInSrc; PNOISE_* param ids
<code>ngspice-46/src/spicelib/analysis/psssetp.c</code>	<code>pnoise</code> / <code>pnoise_out</code> / <code>pnoise_insrc</code> setters + IFparm entries
<code>ngspice-46/src/spicelib/parser/inp2dot.c</code>	<code>dot_pnoise</code> — parse the <code>.pnoise</code> card onto the PSS analysis
<code>ngspice-46/src/spicelib/analysis/dcpss.c</code>	<code>pac_solve_adjoint</code> (transposed conversion solve) and <code>pnoise_sweep</code> — fold every device's noise over sidebands via the device noise routines and a local NOISEAN context; called from DCpss when PSSdoPnoise
<code>examples/rfpss_examples/rc_pnoise.cir</code> , <code>verify_rcpnoise.py</code>	<code>.pnoise</code> example + checks vs. the analytic law and <code>.noise</code>

Scope

`.pnoise` delivers a working periodic-noise analysis that reuses every device noise model and reduces exactly to `.noise` in the linear limit. The first cut evaluates each device's noise PSD at the periodic operating-point sample (a stationary approximation of the cyclostationary source); harmonic (cyclostationary) noise-PSD modulation and a dedicated phase-noise (jitter) output are the natural refinements. The same conversion-matrix adjoint is the last piece needed for PXF (periodic transfer function).

Enhancement-125 — periodic transfer function (**.pxf**)

The third and final RF periodic small-signal analysis, completing the PSS → PAC → Pnoise → PXF suite. PXF is the adjoint counterpart of **.pac**: where PAC injects one input and reads the response at every output, PXF fixes one output and gives the transfer from the input at each sideband — the efficient way to ask “what reaches this node, and from where.”

The idea

PXF solves the adjoint of the conversion matrix,

$$H^T \Psi = e_{\{\text{out}, 0\}},$$

whose sideband- k block $\Psi_k(j)$ is the transfer from a unit injection at node j , sideband k , to the fixed output at sideband 0. Dotting each block with the netlist AC-source pattern $B0$ (the same stamp **.pac/.pnoise** capture) gives the transfer from the input to the output at that sideband:

$$xf_k(f) = \sum_j \Psi_k(j) \cdot B0(j).$$

The sideband-0 result is not merely *close* to the PAC response — it is exactly equal, by the linear-algebra identity

$$(H^{-1} B)_{\text{out}} = e_{\text{out}}^T H^{-1} B = (H^{-T} e_{\text{out}})^T B = \Psi^T B,$$

so **.pxf**'s xf and **.pac**'s output at the same node agree to machine precision. That is the reciprocity cross-check that pins the adjoint solve (**pac_solve_adjoint**, reused from **pnoise**) to the forward **pac_solve_at**.

The command

```
.pxf Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff \  
      OutNode <DEC|OCT|LIN> Npts Fstart Fstop [maxsideband]
```

The first seven fields are the **.pss** parameters, then the output node, an **.ac**-style sweep, and an optional sideband count. It reuses the PSS analysis via a **PSSdoPXF** flag and produces a PXF Analysis plot with xf (sideband 0) plus $xf_{\text{usb}<k>}/xf_{\text{lsb}<k>}$ for the upper/lower conversion transfers.

Verification

The 1 MHz-driven RC low-pass ($R=1k$, $C=1n$), output b , input $V1$ AC 1, swept 10 kHz–1 MHz:

Index	frequency	mag(xf)
0	1.000000e+04	9.980319e-01
...		
20	1.000000e+06	1.571767e-01

- The sideband-0 transfer equals the analytic low-pass $1/\sqrt{1+(2\pi fRC)^2}$ ($0.998 \rightarrow 0.157$) to a worst-case relative error of $1.6e-7$, and it is bit-identical to the **E-123** PAC response $\text{mag}(b)$ at the same node — the reciprocity identity holding to the last digit.
- The $xf_{\text{usb}1}/xf_{\text{lsb}1}$ conversion transfers come back at $\sim 2e-16$ — floating-point zero: a linear circuit does not convert between sidebands. A pumped nonlinear circuit fills them, giving the genuine up/down-conversion transfer functions through the same adjoint.

verify_rcpxf.py asserts xf against the analytic transfer and the conversion sidebands against zero.

Files changed

File	Change
ngspice-46/src/ include/ngspice/ pssdefs.h	PSSan.PSSdoPXF, PxOutNode; PXF_* param ids
ngspice-46/src/ spicelib/analysis/ psssetp.c	pxf / pxf_out setters + IFparm entries
ngspice-46/src/ spicelib/parser/ inp2dot.c	dot_pxf — parse the .pxf card onto the PSS analysis
ngspice-46/src/ spicelib/analysis/ dcpss.c	pxf_sweep — per frequency solve the conversion adjoint (pac_solve_ adjoint) and dot each sideband block with B0, emitting the input→output transfer at each sideband
examples/rfpss_ examples/rc_pxf. cir, verify_rcpxf. py	.pxf example + checks vs. the analytic transfer (== the PAC response by reciprocity)

Scope — the RF periodic small-signal suite is complete

With PXF the trio built on the hardened PSS is done: PAC (conversion gain), Pnoise (folded device noise), and PXF (adjoint transfer). All three share one conversion-matrix solve and its adjoint. The same dense-solve scale caveat and stationary-source note apply; the substrate for full cyclostationary noise, phase noise, and quasi-periodic (multi-tone) extensions is now all in place.

Enhancement-126 — cyclostationary noise

The first refinement of the RF periodic small-signal suite. `.pnoise` folds every device's noise through the conversion matrix, but its first cut made a stationary approximation: it evaluated each device's noise PSD at a single operating point. For a device pumped by a large signal that is exactly the approximation that loses the physics — a diode's shot noise $2qI(t)$, a transistor's $gm(t)$ noise, or a resistor's flicker noise $\propto |I(t)|$ all vary along the PSS period. E-126 adds a cyclostationary mode that captures that variation.

The idea

A cyclostationary noise source has a periodic PSD $S(t)$. Its harmonics couple the noise between sidebands, so the output noise is the double sum

$$\text{onoise}(f) = \sum_k \sum_l \Delta\Psi_k^* \Delta\Psi_l S_{\{k-l\}},$$

where $\Delta\Psi_k$ is the sideband- k adjoint transfer (E-124) and S_m is the m -th harmonic of $S(t)$. Substituting $S_m = (1/P) \sum_s S(t_s) e^{-jm\omega_0 t_s}$ and collapsing the sums gives a form that needs no explicit harmonics of the noise:

$$\begin{aligned} \text{onoise}(f) &= (1/P) \sum_s S(t_s) \cdot |\Delta A_s(f)|^2, \\ A_s(j) &= \sum_k \Psi_k(j) \cdot e^{+j2\pi k s/P} \quad (\text{inverse-DFT of the sideband transfers}). \end{aligned}$$

So the cyclostationary output noise is the period average of the instantaneous noise power $S(t_s)$ weighted by the time-domain adjoint transfer A_s . This is computed by looping over the P retained PSS samples: at each sample its bias is restored (CKTload, so every device's noise PSD is evaluated at that instant), the time-domain transfer A_s is loaded into CKTrhs/CKTlrhs, and the existing device noise routines are called and accumulated — then divided by P . It reuses the device noise models exactly as E-124 does; only the transfer and the averaging change. Enabled by a trailing `cyclo` keyword on `.pnoise`.

Two things it must satisfy

1. Reduction (rigorous). When the noise source is bias-independent — a resistor's thermal noise $4kTG$ — $S(t)$ is constant, and by Parseval the period average equals the sideband sum, so cyclostationary reduces exactly to the stationary result, i.e. to ordinary `.noise`. On the linear RC low-pass the cyclostationary output noise equals $4kTR1/(1+(2\pi fR1C1)^2)$ to every printed digit, identical to E-124 and to a plain `.noise` run — validating the whole per-sample / time-domain-transfer / averaging machinery.
2. Cyclostationary effect (quantitative). A resistor $R1$ carries a *known* periodic current $I(t) = I_{dc} + I_{ac} \cdot \sin(\omega_0 t)$ (driven by a current source) and has a flicker model ($KF, AF = 2$). Its flicker noise $\propto KF \cdot |I(t)|^2$ is genuinely cyclostationary, the circuit is linear (fast PSS), and the transfer to the output is flat ($= R1$), so

$$\text{onoise_flicker}(f) \cdot f = R1^2 \cdot KF \cdot \langle |I(t)|^2 \rangle = R1^2 \cdot KF \cdot (I_{dc}^2 + I_{ac}^2/2),$$

a constant independent of frequency. With $I_{dc} = 1 \text{ mA}$, $I_{ac} = 0.8 \text{ mA}$, $\langle I^2 \rangle = 1.32e-6 \text{ A}^2$ — 32 % above the DC-current value $I_{dc}^2 = 1e-6$ a stationary analysis at the operating point would use. The cyclostationary result matches $R1^2 \cdot KF \cdot \langle I^2 \rangle$ across the sweep, confirming it uses the period average, not a single sample.

The command

```
.pnoise Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff \
      OutNode InSrc <DEC|OCT|LIN> Npts Fstart Fstop [cyclo]
```

The trailing `cyclo` selects the cyclostationary mode; without it, `.pnoise` keeps the E-124 stationary behaviour.

Files changed

File	Change
ngspice-46/src/ include/ngspice/ pssdefs.h	PSSan.PSSpnCyclo; PNOISE_CYCLO param id
ngspice-46/src/ spicelib/ analysis/ psssetp.c	pnoise_cyclo setter + IFparm entry
ngspice-46/src/ spicelib/parser/ inp2dot.c	dot_pnoise parses the optional trailing cyclo
ngspice-46/src/ spicelib/ analysis/dcpss.c examples/rfpss_ examples/rc_ pnoise.cir, verify_rccyclo. py	pnoise_sweep cyclostationary branch — precompute the adjoint per frequency, loop PSS samples restoring each bias, fold the time-domain transfer through the device noise routines, average over the period reduction check (RC → .noise) + quantitative flicker check

Scope

Cyclostationary noise makes .pnoise correct for pumped devices whose noise depends on the operating point (mixers, oscillators, switched circuits). The remaining refinement is a dedicated phase-noise (jitter) spectrum, which reuses the same cyclostationary fold specialised to the oscillator's phase mode.

Enhancement-127 — pseudo-transient continuation

A convergence-robustness enhancement: `.option ptcont` adds a pseudo-transient continuation (PTC) homotopy to the DC operating-point solve. ngspice's convergence cascade already had static gmin stepping, source stepping, and a transient-op fallback; the "principled $\dot{x}$ -embedded continuation" flagged as the remaining gap (alongside the globalized Newton line search that E-111 added) is what this fills.

The method

The DC problem $f(x) = 0$ is embedded in a fictitious backward-Euler pseudo-transient

$$f(x) + Gps \cdot (x - x_{prev}) = 0, \quad Gps = Cps / d\tau,$$

which is one implicit step of $Cps \cdot \dot{x} = -f(x)$ in a *pseudo*-time τ . Marching the pseudo-timestep $d\tau$ from small (Gps large — the system is diagonally dominant, strongly damped, trivially solvable) to large ($Gps \rightarrow 0$ — the pseudo-term vanishes and the equation becomes the true $f(x) = 0$) relaxes the solution to the DC operating point along a stable trajectory.

Each pseudo-step is an ordinary Newton solve of the augmented system, reusing ngspice's existing machinery:

- the **Gps** diagonal is added at factorization time through the same path gmin stepping uses (CKTdiagGmin, applied by LoadGmin / the KLU diagonal load), so it works under both linear solvers;
- the **Gps**·**x_prev** coupling is added to the right-hand side inside `NIiter`, right after the load. This is the essential difference from static gmin stepping: without it the step is a memoryless gmin-shunted solve; with it the step is a genuine backward-Euler move from the previous point, so the iterate follows the operating curve rather than jumping to a distant (possibly spurious) root.

A switched-evolution-relaxation rule adapts $d\tau$: grow it (aggressively when the step converged in few iterations) after a successful step, shrink it and backtrack to the last good point after a failed step. A final solve at $Gps = 0$ polishes the exact DC answer. The driver sits in CKTop as another cascade fallback, gated on `.option ptcont`; it is off by default.

Verification

A behavioral exponential with no junction limiting — a deliberately stiff nonlinearity where plain Newton misbehaves:

```
B1 1 0 I = 1e-14 * (exp(V(1)/0.026) - 1)
R1 2 1 100
V1 2 0 100
```

From $V = 0$, plain Newton overshoots the enormous $\exp$ derivative and settles on a spurious root, $V(1) \approx 70.5$ V. Pseudo-transient continuation follows the stable trajectory to the physically correct operating point,

$$V(1) = 0.837922 \text{ V} \quad (\text{the root of } 1e-14 \cdot (\exp(V/0.026) - 1) = (100 - V)/100),$$

matching the analytic value. `verify_ptcont.py`, run under both KLU and Sparse1.3, checks three things:

1. `.option ptcont` is accepted;
2. result-neutrality — on a battery of normal nonlinear circuits (diode, BJT, two-diode divider, resistor network) the operating point with `ptcont` on is identical to a normal run (a convergence aid must never change the answer);
3. convergence power — on the stiff circuit `ptcont` reaches 0.837922 V (matched to the analytic root), differing from the spurious 70.5 V plain Newton returns.

All 21 checks pass under both solvers.

Files changed

File	Change
ngspice-46/src/include/ngspice/ optdefs.h, tskdefs.h, cktdefs.h	OPT_PTCONT / TSKptcont / CKTptcont + CKTpseudoGmin, CKTpseudoPrev
ngspice-46/src/spicelib/analysis/ cktsopt.c, cktntask.c, cktdojob.c	wire .option ptcont through the task → circuit (off by default)
ngspice-46/src/spicelib/analysis/ cktop.c	pseudo_transient homotopy driver + its slot in the CKTop convergence cascade
ngspice-46/src/math/sni/niiter.c	add the $G_{ps} \cdot x_{prev}$ coupling to the RHS after the load
ngspice-46/src/spicelib/analysis/ cktdest.c	free CKTpseudoPrev
examples/ptcont_examples/	ptcont_demo.cir, verify_ptcont.py, README. md

Scope

PTC is a general DC-convergence aid, verified result-neutral and shown to converge a stiff circuit that plain Newton fails. It composes with the existing cascade (it can be a fallback after gmin/source stepping, or the sole homotopy when those are disabled) and works under both linear solvers. Automatic triggering heuristics (when to reach for PTC without the user asking) and coupling it into the `errpreset` robustness presets are natural follow-ups.

Enhancement-128 — LTE-based dynamic integration-order control

An efficiency enhancement to the transient integrator: `.option dynorder` selects the Gear integration order per step from the local-truncation-error (LTE) limit, so higher-order Gear is used on smooth stretches and far larger timesteps are taken at the same accuracy. Off by default; bounded by the standard `maxord` knob.

The gap

ngspice already has all the machinery for high-order Gear (BDF): `NIcomCof` computes the integration coefficients for orders 1–6, and `CKTtrunc/CKTterr` estimate the LTE-limited timestep at any order. But the stock order controller in `dctran.c` only ever toggles the order between 1 and 2 — once at order 2 it never climbs, so orders 3–6 are dead code in every ordinary transient. On smooth problems that leaves a large efficiency gain on the table: the LTE-limited step grows like $\text{tol}^{1/(p+1)}$ with order p , so a high order permits dramatically bigger steps at a fixed accuracy.

The method

Each accepted timestep, with `dynorder` on, the controller evaluates the raw LTE-limited step (via `CKTtrunc`, given a large trial input so its $2\times$ growth cap does not mask the per-order difference) at the current order and its immediate neighbours, and moves toward the order that permits the largest step. Four guards keep the higher orders — where BDF is only conditionally stable and the divided-difference LTE estimate is delicate — from ever wrecking the answer:

- neighbours only (± 1), never a greedy global maximum — the order cannot oscillate (oscillation makes the LTE history inconsistent and triggers a cascade of step rejections; this was the failure mode of a first, greedy implementation);
- hysteresis — a neighbour must beat the current order's LTE step by $1.2\times$ to win, otherwise the order is kept;
- a settling hold — after any order change the order is held for a few steps so the BDF divided differences (which assume a roughly constant step) can rebuild before another change is considered;
- an order-dependent growth cap — the step is *not* grown the same step the order is raised, and on subsequent steps the growth cap tightens with order ($2\times$ at order ≤ 3 down to $1.3\times$ at order 6), because a large step jump at high order corrupts the very divided differences the LTE relies on.

The whole path is gated on `.option dynorder` and bounded by `maxord`; with the default `maxord = 2` it cannot exceed the stock order-2 behaviour, so enabling it on an ordinary deck is inert.

Robustness on stiff transients

High-order Gear is the wrong tool inside a violent transient (a large fast slew, the first microseconds after a breakpoint). Left unchecked, the controller would climb to a high order there and the timestep would collapse to zero ("Timestep too small"). Two guards, both gated on `dynorder`, make it degrade gracefully instead:

- post-breakpoint hold — a breakpoint (a pulse edge, a source transition) already resets the order to 1; the controller now also *holds* it low for the next few steps (`maxord + 2`), so it does not race back up to a high order inside the post-discontinuity transient before it has settled;
- rejection-rate order drop — a leaky bucket (+2 per LTE-rejected step, -1 per accepted step) detects a *sustained* high rejection rate — the signature of a stiff region where the current order overreaches — and walks the order back down. A single isolated rejection, normal on a smooth high-order run, decays away and leaves the order (and its efficiency) untouched.

The stress case is the transistor-level $\mu\text{A}741$ follower driven by a big $\pm 5\text{ V}$ square wave (Enhancement-83): the output slew-rate-limits for tens of microseconds. With these guards dynamic-order control

completes that slew under both linear solvers and lands on the same answer as fixed low-order Gear (-3.31499 V vs -3.31538 V), where without them it collapsed the step at the pulse edge.

Verification

Three circuits, checked under both Sparse 1.3 (default) and KLU (`verify_dynorder.py`; the heavy reference sweeps run under Sparse, KLU runs the fast subset):

- RC discharge ($V(0)=1$, $RC=1$ ms; analytic $V(8\text{ ms})=e^{\{-8\}}$). At matched tolerance dynorder (`maxord=3`) reaches the stock controller's accuracy in $\geq 2\times$ fewer steps (163 vs 664 rows at `reltol=1e-8`, both $\approx 0.04\%$ error), and its error is monotone in tolerance — the higher-order controller never blows up. Across the Pareto frontier the step saving at matched accuracy is $3-5\times$.
- Parallel-RLC ringdown — a smooth 5.03 kHz sinusoid. dynorder (`maxord=4`) uses 420 steps vs the stock controller's 3734 (a $8.9\times$ reduction) and is simultaneously more accurate against a tight reference (0.13 % vs 0.34 %), because the stock $1\leftrightarrow 2$ toggle never uses order 3–4 even when `maxord=4`. Higher order pays off twice on smooth dynamics.
- Nonlinear diode rectifier — diode switching makes frequent breakpoints that reset the order to 1, so the benefit is modest, but dynorder's final value matches the stock controller to 5 significant figures: a switching circuit must not be perturbed.

Plus a safety check that `.option dynorder` at the default `maxord=2` is result-neutral versus a plain run. All checks pass under both solvers (10/10 Sparse, 6/6 KLU).

Files changed

File	Change
<code>ngspice-46/src/include/ngspice/optdefs.h</code> , <code>tskdefs.h</code> , <code>cktdefs.h</code>	<code>OPT_DYNORDER</code> / <code>TSKdynorder</code> / <code>CKTdynorder</code> + <code>CKTorderCnt</code> (history depth), <code>CKTorderHold</code> (settling), <code>CKTorderRej</code> (rejection-rate bucket), <code>CKTorderMaxUsed</code> (diagnostic)
<code>ngspice-46/src/spicelib/analysis/cktsopt.c</code> , <code>cktnntask.c</code> , <code>cktdojob.c</code>	<code>wire .option dynorder</code> through the task $\rightarrow$ circuit (off by default)
<code>ngspice-46/src/spicelib/analysis/dctran.c</code>	the neighbour + hysteresis + settling-hold + growth-cap order selector, replacing the stock $1\leftrightarrow 2$ toggle when dynorder is set; the stiff-transient guards (post-breakpoint hold, rejection-rate order drop); order-history resets at init and breakpoints; a set <code>ngdebug</code> summary of the highest order used
<code>examples/dynorder_examples/opamp741_examples/verify_opamp741.py</code>	<code>dynorder_demo.cir</code> , <code>verify_dynorder.py</code> dynorder robustness regression: the stiff $\mu A741$ slew under dynorder completes and matches fixed Gear-2

Scope

`dynorder` is a general transient-efficiency aid: on smooth problems it delivers 3–9× fewer timesteps at matched-or-better accuracy by using Gear orders the stock controller never reaches, and it is provably inert on ordinary (default `maxord=2`) decks and result-neutral on nonlinear switching circuits. The robust operating range is `maxord` 3–4; higher `maxord` is more aggressive (and, like all BDF beyond order 2, less robust at loose tolerance — the same reason stock ngspice caps its default Gear order at 2). Automatic engagement (turning it on when the dynamics are detected as smooth) and a per-device rather than whole-circuit order are natural follow-ups.

Enhancement-129 — sweep progress bar

A usability enhancement to ngspice's console output: the throttled "Reference value" status line printed during a DC / AC / transient / noise sweep now carries a live progress bar and percentage.

Before / after

ngspice already printed, every 0.25 s and redrawn in place with a carriage return, a status line showing the current sweep variable:

```
Reference value : 5.91926e-04
```

It tells you *where* the sweep is but not *how far along*. E-129 appends a bar:

```
Reference value : 5.91926e-04 [=====] 74%
```

The fraction

The 0–1 completion fraction is computed per analysis, in `outitf.c`, from data already on the circuit / job (`outp_progress_frac`):

- transient — elapsed time over the run: $(\text{CKTtime} - \text{TSTART}) / (\text{TSTOP} - \text{TSTART})$;
- AC and noise — the current frequency's position in the sweep band, linear for `lin` or logarithmic for `dec/oct`: $\log(f/f_0) / \log(f_1/f_0)$;
- DC — the accepted-point count over the product of the nested sweep step counts (so it is correct for multi-source `.dc` nesting).

Analyses with no well-defined span (operating point, transfer function, ...) return `-1` and keep the original plain line — no bar, no misinformation. The bar is a fixed width so the in-place redraw never leaves stale characters, and it is emitted through the same throttled, `!ft_norefprint && !cp_background` path as before (so `-o / background / set nomodcheck-style` quiet runs are unaffected). It writes only to the status line on `stdout` — never into the rawfile or `wrdata` output.

Verification

`verify_progressbar.py` runs a long-enough sweep of each kind (past the 0.25 s throttle), decodes the carriage-return-updated line, and checks (22/22):

- the bar (`[...] NN%`) is emitted for `tran / AC / DC / noise`;
- the printed percentage matches the analytic sweep fraction at that reference value — within 0.5 % across all four (transient linear-in-time, AC/noise log-in-frequency, DC linear-in-source);
- the bar fill length is proportional to the percentage;
- the percentages are monotone non-decreasing and reach ≈ 100 %;
- an operating-point run prints no bar (and still returns its result).

It is a front-end output feature, independent of the linear solver, so it is checked once (the bytes are identical under Sparse and KLU).

Files changed

File	Change
ngspice-46/src/frontend/outitf.c	<code>outp_progress_frac()</code> (per-analysis 0–1 fraction) + <code>outp_print_reference()</code> (status line with bar); the six "Reference value" print sites now call the helper. Includes <code>acdefs.h</code> / <code>trcvdefs.h</code> for the AC/DC job structs

File	Change
examples/ progressbar_ examples/	progressbar_demo.cir, verify_progressbar.py

Scope

Covers the four swept analyses (DC, AC, transient, noise); non-swept analyses are untouched. A natural follow-up is a bar for the periodic-steady-state shooting loop (`.pss` and the RF suite built on it), whose progress is the shooting-iteration count rather than a swept reference variable.

Enhancement-130 — built-in Nelder-Mead optimizer

A new front-end command, `optimize`, gives ngspice a built-in parameter optimizer: it varies a set of circuit/device parameters, re-runs an analysis, and minimizes a scalar objective expression — a derivative-free downhill-simplex (Nelder-Mead) search. Previously this had to be scripted by hand in a `.control` loop.

Usage

```
optimize -param <name> <init> <lo> <hi> [-param ...]
        -analysis <command ...>
        -minimize <expression ...>
        [-maxiter <N>] [-tol <T>] [-verbose]
```

- **-param name init lo hi** — a parameter to vary. name is an alter target: a device instance (R1, C1) or a device parameter (@m1[w]). Up to 16.
- **-analysis <cmd>** — the analysis to run each iteration (op, ac dec 20 1 1meg, tran 1u 1m, ...). Collects every token up to the next `<letter>` flag, so no quoting is needed.
- **-minimize <expr>** — the scalar cost to minimize, an ordinary ngspice expression over the result vectors, e.g. $(v(out)-0.3)^2$ or $(mag(v(out))-0.5)^2$. Its last value is used.
- **-maxiter** (default 100), **-tol** (default 1e-6), **-verbose** (print the cost each iteration).

Each candidate is applied in place with `alter <name>=<value>` (no re-source), the analysis is run, and the objective is evaluated. The search runs in normalized [0,1] parameter space — each parameter is mapped to its [lo,hi] range — so it is scale-invariant across parameters that differ by orders of magnitude (e.g. a kilohm resistor and a nanofarad capacitor optimized together). On convergence the circuit is left at the optimum and the best parameter values + final cost are printed.

Implementation notes

- The command lives in `frontend/com_optimize.c`. The Nelder-Mead simplex (reflection / expansion / contraction / shrink, bounds-clamped) is standard.
- Sub-commands (`alter`, the analysis) are dispatched synchronously through the command table (`cp_coms`) rather than `cp_evloop()`, which — called re-entrantly from inside a command — would *defer* them to the outer interpreter loop.
- The hundreds of inner analyses would otherwise flood the console. A new `ft_optimizing` flag (set only during an evaluation) gates the "Doing analysis" banner (`cktdojob.c`), the "No. of Data Rows" line, and the E-129 progress bar (`outitf.c`) at their source — an external `stdout` redirect does not survive `docommand's cp_ioreset()`. `-verbose` disables the gate.

Verification

`verify_optimize.py` optimizes circuits with known analytic optima and confirms the command reaches them (9/9):

- DC divider — minimize $(v(out)-0.3)^2$ over R1 (R2=1k). $v(out)=R2/(R1+R2) \Rightarrow R1 = 2333.3 \Omega$ exactly; the optimizer finds 2333.33, $v(out)=0.3$.
- AC low-pass — minimize $(mag(v(out))-0.5)^2$ over R1 at 1 kHz (C=100n). $|H|=1/\sqrt{(1+(2\pi fRC)^2)}=0.5 \Rightarrow R = 2756.6 \Omega$; found 2756.6, $|H|=0.5$.
- Two parameters (2-D simplex) — a divider where R1=3k, R2=2k uniquely gives $v(out)=0.4$ and $R1+R2=5k$ ($i(V1)=-0.2$ mA); minimize the compound objective $(v(out)-0.4)^2 + (|i(V1)|-0.2m)^2$. Found R1=3000, R2=2000 exactly.
- Output is quiet by default (1 banner for ~67 evaluations); `-verbose` prints per-iteration progress.

A front-end command, independent of the linear solver, so it is checked once.

Files changed

File	Change
ngspice-46/src/frontend/ com_optimize.c / .h	the optimize command: option parsing, Nelder-Mead, synchronous sub-command dispatch, objective evaluation (ft_getpnames_from_string/ft_evaluate)
ngspice-46/src/frontend/ commands.c, com_commands.h, Makefile.am (+Makefile.in)	register + build the command
ngspice-46/src/frontend/ options.c, include/ngspice/ fteext.h	the ft_optimizing quiet flag
ngspice-46/src/spicelib/ analysis/cktdojob.c, frontend/outitf.c	gate the per-analysis banner / row count / progress bar on ft_ optimizing
examples/optimize_examples/	optimize_demo.cir, verify_optimize.py

Scope

A general Nelder-Mead optimizer over alter-able parameters, verified to reach analytic optima in 1-D and 2-D. Natural follow-ups: gradient / least-squares methods for smooth problems, multi-analysis objectives (combine several runs), and optimizing .param values directly (which needs a re-source that does not re-run the analysis).

Enhancement-131 — transient checkpoint / restart

Two new front-end commands, **savestate** and **loadstate**, let a (long) transient run be saved to disk and later resumed — including in a fresh ngspice process. A multi-hour analysis can now survive a crash, be split across sessions, or be moved to another machine.

Stock ngspice can only continue a *paused* run in memory (`stop ... resume`): the integration state lives in the CKTcircuit, so once the process exits it is gone. Enhancement-131 serializes that state to a file and rehydrates it into an identically-built circuit, then continues the transient exactly where it left off.

Usage

```
* run part of a transient and checkpoint it
.tran 1u 1m
.control
run
savestate run.ckpt          ; dump the current transient state to disk
.endc
.end

* later -- possibly a fresh process -- resume and continue to 2 ms
.tran 1u 2m                ; the .tran tstop is the new end time
.control
loadstate run.ckpt          ; restore the state and continue the run
wrdata out.dat v(out)
.endc
.end
```

- **savestate <file>** — write the active circuit's current transient state. Valid after a transient has advanced (`CKTtime > 0`); the file is binary and specific to the machine/build that wrote it.
- **loadstate <file>** — restore the state into the active circuit (which must be the same deck) and continue the transient defined by the deck's `.tran` line. The `.tran tstop` becomes the new end time, so the run may be resumed to its original end (crash recovery) or extended past it.

Checkpoint files store a signature (equation count, matrix size, state-vector length, integration order); restoring into a different circuit is rejected with a clear message rather than crashing.

What is saved

Exactly the state the in-memory resume relies on, per CKTcircuit:

- the solution vector `CKTrhsOld` (and `CKTirhsOld`), sized `SMPmatSize+1`;
- the device integration history `CKTstates[0..7]` (charges, currents and their divided differences), each `CKTnumStates` long;
- the current `CKTtime`, `CKTdelta`, `CKTsaveDelta`, `CKTdeltaOld[7]`, `CKTorder`, `CKTmode`, `CKTminBreak`, `CKTdelmin`;
- the pending breakpoint table `CKTbreaks`.

How it works

loadstate:

1. builds the circuit if it is not set up yet (`CKTsetup / CKTtemp`), so the state vectors exist to fill;
2. validates the file signature against the circuit — the solution vector length is keyed off `SMPmatSize(matrix)` (which is not `CKTmaxEqNum`, and is what `NIreinit` actually allocates), so the reads cannot overrun;

3. pours the saved arrays back into CKTrhsOld / CKTstates / CKTbreaks and the scalar time/step/order fields;
4. sets a new one-shot ckt→CKTcheckpoint flag and drives the analysis through the existing resume path (if_run(ckt, "resume", ...)).

DCtran() gains a checkpoint branch (guarded by CKTcheckpoint) that, unlike the in-memory resume, opens a fresh output plot — there is no live plot to 666-relink across a reload — and then jumps into the time-stepping loop with the restored state. It also:

- initializes the XSPICE temporary-breakpoint markers (g_mif_info.breakpoint.current/last = 1e30) — the in-memory resume inherits these from the original run, but a fresh process must set them, or the stepping loop forces CKTdelta = breakpoint.current - CKTtime = -CKTtime;
- rebuilds a minimal breakpoint list (dropping any restored breakpoints at/before the resume time, keeping genuine future ones, and ensuring the — possibly extended — final time is present). Sources (PULSE, SIN, ...) re-schedule their own future edges as the run proceeds.

CKTcheckpoint is a new zero-initialized bit-field on CKTcircuit, so ordinary runs never take the branch.

Solver scope. Checkpoint/restart is supported with the default Sparse 1.3 solver. Under KLU the symbolic/numeric factorization objects are only built during a full run's operating point and are absent on the restore path; both savestate and loadstate reject KLU with a clear message (remove 'option klu') rather than crashing.

Verification

verify_checkpoint.py runs, for each circuit, an uninterrupted reference transient ($0 \rightarrow T_2$), then a split run: part 1 ($0 \rightarrow T_1$) + savestate, then — in a separate ngspice process — loadstate and continue to T_2 . The resumed waveform must match the reference at the end and at an interior sample time (meas ... FIND v(out) AT=...). 19/19 checks pass:

- RC step (linear, uic charging) — resumed end/interior values bit-identical.
- RC pulse (PULSE source) — exercises breakpoint save/restore and source re-scheduling; end value bit-identical, 0.5831499 either way.
- Diode rectifier (nonlinear built-in D, sine drive) — end value bit-identical after 1150 nonlinear steps.
- OSDI diode (compiled Verilog-A, reactive + nonlinear) — end value matches to $\sim 2 \times 10^{-7}$ (OSDI devices carry a little instance-internal state outside CKTstates).
- same-session savestate+loadstate reaches $1 - e^{-2} = 0.8647$.
- robustness — a checkpoint restored into a different circuit is rejected; the KLU solver is rejected with a clear message and writes no file.

Files changed

File	Change
ngspice-46/src/frontend/com_checkpoint.c / .h	new — the savestate / loadstate commands (binary serialize / restore, signature validation, KLU guard) register + build the two commands
ngspice-46/src/frontend/commands.c, com_commands.h, Makefile.am (+Makefile.in)	
ngspice-46/src/spicelib/analysis/dctran.c	the DCtran() checkpoint branch (fresh plot + restored-state continuation + breakpoint fix-up + XSPICE marker init)

File	Change
ngspice-46/src/include/ngspice/ cktdefs.h	new one-shot CKTcheckpoint bit-field
examples/checkpoint_examples/	verify_checkpoint.py, ckdiode.va

Scope

Disk checkpoint/restart of a transient run, verified bit-identical (built-in devices, Sparse solver) across linear, breakpoint-driven, nonlinear and OSDI circuits, including a fresh process. Natural follow-ups: KLU support (rebuild the factorization on restore), checkpointing other analyses, an architecture-portable file format, and a periodic auto-checkpoint option during a long run.

Enhancement-132 — periodic S-parameters (**.psp**)

A new analysis, **.psp**, computes periodic small-signal S-parameters: the scattering matrix of a circuit linearised around a periodic (large-signal) operating point, including conversion between the input frequency and its sidebands $f_{in} + k \cdot f_0$. This is the S-parameter view of a mixer, switched circuit, or any periodically-pumped network — where an ordinary **.sp** (which linearises around a static DC point) cannot see the frequency conversion.

.psp sits on top of the PSS → conversion-matrix suite (E-117–126): it reuses the periodic operating point (E-119), the harmonic Jacobian (E-120), and the same $(2M+1)N$ conversion matrix the PAC / pnoise / PXF analyses share (E-121). It also reuses the RFSPICE port framework (portnum / z0 on voltage sources) and the exact power-wave convention of **.sp**.

Usage

* ports are voltage sources tagged with portnum + z0 (as for **.sp**)

```
V1 in 0 DC 0 AC 1 portnum 1 z0 50
```

```
V2 out 0 DC 0 AC 1 portnum 2 z0 50
```

```
...
```

```
* .psp Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff
```

```
* <DEC|OCT|LIN> NumPts Fstart Fstop [maxsideband]
```

```
.psp 1meg 1u osc 1024 10 50 5u dec 20 10k 1meg 1
```

```
.control
```

```
run
```

```
print s_2_1 s_1_1 ; sideband-0 S-parameters
```

```
print s_2_1_usb1 s_2_1_lsb1 ; ±1 conversion sidebands (with maxsideband ≥ 1)
```

```
.endc
```

- The leading arguments are the ordinary PSS parameters (guessed frequency, stabilisation time, oscillator/probe node, sample points, harmonics, shooting iterations, steady-state coefficient) — identical to **.pss** / **.pac**.
- Then a frequency sweep of the small-signal input frequency: **<dec|oct|lin> NumPts Fstart Fstop**.
- Optional trailing **maxsideband Ksb** emits the conversion sidebands: the output vectors are $S_{<j>,<i>}$ (sideband 0) plus $S_{<j>,<i>_usb<k>}$ / $S_{<j>,<i>_lsb<k>}$ for the upper/lower conversion bands. $S_{<j>,<i>}$ is the wave from source port i to measured port j .

Output is a complex PSP Analysis plot vs frequency; sideband-0 uses the plain $S_{j,i}$ names so it lines up directly with an ordinary **.sp** run.

How it works

After PSS converges, **psp_sweep** (in **dcpsp.c**):

1. extracts the Jacobian harmonics once (**pac_extract_harmonics**, E-120/121);
2. at each swept input frequency, excites each RF port in turn — driving that port's voltage source to 1 in the 0-th sideband through the conversion matrix (exactly as **.sp**'s **VSRCspupdate** does, so the other ports stay z_0 -terminated by the $g_0 = 1/z_0$ shunt already stamped in the Jacobian);
3. reads each port's voltage and branch current at every sideband and forms the Kurosawa power waves $a = k_i \cdot (V + z_0 \cdot I)$, $b = k_i \cdot (V - z_0 \cdot I)$ with $k_i = 1/(2\sqrt{z_0})$ — the same definition **.sp** uses;
4. assembles the incident matrix A (sideband 0) and one scattered matrix $B^{(k)}$ per output sideband, and computes $S^{(k)} = B^{(k)} \cdot A^{-1}$ with the dense complex library (**cinverse** / **cmultiply**) **.sp** also uses.

Because $S = B \cdot A^{-1}$ is invariant to the excitation basis, driving each port with a unit voltage yields the same S as `.sp`'s power-wave excitation. For a time-invariant circuit the conversion matrix is block-diagonal (all Jacobian harmonics above $h = 0$ vanish), so the sideband-0 block reduces exactly to the ordinary `.sp` S -matrix and every conversion sideband is zero.

Implementation notes: `pac_solve_at`'s conversion-matrix assembly was factored into a shared `pac_build_matrix` used by both the PAC solve and the new per-port PSP solve `psp_solve_port`; a `PSSdoPSP` flag + `psp` PSS parameter carry the mode; the `.psp` card is parsed by `dot_psp` (a sibling of `dot_pac`). Like the rest of the periodic small-signal suite it runs under both linear solvers: the $(2M+1)N$ conversion matrix is solved with a standalone dense complex LU (`pss_csolve`) that is independent of KLU/Sparse, and PSS itself runs under both since E-118 (KLU re-factors each shooting step, so it is *slower* but not wrong). Verified bit-identical under `.option klu` and `.option sparse`.

Verification

`verify_psp.py` builds each network, runs `.sp` (reference) and `.psp`, and compares the complex S -parameters. For a time-invariant circuit PSP's sideband 0 must equal `.sp` exactly and its conversion sidebands must be zero (8/8 checks):

- resistive 2-port — $S = B \cdot A^{-1}$ matches `.sp` to $\sim 10^{-16}$.
- reactive 2-port (R + series L + shunt C) — complex S (magnitude and phase) matches `.sp` across a 100–500 MHz sweep, $\max |\Delta S| = 0$.
- 1-port reflection — 75Ω on a 50Ω port gives $\Gamma = 0.2$ exactly (N-general machinery down to a single port).
- 3-port resistive star — matches `.sp` for $N > 2$.
- conversion sidebands — with `maxsideband = 1`, a time-invariant network's ± 1 conversion terms are $\sim 10^{-17}$ while sideband 0 is not (the sideband machinery is exercised and correctly yields zero conversion with no pumping).
- reciprocity — $S_{21} == S_{12}$ for a passive network.
- OSDI / Verilog-A — a compiled VA-resistor (conductance stamp) + VA-capacitor (reactive `ddt` stamp) 2-port matches `.sp` to $\max |\Delta S| = 0$ across a 100–500 MHz sweep, confirming OSDI device G and C Jacobian stamps are captured in the conversion matrix.

Files changed

File	Change
<code>ngspice-46/src/spicelib/analysis/dcpss.c</code>	<code>psp_sweep</code> (per-port conversion-matrix solve $\rightarrow$ periodic S -matrix) + <code>psp_solve_port</code> ; <code>pac_build_matrix</code> factored out of <code>pac_solve_at</code> ; PSP dispatch in DCpss
<code>ngspice-46/src/spicelib/parser/inp2dot.c</code>	<code>dot_psp</code> parser + <code>.psp</code> card dispatch
<code>ngspice-46/src/spicelib/analysis/psssetp.c</code>	<code>psp</code> PSS parameter (setter + table)
<code>ngspice-46/src/include/ngspice/pssdefs.h</code>	<code>PSSdoPSP</code> flag + <code>PSP_D0PSP</code> param id
<code>examples/psp_examples/</code>	<code>verify_psp.py</code> , <code>psp_dev.va</code> (VA resistor + capacitor for the OSDI check)

Scope

Periodic small-signal S-parameters around a PSS operating point, verified to reduce exactly to .sp for time-invariant networks (1/2/3-port, resistive and reactive, magnitude and phase) with correctly-zero conversion sidebands. The conversion machinery is N-port and multi-sideband by construction, so a pumped network reports its frequency-conversion S-parameters; a fast, analytically-checked pumped-mixer reference (mixer conversion gain vs an ideal-switch model) is the natural follow-up, along with periodic noise figure and a Touchstone writer for the sideband blocks.

Enhancement-133 — quasi-periodic (two-tone) steady state (**qpss**)

A new command, **qpss**, computes the two-tone / multi-fundamental steady-state spectrum: driven by two tones f_1 and f_2 , it resolves the response into every mixing product $k_1 \cdot f_1 + k_2 \cdot f_2$, including the third-order intermodulation (IM3 at $2f_1 - f_2$ / $2f_2 - f_1$) that a single-tone AC or PSS analysis cannot show. This is the workhorse of RF linearity characterisation — two-tone tests, IM3/IP3, spectral regrowth.

qpss <expr> <f1> <f2> [periods] [maxorder]

- **<expr>** — the output to analyse, e.g. $v(out)$.
- **<f1> <f2>** — the two tone frequencies (the two-tone sources are in the netlist as ordinary SIN sources).
- **periods** — number of beat periods to run for the transient to settle (default 8); increase for high-Q circuits.
- **maxorder** — report products up to $|k_1| + |k_2| \leq \text{maxorder}$ (default 5).

Output is a labelled spectrum — each product's 2-D harmonic index (k_1, k_2) , its frequency, magnitude and phase:

QPSS: two-tone steady state of $v(out)$

$f_1 = 1e+08$ Hz, $f_2 = 1.1e+08$ Hz, beat $fb = 1e+07$ Hz; 4 beat periods, order ≤ 3

(k_1, k_2)	frequency [Hz]	value	phase [deg]	
(1, 0)	1.000000e+08	1.124860e-03	-90.000	<- f1
(0, 1)	1.100000e+08	1.124856e-03	-90.000	<- f2
(2,-1)	9.000000e+07	3.749541e-04	-90.000	<- IM3 ($2f_1 - f_2$)
(-1, 2)	1.200000e+08	3.749499e-04	-90.000	<- IM3 ($2f_2 - f_1$)

Method

For two commensurate tones (a rational ratio, so they share a beat frequency $fb = \text{gcd}(f_1, f_2)$) the circuit's steady state is periodic at fb . **qpss**:

1. auto-derives fb from the two tones (Euclidean algorithm on the reals);
2. runs an ordinary transient over periods beat periods (to reach steady state) at a step fine enough to resolve the highest reported harmonic;
3. takes the last beat period and evaluates the Fourier coefficient directly at each exact intermod frequency $k_1 \cdot f_1 + k_2 \cdot f_2$ by trapezoidal integration (a direct DFT at the known frequencies — exact for commensurate tones, with no resampling or FFT-bin rounding);
4. labels each product by its 2-D harmonic index (k_1, k_2) .

This transient-sampling method is deliberately not a beat-frequency shooting PSS: shooting is slow, needs reactive state, and here would integrate the whole beat period per iteration. The transient runs once and is fast (~ 0.1 s for the memoryless examples) and robust. As a front-end command it drives an ordinary transient, so it is independent of the linear solver (KLU or Sparse) and works with built-in and OSDI/Verilog-A devices alike.

Verification

verify_qpss.py drives a memoryless weak nonlinearity $i = g_1 \cdot v + g_3 \cdot v^3$ with two tones and checks the analytically-known two-tone products (11/11):

- fundamentals / IM3 / 3f exact — for a pure cubic ($g_3=0.5$, $A=0.1$) the fundamentals are $1.125e-3$, the IM3 products $3.75e-4$, the third harmonics $1.25e-4$, matching $0.5 \cdot (A(\sin + \sin))^3$ to $<2\%$.
- no even-order products — an odd nonlinearity's $(-1, 1)$, $(2, 0)$, ... terms are $\sim 10^{-9}$ (correctly zero).

- IP3 slope law — with a linear+cubic mix, halving the drive drops the fundamental $2\times$ (slope 1) and the IM3 $8\times$ (slope 3) — the defining 3rd-order intermodulation behaviour.
- beat-frequency derivation — $100/110\text{ MHz} \rightarrow \text{fb} = 10\text{ MHz}$; $30/33\text{ MHz} \rightarrow \text{fb} = 3\text{ MHz}$.
- OSDI / Verilog-A — a compiled controlled cubic (`vacube.va`) gives the same fundamentals and IM3 as the built-in behavioural source.

Files changed

File	Change
<code>ngspice-46/src/frontend/com_qpss.c / .h</code>	new — the <code>qpss</code> command: derive fb, run the two-tone transient, direct-DFT the last beat period at each $k_1 \cdot f_1 + k_2 \cdot f_2$, print the labelled 2-D spectrum
<code>ngspice-46/src/frontend/commands.c, com_commands.h, Makefile.am (+Makefile.in)</code>	register + build the command
<code>examples/qpss_examples/</code>	<code>verify_qpss.py, vacube.va</code>

Scope

Commensurate two-tone (multi-tone by extension) steady-state spectrum via transient sampling, verified against the analytic intermodulation products and the IP3 slope law, for built-in and OSDI devices. This covers the common two-tone-IMD / linearity use case. Honest limitations and natural follow-ups: incommensurate tones (an irrational ratio, no common period) need a true frequency-domain harmonic-balance engine; because the tones here are commensurate, high-order products can *fold* onto the same frequency (the reported (k_1, k_2) is the lowest-order label at that frequency — unambiguous for the closely-spaced tones of a real two-tone test); and a small-signal QPAC (periodic AC around the two-tone point) would extend the suite as PAC did for PSS.

Enhancement-134 — Harmonic Balance (**hb**)

A new command, **hb**, computes the periodic steady state directly in the frequency domain by Newton, instead of integrating in time. Each node voltage is a truncated Fourier series $v_i(t) = \sum_{k=-K}^K V_{i,k} e^{jk\omega_0 t}$, and the KCL residual at every node and harmonic is driven to zero. Unlike a transient (or the qpss transient-sampling of E-133), HB needs no settling — it converges on high-Q/slow-settling circuits in a handful of iterations — and it is the classic engine for nonlinear RF steady state (power amplifiers, mixers, multipliers).

ngspice shipped only an unimplemented WITH_HB stub; this is the real analysis.

hb <f0> <K> [points] [maxiter]

- **<f0>** — the fundamental (drive) frequency.
- **<K>** — number of harmonics to solve (0...K, so the system is $(2K+1) \cdot N$).
- **points** — time samples per period for the device evaluation (default ~8K).
- **maxiter** — Newton iteration cap (default 50).

Output is a labelled spectrum — for every node, the magnitude and phase at each harmonic $k \cdot f_0$. set **hb_verbose** prints the per-iteration residual norm.

Method

The residual at node/harmonic k is $F_k = I_{R,k}(V) + [dq/dt]_k - I_{s,k} = 0$, solved by Newton with the **(2K+1)N** conversion matrix (E-121) as the exact Jacobian. Each iteration:

1. inverse-DFT $V_k \rightarrow$ node voltages $v(t_s)$ at P samples over one period;
2. at each sample, drive the device loads at those voltages. Junction devices (diode/BJT/MOS) limit their internal voltage against a stored value, so a single load leaves them pinned at a stale bias (and MODEINITSMSIG alone reads the stored op-point, ignoring the node voltage — which made real diodes look *linear*). So the sample first settles the device: repeated MODEINITFLOAT loads walk the limited junction to the fixed node voltages until the limiter is a no-op, yielding the true companion b and hence the actual resistive current $I_R = G \cdot v - b$ (not the tangent $G \cdot v$). A following MODEINITSMSIG load then reads the settled bias to build the small-signal linearization, and an AC load at $\omega=1$ gives $C(t)$. Behavioural / OSDI devices with no limiting settle on the first pass;
3. DFT $I_R(t_s), G(t), C(t) \rightarrow$ the residual and the conversion-matrix harmonics;
4. dense complex Newton solve (**pss_csolve**) $\rightarrow$ update V_k .

The key that makes nonlinear reactive elements work with no per-device charge extraction: the reactive current is $dq/dt = C(v) \cdot v'$ by the chain rule, so its spectrum is exactly the conversion matrix's reactive term $\sum_m j m \omega_0 \cdot C_{\{k-m\}} \cdot V_m$ applied to V — using the same $C(t)$ samples the Jacobian already needs. Nonlinear charge $Q(v)$ (varactors, junction caps) falls out for free; the nonlinearity is in the sampled $C(t)$.

The independent-source excitation $I_{s,k}$ is captured by loading the circuit at zero node voltage with the sources evaluated at each sample time. Reuses **pac_build_matrix** (Jacobian) and **pss_csolve** (dense solve) from the PSS/PAC suite.

Solver-independent (KLU *and* Sparse). HB does its own dense complex Newton solve, so the sparse linear solver is used only to *read* the periodic $G(t)/C(t)$ off the device matrix. Under Sparse that is **spSetComplex**; under KLU the matrix is a complex CSC, so **hb_extract** binds the device pointers with **DEVbindCSCComplex** (the same guard **pac_extract_harmonics** uses). The **hb** command also copies the task's KLU mode before building the circuit, so a bare **hb** honours **.option klu**. Verified bit-identical under both solvers (the diode rectifier gives the same 18-iteration spectrum either way).

Verification

`verify_hb.py` drives nonlinear circuits with a tone and compares HB's spectrum against ngspice's own transient + fourier steady-state harmonics (8/8):

- nonlinear resistor — fundamental and 3rd harmonic match; Newton converges quadratically in 3 iterations; even-order products are ~0.
- nonlinear R + linear C — the reactive roll-off/phase is captured; still matches the transient fourier.
- built-in diode half-wave rectifier — a real junction device with internal voltage limiting; the sharp rectified waveform's DC...3rd harmonics match the transient to <0.3 % (this is the hard case the per-sample settling above unlocks).
- nonlinear C (OSDI varactor) — a $Q = c j 0 \cdot (V + \gamma V^2)$ charge makes a real 2nd harmonic ($|V_2| \approx 1.4e-2$) that a linear cap cannot; HB matches the transient — the full nonlinear-reactive result, on a compiled Verilog-A device.
- solver parity — the diode rectifier gives a bit-identical spectrum under `.option klu` and `.option sparse` (HB's own dense Newton makes the linear solver a read-only detail).

Also confirmed by hand: HB reproduces the analytic cubic 3rd harmonic $|v_3| = R \cdot g_3 \cdot |v_1|^3/4$, and works under both current- and voltage-source excitation.

Files changed

File	Change
ngspice-46/src/ spicelib/analysis/ dcpss.c	the HB engine: <code>hb_extract</code> (samples I_R/G/C at prescribed voltages, complex-mode via <code>spSetComplex</code> on Sparse or <code>DEVbindCSCComplex</code> on KLU) and <code>HBanalyze</code> (source spectrum + frequency-domain Newton + spectrum output), reusing <code>pac_build_matrix</code> + <code>pss_csolve</code>
ngspice-46/src/ frontend/com_hb.c / .h, commands.c, com_ commands.h, Makefile. am	the hb command (parse, honour <code>.option klu</code> , build the circuit, run <code>HBanalyze</code>)
ngspice-46/src/ include/ngspice/ cktdefs.h	<code>HBanalyze</code> prototype
examples/hb_ examples/	<code>verify_hb.py</code> , <code>vavar.va</code> (nonlinear-cap varactor)

Scope

Single-tone Harmonic Balance with nonlinear resistive and nonlinear reactive devices, current- or voltage-source driven, built-in and OSDI, verified against the transient steady state with quadratic Newton convergence. Honest limitations / follow-ups: strongly-driven circuits (nonlinearity comparable to the linear term) converge but need many iterations — source-stepping / continuation would fix that; the dense $(2K+1)N$ solve is capped for modest circuits (a sparse block solver would scale it); and multi-tone HB (true incommensurate QPSS, extending E-133 and this engine to a 2-D harmonic set) is the next step.

Enhancement-135 — Harmonic-Balance source-stepping continuation

The Harmonic Balance engine of E-134 solves the periodic steady state by Newton on the $(2K+1)N$ conversion matrix. Newton converges quadratically — once it is close enough. For a strongly-driven circuit, where the nonlinearity is comparable to the linear term (a power amplifier near compression, a sharp diode rectifier at large amplitude), a cold full-strength Newton from $V = 0$ overshoots and diverges (the residual blows up to $1e69$ in a step or two).

This enhancement makes hb robust on those circuits with source-stepping continuation — the standard homotopy for hard steady-state problems. No new syntax: it is automatic and transparent.

Method

Every independent source is scaled by a homotopy factor λ , and HB solves a sequence of problems $\lambda: 0 \rightarrow 1$, each warm-started from the previous solution:

- at $\lambda = 0$ the circuit is unexcited and the solution is $V = 0$ (known exactly);
- each level solves $F(V; \lambda) = I_R(V) + [dq/dt](\text{https://github.com/javaNoviceProgrammer/Ngspice_OpenVAF_Enhancements/blob/main/enhancements_doc/V}) - \lambda \cdot I_s = 0$ by the same Newton as E-134, starting from the last converged V ;
- as λ climbs, the steady state moves along a continuous path, so the warm start is always inside Newton's basin.

The stepping is adaptive with backtracking, so it costs nothing on easy circuits and subdivides only as much as a hard one needs:

- the first level is full strength ($d\lambda = 1$), so a circuit that would have converged directly still converges at $\lambda = 1$ on the first try — bit-identical to the plain solve;
- if a level fails (Newton exhausts its iterations, the residual goes non-finite, or the Jacobian is singular), $d\lambda$ is halved and the level retried from the last converged V ;
- if a level converges, $d\lambda$ grows ($\times 1.7$) so the ramp accelerates once past the hard region;
- if $d\lambda$ collapses below $1e-5$ the circuit is reported as having no reachable steady state at that drive (singular or non-convergent), rather than silently returning garbage.

All independent sources — bias and drive — ramp together (classic source stepping). To sweep the RF drive at a fixed DC bias instead, step the drive amplitude with `alter` across separate hb calls.

`set hb_verbose` now prints the λ of each level alongside the per-iteration residual, so the continuation path is visible. The final line reports the total Newton iterations and the number of continuation steps, e.g. HB: converged in 62 iterations, 3 continuation steps ($|F| = 1.7e-11$).

Verification

`verify_hb.py` gains a strongly-driven check (now 9/9):

- strongly-driven diode rectifier — 5 V into a $20\ \Omega$ source resistance and a sharp $IS=1e-14$ junction. A cold full-strength Newton diverges ($|F| \rightarrow 1e69$); with continuation HB ramps through 3 steps (backing off to $\lambda = 0.25$, then climbing) and converges. The spectrum matches the transient fourier steady state to $<0.1\%$ for DC, f_0 and $2f_0$ (DC 1.2442 vs 1.2438, f_0 2.0273 vs 2.0262, $2f_0$ 1.0004 vs 1.0008).

The other eight checks are unchanged and bit-identical — they all converge at $\lambda = 1$ on the first level, so continuation adds no iterations and no numerical difference on circuits that never needed it.

Files changed

File	Change
ngspice-46/ src/ spicelib/ analysis/ dcpss.c	wrap HBAnalyze's Newton loop in the adaptive λ -continuation loop: scale I_s by λ , warm-start/checkpoint/restore V per level, backtrack on failure, grow on success; report iterations + continuation steps
examples/hb_ examples/ verify_hb.py	strongly-driven rectifier check (diverges cold, converges via continuation, matches transient)

Scope

Single-tone HB (E-134) made robust for strongly-driven / stiff nonlinear circuits via automatic adaptive source-stepping. Follow-ups (unchanged from E-134): a sparse block solve to scale past the dense ($2K+1$) $N \leq 900$ cap, and multi-tone HB (true incommensurate QPSS). A drive-only continuation (bias held fixed) is a natural refinement of the all-sources ramp used here.

Enhancement-136 — Two-tone Harmonic Balance (frequency-domain QPSS)

The E-133 qpss computes a two-tone steady state by running a transient over a few beat periods and DFT-ing the last one. That only works for commensurate tones (a rational ratio, so they share a beat period); genuinely incommensurate tones (an irrational ratio — no common period) are out of its reach, and it produces no operating point a small-signal analysis could linearize around.

This enhancement adds a frequency-domain two-tone Harmonic Balance engine, selected with a new hb keyword:

```
qpss <expr> <f1> <f2> hb [K1] [K2]
```

It is the true quasi-periodic steady state — incommensurate-capable — and it retains its operating point for the small-signal qpac (Enhancement-137). The transient path (qpss <expr> <f1> <f2> [periods] [maxorder], E-133) is unchanged.

Method

Each node voltage is a 2-D Fourier series over the truncated harmonic box $|k_1| \leq K_1, |k_2| \leq K_2$:

$$v(t) = \sum_{k_1, k_2} V_{k_1, k_2} \cdot e^{j(k_1 \omega_1 + k_2 \omega_2) \cdot t}$$

The KCL residual $F_{k_1, k_2} = I_R + [dq/dt] - I_s = 0$ is driven to zero by Newton with the 2-D conversion matrix $H_{(n), (m)} = G_{n-m} + j\omega_m \cdot C_{n-m}$ as the Jacobian — the direct 2-D analogue of the E-121 matrix. Two ideas make it work for incommensurate tones:

1. Devices are sampled on a 2-D *phase* grid (**θ_1, θ_2**). At each grid point the node voltages $v(\theta_1, \theta_2)$ are prescribed and the device loaded, so G and C are read as functions of the two phases — *time never appears*, so there is no common-period requirement. A 2-D DFT gives the difference spectra $G_{d_1, d_2}, C_{d_1, d_2}$. As in E-134, the reactive current needs no charge extraction ($dq/dt = C(v) \cdot v' \rightarrow$ the $j\omega \cdot C$ term of the conversion matrix), and junction devices are settled per sample.
2. Sources are captured by an oversampled least-squares APFT. The $v=0$ source RHS is sampled at $N_t \ll N_h$ real times and the almost-periodic Fourier transform $(\Gamma^H \Gamma) I_s = \Gamma^H b$ is solved for the 2-D source spectrum. Oversampling makes $\Gamma^H \Gamma$ well conditioned where a *square* Vandermonde APFT is catastrophically unstable beyond a handful of harmonics.

The dense complex Newton (pss_solve) and the E-135 source-stepping continuation are reused unchanged. Because the linear solver is only used to *read* G/C off the device matrix, the engine carries the same `#ifdef KLU` complex-CSC binding as the PAC / HB extraction and is solver-independent (KLU and Sparse bit-identical). A harmonic box whose tones alias two indices to the same frequency (too commensurate for the order) is rejected with a clear message rather than producing a singular transform.

Verification

verify_qpss_hb.py (7/7), numpy-free, against analytic two-tone mixing and the E-133 transient:

- analytic cubic IM3 — for equal tones the small-signal ratio $|IM3(2, -1)| / |3rd(3, 0)| = 3$ exactly;
- odd nonlinearity — even-order products vanish;
- 3:1 IP3 slope law — doubling the drive scales the fundamental $\times 2$ and IM3 $\times 8$;
- incommensurate tones ($f_2 = \sqrt{2} \cdot f_1$, no beat period) — converge to machine precision with the correct IM3, which the E-133 transient cannot compute;
- HB vs transient — the two methods agree on IM3 for a commensurate pair;
- solver parity — KLU and Sparse spectra are bit-identical (with a reactive C).

E-133's transient verify_qpss.py still passes 11/11 unchanged.

Files changed

File	Change
ngspice-46/src/ spicelib/ analysis/dcpss.c	the QPSS-HB engine: QPSShb (source APFT + frequency-domain Newton + 2-D spectrum output + operating-point retention), qp_extract (2-D phase-grid device sampling + 2-D DFT), qp_build_matrix (2-D conversion matrix), qp_synth, qp_free; retained op-point qpss_hb_saved for qpac
ngspice-46/src/ frontend/com_ qpss.c	detect the hb keyword, honour .option klu, build the circuit, call QPSShb
ngspice-46/src/ include/ngspice/ cktdefs.h, frontend/ commands.c	QPSShb prototype; qpss help string updated for the hb form
examples/qpss_ examples/verify_ qpss_hb.py	the 7-check HB-mode suite

Scope

Two-tone frequency-domain Harmonic Balance with nonlinear resistive and reactive devices, built-in and OSDI, incommensurate-capable, verified against analytic mixing and the transient QPSS. The operating point is retained for QPAC (Enhancement-137, the two-tone small-signal analogue of PAC). Follow-ups: more than two tones, a sparse block solve to scale past the dense $N_h \cdot N$ cap, and diamond (total-order) truncation to trim the harmonic set.

Enhancement-137 — Two-tone small-signal QPAC (quasi-periodic AC)

Enhancement-136 added the frequency-domain two-tone Harmonic Balance engine (qpss ... hb), the true quasi-periodic steady state, and had it retain its operating point. This enhancement adds the small-signal analysis that sits on top of it — QPAC, the two-tone analogue of PAC:

qpac <f_in>

Run after a qpss <expr> <f1> <f2> hb, it injects a small signal at f_in around the retained quasi-periodic operating point and reports the response at every sideband $f_{in} + k_1 \cdot f_1 + k_2 \cdot f_2$. A time-varying (here two-tone-pumped) operating point *mixes* the small signal to those sidebands — the effect a static .ac cannot see.

Method

QPAC is exactly pac_solve_at (E-121/122) on the two-tone harmonic set. Around the QPSS operating point retained in qpss_hb_saved, the small-signal Kirchhoff system is the 2-D conversion matrix $H_{\{(n),(m)\}} = G_{\{n-m\}} + j\omega_m \cdot C_{\{n-m\}}$ built at the input frequency (qp_build_matrix(hd, f_in, ...) — the same matrix the QPSS Newton used as its Jacobian, re-evaluated with $\omega = 2\pi(f_{in} + m_1 \cdot f_1 + m_2 \cdot f_2)$). The stimulus is placed in the (0,0) sideband — a netlist AC-flagged source's RHS B0 (captured, bias-independent, at the operating point) or a unit current fallback — and the dense complex solve (pss_solve) returns the full sideband response $X_{\{k_1, k_2\}}$ in one shot:

$$H(f_{in}) \cdot X = B_0 \rightarrow X_{\{k_1, k_2\}} = \text{response at } f_{in} + k_1 \cdot f_1 + k_2 \cdot f_2$$

Because the conversion data is read straight from the retained operating point, QPAC adds no device evaluation of its own and is solver-independent (KLU and Sparse bit-identical) by construction.

Verification

verify_qpac.py (7/7), numpy-free:

- reduce-to-AC — with the pump driven to ~0 the operating point is time-invariant, so the direct (0,0) response equals the plain .ac response (a 1 A stimulus into $R = 1 \text{ k}\Omega$ gives exactly 1000) and every conversion sideband vanishes — the essential correctness anchor;
- conversion ratio — under a real two-tone pump, $G(t) = 3g_3 v^2$ has an $f_1 \pm f_2$ harmonic twice its $2f_1$ harmonic, so the converted sidebands satisfy $| (1,1) | / | (2,0) | = 2$ exactly;
- conversion present — sidebands are non-zero under the pump (the mixing happens);
- tone symmetry — for equal tones $| (1,1) | = | (1,-1) |$ and $| (2,0) | = | (0,2) |$;
- clean failure — qpac without a prior qpss ... hb reports "no QPSS operating point" rather than crashing;
- solver parity — KLU and Sparse responses are bit-identical.

The QPSS suites are unaffected: E-136 verify_qpss_hb.py stays 7/7 and E-133 verify_qpss.py stays 11/11.

Files changed

File	Change
ngspice-46/src/spicelib/analysis/dcpss.c	QPACAnalyze (build the 2-D conversion matrix at f_in, inject the (0,0) stimulus, solve, print the sideband table); qp_harm gains B0r/B0i/has_src, captured at the QPSS operating point during retention

File	Change
ngspice-46/src/frontend/ com_qpac.c / .h, commands.c, com_commands.h, Makefile. am (+ Makefile.in)	the qpac command
ngspice-46/src/include/ ngspice/cktdefs.h	QPACanalyze prototype
examples/qpss_examples/ verify_qpac.py	the 7-check QPAC suite

Scope

Two-tone small-signal QPAC around the E-136 QPSS operating point, single input frequency per call, AC-source or unit-current stimulus, verified against the reduce-to-AC limit and analytic pump conversion. With E-136 this completes the quasi-periodic (QPSS / QPAC) gap. Follow-ups: an input-frequency sweep (like .pac's dec/oct/lin), quasi-periodic noise (QPnoise) and transfer function (QPXF) on the same retained operating point, and more than two tones.

Enhancement-138 — Two-tone small-signal QPnoise (quasi-periodic noise)

With the quasi-periodic steady state (E-136 qpss ... hb) and its small-signal response (E-137 qpac) in place, this enhancement adds the noise analysis on the same retained operating point — QPnoise, the two-tone analogue of pnoise:

qpnoise <output_node> <f_in>

Run after a qpss <expr> <f1> <f2> hb, it reports the output and input-referred noise density at f_in, folding every device's noise contribution over all sidebands $f_{in} + k1 \cdot f1 + k2 \cdot f2$. Under a two-tone pump a device's noise at each sideband is *converted* (folded) to the output — the effect behind mixer/PA noise figure that a static .noise cannot see.

Method

QPnoise is pnoise (E-124) on the two-tone harmonic set. The transfer from a noise injection at every (node, sideband) to the output is obtained from one adjoint solve of the 2-D conversion matrix:

$$H^T \cdot \Psi = e_{\{out, (0,0)\}}$$

qp_solve_adjoint reuses qp_build_matrix (the same matrix the QPSS Newton used as its Jacobian), transposes it, and puts a unit at the output node in the (0,0) sideband. $\Psi_{\{(k1,k2), j\}}$ is then the transimpedance from (node j, sideband (k1,k2)) to the output. The device noise routines (DEVnoise/NevalSrc, OSDI load_noise) already compute $S \cdot |transimpedance|^2$ reading the transimpedance from CKTrhs/CKTirhs, so loading each sideband's adjoint transfer into CKTrhs and summing over all Nh harmonics folds the noise exactly:

$$onoise(f_{in}) = \sum_{\{k1,k2\}} \sum_{\text{devices}} S_{dev} \cdot |\Psi_{\{(k1,k2)\}}|^2$$

The devices are biased at the quasi-periodic operating point (the phase-(0,0) sample of the retained V) so each PSD is at the periodic bias. Input-referred noise divides by the source→output conversion gain at (0,0) (the QPAC gain from the captured AC-source stimulus). Since QPnoise only *reads* the retained conversion data, it is solver-independent (KLU and Sparse bit-identical) by construction.

The key structural fact: with no pump the conversion matrix is block-diagonal, so Ψ is non-zero only in the (0,0) block and QPnoise collapses to ordinary .noise.

Verification

verify_qpnoise.py (6/6), numpy-free:

- reduce-to-noise — with the pump ~0, QPnoise's onoise equals the plain .noise onoise_spectrum at f_in to machine precision (the definitive correctness anchor);
- thermal law — that value is exactly 4kTR of the 1 kΩ resistor ($4 \cdot 1.380649e-23 \cdot 300.15 \cdot 1000$);
- conversion active — under a real two-tone pump the folded onoise differs from the no-pump value (the matrix is no longer block-diagonal; sidebands contribute and the pump-induced conductance loads the node);
- input-referred — $inoise = onoise / gain^2$;
- clean failure — qpnoise without a prior qpss ... hb reports "no QPSS operating point";
- solver parity — KLU and Sparse onoise are bit-identical.

The QPSS suites are unaffected: E-133 stays 11/11, E-136 7/7, E-137 7/7.

Files changed

File	Change
ngspice-46/src/ spicelib/analysis/ dcpss.c	QPnoiseAnalyze (bias at the QPSS operating point, minimal NOISEAN context, fold device noise over all sidebands via the adjoint, input-refer through the QPAC gain) and qp_solve_adjoint (transpose the 2-D conversion matrix, solve $H^T \Psi = e_{\{out, (0,0)\}}$)
ngspice-46/src/ frontend/com_qpnoise.c / .h, commands.c, com_ commands.h, Makefile.am (+ Makefile.in)	the qpnoise command (resolve the output node, run QPnoiseAnalyze)
ngspice-46/src/include/ ngspice/cktdefs.h	QPnoiseAnalyze prototype
examples/qpss_examples/ verify_qpnoise.py	the 6-check QPnoise suite

Scope

Two-tone small-signal noise around the E-136 QPSS operating point, single spot frequency per call, output and input-referred density, folding all sidebands. Uses the stationary device PSD at the operating-point bias (matching pnoise's default). Follow-ups: cyclostationary device noise (average the PSD over the two-tone period, as pnoise does with cyclo), a frequency sweep, quasi-periodic transfer function (QPXF), and more than two tones.

Enhancement-139 — Cyclostationary QPnoise

E-138 QPnoise folds each device's noise through the 2-D conversion-matrix adjoint, taking the device power spectral density S once at the operating-point bias. But under a two-tone pump a device's bias swings over the period, so its noise $S(t)$ is cyclostationary — a diode's shot noise $2qI_D(t)$ spikes when the junction conducts, a switching mixer's noise is gated by the LO. This enhancement adds that mode:

```
qpnoise <output_node> <f_in> cyclo
```

The stationary form (no `cyclo`) is unchanged.

Method

Mirroring the single-tone cyclostationary pnoise (E-126), instead of the frequency-domain fold $\sum_{\{k1,k2\}} S \cdot |\Psi_{\{k1,k2\}}|^2$ (a single-bias S) the `cyclo` path uses the identity

$$\text{onoise}(f_{\text{in}}) = (1/P) \cdot \sum_s S(t_s) \cdot |A_s|^2, \quad A_s(j) = \sum_{\{k1,k2\}} \Psi_{\{k1,k2\}}(j) \cdot e^{j 2\pi}$$

where Ψ is the same adjoint transfer as E-138 and A_s is its inverse 2-D DFT — the time-domain transimpedance at phase-grid sample $s = (s1, s2)$. The device is re-biased at each sample's quasi-periodic operating point (reconstructed from the retained V by the same `qp_synth` the QPSS Newton used), its noise $S(t_s)$ evaluated there, folded through A_s , and averaged over the $P1 \times P2$ grid. By Parseval this reduces exactly to the stationary sum — and hence to `.noise` — whenever $S(t)$ is constant.

The junction-limiting subtlety (same as E-134). A diode/BJT/MOS limits its internal junction voltage against a stored value, so a single `MODEINITSMSIG` load at the prescribed sample voltage leaves it pinned at a stale bias — and its shot noise would not track the sample, making the "cyclostationary" result collapse back onto the stationary one. Each sample therefore settles the device first (repeated `MODEINITFLOAT` loads walk the junction to the fixed node voltages) before the PSD is read.

Verification

`verify_qpnoise.py` grows to 10/10 — the six E-138 checks plus four for the `cyclo` mode:

- `cyclo` reduce-to-noise — with the pump ~ 0 the PSD is constant, so `cyclo` equals the plain `.noise` at `f_in` (Parseval);
- Parseval under pump — a thermal resistor's noise is *bias-independent*, so even under a full two-tone pump `cyclo` still equals the stationary result exactly;
- cyclostationary diode — a hard-pumped diode (its junction switching on and off) has strongly bias-dependent shot noise, so `cyclo` differs from the single-bias stationary estimate by $>2\times$ (here $\sim 8\times$) — the cyclostationary noise enhancement of a switching mixer, captured only when the per-sample junction settling is in place;
- solver parity — the `cyclo` result is bit-identical under KLU and Sparse.

The other quasi-periodic suites are unaffected: E-133 11/11, E-136 7/7, E-137 7/7.

Files changed

File	Change
ngspice-46/ src/spicelib/ analysis/ dcpss.c	QPnoiseAnalyze gains a <code>cyclo</code> argument and a cyclostationary branch: IDFT the adjoint transfers to the time domain, re-bias (with junction settling) and evaluate each device's PSD at every $P1 \times P2$ phase sample, average $S(t_s)$.

File	Change
ngspice-46/ src/frontend/ com_qpnoise.c, commands.c	parse the optional cyclo keyword; help string
ngspice-46/ src/include/ ngspice/ cktdefs.h	QPnoiseAnalyze prototype gains cyclo
examples/qps_ examples/ verify_ qpnoise.py	four cyclostationary checks (10/10)

Scope

Cyclostationary two-tone noise: the device PSD is averaged over the two-tone period on the P1×P2 phase grid, with per-sample junction settling so real semiconductors are cyclostationary. Follow-ups (unchanged from E-138): a frequency sweep, quasi-periodic transfer function (QPXF), and more than two tones.

Enhancement-140 — Oscillator phase noise (autonomous HB + **phasenoise**)

The periodic-noise machinery (E-124/126) folds device noise through the conversion-matrix adjoint for driven circuits — noise figure, spot noise. The one thing the "Pnoise" gap still promised and lacked was a dedicated oscillator phase-noise spectrum $L(\Delta f)$. This enhancement adds it, with the autonomous-oscillator engine it needs:

```
hbosc <oscnode> <K> [fguess] [tstab]    -- autonomous HB: oscillator steady state + f0
phasenoise <fstart> <fstop> [points]    -- phase-noise spectrum L(df)
```

Autonomous harmonic balance (**hbosc**)

An oscillator has no driving source, so the HB residual is $F(V) = I_R(V) + [dq/dt](\text{https://github.com/javaNoviceProgrammer/Ngspice_OpenVAF_Enhancements/blob/main/enhancements_doc/V}) = 0$, to be solved for the harmonics V and the unknown oscillation frequency ω_0 . Two facts shape the solve:

- the trivial $V = 0$ also satisfies $F = 0$, so a transient seed is used to land on the limit cycle — **hbosc** runs a short transient from the deck's .ic, reads the amplitude and the frequency (from the zero-crossing spacing), and seeds the fundamental;
- the oscillator's phase is free (shifting it is a symmetry), so the Jacobian dF/dV — the conversion matrix H — is singular, its right null space the phase mode $u_k = jk V_k$.

Newton is therefore run on the bordered system, nonsingular by construction:

$$\begin{bmatrix} H & \partial F / \partial \omega_0 \\ u^*{}^T & 0 \end{bmatrix} \begin{bmatrix} \Delta V \\ \Delta \omega_0 \end{bmatrix} = \begin{bmatrix} -F \\ 0 \end{bmatrix}$$

with $\partial F / \partial \omega_0 = I_C / \omega_0$ (the reactive current) and the gauge row u^* removing the phase freedom. It converges quadratically — on the test LC oscillator, 4 iterations to $|F| = 3e-12$, returning f_0 (the true nonlinear oscillation frequency, slightly pulled from the LC resonance) and the harmonic spectrum. Inductors are handled naturally (ngspice's branch-current MNA makes $V_L = j\omega L \cdot I_L$ linear in ω , so they fit the $G+j\omega C$ conversion matrix). The operating point is retained for **phasenoise**.

Phase noise (**phasenoise**)

At each offset Δf , **phasenoise** builds the adjoint of the conversion matrix at input frequency Δf with the unit at the carrier sideband ($m = 1$) of the oscillator node, and folds every device's noise through it (the same DEVnoise machinery as pnoise). As $\Delta f \rightarrow 0$ the matrix approaches the singular limit-cycle H , so the adjoint transimpedance blows up through the phase mode as $1/\Delta f$, and the folded output noise as $1/\Delta f^2$ — the phase-noise skirt. Normalizing to the carrier power $P = 2|V_1|^2$ gives

$$L(\Delta f) = 10 \cdot \log_{10}(S_v(\Delta f) / P) \quad [\text{dBc/Hz}]$$

which is -20 dB/dec near the carrier and flattens into the device noise floor far out — the classic Leeson shape — at a physical absolute level (no PPV normalization constant to mis-scale: it is real folded device noise over carrier power).

Verification

`verify_phasenoise.py` (8/8) on an LC oscillator ($f_0 \approx 5.03$ MHz, cubic negative resistance $i = -g_0 V + g_3 V^3$, describing-function amplitude $A = \sqrt{(4g_0/3g_3)}$):

- autonomous HB converges to $f_0 \approx 5.03$ MHz;
- the fundamental amplitude matches the describing function;
- $L(\Delta f)$ has the -20 dB/dec skirt near the carrier ($L(1k) - L(10k) \approx 20$ dB);
- it flattens into the noise floor far from the carrier;

- the absolute level is physical (≈ -147 dBc/Hz at 1 kHz for this high-Q tank, not the -300 dBc/Hz a mis-scaled PPV gives);
- thermal scaling — doubling the absolute temperature raises L by exactly 3 dB ($10 \cdot \log_{10}(2)$), pinning the absolute noise coupling;
- a clean error with no hbosc operating point, and KLU == Sparse.

The existing periodic-noise suites are unaffected: pnoise stays 9/9, cyclostationary pnoise unchanged, and QPnoise 10/10.

Files changed

File	Change
ngspice-46/src/ spicelib/analysis/ dcpss.c	HBOSCanalyze (autonomous HB: transient-seeded bordered Newton for V and ω_0 , retains the oscillator operating point) and PhaseNoiseAnalyze (carrier-sideband adjoint at each offset, device-noise fold, carrier-power normalization $\rightarrow L(\Delta f)$)
ngspice-46/src/ frontend/com_hbosc.c / .h, commands.c, com_ commands.h, Makefile.am (+ Makefile.in)	the hbosc and phasenoise commands (transient seed + node resolution)
ngspice-46/src/include/ ngspice/cktdefs.h	HBOSCanalyze / PhaseNoiseAnalyze prototypes
examples/phasenoise_ examples/verify_ phasenoise.py	the 8-check phase-noise suite

Scope

Autonomous single-tone HB for oscillators (harmonics + frequency, transient-seeded) and the phase-noise spectrum $L(\Delta f)$ via the carrier-sideband adjoint. This closes the "Periodic / phase noise" gap. Follow-ups: flicker ($1/f^3$) close-in corner from cyclostationary device noise, a jitter (time-domain) figure, and multi-node oscillator seeding beyond the transient-fundamental start.

Enhancement-141 — Two-tone small-signal QPXF (quasi-periodic transfer function)

With QPSS (E-136), QPAC (E-137) and QPnoise (E-138/139) in place, the two-tone small-signal set was one piece short of the single-tone PSS → PAC → pnoise → PXF suite. This enhancement adds that piece — QPXF, the quasi-periodic transfer function, the adjoint of QPAC:

`qpxf <output_node> <f_in>`

Run after a `qpss <expr> <f1> <f2> hb`, it reports the transfer from an input at every sideband $f_{in} + k_1 \cdot f_1 + k_2 \cdot f_2$ to the chosen output at f_{in} — the small-signal frequency conversion of the pumped operating point, seen from the output's perspective.

Method

QPXF is exactly PXF (E-125) on the two-tone harmonic set. It reuses the adjoint of the 2-D conversion matrix already built for QPnoise (`qp_solve_adjoint`):

$$H^T \cdot \Psi = e_{\{out, (0,0)\}}$$

$\Psi_{\{(k_1, k_2), j\}}$ is then the transimpedance from an injection at (node j , sideband (k_1, k_2)) to the output at sideband $(0, 0)$. Dotting each sideband block of Ψ with the netlist AC-source pattern $B0$ (the same stimulus QPAC uses, captured at the operating point) gives the transfer from that input sideband to the output:

$$H_{\{(k_1, k_2)\}} = \sum_j \Psi_{\{(k_1, k_2), j\}} \cdot B0_j$$

The whole set of sideband transfers comes from one adjoint solve — the point of an adjoint/transfer-function analysis. By the reciprocity identity $(H^{-1}B)_{out} = (H^T e_{out})^T B$, the sideband- $(0, 0)$ transfer is bit-identical to the QPAC response at the output node — a cross-check that pins the adjoint solve. Because QPXF only reads the retained conversion data, it is solver-independent (KLU and Sparse bit-identical) by construction.

Verification

`verify_qpxf.py` (6/6), numpy-free:

- reciprocity — the QPXF $(0, 0)$ transfer equals the QPAC $(0, 0)$ response bit-identically (the defining $PXF \leftrightarrow PAC$ cross-check, now two-tone);
- sideband reciprocity — the conversion-sideband magnitudes match QPAC too;
- reduce-to-XF — with the pump ~ 0 the operating point is time-invariant, so the $(0, 0)$ transfer is the plain linear transfer (a 1 A stimulus into $R = 1 \text{ k}\Omega$ gives 1000) and every conversion sideband vanishes;
- clean failure — `qpxf` with no `qpss ... hb` operating point reports "no QPSS operating point";
- solver parity — KLU and Sparse transfers are bit-identical.

The rest of the suite is unaffected: QPSS 11/11, QPSS-HB 7/7, QPAC 7/7, QPnoise 10/10.

Files changed

File	Change
ngspice-46/src/spicelib/analysis/dcpss.c	QPXFanalyze — one adjoint solve (<code>qp_solve_adjoint</code>), each sideband block of Ψ dotted with the AC-source pattern $B0$, printed as the per-sideband transfer
ngspice-46/src/frontend/com_qpxf.c / .h, commands.c, com_commands.h, Makefile.am (+ Makefile.in)	the <code>qpxf</code> command

File	Change
ngspice-46/src/include/ ngspice/cktdefs.h	QPXFanalyze prototype
examples/qpss_examples/ verify_qpxf.py	the 6-check QPXF suite

Scope

Two-tone small-signal transfer function around the E-136 QPSS operating point, single input frequency per call, AC-source or unit-input reference. With QPAC and QPnoise this completes the quasi-periodic small-signal suite (QPSS $\rightarrow$ QPAC $\rightarrow$ QPnoise $\rightarrow$ QPXF), mirroring the single-tone PSS/PAC/Pnoise/PXF. Follow-ups: an input-frequency sweep and more than two tones.

Enhancement-142 — Input-frequency sweep for the two-tone small-signal analyses

QPAC (E-137), QPnoise (E-138/139) and QPXF (E-141) each computed the response at a single input frequency and printed a table. This enhancement makes them swept analyses over `f_in`, emitting an ngspice plot — conversion gain, noise figure and image-rejection curves versus frequency — exactly the way `.ac / .pnoise / .pxf sweep`.

```
qpac <dec|oct|lin> <N> <fstart> <fstop>
qpnoise <output_node> <dec|oct|lin> <N> <fstart> <fstop>
qpxf <output_node> <dec|oct|lin> <N> <fstart> <fstop>
```

The single-frequency forms are unchanged; a `dec/oct/lin` keyword in the sweep position selects the swept form. The result is a normal ngspice plot (made current), read back with `plot / print / setplot`.

Method

Each swept point reuses the exact same single-frequency solve — so a swept value at a given `f_in` equals the single-frequency result there. Around the retained QPSS operating point the sweep steps `f_in` (geometric for `dec/oct`, linear for `lin`) and, per point:

- **qpac** — solves $H(f_{in}) \cdot X = B0$ and records the in-band $(0,0)$ response magnitude for every node (one plot vector per node, like a pumped `.ac`);
- **qpnoise** — folds the device noise through the adjoint and records `onoise_spectrum` and `inoise_spectrum` (the two-tone noise-figure curve);
- **qpxf** — one adjoint solve per point, recording `xf` (the in-band $(0,0)$ transfer) and `xf_conv` (the total conversion, $\sqrt{\sum |H_{\{sb \neq 0\}}|^2}$).

The analysis fills plain arrays; the front-end builds the ngspice plot with a `f` frequency scale via the nutmeg vector API (`plot_alloc / dvec_alloc / vec_new`) — the correct layer for a front-end command, rather than the analysis-job output framework (which needs a live analysis job a front-end command does not have).

Verification

`verify_qpss_sweep.py` (5/5), numpy-free:

- **qpac** — the swept $(0,0)$ response at 0.3 GHz equals the single-frequency **qpac** $(0,0)$ response to machine precision;
- **qpxf** — the swept `xf` at 0.3 GHz equals the single-frequency **qpxf** $(0,0)$;
- **qpnoise** — the swept `onoise` at 0.3 GHz equals the single-frequency **qpnoise**;
- frequency dependence — a reactive circuit's response rolls off with `f_in` (an RC low-pass), so the sweep is genuinely frequency-dependent, not a repeated constant;
- point count — a `lin N` sweep produces exactly `N` points.

The single-frequency suites are unaffected: QPAC 7/7, QPnoise 10/10, QPXF 6/6 (and QPSS 11/11, QPSS-HB 7/7).

Files changed

File	Change
<code>ngspice-46/src/spicelib/analysis/dcpss.c</code>	QPACsweep / QPnoiseSweep / QPXFswEEP — step <code>f_in</code> , reuse the single-frequency solve, fill caller arrays (<code>freqs[]</code> + point-major <code>data[]</code>)
<code>ngspice-46/src/frontend/com_qpac.c / .h</code>	shared sweep helpers — <code>qp_steptype</code> (<code>dec/oct/lin</code>), <code>qp_sweep_maxpts</code> , and <code>qp_emit_plot</code> (builds the ngspice plot)

File	Change
ngspice-46/src/frontend/ com_qpnoise.c, com_qpxf. c, commands.c	detect the sweep keyword, allocate, call the sweep, emit the plot; help strings
ngspice-46/src/include/ ngspice/cktdefs.h	swept-analysis prototypes
examples/qpss_examples/ verify_qpss_sweep.py	the 5-check sweep suite

Scope

dec/oct/lin input-frequency sweep of the two-tone small-signal analyses, emitting plottable ngspice curves that match the single-frequency solves point-for-point. This makes the quasi-periodic small-signal suite production-grade for conversion-gain / noise- figure / image-rejection plots. Follow-ups: a swept cyclostationary qpnoise, and per-sideband vectors for qpac/qpxf (currently the in-band response plus the total conversion).

Enhancement-143 — gradient least-squares curve fitting for **optimize**

Enhancement-130 gave ngspice a built-in **optimize** command: a derivative-free Nelder-Mead search that varies parameters, re-runs an analysis, and minimizes a single scalar objective. That is the right tool for a one-off tuning goal, but the most common optimization job in circuit work is a fit — drive several measurements to their targets at once (curve fitting, device-parameter extraction), which is a *least-squares* problem with a smooth objective. Nelder-Mead ignores that structure and crawls.

Enhancement-143 adds a least-squares mode to **optimize**: give it one or more weighted **-target** <expr> <value> measurements, optionally spread over several **-analysis** stages, and it fits the parameters with Levenberg-Marquardt (a finite-difference Jacobian) — the standard gradient-based least-squares method, which exploits the sum-of-squares structure to converge in far fewer analysis runs. The original scalar **-minimize** / Nelder-Mead path is unchanged.

Usage

```
optimize -param <name> <init> <lo> <hi> [-param ...]
        -analysis <command ...>
        ( -minimize <expression ...>                (scalar, Nelder-Mead)
          | -target <expr> <value> [<weight>] [-target ...] (least squares, LM)
          [ -analysis <command ...> -target ... ] )
        [-method nm|lm] [-maxiter <N>] [-tol <T>] [-verbose]
```

- **-target** <expr> <value> [<weight>] — a measurement to fit. <expr> is a single ngspice expression token (use the no-space forms `v(out)-v(in)`, `mag(v(out))`, `v(out)[3]`), <value> the desired value, <weight> an optional residual weight (default 1; use `1/value` for a *relative* fit when magnitudes span decades). The residual is `weight·(expr - value)`; the optimizer minimizes the sum of squared residuals. Repeatable, up to 64 targets.
- Multi-analysis stages — each **-analysis** opens a stage; every **-target** after it is evaluated on that stage's results. A single fit can therefore combine different analyses, e.g. a DC operating point and an AC response. Up to 8 stages.
- **-method nm|lm** — force the algorithm. Default: least-squares (any **-target**) uses **lm** (Levenberg-Marquardt); a scalar **-minimize** uses **nm** (Nelder-Mead). **nm** also works on a least-squares objective (it minimizes the summed square); **lm** requires **-targets**.
- **-minimize**, **-param**, **-maxiter**, **-tol**, **-verbose** — as in Enhancement-130. **-minimize** and **-target** are mutually exclusive.

As before, each candidate is applied in place with `alter <name>=<value>` (no re-source), every stage's analysis is run, and each expression's last value is read (target a single point with a one-point analysis or a vector index). The search runs in normalized [0,1] parameter space, so it is scale-invariant across parameters that differ by orders of magnitude. On convergence the circuit is left at the optimum; the sum-squared residual (and its RMS) plus the fitted parameters are printed.

Example — two-parameter, two-analysis fit

```
V1 in 0 dc 1 ac 1
R1 in out 3.3k
R2 out 0 3.3k
C1 out 0 100n
.control
optimize -param R1 3.3k 500 8k -param R2 3.3k 500 8k
+       -analysis op           -target v(out)      0.4
+       -analysis ac lin 1 2000 2000 -target mag(v(out)) 0.221061
.endc
```

recovers $R1 = 3\text{ k}$, $R2 = 2\text{ k}$ — the unique circuit whose DC gain is 0.4 and whose $|H(2\text{ kHz})|$ is 0.221061 — from a $3.3\text{ k} / 3.3\text{ k}$ start.

Implementation notes

- All changes are in `frontend/com_optimize.c` (plus the one-line help string in `commands.c`). No new files, no ABI change.
- The objective is generalized to a list of stages (`struct opt_target` with an owning `-analysis` index). `opt_eval()` sets the parameters once, then for each stage runs its analysis and evaluates that stage's targets while the stage's plot is current — filling a residual vector (least squares) or a single scalar (`-minimize`). Console chatter is still gated at source by `ft_optimizing` (see Enhancement-130).
- Levenberg-Marquardt (`levenberg_marquardt()`): a forward-difference Jacobian J of the residuals w.r.t. the normalized parameters (backward difference near the upper bound), the normal equations $A = J^T J$, $g = J^T r$, and the damped solve $(A + \lambda \cdot \text{diag}(A)) \delta = -g$ with a small dense Gauss-elimination solver (`solve_lin`, partial pivoting). λ is decreased on an accepted step and increased until a step reduces the cost — the standard trust-region-like LM loop. Converges on relative cost improvement or step size below `-tol`.
- Nelder-Mead is retained verbatim; in least-squares mode it minimizes the summed square (`opt_eval(..., NULL)`), which is how the `-method nm` comparison runs.

Verification

`examples/optimize_examples/verify_optimize.py` (23/23; the front-end command is solver-independent, so it runs once) — checks [1]–[5] are the Enhancement-130 scalar/Nelder-Mead cases; [6]–[10] are new:

- [6] LM recovers R of an RC low-pass from $|H|$ at three frequencies, one `-target` per frequency on its own `-analysis` stage.
- [7] two-parameter fit of $R1$, $R2$ to a DC gain (`op` stage) and an AC magnitude (`ac` stage) simultaneously $\rightarrow R1 = 3\text{ k}$, $R2 = 2\text{ k}$.
- [8] LM reaches the same optimum as Nelder-Mead in 27 vs 67 evaluations on a two-target RC fit.
- [9] OSDI / Verilog-A device-parameter extraction: recover *both* the saturation current i_s and emission coefficient n of a compiled diode from two measured I-V points by weighted least squares ($i_s \rightarrow 1\text{e-}14$, $n \rightarrow 1.2$).
- [10] input validation: `-method lm` without `-target`, `-minimize` together with `-target`, and multiple `-analysis` with a scalar `-minimize` are rejected.

`examples/optimize_examples/optimize_lsq_demo.cir` is a runnable least-squares demo; `optimize_demo.cir` remains the Nelder-Mead demo.

Scope and follow-ups

Least-squares fitting of alter-reachable device/instance parameters over one or more analyses. Still future work: optimizing symbolic `.param` values directly (needs a re-source that re-evaluates the netlist without re-running the analysis), and analytic (adjoint) sensitivities in place of the finite-difference Jacobian.

Enhancement-144 — optimizing symbolic **.param** values (**-dparam**)

The built-in optimize command ([Enhancement-130](#), [Enhancement-143](#)) could only turn knobs reachable by alter: device instances (R1) and device/model parameters (@m1[w]). But netlists are usually written in terms of symbolic **.param** values — `.param w=1u, R1 in out {500*k}` — and those are *not* alter targets: a **.param** is expanded at parse time, so changing it means editing the deck and re-parsing, not poking the live circuit. This was the last open follow-up from Enhancement-143.

Enhancement-144 adds a second knob kind, **-dparam**, for exactly this. A **-dparam** name is a symbolic **.param**; the optimizer changes it with `alterparam <name>=<value>` (which rewrites the stored deck) followed by a quiet reset that re-sources the deck — re-evaluating every **.param** expression and re-stamping device values — before running the analysis.

Usage

```
optimize (-param|-dparam) <name> <init> <lo> <hi> [...]
    -analysis <command ...>
    ( -minimize <expr> | -target <expr> <value> [<weight>] ... )
    [-method nm|lm] [-maxiter <N>] [-tol <T>] [-verbose]
```

- **-param name init lo hi** — as before: an alter target (device instance or @inst[param]), changed in place (fast, no re-parse).
- **-dparam name init lo hi** — a symbolic netlist **.param** (e.g. the `w` in `.param w=1u`), changed with `alterparam` + a reset re-source.

Everything else — `-analysis`, `-minimize`, `-target`, `-method`, `-maxiter`, `-tol`, `-verbose`, normalized `[0,1]` search, Nelder-Mead / Levenberg-Marquardt — is unchanged. The two knob kinds mix freely in one run.

Example

```
.param rtop=1k
V1 in 0 dc 1
R1 in out {rtop}
R2 out 0 1k
.control
optimize -dparam rtop 1k 100 10k -analysis op -minimize (v(out)-0.3)^2
.endc
```

tunes the **.param rtop** (used as R1's value) until `v(out) = 0.3`, giving `rtop = 2333.3 Ω`.

Implementation notes

- Changes are in `frontend/com_optimize.c` (parameter *kind* tracking + the re-source step) and `frontend/inp.c` (gate two re-source banners); plus the one-line help string in `commands.c`. No new files, no ABI change.
- Each parameter carries a kind: `OPT_ALTER` (device/instance, in place) or `OPT_DECKPARAM` (**.param**, via `alterparam` + reset). If any **-dparam** is present, `opt_eval` applies all deck params first, re-sources once, then applies the in-place **alter** params — because reset rebuilds the circuit from the deck and would otherwise wipe an earlier in-place alter. This ordering is what makes the two kinds mix correctly (verified: a **.param** and an altered device fitted together land on the unique joint solution).
- Circuits with no **-dparam** skip the re-source entirely — the Enhancement-130 / -143 fast path is untouched, no performance regression.
- The per-iteration reset is silenced by the existing `ft_optimizing` flag: the two banners it prints (Reset re-loads circuit ... in `inp_spsource`, and Circuit: ...) are now gated, so hundreds

of inner re-sources are quiet; only the final "leave the circuit at the optimum" run is shown. `ft_` optimizing is re-asserted after each `reset` in case re-sourcing cleared it.

- A reset is heavier than an `alter` (it tears down and rebuilds the whole circuit and re-reads the deck), so `-dparam` costs more per evaluation than `-param` — inherent to changing a parse-time `.param`. Use `-param` when a knob is `alter`-reachable; use `-dparam` only for genuine `.params`.

Verification

`examples/optimize_examples/verify_optimize.py` (31/31; solver-independent, run once) — checks [1]–[10] are Enhancement-130/-143; [11]–[15] are new:

- [11] scalar `.param` fit: `-dparam rtop` (used as a device value) $\rightarrow$ `rtop = 2333.3` for `v(out) = 0.3`.
- [12] mixed `-dparam rtop` + `-param R2` fitted together $\rightarrow$ `rtop = 3 k`, `R2 = 2 k` (the unique joint solution) — proves the deck `param` is re-sourced first and the in-place `alter` re-applied after.
- [13] `.param` inside an arithmetic device expression (`R1 = {500*k}`) $\rightarrow$ `k = 6`.
- [14] least-squares `-dparam` fit (`-target`, Levenberg-Marquardt) $\rightarrow$ `rtop = 2333.3`.
- [15] the inner re-sources are quiet — the `Reset re-loads` banner appears at most once (the final run), not once per evaluation.

Also verified manually with an AC analysis and with a `.param` scaling an OSDI/Verilog-A device value across the re-source. `examples/optimize_examples/optimize_dparam_demo.cir` is a runnable demo. The Enhancement-130/-143 suite is unchanged (23/23 within the 31).

Scope and follow-ups

Optimizing symbolic `.param` values, mixing freely with `alter`-reachable device parameters, in scalar or least-squares mode. Remaining optimizer follow-up: analytic (adjoint) sensitivities in place of the finite-difference Jacobian.

Enhancement-145 — optimizing **.model**-card parameters (**-mparam**)

The `optimize` command could tune two kinds of knob: alter-reachable device/instance parameters ([Enhancement-130](#)/`-param`) and symbolic netlist `.param` values ([Enhancement-144](#)/`-dparam`). The one class left out was **.model**-card parameters — the `is/n` of a diode model, the `r` of a Verilog-A resistor model, and so on. A model parameter is *not* alter-reachable (that reaches only device instances), and — as the companion investigation showed — it is not `.dc`-sweepable either (`.dc` steps only sources, resistors and instance parameters). `ngspice` changes a model parameter with a different command, `altermod`, and Enhancement-145 wires that into the optimizer as a third knob kind.

Usage

```
optimize (-param|-mparam|-dparam) <name> <init> <lo> <hi> [...]
        -analysis <command ...>
        ( -minimize <expr> | -target <expr> <value> [<weight>] ... )
        [-method nm|lm] [-maxiter <N>] [-tol <T>] [-verbose]
```

- **-param name init lo hi** — an alter target: a device instance (R1) or an instance parameter (@m1[w]), changed in place with `alter`.
- **-mparam name init lo hi** — a **.model**-card parameter, named @<model>[<param>] (e.g. @dmod[is], @rmod[r]), changed in place with `altermod <name>=<value>`. New in this enhancement.
- **-dparam name init lo hi** — a symbolic `.param`, changed with `alterparam` + a reset resource.

Everything else is unchanged. Like `-param`, `-mparam` is an in-place knob: a model parameter takes effect immediately (`altermod` re-stamps the affected devices), with no re-source — so `-mparam` is as cheap per evaluation as `-param`, unlike the heavier `-dparam`. All three kinds mix freely in one run.

Example

```
Vd a 0 dc 0.65
D1 a 0 dmod
.model dmod d(is=1e-15 n=1)
.control
optimize -mparam @dmod[is] 1e-15 1e-16 1e-12 -analysis op -minimize
↳ (abs(i(vd))-1m)^2
.endc
```

fits the diode model's saturation current so it passes 1 mA at 0.65 V (`is` → 1.22e-14).

Implementation notes

- Changes are confined to `frontend/com_optimize.c` (a new kind value and the `altermod` branch) plus the one-line help string in `commands.c`. No new source files, no ABI change, no change to `inp.c` or any other file.
- Each parameter's kind is now one of `OPT_ALTER` (instance, `alter`), `OPT_MODELPARAM` (model, `altermod`) or `OPT_DECKPARAM` (`.param`, `alterparam` + reset). In `opt_eval`, the deck params are applied and re-sourced first (as in [Enhancement-144](#)), then the in-place knobs: `alter <name>=<value>` for `OPT_ALTER`, `altermod <name>=<value>` for `OPT_MODELPARAM`. Because `altermod` is in place, `-mparam` never triggers the re-source path — circuits with only `-param`/`-mparam` knobs keep the [Enhancement-130](#)/[143](#) fast path.
- The @<model>[<param>] name is passed straight through to `altermod`, which accepts that accessor form — so `-mparam @rmod[r]` emits `altermod @rmod[r]=...`, exactly mirroring how `-param @m1[w]` emits `alter @m1[w]=...`

Verification

examples/optimize_examples/verify_optimize.py (39/39; solver-independent, run once) — checks [1]–[15] are Enhancement-130/-143/-144; [16]–[20] are new:

- [16] OSDI model param: `-mparam @rmod[r]` fits a Verilog-A resistor model's r (via `altermod`) so $v(out) = 0.25 \rightarrow r = 3\text{ k}$.
- [17] built-in model param: `-mparam @dmod[is]` fits a diode model's is so $I(0.65\text{ V}) = 1\text{ mA} \rightarrow is = 1.22\text{e-14}$.
- [18] determined mixed fit: a model param (`@rmod[r]`) and an instance param (`R2`) fitted together $\rightarrow r = 3\text{ k}, R2 = 2\text{ k}$.
- [19] `-mparam` is the in-place fast path — 0 re-sources (no Reset re-loads banner), unlike `-dparam`.
- [20] all three knob kinds (`-dparam + -mparam + -param`) coexist in one run and converge.

A new `optresm.va` (a Verilog-A resistor whose r is a *model* parameter) is added; `optimize_mparam_demo.cir` is a runnable demo.

Scope and follow-ups

Optimizing `.model-card` parameters (built-in and OSDI/Verilog-A) via `altermod`, mixing freely with instance (`-param`) and symbolic (`-dparam`) knobs, in scalar or least-squares mode. With this the optimizer covers every kind of circuit knob. Remaining optimizer follow-up: analytic (adjoint) sensitivities in place of the finite-difference Jacobian.

Enhancement-146 — universal **sweep** command and **.sweep** card

ngspice's **.dc** card sweeps a knob and records the result, but the knob can only be a source, a resistor, or a device instance parameter (@m1[w]). It cannot step a model parameter or a symbolic **.param** — as the companion investigation for Enhancement-144/145 showed, those need **altermod** and **alterparam** + **reset** respectively, which **.dc** does not do. So there was no way to say "sweep this model's is" or "sweep this **.param** and plot the response."

Enhancement-146 adds a universal **sweep** command (and a matching **.sweep** netlist card) that sweeps *any* knob. It auto-detects which kind each knob is and applies it with the right mechanism — the same three the built-in optimizer uses (Enhancement-130/144/145):

knob	detected as	applied with
device / instance / source / resistor (R1, V1, @m1[w])	instance/ device	alter (in place)
@<model>[<param>] (@dmod[is])	model param	altermod (in place)
a netlist .param (rtop)	.param	alterparam + reset (re-source)

Usage

```
sweep <knob> (<start> <stop> <step> | lin|dec|oct <N> <start> <stop> | list <v>...)
    [-analysis <cmd>] [-output <name>=<expr> ...]
```

```
.sweep <knob> ... (the same, as a netlist card)
```

- sweep specification — a <start> <stop> <step> range, lin|dec|oct <N> (N points total / per decade / per octave), or an explicit list of values.
- **-analysis <cmd>** — the analysis to run at each point (default op); any ngspice analysis command (ac dec 20 1 1meg, tran 1u 1m, ...).
- **-output <name>=<expr>** — an output to record (repeatable). <expr> is any ngspice expression (its last value is taken). The optional <name>= gives the result vector a clean name, so plot gain works even when the expression is a function like mag(v(out)). With no **-output**, every node voltage of the analysis is recorded — the **.dc**-like default.

Results go into a new plot named **sweep** whose scale is the knob values, so plot <output> shows the output versus the swept knob. The per-point analysis plots are kept too (tran1, tran2, ...) for overlaying waveforms. Console chatter from the inner analyses is suppressed via the ft_optimizing flag.

Examples

```
* sweep a model parameter (impossible with .dc) and plot the AC gain
sweep @dmod[is] dec 10 1e-15 1e-12 -analysis ac dec 20 1 1meg -output
↳ g=mag(v(out))
```

```
* sweep a .param straight from the netlist
.sweep rr 1k 5k 1k -analysis ac lin 1 1k 1k -output gain=mag(v(out))
```

Implementation notes

- New front-end command in frontend/com_sweep.c (+ com_sweep.h), registered in commands.c / com_commands.h and the frontend Makefile.am.
- Knob-kind auto-detection (sw_kind): for @X[y], ft_sim->findModel(ckt, X) decides model (→ altermod) vs instance (→ alter); for a bare name, nupa_get_param(name, &found) decides **.param** (→ alterparam + reset) vs device (→ alter). Both are lightweight, silent lookups.

- The engine sets the knob, runs the `-analysis` command (dispatched synchronously through the command table, like the optimizer's `opt_run_cmd`), and evaluates each output expression, collecting the results; then it emits the summary plot via the nutmeg vector API (`plot_alloc / dvec_alloc / vec_new`) — the same layer a front-end command must use (Enhancement-142).
- **.sweep** card: `frontend/inp.c` recognizes a top-level `.sweep` line and adds it (minus the leading `.`) to the post-parse control list, so it runs as a sweep command after the circuit is built.
- Re-entrancy guard: a `.param` knob re-sources the deck (`reset`), which re-runs a `.sweep` card — a static `sweep_active` flag makes that nested invocation a no-op, so the sweep cannot recurse.

Verification

`examples/sweep_examples/verify_sweep.py` (11/11; front-end command, solver-independent):

- [1] an instance/resistor sweep `R1` over a divider reproduces the analytic $R2/(R1+R2)$ and matches the built-in `.dc R1` exactly — the generalization is faithful.
- [2] a voltage-source sweep gives the linear `v(out)`.
- [3] a model-parameter sweep `@rmod[r]` (Verilog-A model `r`, via `altermod`).
- [4] a symbolic **.param** sweep (via `alterparam + reset`).
- [5] the three kinds are each routed correctly by auto-detection.
- [6] the **.sweep** card equals the command form and does not recurse on the `.param` re-source.
- [7] an AC inner analysis with a named output matches the analytic low-pass $|H(1\text{kHz})|$.
- [8] a transient inner analysis (settled node voltage).
- [9] the `lin N / list / start stop step` specs give the right points.
- [10] multiple `-outputs` are all recorded.

`sweep_demo.cir` and `sweep_card_demo.cir` are runnable demos.

Scope and follow-ups

A universal parametric sweep of any circuit knob, with a chosen inner analysis and arbitrary output expressions, from the command line or the netlist. Follow-ups: a nested (multi-knob) sweep, and a `.step`-style automatic overlay of the retained per-point waveform plots.

Enhancement-147 — nested `?:`: no longer takes exponential time to compile

A deep robustness campaign against `openvaf-r` (adversarial inputs, the full production-model corpus, and thousands of mutation-fuzzing iterations) surfaced one high-severity compiler bug: a chain of nested ternary (`?:`) expressions took $O(2^n)$ time to compile, so a module with ~30 nested conditionals — easily produced by macros in a real device model — hung the compiler indefinitely.

Root cause

The body validator in `hir_ty/src/validation/body.rs` visits every expression in `validate_expr`, whose match ends by recursing into the children:

```
_ => ()
}
self.parent.body.exprs[expr].walk_child_exprs(|child| self.validate_expr(child))
```

Arms that fully validate their own operands return early to skip that generic walk — the `Call` and `Path` arms both do. The **Select** (ternary) arm did not: it validated `cond`, then `then_val` and `else_val` via `validate_condition`, and then *fell through* to `walk_child_exprs`, which validated `then_val` and `else_val` a second time. Each ternary therefore validated its branches twice; for a chain of N nested `?:` this compounds to 2^n validations.

Profiling a hanging compile confirmed it exactly — an unbounded `validate_expr` $\rightarrow$ `validate_condition` $\rightarrow$ `validate_expr` $\rightarrow$... recursion with a doubling call count.

The fix

Add a `return`; at the end of the `Select` arm, matching the `Call` and `Path` arms. The arm already validates all three children (`cond`, `then_val`, `else_val`), so the generic `walk_child_exprs` was pure redundant work — removing it is behaviour-preserving and turns validation from $O(2^n)$ into $O(N)$.

```
depth 40:  hang (>30 s)   →  0.09 s
depth 160: (2160, hopeless) →  0.11 s
depth 2000:                →  1.6 s   (linear)
```

Verification

- **examples/nested_cond_examples/verify_nested_cond.py** (7/7): nested `?:`: chains of depth 20/40/80/160 all compile in < 0.2 s; compile time grows ~linearly ($t(160)/t(20) \approx 1.2$, not exponential); and a nested-ternary model still compiles to a valid `.osdi` and computes the correct piecewise value in `ngspice` ($I(2.5\text{ V}) = 2.5/3\text{ k}\Omega = 833.3\text{ }\mu\text{A}$).
- Behaviour-preserving: every one of the 117 production CMC models (BSIM, PSP, HICUM, MEXTRAM, VBIC, EKV, ...) compiles to the identical verdict before and after the fix (0 flips in a head-to-head run), and BSIM4 (12.6 k lines) still compiles in ~2.3 s.
- Unit tests: `hir_ty`, `hir`, `hir_lower` and `sim_back` test suites pass with no snapshot changes.

The robustness campaign (context)

The bug was found by a systematic robustness sweep, which also confirmed the compiler is otherwise solid:

- 117 real production models — 0 crashes / hangs / panics.
- ~50 adversarial hand-crafted inputs (unterminated constructs, malformed numbers, macro bombs, duplicates, unicode/null bytes, ...) — all rejected cleanly; no accepted-invalid inputs.
- 4000 mutation-fuzzing iterations on real models — 0 panics, 0 segfaults; the compiler never crashed on random/garbage input.

Lower-severity findings that remain (all require pathological input a human or real model would never write, and abort cleanly rather than hang) are documented as follow-ups: a recursive-descent stack overflow on extreme expression nesting ($\approx 4\text{ k}$ – 32 k deep unary / binary / parenthesised / call chains); an uncapped **include** self-recursion (a file that ``includes`` itself) overflowing the stack; and a huge array dimension (`real x[0:100000000]`) exhausting memory. Adding a parse-time nesting-depth limit and an include-depth cap (mirroring the existing macro-recursion cap) would turn these into clean diagnostics.

Scope

A one-line, behaviour-preserving correctness/performance fix to the body validator, eliminating an exponential-time compile on nested conditional expressions. Confined to `hir_ty/src/validation/body.rs`; no change to generated code.

Enhancement-148 — compiler hardening: parser depth, include cap, array cap

The robustness campaign behind [Enhancement-147](#) fixed the one *plausible-in-real-code* failure it found (exponential-time nested `?:`). It also turned up three lower-severity ways to make `openvaf-r` crash or hang on pathological input — input a human or real device model would never write, but which a robust compiler should still reject cleanly rather than die on. This enhancement closes all three.

What was fixed

Pathological input	Before	After
deeply nested / chained expression (<code>--...x, x+1+1+..., ((...)), sin(sin(...)), 1?...:0</code>)	recursive-descent stack overflow (SIGABRT), or a later recursive tree traversal overflowing	clean parse error
a file that <code>`includes</code> itself	uncapped include recursion → stack overflow	clean “ <i>nests too deeply</i> ” error
an absurd array / bus / instance range (<code>real x[0:100000000]</code>)	expanded element-by-element → memory exhaustion / hang	clean “ <i>too large</i> ” error

1. Parser expression-depth limit parser’s Pratt expression parser recurses through `atom_expr` for every level of nesting (prefix operators, parentheses, call arguments, ternary branches) and builds a left-leaning tree for operator chains. Both an over-deep recursion *and* an over-deep tree (which later passes recurse over) could overflow the stack. A shared `expr_depth` counter — incremented on each `atom_expr` (recursion) and per operator in the `expr_bp` loop (chain length) — bounds the total expression-tree depth at 1000 (real models nest expressions a few dozen deep) and recovers to the next statement boundary when exceeded.

2. ``include` recursion cap    The preprocessor’s `include_file` recursed into `process_file`, which processes the included file’s own ``includes` — with no bound, so a self-including file overflowed the stack. An `include_depth` counter caps nesting at 64 and emits a new `IncludeRecursionLimit` diagnostic, mirroring the existing macro-recursion guard (Enhancement-65).

3. Array / bus / instance element cap An array-shaped declaration is flattened into one scalar element per index, so an unbounded range materialized (and, for instance arrays, *rendered*) millions of entries. A shared `array_elem_count` helper caps the expansion at $2^{20} \approx 1.05$ M elements and emits a new `ArrayTooLarge` diagnostic (degrading to a single scalar so compilation proceeds). It is applied at every expansion site: variable arrays, parameter arrays, net/port buses, array function returns (`hir_def` item-tree lowering), and instance arrays (both the item-tree lowering and the `hir` elaboration pass, which re-expands them into rendered text).

Files

```
parser/src/{parser.rs, grammar/expressions.rs},    preprocessor/src/{diagnostics.rs,
processor.rs}, basedb/src/diagnostics/preprocessor_error.rs, hir_def/src/item_tree.
rs, hir_def/src/item_tree/{diagnostics.rs, lower.rs}, hir/src/elaborate.rs.
```

Verification

`examples/robustness_examples/verify_robustness.py` (17/17): each pathological input (5 deep-expression shapes, self-include, and 4 huge-array kinds — variable, parameter, net bus, instance) now exits with a clean error in well under a second — no crash, no hang — with the expected diagnostic text;

and valid deep-but- reasonable input (nested ternary depth 30, parentheses depth 100, a 100-term sum, and small variable/instance arrays) still compiles.

Behaviour-preserving: all 117 production CMC models compile to the identical verdict before and after (0 flips in a head-to-head run) — no real model comes near any limit — and the parser, preprocessor, `hir_def`, `hir_ty` and `sim_back` unit-test suites pass with no snapshot changes.

Scope

Turns three pathological-input crash/hang paths into clean, bounded diagnostics. The limits (expression depth 1000, include depth 64, array elements ~1 M) are far beyond anything real Verilog-A produces. With this the robustness campaign's findings are fully resolved.

Enhancement-149 — Latin-Hypercube Monte Carlo sampling (**mcsample**)

ngspice's Monte Carlo works by drawing each random parameter independently from the PRNG — `.param rr = agauss(1k, 100, 3)` re-thrown on every reset. That is plain random sampling: with a modest run count the draws clump in some regions of the distribution and leave gaps in others, and the estimated mean / spread / yield converges only as $1/\sqrt{N}$. Commercial simulators offer low-discrepancy sampling (Latin-Hypercube, Sobol) to fix this; ngspice had none — it was a $\times$ row in the [gap analysis](#).

Enhancement-149 adds Latin-Hypercube sampling (LHS) through a new `mcsample` command. LHS partitions every random dimension's probability range into N equal strata and draws exactly one sample per stratum (with an independent stratum permutation per dimension), so N runs cover each distribution evenly instead of clumping. For smooth responses this cuts the variance of the estimate by one to two orders of magnitude at the same N — measured $\sim 130\times$ lower here.

Usage

```
mcsample lhs <N> [seed <s>]    engage LHS for the next N reset-driven runs
mcsample random | off          revert to independent PRNG draws
mcsample                      report the current sampling mode
```

Once `mcsample lhs N` is set, the existing netlist Monte Carlo idiom is unchanged — the stochastic `.param` functions **agauss** / **gauss** / **aunif** / **unif** / **limit** simply draw stratified values instead of independent ones, one sample per reset:

```
.param rr = agauss(1000, 100, 3)      ; ~ N(1000, 33.33)
V1 a 0 DC 1
R1 a 0 {rr}
.control
  mcsample lhs 64 seed 1
  let run = 0
  dowhile run < 64
    reset          ; steps to the next stratified sample
    op
    let iv[run] = i(V1)
    let run = run + 1
  end
  print mean(iv) stddev(iv)
.endc
```

Each stochastic function call within one reset pass is one LHS dimension, so a deck with several random parameters stratifies each of them independently. Gaussian draws (`agauss/gauss`) are stratified in probability space and mapped through the inverse-normal CDF, so the tails are covered evenly too.

Implementation notes

- `Sampler(maths/misc/randnumb.c, declared in randnumb.h)`: holds the mode, N , current sample index, per-sample dimension counter and seed. Each dimension's stratum permutation (Fisher-Yates over $0..N-1$) and per-sample jitter are generated lazily on first use from a self-contained `splitmix64` seeded by `(seed, dimension)`, so the whole sequence is reproducible and independent of evaluation order. `mc_sample_uniform()` returns `(perm[d][i] + jitter[d][i]) / N`; `mc_sample_gauss()` maps that uniform through **inv_normal_cdf** (Acklam's probit, rel. error $< 1.15e-9$).
- Sample boundary: one full deck re-evaluation pass == one Monte Carlo sample. `numparam/spicenum.c's nupa_signal(NUPADECKCOPY)` edge (once per reset re-source, guarded by the existing `firstsignals` latch) calls `mc_sample_advance()`, which steps the sample index and rewinds the dimension counter before that pass's `.param` draws run.

- Draw hook (numparam/xpressn.c): `agauss/gauss` take one `mc_sample_gauss()` and `aunif/unif/limit` take one `2*mc_sample_uniform()-1` when LHS is active, falling back to the plain `gauss1()/drand()` otherwise — so behaviour with LHS off is byte-identical to before.
- Command `com_mcsample` (randnumb.c, registered in `commands.c`): parses `lhs <N> [seed <s>] / random / off`.
- LHS is a front-end feature — it changes only which uniform/Gaussian values the stochastic `.param` functions draw, not the circuit solve — so results are identical under the Sparse 1.3 and KLU solvers.

Verification

`examples/lhs_examples/verify_lhs.py` (5 groups, all passing under both solvers):

- [1] stratification — $N=48$ LHS draws of a Gaussian `.param` occupy each of the 48 probability strata exactly once (the defining Latin-hypercube property); plain random sampling leaves gaps ($\approx 32/48$ distinct strata).
- [2] multi-dimension — two independent params (`agauss + aunif`) are both fully stratified in the same run.
- [3] variance reduction — over 40 independent trials, $\text{Var}(\text{sample-mean})$ under LHS is $\sim 130\times$ smaller than under plain random MC at the same $N=32$, both converging to the same mean.
- [4] reproducibility — the same seed reproduces the sample set bit-for-bit; a different seed gives a different set.
- [5] correctness — LHS sample mean/stddev match the analytic distribution (`agauss(nom, avar, sig)` has $\sigma = \text{avar}/\text{sig}$; `aunif(nom, avar)` is uniform on $[\text{nom}-\text{avar}, \text{nom}+\text{avar}]$).

`lhs_demo.cir` is a runnable resistor-divider demo ($N=64$ random vs LHS).

Scope and follow-ups

Latin-Hypercube low-discrepancy sampling for the reset-driven `.param` Monte Carlo idiom, engaged with one command and reproducible by seed. Follow-ups: a Sobol sequence option (needs direction-number tables but does not require N up front), extending stratification to the nutmeg-loop `sgauss(0)/sunif(0)` idiom, and — the natural next statistical brick — high-sigma methods (importance sampling / worst-case distance) that build on top of a good sampler.

Enhancement-150 — high-sigma rare-event estimation (**highsigma**)

Enhancement-149 made ordinary Monte Carlo *efficient* (Latin-Hypercube stratification of the whole distribution). This enhancement reaches the part of the distribution ordinary Monte Carlo cannot reach at all: the rare tail. For high-replication circuits — an SRAM bit cell instantiated millions of times, a standard-cell library used everywhere — the quantity that matters is a per-cell failure probability of $1e-7$ to $1e-9$, i.e. a 4–6 sigma event. Plain Monte Carlo would need $1e7$ – $1e9$ runs just to *see* a handful of failures, which is infeasible. This was the last $\times$ statistical row versus a commercial simulator in the [gap analysis](#).

Enhancement-150 adds a **highsigma** command that estimates such probabilities with a few thousand runs, using scaled-sigma importance sampling.

The method

Every Gaussian `.param`'s standard deviation is inflated by a factor `lambda` (the "scaled sigma"), so the rare failure region is sampled *often*; each sample is then reweighted by the likelihood ratio $p_{\text{nominal}}(x) / p_{\text{inflated}}(x)$ so the estimator $\hat{P} = (1/N) \sum w_i \cdot 1[\text{fail}_i]$ is unbiased for the true (nominal) probability. Scaled-sigma sampling is direction-free — unlike mean-shift importance sampling or worst-case-distance search it needs no gradient, sensitivity, or most-probable-failure-point, so it is robust for an arbitrary failure condition. The per-dimension log-weight $\log \lambda - (z^2/2)(1 - 1/\lambda^2)$ is accumulated over a sample's Gaussian draws.

Usage

```
highsigma <N> [-scale <lambda>] [-seed <s>] [-analysis <cmd>]
               -metric <expr> [-max <hi>] [-min <lo>]
```

- **N** importance samples; each re-sources the deck (redrawing the inflated `.params`) and runs `-analysis` (default `op`).
- **-scale <lambda>** sigma inflation (default 2; ~2.5–3 for 4–6 sigma).
- **-metric <expr>** the circuit quantity — one ngspice expression token (`v(out)`, `-1/i(v1)`, `mag(v(out))`).
- **-max <hi> / -min <lo>** the spec: a sample fails if the metric is above `hi` or below `lo` (give both for a two-sided spec; at least one is required). The comparison is done by the command, not inside the expression, because a bare `>/<` in a control-language command is an I/O redirect.

It reports the failure probability, its relative error, the equivalent one-sided sigma-to-fail ($-\Phi^{-1}(\hat{P})$), and the raw failure count, and leaves them in the vectors/variables `highsigma_pfail`, `highsigma_relerr`, `highsigma_sigma`, `highsigma_nfail` for scripting.

```
* fail if R > mean + 4.5 sigma; analytic P = Phi(-4.5) = 3.40e-6
```

```
highsigma 6000 -scale 3.0 -seed 1 -analysis op -metric -1/i(v1) -max 1150
```

```
-> P(fail) = 3.23e-06 (equivalent sigma 4.510) [388 failures in the inflated 6000]
```

Implementation notes

- Sampler (`maths/misc/randnumb.c`): a new `MC_MODE_SSS` reuses the E-149 sampler scaffolding. `mc_sample_gauss()` draws $z = \lambda \cdot \text{gauss1}()$ from the inflated normal and accumulates that draw's log likelihood-ratio into `sss_logw`; `mc_sample_weight() = exp(sss_logw)`. Uniform `.params` are bounded, so SSS does not inflate them (weight 1). `mc_sss_config(N,  $\lambda$ , seed)` engages it (and seeds the global PRNG for reproducibility); `mc_sss_off()` reverts.
- Command `com_highsigma` lives in `frontend/com_sweep.c` — it reuses that file's synchronous command runner (`sw_run_cmd`, for `reset` + the inner analysis) and expression evaluator (`sw_eval_expr`), and is likewise a sampling-driven analysis loop. Registered in `commands.c`.

- The loop is the ordinary reset-driven MC idiom driven from C: for each sample it runs `reset` (redraws the inflated params on the NUPADECKCOPY edge from E-149), runs `-analysis`, evaluates `-metric`, applies the spec, and reads `mc_sample_weight()`. `ft_optimizing` silences per-sample chatter.
- Front-end only, so solver-independent.

Verification

`examples/highsigma_examples/verify_highsigma.py` (Sparse-only — a heavy, thousands-of-re-sources deck; solver-independent). Ground truth: one Gaussian `.param` with failure $R > \mu + \beta \cdot \sigma$ has probability $\Phi(-\beta)$ exactly.

- [1] accuracy at $\beta = 2$ and 4 — P and equivalent sigma match $\Phi(-\beta)$ (e.g. $\beta = 4$: sigma-to-fail 4.000, P $3.16e-5$ vs $3.17e-5$).
- [2] deep tail $\beta = 5$ ($\Phi(-5) = 2.87e-7$) — estimated from 6000 runs (sigma 5.01), where plain MC of the same N expects ~ 0.0017 failures.
- [3] a two-sided spec (`-max` and `-min`) roughly doubles the tail probability and lowers the equivalent sigma accordingly.
- [4] reproducibility — same seed gives the identical estimate bit-for-bit.
- [5] multi-parameter — two independent Gaussians combine as $N(\cdot, \sqrt{s_1^2 + s_2^2})$; the recovered sigma-to-fail matches.

`highsigma_demo.cir` is a runnable 4.5-sigma demo.

Scope and follow-ups

Direction-free rare-event probability estimation, reaching 4–6 sigma with a few thousand runs and reporting an unbiased $P(\text{fail})$ with a confidence (relative error) and an equivalent sigma-to-fail. Follow-ups: worst-case-distance / mean-shift importance sampling (more efficient once the failure direction is known — the most-probable-failure-point can seed the shift), adaptive lambda selection, and combining SSS with the E-149 stratification (stratified importance sampling).

Enhancement-151 — process/mismatch correlations and a packaged yield flow

Enhancement-149 and **Enhancement-150** made Monte Carlo efficient and gave it a rare-event tail. Two pieces of the statistical story were still only *partial* ($\triangle$) versus a commercial simulator:

- Native process/mismatch correlations. ngspice draws every `agauss/gauss` independently — “every textual occurrence of a random {param} draws independently” (the Enhancement-66 gotcha). So *matched* devices could not be modelled, yet whether two devices’ variation is correlated is often exactly what decides the yield.
- A packaged corner + MC + yield flow. Yield had to be assembled by hand from a reset loop and manual pass/fail bookkeeping.

Enhancement-151 closes both — the last two $\triangle$ rows in the statistical section of the [gap analysis](#).

Correlations — **mccorr** + **mvnorm**

```
mccorr <k> <m11> <m12> ... <mkk>    register a k x k correlation matrix
mccorr off                          clear it
```

`mccorr` takes a $k \times k$ correlation matrix (row-major, symmetric, unit diagonal) and Cholesky-factors it once into a lower-triangular L . A new `.param` function **mvnorm**(*i*) returns the *i*-th component of one correlated standard-normal draw per Monte Carlo sample: $y = L \cdot z$ with $z \sim N(0, I)$. So correlated variation is expressed directly in the `.params`:

```
mccorr 2 1 0.9 0.9 1 ; rho = 0.9 between the two factors
.param r1 = 1000 + 50*mvnorm(1) ; r1, r2 vary together
.param r2 = 1000 + 50*mvnorm(2)
```

The underlying z ’s are drawn through the same mode-aware `mc_sample_gauss()` the other sampling modes use, so correlations compose with Latin-Hypercube (E-149) and scaled-sigma importance sampling (E-150) for free; with no matrix registered, `mvnorm()` degrades to independent draws. The common process + mismatch model is the special case `sigma_proc.mvnorm(shared) + sigma_mm.agauss(..)`.

Yield — **montecarlo**

```
montecarlo <N> [-lhs] [-seed <s>] [-analysis <cmd>]
              (-spec <metric> [-max <hi>] [-min <lo>])...
```

Runs N Monte Carlo samples (each re-resources the deck and runs `-analysis`, default `op`), evaluates every `-spec` metric, and counts a sample as pass only if all specs are within their limits. It reports the yield with a Wilson 95% confidence interval and a per-spec violation count, and leaves `montecarlo_yield`, `montecarlo_npass`, `montecarlo_n` for scripting. `-lhs` uses Latin-Hypercube sampling for a much lower-variance yield estimate. Process corners are the ordinary `.lib/.include` corner selection — load a corner model set and run `montecarlo` at that corner; correlations and corners compose.

Implementation notes

- Sampler (`maths/misc/randnumb.c`): `mc_corr_config(k, matrix)` Cholesky-factors the correlation matrix (rejecting a non-positive-definite one); `mc_corr_component(i)` lazily draws the k underlying z ’s once per sample (through `mc_sample_gauss()`, so LHS/SSS apply and SSS weights accumulate), applies L , caches y , and returns $y[i]$. The cache is invalidated every pass in `mc_sample_advance()` — which now runs its reset in every mode, so correlations work under plain MC too. `com_mccorr` parses the matrix; a reusable `mc_lhs_config()` was factored out of `com_mcsample` for `montecarlo -lhs`.
- **mvnorm** is added to the `numparam` expression evaluator (`numparam/xpressn.c`) — one entry in the function list / enum / dispatch, calling `mc_corr_component`.

- **com_montecarlo** lives in `frontend/com_sweep.c` (reuses its synchronous command runner and expression evaluator). Registered in `commands.c` with `mccorr`.
- Front-end only, so solver-independent.

Verification

`examples/yield_examples/verify_yield.py` (Sparse-only — heavy deck):

- [1] an `mccorr` $\rho=+0.7 / -0.6$ matrix reproduces the empirical correlation (0.711 / -0.614) with the right means and sigmas.
- [2] `mvnorm` without a matrix draws independently ($\text{corr} \approx 0$).
- [3] a non-positive-definite matrix is rejected.
- [4] single two-sided spec yield matches $P(|Z| < k)$ (0.8660 vs 0.8664).
- [5] two independent specs' yields multiply (0.7508 vs 0.7506).
- [6] `-lhs` is unbiased and much lower-variance (measured $\sim 800\times$ lower yield- estimate variance at the same N).
- [7] positive parameter correlation raises the joint yield ($0.75 \rightarrow 0.82$).

`yield_demo.cir` — a matched divider that yields $\sim 100\%$ when process-correlated ($\rho=0.9$) but only $\sim 74\%$ when independent: the correlation model is what decides it.

Scope and follow-ups

Native correlated process/mismatch sampling (arbitrary correlation matrix, via `mvnorm`) and a packaged, spec-based Monte Carlo yield command with a confidence interval and optional Latin-Hypercube variance reduction. Follow-ups: multiple independent correlation groups, a covariance (non-unit-diagonal) form, and a one-command corner-sweep-of-yield wrapper.

Enhancement-152 — KLU matrix reordering and scaling controls

This build's KLU direct linear solver (opt-in via `.option klu`, [Enhancement-112](#) onward) ran on its compiled-in defaults — AMD fill-reducing ordering, max row scaling, BTF (block-triangular-form) permutation on — with no way to change them. The only KLU knob exposed, `klu_memgrow_factor`, was moreover broken (it collapsed the real value to a boolean). So "matrix reordering + scaling beyond KLU defaults" was a $\triangle$ row in the [gap analysis](#).

Enhancement-152 exposes KLU's reordering and scaling as `.options` and fixes the `memgrow` knob.

Options

<code>.option klu klu_ordering=amd colamd</code>	fill-reducing ordering (default amd)
<code>.option klu klu_scale=none sum max</code>	matrix row scaling (default max)
<code>.option klu klu_btf=on off</code>	block-triangular form (default on)
<code>.option klu_memgrow_factor=<f></code>	KLU work-array growth (default 1.2)

- **klu_ordering** — the fill-reducing permutation computed before factoring: AMD (approximate minimum degree, default) or COLAMD (column AMD), which can give less fill on some structures.
- **klu_scale** — row equilibration before factoring: max (each row by its largest magnitude, default), sum (by the row's 1-norm), or none; improves the conditioning of badly-scaled matrices.
- **klu_btf** — whether KLU first permutes to block-triangular form (default on).
- **klu_memgrow_factor** — the KLU work-array growth factor; previously a no-op (task-`>TSKkluMemGrowFactor = (val->rValue == 1.2)` set a boolean), now takes the real value.

All of these change only *how* KLU factors the matrix, never the physical solution, and apply only under `.option klu`.

Implementation notes

The knobs follow ngspice's existing option-plumbing chain, exactly mirroring `klu_memgrow_factor`:

1. **optdefs.h** — new `OPT_KLU_ORDERING` / `OPT_KLU_SCALE` / `OPT_KLU_BTF`.
2. **cktsopt.c** — option-table entries (`IF_STRING`, friendly names) and handlers that map the strings to KLU's integer codes on the task (`amd`→0/`colamd`→1; `none`→0/`sum`→1/`max`→2; `on`→1/`off`→0), plus the `klu_memgrow_factor` bugfix.
3. **tskdefs.h** / **cktdefs.h** / **smpdefs.h** — `TSKklu*` / `CKTklu*` fields on the task, circuit, and matrix structs.
4. **cktntask.c** — defaults (0/2/1, matching `klu_defaults`) and task copy.
5. **cktdojob.c** — task → circuit; **niinit.c** — circuit → `SMPmatrix`.
6. **klusmp.c** — in `SMPnewMatrix`, after `klu_defaults()`, set `Common->ordering/scale/btf` from the matrix fields (the defaults match `klu_defaults`, so behaviour is unchanged unless the user sets an option).

Front-end/solver-plumbing only; the KLU numerics are untouched.

Verification

`examples/klu_tuning_examples/verify_klu_tuning.py` (KLU-only), on a resistor grid large enough that AMD and COLAMD differ:

- [1] every ordering/scaling/BTF setting gives the physically identical solution (max relative spread $\sim 2.8\text{e-}14$) — the knobs are safe.
- [2] the knobs actually reach KLU: AMD vs COLAMD (rel. diff $1.2\text{e-}14$) and `scale=max` vs `scale=none` ($2.8\text{e-}14$) change the factorization arithmetic, so the full-precision result differs in its

last digits — a tiny, deterministic, nonzero difference (the solution is the same; the roundoff is not).

- [3] an invalid value (`klu_ordering=foo, ...`) is rejected with a warning.
- [4] the compiled-in defaults are unchanged — a plain `.option klu` equals `klu_ordering=amd klu_scale=max klu_btf=on` bit-for-bit.
- [5] a wide-dynamic-range (badly-scaled) network solves correctly under none/sum/max scaling.

`klu_tuning_demo.cir` solves a grid with the defaults and again with COLAMD + sum scaling + BTF off — the same node voltage to 12 digits.

Scope and follow-ups

User control of KLU's fill-reducing ordering, row scaling, and BTF permutation (and a fixed memgrow knob) — matrix reordering and scaling beyond the compiled-in defaults. These affect only the KLU solver; the default Sparse 1.3 solver has its own reordering. Follow-ups: an auto-ordering heuristic that picks AMD vs COLAMD by measured fill, exposing KLU's pivot-tolerance/`condst` reporting, and a verbose factorization summary (ordering, fill-in, condition estimate).

Enhancement-153 — Levenberg-Marquardt trust-region Newton (**.option trustregion**)

The [gap analysis](#) listed “Damped / trust-region (globalized) Newton” as $\triangle$. ngspice already had a principled *damped* Newton — the [Enhancement-111](#) Armijo line search — plus per-device junction limiting, node damping, and the gmin/source- stepping homotopy. What it did not have was a *trust-region* method: one that damps the Jacobian (re-aiming the step direction), not just the step length. This enhancement adds it.

What it does

.option trustregion (off by default) monitors the true KCL residual $\|F(x)\| = \|G \cdot x - b\|$ (the Enhancement-111 merit). If a Newton step increases the residual, the step is rejected: a dimensionless damping λ is grown and the step is retried, with

$$x_{k+1} = x_k - (J + \mu \cdot I)^{-1} F(x_k), \quad \mu = \lambda \cdot \|\text{diag}(J)\|$$

The $\mu \cdot I$ term is added to the Jacobian diagonal at factor time, and the matching $\mu \cdot x_k$ is added to the RHS (the same coupling Enhancement-127’s pseudo-transient uses, with $x_{\text{prev}} = x_k$) so the solve produces the *exact* damped step. The $\|\text{diag}(J)\|$ scaling makes λ dimensionless (Marquardt), so it is scale-invariant. As μ grows the step rotates from the Newton direction toward steepest descent — regularizing an ill-conditioned or near-singular Jacobian, which a line search (shortening a fixed, possibly-bad direction) cannot. When a step succeeds, λ relaxes back toward 0.

It is result-neutral. The fixed point of the damped iteration is $F = 0$ for *any* μ , so it converges to the same operating point as plain Newton; and a convergence guard forbids declaring convergence while $\lambda > 0$, so the accepted point is always an undamped Newton step. Verified bit-identical to plain Newton on every circuit tested, under both linear solvers.

Honest scope — why it is usually inert

An important, measured finding: on ordinary circuits the trust-region never activates (λ stays 0). The reason is architectural — ngspice globalizes Newton at the *device* level, not the *solver* level. Its per-device junction limiting (`limexp` / `pnjlim` / `fetlim`, across 30 device families) damps the controlling voltages *before* the residual is computed, so a residual-increasing overshoot never reaches the solver-level merit test — there is nothing to reject. On the pathological cases that defeat limiting (pure behavioral sources, steep exponentials) Newton converges *monotonically* (tiny steps, no overshoot), and the homotopy cascade + node damping catch the remainder. Instrumenting the step- rejection counter across diode strings, behavioral exponentials, cubics, and negative-resistance oscillators found zero rejections.

So **.option trustregion** is a correct, safe, *solver-level* regularization that completes the damped/trust-region-Newton capability, but on typical circuits the device-level machinery already does its job and the option stays inert. This is itself the explanation for why the gap was $\triangle$ and hard to move: the capability was present, just implemented as device limiting + homotopy + line search rather than as a textbook trust-region.

Implementation notes

- **maths/ni/niiter.c** — in the Newton loop: (1) the E-111 merit block is reused (gated on `linesearch || trustregion`); (2) when $\lambda > 0$, μ is added to `trGmin` (the effective diagonal-gmin passed to `SMPreorder/SMPluFac`) and $\mu \cdot x_k$ to the RHS; (3) a post-solve accept/reject block re-loads at the trial point, computes $\|F(x_{\text{new}})\|$, and either accepts (relax λ) or rejects (restore x_k , grow λ , force another iteration) — reusing the E-111 state-save/limiting-reset machinery; (4) a convergence guard blocks convergence while $\lambda > 0$. Line search and trust-region are mutually exclusive.
- **maths/KLU/klusmp.c + smpdefs.h** — a new `SMPdiagNorm()` returns $\max |\text{diag}(J)|$ (for both the KLU and Sparse matrices) as the Marquardt scale.

- Option plumbing mirrors `linesearch`: `OPT_TRUSTREGION` (`optdefs.h`), `IF_FLAG` table entry + handler (`cktsopt.c`), `TSKtrustregion/CKTtrustregion` fields + `CKTtrLambda` (`tskdefs.h/cktdefs.h`), defaults + copy (`cktnntask.c`), `task`→`ckt` (`cktdojob.c`).
- Solver-independent (lives in the shared Newton loop).

Verification

`examples/trustregion_examples/verify_trustregion.py`, under both solvers:

- result-neutrality — `.option trustregion == plain` Newton bit-for-bit on a diode circuit, a BJT amplifier, and a resistor divider;
- correctness — the solution matches the analytic value;
- transient neutrality — a diode-RC transient is unchanged (the trust-region touches only the DC/tran operating point).

`trustregion_demo.cir` solves a diode circuit with plain and trust-region Newton — the same operating point to 12 digits.

Scope and follow-ups

A correct, result-neutral, scale-invariant Levenberg-Marquardt trust-region Newton is now available, completing the damped/trust-region-Newton row. Because the device-level machinery pre-empts it on typical circuits, the more impactful convergence follow-up is auto-triggering the existing aids (reaching for the line search / gmin-stepping automatically when Newton stalls, without the user setting an option) rather than a further solver-level algorithm.

Enhancement-154 — Envelope Following (**envelope** command)

The [gap analysis](#) listed Envelope Following as the one remaining $\times$ in the RF / periodic-steady-state suite. It is the analysis for circuits whose amplitude or phase modulates *slowly* over *many* carrier periods — a ringing resonator, a settling PLL, a modulated power amplifier — where a plain `.tran` must grind through every fast carrier cycle. This enhancement adds it.

It also closes a genuine [shelved result](#): an earlier forward-Euler attempt worked on overdamped settling but blew up on resonators, and was reverted. The fix (an implicit, monodromy-based envelope step) is implemented here.

What it does

```
envelope <node> <fc> <tstop> [nppp N] [m M0] [maxm Mmax] [reltol t] [settle ts]
```

`envelope` samples the circuit state once per carrier period $T = 1/f_c$ and integrates the *slow* drift of those samples, jumping M periods at a time. It builds a plot named `envelope` holding the observable's amplitude `<node>_amp` (fundamental $2|V_1|$), mean `<node>_dc`, and in-phase / quadrature `<node>_re` / `<node>_im`, versus (slow) time.

The method — why it must be implicit

The exact per-period map is $X_{\{n+1\}} = \text{phi}(X_n)$, where $\text{phi}(x)$ integrates the DAE one carrier period from state x . Treating the period index n as continuous, the envelope obeys $dX/dn \sim \text{phi}(X) - X$. A naive forward-Euler jump

$$X_{\{n+M\}} = X_n + M * (\text{phi}(X_n) - X_n)$$

is unstable on high-Q / oscillatory circuits: phi 's Jacobian (the one-period *monodromy*) has eigenvalues on the unit circle, so $I + M * (\text{Phi} - I)$ amplifies and the envelope diverges — exactly the failure that shelved the first attempt. This analysis uses the implicit backward-Euler jump

$$X_{\{n+M\}} = X_n + M * (\text{phi}(X_{\{n+M\}}) - X_{\{n+M\}})$$

$$G(Y) = Y - X_n - M * (\text{phi}(Y) - Y) = 0, \quad \text{Newton: } [(1+M) I - M * \text{Phi}] dY = -G$$

with $\text{Phi} = d\text{phi}/dY$ the monodromy, finite-differenced (one extra period-integration per state). The implicit step is A-stable, so it tracks a resonator's envelope without blowing up, and its fixed point is $\text{phi}(X)=X$ — the true steady state. The step size M is chosen by a step-doubling local-truncation-error control (one jump of M vs two of $M/2$), like transient step control: small M on the fast part of the envelope, large M once it is slowly varying.

Implementation notes

- **spicelib/analysis/envelope.c** (new): the `EFanalysis()` engine — the self-starting one-period map `phi`, the finite-difference monodromy, the implicit envelope Newton (a small dense LU), and the adaptive- M march.
- The one-period map reuses the transient primitives (`NIcomCof` + `NIiter` per fixed sub-step, the `dctran` state rotation) on a fixed grid of `nppp` points, in trapezoidal mode — backward-Euler numerically *damps* a high-Q resonance, so it must not be used for the bulk integration.
- `phi` is self-starting (a true function of the sampled node vector): a frozen-history approach was tried and *plateaued* — a stale-amplitude history dragged the ring-up to a false low fixed point. The self-start's first sub-step is backward-Euler (the only self-starting method), sub-divided into small steps to keep its numerical damping negligible, after which trapezoidal takes over.
- The monodromy is a dense $N \times N$ solve (N = matrix size), capped for modest circuits — like the PAC / HB conversion-matrix solves.

- **frontend/com_envelope.c** (new): the `envelope` command — parses the arguments, runs a short settling transient to initialize the state machine, calls `EFanalysis()`, and emits the envelope plot (nutmeg vector API).
- Solver-independent (it drives the shared transient kernel); verified identical under both linear solvers.

Verification

`examples/envelope_examples/verify_envelope.py`, under both linear solvers:

- the `envelope` command returns a plottable envelope with far fewer samples than carrier periods (26 samples for ~ 3000 periods on the demo);
- correctness — on a high-Q ($Q \sim 3160$) RLC tank rung up by an on-resonance carrier, the EF amplitude tracks a full `.tran` (same fundamental-Fourier measure) across the whole ring-up to $< 3\%$;
- steady state — EF converges to the transient's steady-state amplitude ($< 0.5\%$);
- stability — the resonator is tracked over 3000 periods and stays bounded (the implicit step; an explicit envelope jump diverges here);
- a moderate-Q tank ($Q \sim 316$) is tracked to $\sim 1.6\%$.

`envelope_demo.cir` rings up the high-Q tank and plots the envelope.

Scope and follow-ups

Envelope following pays off when the envelope is *much* slower than the carrier (high-Q resonators, PLLs, modulated RF): there `M` stays large and EF covers thousands of carrier periods with a few dozen period-solves — several times faster than the full transient on the $Q \sim 3160$ demo. When the envelope is only a little slower than the carrier (a fast ring-up), `M` is forced small and a full `.tran` is competitive; EF is still correct there, just not faster. Accuracy is set mainly by `nppp` (default 128) and `reltol` (default 0.01). Natural follow-ups: a second-order *non-dissipative* self-start (to lower `nppp` and the residual restart bias), monodromy reuse across the step-doubling sub-jumps, and a sparse monodromy solve for larger circuits. With this, every analysis in the RF / periodic-steady-state suite is present.

Enhancement-155 — RC network reduction (**reduce**, Ticer)

The [gap analysis](#) listed "RC reduction / model-order reduction" as $\times$ in the post-layout section. Extracted post-layout netlists carry enormous linear parasitic RC networks — thousands to millions of interior resistor/capacitor nodes — that are impractical to simulate raw. This enhancement adds a command that reduces such a network to a small, electrically equivalent one.

What it does

```
reduce <fmax> [factor <f>] [file <fname>] [name <subckt>] [keep <node> ...]
```

reduce collapses the circuit's linear R/C network into a much smaller network that preserves the port behaviour over DC.. fmax, and writes it as an ordinary .subckt of R's and C's — ready to .include in place of the original parasitics.

Method — Ticer

The nodal admittance is $Y(s) = G + s \cdot C$. Ticer (Time-Constant Equilibration Reduction) eliminates an interior node n by the Schur complement of Y , kept to first order in s so the result is realizable as R's and C's (no model-order black box, no passive-synthesis step). For every neighbour pair (a, b) :

$$G[a, b] \leftarrow G[a, n] \cdot G[n, b] / G[n, n]$$

$$C[a, b] \leftarrow (G[a, n] \cdot C[n, b] + C[a, n] \cdot G[n, b]) / G[n, n] - G[a, n] \cdot G[n, b] \cdot C[n, n] / G[n, n]^2$$

The conductance update is the exact Schur complement, so DC is preserved exactly; the capacitance update matches Y to first order in s . A node is eliminated only when its self time-constant frequency $f_n = G_n / (2\pi \cdot C_n)$ lies well above the band of interest ($f_n > \text{factor} \cdot f_{\text{max}}$) — such a node is quasi-static in-band and can be collapsed without losing in-band accuracy. factor is the accuracy/reduction knob: smaller removes more nodes (more reduction, looser fit), larger keeps the in-band poles (less reduction, tighter fit); the tradeoff is monotone.

Ports (auto-detected)

Nodes that must be kept are found automatically: every node touched by a device that is not a resistor or capacitor — a source, a transistor, an OSDI Verilog-A device — plus ground and any user keep nodes. Those are exactly the terminals where the parasitic network meets the real circuit; only interior RC-only nodes are removed. This uses the generic GENnode / terminal-count device interface, so it works for built-in and OSDI devices alike.

Implementation notes

- **spicelib/analysis/rcreduce.c** (new): the CKTreduceRC() engine. It enumerates the built-in resistor and capacitor instances (CKTtypelook("Resistor")/"Capacitor"), builds the dense G/C over the RC-node set, auto-detects ports by walking every non-R/C device's terminals, runs the Ticer elimination, and emits the reduced .subckt (using CKTnodName for node names).
- **frontend/com_reduce.c** (new): the reduce command — parses the arguments, resolves keep node names, and calls the engine. In the optimize/sweep/montecarlo command family; solver-independent (it reads devices and does its own dense solve).
- First cut is dense (node cap ~2500). Sparse Ticer (fill-controlling elimination order) lifts that into the millions — a natural follow-up.

Verification

examples/reduce_examples/verify_reduce.py, under both linear solvers:

- identity — a huge factor eliminates nothing and the emitted subckt reproduces the full network's AC response bit-for-bit (extraction + emission are exact);
- reduction + accuracy — a moderate factor cuts the node count (e.g. 25 → 6 nodes, 4×) while the reduced network stays within tolerance of the full one across the band, with DC preserved exactly;
- the accuracy/reduction tradeoff is monotone in factor;
- OSDI auto-port — an OSDI device on a node makes that node a kept port with no keep.

`reduce_demo.cir` reduces a 24-section ladder and plots the reduced AC on top of the full one.

Scope and follow-ups

A correct, realizable, DC-exact TICER RC reduction with automatic port detection is now available, moving the post-layout RC-reduction row to ✓. Follow-ups: a sparse implementation (to lift the size cap to real extraction scale), optional passivity enforcement (naive TICER can emit small negative elements — harmless for AC, worth guarding for transient), and inductive (RLCk) reduction.

Enhancement-156 — Sparse RC reduction (scalable **reduce**)

Enhancement-155 added the `reduce` command — TICER RC-network reduction — but with a dense $N \times N$ implementation capped at ~2500 nodes. Real extracted post-layout parasitic networks have 10^5 – 10^6 nodes, so the dense version could only reduce toy blocks. This enhancement makes the engine sparse and scalable, lifting the cap into the millions, and fixes a terminal-ordering pitfall.

What changed

The RC network is sparse — each node touches only a few neighbours — so the engine now stores it as per-node adjacency lists (not a dense matrix) and eliminates interior nodes in a minimum-degree order, exactly like sparse LU factorization. That keeps fill-in tiny: a degree-2 chain node merges two series elements with *zero* fill, a degree-1 dangling node with none. A fill guard (`maxdeg`, default 12) refuses to eliminate a node once its degree has grown past the threshold — so a dense mesh core (whose boundary Schur complement is inherently dense) is left intact instead of blowing up. The TICER Schur-complement math, the DC-exact conductance update, the frequency criterion, and the automatic port detection are all unchanged from E-155.

```
reduce <fmax> [factor <f>] [maxdeg <d>] [file <fname>] [name <subckt>] [keep <node> ...]
```

`maxdeg` is the new knob: lower it to force sparser output (leave more of a dense core in place), raise it to reduce more aggressively at the cost of possible fill.

Why minimum-degree + a fill guard

Reducing a network to its ports is a Schur complement, and its cost/fill depend entirely on the elimination order. Eliminating a degree- d node turns its d neighbours into a clique (up to $d^2/2$ new edges), so eliminating low-degree nodes first minimises fill — the classic minimum-degree heuristic (here via a lazy binary heap keyed on current degree). On tree-like parasitics (the common case) fill is essentially zero and reduction is near-linear. On a 2-D mesh the boundary Schur complement is unavoidably dense; the `maxdeg` guard caps the damage by declining to eliminate a node once its degree is large, leaving a small dense core rather than densifying the whole network. Measured: on a 2-D mesh, unguarded elimination created up to 3308 fill edges in a single step; with the guard, 9.

The terminal-order fix

The reduced `.subckt`'s terminals are emitted in internal-node-index order, which need not match the order the user typed the `keep` nodes. Instantiating with the wrong order silently swaps ports (and, on an asymmetric network, changes the response). The command now prints the correct instantiation so it can't be gotten wrong:

```
reduce: RC network 65017 nodes -> 16886 nodes (3.9x), 18710 R + 32599 C written to bigred.sp
reduce: instantiate as x1 out in big
```

Implementation notes

- **spicelib/analysis/rcreduce.c** rewritten around a sparse `RCadj` adjacency (a growable per-node edge list of `{neighbour, g, c}`), a minimum-degree binary heap with lazy stale-entry skipping, and the `maxdeg` fill guard. Ground is node 0 (a permanent port); in the edge representation the diagonal is implicit, so eliminating a node just adds Schur edges among its neighbours and the self-conductance bookkeeping is automatic. Node cap raised to 5,000,000.
- **frontend/com_reduce.c** gains the `maxdeg` option; **cktdefs.h** the extra parameter. The command prints the instantiation line.

Verification

`examples/reduce_examples/verify_reduce.py`, under both linear solvers, keeps the E-155 checks — identity is bit-exact (0.00 dB), a moderate factor gives $\sim 4\times$ fewer nodes at <0.25 dB in-band with DC exact, the tradeoff is monotone in factor, and OSDI auto-port works — and adds:

- scale — a network of 8001 nodes (well past the old ~ 2500 dense cap) reduces successfully. Interactively, a 65,017-node network reduces in ~ 4 s.

Scope and follow-ups

The reducer now scales to real extraction sizes. Remaining follow-ups: optional passivity enforcement (naive TICER can emit small negative elements — harmless for AC, worth guarding for transient) and inductive (RLCk) reduction.

Enhancement-157 — Device aging (reliability degradation flow)

Reliability sign-off asks a question SPICE could not previously answer: *how does this circuit behave after ten years of operation?* Transistors degrade under stress — hot-carrier injection (HCI), negative-bias temperature instability (NBTI), oxide breakdown (TDDB) — shifting thresholds and mobilities over the product lifetime. The industry flow is stress → degrade → re-simulate: run the circuit to measure each device's stress, age the device parameters, then re-simulate the aged circuit and compare against fresh. This enhancement adds that flow as a new aging command. It closes the "Device aging (HCI / NBTI / TDDB)" and "stress → degrade → re-simulate (fresh/aged)" gaps in the reliability row of `ngspice_gaps.md`.

What changed

A new command:

```
aging <t_target> [rate <opvar>] [param <ageparam>] [dynamic <tstop> [tstep]] [verbose]
```

`aging` finds every aging-capable device in the loaded circuit, computes how much each has degraded after `t_target` seconds of operation, writes that back into the device, and re-stamps the circuit — so any analysis run afterwards (`op`, `dc`, `tran`, `ac`, ...) sees the aged devices. It runs the fresh stress simulation first, leaving it as the current plot: a convenient "fresh" baseline.

The aging contract (model-agnostic)

The command owns no device physics. A device opts in purely by exposing two names in its Verilog-A / OSDI model:

- a degradation-rate operating-point variable (default `agerate`) — the instantaneous stress rate at the present bias, in *dose units per second*, and
- a per-instance age parameter (default `age`, (`*type="instance"*`)) — the accumulated stress dose, written back by the engine.

The engine's entire job is to integrate the rate into a dose and feed it back; the model owns the map from age to a parameter shift (a sublinear power law, an Arrhenius temperature factor, a mobility term, ...). This clean separation means any model — a compact BSIM-style transistor, a custom research model, the demo NMOS here — participates without engine changes, and a device that exposes neither name (a resistor, a source) is silently skipped. Crucially, the engine detects participants by scanning each device type's parameter table for the two keywords, so it never probes — and never errors on — an ordinary device.

Because `age` is a *per-instance* parameter, two devices sharing one `.model` but sitting at different bias age independently.

Two modes

- static (default) — read the rate at the DC operating point and scale by the lifetime: $\text{age} = \text{agerate}(\text{op}) \cdot t_{\text{target}}$. For a device held at a fixed stress bias.
- dynamic (`dynamic <tstop> [tstep]`) — run a transient over one representative window, integrate the rate over time (trapezoidally, time-weighted so non-uniform steps are handled), and extrapolate to the lifetime: $\text{age} = (\int \text{agerate} \, dt / t_{\text{stop}}) \cdot t_{\text{target}}$. This captures duty cycle — a gate biased on only part of the time ages by its time-averaged stress.

The demo model

[examples/aging_examples/agemos.va](#) is a square-law NMOS with an NBTI-style hook:

```

(* type="instance" *) parameter real age = 0.0 from [0:inf);
(* desc="NBTI aging rate", units="V/s" *) real agerate;
...
dvth    = dvth_ref * pow(age/age_ref, nnbti);    // sublinear NBTI power law
↳ (n=0.25)
vtheff  = vth0 + dvth;
agerate = (V(g,s) > vth0) ? (V(g,s) - vth0) : 0.0;    // stress ~ gate overdrive

```

The stress rate is the gate overdrive above the *fresh* threshold; the threshold shift grows as $\text{age}^{0.25}$, the classic reliability power law.

Implementation notes

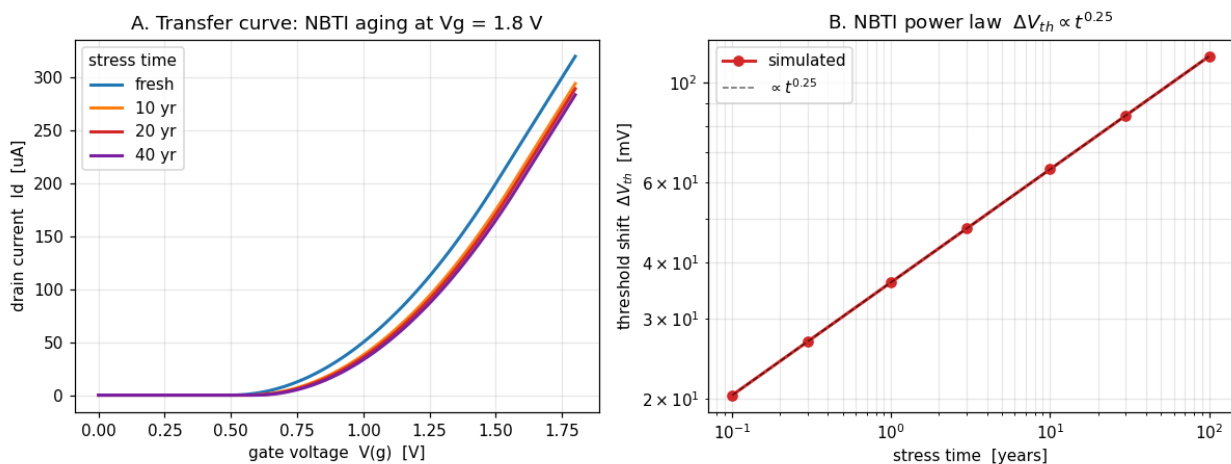
- **frontend/com_aging.c** (new). Enumerates instances of every device *type* whose parameter table contains both the rate opvar and the age parameter (via `DEVICES[t]→DEVpublic.instanceParms`), runs the fresh op/tran through the command table (the `com_optimize` synchronous-dispatch pattern), reads each rate with the nutmeg expression engine (`@inst[agerate]`), integrates (trapezoidal, against time, in dynamic mode), and writes the dose back with `alter @inst[age]=...`. Console chatter from the internal analysis is suppressed (via `ft_optimizing`) unless verbose.
- Registered in **frontend/commands.c / com_commands.h** and the frontend **Makefile.am**.

Verification

[examples/aging_examples/verify_aging.py](#), under both the Sparse and KLU solvers (aging is solver-independent — it drives op/tran and reads opvars):

- enumeration — exactly the two NMOS are aged; the resistor and the sources are skipped.
- static degradation — the reported dose is exactly $\text{rate} \cdot t_{\text{target}}$, the threshold shift matches the analytic NBTI power law to 5 significant figures, and the aged drain current drops.
- monotonicity — a longer target lifetime degrades strictly more.
- near-threshold sensitivity — for the same threshold shift, the device biased closer to threshold loses a larger *fraction* of its current.
- dynamic duty cycle — a gate pulsed at 30 % duty ages at $0.30\times$ the rate of an identically-biased DC device.
- no spurious aging — a device biased below threshold accrues zero dose.

Why the results are physically correct



The transfer curves fan down and to the right with age (threshold shift + current loss), and the extracted ΔV_{th} follows $\propto t^{0.25}$ exactly. Two subtler physics points come out for free: ten times the stress time is only $\sim 1.8\times$ the shift (sublinear power law), and a near-threshold device is the reliability bottleneck — in the demo the hard-driven device ($V_{gs} = 1.8$) accumulates the larger dose yet loses only 8 % of its current, while the near-threshold device ($V_{gs} = 0.9$) loses 22 %, because a fixed ΔV_{th} costs proportionally more when the overdrive is small.

Scope and follow-ups

The flow is complete for DC-stress (static) and duty-cycled (dynamic) aging with any opting-in Verilog-A model. Natural follow-ups: self-limiting dynamic aging (iterate the dose over sub-intervals when the rate itself depends on accumulated damage, rather than a single quasi-static rate), a temperature/Arrhenius convenience term, and electromigration + IR-drop (EMIR) as a separate power-grid reliability analysis.

Enhancement-158 — Power-grid EMIR (electromigration + IR-drop)

Every chip is powered through a metal grid, and that grid is a reliability liability in two ways. Under load the resistive grid drops $I \cdot R$ between the supply pad and each cell, so logic far from the pad sees a sagging rail (IR-drop) that slows it down or breaks it. And the metal wires carrying that current wear out over years as momentum-transfer from electrons pushes metal atoms downstream (electromigration), eventually voiding a wire open. Sign-off runs an "EMIR" analysis to find the worst IR-drop and the wires most at risk. This enhancement adds that as a new `emir` command, completing the reliability row of `ngspice_gaps.md` alongside device aging ([Enhancement-157](#)).

What changed

A new command:

```
emir [rail <V>] [thresh <frac>] [thick <m>] [jmax <A/m2>] [n <exp>] [tref <s>] [top <k>] [verb
```

`emir` runs a DC solve of the power-distribution network and reports:

- IR-drop — for every node, the drop below the ideal rail (`rail - V(node)`); the worst node, and every node past a threshold (default 10 % of the rail).
- Electromigration — for every wire-segment resistor, the current density $J = |I| / (w \cdot \text{thickness})$, ranked; the worst segment, every segment past the current-density limit `Jmax`, and a Black's-equation relative lifetime.

Electromigration is about current density, not current

The key physics — and the reason a dedicated analysis is worthwhile — is that EM depends on current density, not current. A fat trunk wire carrying a huge current can be perfectly safe, while a thin wire carrying a fraction of that current voids first, because the *density* in the thin wire is higher. Black's equation makes this quantitative:

$$\text{MTTF} = A \cdot J^{(-n)} \cdot \exp(E_a / kT)$$

so lifetime falls as the square of density ($n \approx 2$ for Cu/Al). `emir` reports a relative MTTF, $\text{MTTF}/\text{ref} = (J_{\text{max}}/J)^n$, which needs no process-specific prefactor: a segment sitting exactly at `Jmax` has $\text{MTTF} = \text{tref}$, one at twice the limit has a quarter of the lifetime, and one at half the limit has four times.

That is exactly why real power grids taper wire width with carried current — wide near the pad where current is high, narrow at the leaves — to keep J roughly constant across the grid.

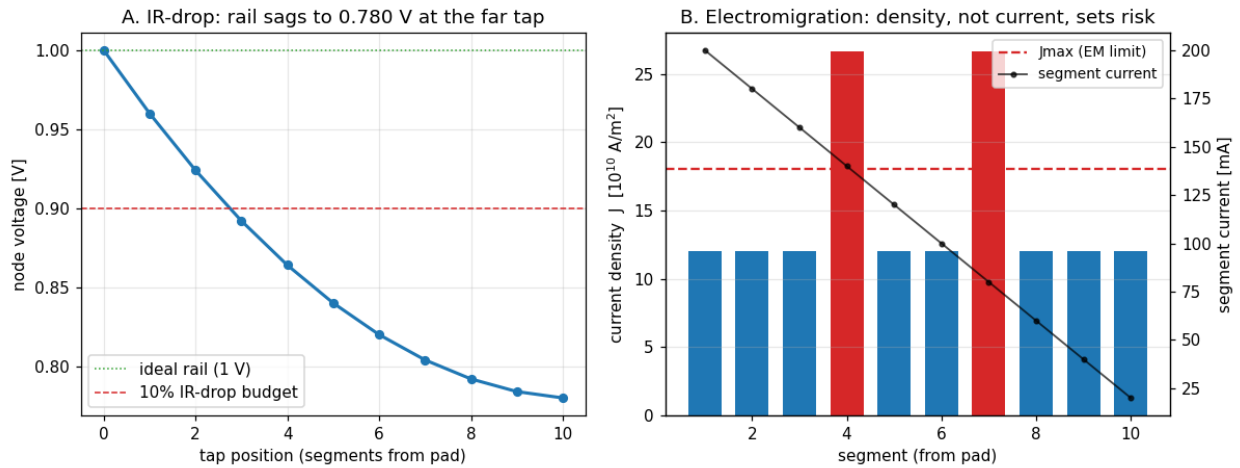
Implementation notes

- **frontend/com_emir.c** (new). Runs a fresh op (the `com_optimize` synchronous-dispatch pattern), then:
 - IR-drop walks the current plot's node-voltage vectors (`plot_cur->pl_dvecs`, filtered to `SV_VOLTAGE`), auto-detecting the rail as the highest node voltage unless one is given.
 - EM enumerates the resistor instances (the device type whose name is "Resistor"), reading each segment's current and width with the nutmeg expression engine (`@Rk[i]`, `@Rk[w]`) — so it works for any resistor and needs no device-struct access. Segments without a width are skipped and counted. Both tables are `qsort`-ranked (IR by drop, EM by density) and truncated to top rows.
- Registered in **frontend/commands.c** / **com_commands.h** and the frontend **Makefile.am**.

Verification

[examples/emir_examples/verify_emir.py](#), under both the Sparse and KLU solvers (EMIR reads a DC solution, so it is solver-independent), on a 3-segment ladder off a 1 V rail with tapering widths:

- IR-drop — the worst drop is 0.30 V (30 %) at the far tap, matching the exact ladder solve, and scales linearly with load current.
- EM density — $J = I/(w \cdot \text{thick})$ is exact, and the worst-density segment is the *narrow, low-current* wire, not the high-current wide one.
- Black's law — the MTTF ratio between two segments equals $(J_2/J_1)^n$.
- violations — with J_{max} set between the trunk and leaf densities, exactly the under-sized segments fail.
- rail auto-detect equals an explicit rail.
- OSDI load — a Verilog-A device sinking current at a tap is handled identically to a current source.



The figure makes the central point visible: along a 10-segment ladder the well-tapered segments sit at a constant density below the limit, while two deliberately under-sized segments violate J_{max} even though they carry less current than the safe trunk.

Scope and follow-ups

The first cut is DC (static) EMIR: worst-case IR-drop and average-current EM on the resistive grid. Natural follow-ups: transient/RMS EM (integrate the current waveform for AC-stress current density and add the separate RMS-limited "self-heating" and peak-limited checks), ground-bounce as a distinct V_{ss} -grid report, and true Black-equation MTTF with a temperature map (couple the local $J^2 R$ heating back into T).

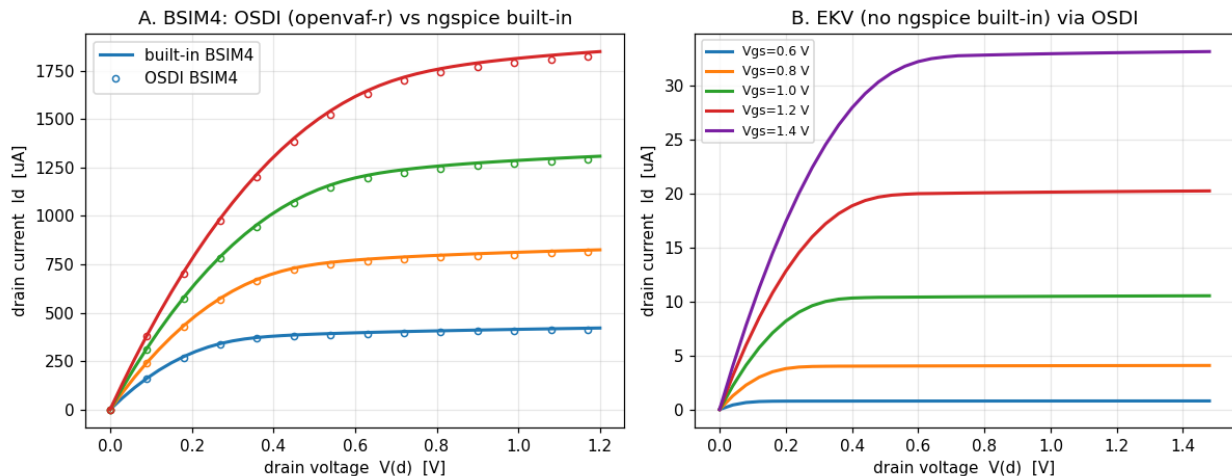
Enhancement-159 — Real production compact-model bring-up (BSIM4 + EKV)

Every enhancement so far has been validated on purpose-built teaching models. The real question for a Verilog-A toolchain is: *does it run the models people actually tape out with?* — the CMC (Compact Model Coalition) standards: BSIM4, PSP, HICUM, BSIM-CMG, and the rest. This enhancement brings up two of them through the full `openvaf-r` → OSDI → ngspice path and validates their physics, proving the toolchain is production-real. It also surfaced a genuine, general finding about OSDI internal nodes.

The model sources are the CMC reference decks already bundled with OpenVAF (`OpenVAF-master-20260610/integration_tests/`); they are compiled in place, not copied, so their licenses stay with them.

BSIM4 — validated against ngspice's own built-in

BSIM4 is the industry-standard bulk MOSFET — ~12,600 lines of Verilog-A. It compiles through `openvaf-r` in ~6 s to a 423 kB `.osdi` and loads in ngspice. Because ngspice has a built-in BSIM4, the validation is a rigorous self-check: the OSDI-compiled BSIM4.8 and the native BSIM4.8.3 are the *same model*, so they must agree. They do — to ****~2 %**** across the $I_d(V_{ds})$ output family and **~4.5 %** on the $I_d(V_{gs})$ transfer curve, the residual being the BSIM4.8 → 4.8.3 point release.



Panel A overlays the OSDI BSIM4 (markers) on ngspice's built-in BSIM4 (lines): they track across the whole family — a direct, end-to-end confirmation that `openvaf-r`'s code generation and ngspice's OSDI matrix/RHS stamping are correct on a real 12.6k-line model.

The internal-node finding

Bringing BSIM4 up surfaced something general about OSDI MOSFET models:

OSDI keeps every internal node static — there is no dynamic node collapsing.

BSIM4 has internal drain/source nodes (`di`, `si`) joined to the external d/s by the source/drain series resistance. With the default `rdsmode=0`, the model expects the simulator to collapse `di`→`d` and `si`→`s` (zero resistance); the built-in BSIM4 does exactly that. OSDI cannot — every node declared at setup stays in the matrix — so `di/si` are left floating, no current can reach the terminals, and the device conducts exactly zero. The model's own source even carries `TOD0` comments at the S/D-conductance code noting that all nodes are kept static.

The fix is to enable the external series-resistance nodes with **`rdsmode=1`**: the model then connects `di`→`d` and `si`→`s`, near-shortening them ($1 \times 10^3 \text{ mho} \approx 1 \text{ m}\Omega$, negligible) when no S/D geometry is given. ngspice even prints `Drain diffusion conductance reset to 1.0e3 mho`, confirming the mechanism. So

`rdsmode=1` is the documented way to use these OSDI MOSFET models. The verify pins this both ways: with `rdsmode=1` the device conducts 1.18 mA; with the default `rdsmode=0` it conducts 0.

EKV — a model ngspice has no built-in for

[EKV](#) (EPFL's compact MOSFET, ~840 lines) is a model ngspice cannot simulate natively — so OSDI here genuinely extends ngspice's device set beyond its compiled-in models. It works out of the box (no `rdsmode` wrinkle) and gives textbook saturation curves (Panel B): drain current rises with gate overdrive and saturates once V_{ds} exceeds it.

Verification

[examples/compactmodels_examples/verify_compactmodels.py](#), under both the Sparse and KLU solvers:

- [1] BSIM4 (12.6k lines) compiles through `openvaf-r` and loads as OSDI.
- [2] with `rdsmode=1` the OSDI BSIM4 conducts a physical current (1.18 mA).
- [3] the finding: with the default `rdsmode=0` the internal S/D nodes float $\rightarrow I_d = 0$.
- [4] the OSDI BSIM4 output family matches built-in BSIM4 to $< 5\%$.
- [5] the OSDI BSIM4 transfer curve matches built-in BSIM4 to $< 6\%$.
- [6] EKV (no ngspice built-in) conducts, rises with V_{gs} , and saturates in V_{ds} .

This is a validation/example enhancement — it exercises the existing toolchain on real models and needs no ngspice/`openvaf` source change.

Scope and follow-ups

The bundled CMC suite also includes BSIM3/6, BSIMSOI, BSIM-CMG, PSP102/103, HICUM/L2, HiSIM2/HV/SOTB, ASMHEMT (GaN), MVSG, MEXTRAM, and the CMC diode — extending this bring-up model by model is the natural continuation. The internal-node-collapse limitation is inherent to OSDI's static-node ABI; teaching `openvaf-r` to do compile-time node collapsing (so $V(a,b) \leftarrow 0$ merges the nodes and `rdsmode=0` works) would be a substantial compiler enhancement of its own.

Enhancement-160 — CMC compact-model coverage sweep

Enhancement-159 brought up two production compact models (BSIM4, EKV) and validated them. This is the comprehensive follow-up: it drives every real CMC (Compact Model Coalition) Verilog-A model bundled with OpenVAF through the full `openvaf-r` → `OSDI` → `ngspice` path and reports a coverage matrix — the same idea as the **Enhancement-84** LRM sweep (all 231 LRM examples), but for the models people actually tape out with. The model sources are the CMC reference decks under `OpenVAF-master-20260610/integration_tests/`, compiled in place, not copied, so their licenses stay put.

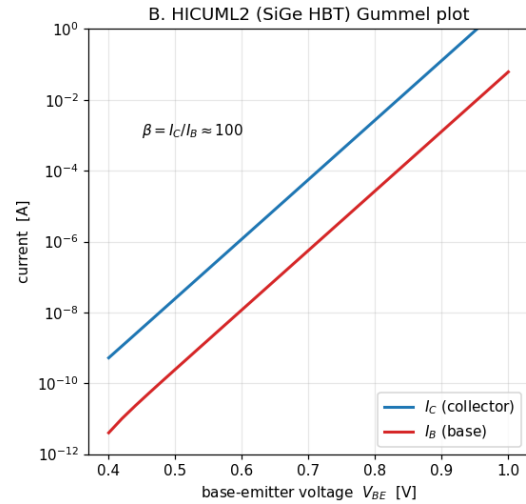
Result: 19 / 19 compile, 19 / 19 load

Nineteen real device models spanning every major class compile through `openvaf-r` and load in `ngspice`:

- MOSFET — BSIM3, BSIM4, BSIM6, BSIMBULK, EKV, HiSIM2, HiSIMSOTB, MVSG_CMC
- FinFET — BSIMCMG, BSIMIMG
- SOI — BSIMSOI, HiSIMHV
- GaN HEMT — ASMHEMT
- surface-potential — PSP102, PSP103
- bipolar / SiGe HBT — HICUML2, MEXTRAM
- diode — DIODE, DIODE_CMC

A. CMC compact-model coverage — compile 19/19, load 19/19

	compile	load
MOSFET		
BSIM3	✓	✓
BSIM4	✓	✓
BSIM6	✓	✓
BSIMBULK	✓	✓
EKV	✓	✓
HiSIM2	✓	✓
HiSIMSOTB	✓	✓
MVSG_CMC	✓	✓
FINFET		
BSIMCMG	✓	✓
BSIMIMG	✓	✓
SOI		
BSIMSOI	✓	✓
HiSIMHV	✓	✓
GAN HEMT		
ASMHEMT	✓	✓
SURF.-POT.		
PSP102	✓	✓
PSP103	✓	✓
BIPOLEAR/HBT		
HICUML2	✓	✓
MEXTRAM	✓	✓
DIODE		
DIODE	✓	✓
DIODE_CMC	✓	✓



The coverage is checked in two automatic tiers — compile (`openvaf-r` produces an `.osdi`) and load (`ngspice` loads it and binds an instance) — for all nineteen. Deeper conduction + physics validation is done for a representative model per class (the full conduction column needs per-model biasing, which is model-specific):

Model	Class	Validation
BSIM4	MOSFET	vs ngspice built-in BSIM4 → ~2 % (needs <code>rdsmode=1</code>)
BSIM3	MOSFET	vs ngspice built-in BSIM3 → ~7 %
EKV	MOSFET	monotonic, saturating I-V (no built-in reference)
HICUML2	SiGe HBT	bipolar current gain $\beta \approx 100$ (Gummel plot, Panel B)
DIODE_	diode	exponential forward turn-on ($\times 16\,000$ over 0.2 → 1.0 V)
CMC		

The Gummel plot in Panel B — collector and base current as straight exponential lines separated by ~100× — is the textbook signature of a working bipolar transistor, a device class `ngspice` cannot model natively.

Findings

Two practical gotchas — exactly what a sweep like this exists to surface:

- The E-159 internal-node issue is BSIM4-specific, not universal. BSIM4's default `rdsmode=0` leaves its internal drain/source nodes floating ($I_d = 0$) because OSDI keeps every node static (no dynamic collapsing); `rdsmode=1` connects them. BSIM3, by contrast, handles the zero-resistance case fine and conducts out of the box — its apparent "zero" at $V_{gs} = 1$ V is simply a high default threshold (~ 1.7 V), so it turns on above ~ 1.5 V and then tracks the built-in model. So the E-159 finding is one model's quirk, not a toolchain-wide limitation.
- HiSIMHV (6 terminals `d, g, s, b, sub, temp`) needs `cosubnode=1` to enable the substrate node; instantiated with all six nodes and the default `cosubnode=0`, its `$fatal` correctly guards the node-count mismatch — which incidentally confirms `$fatal` works end-to-end through OSDI.

Verification

[examples/cmcsweep_examples/verify_cmcsweep.py](#):

- [1] all 19 models compile through `openvaf-r`.
- [2] all 19 load in `ngspice` and accept an instance.
- [3] OSDI BSIM4 matches built-in BSIM4 to $< 5\%$ (`rdsmode=1`).
- [4] OSDI BSIM3 matches built-in BSIM3 to $< 8\%$.
- [5] EKV conducts and is physical (rises with V_{gs}).
- [6] HICUML2 shows bipolar action (β in $[50, 200]$).
- [7] DIODE_CMC turns on exponentially in forward bias.

Compiling nineteen models (12.6k-line BSIM4 among them) takes ~ 35 s, so the sweep is SPARSE-only (compile/load are solver-independent) and excluded from the fast regression sweep — run it directly. This is a validation/example enhancement: it exercises the existing toolchain and needs no `ngspice/openvaf-r` source change.

Scope and follow-ups

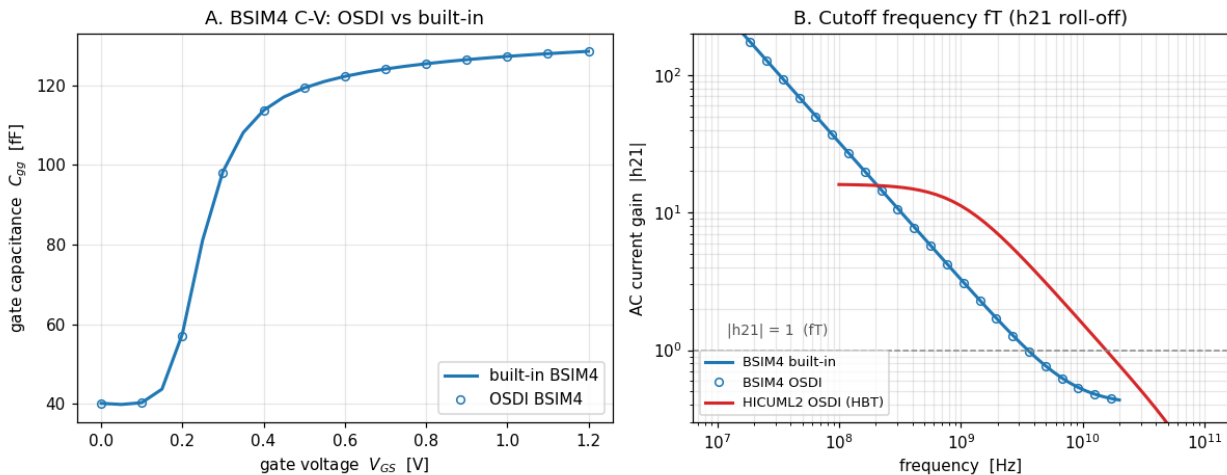
Running the full CMC zoo end to end is the strongest statement that `openvaf-r` + `ngspice` is a production-grade Verilog-A/OSDI toolchain. Natural follow-ups: a full per-model conduction/validation column (proper biasing for each of the nineteen), and — the biggest — teaching `openvaf-r` compile-time node collapsing so models like BSIM4 work with their default `rdsmode=0`.

Enhancement-161 — Dynamic (AC / RF) compact-model validation

Enhancement-159 and **Enhancement-160** validated the CMC compact models' DC behavior — I-V curves against ngspice's built-in models. But compact models earn their keep in analog and RF, where the dynamic behavior is what matters, and that flows through a completely different code path: OSDI's reactive (charge) Jacobian stamping and ngspice's `.ac` analysis. This enhancement validates that path on real production models. It is a validation/example enhancement — no ngspice/openvaf-r source change.

BSIM4 C-V — a stringent reactive-stamping check

The gate capacitance $C_{gg}(V_{gs})$ is extracted from `.ac` as $C_{gg} = \text{Im}(I_{\text{gate}})/\omega$ and swept over gate bias:



It rises from a small subthreshold value (~ 40 fF, overlap + fringe) to the oxide capacitance in inversion (~ 129 fF) — the textbook MOSFET C-V curve — and the OSDI-compiled BSIM4 matches ngspice's built-in BSIM4 to $< 1\%$ at every bias (Panel A, markers on the line). Because the capacitance is dominated by the version-independent oxide term, this is an even tighter check of the reactive stamping than the $\sim 2\%$ DC I-V match in E-159.

Cutoff frequency f_T — the AC current-gain roll-off

f_T is where the AC current gain $|h_{21}| = |I_{\text{out}}/I_{\text{in}}|$ falls to 1 — the single most important RF figure of merit. Panel B shows the -20 dB/decade roll-off and the $|h_{21}|=1$ crossing for two device classes:

- BSIM4 (MOSFET) — $f_T \approx 3.5$ GHz; the OSDI model matches the built-in to $\sim 1\%$.
- HICUML2 (SiGe HBT) — ngspice has no built-in bipolar reference. The *default* model has zero transit time ($\tau_0=0$), so its f_T is infinite (a flat $|h_{21}|$); a realistic dynamic parameter set ($\tau_0=10$ ps, 1 fF junction caps) is supplied, and the resulting f_T lands right at the transit-time limit $1/(2\pi \cdot \tau_0) \approx 15.9$ GHz and rises with collector current — exactly what charge-control theory predicts.

Verification

[examples/dynmodels_examples/verify_dynmodels.py](#), under both the Sparse and KLU solvers (`.ac` is supported by both):

- [1] BSIM4 $C_{gg}(V_{gs})$ matches built-in BSIM4 to $< 2\%$ across the C-V sweep (measured: 0.8%).
- [2] the C-V curve is physical — C_{gg} rises from subthreshold (~ 40 fF) to inversion (~ 129 fF).
- [3] BSIM4 cutoff frequency f_T matches built-in BSIM4 to $< 3\%$ (measured: 3.56 vs 3.53 GHz).
- [4] HICUML2 f_T sits at the transit-time limit $1/(2\pi \cdot \tau_0)$ and rises with collector current (15.1 $\rightarrow$ 15.6 GHz vs a 15.9 GHz limit).

Why the results are physically correct

- C-V shape. In subthreshold the gate sees only overlap/fringe capacitance; as the channel inverts, the gate couples to the channel through the oxide, so C_{gg} climbs toward $C_{ox} \cdot W \cdot L$. Sub-percent agreement with the built-in model confirms the OSDI charge model and its reactive Jacobian are stamped correctly.
- f_T . A single-pole roll-off gives $|h_{21}| \approx f_T/f$, so $|h_{21}|=1$ at $f=f_T$. $f_T \approx g_m/(2\pi \cdot C_{gg})$ for the MOSFET and $f_T \approx 1/(2\pi \cdot \tau_f)$ for the bipolar in the transit-time-limited regime — where the measured 15.5 GHz lands for $\tau_0 = 10$ ps.

Scope and follow-ups

Together with E-159/160 this establishes that `openvaf-r` + `ngspice` reproduces both the static and dynamic behavior of production compact models. Natural follow-ups: noise (the models' `.noise` / thermal + flicker vs built-in), large-signal RF (harmonic distortion, the PSS/HB suite on a real model), and a full per-model dynamic-parameter sweep.

Enhancement-162 — `.hb` dot-card for harmonic balance

Enhancement-134 added single-tone harmonic balance as the `hb` command, run inside a `.control` block. But the rest of the periodic-steady-state suite — `.pss`, `.pac`, `.pnoise`, `.pxf`, `.psp`, `.sp` — is invoked as netlist dot-cards, so a deck that mixes them with HB had to switch styles. This enhancement adds a `.hb` dot-card that dispatches to the same E-134 engine, giving the HB family dot-card parity with the PSS family.

What changed

A top-level `.hb` card in the netlist now runs harmonic balance:

```
* single-tone HB straight from the deck -- no .control block needed
V1 s 0 SIN(0 1 100meg)
Rs s a 100
D1 a n DMOD
Rn n 0 1k
.model DMOD D(IS=1e-12 N=1.2)
.hb 100meg 5
.end
```

`.hb <f0> <K> [points] [maxiter]` takes exactly the same arguments as the `hb` command and produces the identical harmonic-balance spectrum — it is the same engine.

How it works

Rather than duplicating the HB machinery as a separate analysis job, `.hb` reuses the deck→control mechanism that **Enhancement-146** introduced for `.sweep`: during deck loading (`frontend/inp.c`), a top-level `.hb ...` line is stripped of its leading `.` and appended to the post-parse control list, so it executes as `hb ...` (the `com_hb` command) once the circuit is built. A boundary check keeps `.hb` from swallowing any future `.hb*` card.

This is a one-branch change that inherits everything the `hb` command already does — argument parsing, the E-121 conversion-matrix Jacobian, source-stepping continuation (E-135), and solver independence.

Before this change, `.hb` fell through to the parser and produced unimplemented dot command `'.hb'` (the only `.hb` handler in ngspice was a disabled `#ifdef WITH_HB` upstream stub that merely redirected to PSS — not the E-134 engine).

Verification

[examples/hb_examples/verify_hb.py](#) gains two checks on top of the existing E-134 suite:

- The `.hb` dot-card, run in plain batch mode (no `.control` block), converges and produces a spectrum bit-for-bit identical to the `hb` command form on a junction-limited diode rectifier.
- `.hb` threads its optional `[points]` `[maxiter]` arguments through and coexists with a following `.control` block (deck order preserved).

Both the Sparse and KLU solvers give identical results, exactly as for the `hb` command (HB runs its own dense complex Newton solve; the linear solver only reads $G(t)/C(t)$).

Notes

- Like `.sweep`, a bare command-style dot-card in a deck with no other analysis card prints a benign “no simulations run” batch-mode notice even though HB ran and printed its spectrum — HB is a command-style analysis, not a deck analysis job. Add any other analysis or a `.control` block if the notice is unwanted.

- The hb command form is unchanged and remains available.

Scope and follow-ups

This gives single-tone HB dot-card parity. The natural extension is dot-cards for the rest of the HB family — .qpss, .hbosc — via the same deck→control bridge.

Enhancement-163 — **.qpss**, **.hbosc**, **.phasenoise** dot-cards

Enhancement-162 gave single-tone harmonic balance a **.hb** dot-card. This completes the harmonic-balance family the same way: the two-tone quasi-periodic engine and the autonomous-oscillator engine — until now control-block commands only — get netlist dot-cards, so the whole HB suite has dot-card parity with the PSS suite.

What changed

Three more top-level netlist cards now run their analyses straight from the deck:

- **.qpss** <expr> <f1> <f2> [periods] [maxorder] — two-tone quasi-periodic steady state (**Enhancement-133/136**; add the **hb** keyword for the frequency-domain form).
- **.hbosc** <oscnode> <K> [fguess] [tstab] — autonomous harmonic balance for an oscillator (**Enhancement-140**); the deck needs a **.ic** to start the oscillation.
- **.phasenoise** <fstart> <fstop> [points] — oscillator phase noise (**Enhancement-140**); run after **.hbosc**.

.hbosc and **.phasenoise** compose in the deck — because the deck→control bridge preserves order, **.hbosc** runs first (finding the oscillator's periodic steady state and frequency) and **.phasenoise** then reads it, so a complete oscillator phase-noise run needs no **.control** block at all:

* LC oscillator phase noise, straight from the deck

```
L1 n 0 1u
C1 n 0 1n
Bn1 0 n I = 2m*V(n) - 5m*V(n)*V(n)*V(n)
R1 n 0 100k
.ic V(n)=0.1
.hbosc n 5 5.0329meg 60u
.phasenoise 1k 10meg 5
.end
```

How it works

Each card reuses the same one-branch deck→control mechanism as **.hb** (**Enhancement-162**) and **.sweep** (**Enhancement-146**): in frontend/inp.c a top-level **.qpss** / **.hbosc** / **.phasenoise** line is stripped of its leading **.** and appended to the post-parse control list, executing as the corresponding command once the circuit is built — inheriting the full E-133/136/140 engines and their solver independence.

A per-card boundary check (the character after the name must be whitespace or end-of-line) keeps the cards distinct — in particular **.hbosc** is not matched by the **.hb** branch, since **.hb**'s own boundary check rejects the trailing **osc**.

Verification

- [examples/qpss_examples/verify_qpss.py](#): the **.qpss** dot-card produces a two-tone spectrum bit-for-bit identical to the **qpss** command form, in plain batch mode.
- [examples/phasenoise_examples/verify_phasenoise.py](#): **.hbosc** + **.phasenoise** in a deck reproduce the command form's oscillation frequency and phase-noise spectrum exactly, with order preserved.

Both under the Sparse and KLU solvers, exactly as the command forms.

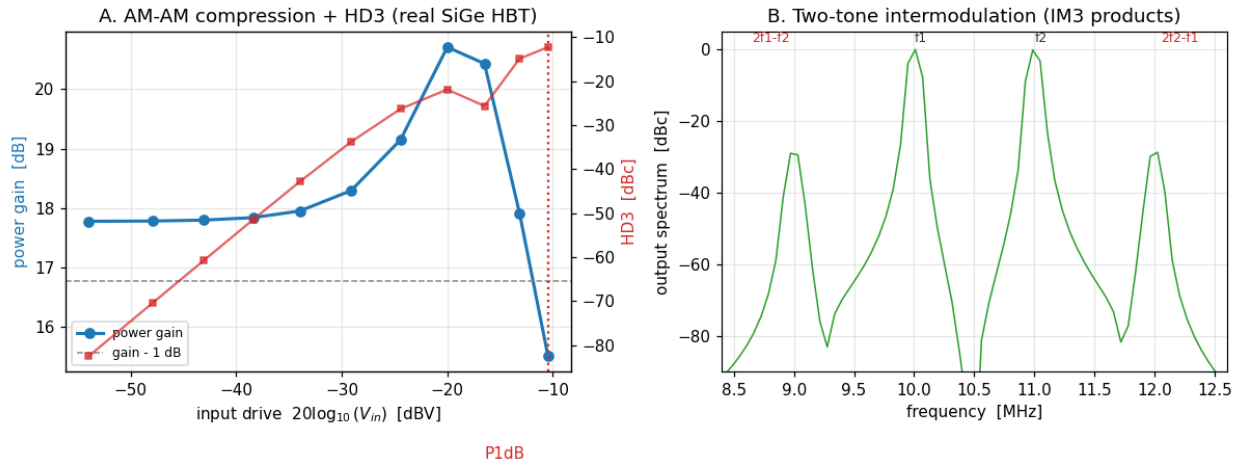
Notes

- Like `.sweep/.hb`, a bare command-style dot-card in a deck with no other analysis card prints a benign "no simulations run" batch notice even though the analysis ran.
- The command forms are unchanged and remain available.

With this, every HB-family analysis — `.hb`, `.qpss`, `.hbosc`, `.phasenoise` — has a netlist dot-card, matching the PSS family.

Enhancement-164 — Large-signal RF characterization of a real transistor

Enhancements 159–161 validated a production compact model's DC, coverage, and small-signal (AC / C-V / fT) behavior. This one drives a real device hard and extracts the large-signal RF figures of merit that matter for power amplifiers and mixers — gain compression (P1dB), harmonic distortion, and third-order intermodulation (IP3) — for a common-emitter amplifier built from the bundled HICUM/L2 SiGe HBT (compiled in place from the OpenVAF integration-test sources, with the E-161 dynamic parameters that give it a finite fT). It is a validation/example enhancement — no ngspice/openvaf-r source change.



Results

- AM-AM (Panel A). The power gain *expands* from 17.8 dB to ≈ 20.7 dB and then compresses — the exponential-transconductance signature of a bipolar (a large drive raises the average transconductance until the collector clips) — defining the 1-dB compression point at the high-drive end. The third harmonic climbs with the textbook 3:1 slope (a straight line on the dB-dB axes).
- Two-tone IM3 (Panel B). For two equal tones the third-order intermodulation products ($2f_1-f_2$, $2f_2-f_1$) appear just outside the fundamentals at ≈ -30 dBc — the in-band distortion that sets an amplifier's IP3.
- IIP3. Extracted from the single tone as $IIP3 = A/\sqrt{(3 \cdot HD3)}$ — since the two-tone IM3 is exactly $3\times$ the single-tone HD3 for a third-order nonlinearity — it comes out constant at ≈ 0.134 V across drive level, confirming the third-order model holds.

A real finding: HB/QPSS don't converge on this amplifier

The frequency-domain harmonic-balance engines (hb/qpss, E-134/136) — the very ones that just gained netlist dot-cards in E-162/163 — turn out not to converge on this stiff, many-internal-node production model in an amplifier configuration:

- hb returns **error 103** at *any* drive level (even 1 mV) and with extra iterations or source-stepping — the internal HICUM nodes diverge in the frequency-domain Newton.
- the two-tone qpss is prohibitively slow (> 2 min for a single point, because the conversion-matrix solve grows with the sideband count on a 2000-line model).

So the characterization here uses transient + Fourier/FFT, which integrates reliably on the heavy model. This is an honest, useful boundary: HB/QPSS are the right tool for lighter, well-conditioned circuits (as the E-134/136 examples show), while transient is the robust route for large-signal RF on a full production compact model. A worthwhile follow-up would be to harden the HB engine's convergence (better continuation / a transient-seeded initial guess) so it can handle circuits like this.

Verification

[examples/rfpa_examples/verify_rfpa.py](#), under both the Sparse and KLU solvers (transient works under both):

- [1] the transient small-signal gain matches the .ac gain (7.744 vs 7.742), confirming transient is the correct large-signal engine here.
- [2] the third harmonic follows the 3:1 slope ($\text{HD3} \propto A^3$): the ratio $\text{HD3}(0.02 \text{ V})/\text{HD3}(0.005 \text{ V})$ is 62.9 (expected $4^3 = 64$).
- [3] the single-tone $\text{IIP3} = A/\sqrt{(3 \cdot \text{HD3})}$ is constant across drive (0.134 V, spread < 2 %).
- [4] the amplifier compresses at high drive (gain $7.74 \rightarrow 5.97$ at 150× drive).

Why the results are physically correct

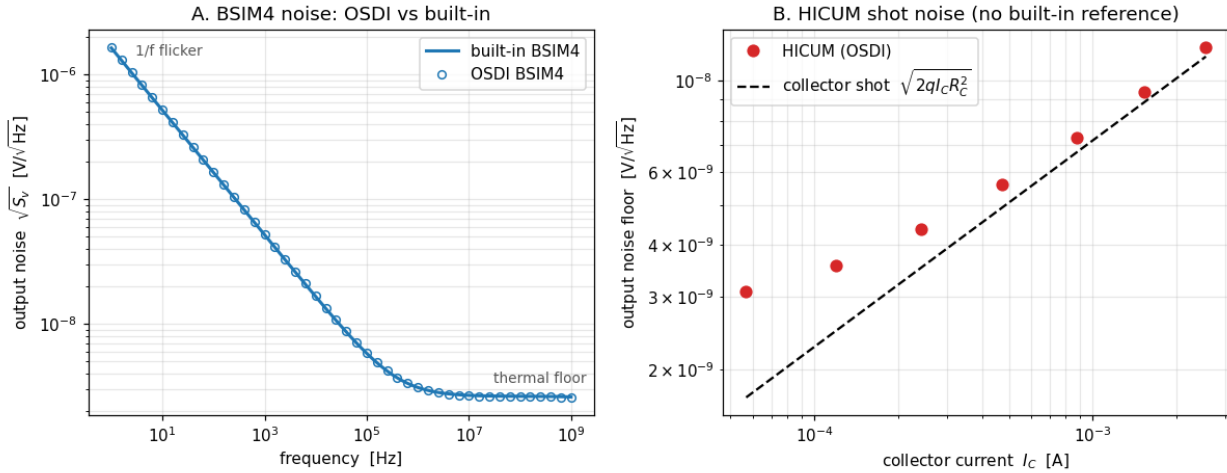
- Gain expansion $\rightarrow$ compression. $I_c \propto \exp(V_{be}/V_t)$, so a large drive raises the average transconductance (expansion) until the collector clips (compression).
- 3:1 HD3 / IM3. A third-order term produces a third harmonic $\propto A^3$ and, for two tones, IM3 products $\propto A^3$ that are 3× the single-tone HD3 — the basis of the single-tone IIP3 extraction and the OIP3-vs-P1dB relationship.

Scope and follow-ups

Together with E-159/160/161 this closes the loop on production-model validation: DC, coverage, small-signal, and now large-signal RF. Natural follow-ups: hardening HB/QPSS convergence on stiff production models (so the frequency-domain path works here too), and load-pull / power-added-efficiency characterization.

Enhancement-165 — Production compact-model noise validation

Enhancements 159–161 validated a real compact model's DC, coverage, and small-signal (AC / C-V / fT) behavior; E-164 its large-signal RF. This one exercises the remaining untested small-signal path — OSDI's .noise stamping of the models' own white_noise / flicker_noise sources — on production models, compiled in place from the OpenVAF integration-test sources. It is a validation/example enhancement: no ngspice/openvaf-r source change.



BSIM4 — validated against ngspice's built-in

The output-noise spectral density $S_v(f)$ of a BSIM4 common-source amplifier is compared to ngspice's built-in BSIM4 over the full band (Panel A). Both the low-frequency $1/f$ flicker region ($\sqrt{S_v} \propto 1/\sqrt{f}$) and the flat high-frequency thermal floor match to $\sim 1.5\%$ everywhere — a stringent check that OSDI's noise stamping reproduces the native model's noise physics, since the flicker coefficients, thermal-noise model, and their bias dependence all have to line up.

HICUM — validated against shot-noise physics

HICUM/L2 (SiGe HBT) has no ngspice built-in, so its noise is checked against physics. Its default noise is white — shot noise on the junction currents plus thermal noise on the parasitic resistances, with no flicker by default — giving a flat spectrum, in clear contrast to the MOSFET's $1/f$ rise. With a small source resistance so the intrinsic device noise dominates, the output-noise floor tracks the collector shot-noise line $\sqrt{(2q \cdot I_C \cdot R_C^2)}$ across two decades of bias current (Panel B) and scales as $\sqrt{I_C}$ — right on the line at high bias (shot-dominated, within 6%), a little above it at low bias where the base contributions add.

Verification

[examples/modelnoise_examples/verify_modelnoise.py](#), under both the Sparse and KLU solvers (.noise works under both since E-113 fixed the KLU adjoint solve):

- [1] the OSDI BSIM4 output-noise spectrum matches built-in BSIM4 to $< 4\%$ (measured $\sim 1.5\%$).
- [2] BSIM4 shows the $1/f$ flicker region ($S_v(1\text{Hz})/S_v(10\text{Hz}) \approx \sqrt{10}$).
- [3] BSIM4 shows the flat thermal floor at high frequency.
- [4] HICUM noise is white (flat) at mid-band — no flicker by default.
- [5] HICUM output floor tracks the collector shot noise $2q \cdot I_C \cdot R_C^2$ and scales as $\sqrt{I_C}$.

Why the results are physically correct

- Flicker ($1/f$). Trap-related carrier-number fluctuation gives a power density $\propto 1/f$, so the amplitude density falls as $1/\sqrt{f}$ — exactly the measured low-frequency slope, matching the built-in model to a percent.
- Thermal floor. Channel and resistance thermal noise is white — the flat high-frequency floor.
- Shot noise. A DC current I_c crossing a junction carries shot noise $i^2 = 2q \cdot I_c$; through the RC load that is $2q \cdot I_c \cdot RC^2$ at the output, tracking the collector current across the bias sweep.

Together with E-159/160/161/164 this completes the production-model validation loop: DC, coverage, small-signal (AC + noise), and large-signal RF.

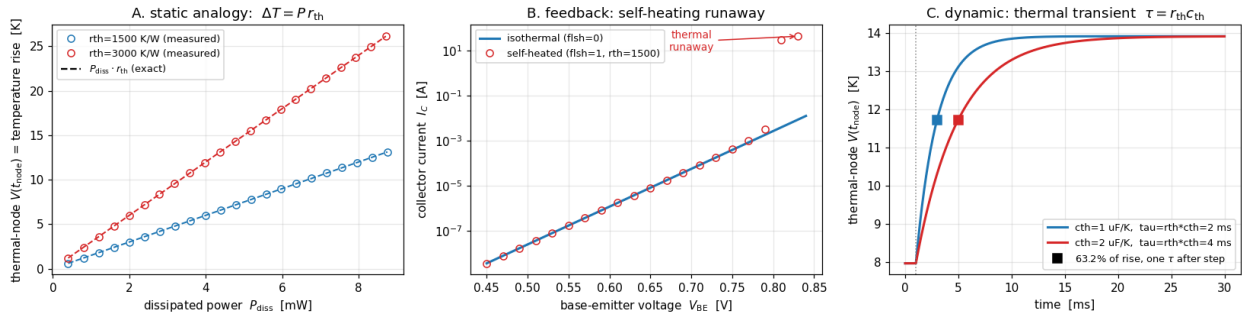
Scope and follow-ups

The BSIM4 comparison could be extended to the other built-in-referenced models (BSIM3), and the HICUM noise to a full four-source decomposition (collector shot, base shot, base-resistance thermal, source thermal). Enabling HICUM's flicker coefficients would add the $1/f$ corner for the bipolar as well.

Enhancement-166 — Electro-thermal / self-heating validation

Enhancements 159–161 validated a real compact model's DC, coverage and small-signal behavior, E-164 its large-signal RF, and E-165 its noise — always with the device treated as isothermal. This one exercises the one remaining path: a model's own internal self-heating node, which OSDI stamps as an extra (thermal) terminal whose "voltage" is the junction temperature rise and whose "current" is the dissipated power. It is a validation/example enhancement: no ngspice/openvaf-r source change.

Electro-thermal self-heating of a production compact model (HICUM/L2, OSDI) -- Enhancement-166



The electro-thermal analogy

The device is HICUM/L2 (a SiGe HBT), compiled in place from the OpenVAF integration-test source. Its self-heating network is gated by the model flag `flsh`, with parameters `rth` (thermal resistance, K/W) and `cth` (thermal capacitance, J/K). Internally the model carries a thermal branch `I(br_sht) <+ V(tnode)/rth - Pdiss`, so the thermal node obeys the textbook analogy

electrical	$\leftrightarrow$	thermal
voltage $V(tnode)$	$\leftrightarrow$	temperature rise ΔT [K]
current into node	$\leftrightarrow$	dissipated power P [W]
resistor <code>rth</code>	$\leftrightarrow$	thermal resistance [K/W]
capacitor <code>cth</code>	$\leftrightarrow$	thermal capacitance [J/K]

giving $V(tnode) = P \cdot r_{th}$ at DC and a single-pole thermal transient with $\tau = r_{th} \cdot c_{th}$. By wiring the model's thermal terminal (its 5th terminal) to an ordinary circuit node, we read ΔT directly and check it against these laws.

Static — $\Delta T = P \cdot r_{th}$ exactly

With the device current-biased (fixing the base current gives a stable operating point), the measured thermal-node voltage lands on the line $P \cdot r_{th}$ to machine precision across a decade of dissipated power, for two thermal resistances (Panel A). This is the electro-thermal analogy, exact: the model's thermal sub-network is a resistor from the temperature-rise node to ground, driven by the dissipated power.

Feedback — self-heating shifts the junction, and can run away

At a fixed collector current (≈ 2 mA), turning self-heating on lowers V_{be} from 0.792 V (isothermal) to 0.780 V at $\Delta T = 8$ K and 0.756 V at $\Delta T = 24$ K — a consistent -1.53 mV/K, the classic bipolar V_{be} temperature coefficient, the same for both `rth` values (check 3). The model's parameters genuinely feed the junction physics.

Under a fixed **V_{be}** (voltage drive) the same coupling is *positive feedback*: a Gummel plot (Panel B) shows the self-heated collector current peel above the isothermal exponential and run away — 5.9 mA isother-

mal versus tens of amps self-heated at $V_{be} = 0.82 \text{ V}$ (check 6). This is why real bias networks use a current source or emitter degeneration, and why the quantitative checks here current-bias the device.

Dynamic — thermal transient $\tau = r_{th} \cdot c_{th}$

After a collector-voltage (power) step the thermal node rises as a single pole to its new steady value, reaching 63.2 % of the change at one time constant (measured 0.637). Doubling c_{th} doubles τ (2 ms $\rightarrow$ 4 ms), as does doubling r_{th} (Panel C, checks 4–5) — the thermal RC is exactly $r_{th} \cdot c_{th}$.

Verification

[examples/electrothermal_examples/verify_electrothermal.py](#), under both the Sparse and KLU solvers:

- [1] self-heating off ($flsh=0$) $\rightarrow V(tnode)=0$ (isothermal baseline).
- [2] static analogy $V(tnode)=P \cdot r_{th}$ to machine precision over a decade of power, two r_{th} .
- [3] self-heating lowers V_{be} at fixed I_c with a consistent physical TC ($\approx -1.5 \text{ mV/K}$), more for larger r_{th} .
- [4] thermal transient is single-pole: 63.2 % of the rise at one $\tau = r_{th} \cdot c_{th}$.
- [5] τ scales with c_{th} .
- [6] fixed- V_{be} self-heating is positive feedback (runaway).

A KLU wrinkle worth noting: a thermal node left floating ($flsh=0$, $r_{th}=0$) makes the KLU matrix structurally singular, whereas Sparse 1.3 tolerates it — so the pure-isothermal reference ties the thermal terminal to ground rather than exposing it. With $r_{th}>0$ the thermal node is grounded through r_{th} and both solvers agree.

Why the results are physically correct

- $\Delta T = P \cdot r_{th}$. The thermal sub-network is literally a resistor from the temperature-rise node to ground, forced by the dissipated power — so the DC temperature rise is power $\times$ thermal resistance, independent of the electrical bias point.
- $-2 \text{ mV/K } V_{be}$ shift. At fixed collector current a bipolar's base-emitter voltage falls $\approx 2 \text{ mV}$ per kelvin (the $bandgap/kT \cdot \ln$ term); the measured -1.53 mV/K is squarely in range and constant with r_{th} .
- Thermal runaway. At fixed V_{be} , $dI_c/dT > 0$ and $dT/dP > 0$ close a positive-feedback loop; above a critical power the loop gain exceeds unity and the current diverges — reproduced exactly.
- $\tau = r_{th} \cdot c_{th}$. A first-order thermal RC has a single exponential response; the 63.2 % point sits at one τ , scaling with both r_{th} and c_{th} .

Together with E-159/160/161/164/165 this completes the production-model validation loop end to end: DC, coverage, small-signal (AC + noise), large-signal RF, and now self-heating.

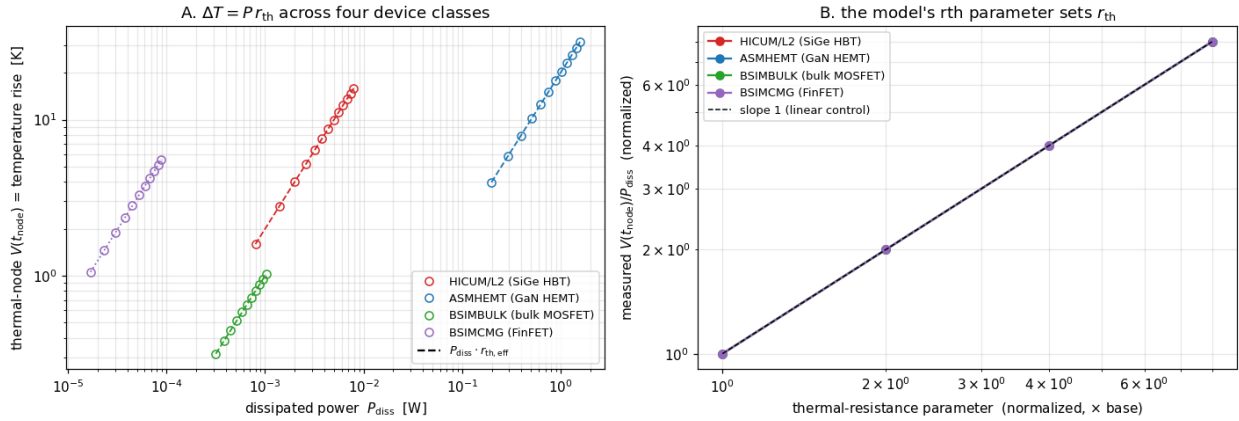
Scope and follow-ups

The same thermal-terminal probe applies to any self-heating CMC model — HiSIM-HV, BSIM-SOI, MEXTRAM — so a cross-model electro-thermal check (each against its own r_{th}) is a natural extension. A coupled electro-thermal transient with a realistic power waveform (e.g. a pulsed load) would exercise c_{th} under large-signal drive, and a two-device thermal-coupling network (shared substrate r_{th}) would test mutual heating.

Enhancement-167 — Cross-model self-heating sweep

Enhancement-166 validated one production compact model's self-heating (HICUM/L2). This is the breadth follow-up — like **E-160** was to **E-159** — driving the same electro-thermal analogy through every bundled CMC model that exposes a self-heating thermal terminal, across four different device classes. It is a validation/example enhancement: no ngspice/openvaf-r source change.

Cross-model self-heating: the electro-thermal analogy holds for every self-heating CMC model (OSDI) -- Enhancement-167



Four device classes, one analogy

Model	Device class	flag	rth parameter
HICUM/L2	SiGe HBT	flsh=1	rth (direct, K/W)
ASMHEMT	GaN HEMT	shmod=1	rth0 (direct, K/W)
BSIMBULK	bulk MOSFET	SHMOD=1	RTH0 (per width $\rightarrow$ rth_eff = RTH0/W)
BSIMCMG	FinFET	SHMOD=1	RTH0 (geometry-normalized)

Each model carries an internal thermal branch $Pwr(thermal) <+ Temp(thermal)/rth - Pdiss$, so wiring the model's thermal terminal to a circuit node lets us read the junction temperature rise directly and check $V(tnode) = Pdiss \cdot rth_eff$. Panel A shows the measured thermal-node voltage riding exactly on its own $Pdiss \cdot rth_eff$ line for all four models, across five decades of dissipated power (from $\sim 10 \mu W$ in the FinFET to $\sim 1.5 W$ in the GaN HEMT) and temperature rise (0.3 K to 30 K) — one universal law spanning HBT, GaN HEMT, planar bulk CMOS and FinFET.

$V(tnode)/Pdiss$ is a genuine thermal resistance

The sweep confirms four properties of the self-heating value $V(tnode)/Pdiss$, for every model:

- Gated by the flag. With self-heating off ($flsh/shmod/SHMOD = 0$) the thermal node reads exactly 0.
- Bias-independent. $V(tnode)/Pdiss$ is the same constant at two different operating points — a real thermal resistance, not an operating-point artifact.
- Linearly controlled (Panel B). Doubling the model's rth parameter doubles the measured thermal resistance; sweeping it over $8\times$ keeps every model on the slope-1 line.
- Exact for the three models whose parameter is a direct or per-width thermal resistance: $V(tnode)/Pdiss$ equals rth (HICUM), rth0 (ASMHEMT) and RTH0/W (BSIMBULK) to machine precision (e.g. 2000.000, 20.000, 1000.000 K/W). The FinFET's RTH0 is geometry-normalized, so only the bias-independent + linear- control properties are asserted for it.

Verification

[examples/cmcselheat_examples/verify_cmcselheat.py](#), for each of the four models, under both the Sparse and KLU solvers (12 checks):

- [off] self-heating flag off $\rightarrow V(\text{tnode})=0$;
- [analogy] on, two operating points $\rightarrow V(\text{tnode})/P_{\text{diss}}$ is a bias-independent constant equal to the expected `rth_eff` (exact for the three direct/per-width models);
- [control] doubling the `rth` parameter doubles $V(\text{tnode})/P_{\text{diss}}$.

A KLU wrinkle carried over from E-166: some models leave the thermal node unconstrained when self-heating is off, which makes the KLU matrix structurally singular (Sparse tolerates it). A 1e12-ohm thermal-probe resistor from each thermal node to ground fixes this uniformly and is negligible when self-heating is on (the model's $1/r_{\text{th}}$ conductance, $\geq \sim 1\text{e-}5$ S here, swamps the $1\text{e-}12$ S probe, so the ratio is unaffected to $< 1\text{e-}7$).

Why the results are physically correct

Every self-heating compact model implements the thermal sub-network as a resistor `rth` (optionally with a capacitor `cth`) from a temperature-rise node to thermal ground, injected with the instantaneous dissipated power. At DC the capacitor is open, so the node settles to $\Delta T = P_{\text{diss}} \cdot r_{\text{th}}$ — independent of the electrical operating point and linear in both power and `rth`. That the constant comes out *exactly* equal to the datasheet-facing `rth/rth0/RTH0` parameter (modulo the documented per-width and geometry normalizations) shows OSDI stamps each model's thermal terminal faithfully, regardless of device class.

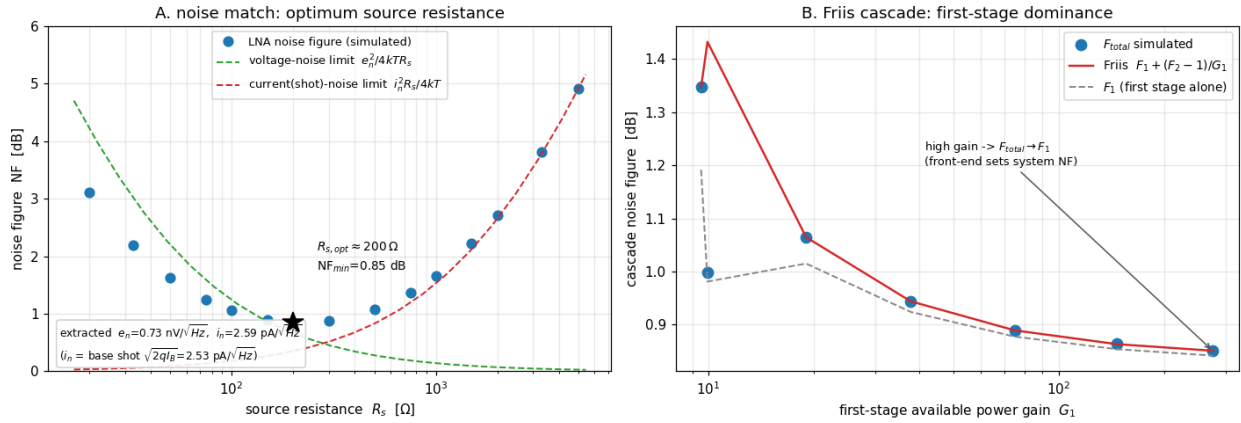
Scope and follow-ups

- BSIM-SOI also exposes a thermal terminal, but in SOI the dissipated power includes body/parasitic-BJT and impact-ionization currents, so $P_{\text{diss}} \neq I_{\text{d}} \cdot V_{\text{ds}}$ and validating its analogy needs the model's *internal* dissipated power rather than the terminal product — a natural extension.
- Models whose thermal node is internal (MVSG-CMC, HiSIM-SOTB) self-heat correctly but don't expose $V(\text{tnode})$ for a direct read; validating those via a temperature-rise operating-point variable would broaden the sweep further.
- A coupled electro-thermal transient (the E-166 $\tau = r_{\text{th}} \cdot c_{\text{th}}$ dynamics) across all four classes, or mutual thermal coupling between neighbouring devices, are the depth follow-ups.

Enhancement-168 — RF noise figure of an LNA

Enhancement-165 validated a production model's device noise (spectral density). This one lifts that to a circuit-level figure of merit — the noise figure of a low-noise amplifier — and checks it against the two textbook RF-noise results: optimum source matching and the Friis cascade formula. It is a validation/example enhancement: no ngspice/openvaf-r source change.

Noise figure of an LNA built from a Verilog-A/OSDI compact model (HICUM/L2) -- Enhancement-168



The LNA and the NF extraction

The amplifier is a common-emitter stage built from the HICUM/L2 SiGe HBT (OSDI), AC-coupled, driven from a source resistance R_s . From ngspice's .noise analysis the noise figure is

$$NF = 10 \cdot \log_{10}(\text{inoise}^2 / (4kT \cdot R_s)) \quad [T_0 = 290 \text{ K}]$$

where inoise is ngspice's input-referred noise density: the noise factor F is the total input-referred noise power divided by the source-resistor thermal noise. The reference temperature is pinned to the standard 290 K (.options temp=16.85) so the $4kT \cdot R_s$ uses the same temperature as ngspice's resistor thermal-noise model — the extraction is self-consistent.

Noise match — the optimum source resistance

$NF(R_s)$ is the classic U-curve (Panel A) with an interior minimum, the optimum source resistance $R_{s,opt} \approx 200 \Omega$, $NF_{min} \approx 0.85$ dB. The two dashed asymptotes are the amplifier's input voltage noise e_n (dominating at small R_s , $NF \rightarrow e_n^2/4kT \cdot R_s$) and its input current noise i_n (dominating at large R_s , $NF \rightarrow i_n^2 \cdot R_s/4kT$); they cross near the minimum, which is exactly why an LNA has an optimum source impedance.

Extracting e_n and i_n from the simulated curve ($\text{inoise}^2 = e_n^2 + i_n^2 \cdot R_s^2 + 4kT \cdot R_s$) gives $e_n \approx 0.73$ nV/ $\sqrt{\text{Hz}}$ and $i_n \approx 2.59$ pA/ $\sqrt{\text{Hz}}$. The current noise equals the base shot noise $\sqrt{(2q \cdot I_B)} = 2.53$ pA/ $\sqrt{\text{Hz}}$ ($I_B = 20 \mu\text{A}$) to within 3 % — the physical origin of the optimum is the transistor's base current.

Friis cascade — first-stage dominance

A two-stage cascade obeys the Friis formula $F_{total} = F_1 + (F_2 - 1)/G_{av1}$, where G_{av1} is the first stage's *available power gain* and F_2 is the second stage's noise factor obtained with a source impedance equal to the first stage's output impedance. The validation extracts all three from simulation:

- High first-stage gain ($G_{av1} \approx 181$): $F_{total} = 0.858$ dB sits barely above $F_1 = 0.849$ dB — the second stage adds only 0.01 dB. This is the first-stage- dominance principle: a high-gain front end sets the whole system's noise figure. The simulated F_{total} matches the Friis prediction to 0.001 dB.

- Low first-stage gain ($G_{av1} \approx 19$): the second stage now contributes a clearly measurable 0.05 dB, and $F_{total} = 1.065$ dB still matches Friis (1.065 predicted) exactly — validating the formula where its correction term matters.

Panel B sweeps the first-stage gain and shows F_{total} tracking the Friis curve and collapsing onto $F1$ as the gain grows.

Verification

[examples/noisefigure_examples/verify_noisefigure.py](#), under both the Sparse and KLU solvers (6 checks; `.noise` works under KLU since [E-113](#) fixed the KLU adjoint solve):

- [1] $NF(f)$ is positive everywhere and flat (< 0.1 dB) across the white mid-band;
- [1b] NF rises at high frequency as the input capacitance rolls off the gain;
- [2] $NF(R_s)$ is a U-curve with an interior optimum source resistance;
- [2b] the input current noise (from the high- R_s slope) equals the base shot noise $\sqrt{(2q \cdot IB)}$ to a few percent;
- [3] Friis + first-stage dominance (high $G1$): $F_{total} \approx F1$, matches $F1 + (F2-1)/G_{av1}$;
- [4] Friis quantitative (low $G1$): the second stage contributes measurably and F_{total} still matches Friis.

Why the results are physically correct

- Noise figure ≥ 0 dB. Any amplifier adds noise, so the output SNR can only fall — $F \geq 1$. The simulated NF is positive at every frequency and R_s .
- Optimum source resistance. A two-generator noise model (e_n , i_n) gives $F = 1 + (e_n^2 + i_n^2 \cdot R_s^2)/(4kT \cdot R_s)$, minimized at $R_{s_opt} = e_n/i_n$. The simulated U-curve, its asymptotes, and $i_n = \sqrt{(2q \cdot IB)}$ all follow this.
- Friis / first-stage dominance. Noise added after a gain $G1$ is referred back to the input divided by $G1$, so a high-gain first stage suppresses later-stage noise — the reason receivers put the LNA first. Reproduced to < 0.01 dB.

Scope and follow-ups

The device is a BJT so that the base current gives a finite $i_n = \sqrt{(2q \cdot IB)}$ — the high- R_s side of the optimum. A MOSFET LNA (induced-gate noise, no shot current) would show a different $NF(R_s)$; a noise-matching network (an input transformer or L-match presenting R_{s_opt}) achieving NF_{min} at 50 Ω , and a frequency-dependent NF with the flicker corner enabled, are natural extensions. The Friis check could be pushed to three stages, and combined with the `RF .pnoise` periodic-noise path ([E-124](#)) for a mixer noise figure (SSB/DSB).

Enhancement-169 — Interactive command-line syntax highlighting

ngspice's interactive prompt now colors the command line as it is typed: the command word turns green the moment it is a recognized command, red when it cannot become one, and stays the normal color while it is still a valid prefix; numbers, quoted strings and `-option` flags get their own colors. A mistyped command is visible before Enter is pressed. This is a real ngspice change (new `src/frontend/syntaxhl.c`, wired into the readline prompt in `src/main.c`).

```
ngspice - interactive prompt

ngspice 1 -> tran 1n 100n uic
ngspice 2 -> ac dec 10 1 1meg
ngspice 3 -> plot v(out) v(in)
ngspice 4 -> let rout=v(out)/i(vin)
ngspice 5 -> write results.raw all
ngspice 6 -> echo "hello world" -foo 3.14
ngspice 7 -> boguscommand 1 2 3
■ command ■ unknown ■ number ■ "string" ■ -option

as you type (the word is re-colored on every keystroke)
ngspice -> plo (still typing... valid prefix)
ngspice -> plot (complete command -> green)
ngspice -> plt (cannot become a command -> red)
```

What it does

- Command word — classified exactly as the interpreter resolves it: looked up in the live command table `cp_coms` (the same `strcasecmp` walk `docommand` uses) plus the control keywords (`if`, `while`, `foreach`, `begin`, `end`, ...). So the color always agrees with whether the command will actually run. Match is case-insensitive.
- Prefix awareness — a partly typed word such as `plo` is a prefix of a real command (`plot`), so it stays neutral rather than flashing red; it turns red only once it can no longer become *any* command (`plt`).
- Other tokens — numbers yellow, "quoted strings" magenta, `-option` flags cyan; everything else (node names, vectors, operators) keeps the terminal default.

How it is wired

- The coloring engine is a pure function `cp_highlight_line()` that tokenizes a line and wraps each token in ANSI SGR color escapes.
- Live coloring overrides GNU readline's `rl_redisplay_function`. The replacement redraws the colored line only when the prompt plus the line fit on one terminal row; otherwise it defers to readline's own `rl_redisplay()`, so a line that wraps is never corrupted. The cursor is repositioned with a plain `CR + CUF` (cursor-forward) so editing mid-line behaves normally.
- Because that manual cursor positioning bypasses readline's internal position tracking, readline's own `accept-line` no longer emits the closing newline, so the Enter key is bound to a small wrapper (`cp_synhl_accept`) that emits the newline itself (only when coloring is active) before running the normal `accept-line` — otherwise the command's output would begin on the input line. With coloring off the wrapper is a no-op and readline handles the newline as usual.

- It is on by default at an interactive terminal and disabled by `set no_syntax_highlight`, by the `NO_COLOR` environment convention, or whenever the output is not a TTY — a piped or redirected session never has color codes injected into its output.

Because as-you-type coloring needs the terminal in raw mode, it requires a readline-enabled build — which the shipped binaries are (CI builds ngspice with `--with-readline=yes`, and the committed binaries link `libreadline`).

The `synhl` command

`synhl <command line>` prints the colorized form of a line non-interactively. It is useful for previewing the highlighting, works in any build (no terminal needed), and is what makes the coloring engine regression-testable in batch mode.

Verification

[examples/syntaxhl_examples/verify_syntaxhl.py](#) — 11 checks in two layers:

- engine (via `synhl` in batch): a valid command is green, an unknown command is red, a valid prefix is neutral, numbers/strings/options get their colors, and matching is case-insensitive.
- live (driven through a pseudo-terminal): typing a complete command renders green on the fly, an impossible one renders red, a valid prefix stays neutral, `NO_COLOR` suppresses coloring, and a piped (non-tty) session leaks no color codes. The live layer auto-skips on a build without readline.

It is a front-end (REPL) feature, independent of the linear solver, so it is not run under the dual-solver harness; the full example regression is unaffected.

Why this is safe

The change is confined to the interactive front end. In batch mode (`ngspice -b`) and in any non-interactive/piped session the redisplay hook never fires (the output is not a TTY), so simulation output is byte-for-byte unchanged — confirmed by the full example regression. The only new externally visible surface is the `synhl` command and the coloring at the interactive prompt.

Scope and follow-ups

The classifier colors the first word as the command; a natural extension is to re-classify after a `;` command separator, and to color known function names in expressions and defined vector names. A configurable palette (`set syntax_colors=...`) and highlighting inside `.control` blocks read from a sourced file are further possibilities.

Enhancement-170 — Semantic syntax highlighting

Enhancement-169 colored the interactive command line *lexically* — the command word green/red, plus numbers, strings and options. This extends it to *semantic* highlighting: it now checks whether the signals and expressions you type actually exist and parse, and colors accordingly, and it draws error output in red. New source in `src/frontend/syntaxhl.c`.

```
ngspice - interactive prompt (semantic highlighting)

ngspice 1 -> print v(a) v(b)                                # valid signals -> default color

ngspice 2 -> print v(a) + v(zzz)                             # unknown signal -> only v(zzz) red

ngspice 3 -> print sqrt(v(a)) - v(bad)                       # unknown signal inside a function -> red

ngspice 4 -> plot v(a)*v(b)                                  # malformed expression -> whole red

ngspice 5 -> plot v(a) vs time                               # keywords (vs) are not flagged

ngspice 6 -> print v(zzz)

Warning: v(zzz) is not available or has zero length.

■ valid signal      ■ invalid signal / expr      ■ number      ■ error output
```

What it adds

1. Invalid signals turn red. A signal reference `v(node)`, `i(source)` or `@device[param]` is looked up in the current plot (read-only, via `vec_fromplot`); if it exists it keeps the default color, if it does not it is red. So `plot v(out)` reads white once `out` is a real node with data, and `plot v(typo)` flags `v(typo)` red.
2. An invalid signal inside a valid expression reddens only that signal. `print v(a) + v(zzz)` colors just `v(zzz)` red — the rest of the expression, including the valid `v(a)`, stays default. Signals nested in functions (`sqrt(v(a)) - v(bad)`) are handled the same way.
3. A malformed expression reddens as a whole. If the argument of an expression command (`plot/print/gnuplot/asciipLOT`) fails to parse — a genuine syntax error such as `v(a)*v(b)` — the whole expression is red. The real parser (`ft_getpnames_from_string`) is used, with its output muted and its tree freed, so the check is silent and side-effect-free.
4. Error output is red. Everything ngspice writes to its error channel (`cp_err`) — `Error:/Warning:` messages — is drawn in red at an interactive terminal.

Not flagged while you are still typing

A half-typed expression is not an error, so it stays neutral: a signal whose parenthesis has not closed yet (`v(bP)`), an expression with unbalanced parens (`v(a)+v(b)`), or one ending in an operator (`v(a)+`) are all left the default color and produce no diagnostics. Only a *settled* malformed expression turns red.

This mattered in practice: the expression parser's error handler (`PPerror`) writes straight to `stderr`, bypassing the `cp_err` muting, so an early version spewed `syntax error in line segment ...` onto the prompt as you typed. The check now also mutes `fd 2` across the parse (restoring it afterwards), and skips obviously-incomplete input entirely.

Scope and safety

- Signal validity is scoped to the unambiguous forms `v(...)` / `i(...)` / `@dev[param]`. Bare words are left the default color, because a bare word could be a plot keyword (`vs`), a function (`sqrt`) or

an option — validating those would produce false reds.

- Signals are validated against the current plot, so before you run a simulation (when no vectors exist yet) signal references read as red.
- Everything is gated exactly as in E-169: on only at an interactive TTY, disabled by `set no_syntax_highlight` and by `NO_COLOR`, and never emitted into a piped/redirected session. The error-stream coloring wraps `cp_err` via `funopen/fopencookie` and points `cp_curerr` at the wrapper too, so ngspice's per-command stream reset (`cp_ioreset`) keeps it in place.
- The parser and vector lookups run on every keystroke but are read-only and do not corrupt command execution — a subsequent `let z = v(a)+v(b)` still computes correctly.

Verification

[examples/syntaxhl_examples/verify_syntaxhl.py](#) — the E-169 lexical checks plus a semantic layer (driven through a pseudo-terminal after simulating a small circuit so its signals exist):

- a valid signal `v(a)` is not red; an invalid `v(zzz)` is red;
- an invalid signal inside a valid expression reddens only that signal;
- a settled malformed expression reddens as a whole;
- a half-typed expression stays neutral with no parser-error spam;
- error/warning output is drawn in red; `NO_COLOR` suppresses it.

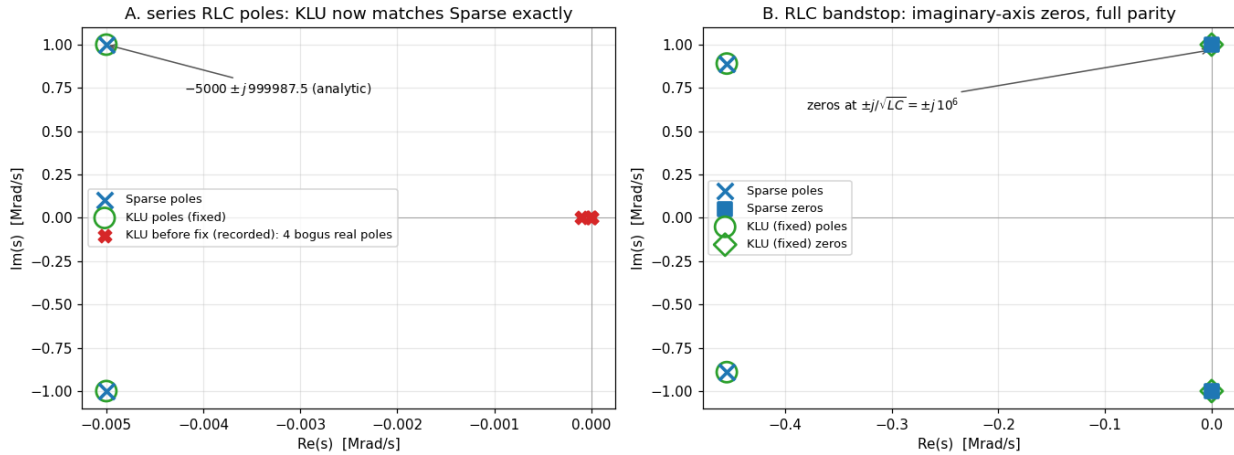
Follow-ups

Bare vector names could be validated too if functions/keywords were excluded via the interpreter's own tables; `let/define` right-hand sides could get the parse check (skipping the `name = prefix`); and the palette could be made configurable (`set syntax_colors=...`).

Enhancement-171 — KLU/Sparse solver audit: complex determinant + pivot tolerance

A deep audit of the two linear-solver stacks — the KLU↔SMP bridge (maths/KLU/klusmp.c, plus ngspice's KLU additions klu_extract.c / klu_utils.c / klu_multiply.c) and the Sparse 1.3 side it mirrors — hunting for defects the way E-37/E-38 audited the compiler. It found that pole-zero analysis under **.option klu** silently produced garbage for any circuit with complex poles or zeros, through two independent, stacked defects. Both are fixed; an obsolete "not supported" guard is removed.

KLU pole-zero: complex determinant + pivot-tolerance fixes (Enhancement-171)



The smoking gun

A series RLC with textbook poles at $-5000 \pm j999987.5$ rad/s:

	pole 1	pole 2	pole 3	pole 4
Sparse	$-5000 + j999987.5$	$-5000 - j999987.5$		
KLU (before)	-100	-10	-0.89	0
KLU (after)	$-5000 + j999987.5$	$-5000 - j999987.5$		

Real-axis poles/zeros were correct all along — which is why the regression's single-real-pole RC check always passed and the defect stayed invisible.

Defect 1 — the complex determinant formula (**spDeterminant_KLU**)

PZ's Muller iteration evaluates $\det(G + sC)$ at complex trial points via SMPcDProd. The KLU determinant was adapted from Sparse's spDeterminant, but the two solvers store pivots differently: Sparse's Diag holds reciprocal pivots (its solve multiplies), while KLU's Udiag holds the actual pivots (its solve divides). The adaptation built each pivot as the mixed quantity $(1/(U_x \cdot R_s), U_z \cdot R_s)$ and took its complex reciprocal — algebraically correct only when $U_z = 0$ (real pivot), garbage otherwise. Every complex-plane determinant evaluation was wrong, so Muller wandered blind.

The real branch had three more latent defects: its pivot loop never executed (the loop index was left at N by the preceding permutation scan, so the determinant was always ± 1.0), it divided instead of multiplying, and it never wrote *piDeterminant, leaving the caller to consume an uninitialized stack value. Both branches computed the permutation sign as $\#non\text{-}fixed\text{-}points / 2$ — wrong for any cycle longer than 2 (a 3-cycle is an *even* permutation but was counted odd).

The rewrite: $\det(A) = \text{sign}(P) \cdot \text{sign}(Q) \cdot \prod (Udiag[k] \cdot Rs[k])$ with the parity computed exactly by cycle decomposition, complex multiplication done properly, and the imaginary part always written. After the fix the KLU determinant matches Sparse to ~14 digits at every trial point.

Defect 2 — the unsanitized pivot tolerance (**SMPcReorder/SMPreorder**)

Pole-zero calls `SMPcReorder(..., PivRel = 0.0)`. Sparse's `spOrderAndFactor` sanitizes an out-of-range threshold (≤ 0 or > 1) back to its stored default; the KLU branch assigned it straight to `Common->tol`. With `tol = 0`, KLU's partial pivoting accepts a diagonal with $|\text{diag}| \geq 0 \cdot \text{max}$ — i.e. an exactly-zero pivot. At PZ's `s = 0` trial an inductor branch has a zero diagonal ($-sL = 0$), so the factorization came back `KLU_SINGULAR`, PZ recorded a spurious root at the origin, and the deflated search never expanded past $|s| \approx 10$ — yielding the four bogus "poles" above. Both `SMPcReorder` and `SMPreorder` now mirror Sparse's sanitization (an in-range `PivRel` still takes effect; DC/AC/transient pass the default $1e-3$ and are unaffected).

Guard removal

With both root causes fixed, the **E-113**-era guard in `pzan.c` — "pole-zero finite-zero computation is not supported with 'option KLU'" (added when the zero search was seen to go spuriously singular, i.e. defect 2) — is removed. The finite-zero search now works under KLU, and a genuine `E_SHORT` reports the same "input shorted" diagnostic as Sparse. The balanced/differential-output PZ guard remains: `SMPcAddCol` genuinely has no KLU implementation.

Verification

[examples/klupz_examples/verify_klupz.py](#) compares the full root set between solvers on six circuits, each anchored to its analytic answer: series RLC (conjugate pole pair), RC lowpass (the old-good real-pole case — no regression), lead network (finite real zero), RC highpass (origin zero), RLC bandstop (purely imaginary zeros $\pm j \cdot 10^6$ — the hardest case), and RLC bandpass (the finite-zero search formerly guarded off). All six: `KLU` $\equiv$ `Sparse`. The full 134-example regression is clean (the tolerance change is a no-op for every analysis that passes an in-range `PivRel`).

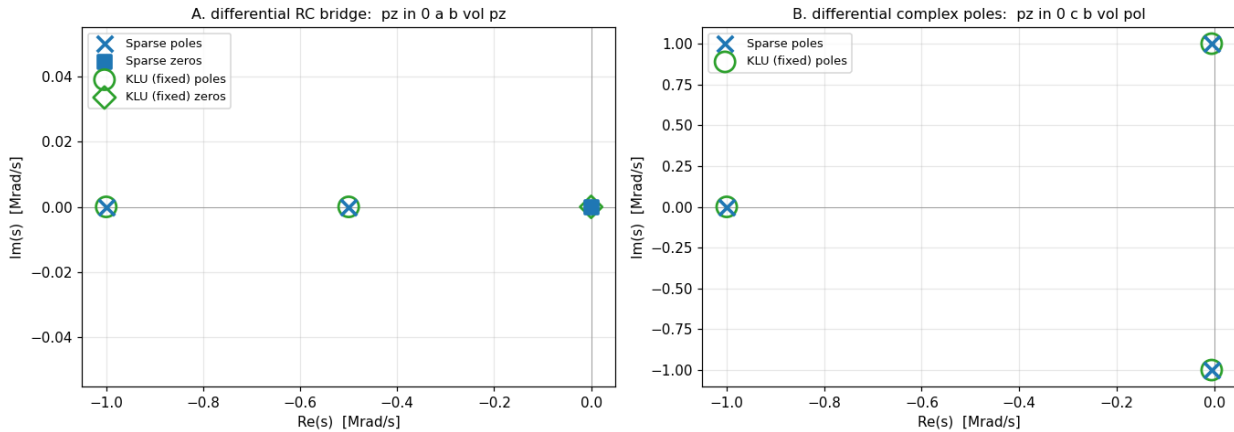
Scope

The vintage spice3 PZ driver stays numerically fragile near its noise floor *independent of solver*: on a twin-T notch KLU stalls on the deflated conjugate zero pair (finds 4 of 6 roots), while on the RLC bandpass it is Sparse that hits its iteration limit (KLU converges cleanly). That is ~1990 Muller-on- determinant fragility, not a solver defect. Also noted during the audit, left as-is: `SMPcSolve/SMPcaSolve` do not apply the (rarely-active) node-collapsing map that `SMPsolve` applies, and `SMPcReorder` reports a failed factorization with status `KLU_EMPTY_MATRIX` as success — both unreachable in normal MNA operation.

Enhancement-172 — KLU balanced pole-zero + full partial pivoting

Enhancement-171 fixed the KLU pole-zero determinant and pivot-tolerance defects, leaving two items on its scope list: the balanced/differential-output `.pz` form was still guarded off as unsupported under KLU, and a twin-T's deflated conjugate zero pair stalled. This enhancement closes both — every `.pz` form now runs under KLU with root sets identical to Sparse, and the last "not supported with 'option KLU'" analysis guard is gone.

Balanced (differential-output) pole-zero under KLU — was "not supported", now exact parity (Enhancement-172)



Part 1 — balanced (differential) output

For an output taken across a floating pair (pz in 0 a b vol), the PZ algorithm changes variables so the new unknown is the difference $V(a) - V(b)$, which requires folding the solution column into the balance column each trial: $\text{col}(b) += \text{col}(a) - \text{SMPcAddCol}$. Under Sparse this creates any missing elements on the fly; KLU's CSC sparsity pattern is fixed at conversion time, so the fold's fill-in cannot be created dynamically — which is why the form was guarded off (E-113-era).

The fix has two halves:

- Union-pattern reservation (`cktpzset.c`): before the $\text{COO} \rightarrow \text{CSC}$ conversion, every row present in the solution column is also registered in the balance column (via `SMPmakeEl` into the COO list; duplicate (`row`, `col`) entries are folded into a single CSC slot by the conversion's group labeling). The reserved entries are structural zeros until the fold fills them.
- A KLU branch for `SMPcAddCol` (`kclusmp.c`): a merge-walk over the two columns' sorted row lists, adding the complex values in place. If an addend row is missing from the accumulator column (impossible once the reservation ran) it reports an error instead of corrupting memory.

The `pzan.c` balanced-output guards are removed. Verified: a differential RC bridge (poles $-1e6$, $-5e5$; zero at 0) and a differential output with a complex pole pair — both digit-for-digit identical to Sparse (previously: Error: ... not supported with 'option KLU').

Part 2 — full partial pivoting for PZ factorizations

While validating a fully-differential configuration, a deeper numerical issue surfaced: KLU minted spurious far-field roots at $|s| \sim 1e19$ – $1e21$ (including a positive-real "pole"), where the true determinant has no roots at all. Diagnosis with an exact-rational (fraction-arithmetic) determinant of the very matrix KLU had factored showed the factorization itself going numerically rotten at extreme $|s|$: at $s = 4.52e19$ the exact determinant was $+4.36e11$ while the pivot product gave $-2.05e12$ — wrong sign and magnitude.

The root cause is architectural: KLU's ordering is fixed at `klu_analyze` time (pattern-only AMD/

BTF), while Sparse re-runs value-aware Markowitz ordering on every PZ trial. PZ sweeps $|s|$ across ~ 20 decades, so the matrix values change by ~ 12 orders of magnitude under a fixed ordering, and with the relaxed default pivot tolerance (`tol = 0.001`) the within-block partial pivoting accepted catastrophically-cancelling pivots. KLU's only value-adaptive lever is that within-block pivoting — so the E-171 sanitization fallback is upgraded from "keep the current tolerance" to full partial pivoting (`tol = 1.0`) whenever the caller passes an out-of-range `PivRel` (only pole-zero does, with `0.0`). Explicit in-range tolerances — e.g. DC/AC/transient's $1e-3$ — are honored unchanged, so nothing outside PZ is affected.

This one change eliminated the far-field junk and cured the twin-T conjugate-pair stall recorded in E-171's scope: what looked like vintage-Muller noise-floor fragility under KLU was largely this factorization inaccuracy.

Verification

[examples/klupz_examples/verify_klupz.py](#) grows from six to nine cross-solver root-set-parity checks (each anchored to its analytic answer): the E-171 six, plus

- [7] differential RC bridge (balanced output — was unsupported under KLU);
- [8] balanced output with a complex pole pair;
- [9] the twin-T notch — all 6 roots now found under KLU, conjugate $\pm j \cdot 10^6$ notch zeros included (KLU's residual real parts are $\sim 1e-12$, an order closer to the exact 0 than Sparse's $\sim 1e-11$).

The dual-solver harness's "balanced-output pole-zero is KLU-unsupported" detection is removed from `examples/_setup.py` — no analysis card remains Sparse-only. Full example regression: 134/134.

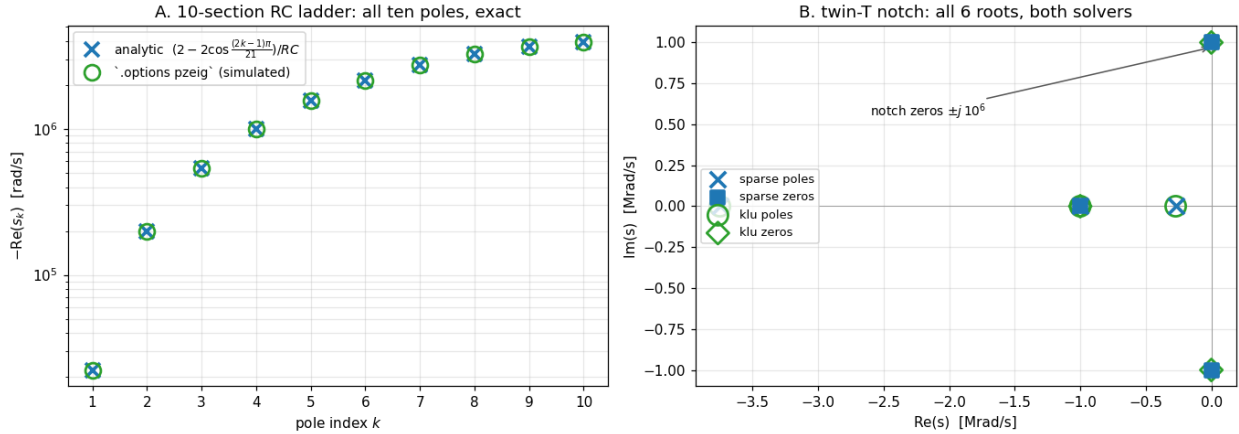
Scope

The vintage spice3 Muller driver keeps its solver-independent quirks (on the RLC bandpass it is *Sparse* that hits its iteration limit while KLU converges cleanly). With both E-171 and E-172 in place, every circuit in the battery produces root sets identical between the two solvers to float precision.

Enhancement-173 — Eigenvalue-based pole-zero (**.options pzeig**)

E-171/E-172 fixed the KLU-side defects in pole-zero analysis, but the root finder itself — a ~1990 Muller iteration on determinant values — remains fragile in both solvers: iteration limits (“giving up after 231 trials” on a plain RLC bandpass), noise-floor stalls, and chaotic search-path sensitivity. This enhancement adds a modern alternative: **.options pzeig** computes every pole and zero at once as a dense eigenvalue problem. The default remains Muller; the new method is opt-in.

Eigenvalue-based pole-zero (**.options pzeig**): shift-invert pencil + Francis QR — no Muller iteration (Enhancement-173)



The method

The existing CKTpzSetup/CKTpzLoad machinery already builds the right matrix for both phases — poles and zeros are the roots of $\det(A(s)) = 0$ for the respective PZ-configured $A(s)$, which is affine in s : $A(s) = G + sC$.

1. Extract the pencil densely. Load at $s=0$ and $s=1$; a new solver-agnostic `SMPdenseExtractReal` (KLU: walk the CSC arrays; Sparse: walk the column lists through the `Int→Ext` translation maps) gives $G = A(0)$ and $C = A(1) - A(0)$. A third load at an arbitrary point verifies affinity, so any hypothetical non-polynomial device falls back with a clear message instead of wrong roots.
2. Shift-invert linearization. C is structurally singular (rows/columns with no dynamic elements), so the pencil is not inverted directly: factor $(G + \sigma C)$ once at a non-root shift σ (tried from a small ladder of shifts, using the circuit’s own sparse solver — Sparse or KLU), then form the dense $M = (G + \sigma C)^{-1}C$ with n sparse solves. From $(G + sC)v = 0 \Leftrightarrow Mv = \mu v$ with $\mu = 1/(\sigma - s)$: every finite root is $s = \sigma - 1/\mu$, and the pencil’s infinite eigenvalues land harmlessly at $\mu = 0$ (dropped by a noise-floor threshold).
3. Dense QR. The eigenvalues of the real matrix M come from a new self-contained eigensolver, `maths/dense/eig.c` — the classical balance / Hessenberg / Francis double-shift QR chain (the EISPACK `balanc/elmhes/hqr` lineage, written fresh; no LAPACK dependency) — unit-tested standalone against companion matrices, complex pairs, a 50×50 tridiagonal with known spectrum, and a badly-scaled balance case.

The roots go into the same `PZtrial` list the Muller driver produces, so `PZpost` output, `print all`, and every downstream consumer are unchanged.

What it fixes in practice

Case	Muller	pzeig
RLC bandpass (Sparse)	iteration-limit warning, correct roots after 231 trials	same roots, no warning, no iteration

Case	Muller	pzeig
twin-T notch	6/6 under Sparse, historically stalled under KLU (pre-E-172)	6/6 under both solvers, notch zeros $\pm j \cdot 10^6$
10-section RC ladder	10/10 (many trials)	10/10 matching the analytic tridiagonal formula $s_k = -(2 - 2\cos((2k-1)\pi/21))/RC$
bandstop imaginary zeros	exact	exact ($0 \pm j \cdot 10^6$ to the printed digit)
balanced/differential output	works (E-172)	works

Plumbing

.options pzeig flows like every task option: OPT_PZEIG (optdefs.h) → cktsort.c IFparm entry + setter → TSKpzEig (tskdefs.h) → copied to CKTpzEig flag in cktdojob.c → dispatch in pzan.c (both the poles and zeros phases call CKTpzEig() instead of CKTpzFindZeros()). New files: spicelib/analysis/cktpzeig.c (the method) and maths/dense/eig.c (the eigensolver); SMPdenseExtractReal added to the KLU↔SMP bridge.

Verification

[examples/pzeig_examples/verify_pzeig.py](#) — 13 checks driving both solvers and both methods: the RLC conjugate pair, the 10-pole ladder against the analytic formula *and* against Muller root-for-root, the bandpass warning-free equivalence, the twin-T 6-root set, the bandstop's exact imaginary zeros, balanced output, the purely resistive no-roots edge case, and that the default method remains Muller. Full example regression: 136/136.

Scope

Dense $O(n^2)$ memory / $O(n^3)$ QR, capped at 2000 unknowns (ample for the small-signal blocks PZ is used on; above the cap it errors with a pointer back to Muller). Root accuracy is dense-QR class — absolute error $\sim \text{machine-eps} \times \text{dominant-root magnitude}$ — so an exact origin-zero can print as $\sim 1e-7$ when poles sit at 10^6 rad/s; Muller refines locally and can resolve wider per-root dynamic range, while pzeig never misses or invents a root. The two methods agree to ≥ 6 digits on every circuit in the battery.

Enhancement-174 — **help all** crash fix (help string used as printf format)

Running **help all** (or `help montecarlo`) in the interactive shell crashed the shipped ngspice with:

```
Error: tvprintf failed
ERROR: fatal error in ngspice, exit(-1)
```

Root cause

ngspice's help printer uses each command's help text as a **printf** format string. In `com_help.c` (and the parallel `com_ahelp.c`):

```
out_printf(ccc[i]->co_help, cp_program);  /* co_help IS the format */
```

Only one argument is ever passed (`cp_program`, the program name, meant to fill a single `%s` — as in the legitimate "Report a `%s` bug."). Any other `%` in a help string is an invalid/incomplete conversion specifier, so `tvprintf` fails and ngspice calls a fatal `exit(-1)`.

The `montecarlo` command's help (added in [Enhancement-151](#)) read:

```
... reports the yield with a Wilson 95% CI and per-spec violations ...
```

The `%` (percent-space) in `95% CI` is that invalid specifier. `help all` walks the command table alphabetically and died the moment it reached `montecarlo`; `help montecarlo` crashed directly too. The bug was present in both command tables (`spcp_coms` for ngspice, `nutcp_coms` for nutmeg).

It is not X11/terminal/solver specific — it is plain format-string handling, so it reproduced on every build. (It was invisible in headless CI because `help all` is an interactive command not exercised by the batch example decks — now it is; see below.)

Fix

Escape the literal percent as `%%` (printf convention), in both tables:

```
... a Wilson 95%% CI ...      → renders as "95% CI"
```

One character; the help line now prints the literal `95% CI` and `help all` completes.

Regression guard

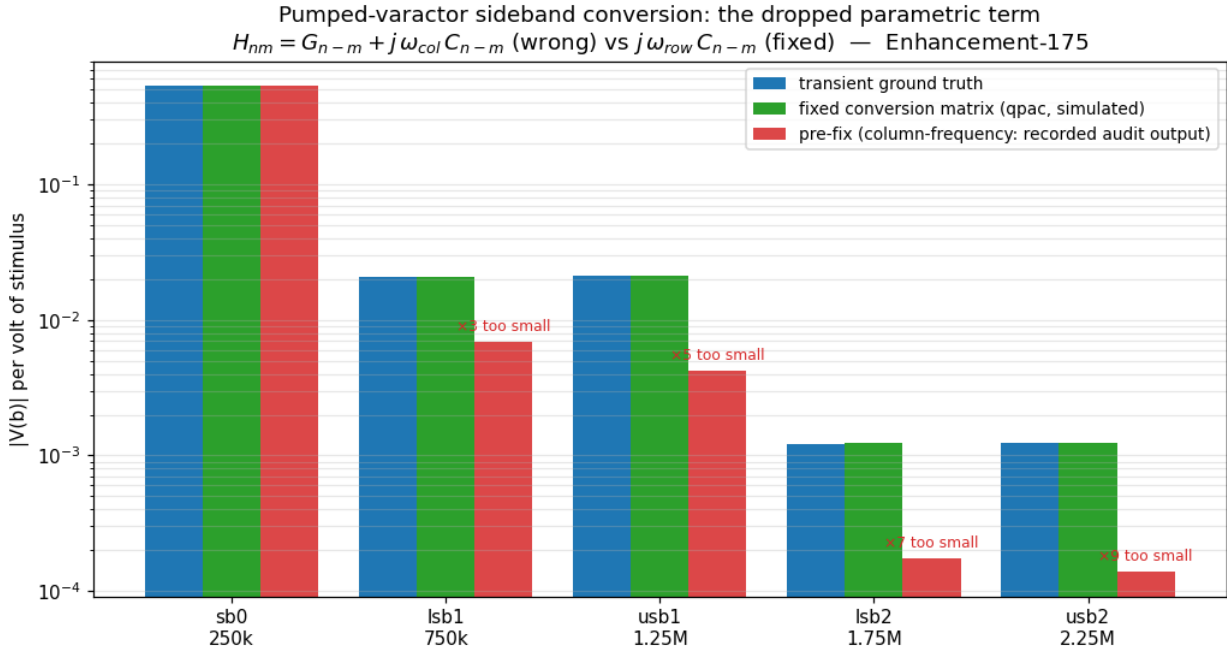
New [examples/helpcmd_examples/verify_helpcmd.py](#) with two layers:

- Runtime — drives an interactive ngspice on a pseudo-terminal and asserts `help`, `help all`, and `help montecarlo` each run to completion (no crash, no `tvprintf failed`), and that the `montecarlo` line renders a literal `95% CI`.
- Static class-guard — scans `commands.c` and fails if *any* command help string carries a format hazard: a `%` that is not `%s` or `%%`, or more than one `%s` (only one argument is passed, so a second `%s` would read a garbage pointer). This catches a future unescaped `%` even when that specific command is never exercised at runtime — the actual class of bug, not just this instance.

The static guard flags the pre-fix string and passes the fix; the runtime guard crashes on the pre-fix binary and passes on the fixed one. Full example regression: 137/137.

Enhancement-175 — RF-suite audit: the dropped parametric term in every periodic small-signal analysis

A full audit of the RF suite — .sp/Touchstone (E-63/E-64/E-72), PSS/PAC/pnoise/pxf (E-117–E-126), PSP (E-132), HB (E-134/E-135), the two-tone QPSS family (E-136–E-142) — driven by analytic anchors and cross-analysis invariants. Most of the suite came back exact; one real defect surfaced, in the mathematical heart of every periodic small-signal analysis.



The bug: column frequency in the conversion matrix

All LPTV small-signal analyses build the harmonic conversion matrix. The code scaled the reactive coupling by the column (input-sideband) frequency:

$$H_{nm} = G_{\{n-m\}} + j \cdot \omega_m \cdot C_{\{n-m\}} \quad (\text{as shipped})$$

For a small signal δv riding a frozen periodic capacitance $C(t) = \partial Q / \partial v |_{\text{orbit}}$, the exact linearized current is $\delta i = d/dt[C(t) \cdot \delta v] = \dot{C} \cdot \delta v + C \cdot \delta \dot{v}$, and the product rule collapses the two terms to the row (output-sideband) frequency:

$$I_n = j \cdot \omega_n \cdot \sum_m C_{\{n-m\}} \cdot \delta V_m \quad \square \quad H_{nm} = G_{\{n-m\}} + j \cdot \omega_n \cdot C_{\{n-m\}}$$

Using ω_m silently drops the parametric-pumping term $\dot{C} \cdot \delta v$ — the very term that makes a pumped varactor a parametric converter. Every conversion sideband through a time-varying capacitance came out scaled by exactly ω_{in}/ω_{out} ; on the audit circuit the $\pm 1/\pm 2$ sidebands were $3\times / 5\times / 7\times / 9\times$ too small (the measured pre-fix/truth ratios were 0.3333, 0.2000, 0.14286, 0.11111 — the ω -ratios to five digits, which is what confirmed the diagnosis beyond doubt).

Affected: pac, psp, pnoise, pxf, qpac, qpnoise, qpxf, and the phasenoise adjoint — for any circuit with nonlinear/pumped capacitance (varactors, junction and MOS capacitances: every real mixer and oscillator). Unaffected: LTI circuits — no off-diagonal C harmonics, and $\omega_n = \omega_m$ on the diagonal — which is exactly what every prior regression check used. The E-171 pattern again: accidental correctness in the untested region.

The subtlety: why HB was innocent (and must not be "fixed")

The harmonic-balance residual uses the same matrix builders — with the column frequency — and that is correct there: HB's reactive current is the exact chain rule $d/dt Q(v(t)) = C(v(t)) \cdot \dot{v}(t)$, whose n -th harmonic is $\sum_m C_{\{n-m\}} \cdot (j\omega_m V_m)$. This is why HB and QPSS-HB matched transient ground truth all along while the small-signal analyses did not — and it means a blanket "fix" would have broken HB.

The repair is therefore a mode flag on the two builders (`pac_build_matrix`, `qp_build_matrix`) and the two inline copies (the PAC adjoint and the phasenoise adjoint):

- `smallsig=1` (`pac/psp/pnoise/pxf/qpac/qpnoise/qpfx/phasenoise`): row frequency — restores the parametric term;
- `smallsig=0` (HB/HBOSC/QPSS-HB residual + Jacobian): chain-rule column frequency — byte-for-byte unchanged behavior.

Evidence

Ground truth is a plain transient + one-beat Fourier projection (the time-domain integrator is independent of all conversion-matrix machinery), on a $1\text{ k}\Omega \rightarrow \text{OSDI varactor } (C(v)=c_0(1+\alpha v))$ pumped at $1\text{ V}/1\text{ MHz}$ with a 1 mV probe at 250 kHz :

sideband	truth /V	fixed /V	pre-fix /V (ratio)
sb0 250k	0.53817	0.53821	0.53821 (diagonal, unaffected)
lsb1 750k	0.020755	0.020765	0.006922 (1/3)
usb1 1.25M	0.021050	0.021058	0.004212 (1/5)
lsb2 1.75M	0.0012221	0.0012284	0.0001755 (1/7)
usb2 2.25M	0.0012407	0.0012439	0.0001382 (1/9)

Regression safety was proven directly: the linear-circuit PAC control produces identical output pre/post fix.

What the audit found clean

- **.sp** + Touchstone (17 checks): series-R/shunt-C/3-port star vs analytic S-parameters to 10 digits; the asymmetric-2-port S_{21}/S_{12} column-order trap; reciprocity; RI/MA/DB, Y/Z (v_1 normalization) and GHz round-trips through `wrsnp/rdsnp`; N-port row-major 4-pairs-per-line layout.
- HB: analytic cubic — fundamental $a_1A+(3/4)a_3A^3$ and $H_3 (1/4)a_3A^3$ exact to 7 digits; even harmonics at machine zero; Sparse/KLU identical. (A wrong-looking first run turned out to be HB *correctly* solving $|v|^3$ — ngspice B-source $^{\wedge}$ is `pow(|x|, y)`; its even harmonics matched $4/3\pi$, $8/5\pi$ analytically.)
- QPSS-HB / QPSS-tran: two-tone fundamentals $a_1A+(9/4)a_3A^3$, $IM_3 (3/4)a_3A^3$, H_3 — exact (HB mode) / within transient truncation (tran mode); the commensurate-tone aliasing guard fires correctly.
- `cktsnoise.c` is a dead fossil (`#ifdef RFSPICE_`, never compiled) — noted.

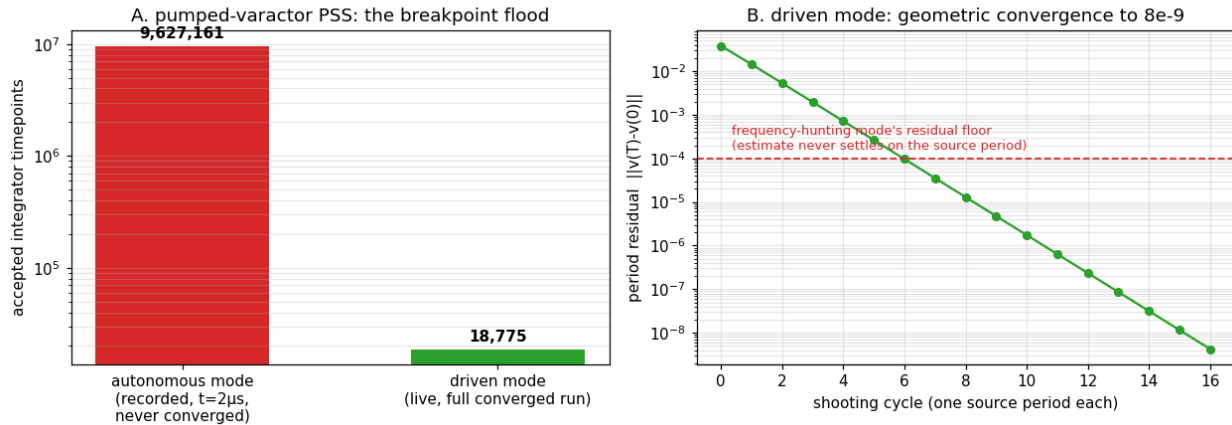
Verification

[examples/rfconv_examples/verify_rfconv.py](#) — 5 checks $\times$ both solvers: transient truth vs QPSS-HB (chain-rule path unchanged), truth vs fixed `qpac` (all five sidebands within 2%), the pre-fix ω -ratio signature is absent, and the HB / QPSS-HB analytic anchors. Full example regression: 138/138.

Enhancement-176 — Driven-mode PSS shooting: no frequency hunt, no breakpoint flood

The [E-175](#) RF audit left one flagged residual: “PSS shooting robustness — the varcap PSS ran 17+ minutes without converging where the linear control took 4; killed rather than diagnosed.” This enhancement is that diagnosis — and the fix turned out to transform the performance of the entire PSS-based RF stack.

Driven-mode PSS shooting: exact source period, no frequency hunt, no breakpoint flood (Enhancement-176)



Diagnosis

Instrumenting the shooting loop (per-cycle residual, guessed frequency, integrator statistics) showed the varactor deck accepting 9,627,161 timepoints by $t = 2\mu\text{s}$ — $\sim 0.2\text{ ps}$ per step, with essentially zero LTE rejections: the integrator was being *told* to take those steps, not failing. The source: ngspice's PSS (the Lannutti shooting) is oscillator-oriented — it hunts the fundamental frequency, and to resolve the hunt it forces mid-period breakpoints at intervals proportional to `steady_coeff`. Two structural problems on driven circuits (which is everything the periodic small-signal stack of [E-117](#)–[E-126](#) runs on):

1. The breakpoint flood. At the standard decks' `steady_coeff = 5e-6`, the forced grid spacing is $T \cdot 0.1 \cdot 5e-6 \approx 0.5\text{ ps}$ — millions of steps per shooting cycle. The *convergence-tolerance* coefficient doubles as the *sampling-grid density*, so asking for tighter convergence quadratically explodes run-time. Even at the default $1e-3$ the grid forces thousands of points per period — the reason a linear RC .pss took ~ 4 minutes.
2. The residual floor. The frequency estimate refines every cycle but can never settle *exactly* on the source frequency; while it is off, the cycle endpoint phase-drifts against the source and the period residual floors ($\sim 1e-4$ on the varactor) — shooting never converges, and the analysis burns its full `sc_iter` budget at flood-limited speed.

Fix: driven mode

Auto-detected when the circuit contains a time-varying independent source (a SIN/PULSE/... V or I source — `funcTGiven`); announced as “PSS: driven circuit detected — shooting at the fixed source period”. In driven mode:

- the period is pinned to the exact source period: no frequency estimation, no estimator grid — each shooting cycle is one plain-transient period, and the residual converges geometrically (varactor: $2e-1 \rightarrow 8e-9$ in 17 cycles, 0.3 s);
- the shooting-phase max step is clamped to **`T/psspoints`** so the orbit is integrated on the *same discretization* as the retained samples. This subtlety surfaced in validation: without the clamp, LTE lets steps grow toward $T/2$ and the shooting converges — to $7e-9$! — onto the fixed point of that *coarse* discretization, several percent off the true orbit (retained swing 0.1651 vs analytic 0).

15718). The old breakpoint flood had been masking this by accident. With the clamp the retained swing is 0.157176 — analytic to 5 digits;

- the post-FFT "relaunch at the strongest spectral line" is disabled: on a driven rectifier-like circuit whose 2nd harmonic dominates, the old logic would relaunch PSS at $2f_0$ and retain the wrong period;
- the autonomous (oscillator) path is untouched — oscillator decks produce byte-identical behavior before/after (verified on both binaries), and circuits with only DC supplies never trigger the detection.

Measured impact

analysis	before	after
varactor .pss	17+ min, never converged	0.3 s, err 8e-9, f = 1 MHz exact
rc_pss (linear RC)	~4 min, f = 999999.8976 Hz	0.05 s, f = 1000000 Hz exact
psp example	406 s in the regression	0.9 s
rfpss battery (5 verifies, 61 checks)	minutes each — regression-excluded	< 0.5 s each
rfanalyses (15 checks)	"several minutes even Sparse-only" — excluded	0.56 s
KLU PSS pass	"prohibitively slow", skipped by default	0.14 s

Harness consequence: rfpss and rfanalyses are removed from both REGRESSION_EXCLUDE and SPARSE_ONLY — the entire RF periodic small-signal suite now runs in every regression sweep under both solvers, instead of "verified once when the feature landed". This also finally made the direct .pac-on-a-pumped-varactor check affordable, closing the E-175 loop on the direct PAC path (sidebands match transient ground truth within 1%).

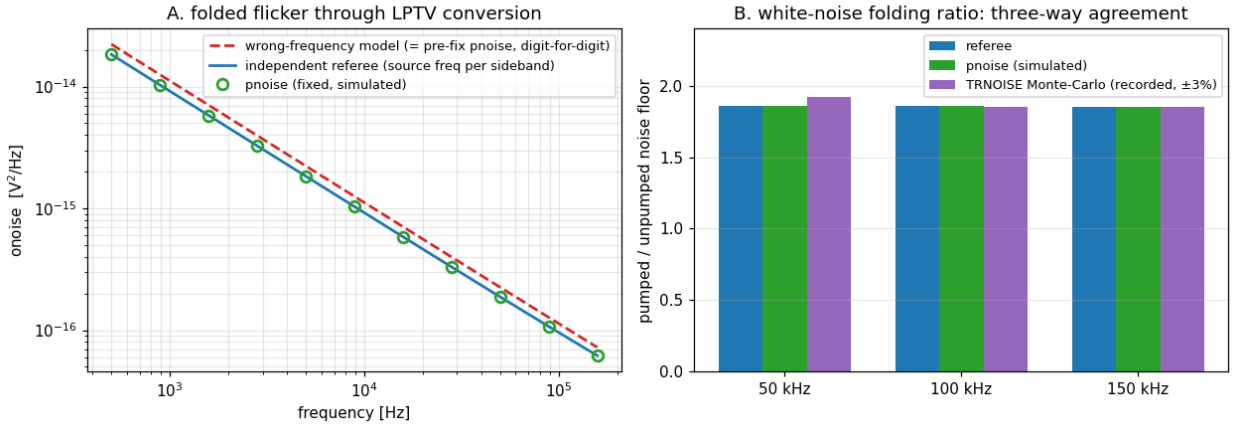
Verification

[examples/pssdriven_examples/verify_pssdriven.py](#) — 6 checks × both solvers: driven detection + retained fundamental exactly the source frequency; retained swing == |H(1MHz)| within 0.1% (step-clamp guard); retained-period self-consistency (span == T, endpoint wraps to start); direct .pac varactor sidebands vs transient truth within 1%; autonomous deck does not trigger driven mode. Plus the newly-included rfpss (61 checks), rfanalyses (15) and psp suites every sweep. Full example regression: 145/145.

Enhancement-177 — pnoise noise-folding referee + the folded-flicker frequency fix

The [E-175](#) audit's honesty ledger left one prominent gap: periodic noise analysis (pnoise/qnoise/phasenoise) was *correct by construction, not correct by measurement* — the folding of device noise through LPTV conversion had never been checked against anything independent. This enhancement builds that referee. It certified most of the machinery — and caught a real bug in the rest.

pnoise noise-folding referee: folded sidebands evaluated at their SOURCE frequency (Enhancement-177)



The referee

Three independent layers, on a 1-node LPTV-conductance circuit ($G(t) = g_0 + g_1 \cdot \sin(\omega_0 t)$) pumping node b, fed by a noisy 10 k Ω — chosen over a varactor because conversion transfers are $O(1)$, making folding errors dominant):

1. A from-scratch Python conversion matrix: $Y_{nm} = \delta_{nm}(1/R + j\omega_n C + g_0) + G_{\{n-m\}}$, $\text{onoise}(f) = \sum_k |Z_k(f)|^2 \cdot S(|f+k \cdot f_0|)$ — the same physics with zero shared code.
2. A TRNOISE transient Monte-Carlo (no shared code *or theory*): the same noise injected as a time-domain source, 198-segment Welch PSD, with the pumped/unpumped ratio cancelling the TRNOISE amplitude convention.
3. The LTI ($\text{pnoise} \equiv \text{.noise}$) and white limits.

Verdict on the white path: measured-correct. pnoise matched the referee to 6 digits at every frequency, and the MC arbiter confirmed the $\sim 1.86\times$ folding ratio within statistical error. The adjoint, the sideband sum, and the thermal densities are right.

The bug the referee caught

The stationary sideband loops set the noise-evaluation frequency once, to the output frequency (`data.freq = freq` outside the `k` loop). But the sideband-`k` adjoint carries noise that originates at $|f + k \cdot f_0|$ — a frequency-dependent source PSD (flicker $1/f$, [noise_table](#)) must be evaluated at the *source-side* frequency. The proof was digit-exact in both directions:

- pre-fix pnoise $\equiv$ the referee's deliberately-wrong "evaluate-at- f " model, digit-for-digit ($1.114323\text{e-}14 \equiv 1.114323\text{e-}14$ at 1 kHz — a 21% overestimate on this circuit, unbounded as $f \rightarrow f_0$);
- post-fix pnoise $\equiv$ the correct model, digit-for-digit ($9.175472\text{e-}15$).

Why three generations of checks missed it: white noise is frequency-flat, and LTI circuits have no $k \neq 0$ transfer — the third occurrence of the accidental-correctness pattern ([E-171](#) determinant, [E-175](#) parametric term). Audits must probe the untested region.

Fixed in all three stationary loops (dcpss.c): pnoise (data.freq = |freq + k·f0| per sideband), qpnoise (|f_in + k1·f1 + k2·f2| per harmonic), and phasenoise — where it matters most: folded far-sideband blocks near a carrier were evaluated at the *tiny offset frequency*, inflating the flicker (1/f³) region.

Cyclostationary scope

The cyclo mode's time-domain identity ($\text{onoise} = (1/P) \sum_s S(t_s) \cdot |A_s|^2$) treats the source PSD as frequency-flat across the folding span — exact for modulated-white noise and for flicker on conversion-free circuits (the shipped E-125/126 use cases), but not for flicker folded through $k \neq 0$ conversion, which cannot be collapsed into that product with the aggregate device-noise API. Documented at both cyclo sites; the stationary mode (now exact) is the right tool when folded flicker matters.

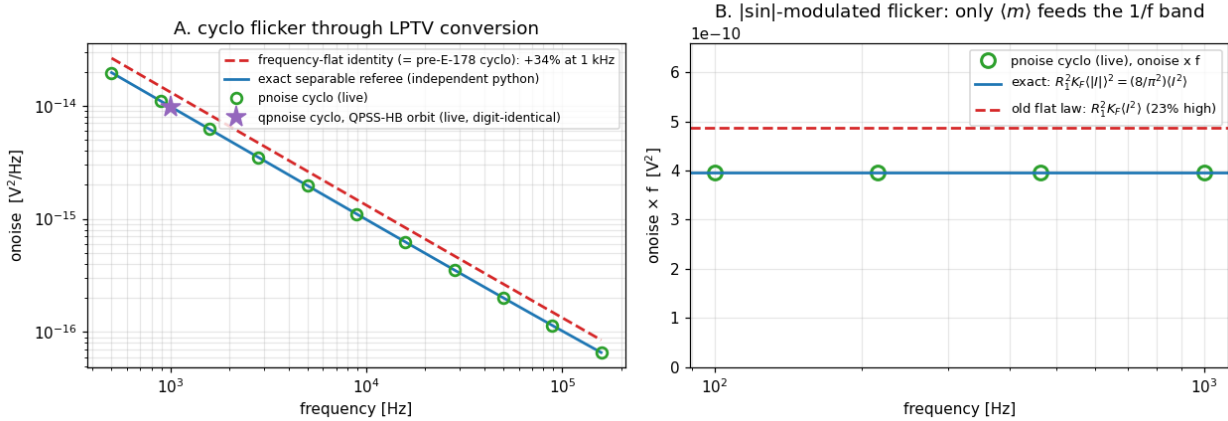
Verification

[examples/pnoisefold_examples/verify_pnoisefold.py](#) — 5 checks × both solvers (runs in ~2 s thanks to E-176): white folding vs referee ($\leq 0.1\%$), flicker folding vs referee ($\leq 0.5\%$, sample-0 bias read from the retained orbit), the pre-fix signature absent, pump/no-pump ratio $\equiv$ referee ratio, LTI limit $\equiv$.noise. The rfpss/qpss/rccyclo suites pass unchanged (their circuits are conversion-free or white — provably unaffected). Full example regression: 146/146.

Enhancement-178 — exact separable cyclostationary folding + the HB DC-source fix

E-177 fixed folded flicker in the *stationary* periodic-noise loops and left the *cyclostationary* mode documented as frequency-flat-limited: its time-domain identity $\text{onoise} = (1/P) \sum_s S(t_s) |A_s|^2$ cannot see that noise folded from sideband q originates at $|f + q \cdot f_0|$. This enhancement removes that limitation exactly — and its cross-machinery check caught a second, unrelated bug that had silently doubled every DC bias voltage inside both harmonic-balance solvers.

Exact separable cyclostationary folding: colored noise folded from its SOURCE frequency, per generator (Enhancement-178)



The exact separable model

For the physical cyclostationary model $S(t, f) = m(t)^2 \cdot g(f)$ — a colored stationary process amplitude-modulated along the orbit — the exact output noise is

$$\begin{aligned} \text{onoise}(f) &= \sum_g \sum_q |B_q(g)|^2 \cdot g_g(|f + q \cdot f_0|) \\ B_q(g) &= (1/P) \sum_s m_g(t_s) \cdot dA_s(g) \cdot e^{+jq \cdot \theta_s}, \quad A_s(j) = \sum_k \Psi_k(j) e^{-jk \cdot \theta_s} \end{aligned}$$

The stationary limit (m const) reduces to the E-177 sum; the flat limit (g const) collapses by Parseval to the old identity — this path strictly generalizes both.

Per-generator amplitudes with no device-API change. The noise-summary machinery (`pntSummary/outpVector`) yields one density per generator, and load polarization against a fixed reference load $R = \Psi_0$ (five DEVnoise sweeps per orbit sample: $A \pm R, A \pm jR, R$) extracts $\hat{c}_{g,s} = \sqrt{S_g(t_s)} \cdot dA_s(g)$ up to a constant phase that cancels in $|B_q|^2$. The spectral shape g_g is measured pointwise at the folded frequencies (no $1/f^{\text{EF}}$ assumption — `noise_table` works), at several probe biases so a generator quiet at one phase still gets its shape from an active one. Family-total summary slots are excluded by the name-prefix rule. Generators whose measured shape is flat keep the original identity — exact for *any* quadratic form, including NevalSrc2-correlated pairs; a colored generator invisible to the reference load falls back to it rather than being dropped. Ported to the two-tone `qpnoise ... cyclo` as the 2-D analog $B_{\{q1,q2\}}$ at $|f + q1 \cdot f1 + q2 \cdot f2|$.

The physics it fixes: flicker sees $\langle m \rangle^2$, white sees $\langle m^2 \rangle$

For modulation far above the analysis band, only the DC component of the envelope $m(t)$ feeds the $1/f$ band — the AC components are shifted to $\pm f_0, \pm 2f_0$ sidebands where $1/f$ is negligible. The old identity collapsed *all* envelope power onto the analysis frequency: for $|\sin|$ -modulated flicker it overestimated by $\pi^2/8$ (23%, the `rcyclo` circuit — its verify now pins the exact $\langle |I|^2 \rangle$ law to $1e-4$), and on the E-177 conversion circuit by 34%. The new cyclo sweep lands on a from-scratch Python referee to $\leq 3e-4$, and the pre-E-178 binary reproduced the referee's deliberately-flat model to $6e-5$ — the error was the model, not the code.

The bug the cross-machinery check caught

Check [6] runs the same circuit through the QPSS-HB orbit (`qpss ... hb + qpnoise ... cyclo`) and expects the 1-D answer. It came out exactly 4×. A differential experiment ($AF = 0/1/2 \rightarrow$ ratio 1/2/4, white exact) pinned a doubled bias current, and a trivial DC deck ($VDC=1 \rightarrow$ harmonic table shows 2.000000) proved it: both HB Newton residuals (`hb`, E-134; `qpss ... hb`, E-136) fold the settle-mode rhs — which already carries the DC source values — into `I_R`, then subtract $\lambda \cdot I_s$ which carries them again. The converged spectrum answered a doubled DC drive: every DC bias voltage came out exactly 2×, silently corrupting all bias-dependent noise and conversion downstream (flicker $\propto I^{AF}$ off by 2^{AF} , diode shot by $2 \times \dots$), while AC content and bias-independent noise were untouched — which is why every earlier AC-driven HB validation passed (the fourth occurrence of the accidental-correctness pattern: E-171, E-175, E-177). The fix adds the unscaled DC source block back to the residual, making the net DC drive exactly $-\lambda \cdot I_{s_DC}$. With it, `qpnoise ... cyclo` agrees with `pnoise ... cyclo` digit-for-digit across two entirely different orbit machineries (QPSS-HB vs PSS shooting).

Follow-up hardening (same enhancement, post-release)

A doc-review sweep re-measured the E-139 hard-pumped-diode circuit and found the `cyclo` result exploding ($\sim 6.5e+1$ V²/Hz): `dionoise.c` computes its three *sidewall* generators only when `DIOresistSWGiven`, but the `prtSummary` block copies all `DIONSRCs` slots — without a sidewall model the summary vector carried uninitialized stack garbage. Latent in stock ngspice for decades (the aggregate total only sums written entries); E-178 is the first *consumer* of per-generator densities. Fixed at the device (unwritten slots zeroed) and hardened in both `cyclo` consumers (densities must be finite, non-negative PSDs; garbage probe ratios fall back to the flat path). The old `verify_qpnoise` “`cyclo > 2× stationary`” diode expectation was itself an artifact of the garbage slots and the doubled HB DC bias — the circuit is purely resistive, so the check now compares against a closed-form torus-average referee (per-sample Newton + instantaneous adjoints), which the fixed `cyclo` matches to ~1%: the true value is dominated by R_n ’s own thermal noise, because when the junction conducts hard its shot PSD rises but its transfer to the output collapses.

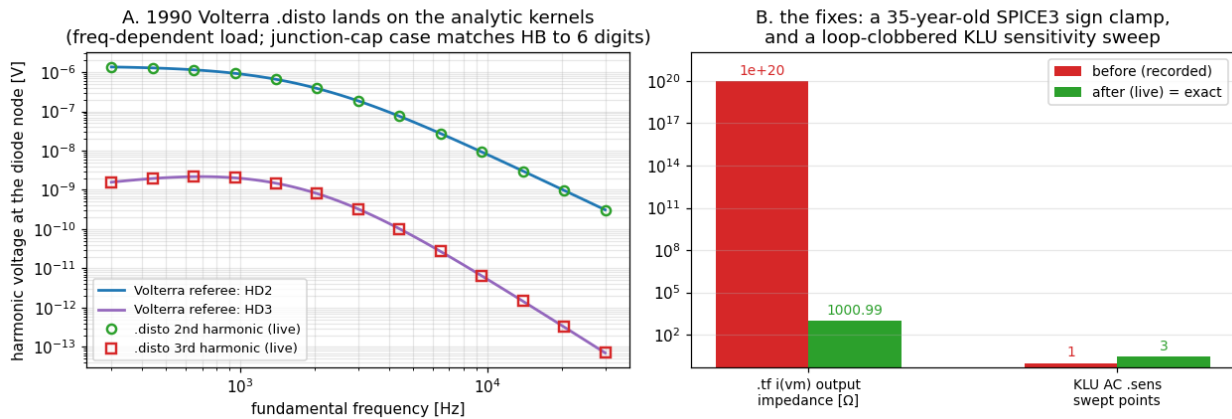
Verification

[examples/cyclofold_examples/verify_cyclofold.py](#) — 6 checks × both solvers: exact-separable referee ($\leq 0.5\%$), old flat-model signature absent (34% here), white pumped `cyclo` $\equiv$ stationary (Parseval, digit-exact), the $(8/\pi^2) I^2$ law ($\leq 2\%$), LTI limit $\equiv$ `.noise`, and the two-tone collapse ($\leq 0.5\%$, digit-identical in practice). `rfpss` (`rccyclo` updated to the exact law), `qpss`, and `pnoisefold` suites pass; full example regression: 147/147 (re-run after the follow-up hardening).

Enhancement-179 — standard-analyses audit: referee battery + three fixes

The gap-analysis doc marks the "Standard analyses (analog)" table all on-par with the commercial reference — but `.tf`, `.sens`, `.disto`, and the `.meas` evaluators are 1990s SPICE3 code whose *values* had never been checked against independent physics. This audit builds the referees (the [E-171/175/177/178](#) pattern: probe the region no prior test could see), certifies most of the table, and fixes the three defects the referees caught — one of them 35 years old.

Standard-analyses audit: referee the 1990s code, fix what the referees catch (Enhancement-179)



The fixes

1. `.tf` current-output impedance — a Berkeley SPICE3 bug (`tfanal.c`). For `.tf i(vm) vin` the output impedance solve forces 1V on the sense branch and inverts the resulting branch current — but clamps it as $1/\text{MAX}(1e-20, \text{rhs})$. The branch current is *negative* for every passive network (the input-impedance path immediately above divides by $-\text{rhs}$), so the clamp always won: every current-output `.tf` reported output impedance $1e20$, since SPICE3 (1988). Fixed with the sign and `fabs` guard of the input path; the feedback-amp referee now gets RL + node-Thevenin to all printed digits (1000.990 vs hand-computed 1000.990001).
2. KLU AC sensitivity silently truncated to one point (`cktsens.c`). The KLU complex-conversion block inside the AC frequency loop reused `i` — the outer loop variable — for its `DEVmaxnum` device walk, so after the first frequency `i` $\approx \text{DEVmaxnum}$ terminated the sweep. One correct row, no error message. This was in the [E-114](#) KLU-enablement code, and E-62's single-point AC-sens check is exactly why it survived: the surviving point *was* correct. Fixed (local index; the dead second block hardened too); the full sweep now matches the analytic $dV/dC = -j\omega R / (1 + j\omega RC)^2$ to 6 digits under both solvers.
3. `.meas DERIV{ACTIVE}` implemented (`com_measure2.c`). The measure parser has accepted `DERIV` since the SPICE3-era port, but evaluation fell into an explicit "*currently not supported*" stub (with an empty `#if 0 measure_deriv()` marked "still some more work to do..."). Implemented: a 3-point Lagrange-quadratic derivative on the nonuniform time grid (exact for the parabolic local model, robust to breakpoint-duplicated timepoints), in both `AT=val` and `WHEN expr` forms, sharing `FIND`'s parse/locate flow. Verified against the analytic sine slope to 5 digits (10882.7 vs 10882.796 at the $v=1$ rising crossing) and ≈ 0 at the crest. The `DERIVATIVE` and `INTEGRAL` long spellings from the manual are now accepted (only `DERIV/INTEG` matched before), and a pre-existing `-Wmissing-prototypes` warning in the same file is silenced.

Measured-correct

- **.disto** — the 1990 Volterra kernels hold up impressively. HD2/HD3 match an analytic diode-kernel referee to $\leq 0.06\%$ including a frequency-dependent load (the harmonic responses cor-

rectly use $Z(2\omega)$, $Z(3\omega)$ — the analog of the E-177 folding-frequency bug is *not* present); the two-tone SIM2 path (f_1+f_2 , f_1-f_2 , $2f_1-f_2$) matches including third-order cascade terms; DISTOF1/DISTOF2 amplitude scaling is exactly quadratic/cubic/bilinear; and nonlinear junction-capacitance harmonics agree with the E-134 Harmonic Balance engine to ~6 digits — two fully independent engines, 36 years apart.

- **.noise** integrals — $\text{onoise_total}^2 \equiv$ the band-limited analytic ($\rightarrow$ kT/C wide-band) to 0.06% at 20 pts/dec, and the flicker log-integral $KF \cdot I^2 \cdot \ln(f_2/f_1) \cdot Zt^2$ to 6 digits (Nintegrate's power-law rule is exact on $1/f$). The diode's per-generator summary slots are clean zeros after the E-178 sidewall fix.
- **.sens** — DC sensitivities at a *nonlinear* operating point (dv/dR_s , dv/dI_S — model parameters included) match central finite differences to 5–6 digits.
- **.pz** at a nonlinear OP — works; the E-62 "nonlinear pz quirk" was the input *convention*: a bias source sitting on the injection node shorts it, and ngspice's "input shorted on the way to the output" refusal is correct. The driving-point form (`.pz a 0 a 0 cur pol`) returns the linearized pole $-(1/R_s + g_d)/C$ to 0.02% under both solvers.
- **.op/.dc/.tran/.meas** — hard-DC homotopy lands both solvers on the identical operating point; trap and Gear both track the analytic RC decay to $\leq 4e-7$ at measure points; the **.meas** battery (RMS/AVG/PP/WHEN/INTEG) is exact to $\sim 1e-6$.

Verification

[examples/stdaudit_examples/verify_stdaudit.py](#) — 8 checks $\times$ both solvers: disto vs Volterra (frequency-dependent), SIM2 + scaling, disto $\equiv$ HB, noise integrals, **.tf** exact incl. the current-output fix (pre-fix 1e20 signature absent), **.sens** DC-vs-FD + the complete 3-point AC sweep (the KLU regression guard), nonlinear-OP pz, and the **.meas** battery incl. DERIV/INTEGRAL. Full example regression: 148/148.

Enhancement-180 — checkpoint/restart under KLU (the last Sparse-only feature falls)

Since E-131, `savestate/loadstate` rejected `.option klu` with a guard, and the solver notes recorded the reason as *"KLU's symbolic/numeric factorization objects are absent on the restore path."* A review question — *why* doesn't it work? — prompted the actual diagnosis: that explanation was wrong, the crash it rationalized was an option-ordering bug in the checkpoint flow itself, and fixing it makes checkpoint/restart work under both solvers — and even across them.

The real mechanism

`.option klu` lives in the *task* (`TSKkluMODE`) and is copied into the circuit only inside `CKTdoJob`, at analysis dispatch. But `loadstate` calls `CKTsetup` *directly* — before any dispatch — so the matrix was built with `ckt->CKTkluMODE` still 0: a Sparse matrix, `SMPkluMatrix = NULL`, regardless of the deck's option. `loadstate` then drives the continuation through `if_run(ckt, "resume", ...)` → `CKTdoJob`, which *now* copies `TSKkluMODE = 1` — but resume deliberately skips re-setup to preserve the restored integration state, so the circuit claims KLU over a Sparse matrix. The first `NIiter` executes `ckt->CKTmatrix->SMPkluMatrix->KLUloadDiagGmin = 1` and dereferences `NULL` (reproduced: `EXC_BAD_ACCESS` at offset `0x90` in `NIiter`, with the guards removed). `savestate` was never broken at all — its guard was over-broad, since the checkpoint file contains only solver-agnostic state (solution vector, device state history, time/step/order, breakpoints).

The fix

`com_loadstate` copies the task's solver selection (and the E-152 KLU knobs — ordering, scaling, BTF, `memgrow`) into the circuit *before* calling `CKTsetup`, exactly the block `CKTdoJob` runs. `CKTsetup` then builds the KLU matrix, converts `COO`→`CSC`, and binds the devices as in any normal run; the later dispatch copy becomes a no-op. Both guards are removed.

Cross-solver restore

Because the checkpoint file is solver-agnostic, the fix yields more than parity: a state saved under one solver restores under the other. All four `save`×`load` combinations — in fresh processes — continue exactly on the uninterrupted run's trajectory (diode + RC transient, compared at $t = 3.5$ ms of a 4 ms run, $\leq 1e-9$ relative):

save under	load under	continuation
Sparse	Sparse	≡ uninterrupted
Sparse	KLU	≡ uninterrupted
KLU	Sparse	≡ uninterrupted
KLU	KLU	≡ uninterrupted

With this, nothing in the simulator remains Sparse-only: E-172 closed the last analysis (balanced-output pole-zero), E-180 the last feature.

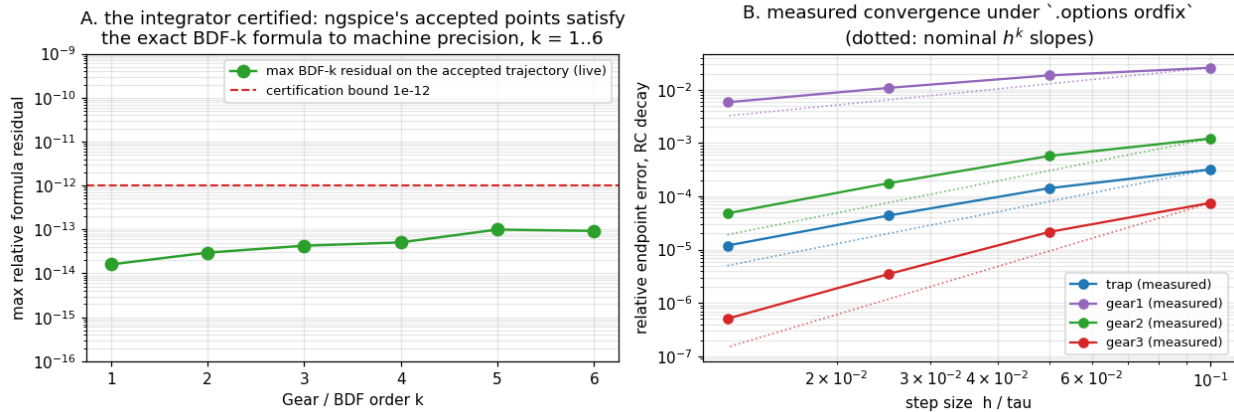
Verification

[examples/checkpoint_examples/verify_checkpoint.py](#) — the E-131 suite's "KLU rejected" robustness check is replaced by the full solver matrix (4 combos, fresh-process restores, ≡ uninterrupted reference), alongside the existing scenarios (RC step/pulse breakpoints, built-in and OSDI diode, same-session `save/load`, mismatched-circuit rejection): 22/22. Solver-notes and gap-analysis tables updated (the $\triangle$ row was this feature). Full example regression: 148/148.

Enhancement-181 — core-numerics audit: the integrator certified exactly, plus `.options ordfix`

The gap-analysis “Core numerics” table has carried ✓ marks for the integration methods since the beginning — but nobody had ever verified that ngspice’s Gear (BDF) corrector coefficients actually deliver their nominal order. Orders 3–6 were dead code for ~30 years until E-128’s dynorder selected them, and E-128 verified order *selection*, not coefficient *correctness*. This audit closes that: the coefficients are now certified exactly, a new verification instrument ships, and the rest of the table (solver precision, convergence aids, the ancient `xmu/lvltim` paths) is referee-verified with no defects found.

Core-numerics audit: certify the 30-years-dormant Gear orders exactly (Enhancement-181)



Why this needed an instrument

Three successive measurement traps — each initially looking like a bug — turned out to be real numerics, and are now documented:

1. Loose tolerances invert the order preference. The LTE-limited step is $h_k = (c_k \cdot \text{tol})^{\{1/(k+1)\}}$: for effective tolerance ≈ 1 it *decreases* with order, so the E-128 controller's refusal to climb under a pinned step with loose `reltol/trtol` is correct-by-theory, not a freeze.
2. The order-1 first step proportional to the pinned step imposes an $O(h^2)$ startup error floor that masks every order above 2 — the classic BDF-starter problem.
3. Step-ratio stability: ramping $\times 2$ at order 6 visibly seeds divergence (variable-step BDF instability at large ratios), and a lossless LC sits outside $\text{BDF} \geq 4$'s $A(\alpha)$ wedges — the observed gear6 blow-ups during the audit were *textbook-correct* behavior of correct coefficients.

`.options ordfix=K` is the shipped instrument: it pins the Gear order at K for verification runs — every converged step is accepted (the step is ruled by `tmax`, not the LTE), the first step is shrunk 1000 $\times$, the order ramps to K while the step is still tiny, and the step then grows gently ($\times 1.15$) to the pin. Off by default; useless (deliberately) for production runs.

The certification

With `ordfix`, the referee is airtight and startup-independent: dump the accepted trajectory at full precision and check, at every stencil of $k+1$ uniformly spaced points, that ngspice's values satisfy the exact BDF- k formula — Lagrange differentiation on the actual nodes — for the circuit ODE. Result: max residual $\leq 1.3e-13$ at every order 1–6 (~90 stencils each). NIcomCof's coefficients are exactly right at all orders, variable-step machinery included. Measured global convergence confirms orders 1–3 asymptotically (trap 1.87, gear1 0.90, gear2 1.87, gear3 2.78 at the finest pair), and the trap-vs-Gear dissipation dichotomy on a lossless LC matches theory: trap preserves the amplitude ($|R(iy)| = 1$) while Gear damps it, less at higher order.

The rest of the table — no defects

- Linear-solve precision (both solvers): errors track conditioning theory ($\kappa \cdot \varepsilon$ on a $1e6$ -spread ladder); a $1e18$ -conductance-spread asymmetric mesh with a VCCS solves to $1e-15$ against an exact-rational MNA referee; $KLU \approx \text{Sparse}$ throughout.
- DC convergence aids: on a 40-diode hard-DC chain, `default` / `noopiter` / `gminsteps=0` / `srcsteps=0` all land on the same operating point ($\text{spread} \leq 2.2e-6$ — tolerance-level), and $KLU \equiv \text{Sparse}$ to $7e-13$ — the two solvers' separate `gmin`-diagonal-loading paths in `NIiter` agree.
- `xmu`: 0.5 is bit-identical to plain trap; 0.45 damps the lossless ring as documented.
- `lvltim=1` (iteration-count timestep control, rarely exercised): works, agrees with the default LTE control.

A pre-existing `-Wmissing-prototypes` warning (`DCtran_step_quit`, declared extern locally in `runcoms2.c`) is silenced with a proper header prototype.

Verification

[examples/corenum_examples/verify_corenum.py](#) — 8 checks $\times$ both solvers: the BDF-residual certification (orders 1–6, $\leq 1e-12$), measured convergence slopes, the LC dissipation conformance, the hard-DC aid matrix, the exact-rational precision mesh, `xmu`, `lvltim=1`, and the `ordfix` order report. The E-128 dynorder suite passes unchanged. Full example regression: 149/149.

Enhancement-182 — pyplot: autoscale by default, pin axes only for explicit user limits

A user-experience refinement of the [E-94/95/98/99](#) pyplot matplotlib bridge: the generated script used to pin every axis with `set_xlim/set_ylim` taken from ngspice's internal plot machinery — grid-rounded ranges computed for ngspice's own renderer. Matplotlib frames data better on its own: the generated script now relies on matplotlib's autoscaling plus **`fig.tight_layout()`**, and emits `set_xlim/set_ylim` only when the user explicitly asked for limits (`xl/xlimit`, `yl/ylimit` on the command).

The change

`plotit()` always fills its `xlims[]/ylims[]` arrays — from the user's `xlimit/ylim` arguments when given, else from the data — so the back-end could not previously distinguish the two cases. The user-given case is exactly when the static `xlim/ylim` pointers (from `getlims`) are non-NULL; those statics are freed *before* the device dispatch, so `plotit` now captures `user_xlim/user_ylim` flags at that point and passes NULL limits to `ft_pyplot` unless the user provided them. `ft_pyplot` already handled NULL limits; the `gnuplot` and `asciiplot` back-ends are untouched.

```
pyplot rc v(out) v(in)                                -> autoscale + tight_layout
pyplot rc v(out) v(in) xlimit 0 1m ylimit -1 1         -> set_xlim(0, 1e-3), set_ylim(-1, 1)
```

Verification

[examples/pyplot_examples/verify_pyplot.py](#) gains two checks (9 total × both solvers): the auto plot's script contains no `set_xlim/set_ylim` and keeps `tight_layout`; an explicit `xlimit 0 1m ylimit -1 1` command still emits both pins with the exact values and renders a valid PNG. The panel (E-98) and export (E-99) suites pass unchanged. Full example regression: 149/149.

Enhancement-183 — pyplot usability: distinct default names, deck-folder output, linewidth & backend

A round of pyplot (the matplotlib plotting command, [E-94/95/98/99/182](#)) usability improvements driven by real use. Four independent changes, all in the pyplot front-end (`com_pyplot.c`, `plotting/pyplot.c`); none affect any other command.

1. Distinct default names for successive no-name plots (a real bug)

In *window(interactive)* mode pyplot launches the Python viewer in the background (`python3 pyplot.py &`) so it doesn't block ngspice. Two plots that both omit the file name shared the same `pyplot.py` / `pyplot.data`: the second call overwrote those files *before* the first viewer had read them, so both windows ended up showing the second plot — its data and its title. (In PNG mode this was invisible, because that path runs Python synchronously.)

Fix: successive omitted-name plots now get distinct base names — the first stays `pyplot` (unchanged), later ones become `pyplot-2`, `pyplot-3`, ... via a per-session counter — so each viewer reads its own files. This was diagnosed from a user's 3-inverter deck where two `pyplot ... title 'no. 3' / title 'no. 4'` windows both showed "no. 4".

2. Artifacts written next to the circuit file

pyplot wrote `<name>.py` / `<name>.data` / `<name>.png` into ngspice's *current working directory* — so running `ngspice -b /path/to/deck.cir` from elsewhere scattered the plot files into the cwd rather than the deck's folder. Now, when the base name is bare (no directory of its own), pyplot prepends the directory of the circuit file (`ft_curckt->ci_filename` via `ngdirname`), and the generated script uses that full path in both `np.loadtxt(...)` and `savefig(...)` so Python finds the data from any cwd. A deck given as a bare relative name (its directory is `.`) still lands in the cwd, exactly as before — the change is opt-in by how you invoke ngspice, with no behavior change for the common in-folder case.

3. `set pyplot_linewidth=<w>`

Sets the matplotlib line width (in points) for every trace, threaded into the `plot(...)/step(...)` calls. Unset or ≤ 0 leaves matplotlib's default. Live setting, changeable between plots.

4. `set pyplot_backend=<name>`

Selects the matplotlib backend explicitly (e.g. `TkAgg`, `QtAgg`, `MacOSX`, `WebAgg`, `Agg`) by emitting `matplotlib.use('<name>')` ahead of `import matplotlib.pyplot`. When set it takes precedence over the automatic backend, including the `Agg` otherwise forced by the `png/svg/pdf` terminals — so a user pairing an interactive backend with a file terminal on a headless host does so deliberately. Unset: unchanged behavior (file terminals use `Agg`, interactive uses matplotlib's default, no `use()` call emitted).

Verification

[examples/pyplot_examples/verify_pyplot.py](#) grows to 13 checks $\times$ both solvers, adding: two no-name plots produce distinct `pyplot/pyplot-2` scripts with their own subtitles; an absolute-path deck run from a different cwd lands its artifacts next to the `.cir` (and the `.py`'s `loadtxt` carries the deck-dir path); `set pyplot_linewidth=3.5` emits `linewidth=3.5`; `set pyplot_backend=Agg` emits `matplotlib.use('agg')`. Full example regression: 149/149. Packaged as a standalone feature bundle under [features/](#).

Enhancement-184 — sweep progress bar reaches 100%

A follow-up fix to the [Enhancement-129](#) sweep progress bar: the bar sometimes stopped short of 100% at the end of a run, e.g.

```
Reference value : 1.32920e+03  [===== ] 82%  
No. of Data Rows : 101
```

The cause

The "Reference value" status line is throttled — it is redrawn in place only when more than 0.25 s has elapsed since the last update (so a fast inner loop does not spend all its time printing). The final sweep point almost always lands *within* 0.25 s of the previous tick, so its print is skipped: the bar freezes at whatever the last throttled update showed (here 82%), and the run then prints "No. of Data Rows" on the next line. The bar reaching 100% was therefore never reliable — it only happened when the last point's timing happened to fall past a 0.25 s boundary.

The fix

`outp_print_reference()` now records the latest reference value and a "a bar was shown this run" flag on every call. A new `outp_finish_reference()` is invoked from the two end-of-run sites (`fileEnd`, `plotEnd`) — immediately before the "No. of Data Rows" line — and, if a bar was shown, reprints it full at 100%, in place, using the sweep's *true* endpoint:

- transient → CKTfinalTime;
- AC → ACstopFreq; noise → NstopFreq;
- DC → the last swept source value.

So the closing frame is always, e.g.

```
Reference value : 1.00000e+06  [=====] 100%  
No. of Data Rows : 101
```

It is a one-shot per run (reset at analysis start and after firing), stays silent for analyses that never showed a bar (operating point, transfer function, ...), and honors the same `ft_optimizing/ft_norefprint/cp_background` guards as the throttled line — so quiet/background/optimizer runs are unaffected, and nothing is written to the rawfile.

Verification

`examples/progressbar_examples/verify_progressbar.py` — the existing 22-check suite, with the end-of-run assertion tightened from "reaches near 100% ($\geq 90\%$)" to "reaches exactly 100%" for all four swept analyses (tran / AC / DC / noise). That check was what let the 82 % case slip through before. Front-end output only, solver-independent (identical bytes under Sparse and KLU). Full example regression: 149/149.

Enhancement-185 — openvaf-r autodiff audit: hypot & atan2 derivative fixes

A deep audit of the openvaf-r compiler's automatic differentiation — the pass (`mir_autodiff`) that builds the small-signal Jacobian used by AC analysis, Newton convergence, noise, pole-zero, and every derivative-dependent result. The probe compared the AC small-signal conductance $g = dI/dV$ of a large battery of nonlinear laws $I = f(V)$ against the *analytic* $f'(V)$, and caught two builtins whose value is correct (so the DC operating point is right) but whose derivative was wrong — the fifth occurrence of the accidental-correctness pattern this project keeps finding ([E-171/175/177/178/179](#)), and the first on the compiler side.

hypot(x,y) — wrong derivative formula

The autodiff rule computed $(x' + y')/(2 \cdot \text{hypot}(x,y))$ — the `sqrt(x)` pattern $(x'/(2 \cdot \text{sqrt}))$ misapplied to a two-argument function. The correct derivative of $\text{hypot}(x,y) = \sqrt{x^2+y^2}$ is

$$d/du \text{ hypot}(x,y) = (x \cdot x' + y \cdot y') / \text{hypot}(x,y)$$

At $V=0.7$, $y=0.5$ the old code gave $1/(2 \cdot 0.860) = 0.581$ where the answer is $0.7/0.860 = 0.814$ — 28% off. It was only *accidentally* correct at $x=0.5$ (with y constant, where $1/(2h) = 0.5/h$ happens to hold) — exactly the point a casual spot-check would use.

Fix: `hypot` is split from `sqrt` in `inst_cache` (its cache now holds `hypot(x,y)` itself, not $2 \cdot \text{hypot}$), and its chain rule computes $(x \cdot x' + y \cdot y')/\text{hypot}$ with the codebase's usual zero/one-derivative folding.

atan2(x,y) — two bugs (magnitude and sign)

`atan2` shares the `Pow|Atan2` chain rule $(x' \cdot c_0 + y' \cdot c_1) \cdot c_2$, but both cached factors were wrong:

1. the common factor c_2 was set to (x^2+y^2) — but the shared rule multiplies by c_2 , so it needed the reciprocal $1/(x^2+y^2)$;
2. the second-argument factor c_1 was $+x$, but $d/du \text{ atan2}(x,y) = (x' \cdot y - y' \cdot x)/(x^2+y^2)$ subtracts the second term, so it needed $-x$.

The result was wrong in magnitude *and* sign: `atan2(V,0.5)` gave $0.5 \cdot 0.74 = 0.37$ (should be $0.5/0.74 = 0.676$), and `atan2(0.5,V)` gave $+0.37$ where the answer is -0.676 . Fix: $c_2 = 1/(x^2+y^2)$, $c_1 = -x$.

Why it matters

`hypot` and `atan2` are used in real compact models — `hypot` for smoothed $\sqrt{x^2+\epsilon^2}$ turn-ons and velocity-saturation limiting, `atan2` for phase and geometry. A model using either compiled to a correct DC bias but a silently wrong AC response, small-signal gain, noise, and Newton Jacobian. Because the fix is at the autodiff-rule level it applies everywhere the derivative is taken — resistive currents, reactive charges (verified via AC susceptance of a `ddt(hypot...)`), higher-order derivatives, and `ddx`.

Verification

[examples/vafautodiff_examples/verify_vafautodiff.py](#) — 9 checks: `hypot d/dV` in both argument orders equals V/hypot , equals the hand-expanded `sqrt(V^2+0.25)` derivative, and is undisturbed at the $V=0.5$ accidental-correctness point; `atan2 d/dV` matches $x/(x^2+y^2)$ and $-y/(x^2+y^2)$ (magnitude and sign) and `atan2(V,V)→0`; the fix reaches a reactive charge via AC susceptance; and a regression battery of 15 other math builtins (`sin/cos/tan`, the inverse and hyperbolic families, `exp/ln/log/sqrt`, `pow`) confirms their derivatives were already correct and stay correct. A compiler property, identical under both linear solvers, so it runs once. Full example regression: 150/150.

Enhancement-186 — openvaf-r autodiff audit: real-modulo (%) derivative fix

Continuing the deep audit of the openvaf-r compiler's automatic differentiation (E-185) — the pass (`mir_autodiff`) that builds the small-signal Jacobian used by AC analysis, Newton convergence, noise, pole-zero, and every derivative-dependent result. The same AC-conductance referee that caught the `hypot` and `atan2` derivative bugs (compare $g = dI/dV$ of a nonlinear law $I = f(V)$ against the *analytic* $f'(V)$) caught a third: real modulo % had its derivative forced to zero. It is the sixth occurrence of the accidental-correctness pattern this project keeps finding (E-171/175/177/178/179/185), and the second on the compiler side.

`x % c` — derivative silently zero

Verilog-A real modulo lowers to the `Frem` opcode. Mathematically

$$x \% c = x - \text{floor}(x/c) \cdot c$$

is a slope-1 sawtooth in x : within each period the fractional remainder rises with x , so $d/dx (x \% c) = 1$ everywhere except the discrete wrap points (where `floor` jumps and the value is discontinuous anyway). Yet `Frem` was grouped with the genuinely-constant opcodes — `floor`, `ceil`, `$clog2`, the integer/bitwise ops, comparisons — and treated as having derivative 0, in *two* places that both had to be fixed:

1. the live-derivative gate (`mir_autodiff/src/lib.rs`, `zero_derivative`) — this decides which SSA values even *depend* on the differentiation variable. With `Frem` listed here, a modulo result was declared independent of the unknown, so the downstream current was considered constant and its derivative was never requested;
2. the chain rule (`mir_autodiff/src/builder.rs`, `inst_derivative`) — the `=>` return arm that emits no derivative edge.

Because the gate came first, the AC/Jacobian contribution of any modulo-dependent term was identically zero — a model computed the right DC value ($V \% c$ tracks V with slope 1) but contributed nothing to the small-signal conductance or the Newton Jacobian. The classic accidental-correctness signature: a .dc sweep of $I = 1e-3 \cdot (V \% 1.0)$ has slope exactly $1e-3$, while the AC conductance read 0.

The fix

`Frem` is removed from the zero-derivative group in both `lib.rs` and `builder.rs`, and given the correct rule

$$d/du (x \% c) = x' - \text{floor}(x/c) \cdot c'$$

For the overwhelmingly common constant divisor ($c' = 0$) this folds to just the dividend's derivative x' (slope 1, scaled by the outer chain). The general bias-dependent divisor term $-\text{floor}(x/c) \cdot c'$ is emitted only when the divisor actually carries a derivative. `floor(x/c)` is itself locally constant, so it contributes no further derivative — matching the exact algebra. `floor/ceil/$clog2/integer` ops stay in the zero group, correctly.

Why it matters

Real modulo appears in compact and behavioral models for phase wrapping (`phase % (2·π)`), periodic geometry, and sawtooth/relaxation constructs. Any such term compiled to a correct DC bias but a silently missing AC response, small-signal gain, noise, and Newton Jacobian entry — and, unlike a wrong *value*, a missing Jacobian entry can also slow or break convergence. Because the fix is at the autodiff-rule level it applies everywhere the derivative is taken — resistive currents, reactive charges, higher-order derivatives, and the explicit `ddx` operator.

Verification

[examples/vafautodiff_examples/verify_vafautodiff.py](#) — the E-185 autodiff battery, extended with five real-modulo checks: $d/dV (V \% 1.0) = 1$ and $d/dV (V \% 0.4) = 1$ (two constant divisors), the derivative scales with the outer prefactor, the chain rule flows *through* the modulo ($d/dV (V\%1)^2 = 2 \cdot (V\%1) = 1.4$ at $V=0.7$), and `floor/ceil` stay at 0 (undisturbed). A separate 3-terminal probe confirms the bias-dependent-divisor branch ($d/dc (x \% c) = -\text{floor}(x/c)$). Cross-checked independently via the `ddx` value path (which exposes the same autodiff result as a DC-readable value) and the compiler's own 19 `mir_autodiff` unit tests (unchanged, all pass). A compiler property, identical under both linear solvers, so the suite runs once. Full example regression: 150/150.

Enhancement-187 — openvaf-r math-identity simplifier: invalid function-inverse cancellations

Continuing the deep audit of the openvaf-r compiler after the autodiff fixes (E-185/186), this one moves off the derivative path and onto the algebraic simplifier (`mir_opt/src/simplify.rs`) — the pass that rewrites $f(g(x)) \rightarrow x$ for function-inverse pairs to avoid redundant work. Six of those cancellations were unconditionally wrong for ordinary finite inputs, corrupting the DC operating point itself (not just a derivative — a strictly more severe class than the autodiff bugs).

The defect

A cancellation $f(g(x)) = x$ is valid only when f is a true left inverse of g over all of g 's range. The simplifier applied it for every inverse pair, including compositions whose outer function only returns principal values:

rewrite (was $\rightarrow x$)	correct result	wrong whenever
<code>asin(sin(x))</code>	principal angle in $[-\pi/2, \pi/2]$	$ x > \pi/2$ (<code>asin(sin 3) = $\pi-3 = 0$. 1416</code>)
<code>acos(cos(x))</code>	principal angle in $[0, \pi]$	$x \notin [0, \pi]$ (<code>acos(cos 4) = $2\pi-4 = 2$. 283</code>)
<code>atan(tan(x))</code>	wrapped angle in $(-\pi/2, \pi/2)$	$ x > \pi/2$ (<code>atan(tan 2) = $2-\pi = -1$. 142</code>)
<code>acosh(cosh(x))</code>	$ x $ (cosh is even)	$x < 0$ (<code>acosh(cosh -2) = 2</code>)
<code>sqrt(x*x), sqrt(x**2)</code>	$ x $	$x < 0$ (<code>sqrt((-3)²) = 3, returned -3</code>)

`atan(tan(x))` is a legitimate angle-wrapping idiom — a model that writes it *wants* the sawtooth folding into $(-\pi/2, \pi/2)$; collapsing it to x silently deletes the wrap. The `sqrt(x2)  $\rightarrow x$` rewrites are the classic “ $\sqrt{(x^2)}$ is $|x|$, not x ” mistake.

Crucially the constant-folding path (`eval_unary`) evaluates each operator numerically and was always correct, so a spot check on literal arguments passes — the bug lived only in the symbolic cancellation on runtime, bias-dependent values, exactly where DC-value tests on constants can't see it.

The fix

The four principal-value cancellations and both `sqrt(x2)` rewrites are removed (`Asin/Acos/Atan/Acosh` and `Sqrt` now decline to simplify). MIR has no `fabs` opcode to fold `sqrt(x2)` to, so the `sqrt` simply stays in place and computes $|x|$ correctly. The cancellations that invert over the whole real line are kept and regression-checked — `tan(atan)`, `ln(exp)`, `asinh(sinh)`, `atanh(tanh)`, `sinh(asinh)`, `cosh(acosh)`, `log10(pow(10, .))`.

Rewrites that are unsound only for Inf/NaN inputs (`x-x  $\rightarrow 0$` , `x*0  $\rightarrow 0$` , `sin(asin(x))`, `exp(ln(x))`, ...) are standard finite-math behavior — Verilog-A models operate on finite quantities — and were deliberately left in place; the boundary drawn is “wrong for a finite, in-range input” = fix, “wrong only for Inf/NaN” = keep.

Why it matters

`asin/acos/atan` and `sqrt(x2)`-as-`abs` appear in real behavioral models — angle geometry, phase wrapping, magnitude/rectification. Any such term compiled to a silently wrong operating point, AC response, and Jacobian on part of its input range, with no diagnostic. Because the rewrite is purely value-level, the wrong number propagated into every downstream analysis.

Verification

[examples/mathident_examples/verify_mathident.py](#) — 12 DC-value checks: each formerly-buggy law compiled to OSDI, node biased, current read back (the wrong cancellation shows directly as a wrong current); the still-valid cancellations confirmed correct; and `sqrt` of a positive argument unaffected. [wrap_demo.va/.cir](#) sweep `atan(tan(V))` to draw the preserved sawtooth. The compiler's own `mir_opt/mir/mir_autodiff` unit tests (13/19/7) are unchanged and pass. A compiler property, identical under both linear solvers, so the suite runs once. Full example regression: 151/151.

Enhancement-188 — Monte Carlo warm-start (`montecarlo ... -warm`)

A performance enhancement for the [Enhancement-151](#) `montecarlo` `yield` command. Each Monte Carlo sample re-sources the deck (fresh random `.param` draws) and solves a DC operating point — but ngspice cold-solves every one, running the full `gmin` / source-stepping homotopy from scratch, even though consecutive samples move the operating point only slightly. `montecarlo ... -warm` reuses the previous sample's converged solution as the initial guess, cutting the per-sample Newton iteration count by roughly an order of magnitude with an unchanged yield.

The observation

Profiling the MC loop showed the DC solve, not the deck re-source, is the expensive part for non-trivial circuits — and every op starts cold. Even two *identical* ops back-to-back each burn the full homotopy: on a Verilog-A diode ladder, a cold op takes ~52 Newton iterations (most of it `gmin`/source stepping), and repeating it unchanged takes 52 again. ngspice's CKTop tries a direct Newton from the caller's `firstmode` and only falls into homotopy if that fails; DCop always passes `MODEINITJCT` (cold junction voltages), so the warm information in the previous solution is thrown away.

The change

`montecarlo ... -warm` wraps the sampling loop in `CKTsetWarmStart(1)` / `CKTsetWarmStart(0)`. In `DCop` (`spicelib/analysis/dcop.c`):

- a small buffer (outside the `CKTcircuit`, which `reset` recreates; indexed by equation number, stable across resets of an identical-topology deck) holds the last converged `CKTrhsOld`;
- when a valid guess of the right size is available, DCop preloads it into `CKTrhsOld` and calls CKTop with `MODEINITFLOAT` (use the existing node voltages) instead of `MODEINITJCT`, so the first Newton starts warm;
- if the guess is poor (a big parameter jump), that first `NIiter` simply fails and CKTop falls through to the normal cold `gmin`/source stepping — so the converged point is identical, only a cheap failed attempt is wasted;
- after every converged solve the solution is snapshotted as the next warm start.

The first sample is cold (no prior); the rest warm. When `-warm` is not given, `dcop_warm_enable` is 0 and the path is byte-identical to before (one extra `SMPmatSize` read). Warm-start lives in the shared DC operating-point code, so it works under both Sparse and KLU and composes with `-lhs`.

Correctness

Warm-start changes only Newton's *starting point*, not the equations, so it converges to the same operating point as the cold path to within the solver's convergence tolerance. The example verifies the warm and cold yields are exactly equal at `reltol=1e-6`, and agree to within a couple of samples at the default `reltol=1e-3` — a tolerance effect on a metric ($v(3) \approx 3.8$ V) whose default `reltol` window ($reltol \cdot |v| \approx 3.8$ mV) is as wide as the narrow spec band, so an edge sample can flip (it flips between two cold runs with different convergence aids too). Tightening `reltol` removes it.

Impact and scope

The win is the iteration *count* ($\sim 52 \rightarrow \sim 4$, $\approx 13\times$), so the wall-clock benefit scales with how much each Newton iteration costs. Large / hard-converging designs — many compact-model devices, `gmin`/source-stepping on every cold op, tens of ms per bias point — benefit most. On small circuits the per-sample deck re-source and command dispatch dominate, so the wall-clock speedup is smaller even though the iteration count still drops. It is opt-in and safe; the deck re-source overhead itself is a separate, complementary lever for a future enhancement.

Verification

[examples/warmstart_examples/verify_warmstart.py](#) — 5 checks on a random-is diode ladder: warm $\equiv$ cold yield exactly at **reltol=1e-6**; they agree to within a few samples at the default tolerance; warm cuts the per-sample iteration count $\geq 3\times$ (measured $52 \rightarrow 2$); -warm composes with -lhs; and warm $\equiv$ cold under KLU (the warm-start hook is in the shared DC-op code). A [warmstart_demo.cir](#) prints the cold-vs-warm yield and iteration counts side by side. Full example regression: 152/152.

Enhancement-189 — Sweep waveform overlay (`sweep ... -overlay`)

A usability follow-up to the [Enhancement-146](#) `sweep` command. `sweep` steps any circuit knob — an instance parameter, a model parameter, a `.param`, a source value — runs an inner analysis at each point, and records each output's last value into a summary sweep plot (a transfer curve vs the knob). The individual per-point analysis plots are retained, but overlaying their full *waveforms* to compare shapes was a manual chore (`setplot` to each run, `rename`, `plot` one at a time). `-overlay` (alias `-ov`) does it in one step, producing the classic HSPICE `.step` overlay.

The change

`sweep ... -overlay` collects every point's full output waveform, resamples them onto a common independent-variable grid, and builds a single sweepwave plot with one vector per (output, knob value), named `<output>_<value>`. The whole family then plots at once.

Three small static helpers were added to `frontend/com_sweep.c`:

- `sw_eval_vec(expr, &x, &y)` — evaluates the output expression and copies its entire waveform (not just the last value the summary keeps) plus its scale (the run's independent variable) into caller-owned buffers; a complex result (AC) is stored as magnitude.
- `sw_interp(x, y, len, xq)` — binary-search piecewise-linear interpolation, flat outside the data range. Each run lands on its own adaptive time/frequency grid, so the captured waveforms are resampled onto one shared grid before they can share a scale vector.
- `sw_wavename(base, val)` — builds a clean nutmeg vector name `<base>_<value>` (non-alphanumeric characters mapped to `_`).

In the sweep loop, when `-overlay` is set, each point's waveform and scale are captured alongside the existing last-value recording (the summary path is unchanged). After the loop, a common grid spanning `[min, max]` of all runs' scales is built at the finest run's resolution, every captured waveform is resampled onto it via `sw_interp`, and the results become the sweepwave plot — which is left current so `plot <out>_<val> ...` works immediately.

Correctness

The example is an RC low-pass ($C = 1\text{ nF}$) driven by a 1 V step, sweeping R so the time constant RC scales $1/2/4/8\text{ }\mu\text{s}$. Each overlaid curve is the exact step response $v(\text{out}) = 1 - \exp(-t/RC)$. The verification compares every resampled sample of every curve against that closed form and finds the worst error $< 1\text{e-4}$ — the resample is faithful, not merely visually close. Because the interpolation is piecewise-linear on the merged grid, it never overshoots a run's own endpoints.

Scope and graceful degradation

`-overlay` is purely front-end and solver-independent — it reshapes vectors that already exist. When the inner analysis yields a scalar (an op, no waveform), there is nothing to overlay: it prints `-overlay ignored (analysis '...' has no waveform to overlay)` and produces the ordinary summary curve. It is never an error. Without `-overlay` the sweep path is byte-identical to [Enhancement-146](#).

Verification

[examples/sweepwave_examples/verify_sweepwave.py](#) — 5 checks: the sweepwave plot is built and the resampled family matches $1 - \exp(-t/RC)$ to $< 1\text{e-4}$; each knob value gets a distinct `<out>_<val>` vector; `-overlay` is ignored gracefully on a scalar op; and the [Enhancement-146](#) summary last-value curve is still recorded correctly (shared path). A [sweepwave_demo.cir](#) overlays four RC step responses and prints matching sample values. It is a front-end command, so it runs once (solver-independent). Full example regression: 153/153.

Enhancement-190 — Nested multi-knob sweep (**sweep ... -vs ...**)

The second usability follow-up to the [Enhancement-146](#) sweep command (the first was the [Enhancement-189](#) -overlay waveform family). sweep stepped one knob and recorded each output's last value into a sweep transfer curve. -vs <knob> <spec> (alias -family) adds outer knobs: the inner (positional) knob stays the x-axis, and the outer knobs' cartesian product forms a curve family — one curve per output per outer combination — the parametric family that .dc ... SWEEP ... and HSPICE's nested .dc produce.

The change

sweep <inner> <spec> [-vs <knob> <spec>]... [-analysis ...] [-output ...] [-overlay]. Each -vs contributes an outer knob (up to SW_MAXKNOB = 4 knobs total, cartesian points capped at SW_MAXPTS). The summary sweep plot uses the inner knob's values as the scale; for each output and each outer-knob combination it emits one curve named <output>_<outerknob>_<value>.... -overlay composes: each cartesian point's full waveform becomes a sweepwave vector named with every knob's value (inner first), so a single knob is <output>_<value> exactly as in E-189.

The spec parser (lin|dec|oct, list, start stop step) was factored into sw_parse_spec and is now used for both the inner knob and each -vs knob, so all knobs accept the same three forms.

By construction, a single knob reduces to E-146: with one knob the cartesian product has one outer combination, so the curve is named <output>, the messages are the E-146 wording, and the data indexing is identical. Existing sweep and sweepwave examples pass untouched.

Set ordering — the subtle part

Each knob is applied with E-146's mechanism: a .param needs alterparam + reset (a full re-source), while an instance/model knob is an in-place alter / altermod. In a cartesian product these interact — a reset re-sources the whole deck and drops every in-place alter — so the order matters. The loop iterates with the inner knob fastest and, at each point:

- re-sources once only when a .param knob's value actually changed since the previous point (i.e. at outer-loop boundaries; an inner .param still resets every point as in E-146, a pure in-place sweep never resets), staging every .param with alterparam before the single reset;
- then re-applies every in-place knob after that reset, so an inner alter survives an outer .param's re-source.

Correctness

The example is an RC low-pass driven by a 1 V step, sweeping R (inner) and C (outer), so each family curve is the exact transfer characteristic $v(\text{out})|_{t=T} = 1 - \exp(-T/(R*C))$ as a function of R at that C. verify_nestedsweep.py compares every family value to that closed form — worst error < 5e-6. A dedicated check uses a .param outer knob (a reset at each outer step with the inner alter re-applied afterward) and confirms the family still matches, proving the ordering.

Verification

[examples/nestedsweep_examples/verify_nestedsweep.py](#) — 5 checks: the cartesian run/curve counts are reported correctly (3×2 = 6 runs, 2 curves); each family curve matches $1 - \exp(-T/(R*C))$ to < 5e-6; a .param outer knob (reset/alter ordering) matches; -overlay composes with the family (<out>_<R>_<C> names); and a single knob still names its curve plainly vo (E-146 unchanged). A [nestedsweep_demo.cir](#) sweeps $R \times C$ into a two-curve family. Front-end and solver-independent, so it runs once. Full example regression: 154/154.

Enhancement-191 — `.ac lin 2 / .sp lin 2` off-by-one fix

A correctness fix found while auditing the data-output path (`wrdata / wrsnp / save / rawfile write`). A ragged-length `wrdata` test wrote an AC vector that came out one point shorter than expected, which traced back not to the writer but to the AC linear frequency sweep: `ac lin 2 fstart fstop` (and the `.ac card`, and `.sp lin 2`) produced one frequency point instead of two.

The bug

The AC (`spicelib/analysis/acan.c`) and S-parameter (`spicelib/analysis/span.c`) analyses computed the linear step behind this guard:

```
case LINEAR:
    if (job->ACnumberSteps - 1 > 1)                /* true only for N >= 3 */
        job->ACfreqDelta = (fstop - fstart) / (job->ACnumberSteps - 1);
    else
        job->ACfreqDelta = 0;                      /* "single point" patch */
```

`numberSteps - 1 > 1` holds only for $N \geq 3$, so `lin 2` fell into the single-point patch and got `freqDelta = 0`. With a zero step the sweep loop terminates after the first point:

```
freq += job->ACfreqDelta;
if (job->ACfreqDelta == 0) goto endsweep;
```

so the sweep emitted a single row at `fstart` and silently dropped the `fstop` point. Only exactly $N = 2$ was affected: `dec / oct` sweeps use a different step formula, and `lin 1`, `lin 3`, `lin 4`, ... were all correct. The single-point patch (Richard McRoberts) was meant for `lin 1`; the `- 1 > 1` form over-reached by one and swallowed `lin 2` as well.

The fix

Restore the intended guard, matching the form the noise analysis (`noisean.c`) already used correctly (`if (NnumSteps == 1) delta = 0; else delta = (stop - start)/(N - 1);`):

```
if (job->ACnumberSteps > 1)
    job->ACfreqDelta = (fstop - fstart) / (job->ACnumberSteps - 1);
else
    job->ACfreqDelta = 0;
```

- $N = 1 \rightarrow 1 > 1$ false $\rightarrow$ `freqDelta = 0` $\rightarrow$ one point at `fstart` (the genuine single-point case, preserved);
- $N = 2 \rightarrow 2 > 1$ true $\rightarrow$ `freqDelta = fstop - fstart` $\rightarrow$ two points at `fstart` and `fstop`;
- $N \geq 3 \rightarrow$ unchanged.

The same one-line change was applied to `span.c` (the `.sp` linear sweep had copied the identical guard). The distortion analysis uses a different $N + 1$ -point convention and was not affected; noise was already correct.

Correctness

The recovered point is a genuine solved point, not a duplicate. `verify_aclin2.py` runs an RC low-pass and checks that both `lin 2` endpoints match the exact transfer function $H(j\omega) = 1/(1 + j\omega RC)$ to $< 1e-6$, that `lin 1` still yields one point, that `lin N` yields N linearly-spaced points ($N = 3, 5, 10$), and that `.sp lin 2` yields two points with the analytic series-R S-parameters ($S_{11} = 1/3$, $S_{21} = 2/3$ for a $50\ \Omega$ series R between $50\ \Omega$ ports).

The wider output-path audit that surfaced this found the writers themselves clean: Touchstone `wrs2p / wrsnp / rdsnp` round-trip exactly across RI/MA/DB formats, S/Y/Z parameters (with the correct `Rbase`

normalization), and frequency units including complex angles; `wrdata` handles complex `(re,im)` columns, single-scale, and ragged-length padding; `save` filtering restricts the vector set; and the `rawfile` `write / load` round-trips bit-exact.

Verification

[examples/aclin2_examples/verify_aclin2.py](#) — 5 checks under both linear solvers (the fix is in the shared analysis setup). A [aclin2_demo.cir](#) prints the `lin 1 / lin 2 / lin 5` frequency lists. Full example regression: 155/155.

Enhancement-192 — Auto-checkpoint on interrupt (**set autosave=<file>**)

A follow-up to the [Enhancement-131](#) transient checkpoint / restart commands, prompted by the question “does savestate fire on a Ctrl-C?” — it did not. E-131 gave savestate / loadstate, but a checkpoint had to be taken by hand, so interrupting a long run lost its progress. E-192 makes an interrupted transient write one automatically, opt-in via a variable:

```
set autosave=mylongrun.chk
run          # ... press Ctrl-C ...
```

A Ctrl-C during that run now writes mylongrun.chk before control returns to the prompt; loadstate mylongrun.chk (even in a fresh process) resumes it.

The safe hook point

Writing a file — buffered I/O, malloc — from inside a signal handler is undefined behaviour, so E-192 deliberately does not checkpoint in ft_sigintr. Instead it uses the path ngspice already unwinds through on an interrupt:

- ft_sigintr only sets a flag (ft_intrpt) and returns (during a run ft_setflag is TRUE, so it does not longjmp);
- at the next accepted timepoint, dctran’s SPfrontEnd->IFpauseTest() → OUTstopnow() sees the flag and returns 1, so dctran returns E_PAUSE;
- E_PAUSE propagates through if_run (which maps it to return code 1) up to dosim (frontend/runcoms.c), where the run reports “interrupted”.

E-192 hooks exactly that `err == 1` branch. So the checkpoint is written on the main thread, at a consistent timestep boundary, with ordinary I/O — never in signal context. A genuine SIGINT, a stop when ... breakpoint, and a Verilog-A \$stop all funnel through this same branch (they differ only in how the flag gets set, upstream of the E-192 code).

The change

- com_checkpoint.c: the body of com_savestate is factored into `bool ckt_write_checkpoint(CKTcircuit *ckt, const char *fname)`; the command is now a thin wrapper. The autosave hook calls the same function, so an autosave file is byte-for-byte what savestate would have written.
- runcoms.c (dosim, the `err == 1` interrupted branch): if `cp_getvar("autosave", ...)` yields a filename and the interrupted run was a transient (`CKTmode & MODETRAN, CKTtime > 0`), it calls `ckt_write_checkpoint`.

It is opt-in (no autosave variable $\Rightarrow$ behaviour is exactly as before, nothing written) and transient-only (the `MODETRAN / CKTtime` guard keeps it from writing stale state after, e.g., an interrupted AC run).

The signal handler itself is only installed in interactive mode (`!ft_batchmode`), so this targets an interactive Ctrl-C — the scenario where ngspice returns to the prompt. In batch (`-b`) mode a Ctrl-C still terminates the process, unchanged.

Correctness

Because the autosave path is the same writer as savestate, an autosave checkpoint is byte-identical to a manual savestate taken at the same pause point, and loadstate resumes it exactly — the E-131 round-trip already verified bit-identical continuation. The E-192 example confirms all of this, including with a real SIGINT.

Verification

[examples/autosave_examples/verify_autosave.py](#) — 5 checks: (1) with `set autosave`, an interrupted (stop-paused) transient writes a checkpoint; (2) it is byte-identical to a manual `savestate` at the same pause; (3) it loads and resumes via `loadstate`; (4) the opt-in gate — without the variable, an interrupt writes nothing; (5) a genuine Ctrl-C (SIGINT) driven through a PTY (so ngspice runs interactive) fires the autosave — lenient, since a machine fast enough to finish the run before the signal lands just skips it, the identical hook being covered by 1–4. Checks 1–4 use a stop breakpoint to hit the exact interrupt path deterministically. A [autosave_demo.cir](#) shows the checkpoint being written mid-run. Full example regression: 156/156.

Enhancement-193 — **.pnoise** honors **sqrnoise** (noise-density units fix)

A correctness fix found while auditing the RF / PSS periodic-steady-state suite. The ngspice manual (§ **.noise**, pp. 326–327) defines `onoise_spectrum` / `inoise_spectrum` as a noise spectral density in $V/\sqrt{\text{Hz}}$ (or $A/\sqrt{\text{Hz}}$) by *default*, switchable to the squared V^2/Hz form with the `sqrnoise` control variable:

”`onoise_spectrum`: ... the output noise voltage ... divided by $\sqrt{\text{Hz}}$ Default setting of ngspice is *unset* `sqrnoise`, which delivers ... Noise Spectral Density [$V/\sqrt{\text{Hz}}$].”

.noise obeys this. **.pnoise** (periodic noise) did not — it always emitted the squared V^2/Hz density and ignored `sqrnoise`. So the *same-named* output vectors carried different units across the two analyses (differing by a square), and the documented `sqrnoise` control silently did nothing for `pnoise`:

	<code>sqrnoise</code> unset (default)	set <code>sqrnoise</code>
.noise <code>onoise_spectrum</code>	4.063e-9 ($V/\sqrt{\text{Hz}}$)	1.651e-17 (V^2/Hz)
.pnoise <code>onoise_spectrum</code> (before)	1.651e-17 (V^2/Hz)	1.651e-17 (V^2/Hz)

The `sqrnoise` toggle test makes the divergence unambiguous: **.noise** flips units with the variable; **.pnoise** returned the squared value regardless.

The change

`pnoise_sweep` (`spicelib/analysis/dcpss.c`) now reads `sqrnoise` (via `cp_getvar`, exactly as `noisearc.c` does) and, when it is unset (the default), applies `sqrt` to the emitted `onoise/inoise` densities — so the default is $V/\sqrt{\text{Hz}}$ and set `sqrnoise` gives V^2/Hz . Both emit paths — the Enhancement-178 cyclostationary folding path and the standard adjoint-folding path — get the same conditional `sqrt`.

	<code>sqrnoise</code> unset (default)	set <code>sqrnoise</code>
.pnoise <code>onoise_spectrum</code> (after)	4.063e-9 ($V/\sqrt{\text{Hz}}$)	1.651e-17 (V^2/Hz)

.pnoise and **.noise** now agree by default, and `sqrnoise` toggles both.

Scope. This fix is **.pnoise** only. The two-tone `qpnoise` command’s single-point diagnostic already prints *both* the V^2/Hz and $V/\sqrt{\text{Hz}}$ forms explicitly (no ambiguity), and its swept and single-point paths share the V^2/Hz convention internally, so reconciling it is deferred; only the **.pnoise** sweep vector was inconsistent with the manual.

Examples updated

Several existing `pnoise` examples asserted the old V^2/Hz values. Their physics (the noise folding) is unchanged — only the final output units — so they were adjusted minimally:

- `rfpss/rc_pnoise` now checks the default $V/\sqrt{\text{Hz}}$ form (`sqrt` of the analytic) and cross-checks against **.noise** without forcing `sqrnoise` — the two now match to 0.00.
- `rfpss/rc_cyclo`, `cyclofold`, `pnoise`fold are inherently power-density relations (`onoise·f == ...`, referee sums of $|Z|^2 \cdot S$), so they set `sqrnoise` to keep the V^2/Hz form; two LTI-limit checks that previously compared `pnoise( $V^2$ )` to `.noise( $V/\sqrt{\text{Hz}}$ )2` become direct equalities.

Also from this audit

The wider RF/PSS audit found the conversion machinery correct: PAC's driving-point response and its ± 1 conversion sidebands (via the `.pac` card's trailing `maxsideband` argument) match a clean transient DFT to $\sim 0.1\%$, and Harmonic Balance matches a transient DFT to 4–5 digits. It also documented the previously-undocumented trailing `maxsideband` argument of the `.pac` card (`dcpsc.c` comment + the `rc_pac.cir` example).

Verification

[examples/pnoiseunits_examples/verify_pnoiseunits.py](#) — 4 checks on a small linear-RC pnoise: default `pnoise == sqrt(4kTR/(1+(\omega RC)^2))` (V/ $\sqrt{\text{Hz}}$); set `sqrnoise pnoise ==` the density (V^2/Hz); default² == the squared form (the exact `sqrt` relation); and default `pnoise == default .noise` (matches to 0.00). A [pnoiseunits_demo.cir](#) prints the spectrum in both unit modes. The updated pnoise-family examples (`rc_pnoise`, `rc_cyclo`, `cyclofold`, `pnoise`) continue to pass. Full example regression: 157/157.

Enhancement-194 — Particle swarm optimization (**optimize -method pso**)

A new search method for the built-in **optimize** command. The optimizer already had two methods, both local:

- Nelder-Mead simplex (**-method nm**, Enhancement-130) — derivative-free, on a scalar **-minimize** objective;
- Levenberg-Marquardt (**-method lm**, Enhancement-143) — gradient least-squares over one or more **-targets**.

Both descend into whichever basin the start point sits in, so on a multimodal objective — several local minima — they return whatever local minimum is nearest the start. E-194 adds a third, global method: particle swarm optimization (**-method pso**).

The method

PSO is a population-based, derivative-free global search. N particles fly through the normalized $[0, 1]^n$ parameter box; each keeps its own best-seen point (**pbest**), and the swarm shares a global best (**gbest**). Each iteration updates every particle's velocity toward **pbest** and **gbest** and moves it:

$$\begin{aligned} v &\leftarrow \chi \cdot (v + \phi \cdot r_1 \cdot (\text{pbest} - x) + \phi \cdot r_2 \cdot (\text{gbest} - x)) & \chi = 0.72984, \quad \phi = 2.05 \\ x &\leftarrow \text{clamp}_{01}(x + v) & |v| \leq 0.5 \end{aligned}$$

with the standard Clerc-Kennedy constriction coefficients (χ, ϕ chosen so the swarm converges without exploding), velocities clamped to half the box, and positions clamped to $[0, 1]$. Particle 0 starts at the user's init point; the rest are seeded uniformly at random, so the whole box is explored. It reuses the existing **opt_eval** (which returns the scalar cost for both objective kinds), so PSO works for a scalar **-minimize** and for **-target** least-squares — unlike **-method lm**, which requires targets.

New options:

- **-swarmsize <N>** (aliases **-swarm**, **-npart**) — population; default auto, $10 + 4 \cdot n_p$ capped at 60.
- **-seed <s>** — makes a run reproducible. PSO draws from a small self-contained splitmix64 PRNG, independent of ngspice's global RNG state, so a given seed gives byte-identical results.
- **-maxiter**, **-tol** are shared with the other methods (**-tol** is the **gbest**-stagnation tolerance, held over several iterations before stopping).

The dispatch is unchanged for the existing methods (**nm/lm** behave exactly as before); PSO is selected only by **-method pso**.

Why it matters

On the classic multimodal function $f(p) = \sin(p) + \sin(10 p / 3)$ over $[2.7, 7.5]$ (global minimum $p^* = 5.1457$, $f^* = -1.8996$; several higher local minima), started at the $p = 2.7$ corner:

Nelder-Mead:	converged, objective = -1.19992	$p = 3.3873$	(local minimum)
Particle swarm:	converged, objective = -1.8996	$p = 5.1454$	(GLOBAL minimum)

Nelder-Mead falls into the nearest local basin; PSO finds the global one. It costs more evaluations (a swarm $\times$ iterations) but does not depend on a good starting guess.

Verification

[examples/psoopt_examples/verify_psoopt.py](#) — 6 checks: PSO finds the global optimum from the trapping corner start; Nelder-Mead from the *same* start is trapped in a higher local minimum (the whole point); a fixed **-seed** is exactly reproducible; several independent seeds all reach the global basin; PSO also minimizes a **-target** least-squares objective (recovers the parameter to a $1e-12$ residual); and PSO solves a 2-D separable multimodal minimum ($f^* = -3.799$). The existing **optimize** example (Nelder-

Mead + Levenberg-Marquardt, 20 checks) continues to pass unchanged. It is a front-end command, independent of the linear solver, so it runs once. Full example regression: 158/158.

Enhancement-195 — Differential evolution (**optimize -method de**)

A second global method for the built-in **optimize** command. **Enhancement-194** added particle swarm (**-method pso**) as the optimizer's first global, population-based, derivative-free search. E-195 adds the other workhorse of that family: differential evolution (**-method de**).

The method

DE keeps a population of NP vectors in the normalized $[0,1]^{np}$ box. Where PSO pulls trial points toward remembered best positions, DE (the classic DE/rand/1/bin) builds each trial from a scaled difference of random members and crosses it with the target:

```
v      = a + F·(b - c)          F = 0.8   (a, b, c distinct, ≠ i)
u[j]   = v[j] if rand < CR or j = jrand, CR = 0.9 (binomial crossover;
          else x[i][j]          one dim always taken)
keep u in place of x[i] iff cost(u) ≤ cost(x[i]) (greedy selection)
```

with the well-tested defaults $F = 0.8$, $CR = 0.9$, positions clamped to $[0,1]$. The difference vector $b - c$ self-scales to the population's own spread: large while the members are far apart (broad exploration), shrinking automatically as they converge (fine local refinement). That self-scaling is what makes DE robust on rugged, discontinuous, or poorly-scaled landscapes without any per-problem step tuning. It reuses the existing `opt_eval`, so it works for a scalar **-minimize** objective and for **-target** least-squares, like PSO.

DE shares all of PSO's infrastructure — the same self-contained `splitmix64` PRNG (so **-seed** is byte-reproducible), the same **-swarmsize** population option (default `auto 10 + 4·np`, capped at 60; clamped to ≥ 5 because DE needs at least four distinct members to form a mutant), and the shared **-maxiter** / **-tol** (gbest-stagnation). The `nm / lm / pso` dispatch is unchanged; DE is selected only by **-method de**.

Why it matters — DE vs PSO

On the multimodal $f(p) = \sin(p) + \sin(10 p / 3)$ over $[2.7, 7.5]$ (global $p^* = 5.1457$, $f^* = -1.8996$; higher local minima) started at the trapping $p = 2.7$ corner:

Nelder-Mead:	objective = -1.19992	p = 3.3873	(local minimum)
Differential evolution:	objective = -1.8996	p = 5.1457	(GLOBAL minimum)
Particle swarm:	objective = -1.8996	p = 5.1454	(GLOBAL minimum)

Both global methods find the global optimum where the local simplex fails. They are complementary — PSO's momentum-driven swarm and DE's self-scaling difference vectors behave differently on different landscapes — so **optimize** now offers a small global toolbox (`pso, de`) alongside the local `nm` and gradient `lm`.

Verification

[examples/deopt_examples/verify_deopt.py](#) — 7 checks: DE finds the global optimum from the trapping corner; Nelder-Mead from the same start is trapped in a higher local minimum; a fixed **-seed** is exactly reproducible; several independent seeds all reach the global basin; DE minimizes a **-target** least-squares objective (recovers the parameter to a $1e-13$ residual); DE solves a 2-D separable multimodal minimum ($f^* = -3.799$); and DE and PSO both reach the global while the local NM does not. The existing **optimize** (20 checks) and **psopt** (6 checks) examples are unchanged. It is a front-end command, independent of the linear solver, so it runs once. Full example regression: 159/159.

Enhancement-196 — Simulated annealing (**optimize -method sa**)

A third global method for the built-in **optimize** command, completing the search-method set. **Enhancement-194** (particle swarm) and **Enhancement-195** (differential evolution) added two *population-based* global optimizers. E-196 adds the classic *single-walker* one: simulated annealing (**-method sa**).

The method

SA holds a single current point in the normalized $[0, 1]^n$ box. Each step it proposes a random neighbour $x_n = \text{clamp}_{01}(x + \text{step} \cdot U(-1, 1))$ and applies the Metropolis acceptance rule:

- accept if $\text{cost}(x_n) \leq \text{cost}(x)$ (a downhill move), or
- accept anyway with probability $\exp(-(\text{cost}(x_n) - \text{cost}(x)) / T)$ (an uphill move).

While the temperature **T** is high, uphill moves are readily accepted, so the walker climbs out of local minima; T is then cooled geometrically ($T \leftarrow \alpha \cdot T$, α set so T drops ~4 decades over the run), so uphill moves become rare and the walker settles into a minimum. The best point ever visited is returned. It evaluates one candidate per step — no population — which makes it the lightest-weight global method, and the natural choice when each analysis is expensive and you want a global search without a whole population of simultaneous evaluations.

Everything is auto-scaled to the problem: the initial temperature **T0** is the mean $|\Delta \text{cost}|$ of a handful of random probes (so an uphill move of that size is accepted about half the time when hot), and the step size shrinks as T cools (wide exploration when hot, fine refinement when cold). There is nothing to tune. It reuses the existing `opt_eval`, so it works for a scalar **-minimize** objective and for **-target** least-squares. **-seed <s>** makes a run reproducible (the same self-contained splitmix64 PRNG as PSO/DE); **-maxiter** is the number of cooling levels. The `nm/lm/ps/de` dispatch is unchanged.

Why it matters — the complete toolbox

On the multimodal $f(p) = \sin(p) + \sin(10 p / 3)$ over $[2.7, 7.5]$ (global $p^* = 5.1457$, $f^* = -1.8996$) started at the trapping $p = 2.7$ corner, all five methods:

Nelder-Mead (local):	objective = -1.19992	p = 3.3873	(local minimum)
Simulated annealing (global):	objective = -1.8996	p = 5.1458	(GLOBAL)
Particle swarm (global):	objective = -1.8996	p = 5.1454	(GLOBAL)
Differential evolution (global):	objective = -1.8996	p = 5.1457	(GLOBAL)

`optimize` now spans local descent (`nm`, `lm`) and three complementary global strategies (`pso`, `de`, `sa`). A single walker refines a little more loosely than a population, so SA reliably reaches the global basin rather than machine precision; a short `nm/lm` polish from its result sharpens it when that matters.

Verification

[examples/saopt_examples/verify_saopt.py](#) — 7 checks: SA finds the global basin from the trapping corner; Nelder-Mead from the same start is trapped in a higher local minimum; a fixed **-seed** is reproducible; several independent seeds all reach the global basin; SA minimizes a **-target** least-squares objective; SA solves a 2-D separable multimodal minimum; and all three global methods (`sa`/`pso`/`de`) reach the global while the local `nm` does not. The existing `optimize` (20), `pssopt` (6) and `deopt` (7) examples are unchanged. It is a front-end command, independent of the linear solver, so it runs once. Full example regression: 160/160.

Enhancement-197 — A 100-parameter circuit optimization (raised **optimize** limits)

The built-in **optimize** command — extended with Levenberg-Marquardt least-squares fitting (143–145) and three global search methods, particle swarm (194), differential evolution (195) and simulated annealing (196) — carried compile-time limits of 16 parameters and 64 least-squares targets. E-197 raises those to 128 each (and the auto-population cap from 60 to 256), and fixes what breaks when the global methods are actually run at that scale, so a genuinely large problem can be optimized in a single call.

Raised limits

com_optimize.c sizes its per-run arrays from two macros, now:

```
#define OPT_MAXP    128    /* max parameters to optimize (was 16) */
#define OPT_MAXT    128    /* max least-squares targets (was 64) */
```

Those bound the `-param/-mparam/-dparam` knob arrays and the LM Jacobian `J[OPT_MAXT][OPT_MAXP]`, so raising them costs a little stack per run and nothing else. The auto-population sizing for the population methods (PSO/DE), previously capped at 60, now caps at 256 so a high-dimensional run gets an adequately sized swarm (`-swarmsize` still overrides). Exceeding a cap is reported cleanly (`optimize: too many -param (max 128)`), not crashed.

Testbed: 100 unknowns

100 independent 1 mA current sources, each driving an unknown resistor R_i to ground, so $v(n_i) = 1e-3 \cdot R_i$. The 100 targets $t_i = 1e-3 \cdot (100 + 9800 \cdot i/99)$ ohm form a linear voltage ramp 0.1 V ... 9.9 V — a well-posed, separable 100-dimensional least-squares.

Levenberg-Marquardt solves it to machine precision. For a well-posed high-dimensional fit, the gradient least-squares solver is the right tool:

```
optimize: converged, sum-sq residual = 2.5e-29 (rms 5.0e-16) after 619 evaluations
v(n0) = 0.100000    v(n50) = 5.049490    v(n99) = 9.900000    (targets hit exactly)
```

all 100 parameters and 100 targets accepted and honored, in about two seconds.

Making the global methods work at scale

The global methods are derivative-free, so at 100 dimensions they are intrinsically far slower than LM — but two changes were needed to make them *function* at that scale rather than stall.

Adaptive crossover for DE. Classic DE/rand/1/bin uses a crossover rate $CR = 0.9$, which mutates almost every coordinate of a trial vector. In high dimension a trial that perturbs $\sim n$ coordinates at once is essentially always worse than its parent and gets rejected by greedy selection — so DE freezes at its starting guess (observed: gbest flat at the initial cost, zero progress). E-197 caps the *expected number of mutated coordinates* at ~ 15 for large problems:

```
const double CR = (n <= 16) ? 0.9 : 15.0 / (double) n;
```

For $n \leq 16$ this is exactly the classic $CR = 0.9$, so the existing low-dimensional DE behavior (and its example) is unchanged; for $n = 100$ it is $CR = 0.15$, and DE descends again — from ~ 1240 to ~ 882 over 35 generations on the testbed.

Dimension-scaled convergence patience. High-dimensional population runs plateau for several generations between improvements, so the stagnation counter that stops PSO/DE on convergence would otherwise cut a still-descending high-D run short. The threshold now grows with dimension:

```
if (++stall >= 8 + n / 4) break;    /* was: >= 8 */
```

For $n \leq 3$ this is exactly the previous 8, so the small- n tests are unchanged; at $n = 100$ it gives 33 generations of patience.

Which method for many unknowns

DE at 100-D is a genuinely hard *global* search and does not fully converge in a short budget — that cost is intrinsic to global optimization in high dimension, not a defect. The division of labor is the point:

Situation	Method
Well-posed fit / extraction (unique best fit)	lm — machine precision, $\sim 10^2$ – 10^3 evals
Smooth, unimodal, no gradient	nm — derivative-free simplex
Multimodal / many local minima	pso / de / sa — global; now usable as dimension grows

Verification

[examples/opt100_examples/verify_opt100.py](#) — 4 checks: LM fits all 100 parameters to 100 targets at machine precision (proving both raised caps at once); DE descends substantially at 100-D (self-calibrated against its own starting cost, $\geq 15\%$ reduction — the adaptive high-dimensional crossover); a fixed –seed is bit-reproducible even at 100 dimensions; and exceeding the raised cap (129 params) is reported cleanly rather than crashing. The existing optimize (39), pssopt (6), deopt (7) and saopt (7) examples are unchanged. It is a front-end command, independent of the linear solver, so it runs once. Full example regression: 161/161.

Enhancement-198 — **stb** stability / loop-gain analysis

A new front-end command that measures a feedback loop's small-signal loop gain $T(f)$ and reports its phase margin and gain margin — the everyday stability check for op-amps, regulators, LDOs, PLLs and any other feedback design. ngspice had no built-in for this; you had to hand-roll a `.control` script. **stb** is the one-command equivalent of Spectre's `stb` analysis (the Middlebrook–Tian injection-probe method).

Usage

```
stb <Vprobe> <Iprobe> (dec|oct|lin <N> <fstart> <fstopt>)
```

Mark the loop break — a single wire carrying the feedback signal — with a probe pair between the driving node A and the loaded node B. Both are quiescent, so the DC operating point is untouched:

```
Vstb A B dc 0 ac 0      series 0 V source (+node = driver A, -node = load B)
Istb 0 B dc 0 ac 0      shunt 0 A source (ground -> load node B)
```

```
stb Vstb Istb dec 20 1 100meg
```

Why double injection

Break a loop and inject a single test signal, and the loading at the break corrupts the measurement: a series *voltage* injection reads the true loop gain only if the break is from a zero-impedance source into an infinite-impedance load. Middlebrook and Tian's double injection cancels the loading error by combining a voltage and a current injection. **stb** runs two AC sweeps, altering the probe AC magnitudes:

```
voltage injection (Vstb ac=1, Istb ac=0):  Tv = -v(A)/v(B)
current injection (Vstb ac=0, Istb ac=1):  Ti = -i(Vstb)/(i(Vstb) + 1)
loop gain          T = (Tv·Ti - 1) / (Tv + Ti + 2)
```

The combined T is independent of where in the loop the probe sits — the defining property a single injection lacks, and the example verifies it directly (a loaded break gives the same margins as a clean break in the same loop).

A singularity-free form. In the common clean-break case the load impedance is high, so $i(Vstb) \rightarrow -1$ (all the injected current returns through the shorted voltage probe) and $T_i \rightarrow \text{inf}$. Computing $T_i = -a/(a+1)$ then divides by zero. **stb** substitutes T_i and clears the $(a+1)$ denominator, evaluating the exact equivalent

$$T = -(T_v \cdot a + a + 1) / (T_v \cdot a + T_v + a + 2), \quad a = i(Vstb),$$

which stays finite at $a = -1$, where it reduces cleanly to $T = T_v$.

Output

The complex loop gain is stored as the vector **loopgain** (vs frequency) in a new **stb** plot, and the margins are printed:

Stability (loop gain via Tian double injection at vstb):

```
DC loop gain      : 79.13 dB
phase margin      : 83.10 deg (at fc = 90497 Hz)
gain margin       : 32.88 dB (at f = 1.732e+06 Hz)
```

```
plot db(loopgain)          magnitude (dB)
plot 180/PI*cph(loopgain)  phase (deg, unwrapped)
```

The phase margin is $180 + \text{angle}(T)$ at the first $|T| = 1$ (0 dB) crossing; the gain margin is $-|T|_{\text{dB}}$ at the first $\text{angle}(T) = -180$ deg crossing. Both crossovers are log-interpolated on an unwrapped phase.

Implementation

`frontend/com_stb.c` (registered in `commands.c`, declared in `com_commands.h`). The probe's two terminal node names are recovered from the voltage source via `CKTinst2Node`; the two injections are driven by dispatching `alter` and `ac` through the command table (as `com_optimize/com_sweep` do), and the resulting complex vectors (`v(A)`, `v(B)`, `Vstb#branch`, `frequency`) are read with `ft_evaluate`. It reuses the AC analysis, so it works under both the Sparse and KLU solvers.

Verification

[examples/stb_examples/verify_stb.py](#) — 5 checks: the phase and gain margin match the closed-form loop gain of an ideal-output op-amp loop; a loaded break (finite op-amp output impedance) gives the same margins as a clean high-impedance break in the same loop (proving the current injection corrects the loading — a single voltage injection would be ~10 % off and diverge past crossover); a higher-gain design's reduced phase margin is tracked and matches analytics; the complex loopgain vector is stored; and an unknown probe source is reported cleanly. Front-end command, independent of the linear solver, so it runs once. Full example regression: 162/162.

Enhancement-199 — N-port Touchstone device (**snp2va.py**)

Instantiate a measured or simulated S-parameter block — a filter, cable, connector, package, or amplifier stored in a Touchstone `.snp` file — as a circuit element that works in AC *and* transient. ngspice had no native n-port element (its `.sp` "ports" are tagged voltage sources used to *measure* a circuit's S-parameters; `rdsnp/wrsnp` from E-64/E-72 are reader/writer commands, not a device), so this fills a real gap.

Approach: Touchstone → Verilog-A, not a native convolution device

S-parameters are frequency-domain; a time-domain model needs a *rational* representation of the response. Rather than write a native convolution device (with its own history buffers and stability headaches), `snp2va.py` converts the file into a Verilog-A n-port realized with `laplace_nd`. OpenVAF's OSDI laplace machinery (E-31, complex poles) then supplies both the AC response and the transient (recursive-state) response — no convolution code at all.

```
parse Touchstone -> S(f) -> Y(f) -> common-pole vector fit -> Verilog-A
I(p_i) <+ sum_j [ laplace_nd(V(p_j), num_ij, den) + e_ij*ddt(V(p_j)) ]
```

- Parse any port count (`.s1p`, `.s2p`, `.s3p`, ...); S, Y, or Z data; MA / DB / RI formats; Hz–GHz; arbitrary reference impedance.
- Convert to admittance $Y = (1/z_0)(I - S)(I + S)^{-1}$.
- Vector-fit every $Y_{ij}(f)$ with a *shared* pole set (Gustavsen). Two details matter: the fit is done in normalized frequency (without it the $s \cdot e$ column is $\sim 10^9$ and the $1/(s-p)$ columns $\sim 10^{-7}$, and the least-squares is hopelessly ill-conditioned), and the pole relocation is done as polynomial roots of the sigma numerator (Durand–Kerner) rather than a matrix eigensolve.
- Emit the model. The subtlety is that Y is *improper* whenever a network has shunt capacitance ($Y \sim sC$ at high frequency), and `laplace_nd` — a ratio of polynomials — cannot represent that (it produced a 63 % AC error at the band edge). The fix is to give `laplace_nd` only the strictly-proper (pole) part and emit the $e \cdot s$ term as an explicit **ddt** (a capacitance). AC error then drops to $\sim 10^{-6}$.

Hardening

- Automatic order selection climbs the pole count and keeps the best *stable* fit. It returns as soon as the fit reaches the tolerance (a genuine rational match — a delay/comb response only "engages" once enough poles are present, and is *uniformly* poor at low orders, so the selection must not quit on a small between-order improvement there). Otherwise it stops at the "knee" — but only once the error is already near a floor, so *noisy* data is fit at its noise floor rather than over-fitted into an unstable model. It breaks when a fit turns unstable (polynomial-root finding degrades at high degree) and returns the best stable fit. (This behavior was refined by stress-testing against a transmission line — a pure delay — which the first version quit on at 2 poles and now fits with ~ 12 .)
- Stability is enforced — right-half-plane poles are reflected into the left half plane — so the emitted model is always BIBO-stable (a noisy, even *non-passive*, fit still stays bounded in transient).
- Passivity is checked and reported; `snp2va` warns when a fit is non-passive (passivity *enforcement* by residue perturbation is a documented non-goal for now).

Pure Python, no numpy

The converter is pure standard library. The three primitives vector fitting needs are reimplemented: least-squares via Householder QR, complex matrix inverse via Gauss–Jordan, and polynomial roots via Durand–Kerner — each validated against numpy to machine precision during development.

Usage

```
python3 snp2va.py bandpass.s2p -o bandpass.va -m bandpass
openvaf-r bandpass.va -o bandpass.osdi
# deck: N1 p1 p2 mm / .model mm bandpass / .control pre_osdi
↪ bandpass.osdi
```

Verification

[examples/nport_examples/verify_nport.py](#) — 6 checks, each generating a Touchstone file from a network whose response is known *exactly*, running `snp2va.py` + OpenVAF, and confirming the device matches the ORIGINAL network in ngspice: the converter runs and compiles; the device matches an R-L-C resonator in AC to 5×10^{-6} (including the transmission peak) and in transient to 5×10^{-3} (one model, both analyses); a 5-pole LC ladder converts (order selection scales past two poles); a 3-port star network matches on both coupled outputs to 4×10^{-9} ; and a noisy measurement is fit at its noise floor, reported non-passive, yet stays bounded in transient (stability enforced). Full example regression: 163/163.

Beyond the pinned checks, the converter was stress-tested to near machine precision on resonators, notches, high-Q blocks, and multi-pole ladders. A delay-dominated transmission line degrades gracefully: the AC response is accurate over the fitted band, while transient pulse edges ring — inherent to rational fitting of a pure delay, for which a T/LTRA line is the right model.

Enhancement-200 — built-in **pre_snp** command

Enhancement-199 shipped `snp2va.py`, a standalone converter that turns a Touchstone `.sNp` S-parameter file into a Verilog-A n-port model. It works, but it is an *external* step: the user runs Python, then `openvaf-r`, then writes a deck. Enhancement-200 folds the whole thing into ngspice itself as a C command, `pre_snp`, so the workflow collapses to one line next to `pre_osdi` — no Python, no external script, nothing to install.

```
.control
pre_snp bandpass.s2p          * parse .s2p -> vector-fit -> write bandpass.va,
                              *   then run openvaf-r -> write bandpass.osdi
pre_osdi bandpass.osdi        * load the freshly compiled n-port model
.endc
```

then instantiate it like any OSDI device:

```
N1 p1 p2 mm
.model mm bandpass            * the module name pre_snp emitted (default: file base name)

pre_snp <file.sNp> [module] — the optional second argument names the Verilog-A module; the
.va and .osdi are written next to the .sNp.
```

The converter in C

`snp2va.c` is a faithful C port of `snp2va.py` — same numerics, same order-selection logic, same output. It is self-contained (only `stdio/stdlib/string/math/ctype/complex`), using C99 double `_Complex` (typedef'd `cplx`) so it does not collide with ngspice's own `complex.h` macros. The three primitives vector fitting needs are reimplemented:

- least-squares via Householder QR (`lstsq_real`),
- complex matrix inverse via Gauss-Jordan (`mat_inv_c`),
- polynomial roots via Durand-Kerner (`poly_roots`).

The pipeline follows E-199: parse Touchstone $\rightarrow S(f) \rightarrow Y(f) = (1/z_0)(I-S)(I+S)^{-1} \rightarrow$ common-pole vector fit in normalized frequency (Gustavsen) $\rightarrow$ emit $I(p_i) <+ \sum_j Y_{ij}(s) \cdot V(p_j)$, with the improper $e \cdot s$ (shunt-C) term split out as an explicit `ddt` because `laplace_nd` cannot represent an improper rational. Each Y_{ij} is realized as a parallel bank of first/second-order **laplace_nd** sections (see *Transient robustness* below). Automatic order selection climbs the pole count and returns the best stable fit (right-half-plane poles reflected $\rightarrow$ always BIBO-stable). Numerically it was checked identical to the Python reference: resonator 4×10^{-6} , ladder 7×10^{-7} , 3-port 3×10^{-8} .

The public entry point is `snp2va_convert(snpfile, vafile, module, msg, msglen)`; the command wrapper `com_pre_snp` (in `com_presnp.c`) calls it, then shells out to `openvaf-r` to compile the `.osdi`.

Transient robustness (order climb + realization)

An N-port stress sweep (generate an N-port R-L-C network, extract its S-parameters with `.sp`, round-trip through `pre_snp`, and compare DC/AC/transient against the original subcircuit) surfaced two defects that the 2-pole example checks never reached, both now fixed:

- Order-selection double-free. The climb that keeps the best *stable* fit let its best and prev buffers alias, then freed the same allocation twice — `pre_snp` aborted (SIGABRT) on any network whose fit needed three or more pole orders (the 2-pole resonator/star escaped by breaking at tolerance first). Fixed by not freeing a buffer the other pointer still holds.
- Transient divergence from a non-passive realization. Each Y_{ij} is fit independently, so two things could destabilize the *time-domain* model even though every pole is in the LHP and the AC response (evaluated pointwise) is exact: a single degree-Np rational per element has coefficients

spanning $\sim |p|^N$ ($\approx 10^{79}$ for eight poles at 10^{10} rad/s), whose companion-form integration is ill-conditioned; and the improper $e \cdot s$ terms form a capacitance matrix with no passivity constraint — an indefinite (“negative-capacitance”) matrix makes the DAE unstable, so the model blew up ($v \rightarrow 10^{28.4}$). Fixed by (a) realizing each element as a parallel bank of first/second-order **laplace** sections (one per real pole / conjugate pair), so every coefficient stays $\leq O(|p|^2)$, and (b) projecting the **e**-matrix onto the symmetric PSD cone (symmetrize $\rightarrow$ eigendecompose via Jacobi $\rightarrow$ clamp negative eigenvalues to 0). Genuinely-improper (shunt-C) networks keep their positive eigenvalues; spurious ones are removed. The [presnp example](#) gained a 4-port coupled-ladder check (fit climbs past two orders; transient must stay bounded and match) that fails on the pre-fix converter.

Runs before **pre_osdi**, always

`pre_snp` is registered as a command named `snp`, so writing `pre_snp ...` in a deck triggers ngspice's existing `pre_mechanism` (`inp.c`): every `pre_`-prefixed control line is stripped of its prefix and executed before the circuit is parsed. That alone would not be enough — `pre_snp` must run before the `pre_osdi` that loads its output, and control lines otherwise execute in deck order.

The fix is a two-pass execution of the collected pre-commands: pass 0 runs every `snp` command (in deck order), pass 1 runs all the rest. So the `.osdi` a `pre_snp` generates is guaranteed to exist by the time any `pre_osdi` tries to load it — even if the deck lists the `pre_osdi` line *first*. The ordering is a guarantee, not a convention the user has to remember.

```
for (pass = 0; pass < 2; pass++)
    for (wl = pre_controls; wl; wl = wl->wl_next) {
        int is_snp = ciprefix("snp ", wl->wl_word) ||
                    strcmp(wl->wl_word, "snp") == 0;
        if (pass == 0 ? !is_snp : is_snp) continue;
        cp_evloop(wl->wl_word);
    }
```

(`snp` / `pre_snp` are also added to `inpcom.c`'s case-preservation list, alongside `osdi` / `pre_osdi`, so the file path in the argument keeps its original case on case-insensitive input handling.)

Finding the compiler

`pre_snp` shells out to `openvaf-r`, which `find_openvaf()` locates via, in order: the `openvaf` ngspice variable (set `openvaf=...` in **spinit** or on the command line — a set inside `.control` runs too late, *after* the pre-commands), the `OPENVAF` environment variable, `$SPICE_LIB_DIR/openvaf-r`, then `PATH`. If none resolve, `pre_snp` reports the failing `openvaf-r` invocation and lists all four options.

Verification

[examples/presnp_examples/verify_presnp.py](#) — 5 checks. Each builds a Touchstone file from a network whose response is known *exactly*, lets `pre_snp` do the whole convert+compile+load inside ngspice, and confirms the device matches the ORIGINAL network: `pre_snp` writes the `.va` and compiles the `.osdi`; the device matches an R-L-C resonator in AC to 5×10^{-6} (including the transmission peak) and in transient to 5×10^{-3} (one compiled block, both analyses); a 3-port star network compiles and matches on both coupled outputs to 4×10^{-9} ; and — with `pre_osdi` written before `pre_snp` — `pre_snp` still runs first so the model loads, proving the ordering guarantee. Full example regression: 164/164.

Enhancement-201 — `pre_snp` scalability (fast vector fitting + shared-pole realization)

The **Enhancement-200** `pre_snp` converter worked, but a stress test (generate an N-port R-L-C network, extract its S-parameters, round-trip through `pre_snp`, compare DC/AC/transient) showed it struggled from about 8 ports up: an 8-port took ~190 s, and larger port counts were simply infeasible. Two things scaled badly, both in `snp2va.c`:

- the pole identification stacked all N^2 elements into one dense least-squares matrix of size $(2 \cdot N_s \cdot N^2) \times (N^2 \cdot (N_p+2) + N_p)$ — $O(N^4)$ memory, $O(N^6)$ compute, re-solved dozens of times over the order climb (≈ 8 TB of RAM at $N=100$);
- the emitted model gave every one of the N^2 elements its own `laplace_nd` bank — $O(N^2 \cdot N_p)$ filter sections and OSDI state.

Four fixes rebuild the scalability; the fit now runs to $N=100$ and the end-to-end ceiling rises from ~8 to ~20–24 ports.

The four fixes

1. Fast Vector Fitting (Deschrijver/Gustavsen). The stacked pole-ID matrix has block-arrow structure: each element's residue columns are independent and couple only through the shared common-pole (σ) columns. Instead of forming that matrix, each element block is Householder-reduced on its own (a small $2N_s \times (N_p+2)$ QR) and only its residual rows — the part orthogonal to that element's own columns — are stacked into one tall $((2N_s - N_p - 2) \cdot N_e) \times N_p$ system solved for σ . Memory drops $O(N^4) \rightarrow O(N^2)$, compute $O(N^6) \rightarrow O(N^2 \cdot N_s \cdot N_p^2)$.
2. Reciprocity. A passive network has symmetric Y; when detected, only the upper triangle ($N(N+1)/2$ elements) is fit and the residues are mirrored — $\sim 2\times$ fewer solves.
3. Iteration cap. Pole relocation now stops as soon as the poles settle (relative move $< 1e-4$) instead of a fixed 12 sweeps — typically 3–5.
4. Shared-pole realization. All N^2 elements share the same poles, so the pole-filters are computed once per input port — a real pole gives one filter $V(p_j)/(s-p)$, a conjugate pair two real basis filters $(s-\sigma)/D$ and ω/D , with the residue entering each output as a real scalar weight. Each output current is then a cheap weighted sum. This is $O(N \cdot N_p)$ `laplace_nd` and OSDI state instead of $O(N^2 \cdot N_p)$ — the difference between a model that compiles/simulates at large N and one that does not.

Three latent bugs the stress test also surfaced

- Order-selection double-free. At higher pole counts a relocated pole set could lose its adjacent-conjugate-pair structure (a "pair" split by numerical noise, or a near-real pole with a tiny imaginary part), and `build_basis/ctil_to_cres` — which walk $i+=2$ on a complex pole — then wrote one slot past the residue array. Fixed by a `canon_poles` step that snaps near-real poles to real and rebuilds exact adjacent conjugate pairs.
- 256-entry stack array. The Touchstone parser assembled each frequency's matrix in a fixed `cp1xpv[16*16]`, hard-capping the port count at 16 ($N=32$ smashed the stack). Moved to the heap; the auto-detect limit was lifted to 512 ports.
- OpenVAF crash on integer literals. A `laplace_nd` coefficient array assigned to a variable (as the shared realization does) crashes OpenVAF when a coefficient is an integer literal like `'{1}'` — which `%12g` produces for `1.0`. The emitter now forces a real literal (`'{1.0}'`).

Results

- Fit at $N=8$: 220 s $\rightarrow$ 3.6 s. $N=64$ fits in 47 s (was ~860 GB of RAM); $N=100$ fits in 2.75 min at 9.6×10^{-4} error.

- Model size: `laplace_nd` count is now $N \cdot N_p$ (65 at $N=8$, 449 at $N=32$) rather than $\sim N^2 \cdot N_p/2$ (256, 7168).
- Correctness preserved. DC/AC/transient still overlay the original subcircuit ($N=8$ unchanged; $N=16$ — previously infeasible — matches to 1.3%). Full example regression 164/164; the E-200 example suite passes (now 9 checks).
- The remaining bottleneck is OpenVAF's compile time for the (now far smaller) shared model — ~ 26 s at $N=16$, ~ 43 s at $N=20$ — so the practical end-to-end ceiling is $\sim N=20-24$ while the fit itself scales to $N=100$. (The separate ngspice `.sp` extraction cost only matters when *generating* S-parameters, not for real `.snp` from a measurement or EM solver.)

Verification

[examples/presnp_examples/verify_presnp.py](#) gains two checks (9 total): an 8-port coupled ladder — the size the old converter struggled at — converts+compiles in a few seconds with the fast fit and the compact shared-pole model, and the resulting device matches the original network in AC. The E-200 order/realization guards (4-port double-free + transient-divergence) and the resonator/3-port checks continue to pass.

Enhancement-202 — `.sp` S-parameter matrix inverse is $O(N^3)$, not $O(N!)$

The `pre_snp` stress test ([Enhancement-201](#)) noted a *separate* wall: producing the input Touchstone file with ngspice's own `.sp` analysis was itself pathologically slow — an 8-port took ~13 s and a 10-port ~18 minutes, growing by roughly a factor of N per added port. That is in the RFSPICE analysis, not the converter, and this enhancement removes it.

The cause: an $O(N!)$ matrix inverse

The `.sp` analysis builds the port S-matrix at every frequency (`CKTspCalcSMatrix` in `cktspdum.c`), which inverts $N \times N$ complex matrices — several per frequency, for the S, Y and Z matrices. The complex inverse (`cinverse` in `maths/dense/dense.c`) was computed by the adjugate / determinant method (Cramer's rule):

```
CMat* cinverse(CMat* A) {
    CMat* B = cadjoint(A);           // adjugate
    cplx  de = cinv(cdet(A));         // 1 / determinant
    return complexmultiply(B, de);
}
```

and `cdet` is a recursive cofactor expansion — $O(N!)$, while `cadjoint` computes N^2 cofactors (each an $(N-1) \times (N-1)$ determinant), so `cinverse` is $O(N \cdot N!)$. With several inverses per frequency across a whole sweep, the cost explodes by a factor of N per port — $6! \rightarrow 7! \rightarrow 8!$ is exactly the measured $\times 7, \times 8, \times 9$ per added port.

The fix: Gauss-Jordan elimination

`cinverse/cinversedest` now invert by Gauss-Jordan elimination with partial pivoting on $[A \mid I]$, in $O(N^3)$. It is a drop-in replacement — same signature, same result (the true inverse), computed the standard way. Profiling had shown ~99 % of an 8-port `.sp` run sat inside `CKTspCalcSMatrix`; with the fix that disappears.

Behavior is preserved: the old `cinverse` never returned NULL (the adjugate always produced a matrix, garbage for a singular input), so the new one returns NULL only on a true allocation failure and zero-fills a singular matrix — the singular case does not arise for a well-posed `.sp` network. `cdet/cadjoint` remain for any other callers.

Results

ports	old <code>.sp</code>	new <code>.sp</code>
8	~13 s	0.3 s
10	~18 min	0.02 s
16	(infeasible)	0.03 s
32	(infeasible)	0.09 s

~50,000 $\times$ at $N=10$. The extracted S-parameters are unchanged — verified against the closed-form network to $\sim 2 \times 10^{-7}$. The same inverse is used by the periodic S-parameter path (`.psp / PSS` in `dcps.c`), which benefits as well; the full example regression (`touchstone`, `rfa` analyses, `psp`, ...) is unchanged.

Together with [Enhancement-201](#) (the `pre_snp` fit and model-size scalability), the whole measured-block workflow — extract or import an N -port `.snp`, convert it to an OSDI device, simulate — now scales to many ports; the remaining limit is OpenVAF's compile time for the generated model.

Verification

[examples/spscale_examples/verify_spscale.py](#) runs a 12-port R-L-C ladder through `.sp + wrsnp` (a port count that took minutes before) and checks two things: the extraction completes in a fraction of a second, and every entry of the extracted 12×12 S-matrix matches the closed-form network across the sweep (max abs error $\sim 2 \times 10^{-7}$) — the fast inverse is exact, not just fast. Full example regression: 165/165.

Enhancement-203 — `.meas ac` gain / phase margin (+ batch `vdb/vp` fix)

ngspice's `.measure` is already broad — TRIG/TARG (with VAL/RISE/FALL/CROSS/ LAST/TD/AT), FIND ... WHEN, AVG/RMS/INTEG/DERIV, MIN/MAX/MIN_AT/ MAX_AT/PP, ERR, and `param='...'`, across tran/ac/dc/sp. An audit against the "richer TRIG/TARG, FIND...WHEN, integral/RMS/derivative, AC margins" wish-list found all of those already present and working — the one genuine gap was AC margins.

Why the manual recipe cannot work

The textbook way to measure margins with the existing primitives is

```
.meas ac pm FIND vp(out) WHEN vdb(out)=0      ; phase margin
.meas ac gm FIND vdb(out) WHEN vp(out)=-180    ; gain margin
```

but the gain-margin form fails: `vp()` returns the phase wrapped to $(-180, 180]$. On a loop whose true phase sweeps past -180° (any 3-pole-ish loop), the wrapped phase jumps from $\approx -179^\circ$ straight to $\approx +179^\circ$ and never equals -180° , so `WHEN vp=-180` reports "out of interval". You cannot find the phase crossover from wrapped phase.

The new functions

Two first-class `.meas ac` functions, computed on the unwrapped phase:

```
.meas ac pm phase_margin v(loopgain)      ; PM = 180 + (unwrapped phase at |gain|=1, i.e. 0 dB)
.meas ac gm gain_margin v(loopgain)      ; GM = -gain(dB) at the -180 deg phase crossover
```

Each prints the margin and its crossover frequency (... `at= <freq>`). The phase is unwrapped by undoing the $\pm 360^\circ$ `atan2` jumps, the 0 dB gain crossover and the -180° phase crossover are found by linear interpolation, and the crossover frequency is interpolated in log space (log-spaced sweep). A loop with no finite -180° crossover (a one- or two-pole loop — infinite gain margin) is reported as such, not as a bogus number. Implemented in `com_measure2.c` (`measure_margin`, dispatched from `get_measure2`).

One subtlety worth noting: ngspice's phase (`vp`, and the measure 'p' type) honors a runtime `cx_degrees` flag that defaults to radians, so `radtodeg()` is a no-op by default; the margin code therefore computes phase in degrees explicitly.

Two bugs the work surfaced

- **gettok_iv** truncated suffixed vectors. The batch (dot-card) measure auto-save pass extracts each measured vector with `gettok_iv` (`misc/string.c`). It copied the leading `v/i`, then broke after the *first* following character on `n_paren == 0` — so `vdb(out)` became `vd` (and `vm/vp/...` were all truncated). The stray `vd` then failed to save ("can't parse 'vd'") and left the node unsaved, so a batch `.meas ac ... vdb(out)` died with "no data saved for AC". Fixed to stop only once the parenthesised part has actually closed (`gettok_iv` is used *only* by `measure`, so the fix is contained). A companion `strip_ac_vec` reduces `vdb(out)` $\rightarrow$ `v(out)` so the base node is what gets `.saved`.

Verification

[examples/acmargin_examples/verify_acmargin.py](#) — 5 checks against closed-form loop gains (buffered one-pole sections, so the poles sit exactly where placed): a stable 2-pole loop's phase margin matches analytic to $<1^\circ$ (84.89° vs 84.89°) and its gain margin is correctly reported infinite; a 3-pole loop's phase margin (-42.75°) and gain margin (-15.92 dB) both match the closed form exactly; and a batch `.meas ac ... vdb(out)` dot-card auto-saves the node and returns the right value. Full example regression: 166/166.

Enhancement-204 — auto-triggering DC convergence aids

A convergence-robustness enhancement: `.option convhelp` turns ngspice's operating-point solver into an automatic escalation ladder that reaches for the strong convergence aids *without the user asking*, and reports which aid produced the point.

Background

ngspice's CKTop cascade already escalates automatically through gmin stepping and source stepping. But the two strongest aids —

- the globalized damped-Newton line search (E-111), and
- pseudo-transient continuation (E-127) —

only fired when the user hand-set `.option linesearch` / `.option ptcont`, and nothing told the user which aid rescued a hard operating point. The ngspice-vs-Spectre gap analysis flagged exactly this: *"what remains is mostly auto-triggering heuristics (reaching for these aids without the user asking) and folding them into the robustness presets."*

The ladder

`.option convhelp` makes the whole cascade one automatic ladder:

Newton (+ line search) → gmin step → source step → pseudo-transient → optran

- Line search is enabled for the operating-point Newton (and, since they call `NIiter` internally, its fallback homotopies). E-111's line search reduces to a plain full Newton step whenever the full step already reduces the weighted residual norm, so points that converge easily take the same iterates and are unaffected.
- Pseudo-transient continuation is tried automatically as a fallback — the E-127 gate becomes `if (ckt->CKTptcont || ckt->CKTconvhelp)`, so no `.option ptcont` is needed.
- When a non-trivial aid produces the operating point, CKTop prints a one-line note, e.g. `Note: DC operating point reached via pseudo-transient continuation.`

CKTop saves the caller's line-search flag on entry and restores it on every exit (the routine was refactored to a single `done: exit`), so the setting never leaks into the transient analysis that follows.

It is off by default — the default (`convhelp unset`) path is byte-for-byte the historical behavior, no new output — and it is turned on by `.option errpreset=conservative`. The E-110 `TSKtolGiven` given-flag mechanism means an explicit `.option convhelp=0` still overrides the preset, regardless of `.options` line order.

Verification

The same deliberately stiff behavioral exponential E-127 used — no junction limiting, $1e-14 \cdot (\exp(V/0.026) - 1) = (100 - V)/100$, physical root $V(1) = 0.837922 V$ — where plain Newton overshoots the huge $\exp()$ derivative to a spurious $\sim 70.5 V$ root (and here even the transient-op fallback lands on it).

With gmin/source stepping disabled so pseudo-transient continuation is the rung that must fire:

- `.option convhelp` reaches the correct $0.837922 V$ and reports *via pseudo-transient continuation* — without `.option ptcont`;
- a plain run lands on the spurious root, so `convhelp` demonstrably changes the outcome;
- `.option errpreset=conservative` enables the same ladder, and an explicit `.option convhelp=0` overrides it.

On a battery of normal circuits (diode, BJT, two-diode, resistor network) convhelp is result-neutral (converged node voltages identical to a plain run) and silent (no aid note — the plain Newton solve already converges). All checked under both linear solvers (KLU + Sparse 1.3): `examples/convhelp_examples/verify_convhelp.py`, 35/35.

Files changed

Seven files, additive, mirroring the E-127 option plumbing:

- `optdefs.h` — `OPT_CONVHELP` enum + `ERRP_CONVHELP` given-flag bit.
- `tskdefs.h` — `TSKconvhelp:1`; `cktdefs.h` — `CKTconvhelp:1`.
- `cktsopt.c` — `OPT_CONVHELP` setter (+ given-flag), `convhelp` in the `errpreset` value table (conservative = on), and the "convhelp" `OPTtbl` keyword.
- `cktntask.c` — copy + default off; `cktdojob.c` — `CKTconvhelp = TSKconvhelp`.
- `cktop.c` — the auto-escalation: enable line search under `convhelp`, auto-fire pseudo-transient continuation, track and report the winning aid, single-exit restore of the line-search flag.

Scope

`convhelp` orchestrates the *existing* aids automatically; it adds no new numerical method. The default path is unchanged, so it is fully backward compatible.

Enhancement-205 — low-rank residue factorization in `pre_snp`

A scalability enhancement for the N-port Touchstone device. The `pre_snp` converter (E-200/E-201) turns a `.snp` S-parameter file into a Verilog-A n-port and compiles it with `openvaf-r`. E-201 made the *vector fit* scale to $\sim N=100$, but the emitted model's compile time — dominated by an $O(N^2)$ -term, $O(N \cdot N_p)$ -filter shared-pole realization — caps the practical port count at $\sim 20\text{--}24$ ($N=32$ compiles in ~ 80 s). E-205 lifts that ceiling for the common class of blocks whose ports couple through a few shared modes.

The structure

The shared-pole realization writes, for each output port i ,

$$I(p_i) \leftarrow \sum_j d_{ij} \cdot V(p_j) + \sum_j e_{ij} \cdot \text{ddt}(V(p_j)) + \sum_{\text{sections}} \sum_j W^{\text{sc}}_{ij} \cdot \text{laplace}(V(p_j))$$

Each pole section's weights W^{sc}_{ij} form an $N \times N$ matrix. For a block whose ports couple through M shared resonant modes (multi-port filters, cavities, packages with a shared plane), each pole's residue matrix is an outer product $a \cdot a^T$ — rank 1 — so W^{sc} has rank $r \ll N$. The dense emit ignores this and pays $O(N^2)$ terms + N filters per section regardless.

The realization

Per channel (d , e , and each pole section) the emitter builds the $N \times N$ real weight matrix and, via a new in-C one-sided Jacobi SVD, chooses the cheaper of a dense emit or a low-rank emit $W = U \cdot V^T$ (rank r kept above `tol_rank` = $1e-7$, so a clean singular-value gap compresses fully while a slowly-decaying channel stays dense):

- cost $2N \cdot r$ (+ r filters) vs dense nnz (+ N filters) — pick the smaller.
- Input combining is the key. Because `laplace` is linear, $\sum_j V[j][m] \cdot \text{laplace}(V(p_j)) = \text{laplace}(\sum_j V[j][m] \cdot V(p_j))$, so the low-rank form filters the r combined inputs $u_m = \sum_j V[j][m] \cdot V(p_j)$ once and distributes them via U . This drops filters from $O(N \cdot N_p) \rightarrow O(r \cdot N_p)$ — the piece that actually dominates the compile — and coupling terms from $O(N^2) \rightarrow O(N \cdot r)$.

A full-rank block keeps the dense form on every channel, so it is emitted exactly as before (verified bit-identical). The passivity projection of the capacitance e (E-201) is preserved, so the low-rank transient stays stable.

Results

Measured on a 3-shared-mode block (`openvaf-r` compile of the emitted `.va`):

N	dense compile	low-rank compile	dense→low-rank filters
16	16.9 s	1.5 s	96 → 8
24	42 s	5.7 s	144 → 10
32	81 s	11.6 s	192 → 8

Compile time goes from super-linear to $\sim$ linear (constant filter count), a $\sim 7\text{--}14\times$ speedup, with the response exact — AC to $\sim 4e-7$, transient to $\sim 3e-5$ — versus a forced-dense build of the same fit. A full-rank ladder is unchanged (bit-identical AC, same filter count).

Files changed

`ngspice-46/src/frontend/snp2va.c` only: a one-sided `jacobi_svd`, an `emit_filter` helper, the per-channel dense/low-rank chooser and input-combined low-rank emit, and the `PRE_SNP_DENSE` env-var escape hatch. Verify: `examples/lowrank_examples/` (5 checks — compression, AC + transient == forced-dense, full-rank fallback).

Scope

E-205 changes only how the fitted model is *realized* in Verilog-A; the vector fit and its accuracy are untouched. Low-rank compression activates automatically when the device's coupling is low-rank and is a no-op otherwise.

Enhancement-206 — design centering / yield optimization

The capstone that fuses two whole subsystems: the optimizer (Nelder-Mead / LM / PSO / DE / SA, [E-130–197](#)) and the Monte-Carlo yield suite (agauss/mccorr sampling + montecarlo specs, [E-149–151](#)). Design centering optimizes the nominal design point to maximize parametric yield / Cpk under process variation — a real Spectre/ADS capability.

The fusion

The optimizer's `opt_eval` already does *apply knobs* → *run analysis* → *evaluate objective*. Design centering replaces the objective with an inner Monte-Carlo run:

```
optimize -dparam xc <init> <lo> <hi> ... (the nominal design point to centre)
        -center -samples N [-lhs] [-seed s]
        -analysis "<cmd>"
        (-spec <metric> [-max HI] [-min LO])... (pass/fail limits, as in montecarlo)
        [-method nm|psa|de|sa]
```

- The outer optimizer searches the design knobs.
- At each candidate, the inner loop runs N Monte-Carlo samples — each reset re-samples the deck's process variation (agauss/.param, and any mccorr correlations) around the current design centre — evaluates every spec, and reduces to the worst-case Cpk and the pass-fraction yield.
- The design knobs feed the agauss *centres*, so moving the design point shifts the whole distribution; the process σ stays in the deck.

Why Cpk is the objective (yield is reported)

Raw yield is a granular step function (with finite N it changes in jumps), which stalls a simplex. Cpk — $\min(\text{gap to each active limit}) / (3\sigma)$ per spec, worst-cased across specs — is continuous and monotone in yield, so it is the smooth objective the optimizer maximizes; the yield is computed and reported alongside. Maximizing Cpk centres the distribution in its spec window (and rewards lower sensitivity), which is exactly design centring.

A fixed inner seed gives every candidate the *same* process draws (common random numbers), so the objective is deterministic and smooth. With `-lhs` the stratified sample-mean is ≈ 0 , so the optimum lands right on the analytic centre.

Implementation

`com_optimize.c` only. `opt_eval`'s in-place-knob application was factored into `opt_apply_inplace` (so the MC loop can re-apply the centre after each reset re-sources the deck), and a new `opt_eval_center` runs the inner MC and returns $-\min(\text{Cpk})$. The methods (nm/psa/de/sa) are unchanged — they call `opt_eval`, which now returns the centering cost. Reuses the E-149/151 sampling (`mc_lhs_config`, `mc_sss_off`, `setseed`). `lm` is rejected for `-center` (Cpk is not a least-squares residual); the default method is Nelder-Mead. Results are published as `dcenter_yield` / `dcenter_cpk` vectors.

Verification

Synthetic problems with known optimal centres (`examples/dcenter_examples`):

- `[center]` output $\sim N(xc, 0.5)$, spec `[4, 6]` ⇒ the yield/Cpk-optimal centre is the midpoint 5; from an off-centre start (4.0) the optimizer recovers $xc \approx 5.00$ and the analytic $Cpk = 1/(3 \cdot 0.5) = 0.667$.
- `[improves]` the centred design's yield ($\sim 95\%$) beats the off-centre start's (a montecarlo at $xc = 4.0$ sits near 50%).
- `[twoknob]` two design params with a lower and an upper spec on two outputs centre independently ($x_a \rightarrow 5$ on `[4,6]`, $x_b \rightarrow 10$ on `[9,11]`).

Scope

Front-end only; solver-independent. Composes with everything the optimizer and the MC suite already support — `-lhs`, `mccorr/mvnorm` correlations, all optimizer methods, and the `-param/-mparam/-dparam` knob kinds.

Enhancement-207 — eye diagram / jitter analysis (**eye**)

A new application domain: the core measurement for serial links / SerDes / high-speed I/O. The eye command folds a received data waveform modulo the unit interval and reports the standard signal-integrity metrics — jitter, eye height/width, and the eye opening at a target BER.

Command

```
eye <expr> -ui <T> [-tstart <t0>] [-threshold <vth>] [-window <frac>]
```

Front-end post-processing over a transient result vector (solver-independent), in the family of `fft / linearize / meas`. Given the data signal `<expr>` and its unit interval `-ui <T>`, it:

1. Rails + threshold — the 20th/80th percentiles of the samples give `level0 / level1`; the decision threshold defaults to their midpoint (or `-threshold`).
2. Crossings — every threshold crossing is found by linear interpolation.
3. Jitter — the UI phase is the circular mean of the crossings mod UI; each crossing's TIE (time-interval error, relative to the ideal UI grid) gives jitter RMS (std of TIE) and peak-to-peak.
4. Eye height — the vertical opening at the sampling instant (UI/2 from the transitions): `min(high samples) - max(low samples)` in a $\pm\text{window}\cdot\text{UI}$ slice.
5. Eye width — UI - jitter_pp at the threshold, plus the width at BER 1e-12 from the Gaussian random-jitter tail (UI - 14.069 · jitter_rms).
6. Folded eye — the waveform folded modulo 2·UI into `eye_wave` vs `eye_t`; the scatter plot of those two vectors is the eye diagram (`plot eye_wave vs eye_t`).

Results are published as permanent vectors (`eye_height`, `eye_width`, `eye_width_ber12`, `eye_jitter_rms`, `eye_jitter_pp`, `eye_level0/1`, `eye_amplitude`, `eye_threshold`, `eye_crossings`, `eye_ui`).

Implementation

New self-contained front-end command `com_eye.c` (+ `com_eye.h`), registered in `commands.c / com_commands.h / Makefile.am`. It reads the vector and its time scale via `ft_evaluate`, and publishes its results into a fresh eye plot (as `stb` does): the folded `eye_wave` and its scale `eye_t` must share a plot of their own length so that `plot eye_wave vs eye_t` and `wrdata eye_wave` pair equal-length vectors — dumping the length-m eye vectors into the transient plot (whose scale is the full, much longer time vector) segfaults. No numerical-core changes.

Verification

Signals with known metrics (`examples/eye_examples`):

- [clean] an ideal 0/1 clock (perfectly periodic edges) recovers rails [0, 1] and amplitude 1, with jitter ~1e-24 s (machine zero) and a full-UI, fully-open eye.
- [jitter] a clock whose edges carry a known injected Gaussian jitter ($\sigma = 20$ ps): the reported `eye_jitter_rms / _pp` match the injected values to a few percent (≈ 18.6 ps rms / 99 ps pp), and `eye_width` equals UI - jitter_pp.
- [vectors] the folded eye (`eye_wave` vs `eye_t`) and the scalar result vectors are published.

Scope

Front-end only; solver-independent. A post-processing command — it consumes whatever transient waveform is in the current plot, so it works on built-in-device, OSDI, and behavioral signals alike.

Enhancement-208 — **pyplot -eye**: one-line matplotlib eye diagrams

A convenience bridge between the **eye** analysis and the **pyplot** matplotlib back-end: `pyplot -eye <expr> -ui <T>` runs the eye analysis and renders the result as a proper persistence-style eye diagram in a single command — no manual `eye` → `wrdata` → external plotting script round-trip.

Command

```
pyplot [name] -eye <expr> -ui <T> [-tstart t0] [-threshold vth] [-window frac]
```

The tokens after `-eye` are exactly the **eye** command's own arguments (the expression and its flags). A single bare token *before* `-eye` is taken as the output base name (`pyplot rxeye -eye v(rx) -ui 0.5n`); omitted, the base name defaults to `eye`.

```
pyplot -eye v(rx) -ui 0.5n -tstart 3n          -> eye.png (or an interactive window)
pyplot rxeye -eye v(rx) -ui 0.5n              -> rxeye.png
```

What it renders

`plot eye_wave vs eye_t` (the raw way to see the E-207 folded eye) draws a single connected line through the folded samples in file order — a scribble, not an eye. `pyplot -eye` instead draws the samples as a 2-D histogram (`hist2d`, log-scaled `LogNorm`, turbo colormap, empty cells masked): overlapping traces accumulate into a persistence eye, exactly how a sampling scope paints one. It is annotated with the metrics the `eye` command reports:

- the decision threshold (dashed) and the eye centre / sampling instant at 1 UI (dotted) after folding;
- a vertical eye-height double-arrow at the eye centre;
- a horizontal eye-width double-arrow along the threshold;
- a title carrying UI, eye height, eye width (as a % of UI), and RMS jitter.

Implementation

`com_pyplot()` detects the `-eye` marker, runs `com_eye()` on the trailing tokens (which folds the waveform and leaves `eye_wave/eye_t` plus the scalar metrics in a fresh current eye plot), then calls the new **`ft_pyplot_eye()`** in `plotting/pyplot.c`. That function reads the folded vectors and metrics back from the current plot, writes `<name>.data` (the folded sample pairs) and a `<name>.py` matplotlib script, and launches Python with the *same* mechanism as `ft_pyplot()` — so it honours every existing `pyplot_*` setting:

- set `pyplot_terminal=png|svg|pdf` → headless (Agg) hardcopy, run synchronously; otherwise an interactive window launched in the background;
- set `pyplot_python=<interp>`, set `pyplot_backend=<name>`, set `pyplot_figsize=W,H`, and set `pyplot_style=<sheet>` (a dark sheet flips the annotation colours to stay legible on a dark ground).

No numerical-core changes; the eye math is entirely E-207's. `ft_pyplot()` and the `gnuplot` / `asciiplot` back-ends are untouched.

Verification

[examples/pyplot_examples/verify_pyplot.py](#) gains four checks (17 total × both solvers). A self-contained data eye — a pseudo-random NRZ bit stream (PWL) through a bandwidth-limiting RC channel ($\tau \approx 0.5$ UI) — is rendered with `pyplot eyefig -eye v(rx) -ui 0.5n`: the eye analysis runs and reports its metrics, `eyefig.png` is a valid PNG, the generated script is a `hist2d` persistence eye referencing the metrics, and the no-name form defaults the base to `eye.png`. Full example regression: 170/170 (E-208 extends the existing `pyplot_examples` suite rather than adding a folder, so the suite count is unchanged).

Scope

Front-end only; solver-independent, like both parents. A convenience wrapper — it adds no new measurement, it packages the E-207 eye + the E-94 matplotlib bridge into one command so the classic eye diagram is a single line.

Enhancement-209 — **hb** publishes its spectrum as nutmeg vectors

The **hb** harmonic-balance command used to only print a spectrum table — the converged Fourier coefficients were freed as soon as the table was written, so there was no way to `plot`, `print`, or `wrdata` the result without scraping `stdout`. (The `amp2n2222` HB reproduction study had to parse that table in Python for every figure.) **hb** now publishes its spectrum as ngspice vectors, so the result is directly accessible like any AC/`tran` result.

What it does

After a converged `hb <f0> <K>`, a fresh **hb** plot is created and left current, containing:

- **hbfrequency** — a real scale vector: `0, f0, 2·f0, ..., K·f0` ($K+1$ points);
- one complex vector per node (named by the node, e.g. `out`, `vcc`, `q1#branch`), carrying the single-sided amplitude at each harmonic — so `mag(out)` and `vp(out)` reproduce the $|V|$ and phase columns of the printed table exactly.

```
hb 100meg 8
plot mag(out)           * the output spectrum
print vdb(out)          * in dB
wrdata spec.dat mag(out) vp(out)
```

All the usual accessors work (`v(out)`, `vm(out)`, `vdb(out)`, `vp(out)`, `mag(out)`, `real(out)`, ...), because the vectors are ordinary complex nutmeg vectors on a frequency scale. The printed table is kept (backward compatible), and the `.hb` dot-card publishes the same vectors.

Implementation

`HBAnalyze()` (`spicelib/analysis/dcpss.c`) already computed the two-sided Fourier solution V_r/V_i and then freed it. It gains an optional out-parameter `struct hbspectrum *out` (declared in `cktdefs.h`): on convergence it hands the V_r/V_i arrays (and $N, K, f0$) to the caller instead of freeing them.

The frontend command `com_hb()` (`frontend/com_hb.c`) receives that struct and, in a new `hb_publish()`, builds the plot with the frontend vector API (`plot_alloc/plot_new/plot_setcur`, `dvec_alloc/vec_new`) — the same idiom the `stb` and `eye` commands use. Node names come from `CKTnames`; a name containing `#branch` is typed `SV_CURRENT`, otherwise `SV_VOLTAGE`. The single-sided scaling ($\times 2$ for $k > 0$, $\times 1$ for DC) matches the table. The command then frees the transferred arrays.

The vectors are published from the frontend (not from the analysis layer) so no analysis `JOB` is required — a bare `hb` in a `.control` block, which may leave `CKTcurJob` unset, publishes cleanly.

Verification

[examples/hb_examples/verify_hb.py](#) gains a check (9 total): after `hb 100meg 5` on a diode rectifier, the published `hbfrequency` + node vectors exist, and `print mag(n)` equals the printed table's $|V|$ column bit-for-bit across all six harmonics. The existing table-based checks (and the `.hb` dot-card parity) are unchanged.

Scope

ngspice-only, purely additive. The numerical HB core is untouched — this only exposes the already-computed spectrum as vectors. `qpss/hb-family` analyses that print their own tables (two-tone `qpss`, `pac`, ...) are unaffected.

Enhancement-210 — PSS polish: **.pss** dot-card runs in batch + complex spectrum vectors

Two usability fixes to the [E-117](#) shooting-method periodic steady state (pss), in the spirit of [E-209](#) (which did the same for hb).

1. The **.pss** dot-card now auto-runs in batch

The pss command (inside `.control`) worked, but a top-level **.pss** card by itself silently did nothing in batch mode — `ngspice -b deck.cir` with just **.pss** ... reported *"no simulations run"*, unlike `.tran`, `.ac`, or `.hb`, which run automatically. A deck had to wrap it in `.control ... run ... .endc`.

.pss is now dispatched to the pss command the same way [E-146](#) (`.sweep`) and [E-162](#) (`.hb`) handle their cards: during deck loading a top-level **.pss** ... line is stripped of its leading `.` and appended to the post-parse control list, so it executes as `pss ...` once the circuit is built (`src/frontend/inp.c`). A boundary check keeps it from matching a longer name, and it never matches `.psp` (periodic S-parameters, a distinct card). Decks that already wrap **.pss** in `.control ... run` keep working (one PSS run, unchanged output).

* now runs straight from the deck, no `.control` needed

```
V1 in 0 SIN(0 1 1meg)
```

```
...
```

```
.pss 1meg 20u 1 1024 10 50 5m uic
```

```
.end
```

2. The frequency-domain spectrum is published as complex vectors

The pss analysis already published two plots — `pss1` (the time-domain periodic waveform) and `pss2` (the frequency-domain spectrum). But `pss2` stored magnitude only (`IF_REAL`): the DFT computed each harmonic's phase and then threw it away, so `vp(out)` was unavailable and `print out` gave a bare magnitude — inconsistent with AC analysis (whose node vectors are complex) and with the `hb` command's [E-209](#) vectors.

`pss2` now publishes complex node vectors (`IF_COMPLEX`): each harmonic is stored as `mag · e^{jφ}` (the DFT already returns ϕ in degrees), built into `IFcomplex` rows and emitted with `OUTpData` (as `PAC/PXF/PSP` do) instead of the real-only `CKTdump`. `mag(out)` reproduces the old magnitude spectrum exactly and `vp(out)` now recovers the phase. The time-domain `pss1` plot is unchanged (a real waveform).

```
.control
```

```
  pss 1meg 20u 1 1024 10 50 5m uic
```

```
  plot mag(out)           * the spectrum (as before)
```

```
  print vp(out)           * ...and now the phase, too
```

```
.endc
```

Backward compatibility Because `pss2` node vectors are now complex, `print out / wrdata out` emit `re, im` columns (exactly as AC analysis does), not a bare magnitude. Decks that parsed the magnitude directly should use `mag(out)`; the bundled `rc_pss.cir` was updated from `print b` to `print mag(b)` accordingly. This is the same magnitude-vs-complex convention the rest of `ngspice`'s frequency-domain analyses use.

Verification

[examples/rfpss_examples/verify_rcpss.py](#) gains two checks: a top-level **.pss** card (no `.control`) auto-runs in batch and reaches convergence; and the frequency-domain spectrum resolves as complex — both `vm(out)` (magnitude) and `vp(out)` (phase) return, with a driven diode rectifier's harmonics showing the expected non-trivial phase. The existing PSS / PAC / PXF / solver-parity checks pass unchanged (`rc_pss.cir` migrated to `mag(b)`).

Scope

ngspice-only, `src/frontend/inp.c` (dispatch) + `src/spicelib/analysis/dcpss.c` (complex pss2 output). The shooting-PSS numerical core, the retained periodic operating point, and every analysis built on it (PAC/Pnoise/PXF/PSP) are untouched.

Enhancement-211 — code-analysis bug fixes (XSPICE DC-op else + LTSPICE table free)

A static-analysis pass over the ngspice tree (clang's analyzer on every project-modified file, each finding then confirmed by reading the code) surfaced two real defects. Both are fixed here.

Bug 1 — mixed-signal DC operating point runs the analog solve twice

DCop() (src/spicelib/analysis/dcop.c) chooses between the event-driven XSPICE solver and the analog solver with a braceless else:

```
if (ckt->evt->counts.num_insts != 0) {           /* event-driven instances present
↳ */
    converged = EVTop(...); ...
} else
    converged = CKTop(...);                     /* ORIGINAL: analog solve,
↳ else-only */
```

Enhancement-188 (DC warm-start) inserted its preload code between the else and CKTop. Because the else had no braces, it then covered only its first statement (`wsiz = SMPmatSize(...)`), and everything after — including `converged = CKTop(...)` — ran unconditionally. Consequences, with XSPICE compiled (it is):

- For any deck with event-driven code-model instances (A-devices), both EVTop *and* CKTop ran; CKTop re-solved the analog part and overwrote the event-driven DC result. The original ran exactly one of the two.
- `wsiz` was read uninitialised at two later points (the warm-start guard and snapshot) on the EVTop path, where the else body that assigns it was skipped — undefined behaviour, masked today only because DC warm-start is off by default.

Fix: hoist `wsiz = SMPmatSize(...)` above the branch (always initialised, and the warm-start snapshot below needs it on both paths), and add braces so the else covers the warm-start preload and CKTop — restoring the original “exactly one solve” semantics. The `#ifdef XSPICE` braces vanish together when XSPICE is off, so the non-XSPICE build is unchanged.

Bug 2 — free of an uninitialised pointer on LTSPICE single-pair table import

`inp_compat()` (src/frontend/inpcom.c) converts an LTSPICE `E ... table=(...)` controlled source using `char *ckt_array[100]` — an uninitialised stack array. The LTSPICE branch sets only `ckt_array[1]`; a single-pair table (`table=(x0, y0), ipairs == 1`) then does `tfree(ckt_array[2])` — a free of a garbage stack pointer (`ckt_array[2]` is set only on the *other*, non-LTSPICE, branch). Undefined behaviour; a likely crash on malformed/degenerate input.

Fix: zero-initialise the array (`char *ckt_array[100] = { NULL };`). ngspice's `tfree(NULL)` is a safe no-op (`txfree` guards `if (ptr)`), so a `tfree` of any slot not written on a given branch does nothing instead of freeing garbage.

Verification

New suite [examples/codeanalysis_examples/verify_codeanalysis.py](#) (5 checks × both solvers) exercises the fixed paths: a mixed-signal DC op (an analog divider + an `adc_bridge`, an event-driven instance) solves the analog node to the correct 0.5 V without error; a single-pair LTSPICE `E ... table=(0.5, 3)` imports and yields the constant 3 V; and a plain analog DC op is unchanged. A would-fail before/after test is impractical for either bug (the redundant CKTop leaves a simple circuit's DC point unchanged, and freeing a garbage pointer is UB that may or may not crash), so these are behavior guards. Full regression: 171/171.

Scope

ngspice-only, two files (`dcop.c`, `inpcom.c`). No functional change to correct analog decks; the fixes remove undefined behaviour and a double analog solve on the XSPICE and LTSPICE-import paths respectively.

Enhancement-212 — crash hardening: seven user-triggerable crashes

A static-analysis pass (clang's analyzer over the ngspice tree) combined with argument fuzzing of every frontend command and malformed/degenerate netlists surfaced seven reproducible, user-triggerable crashes (SIGSEGV / SIGABRT) in stock ngspice code. Every one is in user-input handling — command parsers, the `.op` output path, and netlist recursion — never in the numerical core (the device models, the OSDI loader, Sparse 1.3, KLU, and the analysis drivers were all audited and found clean). All seven are fixed here.

The common thread is a missing guard on an edge of the input: a trailing flag, an empty argument list, a missing operand, an empty circuit, a bsearch miss, or an unbounded self-reference.

Command-parser NULL dereferences

1. **ipplot** `-w` / **ipplot** `-d` with the flag as the trailing token (`breakp.c`, `com_ipplot`). The flag handler consumes the next word (`wl = wl->wl_next`) and guards it with `if (wl) {...}` — which proves `wl` can be NULL — then at the loop bottom does `wl = wl->wl_next` unconditionally, dereferencing the NULL. Repro: `ipplot -w` with no window value. Fix: guard the advance (`if (wl) wl = wl->wl_next;`).
2. bare **altermod** (`device.c`, `com_altermod` → `com_alter_common`). `com_alter` guards `if (!wl)`, but its sibling `com_altermod` does not, passing an empty list straight into `com_alter_common`, which sets `wlin = wl_head` (NULL), calls `wl_nthelem(100, NULL)` → NULL, then `eq(wlin->wl_word, "I")` derefs NULL. Repro: `altermod` with no arguments. Fix: guard the empty `wl_head` with a clean error.
3. **meas ... FIND / WHEN** with a missing operand (`vectors.c`, `vec_get`). A malformed measure statement (`meas tran x FIND, ... WHEN, FIND v(1)` with no WHEN/AT, across `tran/dc/ac/sp`) leaves `meas->m_vec` NULL, which flows into `com_measure_when` → `vec_get(NULL)` → `copy(NULL) = NULL` → `strchr(NULL, '.')`. Fix: make `vec_get` NULL-safe (`if (!vec_name) return NULL;`) — the common sink, so every trigger reports "no such vector" instead of crashing. Every caller already handles a NULL return.

Output-path abort

4. empty / all-commented deck + **.op** (`dotcards.c`, `ft_cktcoms`). A circuit with no non-ground nodes — e.g. a deck whose every component is commented out — produces an `op` plot with no data vectors, so `pl_dvecs` is NULL. The former `assert(pl_dvecs != NULL)` aborted (SIGABRT; ngspice's autotools build defines no NDEBUB, so asserts are live), and with `-DNDEBUB` the next line would dereference NULL. Fix: guard with `if (plot_cur && plot_cur->pl_dvecs)` and skip the (empty) `op` printout. The solver had already handled the empty matrix correctly — this was purely in output formatting.

Solver-glue latent guard

5. KLU pole-zero setup (`cktpzset.c`). A bsearch miss for the drive pointer was reported with a `printf` and then dereferenced anyway (`job->PZdrive_pptr = matched->CSC_Complex`). This is latent — `SMPmakeElt` creates the element just above, so it is always in the bind table and the miss cannot occur today — but the guard was objectively broken. Fixed by adding the missing `else/NULL` path, matching the correct idiom already in `cktsetup.c`.

Netlist-recursion stack overflows

6. **.include** recursion (`inpcom.c`, `inp_read`). `inp_read` tracked a `call_depth` but never limited it, so a file that `.includes` itself — directly, via an `A → B → A` cycle, or via a `.lib` section that `.includes` its own file — recursed until the C stack overflowed (SIGSEGV). (`.lib files` are de-duplicated by `find_lib` and read once; the crash is always via `.include`.) Fix: cap the depth at `INP_MAX_INCLUDE_DEPTH = 50`; a cycle now reports an error. No legitimate netlist nests includes anywhere near this deep — a 49-level chain still reads fine.

7. **.func** recursion ([inpcom.c](#), `inp_expand_macro_in_str`). The expander re-expands a substituted function body with no depth guard, so `.func f(x)={f(x)}` (or a mutual `f ↔ g`) expanded forever and overflowed the stack — each recursion frame also holds a `params [FCN_PARAMS]` array (~8 KB), so it fails near ~1000 deep. Fix: a `macro_depth` counter capped at `INP_MAX_MACRO_DEPTH = 100`, reset on every error path. Legitimate nested and mutual functions still expand to the correct values.

Verification

New suite [examples/crashfix_examples/verify_crashfix.py](#) (17 checks × both solvers) drives every repro and asserts it now exits gracefully — no signal — instead of crashing, while every valid form still works (`iplot -w 3 v(1)`, `altermod nm vto=0.7`, `meas tran vmax MAX v(1) → 1.0`, a normal `.op`, `KLU .pz`, a legitimate nested `.include`, and `.func sq(x)={x*x} → sq(4) = 16`). Crashes are detected from the child's signal exit code (SIGSEGV/SIGABRT), which the pre-fix binary returned. Full regression: 172/172, both solvers.

Scope

ngspice-only, six files (`breakp.c`, `device.c`, `vectors.c`, `dotcards.c`, `inpcom.c`, `cktpzset.c`). No functional change to correct decks — the fixes turn crashes on malformed or degenerate input into graceful errors. The netlist-recursion caps mirror the include/expression-depth hardening OpenVAF-r received in [Enhancement-148](#); ngspice's netlist reader had lacked the equivalent guards.

Enhancement-213 — openvaf-r crash hardening: four compiler panics

Fuzzing openvaf-r with malformed Verilog-A found four distinct panics. Instead of printing a diagnostic, the compiler aborted with

```
OpenVAF encountered a problem and has crashed!  
A log file has been generated at ".../openvaf-crash-...log".  
To help us fix the problem, please open an issue at ...
```

and exited 101 — telling the user to file a bug report for what is simply a typo in their model. Every one is reachable from ordinary source mistakes; the headline case is a module that is merely missing its **endmodule**, one of the most common Verilog-A editing errors. All four are fixed here, so each input now produces a normal error message.

This is the OpenVAF-r counterpart to [Enhancement-212](#) (the same campaign against ngspice), and a direct continuation of [Enhancement-148](#), which hardened the compiler against pathological *depth* (parser recursion, ``include` nesting, array expansion). E-148 covered inputs that were too big; E-213 covers inputs that stop too early or are simply malformed.

1. Parse errors at end of file (the **endmodule** crash)

[syntax/src/parsing/tree_builder.rs](#). When a parse error lands at EOF, the error's span was built as

```
span: TextRange::at(  
    self.text_pos,                                     // already  
    → the end of the file  
    self.tokens.last().map_or_else(|| TextSize::from(0), |t| t.span.range.len()),  
    → // the LAST token's length  
) ,
```

`text_pos` is *already* the end of the source, so adding the previous token's length yields a span that runs past the end of the file — for the 6-byte input `module`, the span is `6..12`. Mapping that back to its source file then failed the `assert!(range.end() <= self.range.end())` in `FileSpan::with_subrange` (`preprocessor/src/sourcemap.rs`), which crashed the compiler *while trying to print the error*. The panic message told the story exactly:

```
subrange 6..12 -> 6..12 must fit into the total range 0..6
```

Fix: the error belongs *at* EOF, so use an empty range there (`TextRange::at(self.text_pos, 0.into())`) — precisely what the neighbouring `expected_at` field already does.

Additionally, [syntax/src/lib.rs](#) `find_ctx_range` maps a global position to its source context through half-open `[start, end)` ranges, which by construction never cover the EOF position itself (`pos == last_range.end()` compares Less against every range). It `.expect()`ed and panicked; it now clamps to the nearest range.

Triggers fixed: a module missing `endmodule`; a bare `module` keyword; a module header followed by EOF; an unclosed `analog begin`; an unterminated string literal.

2. Real literal with an exponent marker but no exponent

[lexer/src/lib.rs](#). The lexer consumed `e/E` as an exponent marker without checking that an exponent follows (the result of `eat_float_exponent()`, which reports whether it saw a digit, was discarded). So `1e` became a `Float` token whose text does not parse as an `f64`, and `ast::StdRealNumber::value()`'s `src.parse().unwrap()` panicked.

Fix: an `e` only joins the number when a digit (optionally after a sign) actually follows it. A bare `1e` now lexes as `1` and an identifier `e`, and surfaces as an ordinary parse error — the same approach based_

literal_body already takes for a malformed 8'squark. Valid exponents (1.5e3, 2e-3, 1E6) are untouched.

Triggers fixed: 1e, 1e+, 99e, 1.5e, and parameter real p=2e.

3. Preprocessor directives that end the file

[preprocessor/src/parser.rs](#). Two accessors indexed the token list directly, which is out of bounds when the parser sits one past the last token — a bare ``define` that ends the file:

- `previous_range()` — `self.full_tokens[pos]`, reached while building the "expected an identifier" diagnostic;
- `followed_by_bracket_without_space()` — `self.relevant_tokens[self.pos + 1u32]` (index out of bounds: the len is 2 but the index is 2).

Fix: both use `.get()` with a sensible fallback — a zero length, and "nothing follows, so it is not followed by a bracket" — mirroring the `.get().map_or()` idiom `current_range()` already used.

Triggers fixed: ``define`, ``define M(a,`ifdef`, ``include` at EOF.

4. `path()` asserted a precondition its callers do not check

[parser/src/grammar/paths.rs](#). `path()` began with `assert!(p.at_ts(PATH_SEGMENT_TS))`. Several callers do not check that precondition and reach it on plausible input, crashing the compiler:

input	caller
aliasparam x = 5; — a literal where a parameter name belongs	alias_parameter_decl
aliasparam x = ;	alias_parameter_decl
I(<1>), I(<>)	port_flow
discipline d 1 = 2;	discipline

Fix: drop the assert. The very next line, `p.expect_ts(PATH_SEGMENT_TS)`, already emits "expected identifier" and returns false, so the error is reported and an empty path node completed. (nature's parser already guarded its own call site this way in [Enhancement-39](#); this makes the helper itself safe for every caller.)

Verification

New suite [examples/vafcrash_examples/verify_vafcrash.py](#) (25 checks) drives every repro and asserts it now yields a clean error instead of a crash, plus regression checks that valid code is unchanged (a resistor; real exponents 1.5e3/2e-3/1E6; an aliasparam bound to a real parameter; a ``define` with arguments).

One detail worth noting: `openvaf-r` installs a custom panic hook that prints its own message and exits 101, rather than dying on a signal or printing the usual Rust panicked at. A crash check that only looks for those two — as E-148's suite does — would score these panics as ordinary errors, so the new suite treats exit code 101 and the hook's message as a crash.

Toolchain tests pass unchanged (lexer 8, preprocessor 6, basedb 9, sim_back 26, hir 15, hir_lower 4). Full regression: 173/173.

Scope

openvaf-r only, five files (`syntax/src/lib.rs`, `syntax/src/parsing/tree_builder.rs`, `lexer/src/lib.rs`, `parser/src/grammar/paths.rs`, `preprocessor/src/parser.rs`). No change to any accepted program: every fix is on a path that previously aborted the compiler. The generated OSDI for every existing model is unchanged — the full example suite compiles and simulates identically.

Enhancement-214 — whole-array type coercion: a recurring crash class, fixed at its root

Some bugs are found once. This one was found four times, in four different corners of the compiler, and fixed four times — because each fix landed on the symptom rather than the cause. This enhancement fixes the last two instances, and then removes the trap that kept producing them.

The symptom is always the same: an integer Value reaches a float MIR op (feq/fmul/fsub/fdiv), and `mir_opt::const_eval::eval_binary` has no (Int, Float) case for it, so the compiler panics —

```
invalid operation fdiv Int(1) Float(Ieee64(1.0))
OpenVAF encountered a problem and has crashed!
```

— exit 101, with the usual invitation to file a bug report. If constant propagation happens *not* to fold the expression, the very same defect instead survives to codegen and reaches LLVM as `i32 = fadd . . , ConstantFP:f64`, aborting with **LLVM ERROR: Cannot select**. One bug, two faces, depending on an optimization that has nothing to do with it.

Like [E-213](#), none of this requires exotic input. The trigger is writing `{1}` where a real is expected — which is simply how a unity coefficient or a small selector is naturally written.

The four instances

#	Where	Fixed in
1	case over an integer array — the array element type was hardcoded real, so an feq landed on i32	E-33
2	laplace_*/zi_* integer-literal coefficients: <code>laplace_nd(V(in), {1}, ...)</code>	b77266ec
3	laplace_*/zi_* integer array-variable coefficients	c55812d6 (here)
4	An integer case item against a real array discriminant	8d0ab057 (here)

Instance 3 is a good illustration of how a symptom-level fix propagates the problem. The fix for instance 2 cast integer-literal coefficients to real but *explicitly excluded* the array-variable path, with this reasoning recorded in a comment:

```
    a whole-array variable reference is already real by its declaration, so the var-ref path needs
    no cast
```

That is false for an integer array:

```
integer c[0:0];
analog begin
    c[0] = 1;
    V(out) <+ laplace_nd(V(in), c, '{1.0, 1e-6}'); // panicked
end
```

Its element reads are i32 and hit exactly the mixed-type MIR the earlier fix set out to prevent. Coefficient vectors are real per LRM 9.19; the var-ref path is now coerced from the variable's declared type, and the comment corrected.

Instance 4 comes from the opposite direction — a real discriminant with integer items:

```
real r[0:0];
analog begin
    r[0] = 1.0;
    case (r)
```

```

    {1}: g = 7.0;          // panicked: invalid operation feq Int(1) Float(..)
    default: g = 1.0;
  endcase
end

```

The comparison opcode is chosen from the discriminant's type — Feq, for a real array — but the item's integer elements lower to i32.

The root: a cast that was silently dead

Type inference is not at fault here. It *does* record the coercion: `expect()` inserts a cast for the offending array, `casts.insert(item, Array{Real})`. The problem is where that cast lands and who reads it.

The cast is recorded on the array expression. But every whole-array consumer goes through `hir_lower's lower_array_elems_impl`, and that function decomposes the array and lowers each element itself — it never routes the array expression through `lower_expr`, which is the only place that consults `needs_cast()`. So the cast was never applied. It was dead, and nothing reported that.

That is why the bug kept coming back. Each new context that consumed a whole array inherited a trap that was invisible at the call site, and each was patched locally as it was discovered. The fix is at the chokepoint every consumer already passes through:

```

// openvaf/hir_lower/src/expr.rs
let coerce_real = coerce_real
  || matches!(self.body.needs_cast(expr), Some( (_, dst)) if *dst.base_type() ==
    ↪ Type::Real);

```

Inference's intent is now effective for every consumer, present and future, instead of requiring each call site to remember to ask. Verified in isolation: with the case-site flag disabled, this guard alone fixes every case repro.

The explicit `coerce_real` flag is kept, for a reason worth stating: it serves consumers whose element type is fixed by *the language* rather than by an inferred cast. A `laplace_*/zi_*` coefficient vector is real per LRM 9.19 and `inference_laplace` records no cast for it, so there is nothing for the structural guard to honour there. `lower_case` likewise keeps its own coercion tied to its opcode (`discr_op == Opcode::Feq`) — choosing the opcode from the discriminant makes matching the operand types *that function's own invariant*, independent of inference's bookkeeping. The two mechanisms are complementary, not redundant.

Verification — coercion that does not miscompile

A compiles-without-crashing test would pass even if the coercion silently changed a coefficient, so the suite checks meaning, not just survival.

The same first-order filter is compiled twice — once with the integer spelling `{1}`, once with `'{1.0}` — and swept over 13 AC points from 1 kHz to 10 MHz:

	worst deviation
integer literal <code>{1}</code> vs real <code>'{1.0}</code>	0 dB (bit-identical)
integer array variable vs real <code>'{1.0}</code>	0 dB (bit-identical)
integer-coefficient filter vs analytic $1/(1+j\omega\tau)$	3.4e-08 dB

The integer case item still matches: the arm is taken (`g = 7`, not the default 1), giving a current identical to the `'{1.0}` spelling. Valid code is unaffected — real coefficients, real array variables, real/integer/string scalar cases, and E-33's element-wise real array case all behave exactly as before.

Mutation-tested. A guard that cannot fail is worth nothing, so the fixes were reverted in a scratch build and the repros re-run against it: all four crash again with exit 101. The suite catches every instance of the class.

Verification guards hardened

Two suites were sharpened during the same hunt, each with a blind spot that could hide exactly the kind of defect it was meant to catch:

- **vafautodiff_examples** ($8 \rightarrow 16$ checks). The suite held every builtin's second argument constant, so it never exercised that argument's chain rule — precisely where **E-185's** hypot bug lived. Added a cross-derivative referee (both arguments live circuit unknowns, read off the off-diagonal AC Jacobian) at asymmetric bias points, since hypot's wrong rule was accidentally *correct* at $x == y$; plus a 44-point battery, as the old suite used a single point per builtin. Re-injecting E-185's bug makes it fail at 50% on both partials.
- **operator_examples** (+14 checks). Every check used literal operands, so it exercised only the constant folder. But **E-37's** `>>` bug was a *disagreement* between the folder and the runtime path — a literals-only test exercises just the side that happened to be broken. Each operator is now computed twice, once folded and once through `$rtoi(V(in))` so it cannot be folded, and the two are compared. Breaking only the runtime path leaves the old check passing while the new one fails with score 9.

Scope

openvaf-r only, two files (`hir_lower/src/expr.rs`, `hir_lower/src/stmt.rs`). No accepted program changes meaning: every fix is on a path that previously aborted the compiler, and the generated OSDI for every existing model is identical — the full example suite compiles and simulates unchanged. Toolchain tests pass. New `examples/arraycast_examples` (23 checks, both solvers). Full regression: 174/174.

If a fifth instance ever appears, the question to ask first is whether the context bypasses `lower_array_elems_impl` (reading variables directly, say), and whether inference records a cast for it at all.